

Canonical Diophantine Certificates

Witness-faithful polynomial representations of bounded computation: trace classes, accelerators, polynomial trajectories, memory logs, queues, rewriting, heaps, self-assembly, reaction networks, routing networks and interaction combinators

Research report prepared for Vladimir Reshetnikov. Manuscripts dated September 30 and October 2, 2026; merged in the batch-60/61 write phase and extended in the batch-62, batch-63 and batch-78 write phases.

Abstract

[write] An existential Diophantine representation says that a computation exists; it need not preserve which computation occurred, how many computations occurred, or how many arithmetic witnesses describe one of them. This report collects fifteen manuscripts that construct *witness-faithful* (single-fold or parsimonious) polynomial representations of bounded computation and locate the boundary beyond which such representations are as hard as the open single-fold and finite-fold problems.

For Petri nets and interval-guarded translations, an explicit convex quadratic integer polynomial has its natural zeros in bijection with the commutation classes of enabled executions of exact length T ; its nonnegative real zero set is a bounded rational polytope whose lattice points are exactly those zeros (Part II, manuscript 05). Quartic second routes (manuscripts 04 and 06), a Cartier-Foata variant for bounded causal height (manuscript 01), exact all-marking commutation tests, an exponential automaton-state lower bound and #P-completeness of counting accompany it (Parts I and II). Single-fold quartics of fixed arity describe multisets, fixed-word powers and nested repetitions of counter actions independently of the repetition counts (Part III); the relation “ $p(i) \geq 0$ for all $i \leq T$ ” with $O((d+1)^3)$ witnesses independent of T (Part IV); random-access memory logs with $24L$ witnesses (Part V); cyclic-tag, deletion-tag and fixed-read FIFO executions with one coordinate per consumed symbol and one guard slack, together with a proof that faithful polynomial word memory needs two natural coordinates (Part VI); and FRACTRAN runs and scheduled SKI reductions (Part VII). Part VIII collects further substrates. Four later manuscripts add dynamic heaps modulo fresh names (Part X), a sharp degree threshold for orthant-nonnegative polynomials and order-free tile assembly (Part XI), priority pumping (Part XII) and a reaction network with one shared fallback (Part XIII); a twelfth adds history-free certificates for chip-routing networks, one quartic per routing topology, uniform in the initial loads and run lengths, whose unique natural zero carries the exact firing counts (Part XIV). Three manuscripts of batch 78 add exact port wiring for Lafont’s interaction combinators: a sharp $2/5$ – $2/3$ bound on how often the parity of the cyclic wires of a random closure tells two perfect matchings apart, bounded six-rule quartic compilers with $O(C_0 + F + T)$ coordinates and one witness per legal schedule, built on a ten-pattern annihilation kernel and on variants of Part V’s memory certificate, and a canonical permutation-orbit certificate with an implemented fixed-schedule compiler (Part XV).

Part IX proves, with proofs from four manuscripts and a sketch in a fifth, that a single-fold (finite-fold) polynomial for universal halting exists exactly when every computably enumerable set has a single-fold (finite-fold) Diophantine representation, which is an open problem that this report does not solve. Part XIV proves that a single-fold (finite-fold) representation of the graph of one monotone Boolean function computable in polynomial time is equivalent to the same open statement.

Status. [write] A merge of fifteen AI-assisted, unrefereed manuscripts, six delivered in batch 60, one in batch 61, four in batch 62, one in batch 63 and three in batch 78 of ProveIt's incoming reports (Appendix [K](#)). The theorems carry ordinary mathematical proofs; the accompanying Python programs perform finite exact checks and are not proofs. No statement of this report is formalized in Lean or Rocq, although the report sits beside ProveIt's formal Hilbert's-tenth-problem project; the project declarations it cites are named in the README. Global priority is not claimed for any construction, and no resolution of the general single-fold or finite-fold problem is claimed. Every manuscript's own status statement is printed in Section [3](#).

Contents

1	The research problem and its relation to ProveIt	10
2	Conventions	12
3	The fifteen manuscripts	21
I	Exact commutation and resource algebra	48
4	An exact commutation test for Petri transitions [05]	49
5	Guarded translations and exact commutation [01]	52
6	The exact resource envelope of a multiset [01]	54
7	The exact resource-summary algebra [06]	55
8	A first-failure product for resource legality [07]	58
II	Canonical trace classes	60
9	Canonical schedules with linear Boolean memory [05]	60
10	A parsimonious quadratic encoding of raw executions [05]	62
11	Canonical traces: compact quartic and convex quadratic forms [05]	64
12	Costs, exact counts, and a closed-form example [05]	67
13	Counting hardness survives maximal commutation [05]	69
14	A symbolic normal-form monitor for concurrent traces [04]	70
15	Quartic certificates for Petri-net trace classes [04]	72
16	Counting, weights, and the limits of low degree [04]	74
17	A compact signature for lexicographic trace normality [06]	76
18	A bijection between bounded trace classes and polynomial roots [06]	78
19	Causal traces and canonical layers [01]	80
20	An explicit quartic for causal histories [01]	81

III	Compressed repetition and accelerators	84
21	A fixed-arity single-fold quartic accelerator [01]	85
22	Fixed block programs with unbounded repetition [01]	86
23	Worked examples [01]	88
24	Saturation: nested normality only sees zero, one, and two [06]	90
25	A single-fold quartic compiler [06]	91
26	Expressiveness and an exact nesting boundary [06]	94
IV	Polynomial trajectories	98
27	The problem addressed and the precise restricted result [02]	98
28	The finite-difference sign certificate [02]	99
29	The single-fold quartic theorem and its cost [02]	102
30	Boolean guards and empty execution prefixes [02]	103
31	Accelerating polynomial trajectories [02]	104
32	Computational substrates covered by the theorem [02]	105
33	Canonical nontermination certificates and input bounds [02]	108
34	Worked examples [02]	110
35	Manuscript 02: Conclusion [02]	112
V	Memory logs, RAM and graph evaluation	112
36	Canonical arithmetic with quadratic residuals [03]	112
37	Deterministic sorting as a canonical polynomial circuit [03]	114
38	The canonical random-access memory theorem [03]	115
39	Size, height, and the cost of canonical routing [03]	118
40	A linear-size alternative: exact large-base products [03]	119

41	From memory logs to complete RAM executions [03]	123
42	Applications to computational substrates [03]	125
43	Exact counting and first-halting probabilities [03]	127
44	Manuscript 03: Conclusion [03]	128
VI	Queues: tag systems and FIFO networks	129
45	Little-endian words and faithful concatenation [07]	130
46	Cyclic tag systems without intermediate queues [07]	130
47	Two examples: a real computation and a self-starting fiction [07]	132
48	Variable count, degree, and witness height [07]	134
49	General deletion-tag systems [07]	136
50	The exact dimension of polynomial word memory [07]	138
51	A quartic certificate from two-coordinate memory [07]	141
52	FIFO networks and switched arithmetic dynamics [07]	142
53	Manuscript 07: Conclusion [07]	145
VII	FRACTRAN, SKI and term rewriting	145
54	A linear-size, unique-witness FRACTRAN priority certificate [04]	146
55	Context-local Diophantine compilation of SKI rewriting [04]	149
56	Local FRACTRAN and combinatory-logic interfaces, and bounded rewriting [02, 01, 05]	152
VIII	Other substrates	154
57	Other computational substrates [05]	154
58	Other computational substrates through the same interface [04]	156
59	Other computational substrates and safe adapters [01]	158
60	Other substrates and the limit of the construction [02]	159

61	Other computational substrates: transfers and limits [06]	159
IX	The boundary of fixed-arity compression	161
62	Fixed-dimensional compression and the finite-fold boundary [05]	161
63	The boundary of unbounded canonicalization [01]	164
64	The exponentiation boundary and universal first halting [02]	166
65	The exact boundary of uniform single-fold compression [03]	168
66	Unbounded computation and the fixed-arity frontier [04]	169
67	The exact finite-fold obstruction [06]	171
68	The fixed-arity and finite-fold boundary [07]	172
69	Implementation and validation	173
70	Formalization targets	186
71	Research questions	194
72	Conclusions of manuscripts 05, 01, 04 and 06	208
X	Dynamic heaps, fresh names and local graph computation	210
73	Conventions of Part X	210
74	Repository interface and relevant prior work [08]	211
75	Witness-faithful polynomial systems [08]	212
76	A complete memory-log compiler [08]	215
77	Sorting costs and a restricted optimality statement [08]	217
78	Fresh names and canonical finite histories [08]	219
79	The local-heap compilation theorem [08]	220
80	A universal pointer-only substrate [08]	224
81	Local graph computation and the exact transfer boundary [08]	225

82	A worked certificate and executable evidence [08]	226
83	Unbounded properties and two necessary obstructions [08]	228
84	A formalization and integration plan [08]	230
85	Further research questions and proposed topics [08]	231
86	Conclusion [08]	233
87	Reproduction and artifact map [08]	233
88	Dependency and claim ledger [08]	234
XI	Order-free self-assembly and a positivity-restricted degree threshold	235
89	Conventions of Part XI	236
90	A sharp degree threshold under real-orthant nonnegativity [09]	236
91	Order-free arithmetic certificates for self-assembly [09]	244
92	Reproduction, formalization, and further research [09]	254
XII	Priority repetition: pumping bounds, program-uniform certificates and periodic tails	259
93	Conventions of Part XII	260
94	Ordered arithmetic computation [10]	260
95	Exact finite-block acceleration [10]	261
96	Least seeds and the exact repetition capacity [10]	263
97	A sharp pumping bound [10]	265
98	A canonical quartic Diophantine compiler [10]	266
99	Variants, composition, and what is not free [10]	271
100	Infinite periodic instruction tails [10]	272
101	Discovering periodic tails is still undecidable [10]	273
102	Beyond periodic instruction words [10]	275

103 Reproducible artifacts and finite validation [10]	277
104 A formalization roadmap for ProveIt [10]	279
105 Further research questions [10]	280
106 Conclusion [10]	282
107 An executable specification of the interval algorithm [10]	282
108 An executable specification of the capacity formula [10]	283
109 Claims and non-claims ledger [10]	283
 XIII Conservative priority reaction networks and a canonical-fuel form of the finite-fold problem	 284
110 Conventions of Part XIII	285
111 Models and basic notation [11]	285
112 The one-fallback conservative compiler [11]	286
113 Reservoir thresholds and two notions of universality [11]	288
114 Why priority cannot simply be erased [11]	290
115 Decidable bounded resources and unbounded thresholds [11]	292
116 Explicit quartic certificates with unique witnesses [11]	293
117 Single-fold quartics for bounded minimum fuel [11]	295
118 A canonical-fuel normal form for the finite-fold problem [11]	297
119 Pareto-minimal resources: a general extension [11]	299
120 Worked example and executable verification [11]	300
121 A concrete integration plan for ProveIt [11]	301
122 Further research questions and priorities [11]	302
123 Conclusion [11]	303
124 The universal source machine and a source-table discrepancy [11]	304

125 Manifest and assurance levels [11]	305
XIV History-free routing certificates: last exits, compressed periods and a rank-function bottleneck	307
126 Conventions of Part XIV	308
127 A history-free characterization of exact routing [12]	309
128 A parametric single-fold quartic for run-length routing [12]	311
129 Shared grammar periods without word expansion [12]	314
130 Certificate height, both outcomes, and fixed-period structure [12]	317
131 A circuit-defined period already hides a counting problem [12]	319
132 A universal one-router system with easy prefix ranks [12]	320
133 An exact finite-fold and single-fold bottleneck [12]	322
134 Executable artifacts and the formalization boundary [12]	325
135 Further research questions and concrete targets [12]	327
136 Conclusion [12]	328
137 Reproduction instructions and archive contents [12]	329
138 A compact dependency map [12]	329
XV Interaction combinators: exact wiring, loop-exact gluing and bounded quartic frontends	331
139 Conventions of Part XV	333
140 Port-labelled nets and the six interaction rules	334
141 Exact gluing and loop accounting	339
142 Contextual information: loop parity, injectivity and tomography	343
143 The local annihilation kernel	348
144 Memory logs: second routes to Part V	352

145	The bounded six-rule compiler	359
146	A second route: permutation orbits and an implemented scheduled compiler	370
147	Boundaries: fixed horizon, schedules and the MRDP interface	378
148	Implementation, validation and formalization proposals	381
149	Research questions and conclusions	388
150	A compact mathematical specification of the memory compiler [13]	394
151	Literal functional gadgets [14]	394
152	Assembler specification and trust boundaries [14]	396
153	Package contents and replay [14]	396
154	The complete residual schema in one place [15]	397
155	Small omissions that would invalidate the result [15]	398
156	Data format and verification of the exported polynomial [15]	398
	Appendices	399
A	Manuscript 05: A minimal end-to-end example	399
B	Manuscript 05: Edge cases and specification checklist	400
C	Manuscript 05: Source and verification provenance	401
D	Manuscript 01: Reproducibility and source-audit notes	401
E	Manuscript 02: The finite checking formula in one place	401
F	Manuscript 02: Reproduction and the exported polynomial format	402
G	Manuscript 03: Additional validation of the linear backend	403
H	Manuscript 03: Claim and dependency ledger	403
I	Manuscript 04: Notation and resource-count reference	404
J	Manuscript 06: A completely displayed local power polynomial	404
K	Provenance of this report	406

References**411**

1 The research problem and its relation to ProveIt

1.1 Beyond an existence-only translation

A conventional application of the Matiyasevich–Robinson–Davis–Putnam theorem turns an effectively enumerable reachability predicate into a polynomial solvability problem. It does not, by itself, identify polynomial solutions with executions. One execution can acquire many auxiliary encodings; a nondeterministic execution may have many schedules; and a scheduling quotient can introduce another layer of multiplicity.

For a compiler intended for proofs, counting, or optimization, the natural stronger specification is a bijection

$$\{\text{computational objects of interest}\} \longleftrightarrow \{z \in \mathbb{N}^n : P(z) = 0\}. \quad (1)$$

The entire right-hand tuple must be determined, not merely its projection to a few distinguished coordinates. Otherwise even the number of solutions of the equation can be unrelated to the number of computations.

Here the main computational objects are executions modulo specified swaps of adjacent transitions. We resolve four concrete design questions. First, which swaps preserve enabledness at *every* marking? Second, how can one enforce one representative per commutation class without expanding an exponentially large automaton? Third, can all arithmetic auxiliaries be unique while keeping the polynomial degree small? Fourth, what does this genuinely bounded construction imply—and not imply—about a single polynomial representing arbitrarily long universal computations?

1.2 Repository baseline and the extension developed here

The inspected ProveIt snapshot has commit

e8bb0931d67f80d9fce87a8cddb0f661ff19f956.

Its Hilbert-tenth-problem development contains theorems `boundedForall_dioph`, `exactIter_dioph`, and `existsExactIter_dioph`; the last two provide Diophantine exact iteration and finite reachability under appropriate Diophantine substitutions [3]. These are the correct existing interfaces to reuse for existence-level consequences.

The present article adds an explicit *semantic fibre* specification. We keep the transition word, its states, the normal-form recognizer, and every auxiliary variable visible, prove unique extension, and only then collect the constraints into one polynomial. The bounded construction is elementary and does not need MRDP. Its relation to the repository is therefore complementary: it supplies a structured front end and a stronger correctness theorem, rather than another proof of existential trace closure.

The repository’s status register distinguishes completed results from pending formalization [2]. No repository build was performed for this article. Statements about its interfaces are based on the inspected sources, not on a new independent kernel audit. No result below relies on an unverified numerical optimization in the repository.

1.3 Theorems, classical ingredients, and novelty

The exact encoding, its size accounting, its unique-extension proof, the graph realization, the memory lower bound, and the particular counting reduction are all supplied in full. They form a concrete extension suitable for review and implementation. We do not claim that an exhaustive literature search establishes priority for each individual observation.

The trace-monoid forbidden-factor characterization of lexicographic normal forms is classical; an explicit published form appears in Barthélemy [18]. Sum-of-squares conjunction and arithmetic encodings of Boolean constraints are also standard ideas. Register-machine Diophantine representations have a long history, including Jones–Matiyasevich [8]. The contribution pursued here is the *combined guarantee*: exact semantic commutation, canonical representative selection, no free witnesses, bounded real geometry, and explicit separation from fixed-dimensional finite-fold compression.

The distinction between polynomial representability and finite-fold or single-fold representability is essential. Matiyasevich discusses this distinction and the associated conjectures [9]; Cantone, Cuzziol, and Omodeo discuss recent single-fold specification methods and the danger of introducing uncontrolled witnesses [10]. Our boundary theorem concerns these stronger specifications. It is not presented as their solution.

1.4 How this report is organized

[write] This report merges fifteen manuscripts written independently for the same purpose, witness-faithful Diophantine compilation of bounded computation. Six (numbered 01–06 here) were delivered in batch 60 of ProveIt’s incoming reports and pin the repository at e8bb0931d; the seventh (07) was delivered in batch 61, pins 725d2ebb6, and names manuscripts 04 and 06 as its companions. Four more (08–11) were delivered in batch 62, pin 4e128356d (manuscript 11 also names ccfb084ad), and form Parts X–XIII; they were written without knowledge of this report. Manuscript 12 was delivered in batch 63, pins e18718e83 and forms Part XIV; it too was written without knowledge of this report. Manuscripts 13–15 were delivered together in batch 78, pin 439c0a2d9, 6914ccca6 and 928ea9701, and are merged by subject into Part XV; manuscript 14 cites this report’s README, and the other two do not cite the report. Manuscript 05, whose introduction the preceding subsections reproduce, is the base: its trace-class theorem has the weakest hypothesis and the strongest conclusion among the four manuscripts that prove one (Part II). The fifteen titles, abstracts and status statements, and every manuscript’s own introduction, are printed in Section 3; Appendix K records their archives, pins and commits.

After the conventions (Section 2), the material is arranged by subject in fifteen Parts:

Part	Subject	Manuscripts
I	Exact commutation and resource algebra (Part I)	05, 01, 06, 07
II	Canonical trace classes (Part II)	05; 04, 06 (second routes); 01
III	Compressed repetition and accelerators (Part III)	01, 06
IV	Polynomial trajectories (Part IV)	02
V	Memory logs, RAM and graph evaluation (Part V)	03
VI	Queues: tag systems and FIFO networks (Part VI)	07
VII	FRACTRAN, SKI and term rewriting (Part VII)	04, 02, 01, 05
VIII	Other substrates (Part VIII)	05, 04, 01, 02, 06
IX	The boundary of fixed-arity compression (Part IX)	all seven
X	Dynamic heaps, fresh names and local graph computation (Part X)	08
XI	Order-free self-assembly and a positivity-restricted degree threshold (Part XI)	09
XII	Priority repetition: pumping bounds, program-uniform certificates and periodic tails (Part XII)	10
XIII	Conservative priority reaction networks and a canonical-fuel form of the finite-fold problem (Part XIII)	11
XIV	History-free routing certificates: last exits, compressed periods and a rank-function bottleneck (Part XIV)	12
XV	Interaction combinators: exact wiring, loop-exact gluing and bounded quartic frontends (Part XV)	13, 14, 15

The back matter collects each manuscript’s implementation and validation record (Section 69), its formalization proposals (Section 70), the merged research questions (Section 71), the remaining conclusions, the manuscripts’ appendices, the provenance record and one merged bibliography. These back-matter sections cover manuscripts 01–07; each of Parts X–XIV is one of manuscripts 08–12 in its own order, closing with that manuscript’s validation, formalization plan, research questions, conclusion and appendices. Part XV merges manuscripts 13–15 by subject and closes with their validation, research questions, conclusions and appendices.

A result proved by several manuscripts is printed once, at the place named in Appendix K, and credits every manuscript that proves it; a genuinely different proof or construction of the same result is kept as a marked second route. Two such mergers change the shape of the text: the universal-halting equivalence is one theorem, Theorem 62.1, and the absence of a computable witness-height bound is one proposition, Proposition 62.2. At the places where the other manuscripts stated them, a note points to the merged statement and the manuscript’s own proof follows.

2 Conventions

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. Every witness ranges over $\mathbb{N}$; every polynomial has integer coefficients, and subtraction in a displayed polynomial is subtraction in $\mathbb{Z}$, never truncated subtraction. A *residual system* is a finite list of integer polynomials required to vanish; its single equation is the sum of their squares, of degree at most four when every residual is quadratic and at most two when every residual is affine. A node of an arithmetic circuit names a polynomial subexpression and is not a witness; only an added coordinate is. Parameters are natural except in Part IV, whose coefficient parameters are signed integers. In Part VI the program, the input and terminal words and the horizon are data of the construction, not unknowns.

A representation is *single-fold* if each parameter tuple has at most one witness, *finite-fold* if it has finitely many, and *parsimonious* for a set of semantic objects if its witnesses are in a specified bijection with them. “Canonical certificate” (Part V) means a single-fold representation.

Words used in two senses.

- *Certificate*. Here, a natural witness tuple of a polynomial. In ProveIt’s Hilbert’s-tenth-problem project a “straight-line certificate” of the 1980 Theorem 5 is a program whose arithmetic operations are counted; the project’s 75-operation universal certificate is of that kind. No result of this report concerns operation counts.
- *Height*. Part II’s *causal height* H (from manuscript 01) is the number of Cartier–Foata layers of a trace. In Part VI, H is an upper bound on queue length. Elsewhere a *witness height* bounds the magnitude of witness coordinates.
- *Length* T is the number of steps of a bounded execution in Parts II, VI, VII and VIII; in Part IV T is the horizon of $\{0, \dots, T\}$.
- *Memory*. Part V certifies logs of a random-access memory. Part VI’s *word memory* is a numerical encoding of a finite word; its dimension is the number of natural coordinates of the encoding.
- *Universal*. The Jones articles’ *universal pair* (ν, δ) bounds unknowns and degree of an existence representation; Theorem 62.1 concerns a universal polynomial that is moreover single-fold or finite-fold, which is equivalent to an open problem.

[write] Three further collisions need the same care. The letter H has two more meanings: in Part VII it is the total context depth of an SKI schedule (manuscript 04), and in Part II $H_{i,a}$ (manuscript 05) and

$H_{a,i}$ (manuscript 06) are linear forms in the selectors of one step. In Part II, “memory” (Theorems 9.5 and 14.3) counts the states or bits of a deterministic finite-state recognizer of normal forms, which is neither Part V’s nor Part VI’s notion. Finally, “canonical” qualifies three different objects: a canonical *representative* (the lexicographically least word of a trace class, or the Cartier–Foata normal form), a canonical *witness* (the unique auxiliary tuple of a single-fold representation, Part V’s “canonical certificate”), and canonical *padding* (a uniquely forced way of filling a fixed-length record). The tempting false reading of a “canonical certificate for a trace” is that the trace has one witness among all the traces with the same endpoints; the true reading, throughout Part II, is that each trace class has exactly one witness, so the number of zeros at fixed endpoints is the number of classes.

Letters that change meaning between Parts. [write] Each Part keeps the letters of the manuscript it prints. The dictionary below lists the letters that change meaning; the tempting false reading is to carry a letter across a Part boundary.

Symbol	Meaning by manuscript (and Part)
places, transitions	05: d places, m transitions (Parts I, II, VIII); 01: p coordinates, m actions, alphabet A (Parts I–III); 04: p places, r transitions, and in FRACTRAN r fractions (Parts II, VII); 06: p resources, q transition labels (Parts I–III); 07: p is the period of a cyclic-tag program, and also the number of resources in the first-failure product and of queues in a FIFO network (Parts I, VI).
m	01: number of actions; 05: number of transitions; 06: an endpoint marking ($m \xrightarrow{w} n$); 04: marking variables $m_{t,i}$; 07: appendant lengths m_i .
u, v	01: upper guard u_a and displacement v_a ; 05: upper guard u_a (interval guards), production v_a , and the auxiliaries $u_{i,a}$; 06: least input u_a and its output v_a ; 07: guard slack u ; 04: production matrix V .
D	01: dependence relation $D = (A \times A) \setminus I$; 05: sequential demand D_{ab} ; 06: linear forms $D_{a,i}$; 07: total number of consumed symbols D and prefix factors D_i .
d	05: number of places; 01: $d = \sum_a D(a) $; 02: nominal degree bound; 07: deletion number.
N	05: final marking; 01: witness counts N_{acc} , N_H ; 03: number of padded records; 06: number of syntax nodes; 07: $N_{B,d}$, the number of halted words.
B	01, 04: witness-height bounds; 04: stack base; 05: coordinate bounds B_p ; 07: alphabet size.
K	01: universal halting $K(e, x)$; 05: label-additive cost $K(w)$; 07: $K = (B - 1)!$.
C	03: number of comparators; 05: linear forms $C_{i,a}$; 07: the largest number of symbols read in one event.
L	01: loss totals L_j (and $L = n_a + 2n_b$ in an example); 03: log length; 05: lower box L_{ab} ; 07: length residual.
G	01: gain totals G_j ; 07: the first-failure guard.
I	a symmetric irreflexive independence relation in every manuscript that uses one; 05 and 06 name maximal ones I_{max} and I_* .

Reading the merged text. [write] Text printed from a manuscript keeps that manuscript’s wording. In it, “this article”, “this paper”, “the present report” and “below” refer to that manuscript; theorem, section and equation numbers are this report’s. Where a manuscript names one of its files, the shipped name in code/ or data/ is printed (the README maps every delivered name to its shipped name), except inside a displayed command listing, which keeps the delivered layout because the programs import one another by their delivered names. A manuscript’s repository citations are to the pinned commit it names. Renamed macros and symbols: manuscript 02’s `\code` (small typewriter) and manuscript 05’s `\code` (a non-breaking URL) print as manuscript 01’s `\code`; the pin macros of manuscripts 01, 02, 04 and 07 (01’s and 07’s share the name `\repoSHA` but hold different commits) are printed as the literal commit identifiers; manuscript 06 defined a `\word` macro it never used, so manuscript 07’s is the only one. No mathematical symbol was renamed in Parts I–IX. The rest of this section prints each manuscript’s own conventions.

Parts X–XIII. [write] Manuscripts 08–11 (batch 62) keep their own letters and conventions, fixed at the start of each Part (Sections 73, 89, 93 and 110); the dictionary above covers manuscripts 01–07. Three letters acquire further meanings there: H is the largest consumption exponent of a priority program in Part XII; R counts the reactions of a network in Part XIII, not residuals; and L is a resource cutoff in Part XIII. Renamed for batch 62: manuscript 09’s tile palette T is written $\mathcal{T}$, and manuscript 08’s birth-order normal form is written NF_{birth} . Renamed macros that print the same glyphs: 08’s $\backslash\text{Bool}$ as $\backslash\text{B}$, 09’s $\backslash\text{ind}$ as $\backslash\text{indset}$, 11’s $\backslash\text{E}$ as $\backslash\text{cE}$; the $\backslash\text{repo}$ macros of 09 and 10, which print the word *ProveIt*, are replaced by the word, and their pin macros by the literal *commit*. The equations of Parts X–XIII are numbered within sections. Part XI’s lemmas on nonnegative cubics admit rational or real coefficients where they say so, and its comparison penalty accepts integer inputs; Part XII’s drifts are signed data derived from natural program entries. In Parts X and XIII, T is again a number of (macro)steps.

Part XIV. [write] Manuscript 12 (batch 63) keeps its letters too, fixed in Section 126. Its Cantor pairing is written π_C , because Part IX’s π is a different pairing, and its names *SF* and *FF* are Part XIII’s. In Part XIV, h_v is a height in a forest, not a history, and T counts the firings of an execution, a number determined by the input rather than a given horizon.

Part XV. [write] Manuscripts 13–15 (batch 78) keep their letters, fixed in Section 139. Several letters clash among them: L is a record count, a log length or a number of new cyclic wires; B is a carry-free base or a boundary; M is a matching, a partial matching or a packing bound; and $D_n = (2n - 1)!!$ is not Part V’s radix D . In Part XV, T is again an external horizon, and ℓ counts port-free cyclic wires. Its equations are numbered within sections; manuscript 14’s hand-set equation numbers are replaced by these.

2.1 Manuscript 05: Conventions and witness-preserving arithmetic

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. All unknowns are nonnegative integers unless a real relaxation is explicitly being considered. Polynomial equalities, including their subtractions, are interpreted in $\mathbb{Z}$ or $\mathbb{R}$, not using truncated subtraction on $\mathbb{N}$. A transition label identifies a transition, not merely its visible action name; distinct transitions with identical effects remain distinct labels.

Definition 2.1 (Representations and fibres). A polynomial $P(x, z) \in \mathbb{Z}[x, z]$ represents a relation R if

$$R(x) \iff \exists z \in \mathbb{N}^n P(x, z) = 0.$$

It is *finite-fold* if for each fixed x there are only finitely many such z , and *single-fold* if there is at most one. A representation of a set of computational objects is *parsimonious* here when an explicitly defined decoding map is a bijection from its entire natural zero set to those objects.

A finite number n of witness *coordinates* says nothing about the cardinality of a fibre. Even a one-variable equation such as $0z = 0$ has infinitely many natural solutions. Conversely, the polynomials in the main theorem generally have many solutions at a fixed pair of endpoints: one for each execution class. Their unique-extension property is conditional on the selected canonical word.

Lemma 2.2 (Unique slack and scalarization). Suppose $L_1(z), \dots, L_s(z)$ are integer polynomials. Then

$$\sum_{j=1}^s L_j(z)^2 = 0 \iff L_j(z) = 0 \text{ for every } j$$

over $\mathbb{Z}$ and over $\mathbb{R}$. For integer-valued expressions A, B , the inequality $A \leq B$ is equivalent to the existence of a unique $s \in \mathbb{N}$ satisfying $A + s = B$.

Proof. Every square is nonnegative. A sum of nonnegative reals is zero exactly when every summand is zero. The slack is necessarily $B - A$, which is a natural number exactly under the stated inequality. $\square$

Lemma 2.3 (Affine Boolean conjunction). *For fixed $p, q \in \{0, 1\}$, the natural solutions of*

$$p = u + s_1, \quad q = u + s_2, \quad 1 + u = p + q + s_3 \quad (2)$$

consist of exactly one tuple (u, s_1, s_2, s_3) , with $u = pq$. For real $p, q \in [0, 1]$, the same equations imply

$$\max(0, p + q - 1) \leq u \leq \min(p, q).$$

Proof. Nonnegativity gives $u \leq p$, $u \leq q$, and $u \geq p + q - 1$. For Boolean inputs these bounds determine u : it is zero when either input is zero and one when both are one. Each slack then has a prescribed value. The displayed real bounds are precisely the same inequalities. $\square$

This small gadget is responsible for the quadratic, rather than quartic, final polynomial. We do not use a nonunique decomposition into sums of squares to enforce nonnegativity: natural-variable domains provide it directly. Replacing all natural variables by arbitrary integer variables would invalidate the later one-hot and slack arguments. Replacing a natural variable by a sum of four squares preserves existence but not the claimed bijection.

2.2 Manuscript 01: Three different meanings of an exact representation

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. For an integer polynomial $P(x, w)$ and fixed parameter tuple x , let

$$Z_P(x) = \{w \in \mathbb{N}^k : P(x, w) = 0\}.$$

We distinguish three contracts.

Definition 2.4. An *existence representation* of a relation R satisfies $R(x) \iff Z_P(x) \neq \emptyset$. It is *single-fold* if $|Z_P(x)| \leq 1$ for every x , and *finite-fold* if $|Z_P(x)| < \infty$ for every x . Given a set $C(x)$ of semantic objects, a *parsimonious representation* of C is a specified bijection $Z_P(x) \simeq C(x)$.

A parsimonious history representation need not be single-fold: there can be several genuinely different histories between the same endpoints. The goal is not to identify those histories, but to avoid multiplying each of them by irrelevant choices. Conversely, a single-fold decision representation need not display a recognizable computation at all.

The universal single-fold and finite-fold questions are not solved by ordinary MRDP. Matiyasevich distinguishes ordinary polynomial representability from single-fold *exponential* representability and asks whether the two improvements can be combined [9]. We use this distinction as a precise boundary, rather than claiming that deterministic execution automatically yields a unique polynomial witness.

2.3 Manuscript 02: Single-fold arithmetic without hidden choices

Definition 2.5 (Single-fold representation). For a relation $R(a)$, a polynomial representation

$$R(a) \iff \exists w \in \mathbb{N}^m P(a, w) = 0$$

is *single-fold* if for every fixed input a there is at most one w satisfying the equation. Thus a true instance has exactly one witness and a false instance has none. It is *finite-fold* if every input has only finitely many witnesses.

Our convention is $0 \in \mathbb{N}$. Free parameters may range over $\mathbb{Z}$, while existential coordinates always range over $\mathbb{N}$. To obtain an entirely natural-input relation, replace each signed free parameter a by $a^+ - a^-$, with $a^+, a^- \in \mathbb{N}$. These are *free inputs*; their possible redundancy does not create extra existential witnesses for a fixed input pair. When a signed quantity itself is existentially quantified, its pair must be normalized as below.

2.3.1 Canonical signed coordinates

Lemma 2.6 (Signed normal form). *Every $v \in \mathbb{Z}$ has a unique representation*

$$v = v^+ - v^-, \quad v^+, v^- \in \mathbb{N}, \quad v^+ v^- = 0.$$

It is $v^+ = \max(v, 0)$ and $v^- = \max(-v, 0)$.

Proof. The product condition implies at least one coordinate is zero. If $v > 0$, then v^- cannot be positive or v^+ zero, so $(v^+, v^-) = (v, 0)$. The case $v < 0$ is symmetric, and $v = 0$ forces both coordinates to vanish. $\square$

Without the product condition, the pairs $(v^+ + k, v^- + k)$ would give infinitely many encodings. This elementary issue is not cosmetic: it is exactly the sort of witness multiplicity that the construction must avoid.

Lemma 2.7 (A total single-fold sign test). *For a given integer v , the following equations have exactly one solution $(b_-, b_0, b_+, s) \in \mathbb{N}^4$:*

$$\begin{aligned} b_\epsilon(b_\epsilon - 1) &= 0 \quad (\epsilon \in \{-, 0, +\}), \\ b_- + b_0 + b_+ &= 1, \\ v &= (b_+ - b_-)(s + 1), \\ b_0 s &= 0. \end{aligned} \tag{3}$$

The three bits respectively indicate $v < 0$, $v = 0$, and $v > 0$.

Proof. Exactly one of the three bits is 1. If $b_+ = 1$, the magnitude equation says $v = s + 1 > 0$ and determines $s = v - 1$. If $b_- = 1$, it says $v = -(s + 1) < 0$ and determines $s = -v - 1$. If $b_0 = 1$, it forces $v = 0$; the last equation then forces $s = 0$. The indicated assignment exists in each case, and the cases are mutually exclusive. $\square$

All equations in (3) are quadratic or linear when v is a linear expression in existing wire coordinates. In particular, the sign of a negative integer needs neither a square-root witness nor a choice of squares representing an inequality.

2.3.2 Total circuits and the quartic conversion

Lemma 2.8 (Canonical circuit lemma). *Let $\Phi(\mathbf{a})$ be a fixed quantifier-free Boolean combination of integer polynomial equalities and inequalities. Given an arithmetic and Boolean circuit of size S for Φ , one can effectively construct $O(S)$ natural auxiliary coordinates and quadratic integer residuals $Q_1, \dots, Q_N$ such that*

$$\Phi(\mathbf{a}) \iff \exists \mathbf{u} \in \mathbb{N}^{O(S)} \sum_{j=1}^N Q_j(\mathbf{a}, \mathbf{u})^2 = 0.$$

The auxiliary tuple is unique whenever it exists. The resulting polynomial has degree at most four in all its free and existential variables.

Proof. Evaluate the entire circuit in a fixed topological order. Represent every signed arithmetic output by its canonical pair from Lemma 2.6. For an addition gate, impose

$$z^+ - z^- = (x^+ - x^-) + (y^+ - y^-), \quad z^+ z^- = 0.$$

For multiplication, replace the right-hand side by $(x^+ - x^-)(y^+ - y^-)$. These equations have degree at most two, and the output pair is uniquely determined by the previously determined inputs. Fixed integer constants and signed free inputs are allowed in the same formulas. Affine expressions may be retained as expressions rather than stored as extra wires.

For an atomic comparison, first evaluate its polynomial difference and then use Lemma 2.7. Equality is the zero bit; strict inequality is one of the other bits. Non-strict inequality is the sum of the appropriate two bits. The Boolean operations are represented by

$$\neg b = 1 - b, \quad b \wedge c = bc, \quad b \vee c = b + c - bc.$$

Each output is uniquely determined and is automatically a bit when its inputs are bits. Store outputs as additional wires as necessary to keep every defining residual quadratic. Finally require the truth-output bit to be 1.

All internal computations, including those belonging to a disjunct that is not needed to make Φ true, are still evaluated. Consequently they have unique values and cannot introduce unconstrained witnesses. Induction along the circuit proves that all internal coordinates are uniquely determined by α before the final truth assertion is made. That assertion holds precisely when Φ is true.

A sum of squares of integers is zero exactly when each summand is zero. Squaring the quadratic residuals and summing them yields the advertised single equation and the degree bound. The number of residuals and variables per gate is bounded by an absolute constant. $\square$

Remark 2.9 (What is and is not being eliminated). The lemma removes Boolean structure and comparisons from a *fixed finite* formula. It does not eliminate an arbitrary bounded universal quantifier, nor a variable-length circuit. The next sections provide a special finite formula for one such quantifier, using polynomial degree.

Example 2.10 (An unsafe disjunction). The relation $a = 0 \vee b > 0$ can be expressed by $\exists s \in \mathbb{N} \ a(b - s - 1) = 0$. But when $a = 0$, every s is allowed. A product representing a disjunction preserves existence but not uniqueness. A total sign circuit has no such freedom. Canonical padding of inactive certificate components will play the same role below.

2.4 Manuscript 03: Conventions

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. Every polynomial has coefficients in $\mathbb{Z}$, but all input and witness coordinates range over $\mathbb{N}$. Subtraction in a displayed polynomial is subtraction in $\mathbb{Z}$, *not* truncated natural subtraction. These conventions are essential to the comparison gadget.

A *residual system* is a finite list $F_1, \dots, F_R$ of integer polynomials, required to vanish. Its associated polynomial is

$$P = \sum_{i=1}^R F_i^2. \tag{4}$$

At natural, integer, or real arguments, $P = 0$ is equivalent to simultaneous vanishing of all residuals. In particular, quadratic residuals produce a polynomial of degree at most four. We keep both representations: the residual list exposes the semantics, while (4) supplies the single equation.

Definition 2.11 (Canonical certificate). For a relation $R(x, y)$, a certificate $F(x, y, z) = 0$ is *canonical* when $R(x, y)$ holds exactly when there exists a witness z , and for each fixed (x, y) there is at most one such z . If R is the graph of a total function, there is exactly one pair (y, z) for each x .

The word *parsimonious* will mean that projection from polynomial solutions to the specified semantic executions is a bijection. Distinct labelled nondeterministic choices count as distinct executions, even when they later merge into the same machine state.

2.5 Manuscript 04: Arithmetic interfaces and canonical auxiliary variables

2.5.1 Domains, residuals, and witnesses

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. Every witness variable ranges over $\mathbb{N}$, but all polynomial expressions are evaluated in $\mathbb{Z}$. In particular, subtraction is ordinary integer subtraction, not truncated natural subtraction. A *residual system* is a finite list

$$F_1(\mathbf{x}) = 0, \dots, F_E(\mathbf{x}) = 0, \quad F_i \in \mathbb{Z}[\mathbf{x}]. \quad (5)$$

The single associated polynomial is

$$P(\mathbf{x}) = \sum_{i=1}^E F_i(\mathbf{x})^2. \quad (6)$$

Since an integer square is nonnegative, $P = 0$ if and only if every residual vanishes. If $\deg F_i \leq 2$, then $\deg P \leq 4$. This conversion introduces no variables and preserves the entire solution set, not just solvability.

A displayed sum of squares is already an explicit polynomial. Expanding it into a list of monomials can enlarge its description. Our exact equation counts count the residuals; they are not claims about a minimum number of operations or about the number of monomials in a fully expanded quartic.

Definition 2.12 (Witness-faithfulness). For fixed external data d , a residual system is witness-faithful for a set C_d if it comes with mutually inverse maps between C_d and all its natural solutions. If the objects are classes under an equivalence relation, the maps must be defined on classes, rather than on arbitrary representatives.

2.5.2 Parsimonious quadratization

The following familiar circuit principle will also prevent several common multiplicity errors.

Lemma 2.13 (Unique-extension quadratization). *Given a finite system of integer polynomial equations in natural variables $\mathbf{x}$, one can effectively construct a system of equations of degree at most two in $(\mathbf{x}, \mathbf{w})$ such that every original solution has exactly one extension $\mathbf{w}$, and no other solutions exist. Consequently the construction preserves finite-fold and single-fold fibers with respect to any distinguished input coordinates.*

Proof. Write each polynomial as $F^+ - F^-$, where F^+ and F^- have nonnegative integer coefficients. Construct acyclic arithmetic circuits for these two polynomials using natural constants, addition, and multiplication. For each gate introduce a fresh natural variable w . An addition gate gives $w - u - v = 0$ and a multiplication gate gives $w - uv = 0$; a constant gate gives $w - c = 0$. Finally equate the two output variables.

For any assignment to $\mathbf{x}$, evaluating the circuits gives an extension satisfying all gate equations. Conversely, topological induction through the circuit forces each gate variable to equal that value. The final output equality is precisely $F = 0$. Applying the construction to all residuals proves the assertion. Combining the quadratic residuals by (6) preserves this unique extension. $\square$

An arbitrary integer variable should not instead be replaced by $u - v$ with unrestricted $u, v \in \mathbb{N}$: the replacements $(u + k, v + k)$ immediately give infinitely many encodings of the same integer. The lemma avoids this problem by computing the positive and negative polynomial parts as *functions* of the original variables.

Lemma 2.14 (Canonical branch activation). *Suppose each labeled branch has a natural-witness certificate with at most one auxiliary assignment for fixed input and output. A finite branch choice can be compiled into quadratic residuals so that the resulting solutions correspond exactly to valid labeled choices, with no multiplicity from inactive branches.*

Proof. Use one-hot natural selectors x_a with $x_a(x_a - 1) = 0$ and $\sum_a x_a = 1$. For every branch-local witness $w_{a,j}$ impose $(1 - x_a)w_{a,j} = 0$. Thus all inactive branch witnesses are zero. Compute the positive and negative parts of each branch residual by the always-active circuits of Lemma 2.13; their outputs u, v are uniquely determined even when that branch is inactive. Gate the final equality by $x_a(u - v) = 0$, a quadratic equation. In the selected branch the original certificate holds and has its assumed unique extension; elsewhere the witnesses are fixed at zero and the circuit wires are forced. This proves the bijection. $\square$

The branch *label* is part of the computational object in this lemma. If two labels describe the same semantic event, a further canonicalization is needed. Also, directly multiplying a selector by a quadratic residual gives a cubic: computing its two polynomial parts first is what retains quadratic residual degree.

2.6 Manuscript 06: Domains, schedules, and three kinds of uniqueness

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. Every unknown in a Diophantine equation ranges over $\mathbb{N}$, while coefficients and polynomial subtraction belong to $\mathbb{Z}$. Thus $x - y$ in an equation is ordinary integer subtraction, never truncated natural subtraction. Vector inequalities, minima, and products explicitly called coordinatewise are taken coordinate by coordinate.

Definition 2.15 (Token transition). Fix $p \geq 1$. A transition a is a pair

$$u_a, v_a \in \mathbb{N}^p.$$

It is enabled at marking m when $m \geq u_a$, and its result is $m - u_a + v_a$. A finite ordered alphabet A labels the transitions; distinct labels may have identical numerical behavior. Write $q = |A| \geq 1$.

The model is exactly the firing semantics of a weighted Petri net once labels are assigned to transitions. It also describes multiset rewriting with a fixed finite species set. There is no test for the absence of tokens, no reset, copying operation, or multiplication of the current marking. An explicit input requirement permits a transition to consume and restore a catalyst. Such a transition cannot in general be replaced by its net displacement.

For a word $w = a_1 \cdots a_L$, write $m \xrightarrow{w} n$ when every successive transition is enabled and the final marking is n . The empty word ε is executable at every marking and leaves it unchanged.

Definition 2.16 (Parametrized repetition expression). Fix parameter names $k_1, \dots, k_r$. An expression has grammar

$$E ::= \varepsilon \mid a \mid (EF) \mid E^{k_j}.$$

An assignment $\mathbf{k} \in \mathbb{N}^r$ determines a finite word $\llbracket E \rrbracket_{\mathbf{k}}$ by concatenation and ordinary word powers. In particular, E^0 expands to ε , whether or not the expansion of E could execute from the given marking. The same parameter may occur at many nodes. The repetition depth is the maximum number of repetition nodes on a root-to-leaf path.

We count each empty expression, letter, concatenation, and repetition as a syntax node. Unless stated otherwise, the syntax is a tree. The template is fixed before the free numerical parameters are supplied. This is not a program that chooses a different syntactic body in response to its current counter values. Parameter-dependent choices will occur only inside the *certificate compiler*, and their values will be uniquely constrained.

Definition 2.17 (Single-fold auxiliary representation). A polynomial $Q(x, z) \in \mathbb{Z}[x, z]$ represents a relation $R(x)$ single-fold when

$$\#\{z \in \mathbb{N}^s : Q(x, z) = 0\} = \begin{cases} 1, & R(x), \\ 0, & \neg R(x). \end{cases}$$

A representation is finite-fold when this set is finite for every x and nonempty exactly on R .

The declaration of the free tuple x is part of the theorem. In our compressed compiler it includes the starting marking, ending marking, and *all* repetition parameters. If the parameters are subsequently quantified, the number of solutions can increase. A second uniqueness notion concerns the word represented by a parameter tuple. A third concerns the word selected inside a commutation class. These three notions will be kept separate.

Example 2.18 (An infinite multiplicity hidden by projection). For $E = (a^{k_1})^{k_2}$, every tuple with $k_1 = 0$ or $k_2 = 0$ expands to the empty word. Our compiler has exactly one auxiliary solution for each such tuple and equal endpoints. If (k_1, k_2) become witnesses too, the empty execution has infinitely many certificates. No local uniqueness gadget can remove this multiplicity without changing the parameterization.

2.7 Manuscript 07: Arithmetic and word interfaces

2.7.1 Natural witnesses, integer evaluation

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. Witness coordinates lie in $\mathbb{N}$, but polynomial coefficients and evaluation lie in $\mathbb{Z}$. In particular, every subtraction in this article is signed subtraction. Formal queue lengths after an illegal event may be negative; silently replacing them by truncated natural subtraction would invalidate several arguments.

Definition 2.19. A *residual system* is a finite list $R_1, \dots, R_s \in \mathbb{Z}[X_1, \dots, X_k]$. Its associated polynomial is

$$P = \sum_{j=1}^s R_j^2.$$

Its natural zero set is the set of $\mathbf{x} \in \mathbb{N}^k$ satisfying every residual equation $R_j(\mathbf{x}) = 0$.

The equivalence follows because each square is a nonnegative integer. A quadratic residual system therefore gives a polynomial of degree at most four. Introducing an extra coordinate for a circuit gate changes the witness count; simply naming a polynomial subexpression does not.

2.7.2 Arithmetic circuits are polynomial syntax

A directed acyclic arithmetic circuit with integer constants and addition, subtraction, and multiplication denotes one ordinary integer polynomial. Reusing a node is a way of writing a polynomial compactly. It is not existential quantification over the node's value. Our JSON artifacts make this distinction explicit: only the listed input variables are witnesses; internal nodes are evaluated deterministically.

Consequently an $O(T)$ circuit need not expand to $O(T)$ monomials. For example, a product of T affine factors may have exponentially many monomials when expanded. Circuit operations on large integers are also not unit-cost bit operations. We report degree, number of witness coordinates, arithmetic size, and witness bit height separately.

The horizon T , the input word, and the program are parameters of the *construction*, not additional unknowns. A symbol such as 2^T is an integer coefficient computed before solving the equation. Products indexed by $i < T$ are finite syntactic products for that particular polynomial.

3 The fifteen manuscripts

[write] Each subsection below gives one manuscript’s title, author line and date as delivered, its abstract and status statement verbatim, and then its own introduction (except manuscript 05’s, which is Section 1). The introductions describe each manuscript’s results in its own terms and numbering-free words; where they name a theorem, the reference points to where this report prints it.

3.1 Manuscript 05 (base): Canonical Trace Polytopes

[write] *Subtitle: “Witness-Preserving Diophantine Representations of Computational Substrates”. “Research memorandum prepared for Vladimir Reshetnikov”, September 30, 2026; pinned at e8bb0931d. Batch 60, manuscript 05 (canonical_trace_polytopes.zip). Printed in Section 1 (introduction), Section 2 (conventions), Parts I, II, VII, VIII and IX, and the back matter.*

Abstract. An existential Diophantine representation says that a computation exists; it need not preserve which computation occurred, how many computations occurred, or how many arithmetic witnesses describe it. We construct a stronger, bounded-horizon representation. For a Petri net with d places, m distinctly labelled transitions, and horizon T , an explicit convex quadratic integer polynomial has its nonnegative integer zeros in bijection with the commutation classes of enabled length- T executions between prescribed endpoints. The construction uses

$$(2T + 1)d + (7T + 1)m$$

unknowns. Its nonnegative real zero set is a bounded rational polytope, and every slack variable is uniquely determined by the canonical execution. A smaller quartic presentation is also given.

We derive an exact all-marking criterion for permissible commutations, realize arbitrary independence graphs by unit-weight nets, and prove an exponential automaton-state lower bound that explains the use of Boolean memory rather than an explicitly expanded automaton. Counting classes remains #P-complete even for unit-weight, 1-safe nets and their maximal static commutation relation. The same witness-preserving arithmetic methods apply to bounded counter programs, Boolean circuits, cellular automata, and explicitly size-bounded rewriting.

Finally, we prove a precise boundary theorem: a single finite-fold polynomial representation of the decidable verifier for canonical first-halting universal computations is equivalent to the general finite-fold Diophantine conjecture; the analogous statement holds for single-fold representations. The bounded constructions do not cross this boundary. Complete mathematical proofs, an exact-arithmetic reference implementation, reproducible finite checks, and a formalization and research agenda are included.

Status. This is a proved theorem package and executable research prototype, not a claimed resolution of the general finite-fold or single-fold conjecture. Classical ingredients are identified in the text. Global priority for the combined results has not been established, and the new proofs have not been checked in Lean.

[write] *Manuscript 05's introduction is Section 1.*

3.2 Manuscript 01: Canonical Diophantine Certificates for Causal Computation

[write] *Subtitle: “Exact commuting-block acceleration, quartic trace quotients, and the boundary of single-foldness”. “Research report prepared for the ProveIt project”, 30 September 2026; pinned at e8bb0931d. Batch 60, manuscript 01 (Diophantine_Causal_Computation.zip). Printed in Parts I, II, III, VII, VIII and IX and the back matter.*

Abstract. Ordinary Diophantine representability records whether a computation exists; it need not preserve how many computational objects exist. We develop explicit ordinary polynomial representations which control the entire witness fibre for a substantial class of computational substrates. An action is an integer translation restricted to a coordinatewise lower-and-upper guard. This includes weighted Petri transitions and finite-control counter instructions with zero tests.

Our first result gives the exact initial-resource box in which *every* ordering of a prescribed multiset of actions can execute. When the active actions commute as partial maps, the same box characterizes *some* ordering. For Petri transitions it reduces to total resource loss plus the largest retained resource requirement. The resulting accelerator is a single-fold polynomial of degree at most four, with witness arity independent of the multiplicities. A concrete three-action example uses eleven witnesses even for multiplicities 10^{100} , 10^{80} , 10^{90} .

Our second construction combines these exact resource tests with the classical Cartier–Foata normal form. For each fixed causal-height bound, the nonnegative roots of an explicit quartic are in bijection with executable trace classes, not with their interleaved schedules. All intermediate markings, guard slacks, and normal-form auxiliaries are uniquely determined. We give complete proofs, exact variable and equation counts, block-composition extensions, an executable compiler, and differential tests. Finally, we prove that a single fixed-arity single-fold representation for universal halting would imply the full single-fold MRDP principle. The bounded-height family and commuting-block accelerator do not make that unproved leap.

Status. This is an unrefereed mathematical development with reproducible exact-integer tests, not a Lean-checked extension of ProveIt. The particular resource formulas and compiler assemblies are derived and proved here; their priority in the wider literature is not established. Classical arithmetization, trace normal forms, and the finite-fold conjecture are explicitly distinguished from these constructions.

3.2.1 The representation problem beyond MRDP

What is already available in ProveIt The starting point is the ProveIt repository at the pinned revision

e8bb0931d67f80d9fce87a8cddb0f661ff19f956.

Its Hilbert-tenth development already supplies MRDP, finite-witness integer polynomials, and Diophantine closure under exact finite iteration. In particular, the inspected source `Diophantine/Common/DiophantineTrace.lean` contains the interfaces `boundedForall_dioph`, `exactIter_dioph`, and `existsExactIter_dioph`. These are existence statements about Diophantine representations; they do not assert uniqueness of auxiliary witnesses [1, 3].

This is an important distinction in selecting a research target. Merely observing that finite executions of an effective substrate form a computably enumerable relation, and then applying MRDP, would add little to that development. Nor would another general register-machine-to-Diophantine proof be new:

the repository includes the Jones–Matiyasevich 1984 route, and independent mechanized work uses Minsky machines and FRACTRAN [1, 8, 14].

The present target is *fibre control*: eliminate witnesses that encode nothing more than a choice of interleaving, an oversized coding base, an unconstrained inactive branch, or a noncanonical padding length. This is useful both mathematically, when counting solutions, and architecturally, when designing verified compilers that should preserve a specified semantic object rather than only existential truth.

Questions answered in this report The concrete questions are as follows. Can one characterize the exact resources needed to serialize a multiset without inspecting all its permutations? Can this become an ordinary low-degree polynomial with one complete witness tuple? Can a polynomial count causal traces rather than scheduling permutations? And which step would be needed to turn these constructions into a universal single-fold theorem?

The answers are given by the envelope theorem (Theorem 6.1), the fixed-arity accelerator (Theorem 21.1), the causal-height compiler (Theorem 20.1), and the universal-halting equivalence (Theorem 62.1). These are scoped representation results, not a claim to have settled a previously named major open conjecture.

Component	Role and provenance
MRDP and exact iteration	Existing repository infrastructure; used as context, not as the proof of the explicit compilers.
Cartier–Foata layers	Classical normal form; a self-contained proof of the version used here is supplied.
Resource envelope	Direct extremal-prefix derivation for arbitrary repeated box-guarded translations.
Quartic compilers	Explicit constructions with proofs of all auxiliary-witness uniqueness claims.
Global compression	Reduced to the single-fold or finite-fold MRDP principle; not claimed to follow from the bounded construction.
Formal verification	Proposed integration work. The attached Python tests are not proof-assistant verification.

3.3 Manuscript 02: Canonical Diophantine Certificates for Polynomial Trajectories

[write] Subtitle: “Single-fold acceleration, finite-difference sign tables, and the boundary of general computation”. “Research manuscript for the ProveIt project. Prepared with ChatGPT”, September 30, 2026; pinned at e8bb0931d. Batch 60, manuscript 02 (canonical_diophantine_certificates.zip). Printed in Section 2 (its Section 2) and Parts IV, VII, VIII and IX and the back matter; the first two subsections of its introduction open Part IV (Section 27).

Abstract. The existence of a Diophantine representation of finite computation does not by itself control the number of representing witnesses. We develop a canonical certificate construction for a substantial restricted class of computations: trajectories polynomial in the iteration counter, with arbitrary fixed Boolean combinations of polynomial guards.

The main construction eliminates a bounded universal quantifier over one integer variable without exponentiation and without an existentially chosen Chinese-remainder modulus. For every fixed degree bound d , the relation $\forall i \in \{0, \dots, T\} p(i) \geq 0$ has a single-fold representation by one integer polynomial of degree at most four, with $O((d+1)^3)$ natural witnesses, independently of T . The

geometric part consists of exactly $(d + 1)(2d + 1)$ coordinates. These record the unique maximal sign intervals of $p, \Delta p, \dots, \Delta^d p$, with canonical padding. Monotonicity between successive difference levels reduces verification to endpoint tests.

We prove single-fold representations for exact guarded execution and first exit of polynomially iterable maps, including unitriangular nonlinear updates, unipotent affine maps, and a certified eventually quasi-polynomial affine extension. A canonical root bound also gives single-fold certificates for nontermination from a specified input. We provide a reproducible integer-only certificate generator, checker, and compiler to explicit quadratic residuals whose squared sum is the promised quartic.

The contribution is a constructive restricted result, not a solution of the general finite-fold or single-fold representation problem. The proofs are given here; historical priority of this particular construction has not been established, and no proof-assistant verification is claimed.

3.3.1 The problem addressed and the contribution

What is inherited from ProveIt We inspected the ProveIt repository at commit `e8bb0931d67f80d9fce87a8cddb0f661ff19f956`. Its `Computability/HilbertTenthProblem` development contains an MRDP interface and a finite-trace interface. In particular, `DiophantineTrace.lean` provides ordinary Diophantine closure for bounded universal quantification, exact iteration, and finite reachability [3]. These interfaces do not state uniqueness of the auxiliary witnesses. The project also maintains corrected editions of the Jones articles, including the Jones–Matiyasevich register-machine treatment of exponential Diophantine representation [1].

We use this as a precise project-level motivation: ordinary finite reachability is already available, so another invocation of MRDP would not constitute the intended advance. The new target is a restricted *multiplicity-preserving acceleration interface*. None of our proofs requires assuming an unproved repository theorem. Conversely, source inspection is not a fresh compilation or transitive axiom audit of the repository, and we make no such claim.

Relation to prior work and novelty status There are two relevant bodies of prior work. Single-fold specifications require care with arithmetic auxiliary variables and disjunction; the recent treatment by Cantone, Cuzzio, and Omodeo discusses these issues and the role of exponentiation [10]. Our total-circuit normalization is an explicit implementation-oriented formulation of elementary ingredients, not a claim that canonical arithmetic gadgets are new.

Polynomial and triangular loop analysis also has substantial prior literature. Hark, Frohn, and Giesl study triangular weakly nonlinear loops with polynomial Boolean guards and reductions for termination [25]. Lommen, Meyer, and Giesl use closed forms and stabilization thresholds in modular complexity analysis [26]. We therefore do not claim to introduce polynomial loop acceleration, polynomial closed forms, or input-dependent termination bounds.

The result developed here is the particular *canonical finite-difference certificate*, its explicit cubic witness bound, and its use in single-fold ordinary polynomial representations of execution. The cited papers were checked for the surrounding landscape, not exhaustively mined to prove historical priority. Accordingly, this manuscript offers complete proofs of a specific constructive theorem, while leaving priority and publication novelty to further expert comparison. Its usefulness does not depend on labeling a standard consequence as a breakthrough.

3.4 Manuscript 03: Linear-Size Canonical Diophantine Certificates for Random-Access and Graph Computation

[write] Subtitle: “Quartic equations, unique witnesses, and exact path weights”. “Research note prepared for Vladimir Reshetnikov”, September 30, 2026; pinned at e8bb0931d. Batch 60, manuscript 03 (ProveIt_Linear_Size_Diophantine_Certificates.zip). Printed in Section 2 and Parts V and IX and the back matter.

Abstract. MRDP settles the existence of Diophantine descriptions of effective computation, but does not automatically preserve individual executions or control artificial witness multiplicity. We give two explicit canonical compilers for finite random-access memory logs over the natural numbers. Every valid log has exactly one auxiliary witness, and every invalid log has none.

The stronger scalar-size result uses exactly $24L$ witnesses and $22L + 1$ quadratic residuals for a length- L log. Their sum of squares is a sparse polynomial of degree at most four. A proposed sorted list is certified by packing complete records in a uniquely fixed radix and comparing two products at a base large enough to make coefficient carries impossible. The product identity is deterministic and exact, not a probabilistic fingerprint. All large powers are computed by quadratic chains. If the input coordinates have b bits, the largest witness has $O(L^2(b + \log(L + 2)))$ bits.

A complementary low-height compiler uses a canonical two-equation comparison gadget and a sorting network. Its exact counts are $10C + 2L + 3(N - 1)$ witnesses and $10C + 3L + 4(N - 1) + 1$ residuals for N padded records and C comparators. It gives explicit $O(L \log^2(L + 2))$ size, or $O(L \log(L + 2))$ using optimal networks, while keeping every witness to $b + O(\log(L + 2))$ bits.

We prove parsimonious composition with RAM instruction certificates, apply it to counter, cellular, stack, and immutable combinator-graph computation, and derive exact execution-count and first-halting probability formulas. We also prove the boundary: fixed-variable single-fold compression of one universal halting substrate is equivalent to the general single-fold conjecture. Our constructions are horizon-indexed families, not that compression. Both memory backends emit actual sparse quartics and pass reproducible exhaustive and adversarial tests. The proofs are ordinary mathematical proofs, not newly Lean-verified results; literature-wide priority is not claimed.

Keywords. Diophantine representation; random-access memory; parsimonious compilation; sorting networks; combinatory logic; finite traces.

3.4.1 The research target and its relation to ProveIt

What is already available The repository snapshot used for this investigation is

e8bb0931d67f80d9fce87a8cddb0f661ff19f956.

Its Hilbert-tenth documentation reports MRDP in both directions, a register-machine encoding, and reusable interfaces for bounded quantification, exact iteration, and finite reachability. Its combinatory-logic documentation also carefully distinguishes positive-step simulations from operational reflection and full abstraction. These are the relevant starting points [1, 2, 4]. We inspected these documents at the pinned revision; we did not rebuild the repository or independently audit its entire formal development.

The classical register-machine route is represented by Jones and Matiyasevich [8]. Carneiro’s Lean development supplies relevant historical context for machine-checked Diophantine exponentiation [15]. Neither the classical theorem nor an existential library interface automatically gives a one-to-one correspondence between program executions and polynomial witnesses. That missing contract is the target here.

A statement that a Turing-complete substrate is Diophantine, obtained only by composing its interpreter with MRDP, would not be a new mathematical result. Instead we ask a quantitative and intensional question:

Can arbitrary finite memory access be represented by a sparse quartic equation, with exactly one auxiliary witness for each valid access log, without expanding the entire memory address space or introducing a factor proportional to address bit width in the number of arithmetic constraints?

The answer is yes. Sections 36–40 give two complete constructions: a low-height sorting-network compiler and a linear-size large-base-product compiler, both with exact counts and executable artifacts.

Prior art and the actual contribution Sorted memory transcripts and routing-network checks are established techniques. In particular, the RAM arithmetization in *SNARKs for C* checks local transitions and a transcript ordered by address and timestamp, using routing constraints [27]. We do not claim to invent that architecture or quasilinear verification. The distinction pursued here is an exact natural-number polynomial with a fully specified, unique auxiliary fiber, including the comparison and routing data. A deterministic comparison network makes this contract transparent. This is a different guarantee from merely exhibiting some valid routing.

Batcher’s sorting construction and the asymptotically optimal sorting-network theorem are also established results [28, 29]. Our proofs identify the precise arithmetic interface they need and quantify the complete resulting Diophantine certificate. The explicit bitonic version is proved and implemented below. Its improved network bound uses the external optimal-network theorem; no AKS implementation is supplied. The separate linear-size construction in Section 40 does not use that theorem or any sorting network. It uses an elementary carry-free integer evaluation argument to certify a unique sorted record list, paying for fewer coordinates with greater height.

Status	Content
Established background	MRDP; sorting-based RAM verification; Batcher and AKS sorting networks.
Proved in this note	Two exact canonical memory compilers; linear scalar size; witness and residual counts; size–height tradeoffs; composition and weighted-path consequences.
Executable artifact	A general symbolic memory-log compiler, sparse quartic export, canonical witness generation, and deterministic regression tests.
Not established here	Literature-wide novelty of every formulation; fixed-variable single-fold representation of arbitrary halting; a formal verification of this new compiler; full abstraction of ProveIt’s existing lambda-to-combinator translations.

This status distinction matters. The result is a concrete research extension with complete proofs, not a declaration that a famous open conjecture has been solved. Its potential value to ProveIt is a stronger reusable specification: existence, uniqueness, decoding, and quantitative bounds travel together.

3.5 Manuscript 04: Witness-Faithful Diophantine Compilation

[write] *Subtitle: “Concurrent traces, priority arithmetic, and context-local rewriting”. “Research article prepared for Vladimir Reshetnikov”, September 30, 2026; pinned at e8bb0931d. Batch 60, manuscript 04 (Witness_Faithful_Diophantine_Compilation.zip). Printed in Section 2 and Parts II, VII, VIII and IX and the back matter.*

Abstract. Existential Diophantine representability is already available for every computably enumerable computational semantics. A more discriminating question is whether arithmetic solutions faithfully represent the intended computational objects, without spurious choices of auxiliaries or repeated encodings of the same concurrent execution. We develop three explicit constructions addressing this question. For a Petri net with p places and r labeled transitions, exact T -step executions between fixed endpoints, modulo a specified disjoint-support commutation relation, are in bijection with the natural zeros of a polynomial of degree at most four in $(p + r)(2T + 1)$ variables. The construction uses a quadratic, r -bit monitor for lexicographic trace normal forms; a matching linear-in-alphabet memory lower bound follows from an explicit family requiring exponentially many deterministic automaton states. For an ordered r -fraction FRACTRAN program, an exact run is represented uniquely using $T(7r + 2) + 1$ variables and $T(8r + 3) + 2$ quadratic residuals. For SKI rewriting along a fixed schedule, a context-local encoding uses whole-subterm integer codes, with size linear in the schedule and its total context depth, rather than in the number of tree nodes traversed by a full tableau.

All constructions include two-way proofs, exact resource counts, and executable exact-integer certificates. We distinguish these bounded, witness-faithful constructions from fixed-arity single-fold and finite-fold representations of unbounded computation. The latter are not obtained by determinism alone. The article identifies the precise compression frontier, gives a witness-height obstruction, and proposes a research program linking these constructions to the existing computability developments in ProveIt. Global priority for the particular constructions is not established; no resolution of the general finite-fold or single-fold problem is claimed.

Keywords. Diophantine representation; arithmetic certificates; Petri nets; trace monoids; FRACTRAN; combinatory logic; single-fold representations.

3.5.1 The problem beyond existential representability

What is already settled The Matiyasevich–Robinson–Davis–Putnam theorem, abbreviated MRDP, identifies computably enumerable subsets of the natural numbers with Diophantine sets [7]. Thus a computational substrate with an effective finite-execution semantics generally presents no new *existence* problem: its finite reachability relation is computably enumerable and hence Diophantine. The constructive route through register machines has a substantial history; Jones and Matiyasevich give an exponential-Diophantine register-machine construction, and Larchey-Wendling and Forster’s full mechanization uses FRACTRAN as an intermediate language [8, 13]. Re-proving that FRACTRAN, SKI reduction, or Petri-net reachability is merely Diophantine would therefore miss the interesting issue.

The inspected ProveIt snapshot already contains MRDP interfaces, bounded universal quantification, exact iteration of a Diophantine relation, and finite reachability [1, 3]. Its combinatory-logic development supplies concrete operational semantics and compiler simulations [4]. These are appropriate foundations, but ordinary existential equivalence says nothing about how many arithmetic witnesses encode one execution.

We instead ask for an explicit map

$$\{\text{intended computational objects}\} \longleftrightarrow \{\text{natural solutions of an arithmetic certificate}\}$$

that is a *bijection*. Depending on the substrate, the intended object is a labeled sequential run, a commutation class of concurrent runs, or a reduction along a specified schedule. The distinctions matter. Two different firing words may describe the same concurrent trace, while two distinct traces may have the same endpoint and the same transition counts.

The contributions and their scope The principal construction is a direct arithmetic quotient of Petri-net executions by independent swaps. It does not first enumerate a potentially exponential-state

normal-form automaton. A bit for each transition records exactly the information needed to prohibit a nonminimal linearization. The resulting polynomial has one solution per concurrent trace class, not one solution per scheduling permutation.

The second construction handles *priority*. FRACTRAN is deterministic only after the least applicable fraction is selected. A polynomial that merely allows any applicable fraction models a different language. We build the least-index choice into a cumulative prefix circuit and force every division and inactive slack to be canonical.

The third construction handles *locality*. A single integer represents a whole SKI subtree. A prescribed contextual contraction need decompose only the path to the redex and the finite pattern at its root. The result is a compact certificate in variable count, but not necessarily in bit length. This is a useful separation of two notions of succinctness, not a claim that integer coding removes all costs of large terms.

Construction	Object represented	Natural-solution multiplicity
Petri net, fixed T	A class of enabled words under the chosen independent swaps	Exactly one per class
FRACTRAN, fixed T	The exact priority-respecting run	Zero or one
SKI, fixed schedule	The scheduled contextual reduction	Zero or one
Unbounded reachability	Existence of some finite execution	MRDP supplies existence, not these multiplicity guarantees

The exact compiler theorems, their joint interpretation, and the resource accounting are the results developed in this article. The normal-form criterion, circuit encodings, and general MRDP implications are established background or elementary consequences, and are identified as such. The constructions have not undergone independent peer review or a new proof-assistant check. The accompanying tests are reproducibility evidence, not substitutes for the proofs.

Related work and a necessary distinction Lexicographic trace normal forms have a classical forbidden-factor characterization, often attributed to Anisimov and Knuth; a modern use appears in Lohrey, Stober, and Weiß [20]. Counting trace classes is also an active subject. De Colnet, Meel, and Mathur study counting and sampling the classes intersected by a regular language, presented by a finite automaton, and obtain complexity and approximation results [21]. Our purpose is different: a succinct operational model is compiled into an exact polynomial with controlled fibers. No new general trace-counting principle is asserted.

A subtlety is that a class intersecting an arbitrary language need not have its lexicographically least member in that language. Our Petri-net endpoint languages are *trace-closed* for the specified independence relation. This closure, proved below, is what makes normal-form filtering preserve the desired set of classes.

3.6 Manuscript 06: Saturation and Single-Fold Diophantine Compilation of Concurrent Counter Programs

[write] *Subtitle*: “Resource summaries, canonical traces, and nested repetition schemes”. “Research article. Prepared for Vladimir Reshetnikov”, September 30, 2026; pinned at e8bb0931d. Batch 60, manuscript 06 (diophantine_counter_programs.zip). Printed in Section 2 and Parts I, II, III, VIII and IX and the back matter.

Abstract. We give an explicit, witness-preserving route from structured concurrent counter executions to ordinary polynomial equations over the nonnegative integers. A firing word is summarized by the least initial resources it needs and the resources it leaves at that least input. This classical partial-translation algebra admits exact composition and arbitrary-power formulas with unique quadratic witnesses. Independently, we construct a Boolean transfer signature for lexicographic trace normality and prove that every signature satisfies $S^2 = S^3$. Consequently, in an arbitrarily nested repetition expression, normality depends on each repetition parameter only through the three cases 0, 1, and at least 2.

Combining the two summaries gives a polynomial of degree at most four with $O((p+q)N)$ auxiliary variables for a schedule with N syntax nodes, p resources, and q transition labels. For fixed endpoints and repetition parameters, there is exactly one auxiliary solution precisely when the specified execution is valid and, optionally, already canonical. The expanded execution length does not occur in this variable bound. A separate bounded-word construction gives a genuine bijection between polynomial solutions and executable trace classes.

We also locate a precise expressiveness boundary. Flat repetition schemes have Presburger-definable execution relations, whereas separated increment/decrement schemes of nesting depth two represent arbitrary systems of quadratic equations. For one counter, separated depth- d schemes correspond exactly to degree- d polynomial equations. Finite-fold and single-fold Diophantine representability are equivalent to the corresponding bounded-multiplicity parameterizations by separated one-counter schemes of depth four. These equivalences identify the remaining obstruction; they do not resolve the finite-fold or single-fold conjectures. An executable compiler and 49,722 finite checks accompany the proofs.

Status. The article contains mathematical proofs and tested prototype code, not a new kernel-checked formalization. Classical ingredients are identified explicitly. Historical priority for the combined constructions and formulations is not established by this study.

3.6.1 Purpose, context, and the main results

A general existence theorem can hide exactly the information needed by a compiler. The Matiyasevich–Robinson–Davis–Putnam theorem says that every computably enumerable relation has a Diophantine representation [7]. It does not, by itself, say how to preserve a particular execution, how many witnesses that execution acquires, how large those witnesses are, or whether two witnesses encode the same concurrent behavior. These distinctions become essential when Diophantine representation is used as a computational substrate rather than only as an undecidability theorem.

The motivating repository, *ProveIt*, already documents MRDP in both directions, finite-trace and bounded-quantifier interfaces, register-machine encodings, and universal Diophantine equations [6]. Its relevant status register also distinguishes completed formal results from remaining obligations [6]. Repeating that a computable interpreter has a Diophantine graph would add little to that program. We instead develop a direct compiler with a precise contract: a specified structured execution receives *one and only one* auxiliary certificate, with a degree and size bound visible from its syntax.

There are two logically different sources of multiplicity. An arithmetic encoding can create many witnesses for one computation. Independently, a concurrent computation can have many interleavings, or a repetition expression can have many parameter tuples that expand to the same word. We eliminate the first multiplicity. For all words of a fixed length we can also eliminate the interleaving multiplicity. We do not silently identify either achievement with eliminating multiplicity from arbitrary existential repetition parameters.

The three principal constructions For a finite collection of token-consuming and token-producing transitions, a word induces a partial translation of $\mathbb{N}^P$. Its domain is a principal upper orthant. The least

input and its output provide an exact resource summary. Composition uses coordinatewise cancellation; repetition has a closed formula including the exceptional zero-power case. All local choices can be expressed by uniquely solvable quadratic systems.

The second construction concerns a different object: the lexicographically smallest word in a trace class. The familiar normal-form language is regular, but constructing a full transition table on all subsets of the alphabet would be wasteful. We describe its block action with $3q + 1$ Boolean coordinates, subject to a disjointness invariant. This description admits linear-size composition circuits. More importantly, its powers stabilize after the second power. The stabilization remains valid inside arbitrary nested repetitions.

The third construction combines these branches without conflating them. The resource summary is invariant under the permitted commutations. The normal-form signature is intentionally *not* invariant: it distinguishes the chosen representative from the other words in the same trace. Their quadratic constraints can nevertheless be joined in a single sum of squares.

Theorem 3.1 (Summary of the compilation results). *Let E be a fixed parametrized repetition expression with N syntax nodes over q transitions acting on p nonnegative counters. Let I be a fixed symmetric, irreflexive relation of globally commuting transition labels.*

- (i) *Whether the expansion of E is lexicographically least in its I -trace class is unchanged when every repetition parameter k is replaced by $\min(k, 2)$.*
- (ii) *One can construct an integer polynomial of degree at most four with $O((p + q)N)$ auxiliary variables whose natural solutions, for fixed endpoints and repetition parameters, form a singleton exactly when the specified execution is valid and already lexicographically least. Without the normality requirement the auxiliary bound is $O(pN)$.*
- (iii) *For fixed length $T \geq 1$, one can instead construct a quartic polynomial with*

$$q(2T + 1) + p(7T - 4)$$

witness variables whose solutions are in bijection with the executable I -trace classes of length T between the specified endpoints.

Part (i) is proved in Section 24, part (ii) in Section 25, and part (iii) in Section 18. A second group of results, in Sections 26 and 67, characterizes exactly where unbounded parameter search restores the full difficulty of Diophantine solvability.

What is classical, and what is proved here The algebra of one partially blind counter is the bicyclic monoid; products of such storage components are standard in the study of valence automata [23]. Lexicographic trace normal forms go back to Anisimov and Knuth [19]. The conversion of arithmetic circuits into quadratic systems, followed by a sum of squares, is a standard degree-reduction method. Single-fold representation and its closure issues have an extensive literature [9, 10]. None of these ingredients is presented as a new discovery.

The contributions established in this article are the explicit synchronized construction, its local-to-global uniqueness proof, the compact normality signature and its cap-at-two consequence for nested schemes, the stated witness and trace-counting bounds, and the solution-preserving substrate normal forms. We give full proofs rather than relying on novelty labels. A targeted literature review did not establish historical priority for every equivalent formulation. In particular, a useful new implementation or packaging of known algebra should not be confused with a first theorem about that algebra.

All statements below are independent of any unverified theorem elsewhere in ProveIt. The repository is the motivation and a prospective integration target. The source audit used its tree at commit e8bb0931d67f80d9fce87a8cddb0f661ff19f956; it was not a rebuild or an independent audit of the repository's proof kernel dependencies.

3.7 Manuscript 07: Causality Without Tableaux: Compact Diophantine Certificates for Queue Universality

[write] Subtitle: “Product guards, tag systems, and optimal polynomial word-memory dimension”. “Research article. Prepared for Vladimir Reshetnikov”, September 30, 2026; pinned at 725d2ebb6. Batch 61, its only manuscript (*Causal_Diophantine_Queue_Compilation.zip*); numbered 07 in this report. Printed in Section 2 and Parts I, VI and IX and the back matter.

Abstract. We construct ordinary integer polynomials whose natural zeros faithfully encode finite computations of cyclic tag systems and general deletion-tag systems. The main cyclic-tag construction uses exactly $T + 1$ variables for a T -step execution, has degree at most $\max\{4, 2T\}$, and admits a linear-size arithmetic circuit. Its variables are the consumed bits and one uniquely determined slack; no intermediate queue is quantified. The key is a causal reconstruction theorem: one global word identity becomes sufficient when accompanied by a product guard. A first-failure lemma proves that this guard is positive for legal executions and exactly zero for every illegal candidate, even if later formal resource values become negative.

Two alternatives expose the cost tradeoff. A bounded-root construction removes the slack at the expense of a larger circuit. A local radix construction yields a quartic polynomial in $3T - 2$ variables. The general deletion- d construction uses $dT + 1$ variables, including all consumed symbols, with an explicit degree bound and explicit denominator clearing. Fixed-read FIFO networks admit a corresponding consumed-symbol-plus-one bound.

A separate obstruction explains the radix architecture. Finite words admit a faithful two-coordinate natural-number encoding with affine insertion at either end. No faithful one-coordinate natural-number encoding can make both binary prepends and even one fixed append polynomial. An elementary positive-leading-coefficient centralizer argument proves this dimension lower bound.

The article gives complete mathematical proofs, executable compilers, an independent exact-integer verifier, finite exhaustive checks, and a proposed formalization and research program. These are horizon-indexed, witness-faithful constructions, not a fixed-arity finite-fold representation of unbounded halting. Historical priority for the particular constructions is not established; no general finite-fold conjecture is claimed solved.

Verification status. The proofs below are mathematical proofs and the supplied Python artifacts have been tested. This package does not contain a new kernel-checked Lean or Rocq formalization. Repository audit reports are distinguished from checks performed for this article.

3.7.1 The question beyond representability

Why another Diophantine construction? The Matiyasevich–Robinson–Davis–Putnam theorem identifies computably enumerable sets of natural numbers with Diophantine sets. Thus, once a computational substrate has an effective finite-execution semantics, the mere existence of a Diophantine representation of its halting relation is not a new universality theorem. The interesting questions concern the structure of the representing polynomial: its variables, degree, coefficient size, circuit size, and the relationship between its solutions and actual computational histories. The formal developments of Carneiro and of Larchey-Wendling and Forster provide important precedents for making such reductions precise [15, 13].

ProveIt’s Hilbert-tenth-problem development already supplies the forward and reverse MRDP interfaces, finite polynomial representations, and trace infrastructure [5, 1]. We therefore do not repackage “this machine is effective, hence MRDP applies” as a new result. Instead we ask three concrete questions.

Can a certificate for a variable-length queue avoid storing every intermediate queue? Can a global equality between the words consumed and produced certify an execution without permitting a symbol to create itself after an illegal read? Finally, can a natural-number memory with polynomial operations at both ends really be reduced from two coordinates to one? Theorems 46.1, 8.1, and 50.4 answer these questions in explicit models.

Cyclic tag systems are a useful test case: their binary queue semantics is very small, yet the class is computationally universal. Cook uses them as an intermediate substrate in his universality proof for Rule 110 [31]. Tag-system simulation also has a quantitative literature; Woods and Neary establish polynomial-time simulation results rather than merely abstract universality [32]. We use these existing universality results as context. Our proofs concern arithmetic certificates for the precisely defined queue dynamics below.

The results at a glance For a fixed cyclic-tag program, input word, terminal word, and positive horizon T , the main construction represents the exact T -step execution relation. For empty terminal word it represents first halting at exactly T . Every successful instance has exactly one natural solution.

Construction	Variables	Degree bound	Residuals
Causal word identity	$T + 1$	$\max\{4, 2T\}$	$T + 3$
Bounded-root, no slack	T	$2 \max\{2, T, H\}$	$2T + 2$
Local radix, empty endpoint	$3T - 2$	4	$3T$

Here H is an explicit upper bound on queue length, defined in Section 48. The first and third rows have $O(T)$ arithmetic-circuit size for a fixed program; the second has $O(T(1 + H))$ size. All three denote ordinary polynomials, not expressions with variable exponents or an unbounded product operator.

For deletion- d tag systems over an alphabet of size B , we give $dT + 1$ variables and degree at most

$$2 \max\{B, T(B - 1), d(T - 1)(B - 1), 1\}.$$

The separate memory theorem is an actual lower bound, but in a carefully specified architecture: faithful word encodings with polynomial insertion maps. No lower bound on the number of variables of arbitrary Diophantine representations is asserted.

Relation to earlier work and scope of novelty [write] Manuscripts 04 and 06 of this report treat concurrent resource traces, priority arithmetic in FRACTRAN, context-local SKI rewriting and compressed counter schedules (Parts II, III and VII). This article instead treats FIFO word content, temporal availability of produced symbols, and the dimension of polynomial word memory. The companion articles are not proof dependencies.

Radix encodings, sums of squares, finite-table interpolation, and polynomial centralizers are established mathematical tools. Our contribution is the explicit construction and proof of the displayed resource guarantees, the causal obstruction and its repair, and the sharply stated word-memory consequence. The centralizer proof is self-contained; the broader theory has a substantial literature, for example Atela and Hu [33]. A targeted literature check does not establish historical priority for every formulation here. These results should be evaluated as proved constructions and a proved architectural obstruction, not as a claim that a named longstanding conjecture has been settled.

3.8 Manuscript 08: Names to Numbers

[write] Manuscripts 08–11 were delivered in batch 62 of ProveIt’s incoming reports, after the rest of this report had been written; each is printed in a Part of its own (Parts X–XIII), and this section gives its title page,

abstract, status statement and introduction. Title-page heading “Computability / Diophantine representation”; subtitle “Exact Quartic Certificates for Dynamic Heaps and Local Graph Computation”. “A research article developed for the ProveIt program”, 30 September 2026; pinned at 4e128356d. Batch 62, manuscript 01 (Names_to_Numbers_Diophantine_Heaps.zip). Printed here (abstract, status, Section 1) and in Part X (Sections 2–14 and both appendices).

Abstract. An effective finite-trace computational model already has a Diophantine halting relation by MRDP. The more discriminating problem is to preserve the internal structure of its computations: memory reads must see the latest write, fresh object names must not create spurious multiplicity, and arithmetic helper variables must not introduce extra witnesses. We construct an exact compiler for these requirements over the nonnegative integers.

For a log padded to N memory events and any sorting network with C comparators, the memory layer uses exactly $10C + 4N - 3$ auxiliary variables and $10C + 7N - 4$ quadratic residual equations. Their sum of squares is one polynomial of degree at most four. A fixed log has exactly one natural auxiliary solution when it is valid and none otherwise. Addresses and stored values need not be bounded in advance. An implemented bitonic network gives $C = Nk(k + 1)/4$ for $N = 2^k$; substituting an established linearithmic sorting network gives an $O(N \log N)$ existence result.

A second construction canonically numbers objects by their ordered birth events. For fixed local heap programs this yields a bijection between natural zeros and finite runs modulo fresh-name permutations, while retaining genuine schedule choices. We give a direct Turing-machine simulation by pointer-only two-stack heaps, a transfer theorem for bounded-local graph programs, a canonical quadratic treatment of inactive branches, explicit height bounds, and a computability obstruction to existential certification of perpetual memory safety. The archive contains the implemented memory compiler, executable checks, and a numerical certificate.

Status and scope. Full mathematical proofs are supplied for the stated constructions. The memory layer and pointer-log demonstrations are implemented and tested; the complete source-language compiler is a mathematical construction, not a delivered implementation. This work is not Lean-certified or independently refereed. The specific assembly and quantitative contracts are research contributions of this draft; historical priority is not established. No resolution of the general finite-fold or single-fold MRDP questions is claimed.

3.8.1 The problem being solved

Beyond the existence of a representing polynomial Suppose a computational system has effectively encoded finite configurations and an effectively checkable one-step relation. Finite accepting computations can be enumerated. Consequently, the existence of some polynomial representing acceptance is already a consequence of MRDP. Merely replacing a Turing machine by an unusual substrate does not, by itself, produce a new representability theorem. The repository’s formal MRDP development already provides this existence-level foundation [1].

The useful additional questions concern the representation itself. Can it recognize arbitrary dynamic memory accesses without writing a complete heap at every time? Can it preserve the number of computational witnesses? Can it remove the infinitely many choices of otherwise irrelevant fresh names? Can all the resulting equations remain quadratic, so that one sum of squares has degree at most four? Finally, which parts survive when the time bound is removed?

We answer these questions for a precisely specified class of finite computations. The central achievement is a *composable exact certificate contract*, rather than a claim that a previously non-Diophantine universal substrate has become Diophantine.

Main results at a glance Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. An auxiliary solution always means a solution in $\mathbb{N}$, not an unrestricted integer or real solution.

Theorem 3.2 (Overview). *The following constructions are effective.*

- (i) *A length- $m \geq 1$ memory log can be represented by one nonnegative quartic polynomial with unique auxiliary extension for every valid source log. For $N = 2^{\lceil \log_2 m \rceil}$ and a C -comparator sorting network, the exact memory-layer counts are*

$$A_{\text{mem}} = 10C + 4N - 3, \quad E_{\text{mem}} = 10C + 7N - 4.$$

Here the $3m$ address/flag/value source coordinates are not included in A_{mem} , and E_{mem} counts emitted residual slots, including constant-zero slots from padding.

- (ii) *For a fixed finite bounded-local heap program, an initial heap of S objects, and a bound of T macrosteps, accepting runs modulo fresh-name permutations have a bijective quartic representation with $O((S+T) \log^2(S+T+2))$ variables under the implemented bitonic construction. An established $O(N \log N)$ sorting network improves the mathematical bound to $O((S+T) \log(S+T+2))$.*
- (iii) *A pointer-only two-stack substrate is Turing complete and satisfies the local hypotheses of (ii). Its source semantics requires no arithmetic on object names.*
- (iv) *Finite memory faults in this class admit Diophantine representations, but the parameterized property that a deterministic program is memory-safe forever need not admit any existential polynomial representation.*

The separate theorems below establish all four assertions. The constants in (ii) depend on the fixed program, its record width, its maximum number of accesses and births per macrostep, and the chosen local instruction certificates. The result does not assert one small constant for arbitrary source languages or arbitrary program descriptions.

What is, and is not, quotiented The quotient in (ii) removes names of dynamically born objects. It does *not* identify two different rewrite schedules, two different chosen redex occurrences, or two derivations merely because their final graphs are isomorphic. The initial objects have fixed identities, and objects born in a single macrostep occupy specified ordered output slots. These qualifications are essential to the bijection, not presentation details.

For a deterministic source program with a fixed input and a fixed padding convention, at most one run survives. For a nondeterministic source program, there can be many zeros, but exactly one for each retained computational run. This is bounded, witness-faithful compilation. It is not a fixed-arity single-fold representation of an arbitrary recursively enumerable set.

3.9 Manuscript 09: Order-Free Diophantine Self-Assembly and a Sharp Positivity-Restricted Degree Threshold

[write] *Title-page heading “Computability • Diophantine geometry • Self-assembly”; subtitle “Unique arrival ranks, assembly-faithful quadratic certificates, and the transition from semilinearity to universality”. “Research prepared for Vladimir Reshetnikov”, September 30, 2026, with the line “Mathematical proofs, a standard-library Python compiler, exact finite checks, and independently replayable arithmetic certificates”; pinned at 4e128356d. Batch 62, manuscript 02 (order_free_diophantine.zip). Printed here (title-page summary, abstract, keywords, status, Section 1) and in Part XI (the manuscript’s Parts I–III).*

Title-page summary. The manuscript’s title page carries this box:

For integer polynomials that are nonnegative on the *entire real nonnegative orthant*, existential natural-number representation has the exact expressive-power threshold

$$\mathcal{D}_2^+ = \mathcal{D}_3^+ = \text{SL}, \quad \mathcal{D}_4^+ = \text{CE}.$$

For a seeded, irreversible tile system on a finite candidate domain, there is an explicit convex quadratic whose natural zeros correspond *bijectionally to producible labelled assemblies*, not to their many possible assembly orders.

Abstract. The existence of Diophantine representations of computable physical or combinatorial systems is an immediate consequence of the Matiyasevich–Robinson–Davis–Putnam theorem once a finite execution can be recognized. That observation alone says little about the degree of an explicit certificate, its size, or the meaning and multiplicity of its witnesses. This article addresses these issues through two complementary results.

First, for every finite number of input coordinates, the sets represented by an integer polynomial of degree at most three that is nonnegative on the whole real nonnegative orthant are exactly the semilinear sets. Every semilinear set already has such a representation of degree two, and can in fact be represented with exactly one natural witness per member. Degree four, in contrast, represents every computably enumerable set. The lower bound follows from a facewise rigidity lemma: the zeros of a nonnegative cubic in an open convex region form an affine slice determined by a Hessian kernel. The natural zero set is consequently a finite union of natural solutions of rational affine systems. We give both an elementary existence proof and an effective version using real-algebraic sampling. Even the set of squares requires degree four within this positivity class.

Second, finite producibility in the seeded abstract Tile Assembly Model admits a direct, order-free, assembly-faithful compiler. Each occupied site carries its unique first-arrival round. Two local threshold tests, one at the preceding round and one a round earlier, characterize these ranks without a time-expanded tableau. Boolean labels, comparisons, and local conjunctions are enforced by affine equations with unique nonnegative slacks. Summing their squares gives a convex quadratic with one natural zero per producible labelled assembly. We give exact variable and equation counts, a whole-grid terminality extension, an antimatroid interpretation, and counterexamples explaining why negative interactions and static partial-order schedules require separate treatment.

These results fit the two existing interfaces in ProveIt: its finite-witness MRDP theorem and its Presburger decision procedure. The accompanying implementation verifies four complete small assembly families, checks rank uniqueness by exhaustive finite search, and exports literal affine systems and witnesses. The combination distinguishes bounded spatial certificates from fixed-arity universal equations and identifies a sharp obstruction to compressing the latter into nonnegative cubics.

Keywords. Diophantine representation; semilinear set; nonnegative cubic; MRDP; abstract Tile Assembly Model; canonical witness; monotone activation; Presburger arithmetic.

Status. The theorems below are proved in the article. Historical priority is not asserted, and the new results have not been checked by a proof-assistant kernel. The fixed-arity finite-fold MRDP problem and the unrestricted cubic Diophantine problem are not claimed to be solved.

3.9.1 Questions, contributions, and the repository boundary

Two questions worth separating A computational substrate may be a register machine, a term-rewriting system, a growing tile assembly, or any other effectively described transition system. For finite reachability, MRDP gives an existential polynomial after an encoding is chosen. A useful next question is not merely whether a polynomial exists, but what arithmetic structure is necessary and what its solutions actually count.

We study the following two questions.

- Q1.** Can an everywhere nonnegative cubic be a universal arithmetic certificate, or is there an exact obstruction below degree four?
- Q2.** Can a genuinely asynchronous spatial computation be represented without choosing a construction order, with exactly one witness for each final labelled object and an explicit linear-size certificate?

The first question is answered by the complete classification in Theorem 90.2. The second is answered for seeded irreversible self-assembly by Theorem 91.7, including the unique earliest-rank characterization in Theorem 91.3. Neither question is treated by invoking universality alone.

The hypotheses of Q1 are deliberately precise: nonnegativity holds at *all real nonnegative assignments to inputs and witnesses jointly*. For Q2 the finite candidate domain is compiler data, and the number of arithmetic coordinates grows with that domain. This is not a fixed-arity single-fold representation of an unbounded halting predicate.

What was inspected in ProveIt The repository was inspected at commit

4e128356d0ef75308be8ed405d89aea2ffdb8a57.

The source file `Computability/HilbertTenthProblem/Lean/Diophantine/MRDP.lean` contains the finite-polynomial theorem `Diophantine.mrdp` and its converse `mrdp_iff`. Its quantifiers choose one finite witness number and one integer polynomial before the natural input is supplied. The same source exposes `mrdp_dioph_iff` through Mathlib’s `Dioph` interface [34].

The Hilbert-tenth README and status register document the surrounding Jones-paper formalizations, finite-trace infrastructure, and the separate status of numerical universal certificates [1]. No optimized operation count in that work is used as an assumption here. In particular, this article does not claim to improve the repository’s 75-operation certificate.

The directory `Logic/PresburgerArithmetic` documents an executable Cooper quantifier-elimination development and the interface `PresburgerArithmetic.Formula.presburgerArithmetic_decidable` [35]. The present classification therefore connects two relevant repository themes: arithmetic relations that collapse to Presburger arithmetic, and relations that require full MRDP strength. These are source inspections, not a fresh build or an independent axiom audit of the repository.

Prior work and the meaning of the contribution Semilinear sets and Presburger-definable sets coincide by the classical work of Ginsburg and Spanier [42]. The geometry of zeros of copositive quadratic forms already uses support faces and positive-semidefinite principal submatrices [44]. Ahmadi and Zhang study the geometry and computation of local minima of cubics; their Lemma 4.2 and Theorem 4.3 give closely related Hessian-kernel and line-restriction phenomena [43]. Our short geometric argument is included in full; the arithmetic classification is not presented as a discovery of those antecedents.

The seeded abstract Tile Assembly Model is an established model of algorithmic self-assembly. Its computational and intrinsic universality are documented in [46, 47]. Producibility and compatible

unions also have a substantial literature. For example, Doty’s polynomial-time results in [48] concern the *hierarchical* model, which is not the seeded single-tile model compiled here. We do not conflate the two.

The contribution developed here is the combination of a precise positivity-restricted degree classification and a completely specified, order-free arithmetic compiler with exact witness semantics. All claimed mathematical implications have proofs below. Establishing publication priority for this formulation would require a broader literature and expert review; no claim of historical priority or a named published open problem’s resolution is made.

3.10 Manuscript 10: Sharp Pumping Bounds and Canonical Diophantine Certificates for Priority Arithmetic Computation

[write] Subtitle “*FRACTRAN, ordered multiset rewriting, and finitely certified nontermination*”. An AI-assisted research manuscript (its author line, recorded in Appendix K, names only the assistant), 30 September 2026; pinned at 4e128356d. Batch 62, manuscript 04 (*ProveIt_Priority_Pumping_Diophantine.zip*). Printed here (abstract, status, Section 1) and in Part XII (Sections 2–14 and the three appendices).

Abstract. An ordered arithmetic program chooses the first enabled rule. This priority condition is precisely what a naive product-of-transitions certificate loses. We study finite repetitions of a prescribed instruction word, retaining all priority tests. For rules consuming vectors bounded by H coordinatewise, we prove a sharp, word-length-independent pumping theorem: if some initial state executes $2H + 1$ copies of the word, then every finite number of copies is executable from a suitable initial state. A closed formula computes the exact repetition capacity, including the case of infinite capacity; a three-fraction family attains capacity $2H$.

We also construct an explicit polynomial of degree at most four for the endpoint relation of a fixed repeated word. The program coefficients, initial state, repetition count, and final state are free natural-number inputs. Every true instance has exactly one auxiliary solution. The number of auxiliary variables is linear in the word length and the number of priority comparisons, independent of the repetition count. The construction uses totalized comparisons and Euclidean division rather than an appeal to MRDP. Finally, we characterize infinite periodic instruction tails, prove that existence of an eventually periodic instruction trace is computably enumerable complete even for one fixed squarefree-denominator program, and give a six-fraction nonperiodic computation with a finite affine macro certificate. Complete proofs, a reproducible compiler, explicit polynomial data, finite tests, and a formalization roadmap are included.

Status and scope. This is an unrefereed research manuscript. The mathematical results stated as theorems are proved below, but have not been checked in Lean or Rocq. The accompanying Python checks are finite tests, not substitutes for those proofs. The precise capacity formula, sharp threshold, and quantitative canonical compiler are the contributions developed here. Literature-wide priority for these formulations is not established. Neither FRACTRAN universality nor Diophantine representability of its halting problem is claimed as new. This work does not establish a universal single-fold MRDP theorem or improve ProveIt’s universal arithmetic-operation record.

3.10.1 The research problem and the repository boundary

Why another Diophantine computation construction? The statement that a Turing-complete effective substrate has a Diophantine halting relation is a consequence of MRDP, not a new representation theorem by itself. A useful additional result should preserve information that this existence statement

does not quantify: priority, witness multiplicity, certificate size, endpoints, or a specific compression mechanism.

The relevant ProveIt development already supplies MRDP in both directions and a substantial formalization of Jones’s constructions [1]. Its research note on alternative arithmetic machines identifies the first-applicable-rule requirement as an obstacle to a shorter FRACTRAN history compiler [36]. The vendored Rocq/Coq library also contains Diophantine one-step, transitive-closure, stopping, and evaluation relations for FRACTRAN [37]. Thus a bare translation of FRACTRAN halting would duplicate existing work. [write] Part VII of this report prints another bounded FRACTRAN certificate with first-applicable-fraction priority, on raw integers (Theorem 54.2).

We instead ask three narrower, quantitative questions.

1. Can an arbitrarily long repetition of a prescribed word be checked without enumerating its individual steps, while preserving every priority test?
2. Is there a sharp upper bound on a word’s finite repetition capacity when the initial state is allowed to vary?
3. Can the accelerated checker be made into a small, explicit polynomial with a unique auxiliary witness, rather than merely an existential arithmetic formula?

The answers are affirmative. A secondary question—how much nontermination can be captured by finite descriptions—exposes both a useful extension and an unavoidable incompleteness boundary.

Main conclusions For an ordered program with production and consumption matrices $A, B \in \mathbb{N}^{m \times d}$, let

$$H = \max_{i,p} B_{ip}.$$

For any fixed nonempty instruction word w , the exact capacity $\text{Cap}(w)$ belongs to $\{0, 1, \dots, 2H\} \cup \{\infty\}$. Theorem 96.4 gives a closed formula, and Theorem 97.3 proves that $2H$ cannot be reduced. The statement is about existence of a seed for each finite repetition count; it is not automatically about one seed supporting an infinite run.

For $w = (j_0, \dots, j_{\ell-1})$, with one-based instruction indices, put

$$R = \sum_{r=0}^{\ell-1} j_r, \quad S = dR. \tag{3.10.1}$$

The endpoint compiler in Theorem 98.6 has

$$n_z = 29S + 3d\ell + 12d + 3R + 3\ell, \tag{3.10.2}$$

$$n_f = 32S + 3d\ell + 15d + 4R + 4\ell + 1 \tag{3.10.3}$$

auxiliary variables and quadratic residual equations, respectively. Their sum of squares is the promised quartic polynomial. These are counts for the specified unoptimized compiler, not minimality claims.

Theorem 100.1 characterizes infinite repetition by nonnegative drift and a permanent zero-drift blocker for every earlier rule. Theorem 101.1 then shows that discovering an eventually periodic instruction tail is computably enumerable complete, despite decidability of checking a supplied tail. Section 102 gives an explicit nonperiodic computation that is nevertheless certified by a finite affine family of accelerated blocks.

Provenance and related work The repository inspection is pinned to commit

4e128356d0ef75308be8ed405d89aea2ffdb8a57.

The inspected scope consists of the Hilbert-tenth-problem README, the FRACTRAN alternatives note, and the vendored FRACTRAN Diophantine file. This is not a repository-wide correctness audit or a build of its formal libraries. The distinction matters because the historical alternatives note uses a 90-operation baseline, while the newer README reports a 75-operation complete certificate. Our residual counts measure a different object.

Conway’s FRACTRAN uses the first fraction that produces an integer and supports universal arithmetic computation [30]. Larchey-Wendling and Forster use it as an intermediate model in a formal proof of DPRM [14]. Acceleration of prescribed counter-machine paths and flat schemas is well established; Barrett, Demri, and Deters discuss Presburger descriptions of such runs [49], while Decker and Pirogov study compressed path schemas for temporal verification [50]. Our elementary interval analysis belongs to that tradition. We do not claim that fixed-loop acceleration or quarticization by arithmetic circuits is new in general. The specific package here is the exact first-applicable-rule capacity calculation, its sharp bound, and a fully specified canonical arithmetic compiler with an audited count. [write] Within this report: for a fixed program, single-fold quartic acceleration of the block relation is an instance of Theorem 31.2 (Part IV), and the priority-free power of a fixed word is Proposition 22.2 (Part III). The new content of Part XII is uniformity in the program entries, the priority capacity formula and the sharp threshold $2H + 1$.

3.11 Manuscript 11: One Shared Fallback

[write] *Subtitle “Conservative Reaction Computation, Unique Quartic Certificates, and a Canonical Fuel Form of the Finite-Fold Problem”. “Research draft prepared for Vladimir Reshetnikov”, 30 September 2026, with a second author line saying that proofs and executable verification are included (it also names the AI assistant; Appendix K); pinned at 4e128356d, and names a later commit ccfb084ad. Batch 62, manuscript 05 (one_shared_fallback_research.zip). Printed here (abstract, Section 1) and in Part XIII (Sections 2–14 and both appendices). Manuscript 11 has no separate status statement; its scope statements are the last two paragraphs of its Section 1, printed below.*

Abstract. We give an explicit compiler from deterministic increment/decrement register machines to priority reaction systems in which every reaction consumes exactly two molecules and produces exactly two, and only one reaction has low priority. An eight-register universal source machine yields a fixed system with 61 species and 62 reactions. Its closed finite-reservoir instances are finite-state; universality holds after existentially quantifying the initial reservoir, or in the corresponding open-reservoir semantics. The least sufficient reservoir is exactly the peak total source-register population minus its initial value.

We construct ordinary quartic polynomial certificates whose natural solutions are in bijection with bounded, canonically padded computations. For a fixed resource cutoff, these certificates are single-fold and have explicit size and height bounds. The minimum-reservoir relation is primitive recursive and even decidable in logarithmic space on unary inputs. Nevertheless, a finite-fold Diophantine representation of this one relation is equivalent to the general finite-fold conjecture; the analogous single-fold equivalence also holds. We additionally prove a priority-erasure obstruction using a self-contained coverability argument, extend the canonicalization principle to Pareto-minimal resources, and provide a tested compiler and an expanded example polynomial. The results are normal-form constructions and reductions, not a resolution of the finite-fold conjecture or a claim of record-small universality.

3.11.1 Research question, contribution, and provenance

Why another computational substrate is not enough The MRDP theorem identifies computably enumerable relations with Diophantine relations. Consequently, proving that a new finitely described computational substrate has Diophantine halting sets is often an immediate corollary of an already established universality theorem. A useful contribution should retain something that this abstract existence argument forgets: resource usage, operational semantics, an explicit low-degree certificate, or control of arithmetic witness multiplicity.

ProveIt’s Hilbert-tenth-problem development already provides the appropriate existence machinery. Its MRDP guide distinguishes a fixed finite polynomial representation from an explicit numerical degree bound or a practical polynomial-generating algorithm. Its trace interface supplies bounded universal quantification, exact iteration, and existential finite reachability [5, 3]. These are the interfaces on which the present work builds conceptually; our reaction construction and bounded polynomial compiler do not assume any unproved repository theorem.

There is substantial existing work on register-machine arithmetization, including Jones and Matiyasevich’s exponential-Diophantine construction, and on certified reductions through FRACSTRAN [8, 14]. Priority semantics and universal multiset rewriting are likewise established subjects [55, 53]. The objective here is not to rename one of those models, but to prove a particular conjunction of restrictions and exact correspondences.

What is proved here The principal structural result is an effective compiler with all of the following properties simultaneously: two-to-two reactions, total molecule conservation, a single shared low-priority fallback, determinism on legal initial markings, and an exact resource threshold. For the universal instance, the bound is 61 species and 62 reactions. These numbers are construction bounds, not optimality claims.

The arithmetic result has two levels. At a fixed horizon we give the actual quadratic residuals whose sum of squares is a quartic polynomial, and prove uniqueness of every auxiliary variable. At a fixed resource cutoff we combine two such traces to certify the *least* sufficient resource. Finally, we prove that compressing this particular decidable minimum-resource relation into a fixed finite-fold polynomial would settle the general finite-fold conjecture. This locates a genuine unresolved issue rather than silently assuming it away.

The research questions answered affirmatively are construction questions formulated in this manuscript. We do not assert that their exact formulations were previously published open problems. The contribution is the explicit compiler, its exact quantitative and witness-preserving analysis, and the resulting normal form. Historical priority for this combination has not been established by an exhaustive literature review.

Three semantic qualifications A finite closed reservoir cannot provide an infinite supply of memory. Every closed instance below is finite-state, and its acceptance is decidable. The undecidable question is whether *some* finite initial reservoir suffices. The open-reservoir interpretation is the genuinely Turing-complete single-machine semantics. These two statements must not be conflated.

The word *fuel* means reusable storage reserve: a decrement returns a molecule to the reservoir. It is not a monotonically expended energy budget. The exact formula therefore measures peak register population, not running time.

Finally, priority is an ideal execution rule, not an assertion about physical reaction rates. Section 114.3 proves that merely assigning a large but finite rate to the high-priority reaction does not implement the exact zero test used here.

Source and verification scope The repository review was targeted to the Hilbert-tenth-problem README, the MRDP guide, and the trace implementation. The trace source was read at revision 4e128356d0ef75308be8ed405d89aea2ffdb8a57. [write] [The manuscript adds that a later read of the live branch returned ccfb084adaa2f32e8d2738a25f82a00377fb3a8c; the files it cites are identical at the two commits.] No assertion is made that every newly incoming report was reviewed. The paper cites the inspected interfaces, not a purported exhaustive novelty audit of the repository.

All new mathematical statements below are proved in ordinary mathematics. The supplied Python tests were executed, but the new construction has not been Lean-formalized or independently refereed. The inherited universality of the source machine is a literature dependency, not something established by finite simulation tests.

3.12 Manuscript 12: Diophantine Certificates Without Execution Histories

[write] *Manuscript 12 was delivered in batch 63 of ProveIt’s incoming reports, after Parts I–XIII had been written; it is printed in Part XIV, and this subsection gives its title page, abstract and introduction. Title-page heading “Computability / Diophantine representation”; subtitle “Single-fold quartics for succinct routing, and a universal one-router boundary”. “Research article prepared for the ProveIt program”, September 30, 2026. Title-page note: “Repository snapshot: e18718e837d43e162252f9a314e8cb797fbd1a1f. All Diophantine unknowns in this article range over $\mathbb{N} = \{0, 1, 2, \dots\}$; polynomial evaluation is over $\mathbb{Z}$.” Batch 63, manuscript 02 (Diophantine_Certificates_Without_Histories.zip). Printed here (abstract, Section 1) and in Part XIV (Sections 2–11 and both appendices). Manuscript 12 has no separate status statement; the last paragraph of its abstract and its contribution and limitation ledger, printed below, state its scope.*

Abstract. A generic application of the Matiyasevich–Robinson–Davis–Putnam theorem represents a computably enumerable execution property by a polynomial, but does not expose its operational meaning or control auxiliary multiplicity. We develop a different route for finite particle-routing networks: replace the entire execution history by prefix counts and a canonical last-exit forest.

For a fixed routing topology with n active vertices, s absorbing sinks, and m run-length blocks, we construct one integer polynomial of total degree at most four with exactly $4n + 4m + s$ natural witness coordinates. Initial chip counts and all run lengths are parameters. Its natural zero set above each parameter tuple is empty for nontermination and a singleton for termination; the unique root includes the exact firing counts and every sink output. The construction uses $6n + 2m + s$ residual equations of degree at most two. A second compiler handles periods represented by shared concatenation grammars, with size linear in the grammar at the level of variables and residual equations.

The representation of the routing instructions matters. Circuit-defined periods already encode Boolean counting in a one-vertex acyclic network. More sharply, an explicitly constructed one-router system has a uniformly polynomial-time prefix-rank function and an eventually constant stack on every input, yet has computably enumerable complete halting behavior. Its single possible sink event occurs at firing 2^T , where T is a universal machine’s first halting time. We prove that giving this one monotone, polynomial-time rank function a single-fold, respectively finite-fold, Diophantine graph is equivalent to the corresponding representation principle for all computably enumerable sets.

The last-exit mechanism is classical. The contributions developed here are explicit canonical algebraic compilers, their quantitative accounting, and precise representation-boundary theorems. All proofs are supplied; executable exact-arithmetic checks and a sparse polynomial compiler accompany the paper. Priority for these refinements has not been established, and the new results have not been formalized in Lean.

3.12.1 The research problem and the claims made here

Beyond a black-box application of MRDP The ProveIt repository provides a natural foundation for studying arithmetic representations of computation. Its Hilbert’s tenth problem development exposes `Diophantine.mrdp` and its converse: a computably enumerable subset of $\mathbb{N}$ is the projection of the natural zero set of a fixed integer polynomial. The repository’s interface allows input zero and natural witnesses, and does not claim that the auxiliary witnesses are unique or provide a practical polynomial-generating algorithm [1, 5].

[write] [ProveIt does formalize the single-fold *exponential* representation of every enumerable set (Remark 64.2); the missing uniqueness concerns ordinary polynomials. The files cited here are unchanged between the pin e18718e83 and this write.]

Thus “this effective substrate has Diophantine halting” is usually not the interesting conclusion. The useful questions are more structural. Can an arithmetic certificate retain the exact output and firing counts? Can it avoid one row per computation step? Can every auxiliary choice be made canonical? Can instructions be compressed without expanding their repeated actions? Finally, which of these benefits survive when the underlying model becomes Turing complete?

Here a *single-fold representation* of a relation $A(\mathbf{x})$ means a polynomial $Q(\mathbf{x}, \mathbf{w})$ such that

$$A(\mathbf{x}) \iff \exists \mathbf{w} \in \mathbb{N}^r Q(\mathbf{x}, \mathbf{w}) = 0,$$

with at most one complete tuple $\mathbf{w}$ for each $\mathbf{x}$. A *finite-fold representation* requires only finitely many witness tuples. The adjective applies to *all* auxiliary coordinates, not merely to a unique decoded computation hidden inside a highly nonunique arithmetic encoding. Parameters and witnesses must also be kept distinct: fixed polynomial arity is a stronger requirement than one polynomial for each externally chosen time bound.

Relation to previous work The conceptual starting point is established mathematics, not a claim of a new least-action principle. Friedrich and Levine characterize abelian-stack aggregation by a local balance condition and an acyclic final rotor structure, and use that characterization to compute final states without traversing all intermediate states [57]. Bond and Levine develop the general abelian network framework [58]. For the one-token switching problem ARRIVAL, Gärtner and coauthors give unique verifiable run-profile certificates and prove membership in $\text{UP} \cap \text{coUP}$ [59]. Fearnley and coauthors explicitly study switching periods encoded by binary run lengths and extend structural certificate methods to succinct switching games [60]. In particular, neither run-length compression nor the use of last exits is new here.

The paper specializes and recompiles these mechanisms in a way suited to Diophantine research. We use zero-retention routing to absorbing sinks, permit arbitrarily many input chips, make every slack and forest-height variable unique, count the exact coordinates, and keep the run lengths themselves as polynomial parameters. We also give a grammar-DAG compiler and explicit counterexamples to overly broad generalizations. All arguments for this model are proved below, rather than imported by analogy from a slightly different aggregation convention.

A contribution and limitation ledger The central positive result is Theorem 128.1: the parametric quartic with $4n + 4m + s$ natural witness coordinates. Its grammar counterpart is Theorem 129.1. The resulting certificates have polynomial binary length on finite succinct instances, and a trap-collapse construction supplies canonical certificates for both outcomes of the termination question (Theorem 130.2).

The central boundary result is Theorem 132.1: one uniform, polynomial-time rank routine defines eventually constant stacks with universal halting behavior. Theorem 133.1 then identifies the exact

single-fold and finite-fold obstruction for that routine, with an explicit $2r + 3$ witness bound conditional on a local r -witness representation. These are positive mathematical constructions and reductions, not claims that the unrestricted representation principles have been proved.

No claimed result here gives a polynomial-time algorithm for finding the unique routing certificate. No claimed result improves a universal variable-degree record. The polynomial families for finite compressed networks have dimensions depending on the network description. The one-router universal construction uses an unbounded counter and a fixed bounded-simulation algorithm; it is not a finite-state rotor in disguise. We distinguish these points because confusing them would turn useful structural results into false breakthrough claims.

The bibliography establishes relevant prior art but is not a complete priority search for every formulation below. The explicit refinements are presented as research results developed and proved in this article, with priority unverified. The accompanying Python checks are independent finite sanity checks, not substitutes for the proofs or a Lean kernel audit.

3.13 Manuscript 13: Exact Wiring in Diophantine Computation

[write] *Manuscripts 13–15 were delivered together in batch 78 of ProveIt’s incoming reports (arrival commit 1977e6ea6), after Parts I–XIV had been written. They are merged by subject in Part XV; these three subsections give their title pages, abstracts, status statements and introductions. Manuscript 13: title-page heading “ProveIt research report”; subtitle “Sharp loop-parity separation and a cycle-preserving compiler for interaction combinators”; “Prepared for Vladimir Reshetnikov”, 2 October 2026. The name of the AI assistant, on its title page and in its PDF metadata, is recorded in Appendix K. Pinned at 439c0a2d9c1052595f3de6a29c11511a24fb2e11. Batch 78, manuscript 01 (Exact_Wiring_Diophantine_Report.zip, inner directory Exact_Wiring_Diophantine; 28-page PDF). Printed here (abstract, package statement, Section 1), in Part XV (Sections 2–13 and Appendix A) and, for its Appendix B (source audit), in the provenance appendix.*

Abstract. A small controller does not make graph memory free. Motivated by an explicit wiring obstruction in the ProveIt repository, we study precisely how much port incidence a sound arithmetic representation must preserve, and how to preserve it without storing the entire graph at every time step.

The first main result is a sharp separation theorem. If $M \neq N$ are perfect matchings of $2n$ labelled ports and Q is a uniformly random matching, then closing M and N with Q gives different cyclic-wire parities with probability between $2/5$ and $2/3$. Both constants are optimal, and the equality cases are classified. An explicit recurrence proves this for every n . Consequently, even one-bit contextual observations force lossless wiring summaries; conversely, $O(n \log n)$ nonadaptive one-bit probes suffice for simultaneous identification of all matchings.

The second main result is a constructive bounded-horizon compiler. An interaction-combinator history with C_0 initial cells, F free ports and T sequential interactions has a natural-number Diophantine certificate with $O(C_0 + F + T)$ variables and quadratic equations, or one polynomial of degree at most four. Exact cyclic-wire counts are preserved. The construction uses constant-time port splicing and an exact, collision-free, large-integer memory argument. For a fixed legal schedule, its ancillary witnesses can be made unique. An executable sparse-polynomial generator implements the memory component, and separate finite tests check the combinatorial and rewriting lemmas.

These are proved results within the stated models, not a claim to a new smallest universal polynomial or to a general single-fold MRDP theorem. Historical priority for the sharp separation theorem has not been certified.

Package. Self-contained mathematical proofs, source code, finite-test receipts, and an expanded example quartic. No Lean build or independent peer review is claimed.

3.13.1 Research target, scope, and relation to existing work

A specific unresolved repository interface The ProveIt note *A finite wiring obstruction for interaction combinators* exhibits two four-cell nets with identical labelled underlying graph, agent counts, port-type histogram and initially active pair, but different agent-free reachability. It explicitly leaves a topology-aware arithmetic compiler open [63]. This is a useful target: it is not enough merely to invoke the fact that every computably enumerable relation is Diophantine. One wants to know what the graph memory must retain, how local rewiring is certified, and which costs grow with the computation.

We address two precise questions.

Contextual information question. Can an encoding forget part of a matching while remaining exact under every possible wiring context, even if the observable is only cyclic-wire parity?

Bounded compilation question. Can exact interaction-combinator histories, including cyclic components, be compiled with only linearly many arithmetic coordinates, rather than a graph-sized tableau at every step?

The answers are respectively *no*, with a sharp probabilistic strengthening, and *yes*, with a completely specified arithmetic construction. The first result is independent of the universality of the substrate. The second applies to all six rules, not only the terminating all- δ fragment used by the repository counterexample.

What is old, what is proved here, and what remains to be built Lafont’s interaction combinators form a three-symbol, six-rule universal interaction system [41]. Port numbering is semantic, and isolated cyclic wires belong to the formalism. Exact permutation-based gluing, execution formulas and their associativity already occur in de Falco’s framework [68]. Matching composition with a recorded loop scalar also belongs to the established Brauer-category framework [69]. None of those concepts is claimed as an invention here.

Memory verification by comparing suitable collections of access records has a substantial prior history [70, 71]. Our arithmetic variant uses a deterministically computed large integer in place of a random field fingerprint, and proves the resulting natural-number zero set exactly. The price is large witness height; this is not a cryptographic improvement.

The report’s principal mathematical contribution is the explicit signed closure recurrence and the sharp $2/5$ – $2/3$ theorem, including its equality cases. Its principal compilation contribution is the combination of exact loop-preserving local rewriting, canonical sorted-memory witnesses, bounded allocation, and a linear-description quartic family. These are new derivations supplied for the project. A literature search located the related frameworks above, but did not establish priority for these exact statements; the report therefore makes no categorical claim that they have never appeared elsewhere.

The attached code implements every interaction rule and both gluing algorithms. It also implements the complete sparse-polynomial memory component. The *entire* interaction-net polynomial family is specified and proved constructively in Sections 145.3–145.4, but a fully instrumented exporter for every controller microstep is not included. That implementation distinction does not replace, or enlarge, the mathematical theorem.

Conventions and the size model Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. Every quantified arithmetic coordinate is natural; every polynomial has integer coefficients. A certificate with R quadratic residuals $E_1, \dots, E_R$ can be replaced by

$$P = \sum_{i=1}^R E_i^2. \quad (3.13.1)$$

Over $\mathbb{N}$, $P = 0$ is equivalent to all $E_i = 0$. Thus $\deg P \leq 4$. Degree four by itself is a standard normal-form phenomenon, not the novel content of the compiler.

An *external horizon* T indexes a different finite polynomial for each T . It is not an extra existential variable inside a single fixed-arity polynomial. Linear size below means linearly many variables, residuals and bounded-fanin arithmetic instructions in this family. It does not mean linear bit complexity, nor a universal bound independent of T .

For contextual results, ports are labelled and there are no agents. For the compiler, cells have canonical allocated identifiers and an explicitly ordered interface. Exact equality of outputs refers to that labelling; an arbitrary quotient by graph isomorphism is not silently included.

3.14 Manuscript 14: Topology Is Not Free

[write] *Title-page heading “ProveIt research continuation”, 2 October 2026; subtitle “Loop-Sensitive Diophantine Certificates for Interaction Nets”, second subtitle “Exact port gluing, a ten-state boundary calculus, and linear-size quartics with canonical witnesses”; “Research prepared for Vladimir Reshetnikov’s ProveIt program”. The name of the AI assistant, on its author line and in its PDF metadata, is recorded in Appendix K. Pinned at 6914ccca6685baf53b7a35f25efc89366c76ba74; motivating note blob 764c982297342093307a0e831bc0d9ed5084c87f. Batch 78, manuscript 04 (Topology_Is_Not_Free_Interaction_Nets.zip, inner directory Interaction_Net_Diophantine; 23-page PDF). Printed here (the title-page box “Main result”, the status statement, abstract and Section 1), in Part XV (Sections 2–13 and Appendices A–C) and, for its Appendix D (source audit), in the provenance appendix. Its equations carried hand-set numbers such as “(3.3)”; here they are numbered like the other equations of Part XV, and every reference to them points to the new number.*

Main result. A finite reduction of Lafont’s interaction combinators, including its exact port wiring and every agent-free cyclic wire, has a quartic Diophantine certificate with linearly many coordinates in the input size and the external reduction horizon. Each legal scheduled execution has exactly one auxiliary witness. The local annihilation kernel is an explicit 23-equation system; the bounded-memory kernel uses exactly $17L - 4$ auxiliaries and $17L - 4$ quadratic equations for $L \geq 1$ accesses.

Status. Mathematical proofs and reproducible finite checks. The two arithmetic kernels are implemented and exported as expanded quartics. The full interaction-net assembler is specified constructively and proved correct, but is not supplied as a complete symbolic compiler. No Lean formalization, fixed-arity compression, universal-operation record, or resolution of the general finite-fold problem is claimed.

Abstract. A recent ProveIt note exhibits two four-agent interaction nets with identical coarse graph statistics but different agent-free reachability, and explicitly leaves a topology-aware arithmetic compiler open. We address that bounded, explicit-input interface rather than re-proving that a universal substrate has some Diophantine representation. The central observation is that wire suppression never traverses surviving agents: all temporary degree-two vertices belong to the bounded rule interface. For binary annihilation this interface has exactly ten partial-matching patterns. We give quadratic formulas for the number of newly closed wires, quadratic extensions for the surviving endpoint matching, and a sharp ten-state distinguishability result for this local abstraction. A deterministic integer-product certificate then checks an entire bounded random-access memory log with $17L - 4$ auxiliary coordinates and the same number of quadratic residuals. Its proof uses a carry-free integer base, not a randomized fingerprint. Composing these constructions yields an exact, loop-sensitive, witness-faithful quartic family of size $O(C_0 + f + T)$, with polynomial bit-length witnesses. A literal 36-access rewrite schedule, the soundness and uniqueness arguments, native bounded-input loading, terminal predicates, implementation boundaries, and research directions are included. Exhaustive and randomized checks

compare two independently organized reducers, including all 10,395 matchings on four labelled binary cells.

3.14.1 The research target and what is actually established

A specific repository gap The motivating file is *A finite wiring obstruction for interaction combinators* in the ProveIt research continuation [63]. It retains unusually substantial statistics—the labelled underlying graph, port-type histogram, agent counts, and initially active pair—and nevertheless finds two inputs that disagree on agent-free reachability. The note correctly separates this obstruction from a lower bound on topology-aware certificates. Its outstanding implementation interface is exact port memory, rewiring, and cyclic-component accounting.

The surrounding repository already contains general witness-faithful memory and bounded-local heap compilers [66]. Those results are a foundation, not something to claim anew. The present continuation supplies a small, exact interface for this particular graph semantics and proves its composition with a self-contained capped-memory certificate. In particular, we do not replace the graph by a count vector or treat wire topology as a cost-free oracle.

The inspected repository snapshot is 6914ccca6685baf53b7a35f25efc89366c76ba74. The obstruction note has Git blob hash 764c982297342093307a0e831bc0d9ed5084c87f. Repository findings here refer to that snapshot, not to every incoming archive or every later commit.

Result ledger The central proved statements are as follows.

1. **Exact local topology.** A four-port annihilation is determined by its partial internal matching and ordered external endpoints. A 23-residual quadratic system has exactly one natural extension for each of the ten legal internal patterns and none for any illegal pattern. Its output includes the new cycle count. The local ten-state abstraction is minimal under a precisely specified family of observations.
2. **An explicit memory bound.** For fixed positive L, A, V , validity of a zero-initialized L -access log with address cap A and value cap V has exactly one extension to a system of $17L - 4$ quadratic equations in $17L - 4$ auxiliaries. Its sum of squares is quartic. This bound includes all sorting, range, and product witnesses.
3. **A complete bounded semantic interface.** A legal scheduled interaction-combinator execution has exactly one natural zero of an effectively constructed quartic family. There are $O(C_0 + f + T)$ coordinates and equations. Bare loops, self-wires between distinct ports of an agent, parallel temporary edges, free ports, and fresh names are all covered.

The last theorem is a mathematical construction, assembled from explicitly described finite circuits and the memory system. The delivered code emits the two arithmetic kernels and implements two reducers; it does not emit the full unknown-schedule quartic. This distinction is maintained throughout.

Novelty and prior ingredients Lafont’s interaction nets and universal interaction combinators are classical [67, 41]. Precise permutation-based gluing, the execution formula, and treatment of closed components are also established mathematics; de Falco gives a particularly relevant account [68]. Our contribution is the explicit bounded Diophantine interface, its local resource ledger, the particular capped-memory equations, and their proved composition. The formulas are proved here, but independent priority for each elementary lemma is not asserted. This is a resolution of a concrete repository-level construction gap, not a claim that general arithmetization or interaction-net semantics was previously unknown.

3.15 Manuscript 15: No Ghost Wires

[write] *Title-page heading “Diophantine representations of computation”; subtitle “Loop-Exact Quartic Certificates for Interaction Combinators” and the line “A topology-aware extension of the ProveIt research program”; “Prepared for Vladimir Reshetnikov”, October 2, 2026; running head “Research draft / October 2026”. Its PDF metadata, which names the AI provider, is recorded in Appendix K. It records the repository revision 928ea97017a25ebe56d240c84f27d2275d818c75, which it calls a tree identifier; in the repository that identifier is a commit, of 2 October 2026 (Section 148.5). Batch 78, manuscript 05 (no_ghost_wires.zip, inner directory no_ghost_wires; 25-page PDF). Printed here (abstract, status statement, Section 1) and in Part XV (Sections 2–13 and Appendices A–C); its provenance note and claims ledger are shipped as 15-no-ghost-wires-PROVENANCE.md and 15-no-ghost-wires-CLAIMS.md.*

Abstract. An interaction-combinator computation rewrites a port-labelled graph. Its wires cannot be replaced by agent counts, edge counts, or port-type histograms: such summaries can identify nets with different reachability behavior. Moreover, rewriting may create closed wires with no remaining ports. We give an explicit arithmetic treatment of both phenomena.

The central construction associates to the gluing of two matchings a permutation whose boundary-containing orbits encode surviving wires and whose other orbits encode closed components in pairs. A canonical minimum-and-distance certificate for a permutation has exactly one natural witness: it uses $4n$ variables with fixed successor indices, or $5n$ variables when the successor is selected by an input permutation matrix. Its equations are quadratic. We combine this certificate with literal templates for all six of Lafont’s interaction-combinator rules, obtaining a sum-of-squares polynomial of total degree at most four for each finite lifetime schedule. For fixed endpoint nets, that polynomial has exactly one natural zero if the scheduled computation is valid, and none otherwise.

We also prove a division-free matching-elimination identity, an order-independent closed-loop accounting law, a sixteen-port locality bound, and a contextual completeness result for boundary matchings together with loop counts. A complete Python implementation exports the polynomial and its witness. Exact tests cover 46,234 permutations, 81,393 gluing instances, 412 contexts spanning all six rules, and all 10,395 closed four- δ nets. The repository’s distinguishing example has a certificate with 126 witnesses and 169 quadratic residuals.

Status. Theorems are proved in this manuscript and supported by exact finite checks. The work supplies the missing topology-aware compiler at finite horizon. It does not supply a fixed-arity universal equation, establish historical priority for the specific certificate, resolve finite-fold MRDP, or claim a new Lean verification.

3.15.1 The research question and the precise advance

A concrete gap in the repository The ProveIt Hilbert’s-tenth-problem project contains an extensive arithmetic representation program, including an MRDP interface and research on alternative computational substrates. The directly relevant starting point is its note *A finite wiring obstruction for interaction combinators* [63]. That note constructs two four-agent nets with the same labelled underlying graph, the same unordered port-type histogram, the same agent types, and the same initially active pair. One reaches an agent-free net with two cyclic wires; the other reaches a normal form containing two agents. Both terminate. The distinction is reachability of the specified target, not termination versus nontermination.

The note explicitly leaves a topology-aware arithmetic compiler to be supplied. Its warning is justified: any representation whose entire dependence on the input factors through the displayed summary must

give the same answer for the two nets. Changing the polynomial degree, increasing its number of witnesses, or appealing to universality cannot repair information that has already been discarded.

We retain the exact port matching, implement every rule, and make cyclic wires an explicit scalar component of the state. The advance is a concrete, proof-backed finite-horizon compiler and a reusable arithmetic gluing theorem, not a new assertion that this already-universal substrate is computable.

What is inherited and what is constructed here Interaction nets and the three-symbol, six-rule universal system are due to Lafont [67, 41]. Matching composition with closed components also belongs to the established diagrammatic setting of Brauer categories [69]. Those facts are background, not novelty claims. In particular, we do not rebrand ordinary matching concatenation as a newly discovered mathematical operation.

The constructions developed here are the explicit canonical orbit certificate, its low-degree selector form, the loop-exact integration with all six literal rules, and the complete witness and residual ledger. The contextual and locality lemmas explain exactly what can and cannot be compressed. Historical priority for these particular arithmetic formulations has not been established.

Result	Scope and significance
Canonical orbit certificate	A unique natural witness for every finite permutation; no guessed component roots or unconstrained root slacks.
Loop-exact gluing	Recover the surviving matching and the exact number of port-free cycles, including two-edge cycles.
All-six-rule compiler	Quadratic residuals and a quartic sum of squares for an arbitrary externally specified finite lifetime schedule.
Multiplicity control	Exactly one witness per valid scheduled trace; finite disjunction preserves one witness per successful schedule.
Concrete implementation	Parametric endpoint exporter, witness generator, three gluing algorithms, and reproducible test receipts.

Conventions Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. All polynomial coefficients are in $\mathbb{Z}$; all parameters and existential witnesses range over $\mathbb{N}$. A *quartic certificate* means a polynomial of total degree at most four, usually displayed as a sum of squares of quadratic residuals. Total degree counts endpoint parameters as well as witnesses.

Agent identities and free-port labels are explicit. Auxiliary ports are ordered. Statements about exact endpoints mean equality of these labelled data unless an explicit renaming is supplied. A *lifetime schedule* specifies the two agent identities consumed at each step. Its legality with respect to agent existence and types can be checked without knowing the wiring; whether the two principal ports are actually connected is a separate arithmetic condition.

[write] *Batch 78, 2 October 2026. The three introductions describe the research note [63] as leaving a topology-aware arithmetic compiler open. That was the state at their pins, and the note is unchanged (its blob is 764c98229 at all three pins and at this write). Since then the repository's Hilbert's-tenth-problem tree has gained a quadratic relation for one externally selected $\delta\delta$ interaction and for fixed schedules of the terminating δ fragment [64] (commit b7a9404d4, after the three pins) and a review of the three archives [65], and Part XV prints the three compilers. The fixed-arity, schedule-quotient and universal-loader questions that the introductions leave open remain open.*

Part I

Exact commutation and resource algebra

Source and scope. [write] This Part prints four manuscripts' treatments of one question: when does a sequence of guarded resource updates execute, and when may two adjacent updates be exchanged at every state? Manuscript 05 gives the exact all-marking commutation criterion for Petri transitions, realizes every independence graph, and, in a subsection moved here from its section on other substrates, extends the criterion to interval-guarded translations (Theorems 4.1, 4.4 and 4.5). Manuscript 01 proves the same box criterion for its guarded translations (Proposition 5.1) and its scalar Petri classification, and then the exact resource envelope of all serializations of a multiset (Theorem 6.1). Manuscript 06 develops the resource-summary algebra of consume/produce words: principal partial translations, composition by cancellation, exact powers, and its own global commutation test (Theorem 7.7). Manuscript 07 closes the Part with its first-failure product (Theorem 8.1), used by the queue certificates of Part VI.

Duplicates. [write] The commutation criterion is proved three times. Manuscript 05's Theorems 4.1 and 4.5 are printed as the main statements, since they name the maximal relation and cover interval guards; manuscript 01's Proposition 5.1 has the same box formulas (12)–(13) as Theorem 4.5 and is kept, with its own proof, as a second route; manuscript 06's Theorem 7.7 states the criterion through least inputs, $\min(u_b, v_a) = \min(u_a, v_b)$, which is the counter form of $D_{ab} = D_{ba}$. Manuscript 01's envelope and manuscript 07's first-failure product are different theorems, proved by the same induction on formal prefix states (as is manuscript 01's Proposition 22.2 in Part III); manuscript 06 exponentiates one word exactly (Theorem 7.5), and manuscript 01 does so in Proposition 22.2.

Setting and hypotheses. [write] Each manuscript keeps its own model. Manuscript 05: a Petri net with d places and m transitions, consumption c_a and production v_a ; interval guards $l_a \leq x \leq u_a$ with translation updates in Theorem 4.5. Manuscript 01: box-guarded translations $F_a(x) = x + v_a$ on $\ell_a \leq x \leq u_a$ in $\mathbb{N}^p$, with lower and upper guards, m actions. Manuscript 06: token-consuming and token-producing transitions on p counters with q labels, and no upper guards. Manuscript 07: candidate event sequences on p resources whose formal integer histories are allowed to become negative after an illegal step.

4 An exact commutation test for Petri transitions [05]

4.1 Model and sequential demands

Let $\mathcal{N}$ have $d \geq 0$ places and a finite ordered alphabet $\mathcal{A} = \{1, \dots, m\}$ of transitions, where $m \geq 1$. Transition a has consumption and production vectors

$$c_a, v_a \in \mathbb{N}^d.$$

At a marking $x \in \mathbb{N}^d$ it is enabled exactly when $x \geq c_a$ coordinatewise, and its result is

$$a(x) = x - c_a + v_a.$$

Thus every label acts as a partial function. In a word ab , transition a is performed first.

For a vector $y \in \mathbb{Z}^d$, write y_+ for its coordinatewise positive part. Define the *sequential demand*

$$D_{ab} = c_a + (c_b - v_a)_+ = \max(c_a, c_a + c_b - v_a). \quad (7)$$

Theorem 4.1 (Exact all-marking commutation criterion). *The two-step word ab is enabled at x if and only if $x \geq D_{ab}$. Transitions a, b commute as partial functions if and only if*

$$D_{ab} = D_{ba}. \quad (8)$$

Consequently

$$I_{\max} = \{(a, b) : a \neq b, D_{ab} = D_{ba}\}$$

is the maximal symmetric irreflexive, context-independent relation whose adjacent swaps preserve enabledness and final markings for all starting markings. It can be computed using $O(dm^2)$ coordinate comparisons and arithmetic operations.

Proof. The two enabling conditions are $x \geq c_a$ and $x - c_a + v_a \geq c_b$. Their conjunction is exactly $x \geq \max(c_a, c_a + c_b - v_a) = D_{ab}$. Whenever both orders are enabled they have the same result,

$$x - c_a + v_a - c_b + v_b.$$

They are therefore the same partial function exactly when their domains agree. Two principal coordinate orthants $\{x : x \geq D\}$ and $\{x : x \geq E\}$ agree exactly when $D = E$: inclusion in one direction forces the corresponding coordinate inequality by testing the corner of the smaller orthant. This proves necessity as well as sufficiency.

A swap inside a longer word has a common prefix marking. Equality of the two partial maps preserves whether the two-step fragment exists and, if it exists, the marking at the beginning of the unchanged suffix. Hence every pair in the displayed relation is sound in all contexts. Conversely, a relation sound in every context is sound for the empty prefix and suffix at every marking, so every one of its pairs satisfies (8). Computing both demand vectors for each pair gives the operation bound. $\square$

The operation bound is not a claim of constant-cost arithmetic on unbounded integers; the bit complexity includes the arc-weight bit lengths. Only additions, comparisons, and positive parts of input-size integers are needed.

Example 4.2 (Why effect vectors are insufficient). On one place let a produce a token and let b consume a token:

$$(c_a, v_a) = (0, 1), \quad (c_b, v_b) = (1, 0).$$

The net changes $+1$ and -1 commute algebraically, but ab is enabled at marking zero and ba is not. Indeed $D_{ab} = 0$ while $D_{ba} = 1$.

Remark 4.3 (Sequential commutation is not simultaneous independence). The relation in Theorem 4.1 means equality of sequential partial maps. It is not a claim that the two transitions can fire simultaneously, nor that their token occurrences are causally independent in an unfolding. Two pure consumers sharing a place satisfy $D_{ab} = c_a + c_b = D_{ba}$, although they compete for the same resource. The quotient in this article is explicitly the sequential commutation quotient.

4.2 There is no hidden graph restriction

Theorem 4.4 (Realizing arbitrary independence graphs). *Every symmetric irreflexive relation I on m labels is exactly $I_{\max}$ for a Petri net with unit arc weights and at most $\binom{m}{2}$ places.*

Proof. For each unordered nonedge $\{a, b\}$, with $a < b$, introduce a private place p_{ab} . Transition a produces one token there, transition b consumes one token there, and all other transitions leave it untouched. At this coordinate $D_{ab} = 0$ and $D_{ba} = 1$, so the nonedge is not in $I_{\max}$.

If $\{a, b\}$ is an edge of I , no private place is assigned to this pair. At every existing place, at most one of these two transitions acts. A transition and an identity action have equal demands in either order. Thus all coordinates satisfy $D_{ab} = D_{ba}$, and the pair belongs to $I_{\max}$. The arcs have weights zero or one, and there is one place per nonedge. $\square$

For a complete graph, the construction has zero places and all transitions are identity actions. An unused dummy place can be added when a convention requires a nonempty place set; this has no semantic effect. The theorem concerns commutation at all markings, not boundedness or safety from a particular initial marking.

4.3 Exact commutation with interval guards

[write] *This subsection is moved from manuscript 05's section on other substrates (Section 57, Part VIII), where it follows the treatment of counter and register programs with zero tests; its corollary for guarded programs is in Part II (Corollary 11.5). Manuscript 01's Proposition 5.1 below proves the same box formulas.*

The Petri demand formula cannot simply be reused after adding zero tests. Domains now contain equalities as well as lower bounds, and a previously commuting update may destroy a guard. The needed stronger test is still explicit for interval-guarded translations.

Theorem 4.5 (Exact commutation with interval guards). *Suppose transition a has domain*

$$D_a = \{x \in \mathbb{N}^d : l_a \leq x \leq u_a\}$$

and update $x \mapsto x + \delta_a$, where $l_a \in \mathbb{N}^d$, $u_a \in (\mathbb{N} \cup \{\infty\})^d$, $l_a \leq u_a$, and $l_a + \delta_a \in \mathbb{N}^d$. Define

$$L_{ab} = \max(l_a, l_b - \delta_a),$$

$$U_{ab} = \min(u_a, u_b - \delta_a),$$

using $\infty - t = \infty$. The domain of ab is the box $[L_{ab}, U_{ab}] \cap \mathbb{N}^d$, declared empty if any lower endpoint exceeds its upper endpoint. Two transitions commute as partial maps exactly when either both composed boxes are empty, or both are nonempty and

$$L_{ab} = L_{ba}, \quad U_{ab} = U_{ba}.$$

The maximal such relation is computable with $O(dm^2)$ coordinate operations.

Proof. The two domain conditions are $l_a \leq x \leq u_a$ and $l_b \leq x + \delta_a \leq u_b$. Intersecting their coordinate intervals gives exactly the stated box. The final update, when defined, is $x + \delta_a + \delta_b$ in either order. Empty compositions agree as partial functions. For nonempty integer boxes, equality of domains is equivalent to equality of all lower and upper endpoints: each finite extreme is attained, and an unbounded coordinate is distinguished from a bounded one by an arbitrarily large coordinate value. This proves the criterion and the direct pairwise algorithm. $\square$

Lower guards express positivity and consumption; upper guard zero expresses a zero test. Finite control can itself be a coordinate with lower and upper endpoints both equal to the source state, and update equal to target minus source. Thus the interval model includes the usual zero-testing counter programs, not just ordinary Petri nets. Reset and copy updates are not translations and are not covered by this theorem.

5 Guarded translations and exact commutation [01]

5.1 A substrate with natural-number states

Fix a dimension $p \geq 1$ and a finite nonempty action alphabet A of size m . Each action a has a lower guard $\ell_a \in \mathbb{N}^p$, a displacement $v_a \in \mathbb{Z}^p$, and an upper guard $u_a \in (\mathbb{N} \cup \{\infty\})^p$. We require, coordinatewise,

$$\ell_{aj} \geq \max(0, -v_{aj}), \quad \ell_{aj} \leq u_{aj}. \quad (9)$$

The action is the partial map

$$F_a(x) = x + v_a \quad \text{defined precisely when } \ell_a \leq x \leq u_a. \quad (10)$$

Condition (9) guarantees natural outputs. An infinite upper guard imposes no upper restriction.

For a word $w = a_1 \cdots a_t$, execution means that every successive partial map is defined. Its endpoint, when it exists, is $x + \sum_{i=1}^t v_{a_i}$. The endpoint equation alone is insufficient: intermediate guards can fail even if the final vector is nonnegative.

A weighted Petri transition with consumption $c_a \in \mathbb{N}^p$ and production $d_a \in \mathbb{N}^p$ is obtained by taking

$$\ell_a = c_a, \quad v_a = d_a - c_a, \quad u_a = \infty.$$

Upper guards also permit exact-value tests, in particular a zero test $\ell_{aj} = u_{aj} = 0$. This elementary model is sufficiently broad to expose the distinction between resource feasibility and mere net displacement.

5.2 An exact and computable independence relation

We call distinct actions independent when they commute *as partial maps*, including equality of domains:

$$F_b \circ F_a = F_a \circ F_b. \quad (11)$$

This is stronger than equality of final translations on the common domain. All translations have commuting displacements; their guards need not commute.

Proposition 5.1 (Exact box test; manuscript 01, second route to Theorem 4.5). *The domain of “first a , then b ” is the box with bounds*

$$L_{ab,j} = \max(\ell_{aj}, \ell_{bj} - v_{aj}), \quad (12)$$

$$U_{ab,j} = \min(u_{aj}, u_{bj} - v_{aj}). \quad (13)$$

It is empty if $L_{ab,j} > U_{ab,j}$ for some j . Two actions commute if and only if both composition boxes are empty, or both are nonempty and their lower and upper bounds agree.

Proof. The two enabling conditions are $\ell_a \leq x \leq u_a$ and $\ell_b \leq x + v_a \leq u_b$. Intersecting them gives the displayed box. Whenever both compositions are defined, their outputs are the same vector $x + v_a + v_b$. Thus equality of partial maps is exactly equality of their domains. Nonempty boxes in $\mathbb{N}^p$ of this form have uniquely determined finite lower endpoints and finite or infinite upper endpoints, so equality is detected coordinatewise. Empty boxes must be compared as empty sets, not by their possibly different displayed bounds. $\square$

Testing one pair requires $O(p)$ integer comparisons and additions. Testing all unordered pairs gives the largest static irreflexive independence relation in $O(pm^2)$ such operations. Any symmetric irreflexive subrelation of this relation can be used below, at the cost of retaining more distinct traces.

Remark 5.2 (Not physical simultaneous firing). Our independence concerns interchangeable *sequential* schedules. It is not the Petri-net step convention that reserves all input tokens simultaneously. For example, two named transitions each consuming one token and producing none commute as partial maps: either order is defined exactly at markings at least two. They are both individually enabled at marking one, but their two-event word is not executable there. The resource theorem below retains this distinction.

5.3 A closed classification for one Petri coordinate

The box test has a useful explicit specialization. Let (c, d) and (e, f) be the consumption–production pairs of two actions on one coordinate, and put $v = d - c$, $w = f - e$.

Proposition 5.3 (Scalar Petri commutation; the one-coordinate case of Theorem 4.1). *The two lower-guarded translations commute precisely in the cases in Table 2. For vector-valued Petri transitions the criterion must hold in every coordinate.*

Displacement signs	Necessary and sufficient condition
$v = w = 0$	Always
$v > 0, w > 0$	$c = e$
$v < 0, w < 0$	$d = f$
$v = 0, w \neq 0$	$c \leq \min(e, f)$
$w = 0, v \neq 0$	$e \leq \min(c, d)$
$vw < 0$	Never

Table 2: Commutation as equality of partial sequential maps, not concurrent token reservation.

Proof. There is no upper bound, so the criterion is

$$\max(c, e - v) = \max(e, c - w). \quad (14)$$

If $v, w > 0$ and $c > e$, the left side equals c , while both arguments on the right are less than c . The case $e > c$ is symmetric; $c = e$ works.

If $v = -r < 0$ and $w = -s < 0$, substitute $c = d + r$ and $e = f + s$. Both sides of (14), after subtracting $r + s$, become respectively $\max(d - s, f)$ and $\max(f - r, d)$. If $d > f$, the second is d and the first is strictly smaller; the reverse inequality is symmetric. Equality of d, f is sufficient.

For $v = 0$, (14) reads $\max(c, e) = \max(e, c - w)$. With $w > 0$ it holds exactly when $c \leq e$; with $w < 0$ it holds exactly when $c \leq e + w = f$. The other zero-displacement case is symmetric. If $v > 0$ and $w = -s < 0$, the right side is $\max(e, c + s)$, strictly greater than both c and $e - v$, so equality fails. Finally, two zero displacements give the identical threshold $\max(c, e)$.

For vectors, all composition domains are nonempty upward boxes. Equality of their coordinate bounds is necessary and sufficient by Proposition 5.1. $\square$

This classification recognizes shared-resource commutation that a disjoint-coordinate test would miss. A read-only transition can commute with a consuming transition when its read threshold is no greater than the resource retained after the latter fires.

6 The exact resource envelope of a multiset [01]

Write

$$v_{aj}^+ = \max(v_{aj}, 0), \quad v_{aj}^- = \max(-v_{aj}, 0).$$

For multiplicities $n = (n_a)_{a \in A} \in \mathbb{N}^m$, let

$$S_j^- = \sum_a n_a v_{aj}^-, \quad S_j^+ = \sum_a n_a v_{aj}^+, \quad A_n = \{a : n_a > 0\}. \quad (15)$$

For each action-coordinate pair define the fixed constants

$$r_{aj} = \ell_{aj} - v_{aj}^- \geq 0, \quad s_{aj} = u_{aj} + v_{aj}^+ \quad (u_{aj} < \infty). \quad (16)$$

The extrema below are coordinatewise. The maximum of an empty set of nonnegative lower margins is zero; the minimum of an empty set of upper margins is infinity.

Theorem 6.1 (Universal-serialization envelope). *For arbitrary actions satisfying (9), the following are equivalent:*

- (i) Every word with multiplicity vector n is executable from $x \in \mathbb{N}^p$.
- (ii) For every coordinate j ,

$$x_j \geq S_j^- + \max_{a \in A_n} r_{aj}, \quad (17)$$

$$x_j \leq \min_{\substack{a \in A_n \\ u_{aj} < \infty}} s_{aj} - S_j^+. \quad (18)$$

When these conditions hold, every such word ends at

$$y = x - S^- + S^+. \quad (19)$$

No pairwise commutation assumption is needed for this theorem.

Proof. Fix a coordinate j and distinguish one occurrence of an active action a . Just before that occurrence, let δ be the sum of displacements contributed by the preceding occurrences in this coordinate. Its guard is

$$x_j \geq \ell_{aj} - \delta, \quad x_j \leq u_{aj} - \delta \quad (u_{aj} < \infty).$$

Among all permutations and all choices of this distinguished occurrence, the smallest possible preceding displacement is

$$\delta_{\min} = -S_j^- + v_{aj}^-.$$

Indeed, put all other negative-displacement occurrences before it and all positive-displacement occurrences after it; zero-displacement occurrences are immaterial. No subset of the other occurrences has smaller total displacement. The strongest lower requirement due to a is consequently

$$\ell_{aj} - \delta_{\min} = S_j^- + \ell_{aj} - v_{aj}^- = S_j^- + r_{aj}.$$

Similarly the largest possible preceding displacement is $\delta_{\max} = S_j^+ - v_{aj}^+$, obtained by putting all other positive occurrences first. The strongest finite upper requirement due to a is

$$u_{aj} - \delta_{\max} = s_{aj} - S_j^+.$$

Taking the maximum or minimum over active a proves that the displayed box is the intersection of every guard constraint at every position of every ordering.

For sufficiency, fix any ordering. Each of its prefix displacements lies between the two extremal values just used for its current action. The assumed inequalities therefore imply that every formal intermediate state satisfies its next guard. Induction on the word length then makes these formal states an actual execution in $\mathbb{N}^P$; condition (9) ensures the next output is natural.

For necessity, if all orderings execute, in particular the extremal ordering for each action and coordinate executes. Thus each of its extremal guard inequalities holds. Different coordinates may use different extremal permutations: this is legitimate because the hypothesis quantifies over *all* permutations. If $n = 0$, there are no occurrences, the empty word is always executable, and both formulas give the identity domain and endpoint. Summing displacements gives (19). $\square$

Corollary 6.2 (Exact acceleration under commutation). *If the actions in A_n commute pairwise as partial maps, then some ordering of the multiset is executable from x if and only if the envelope conditions hold. Thus some ordering, every ordering, and the explicit resource box are equivalent.*

Proof. Any two multiset words are connected by adjacent transpositions. Swapping identical letters changes nothing; swapping distinct active letters replaces two equal composite partial maps. Pre- and post-composition preserve that equality. All complete words therefore denote the same partial map, with the same domain. Apply Theorem 6.1. $\square$

Corollary 6.3 (Petri resource formula). *For Petri transitions, every ordering of multiplicities n executes from x exactly when, for every j ,*

$$x_j \geq \sum_a n_a (c_{aj} - d_{aj})_+ + \max_{a \in A_n} \min(c_{aj}, d_{aj}). \quad (20)$$

Under pairwise commutation of the active transitions, the same condition characterizes existence of an ordering. The endpoint is $x + \sum_a n_a (d_a - c_a)$.

Proof. Here $v^- = (c - d)_+$ and $c - (c - d)_+ = \min(c, d)$; there are no upper guards. Substitute into Theorem 6.1 and Corollary 6.2. $\square$

The two terms in (20) have different meanings. The sum measures total irreversible loss over the block. The maximum measures the largest resource that must remain available while a transition fires. Shared read-only requirements are maximized, not summed.

Example 6.4 (Why existential and universal feasibility differ). Take $a : x \mapsto x + 1$ with lower guard zero and $b : x \mapsto x - 1$ with lower guard one. From zero, ab executes and ba does not. The envelope for one occurrence of each requires $x \geq 1$. This is the correct universal condition and an incorrect existential condition without commutation. The two partial maps fail the box test.

Example 6.5 (A repeated read versus repeated loss). For a read $x \mapsto x$ guarded by $x \geq r$, any positive number of repetitions requires only $x \geq r$. For a unit loss guarded by $x \geq 1$, n repetitions require $x \geq n$. A test that reserves r fresh tokens for each repeated read overestimates requirements; a test that only checks individual enabling underestimates repeated loss.

7 The exact resource-summary algebra [06]

7.1 Every word is a principal partial translation

For $U, V \in \mathbb{N}^p$, let $[U, V]$ denote the partial map

$$U + r \mapsto V + r \quad (r \in \mathbb{N}^p). \quad (21)$$

Its domain is $U + \mathbb{N}^p$. We compose such maps in execution order: the left factor executes first.

Lemma 7.1 (Faithful summaries). *Each word w has a unique pair $(U_w, V_w) \in \mathbb{N}^p \times \mathbb{N}^p$ such that*

$$m \xrightarrow{w} n \iff \exists r \in \mathbb{N}^p (m = U_w + r \wedge n = V_w + r). \quad (22)$$

For fixed m, n , the slack r , if it exists, is unique.

Proof. The empty word has summary $(0, 0)$ and a letter a has summary (u_a, v_a) . Closure under concatenation is proved in Lemma 7.2, so these base cases give existence by induction on length. For uniqueness, the domain of $[U, V]$ has the unique least element U , and the value at that element is V . Thus equality of the partial maps forces equality of both coordinates. Finally, $m = U_w + r$ determines r uniquely. $\square$

An independent closed description makes the enabling information explicit. Set $\Delta_i = v_{a_i} - u_{a_i} \in \mathbb{Z}^p$. For each coordinate j ,

$$(U_w)_j = \max \left(0, \max_{1 \leq i \leq L} \left((u_{a_i})_j - \sum_{h < i} (\Delta_h)_j \right) \right), \quad V_w = U_w + \sum_{i=1}^L \Delta_i. \quad (23)$$

The inner maximum is absent when $L = 0$. Indeed, the inequalities saying that all steps are enabled are exactly the inequalities whose largest right-hand side occurs in (23). Nonnegativity of V_w follows from executability at U_w . In particular, every finite word is executable from *some* sufficiently large marking, although not necessarily from the marking of interest.

7.2 Composition by cancellation

Lemma 7.2 (Resource product). *For $x = (u, v)$ and $y = (u', v')$, set $c = \min(v, u')$. Then*

$$x \star y = (u + u' - c, v + v' - c) \quad (24)$$

is the summary of executing x followed by y .

Proof. It suffices to work in one coordinate. Write $v = c + s$ and $u' = c + t$, where $s, t \geq 0$ and at least one of s, t is zero. Starting at $u + r$, the first partial map produces $v + r$. The second is enabled precisely when $c + s + r \geq c + t$. Since $st = 0$, this is equivalent to $r \geq t$: if $t = 0$ it is automatic, and if $t > 0$ then $s = 0$. Writing $r = t + r'$ therefore gives the composite action

$$u + t + r' \mapsto v' + s + r'.$$

Here $u + t = u + u' - c$ and $v' + s = v + v' - c$, which proves the formula. $\square$

Corollary 7.3. *The operation $\star$ is associative with identity $(0, 0)$. For p counters its algebra is the direct product of p copies of the one-counter partial-translation algebra.*

Proof. Composition of partial maps is associative. The summary map is faithful by Lemma 7.1, so the corresponding pairs are equal. The identity and coordinatewise product assertions follow directly from the formula. $\square$

In one coordinate this is a concrete realization of the bicyclic monoid: first decrement u times, then increment v times. An increment followed by a decrement cancels on all inputs, whereas a decrement followed by an increment remains undefined at zero. We use the explicit pairs rather than an abstract presentation because they display exactly which natural-number witnesses a compiler needs. The broader connection between bicyclic storage, partially blind counters, and graph monoids is classical [23].

Lemma 7.4 (Unique minimum gadget). *For fixed $x, y \in \mathbb{N}$, the equations*

$$c + s = x, \quad c + t = y, \quad st = 0 \quad (25)$$

have the unique solution $c = \min(x, y)$, $s = x - c$, $t = y - c$ in $\mathbb{N}^3$.

Proof. The displayed values satisfy the equations. Conversely, $c \leq x, y$. Since $st = 0$, either $s = 0$ and $c = x \leq y$, or $t = 0$ and $c = y \leq x$. These cases agree when $x = y$ and determine all three variables. $\square$

For composition we apply (25) to $x = v$, $y = u'$ and set the new coordinates to $u + t$ and $v' + s$. Every defining residual has degree at most two, and every auxiliary coordinate is forced. This is stronger than merely asserting that a minimum is Diophantine.

7.3 Exact powers, including zero repetitions

Theorem 7.5 (Power formula). *Let $x = (u, v)$, and put $c = \min(u, v)$, $\alpha = u - c$, $\beta = v - c$ coordinatewise. Then*

$$x^{\star k} = \begin{cases} (0, 0), & k = 0, \\ (c + k\alpha, c + k\beta), & k \geq 1. \end{cases} \quad (26)$$

In every coordinate $\alpha\beta = 0$.

Proof. In one coordinate, either $u \geq v$, so $\beta = 0$, or $v \geq u$, so $\alpha = 0$. For $k = 1$ the formula recovers (u, v) . Suppose $k \geq 1$. If $\beta = 0$, the output after k copies is c and its cancellation against the next input $c + \alpha$ is c . The composite is $(c + (k + 1)\alpha, c)$. If $\alpha = 0$, the input stays c and the output becomes $c + (k + 1)\beta$. This proves the positive-power formula by induction. The zero-power formula is the identity of the monoid. $\square$

Example 7.6 (A catalyst cannot be retained at the zeroth power). The command decrement-then-increment on one counter has summary $(1, 1)$. Every positive power still requires one token, but its zeroth power is the empty word and is enabled at zero. Substituting $k = 0$ into the positive-power line of (26) would incorrectly retain the requirement $(1, 1)$.

A two-way zero test can itself have unique polynomial witnesses. Let $e, \ell \in \mathbb{N}$ satisfy

$$e(e - 1) = 0, \quad k = e + \ell, \quad (1 - e)\ell = 0. \quad (27)$$

If $e = 0$ these equations force $k = \ell = 0$; if $e = 1$ they force $k \geq 1$ and $\ell = k - 1$. Thus $e = \mathbf{1}_{k > 0}$ uniquely. The power coordinates are then

$$U' = ec + k\alpha, \quad V' = ec + k\beta. \quad (28)$$

Together with (25), all residuals remain quadratic. Notice that the child summary and its minimum witnesses are still constrained when $k = 0$. Disabling their equations would create free witnesses and destroy single-foldness.

7.4 A sharp global commutation test

Disjoint resource footprints provide one sufficient commutation relation, but are not necessary. Exact summaries give the largest relation appropriate for exchanging adjacent labels on *all* markings.

Theorem 7.7 (Global adjacent-swap criterion; the counter form of Theorem 4.1). *For two transition labels a, b , the following conditions are equivalent:*

- (i) *The partial actions of ab and ba are equal, including their domains.*
- (ii) *Their minimum input vectors are equal.*
- (iii) *Coordinatewise,*

$$\min(u_b, v_a) = \min(u_a, v_b). \quad (29)$$

Consequently, the relation I_ consisting of the distinct pairs satisfying (29) is the largest symmetric, irreflexive label relation for which every permitted adjacent swap preserves executability and result at every starting marking.*

Proof. By (24), the minimum inputs of ab and ba are respectively

$$u_a + u_b - \min(v_a, u_b), \quad u_a + u_b - \min(v_b, u_a).$$

Equality is exactly (29). Both words have the same total increment $v_a - u_a + v_b - u_b$, so equality of minimum inputs also implies equality of minimum outputs. Faithfulness proves the equivalence. The last assertion follows by applying the equivalence to each pair separately. $\square$

Two pure consumers of the same resource satisfy the criterion, as do two pure producers. Pure increment and pure decrement of one counter do not. The criterion is about equality of the *two-step partial maps*; it is not the usual stronger assertion that two individually enabled transitions can always both execute. Two consumers may compete for a marking that enables each separately but not their composition. This distinction is important when relating the construction to other notions of concurrency.

Fix henceforth any symmetric, irreflexive $I \subseteq I_*$. Define $\equiv_I$ as the congruence on A^* generated by $ab \equiv_I ba$ for $(a, b) \in I$. Every class is finite, since swaps preserve length and letter multiplicities. Equality of partial actions is preserved under composition, so

$$w \equiv_I w' \implies \text{res}(w) = \text{res}(w'). \quad (30)$$

The converse need not hold: distinct traces can have the same partial action. Our later counting theorem counts these trace classes, not all semantic partial maps.

8 A first-failure product for resource legality [07]

8.1 Formal histories may continue after failure

Fix p resources, an initial vector $r_0 \in \mathbb{N}^p$, and a candidate sequence of T events. Event i consumes $c_i \in \mathbb{N}^p$ and produces $a_i \in \mathbb{N}^p$. Define its *formal history* over $\mathbb{Z}^p$ by

$$r_{i+1} = r_i - c_i + a_i \quad (0 \leq i < T),$$

regardless of whether event i is enabled. The sequence is legal when

$$r_{ij} \geq c_{ij} \quad \text{for every } i < T \text{ and } j < p.$$

Let $(x)_{\underline{d}} = \prod_{h=0}^{d-1} (x - h)$, with $(x)_{\underline{0}} = 1$.

Theorem 8.1 (First-failure product). *The integer*

$$G = \prod_{i=0}^{T-1} \prod_{j=0}^{p-1} (r_{ij})_{\underline{c_{ij}}}$$

is positive for a legal event sequence and is exactly zero for an illegal one. Hence legality is equivalent to

$$\exists u \in \mathbb{N} \quad G = u + 1,$$

and the witness u is unique whenever it exists.

Proof. If the sequence is legal, every factor in a nonempty falling factorial satisfies $r_{ij} - h \geq c_{ij} - (c_{ij} - 1) = 1$. Empty products equal one, so $G \geq 1$.

Suppose instead that the sequence is illegal and choose its first disabled event i . Every earlier event is legal. Induction on the earlier events gives $r_i \in \mathbb{N}^p$: subtracting an enabled consumption and adding a nonnegative production cannot create a negative resource. There is a coordinate j with

$$0 \leq r_{ij} < c_{ij}.$$

The falling factorial for this coordinate includes the factor with $h = r_{ij}$, which is zero. Thus the entire product is zero. Formal resource values at later events can be negative, but they cannot undo this zero factor.

The final assertion follows because the positive integers are exactly the numbers $u + 1$ with $u \in \mathbb{N}$. Such a u must equal $G - 1$. $\square$

8.2 Why this is not a generic positivity trick

Replacing an arbitrary collection of strict inequalities by positivity of their product is unsound: two negative factors have positive product. The theorem avoids this problem for a structural reason. The first disabled state is still nonnegative, and the falling factorial vanishes at every forbidden consumption level $0, \dots, c_{ij} - 1$. A negative factor arising later is always preceded by a zero factor.

For a queue deleting one symbol per event, put $p = 1$ and $c_i = 1$. If event i appends $q_i \geq 0$ symbols, then

$$\ell_i = n - i + \sum_{h < i} q_h, \quad G = \prod_{i < T} \ell_i.$$

Legality is exactly $G \geq 1$. For deletion of d symbols, use $\prod_{i < T} (\ell_i)_d$ instead.

Remark 8.2 (Polynomial scope). The theorem is stated for candidate *integer* consumptions and productions. It becomes a fixed polynomial formula directly when the falling-factorial orders are fixed, as in the queue constructions below. An unrestricted variable consumption would make the displayed product length variable; that notation alone would not be an ordinary polynomial. Finite choices of consumption can be treated by separate finite-table constructions, but are not silently included in the degree claims here.

8.3 Separating content from resources

The product sees only resource availability. It does not check which symbol is read, whether an instruction is selected correctly, or whether a produced word is the prescribed one. Its role is complementary: a content equation will identify the candidate symbols, and the resource product will guarantee that these symbols are available in the correct temporal order. This separation is the mechanism that permits us to eliminate intermediate queues.

Part II

Canonical trace classes

Source and scope. [write] Four manuscripts prove that the natural zeros of an explicit polynomial are in bijection with commutation classes of bounded executions. Manuscript 05 is the base: its Sections 4–8 (normal forms with linear Boolean memory, raw runs, the quartic and the convex quadratic certificates, costs and a ballot formula, and counting hardness) are printed first, together with its corollary for interval-guarded programs, moved here from its section on other substrates (Corollary 11.5). Manuscript 04’s Sections 3–5 (normal-form monitor, Petri quartic, counting and Parikh refinements), manuscript 06’s Sections 4 and 7 (transfer signatures for trace normality and the bounded trace-class bijection) and manuscript 01’s Sections 6–7 (Cartier–Foata layers and the causal-history quartic) follow.

The shared theorem and its second routes. [write] For exact length T the bijection is Theorem 11.1 (degree four) and Theorem 11.2 (degree two, convex, with a rational polytope as real zero set), from manuscript 05. It needs only a sound independence relation $I \subseteq I_{\max}$ and extends to interval guards and zero tests (Corollary 11.5); manuscript 04’s Theorem 15.2 needs independence by *disjoint supports* (60), and its variable count $(p + r)(2T + 1)$ is the count $(2T + 1)(d + m)$ of Theorem 11.1 in manuscript 04’s letters. Manuscript 06’s Theorem 18.1 (quartic, $q(2T + 1) + p(7T - 4)$ witnesses, resource summaries on a binary concatenation tree) is a third construction. Both are kept, with their proofs, as marked second routes. Manuscript 01’s Theorem 20.1 is a different theorem: it bounds the Cartier–Foata height H of a trace rather than its length, and its box-guarded actions carry upper guards.

Other duplicates. [write] Manuscript 04’s monitor (Theorem 14.2) is the flag recurrence of Proposition 9.4, and its forbidden-factor lemma, like manuscript 06’s Lemma 17.1, is Lemma 9.2; all are kept with their proofs. The exponential automaton lower bound is proved twice with different families: Theorem 9.5 uses $2k$ labels, and Theorem 14.3 uses $2k + 1$ letters and realizes the relation by disjoint supports in a 1-safe net. $\#P$ -completeness of class counting is proved for the maximal relation with unit weights (Theorem 13.1) and for empty independence (Theorem 16.2); these are different quotients and both are kept. The Parikh and additive-cost refinement is printed in manuscript 05’s generating-function form, Corollary 12.1; manuscripts 01 (Corollary 20.2), 04 (Corollary 16.1) and 06 (the corollary after Theorem 18.1) state it for their own certificates.

Setting and hypotheses. [write] Manuscript 05: a Petri net with d places and m distinctly labelled transitions, any symmetric irreflexive $I \subseteq I_{\max}$, optional interval guards. Manuscript 04: p places, r transitions, I restricted to disjoint supports. Manuscript 06: consume/produce transitions on p counters, q labels, I a relation of globally commuting labels, no upper guards. Manuscript 01: box-guarded translations with lower and upper guards, I a valid relation of commuting pairs, a fixed height bound H .

9 Canonical schedules with linear Boolean memory [05]

Fix any symmetric irreflexive $I \subseteq I_{\max}$. On words over $\mathcal{A}$, let $\equiv_I$ be the equivalence relation generated by adjacent swaps $ab \leftrightarrow ba$ for $(a, b) \in I$. Equivalent words have equal length and equal multiplicities of all labels. Every class of words of length T is finite and has a unique lexicographically least word, denoted its normal form.

Lemma 9.1 (Execution invariance). *If $w \equiv_I w'$, then w is enabled from M if and only if w' is, and their final markings agree whenever they are enabled.*

Proof. Apply Theorem 4.1 to each generating swap and compose the resulting partial-function equalities. $\square$

9.1 The forbidden-factor test

The next characterization is a classical trace-language fact [18]; we record a proof to make its use in the compiler explicit.

Lemma 9.2 (Lexicographic forbidden factors). *A word is not lexicographically least in its I -class if and only if it has a factor*

$$bua, \quad a < b, \quad (a, b) \in I, \quad (a, c) \in I \text{ for every letter } c \text{ of } u. \quad (31)$$

Proof. Such a factor allows the final a to move left across u and b , making the word smaller. Conversely, order the occurrences of a word by the transitive closure of $i < j$ whenever their labels do not commute. Equivalent words are precisely the labelled linear extensions of this occurrence order: adjacent incomparable swaps preserve it, and moving the first occurrence of a target linear extension to the front, then inducting, connects any two extensions.

If the given extension is not lexicographically least, consider its first nongreedy choice. A later occurrence with smaller label a was already available. It is incomparable with every occurrence skipped before reaching it, since a predecessor would contradict availability and a successor cannot occur earlier in the given extension. Hence all these skipped labels commute with a . The first skipped label is some $b > a$, giving (31). $\square$

Example 9.3 (Adjacent inversion checks miss normal-form violations). Let $a < b < c$, with aIb and bIc but not aIc . The word cab has no adjacent decreasing independent pair: ca is dependent and ab is increasing. Nevertheless

$$cab \equiv_I cba \equiv_I bca,$$

and bca is lexicographically smaller. Normalization must remember more than the immediately preceding label.

9.2 A forbidden-set recurrence

For each candidate next label a , keep one flag F_a . After a prefix w , set $F_a(w) = 1$ exactly when w has a suffix bu with

$$b > a, \quad aIb, \quad \text{every letter of } u \text{ independent of } a.$$

The empty suffix u is permitted. All flags initially vanish. Appending a is forbidden precisely when $F_a = 1$.

Proposition 9.4 (Exact flag update). *On appending a label b , the flags change by*

$$F'_a = \mathbf{1}_{aIb} (\mathbf{1}_{a < b} \vee F_a). \quad (32)$$

A word is a normal form if and only if no step appends a label whose old flag is one.

Proof. If a and b are dependent, any suffix whose letters must commute with a is broken, and b itself cannot start a new forbidden suffix. If aIb and $a < b$, the one-letter suffix b starts such a suffix regardless of the old flag. If aIb and $b < a$, the new letter can only prolong a previously existing suffix, not start one. These are exactly the three cases in (32). The last assertion is Lemma 9.2 applied at the right endpoint of every forbidden factor. $\square$

For arithmetic use, at step i let $e_{i,b}$ be one-hot selectors. Write

$$C_{i,a} = \sum_{b:aIb} e_{i,b}, \quad H_{i,a} = \sum_{b>a:aIb} e_{i,b}, \quad W_{i,a} = C_{i,a} - H_{i,a}. \quad (33)$$

On a natural one-hot assignment, C, H, W are Boolean, $H \leq C$, and H and W cannot both equal one. The recurrence is

$$F_{i+1,a} = H_{i,a} + W_{i,a}F_{i,a}. \quad (34)$$

It uses only m bits of state. All indices range over $i = 0, \dots, T - 1$ unless indicated otherwise.

9.3 Why an exponentially large automaton is avoidable but real

Theorem 9.5 (Exponential state lower bound). *For every $k \geq 1$ there is an independence relation on $2k$ ordered labels whose language of lexicographic normal forms requires at least 2^k nonrejecting states in any deterministic finite automaton. Thus any deterministic finite-state implementation requires at least k bits of memory in the worst case. The m flags of Proposition 9.4 are optimal in order of growth, up to a constant factor.*

Proof. Order the alphabet as

$$a_1 < b_1 < a_2 < b_2 < \dots < a_k < b_k.$$

Let $a_i I b_j$ exactly when $j \leq i$, impose symmetry, and allow no independence between two a 's or between two b 's. For $S \subseteq \{1, \dots, k\}$, let w_S consist of the letters b_j with $j \in S$, in decreasing index order. Each w_S is normal, since all b -letters are dependent.

After this prefix, $F_{a_i} = 1$ exactly when $i \in S$. Any selected b_j with $j > i$ is dependent with a_i and occurs before all smaller indices; it cannot leave a final flag. The letter b_i , if present, sets the flag. Every later b_j has $j < i$, commutes with a_i , and is smaller than a_i , so it preserves but does not set the flag. Therefore $w_S a_i$ is normal exactly when $i \notin S$.

For distinct S, S' , choose i in their symmetric difference. The continuation a_i is accepted after exactly one of $w_S, w_{S'}$. A deterministic automaton must assign distinct states to these 2^k prefixes. They are all accepted prefixes, so these are nonrejecting states. A device with b bits has at most 2^b states, giving $b \geq k$. $\square$

By Theorem 4.4, these difficult relations occur as exact semantic commutation graphs of unit-weight Petri nets. This lower bound is not a lower bound on the number of unbounded arithmetic variables: one integer can store many bits. It specifically explains why expanding the recognizer into one variable per automaton state would be the wrong compiler architecture.

10 A parsimonious quadratic encoding of raw executions [05]

Fix a net, natural endpoints $M, N \in \mathbb{N}^d$, and an exact horizon $T \in \mathbb{N}$. Introduce the variables

$$x_{i,p} \quad (0 \leq i \leq T), \quad r_{i,p} \quad (0 \leq i < T), \quad e_{i,a} \quad (0 \leq i < T),$$

where $1 \leq p \leq d$ and $1 \leq a \leq m$. They all range over $\mathbb{N}$. The equations are

$$x_{0,p} = M_p, \quad x_{T,p} = N_p, \quad (35)$$

$$\sum_{a=1}^m e_{i,a} = 1, \quad (36)$$

$$x_{i,p} = \sum_a c_{a,p} e_{i,a} + r_{i,p}, \quad (37)$$

$$x_{i+1,p} = r_{i,p} + \sum_a v_{a,p} e_{i,a}. \quad (38)$$

Let P_{raw} be the sum of squares of the left-minus-right residuals of these equations.

Theorem 10.1 (Exact raw-run fibres). *The natural zeros of P_{raw} are in explicit bijection with the enabled length- T transition words taking M to N . The polynomial has degree at most two. The displayed, unoptimized presentation has*

$$V_{\text{raw}} = (2T + 1)d + Tm,$$

$$R_{\text{raw}} = 2(T + 1)d + T$$

variables and squared residuals, respectively.

Proof. A natural solution of (36) has exactly one coordinate equal to one and all others zero. Let a_i be its selected label. Equations (37) and the nonnegativity of r give $x_i \geq c_{a_i}$, so the step is enabled. They also force

$$r_i = x_i - c_{a_i}, \quad x_{i+1} = x_i - c_{a_i} + v_{a_i}.$$

Together with (35), this decodes a valid execution.

Conversely, a valid word fixes the selectors. Starting from $x_0 = M$, it uniquely determines each marking x_i and then each remainder r_i . Enabledness guarantees that these are natural numbers. They satisfy every equation, and there are no other variables. This proves both existence and uniqueness of extension. All residuals are affine, so their squared sum has degree at most two. Counting the three variable arrays and the equations gives the formulas. $\square$

Lemma 10.2 (Uniform coordinate bounds). *Every natural solution of (35)–(38), and every nonnegative real solution of these same equations, satisfies*

$$0 \leq x_{i,p}, r_{i,p} \leq B_p := M_p + T \max_a v_{a,p}, \quad 0 \leq e_{i,a} \leq 1. \quad (39)$$

Proof. The selector sum bounds each nonnegative selector by one. From (37), $0 \leq r_{i,p} \leq x_{i,p}$, and from (38),

$$x_{i+1,p} \leq x_{i,p} + \max_a v_{a,p}.$$

Induction from M_p proves the sharper bound with i in place of T for x_i , and hence the stated common bound. $\square$

The raw-run encoding already avoids a frequent error: gating a separate transition equation by a selector while leaving inactive-branch auxiliaries free. There is only one enabling remainder per place and time, and it belongs to the selected step.

11 Canonical traces: compact quartic and convex quadratic forms [05]

11.1 A small quartic presentation

Add the flag variables $f_{i,a} \in \mathbb{N}$ for $0 \leq i \leq T$. Besides the raw-run equations, impose

$$f_{0,a} = 0, \quad (40)$$

$$f_{i+1,a} = H_{i,a} + W_{i,a}f_{i,a}, \quad (41)$$

$$\sum_a e_{i,a}f_{i,a} = 0. \quad (42)$$

The linear forms H, W are those of (33). Let P_4 be the sum of the squares of all displayed residuals.

Theorem 11.1 (Compact canonical certificate). *Natural zeros of P_4 are in bijection with the I -commutation classes of enabled length- T words from M to N . It has degree at most four, with*

$$V_4 = (2T + 1)(d + m), \quad (43)$$

$$R_4 = 2(T + 1)d + (T + 1)m + 2T. \quad (44)$$

At every zero all selectors and flags are Boolean; no separate equations $f(f - 1) = 0$ are required.

Proof. Decode a raw run using Theorem 10.1. Since each selector row is one-hot, the recurrence (41), starting from zero, forces f_i to be exactly the Boolean flags of Proposition 9.4. In (42), every summand is nonnegative and only the selected label can contribute. Thus the selected old flag vanishes at every step, so the word is normal.

Conversely, each normal enabled word uniquely determines its raw witness and all flag rows. It satisfies the rejection equations. By Lemma 9.1, every enabled commutation class has its normal representative among the executions with the same endpoints. Uniqueness of this representative and of all its auxiliaries proves the bijection. The extra flag array contributes $(T + 1)m$ variables; its initial and recurrence equations contribute $(T + 1)m$ residuals, and rejection contributes T . Recurrence and rejection are at most quadratic, so the squared sum is at most quartic. $\square$

11.2 Eliminating products without introducing multiplicity

We next replace each recurrence and rejection test by affine constraints. At each (i, a) introduce five variables

$$u_{i,a}, \quad s_{i,a}^1, \quad s_{i,a}^2, \quad s_{i,a}^3, \quad s_{i,a}^4 \in \mathbb{N}.$$

Replace (41)–(42) by

$$f_{i,a} = u_{i,a} + s_{i,a}^1, \quad (45)$$

$$W_{i,a} = u_{i,a} + s_{i,a}^2, \quad (46)$$

$$1 + u_{i,a} = f_{i,a} + W_{i,a} + s_{i,a}^3, \quad (47)$$

$$f_{i+1,a} = H_{i,a} + u_{i,a}, \quad (48)$$

$$1 = e_{i,a} + f_{i,a} + s_{i,a}^4. \quad (49)$$

Keep the raw-run equations and (40). Let $L_1, \dots, L_{R_2}$ be all their affine residuals, including (45)–(49), and define

$$P_2(z) = \sum_{j=1}^{R_2} L_j(z)^2. \quad (50)$$

This is an explicit polynomial specified without an expensive expansion into monomials. The implementation also supports expansion for small examples.

Theorem 11.2 (Canonical trace polytope theorem). *For every N, M, N, T and symmetric irreflexive $I \subseteq I_{\max}$ as above, the polynomial (50) has all the following properties.*

- (i) *It belongs to $\mathbb{Z}[z_1, \dots, z_{V_2}]$, has degree at most two, and is convex on $\mathbb{R}^{V_2}$.*
- (ii) *Its natural zero set is in explicitly computable bijection with the I -commutation classes of enabled length- T executions from M to N .*
- (iii) *Its nonnegative real zero set is a bounded rational polytope. Its lattice points are exactly the natural zeros in (ii).*
- (iv) *The unoptimized presentation has*

$$V_2 = (2T + 1)d + (7T + 1)m, \quad (51)$$

$$R_2 = 2(T + 1)d + (5T + 1)m + T. \quad (52)$$

- (v) *At every natural zero, each selector, flag, u , and s^j is at most one, and all markings and enabling remainders satisfy (39). Thus the polynomial has finitely many natural zeros, with an explicit coordinate bound.*

Proof. We separate arithmetic soundness, uniqueness, and real geometry.

Natural soundness. The raw constraints decode an enabled run. Each $W_{i,a}$ and $H_{i,a}$ is Boolean and they are disjoint. Start with the forced Boolean row $f_0 = 0$. If f_i is Boolean, Lemma 2.3 applied to (45)–(47) forces

$$u_{i,a} = f_{i,a} W_{i,a}.$$

Then (48) gives the correct next flag. It is Boolean because $u \leq W$ and $H + W = C \leq 1$. Induction proves the Boolean flag interpretation at every layer. Equation (49) is exactly $e_{i,a} + f_{i,a} \leq 1$. It rules out selecting a forbidden label. The decoded run is consequently normal.

Existence and uniqueness. Starting from a canonical enabled word, set the raw variables and flags as before. Each u and each slack must be

$$\begin{aligned} u &= fW, & s^1 &= f - u, & s^2 &= W - u, \\ s^3 &= 1 + u - f - W, & s^4 &= 1 - e - f. \end{aligned}$$

All these values are zero or one. Nonnegativity of the last expression is precisely the no-forbidden-selection property. They satisfy every affine equation. Conversely, the equations and the preceding induction force exactly these values. Thus the extension is unique, and the class bijection follows as in Theorem 11.1.

Real geometry. Write the affine system as $Az = b$ with integer A, b . Scalarization gives

$$\{z \in \mathbb{R}_{\geq 0}^{V_2} : P_2(z) = 0\} = C := \{z \in \mathbb{R}_{\geq 0}^{V_2} : Az = b\}.$$

This is a rational polyhedron, possibly empty. The selectors lie in simplices, so $0 \leq H, W$ and $H + W = C \leq 1$. Inductively assume $0 \leq f_i \leq 1$. Equations (45)–(47) give $0 \leq u \leq f_i, W$. Hence

$$0 \leq f_{i+1} = H + u \leq H + W = C \leq 1.$$

The first two slacks are at most one. The third is $1 + u - f_i - W \leq 1$ because $u \leq f_i$ and $W \geq 0$; the fourth is $1 - e - f_i \leq 1$. All are nonnegative by the variable domains. The raw variables obey Lemma 10.2. Every coordinate of C is therefore bounded. A bounded rational polyhedron is a rational polytope, with the empty case allowed.

Finally, $P_2 = \|Az - b\|_2^2$ has Hessian $2A^\top A$, which is positive semidefinite. The counts are the raw counts, plus $(T + 1)m$ flag variables and $5Tm$ local auxiliaries; the added residuals are $m + 5Tm$. This gives (51) and (52) and completes all assertions. $\square$

Remark 11.3 (What is fixed and what is a parameter). The net, the alphabet order, the horizon, and the relation I are fixed when this polynomial is formed. The endpoints can alternatively remain free polynomial parameters; they enter only affinely, so the joint degree is still at most two. Treating arbitrary arc weights as additional variables would create products with selectors and would change this degree accounting. Treating T as a variable in one fixed polynomial is a different problem altogether.

The number of variables and equations is linear in $T(d + m)$. In a sparse residual representation, the coefficient occurrences require at most order $T(dm + |I| + d + m)$ space, up to the endpoint terms and bit lengths. This is not a claim that a fully expanded square has the same sparsity: squaring a dense affine residual introduces cross terms. Nor are the displayed constants claimed optimal; obvious deterministic or redundant arrays can be eliminated in particular instances.

11.3 The real relaxation is not automatically integral

Example 11.4 (A nonempty polytope with no lattice points). Take two places, two transitions, and

$$c_a = (2, 0), \quad c_b = (0, 2), \quad v_a = v_b = (0, 0), \quad M = (1, 1), \quad N = (0, 0), \quad T = 1.$$

The two pure consumers commute by Theorem 4.1. Neither is enabled at M , so there are no integer solutions. Nevertheless set

$$e_{0,a} = e_{0,b} = \frac{1}{2}, \quad x_0 = (1, 1), \quad x_1 = r_0 = (0, 0).$$

These satisfy the raw affine constraints. In the order $a < b$, take $f_0 = 0$, $u = 0$, and $f_{1,a} = 1/2$, $f_{1,b} = 0$. For each label set

$$s^1 = 0, \quad s^2 = W, \quad s^3 = 1 - W, \quad s^4 = 1 - e.$$

All are nonnegative and satisfy (45)–(49). Thus C is nonempty but $C \cap \mathbb{Z}^{V_2}$ is empty.

Convexity of P_2 is not an algorithm for its natural zeros. Linear programming can find the fractional point in this example and still decide nothing affirmative about a genuine execution.

11.4 A concrete scheduling example

Let a produce one token, let b be a no-op, and let c consume one token, on one place. In the order $a < b < c$, the maximal commutation relation has precisely the pairs aIb and bIc and their reverses. The word cab is enabled from marking one but not canonical, as in Example 9.3. Its normal representative is bca ; its marking trace is

$$1 \xrightarrow{b} 1 \xrightarrow{c} 0 \xrightarrow{a} 1.$$

After the first b , the flag for a is one. The subsequent c , which is dependent with a , clears it before the final a is selected. The construction records that flag history uniquely; it does not merely impose a global sorting of all labels.

For this net, horizon six and endpoints $M = N = 2$, there are 140 enabled words and 28 commutation classes. The quadratic certificate has 142 variables and 113 residuals; the quartic one has 52 variables and 47 residuals. These counts are reproduced by the included program, not inferred from a floating-point solver.

11.5 Canonical quadratic certificates for guarded programs

[write] *This corollary and the paragraph after it are moved from manuscript 05's section on other substrates (Part VIII), where they follow Theorem 4.5 (printed in Part I, Section 4.3); the bound $B_p = M_p + T \max_a v_{a,p}$ is that of (240) there.*

Corollary 11.5 (Canonical quadratic certificates for guarded programs). *Fix an interval-guarded translation system as in Theorem 4.5, numerical endpoints, and horizon T . Let g be the number of finite upper-guard entries among all transition coordinates. Its executions modulo the maximal commutation relation in Theorem 4.5 have a parsimonious convex quadratic certificate with*

$$V_2 + Tg \quad \text{variables}, \quad R_2 + Tg \quad \text{residuals},$$

where V_2, R_2 are given by (51)–(52). Its nonnegative real zero set is bounded.

Proof. Use the underlying Petri updates $c_a = l_a$ and $v_a = l_a + \delta_a$, which are nonnegative by assumption. Add, for every finite $u_{a,p}$ and each time step, a unique natural slack $g_{i,a,p}$ with

$$x_{i,p} + g_{i,a,p} = u_{a,p}e_{i,a} + B_p(1 - e_{i,a}), \quad (53)$$

where $B_p = M_p + T \max_a v_{a,p}$. At a natural one-hot selector, this enforces the selected upper guard; each inactive guard enforces only the already established bound $x_{i,p} \leq B_p$. Hence each enabled guarded word has exactly one raw extension, including all upper-guard slacks, and no other words have extensions.

Use the sound relation from Theorem 4.5 in the same flag constraints. The proof of the class bijection needs only partial-map commutation, now supplied by that theorem, so it applies without change. In the real relaxation the new right-hand side is between B_p and $u_{a,p}$, and the added slack is bounded by their maximum. The old coordinate bounds remain valid. There is one new variable and one affine residual per finite guard and time step. $\square$

The distinction between the two maximal relations matters. A pair can commute vacuously because both guarded orders have empty domains, even if the underlying unguarded Petri transitions do not commute. It is the *guarded* execution set that is then invariant. The reference compiler implements these optional upper guards and recomputes their exact commutation relation; it does not silently reuse the unguarded relation.

[write] Order-free assembly. Part XI gives a convex quadratic for monotone tile accretion whose natural zeros are in bijection with the producible labelled assemblies (Theorem 91.7); its quotient identifies the legal orders of an accretion, which are not the commutation classes of a fixed independence relation.

12 Costs, exact counts, and a closed-form example [05]

12.1 Which observations survive the quotient?

For label costs $\kappa_a \in \mathbb{N}^q$, define

$$K(w) = \sum_{i=0}^{T-1} \kappa_{a_i}.$$

This quantity is constant on commutation classes because adjacent swaps preserve the multiplicity of each label. It can be retained by adding the affine equations

$$y_j = \sum_{i,a} \kappa_{a,j} e_{i,a} \quad (1 \leq j \leq q). \quad (54)$$

The new output variables y_j are unique. Prescribing a target value of y , or imposing $y_j + s_j = B_j$ for a budget, retains unique extension. These additions do not increase the polynomial degree.

Corollary 12.1 (Generating functions are preserved). *Let $\mathcal{R}_T(M, N)/I$ be the finite set of commutation classes represented by P_2 . Then, with commuting indeterminates $t_1, \dots, t_q$,*

$$\sum_{[w] \in \mathcal{R}_T(M, N)/I} t^{K(w)} = \sum_{z \in \text{Sol}_{\mathbb{N}}(P_2)} t^{\sum_{i,a} \kappa_a e_{i,a}(z)}.$$

Here $t^v = \prod_j t_j^{v_j}$. In particular, the number of natural zeros is the number of classes, and taking one cost coordinate per label retains the entire Parikh-vector generating polynomial.

Proof. Apply the bijection of Theorem 11.2; each matched pair has the same displayed cost. Both sums are finite by the explicit bounds. $\square$

Neither state-dependent costs nor maximum intermediate resource usage are automatically preserved. For the one-place producer a and no-op b , the orders ab and ba commute, yet a cost charged to b equal to the current marking has different values. More generally, any observation depending on the prefix state must be included in the partial-map semantics before commutation is certified. A label-additive cost is a sufficient condition, not a universal description of observable equivalence.

A stochastic interpretation also requires care. Counting one representative per class is not the same as summing the probability of all schedules in that class. The latter requires a separate class-weight calculation, potentially including the number and probabilities of its linear extensions.

12.2 A ballot formula for the producer–idle–consumer net

The small example from Section 11 admits an exact family of counts. This gives an independent combinatorial check on both the semantics and the enumerator.

Theorem 12.2 (Class and schedule counts for one resource). *For the one-place net with producer a , idle transition b , and consumer c , fix $M, N, T \in \mathbb{N}$. Let*

$$\mathcal{L} = \{\ell : 0 \leq \ell \leq T, \quad r_\ell = (\ell + N - M)/2 \in \{0, 1, \dots, \ell\}\}.$$

Use $\binom{n}{k} = 0$ when k is an integer outside $0 \leq k \leq n$. Set

$$A_\ell = \binom{\ell}{r_\ell} - \binom{\ell}{r_\ell + M + 1}.$$

Then the number of maximal-commutation classes and the number of raw schedules are respectively

$$\#(\mathcal{R}_T(M, N)/I_{\max}) = \sum_{\ell \in \mathcal{L}} A_\ell, \quad \#\mathcal{R}_T(M, N) = \sum_{\ell \in \mathcal{L}} \binom{T}{\ell} A_\ell. \quad (55)$$

The class Parikh generating polynomial is

$$\sum_{\ell \in \mathcal{L}} A_\ell U^{r_\ell} V^{\ell - r_\ell} B^{T - \ell},$$

where U, V, B mark producer, consumer, and idle occurrences.

Proof. Deleting all idle letters leaves a word over a, c , and adjacent allowed swaps change neither this reduced word nor the number of idle letters. Conversely, all placements of a fixed number of idle letters around the same reduced word are equivalent, since b commutes with both other labels. Thus a class is exactly an enabled reduced word of some length ℓ , together with $T - \ell$ idle letters.

A reduced word ending at N has r_ℓ up-steps and $\ell - r_\ell$ down-steps. Of the $\binom{\ell}{r_\ell}$ unrestricted words, the ones that go below zero are counted by $\binom{\ell}{r_\ell + M + 1}$. To see this, reflect the part of the path from its starting height M through its first visit to -1 . This maps it bijectively to an unrestricted path from $-M - 2$ to N of length ℓ . Such a path has $r_\ell + M + 1$ up-steps; reflecting through its first crossing of -1 reverses the construction. Hence A_ℓ is exactly the enabled reduced-word count.

There are $\binom{T}{\ell}$ choices of positions for a fixed reduced word in a raw schedule. Idle steps cannot alter enabledness. Summing proves both counts and recording the three multiplicities proves the generating polynomial. $\square$

For $M = N = 2, T = 6$, the nonzero terms A_ℓ for $\ell = 0, 2, 4, 6$ are 1, 2, 6, 19. Thus the class count is 28, whereas the raw count is

$$1 + 15 \cdot 2 + 15 \cdot 6 + 19 = 140.$$

The formula also explains why a canonical class count cannot generally be obtained by dividing the raw count by one uniform symmetry factor: the factors $\binom{T}{\ell}$ differ among classes.

13 Counting hardness survives maximal commutation [05]

We now let the net and endpoints vary as finite input data. Arc weights and markings are encoded in binary; the exact horizon T is encoded in unary. The latter convention is essential for a polynomial-size unrolling. The complexity class $\#P$ counts accepting branches of polynomial-time nondeterministic computations; Valiant's work is a foundational source for this counting framework [22].

Theorem 13.1 (Parsimonious counting completeness). *Counting maximal-static-commutation classes of enabled length- T Petri-net executions from M to N is $\#P$ -complete under parsimonious polynomial-time reductions. Hardness holds even for nets with only unit arc weights that are 1-safe from the specified initial marking. Consequently, counting natural zeros of the quadratic certificates produced by Theorem 11.2 is $\#P$ -complete on this promised family of certificates.*

Proof. Membership. Compute $I_{\max}$ by Theorem 4.1. Guess a word of length T using a fixed-width binary encoding of each label, rejecting unused binary codes. Every word has exactly one guessing branch. Simulate it, check the endpoints, and run the flag recognizer. Exactly the canonical enabled words accept. Since T is unary, the word length is polynomial in the input size. By Lemma 10.2, intermediate marking bit lengths are polynomial as well. The arithmetic and flag checks run in polynomial time. Hence each commutation class contributes exactly one accepting branch.

Hardness construction. Start with a Boolean circuit on n input wires, with g gates of fan-in at most two and designated output wire o . Gates are topologically ordered. Write $w = n + g$. We construct a net that guesses the input, evaluates every gate deterministically, tests that the output is one, and finally erases the stored values. The erasure is important: all accepting inputs must lead to the same final marking.

For each wire j , create two value places V_j^0, V_j^1 , initially empty. Create control places $Q_0, \dots, Q_{2w+1}$ with a single token initially at Q_0 . Every transition moves the control token from one stage to the next. The data operations at the stages are as follows.

1. At input stage $j < n$, two distinct transitions write a token to V_j^0 or V_j^1 .
2. At gate stage j , include one transition for each assignment to its distinct predecessor wires. There are at most four. The transition reads the corresponding predecessor value places catalytically: a read place has both a unit input and a unit output arc. It writes one token to the correct value place of the new output wire. For repeated predecessors, consolidate duplicate reads rather than consuming two tokens from one place.

3. At stage w , the only transition requires a token at V_o^1 , retains it, and advances the control. No transition accepts an output zero.
4. At the next w stages, two possible transitions for each wire erase its zero or one token. Exactly one is enabled. At the end all value places are empty.

The common final marking has its only token at Q_{2w+1} , and the horizon is

$$T = 2w + 1.$$

Every reachable marking is 1-safe: each wire is written once, read without change, and then erased once; only one control token exists. All arcs have weight zero or one. After the input choices, exactly one gate or erasure transition is enabled at each stage. Thus satisfying input assignments are in bijection with accepting runs of this common length and common endpoint. The construction is polynomial in the circuit size.

The quotient does not merge assignments. Distinct input assignments select different labelled input transitions. Their Parikh vectors are different, and all commutations preserve Parikh vectors. Thus no two accepting runs belonging to different assignments can become equivalent, even when the relation is the maximal one computed from the constructed net. There is already only one run for each assignment. Soundness of the relation also ensures that any word equivalent to an accepting run is an accepting run. Consequently each such class has exactly one run.

For completeness, counting satisfying assignments to circuits is itself a parsimonious universal #P problem: write a polynomial-time accepting predicate as a Boolean circuit on a fixed-length certificate, using deterministic padding to make the certificate representation unique. Its satisfying input strings are exactly the accepting certificates. Applying the construction above proves hardness. Finally, Theorem 11.2 preserves the count when passing to the quadratic certificate. $\square$

This theorem pinpoints the limitation of the geometric representation. A compact convex quadratic polynomial can encode a hard *integer counting* problem because its real zero polytope need not be integral. It does not imply hardness for solving the corresponding unconstrained real least-squares problem. The theorem also does not say that every family with a large commutation relation is hard; graph and net structure can still support useful parameterized algorithms.

14 A symbolic normal-form monitor for concurrent traces [04]

14.1 Words, independence, and the forbidden-factor criterion

Let $\mathcal{A} = \{0, \dots, r-1\}$ be a totally ordered alphabet and let $I \subseteq \mathcal{A}^2$ be symmetric and irreflexive. Letters in I are called independent. Write $u \equiv_I v$ if u and v are related by finitely many swaps of adjacent independent letters. This is a length-preserving congruence; each class is finite and therefore has a unique lexicographically least word, denoted $\text{NF}_I(u)$.

Lemma 14.1 (Classical normal-form criterion). *A word is lexicographically least in its I -class if and only if it has no factor bva such that $a < b$ and a is independent of b and of every letter of v .*

Proof. Such a factor is an immediate obstruction: swap its last a successively left through v and b . The resulting equivalent word is lexicographically smaller.

For the converse, distinguish repeated occurrences by their order among occurrences of the same letter. Equivalent words have the same occurrences, and the relative order of every pair of dependent occurrences is unchanged by an allowed swap. Suppose u' is an equivalent, smaller word. Delete the common prefix, matching the distinguished occurrences there. Let b be the next letter of u , and $a < b$

the next letter of u' . The occurrence of a used by u' occurs later in u , so u has a factor bva at that position. Every occurrence in bv is still unmatched. If one were dependent on a , its order before a would persist in u' , contradicting that a is the next unmatched occurrence of u' . Thus a is independent of every letter of bv , giving the forbidden factor. $\square$

This proof uses only the invariance of dependent-occurrence order; it does not require a construction of a full automaton. In particular, checking only adjacent inversions would be inadequate. The middle word v can contain letters that prevent a local sorting rule from exposing the obstruction immediately.

14.2 Flags with a direct arithmetic update

For a prefix u , define $f_a(u) \in \{0, 1\}$ to record whether there exists an occurrence of some $b > a$ in u such that b is independent of a and every letter after that occurrence in u is independent of a . Initially all flags are zero. On appending c , the update is

$$f_a(uc) = \begin{cases} 0, & (c, a) \notin I, \\ 1, & (c, a) \in I \text{ and } c > a, \\ f_a(u), & (c, a) \in I \text{ and } c < a. \end{cases} \quad (56)$$

The equality case belongs to the first line because I is irreflexive. An appended letter c creates a forbidden factor precisely when $f_c(u) = 1$.

Encode the letter at time t by one-hot variables $x_{t,c}$. The case distinction becomes the quadratic equation

$$f_{t+1,a} = f_{t,a} \sum_{\substack{c < a \\ (c,a) \in I}} x_{t,c} + \sum_{\substack{c > a \\ (c,a) \in I}} x_{t,c}. \quad (57)$$

Normality is imposed by the single equation per time step

$$\sum_{a \in \mathcal{A}} x_{t,a} f_{t,a} = 0. \quad (58)$$

Theorem 14.2 (Quadratic normal-form monitor; the flag recurrence of Proposition 9.4). *For a fixed word of length T , equations $f_{0,a} = 0$ and (57) force a unique flag tableau. Its entries lie in $\{0, 1\}$ without separate Boolean equations. Adding (58) at every time accepts exactly the lexicographically least words of their trace classes.*

Proof. Because each row of selectors is one-hot, exactly one case of (56) applies. Induction on t proves both uniqueness and Booleanity, and proves that the computed flags have the defining suffix-obstruction meaning. Equation (58) is exactly $f_{t,c} = 0$ for the selected c . Therefore all those equations hold exactly when no forbidden factor ever appears. Apply Lemma 14.1. $\square$

The monitor uses r bits of working memory, plus a rejection state in an ordinary automaton implementation. The displayed arithmetic description uses $O(T(r + |I|))$ terms, where $|I|$ counts ordered independent pairs. The number of equations is $O(Tr)$, even for dense I ; this must not be confused with an $O(Tr)$ bound on a fully expanded description for every graph.

14.3 Why a symbolic representation matters

Theorem 14.3 (Exponential-state lower-bound family; a variant of Theorem 9.5). *For every $k \geq 1$ there is an independence relation on $2k + 1$ ordered letters such that every deterministic finite automaton recognizing its lexicographic trace normal forms has at least 2^k states. Thus, in the worst case, a deterministic streaming recognizer needs $\Omega(r)$ bits of memory. This remains true for independence relations realized by disjoint supports in an always-enabled, 1-safe Petri net.*

Proof. Order the alphabet as

$$r_1 < \dots < r_k < a_1 < \dots < a_k < b.$$

Let the independent pairs, together with their reverses, be (b, a_i) for all i and (r_i, a_j) for $i \neq j$. There are no other independent pairs.

For $S \subseteq \{1, \dots, k\}$ let p_S be b followed by the r_i with $i \in S$, in increasing order. Each p_S is a normal form. After reading b , every flag f_{a_i} is one. Reading r_i clears f_{a_i} , while each other r_j preserves it. Consequently

$$p_S a_i \text{ is a normal form} \iff i \in S. \quad (59)$$

If a deterministic automaton reached the same state after distinct prefixes p_S and $p_{S'}$, choose i in their symmetric difference. Appending a_i would have to produce the same acceptance outcome, contradicting (59). Hence all 2^k prefixes require distinct states.

For the Petri-net realization, create one place for every unordered pair of distinct dependent letters. Put one token in every such place. Each transition has a consume-one/produce-one self-loop at precisely the places corresponding to pairs containing that transition. Every transition is always enabled and every marking remains unchanged and 1-safe. Two distinct transition supports overlap exactly when their letters are dependent. Thus disjoint support realizes the prescribed I . $\square$

The lower bound concerns deterministic recognition of normal forms. It is *not* a lower bound on all Diophantine encodings, nondeterministic certificates, or the number of variables in a polynomial. Moreover, the net in the proof uses structural resource independence; transitions with the same overall effect are not automatically identified.

15 Quartic certificates for Petri-net trace classes [04]

15.1 Operational semantics and trace closure

A Petri net here has $p \geq 1$ places and $r \geq 1$ distinctly labeled transitions. Let $U_{ia}, V_{ia} \in \mathbb{N}$ be the numbers of tokens consumed and produced by transition a at place i . At marking $m \in \mathbb{N}^p$, transition a is enabled if $m_i \geq U_{ia}$ for every i ; its successor is

$$m'_i = m_i - U_{ia} + V_{ia}.$$

Its support is $\text{supp}(a) = \{i : U_{ia} + V_{ia} > 0\}$. Fix a symmetric irreflexive I satisfying

$$(a, b) \in I \implies \text{supp}(a) \cap \text{supp}(b) = \emptyset. \quad (60)$$

One may use all disjoint-support pairs, or any symmetric subset.

Lemma 15.1 (Enabled-swap property). *If a firing word contains adjacent independent transitions ab , swapping them gives another enabled firing word with the same final marking. Hence the set of firing words of length T from s to z is a union of complete I -classes.*

Proof. Suppose a fires at m and b fires after a . Since a changes no place in the support of b , all enabling inequalities for b already hold at m . After b , the enabling inequalities for a still hold. The effects add coordinatewise and have disjoint supports, so both orders reach $m - U_a + V_a - U_b + V_b$. The rest of the firing word is unchanged. Iterating this argument along any finite sequence of allowed swaps proves closure. $\square$

This quotient is by *the specified transition commutations*. It does not identify arbitrary executions with equal endpoints, equal Parikh vectors, indistinguishable token identities, or isomorphic occurrence processes. These are different semantic quotients.

15.2 The complete residual system

Fix a horizon $T \in \mathbb{N}$ and endpoint markings $s, z \in \mathbb{N}^p$. Introduce the following natural variables:

$$\begin{aligned} m_{t,i} & \quad (0 \leq t \leq T, 1 \leq i \leq p), \\ x_{t,a} & \quad (0 \leq t < T, 0 \leq a < r), \\ e_{t,i} & \quad (0 \leq t < T, 1 \leq i \leq p), \\ f_{t,a} & \quad (0 \leq t \leq T, 0 \leq a < r). \end{aligned}$$

The m variables are markings, x selects a transition, e is the enabling slack, and f is the monitor. The system consists of

$$m_{0,i} = s_i, \quad m_{T,i} = z_i, \quad (61)$$

$$x_{t,a}(x_{t,a} - 1) = 0, \quad \sum_a x_{t,a} = 1, \quad (62)$$

$$m_{t,i} = \sum_a U_{ia}x_{t,a} + e_{t,i}, \quad (63)$$

$$m_{t+1,i} + \sum_a U_{ia}x_{t,a} = m_{t,i} + \sum_a V_{ia}x_{t,a}, \quad (64)$$

$$f_{0,a} = 0, \quad (65)$$

$$f_{t+1,a} = f_{t,a} \sum_{\substack{c < a \\ (c,a) \in I}} x_{t,c} + \sum_{\substack{c > a \\ (c,a) \in I}} x_{t,c}, \quad (66)$$

$$\sum_a x_{t,a} f_{t,a} = 0. \quad (67)$$

Unspecified indices range over all values for which their variables are defined. Every residual has degree at most two. Let $P_{N,I,T,s,z}$ be the sum of their squares.

Theorem 15.2 (Witness-faithful concurrent compilation; manuscript 04, second route to Theorem 11.1 under disjoint supports). *The natural zeros of $P_{N,I,T,s,z}$ are in canonical bijection with the I -classes of enabled length- T firing words from s to z . The polynomial has degree at most four. The displayed construction uses exactly*

$$V_{\text{Petri}} = (p + r)(2T + 1), \quad (68)$$

$$E_{\text{Petri}} = 2p + r + T(2p + 2r + 2) \quad (69)$$

variables and quadratic-or-linear residuals, respectively. Every solution coordinate is at most

$$B = \max \left\{ 1, \max_i s_i + T \max_{i,a} V_{ia} \right\}. \quad (70)$$

If all markings reachable from s are 1-safe, every solution coordinate is Boolean.

Proof. From a class to a solution. By Lemma 15.1, the unique least word $w = a_0 \cdots a_{T-1}$ in the class is still an enabled run from s to z . Let m_t be its successive markings, set $x_{t,a} = 1$ exactly when $a = a_t$, and put $e_{t,i} = m_{t,i} - U_{i,a_t}$. Enabling makes every $e_{t,i}$ natural. Use the unique monitor tableau of Theorem 14.2. All equations hold, including the normality tests.

From a solution to a class. Because the domain is $\mathbb{N}$, the equations in (62) select exactly one transition a_t at each step. Equation (63) proves its enabling inequalities. Equation (64) is precisely its successor marking. The endpoint equations therefore produce a length- T run from s to z . The flag equations and Theorem 14.2 force this word to be least in its class.

Uniqueness. For a fixed word, the initial marking and update equations force all markings; the enabling equations force all slacks; and the monitor equations force all flags. The selectors are the word itself. Thus there is exactly one solution per normal word, and hence per class.

Counts. The four variable families contribute $(T+1)p$, Tr , Tp , and $(T+1)r$, giving (68). Endpoints contribute $2p$ residuals and initial flags contribute r . Each step contributes r Boolean equations, one selector sum, p enabling equations, p updates, r monitor updates, and one normality equation, giving (69).

Height. At each step, any place increases by at most $\max_{i,a} V_{ia}$. Hence every marking coordinate satisfies (70). Each enabling slack is between zero and the corresponding marking; all other coordinates are bits. If every reachable marking is 1-safe, this also makes every slack a bit. The sum-of-squares conversion proves the degree statement. $\square$

For $T = 0$, the same formulas apply: there is exactly one solution if $s = z$ and none otherwise. Some equations then refer to the same marking variables. They are still counted as displayed residuals. Counts throughout this paper are construction counts, not minimality claims.

15.3 Why token-balance equations are insufficient

An incidence equation

$$z = s + (V - U)v$$

records only a transition-count vector $v \in \mathbb{N}^r$. It does not certify enabling or an order in which transitions can fire. For example, take two places and transitions moving a token in opposite directions. From the zero marking, neither transition is enabled, but $v = (1, 1)$ satisfies the zero-to-zero incidence equation. Equations (63) rule out this spurious object. The normal-form monitor then removes a different kind of redundancy: multiple valid orders of independent transitions.

Example 15.3 (A fork and a join). Use five places and three transitions $a < b < c$:

	Consumed vector	Produced vector
a	$(1, 0, 0, 0, 0)$	$(0, 1, 0, 0, 0)$
b	$(0, 0, 1, 0, 0)$	$(0, 0, 0, 1, 0)$
c	$(0, 1, 0, 1, 0)$	$(0, 0, 0, 0, 1)$.

From $s = (1, 0, 1, 0, 0)$ to $z = (0, 0, 0, 0, 1)$ in three steps, the enabled words are abc and bac . Only a and b are independent, so there is one trace class, represented by abc . The certificate has 56 variables and 67 residuals, with a unique Boolean solution. The accompanying file `data/04-witness-faithful-petri-fork-join.json` contains the complete residuals and that solution.

16 Counting, weights, and the limits of low degree [04]

16.1 Exact counting and Parikh refinements

Write $C_{T,s,z}$ for the class set in Theorem 15.2. Then

$$\#C_{T,s,z} = \#\{\mathbf{y} \in \mathbb{N}^{V_{\text{Petri}}} : P_{N,I,T,s,z}(\mathbf{y}) = 0\}. \quad (71)$$

Both sides are finite, and the height bound supplies a finite search box. The count is at most r^T . This is a parsimonious representation: no correction factor for permutations, unused branches, or arithmetic coding choices is needed.

Corollary 16.1 (Parikh and additive-cost refinement; compare Corollary 12.1). *Adjoin variables n_a and equations $n_a = \sum_{t < T} x_{t,a}$. The extension remains witness-faithful, and n_a is the number of occurrences of transition a in the represented class. For fixed natural transition costs c_a , a further equation $C = \sum_a c_a n_a$ gives a unique cost coordinate. Thus coefficient counts in*

$$G_{T,s,z}(\mathbf{u}) = \sum_{[w] \in C_{T,s,z}} \prod_a u_a^{\text{Par}_a(w)}$$

are exact solution counts of quartic certificates with the corresponding linear constraints.

Proof. The new variables are uniquely determined by the selectors. Independent swaps preserve each transition count and hence every additive cost. Imposing prescribed values filters precisely the desired classes. The new residuals are linear, so degree at most four is retained after summing squares. $\square$

This does not imply that equal Parikh vectors determine equal trace classes. The polynomial G can have coefficients larger than one for exactly that reason.

16.2 A complexity consequence for succinct nets

The following consequence explains why the small degree should not be interpreted as an efficient generic solution method. It is a standard counting-reduction phenomenon instantiated by our explicit compiler, rather than a new general complexity classification of traces.

Theorem 16.2 (Counting hardness for the bounded-net family; empty independence, compare Theorem 13.1). *Counting the classes represented in Theorem 15.2, with the net and a unary horizon as input, is in #P. It is #P-hard even for 1-safe nets in which all transitions share a self-loop place, so that disjoint-support independence is empty. Consequently the corresponding structured family of quartic zero-counting problems is #P-complete, even on this Boolean-witness subclass.*

Proof. For membership, nondeterministically choose a length- T word using a fixed binary encoding of each letter, rejecting unused encodings. There is exactly one choice string per word. Simulate the net, check the endpoint, and run the monitor. Marking bit lengths are bounded by the input bit length plus $O(\log(T + 1))$ using (70). With T unary, these operations take polynomial time. Exactly one word from each class accepts.

For hardness, start with a Boolean circuit over fan-in-two AND and unary NOT gates. Counting its satisfying assignments is the defining standard circuit form of #P. Let it have n input bits and g gates in topological order. Build a net with one control token moving through successive stages and a dual-rail pair of places for each input and gate value. At an input stage, two transitions choose the bit and put a token on its zero or one rail. At each gate stage, transitions for its possible input valuations read the relevant rails by self-loops, put one token on the fresh correct output rail, and advance control. If an input wire occurs twice, its rail is read only once in the enabling pattern; this avoids an unintended demand for two tokens. Exactly one gate transition is enabled for each assigned input valuation.

An additional stage advances only when the designated circuit output is true. Finally, erase the $n + g$ stored bits in a fixed order, with a zero and a one transition at each stage. Precisely one erasure transition is enabled. The target is the final control token with all value rails empty. Add a common consume-one/produce-one lock place to every transition. It makes disjoint-support independence empty without changing the executions.

All stages are visited once, all value rails receive at most one token, and the net is 1-safe. Every satisfying assignment determines exactly one accepting word, including its deterministic gate evaluations and erasures. No other accepting words exist. Every such word has

$$T = (n + g) + 1 + (n + g) = 2(n + g) + 1$$

steps and the same target marking. The construction is polynomial in circuit size, and because I is empty, words are classes. Theorem 15.2 then gives the same count as natural quartic zeros, all of whose coordinates are Boolean. $\square$

The theorem concerns a succinct operational representation, not an explicitly enumerated reachability automaton. This matters when comparing it with algorithms whose input size is the number of automaton states. It also shows that exact uniqueness of the auxiliary variables removes encoding redundancy, not the intrinsic combinatorial difficulty of choosing a computation.

17 A compact signature for lexicographic trace normality [06]

Let $<$ be the fixed total order on A . A word is *normal* when it is the lexicographically least word in its I -trace class. Equal-length words are being compared, so no convention about comparing a proper prefix with a longer word affects the definition. We first recall the classical local obstruction to normality and then derive a small block signature.

17.1 The forbidden-factor criterion

Lemma 17.1 (Lexicographic obstruction). *A word w is nonnormal if and only if it has a factor bua such that $a < b$ and every letter in bu is independent of a .*

Proof. If such a factor occurs, the last a can be commuted left across all of bu . This replaces bua by abu , producing a lexicographically smaller equivalent word.

For the converse, distinguish occurrences even when their labels agree. On the occurrences of w , place an order relation from position i to position j whenever $i < j$ and their labels are not independent, and take transitive closure. This is a partial order. The words in the trace class are exactly the labelings of its linear extensions: an adjacent independent swap preserves all these precedence constraints, and any two linear extensions can be connected by swapping adjacent incomparable occurrences. The latter claim follows by moving the first occurrence of a target linear extension leftward, then applying induction to the remaining occurrences. Every occurrence passed in this move must be incomparable.

The smallest labeling of a linear extension is obtained greedily by choosing the least label among the available minimal occurrences. Equal-labeled occurrences are dependent and retain their order, so ties do not introduce a different labeled word. If w is not the smallest labeling, let its first greedy failure be at a position labeled b . An available occurrence with a smaller label a appears later in the displayed word. Availability means that none of the intervening remaining occurrences is a predecessor of that a ; in particular, each intervening label, including b , is independent of a . They form a factor bua as asserted. $\square$

The normal form and the underlying sorting interpretation are classical [19]. We include the proof because the exact obstruction, not just regularity of the normal-form language, determines the compiler.

17.2 One bit of memory per possible smaller letter

For each $a \in A$, maintain a bit F_a . After reading a prefix, the meaning of $F_a = 1$ is that the prefix contains an occurrence of some $b > a$ such that this occurrence and every subsequent letter are independent of a . Thus reading a next would complete a forbidden factor. Initially $F_a = 0$.

When the next letter is b , reject if $F_b = 1$. The memory update is

$$F'_a = d_{ab}F_a + h_{ab}(1 - F_a), \quad d_{ab} = \mathbf{1}_{(a,b) \in I}, \quad h_{ab} = \mathbf{1}_{(a,b) \in I \wedge b > a}. \quad (72)$$

If a is dependent on b , every old candidate is blocked and the new bit is zero. If $b > a$ and a is independent of b , the letter b itself is a new candidate and the bit becomes one. In the remaining independent case the bit is unchanged. These alternatives prove the memory invariant by induction. Together with Lemma 17.1, they prove that this deterministic recognizer accepts exactly the normal words.

It is convenient to continue applying the memory update even after a rejection. Acceptance and memory evolution are separate: a block may have a well-defined final memory even though some earlier position caused failure.

Definition 17.2 (Block signature). A signature is a tuple

$$S = (C, R, \alpha, \beta), \quad C \in \{0, 1\}, \quad R, \alpha, \beta \in \{0, 1\}^q, \quad \alpha_a \beta_a = 0.$$

It describes a block w when, for every incoming bit vector F ,

$$f_w(F)_a = \alpha_a F_a + \beta_a, \quad (73)$$

$$w \text{ causes no rejection from } F \iff C = 1 \text{ and } F_a R_a = 0 \text{ for all } a. \quad (74)$$

Here f_w is the memory transformation obtained by processing the whole block.

The disjointness condition means that each coordinate transformation is exactly one of the identity map, the constant-zero map, and the constant-one map. The bit C records acceptance from the all-zero incoming memory. The vector R records which incoming bits can trigger a rejection. When $C = 0$, the acceptance predicate is already false and different R vectors can describe the same predicate. We do not require a minimal semantic signature. We use the particular signature determined by the following recursive formulas; that choice is fully deterministic.

Lemma 17.3 (Empty and one-letter signatures). *The empty block has signature*

$$\mathbf{1}_S = (1, \mathbf{0}, \mathbf{1}, \mathbf{0}).$$

For the one-letter block b , a signature is

$$C = 1, \quad R_a = \mathbf{1}_{a=b}, \quad \alpha_a = \mathbf{1}_{(a,b) \in I \wedge b < a}, \quad \beta_a = \mathbf{1}_{(a,b) \in I \wedge b > a}. \quad (75)$$

Proof. The empty word does nothing and never rejects. For one letter b , rejection is precisely the incoming condition $F_b = 1$. Formula (72) is constant one in the independent $b > a$ case, identity in the independent $b < a$ case, and constant zero otherwise. Irreflexivity of I excludes a fourth case with $a = b$ independent of itself. $\square$

17.3 Composition without enumerating subsets

For bits x, y , write $x \vee y = x + y - xy$. All vector operations below are coordinatewise except for the displayed scalar product.

Theorem 17.4 (Signature product). *Suppose $S = (C, R, \alpha, \beta)$ describes x and $T = (D, S_0, \gamma, \delta)$ describes y . Then the following signature $S \odot T = (C', R', \alpha', \beta')$ describes xy :*

$$\begin{aligned} \alpha'_a &= \gamma_a \alpha_a, & \beta'_a &= \gamma_a \beta_a + \delta_a, \\ R'_a &= R_a \vee ((S_0)_a \alpha_a), & C' &= CD \prod_{a \in A} (1 - (S_0)_a \beta_a). \end{aligned} \quad (76)$$

All resulting coordinates are bits and $\alpha'_a \beta'_a = 0$.

Proof. Substitution in the final-memory formula gives

$$f_y(f_x(F))_a = \gamma_a(\alpha_a F_a + \beta_a) + \delta_a = (\gamma_a \alpha_a) F_a + (\gamma_a \beta_a + \delta_a).$$

The summands in the last constant term cannot both be one because $\gamma_a \delta_a = 0$. If $\gamma_a \alpha_a = 1$, then $\gamma_a = \alpha_a = 1$, forcing $\beta_a = \delta_a = 0$ and hence disjointness.

For acceptance, both blocks must accept. Their conditions are

$$C = D = 1, \quad F_a R_a = 0, \quad (\alpha_a F_a + \beta_a)(S_0)_a = 0 \quad (a \in A).$$

All terms in the last product are nonnegative. It vanishes precisely when $\beta_a(S_0)_a = 0$ and $F_a \alpha_a(S_0)_a = 0$. The conditions independent of F give C' , and the remaining conditions forbid an incoming one whenever R_a or $\alpha_a(S_0)_a$ is one. This is the stated R'_a . $\square$

These formulas can also be viewed as an associative product on all such signatures, not merely as a rule on signatures arising from words. To verify associativity, the memory coefficients associate by composition of affine maps. For three factors, the forbidden incoming bits are the coordinatewise union

$$R \vee (\alpha S_0) \vee (\alpha \gamma T_0),$$

independent of bracketing. The conditions on constant terms say, in either bracketing, that none of the constant-one contributions at a later rejection test is present. Distributivity of Boolean conjunction and disjunction makes the scalar expressions equal. The empty signature is a two-sided identity. Thus recursive multiplication agrees with reading letters one at a time.

There are at most $2 \cdot 6^q$ possible signatures: 2 choices for C , 2^q for R , and 3^q for the disjoint pairs (α_a, β_a) . A compiler need not list them. Each product requires only $O(q)$ Boolean gates. This is the useful distinction between a small *description of a transformation* and an explicit transition table on all 2^q incoming memory states.

18 A bijection between bounded trace classes and polynomial roots [06]

The compressed compiler fixes a schedule template and its parameters. For a stronger statement about concurrent *behaviors*, we now allow every word of a fixed length T . The witness tuple will include the chosen word itself, and normality will remove exactly the redundant commutation representatives.

18.1 One-hot word selection and deterministic memory

Fix $T \geq 1$. For each position $0 \leq i < T$ and label $a \in A$, introduce $z_{i,a} \in \mathbb{N}$ with

$$z_{i,a}(z_{i,a} - 1) = 0, \quad \sum_{a \in A} z_{i,a} = 1. \quad (77)$$

These variables select exactly one word w of length T . Its i th letter summary has coordinates

$$U_i = \sum_a u_a z_{i,a}, \quad V_i = \sum_a v_a z_{i,a}. \quad (78)$$

Use any full binary concatenation tree on these T leaves, with the resource minimum gadget at each internal node. Impose the root endpoint equations (112).

For canonicity, introduce memory variables $F_{a,i}$ for $a \in A$ and $0 \leq i \leq T$, beginning with $F_{a,0} = 0$. Define the following *linear expressions*, without additional variables:

$$D_{a,i} = \sum_{b:(a,b) \in I} z_{i,b}, \quad H_{a,i} = \sum_{b:(a,b) \in I, b > a} z_{i,b}.$$

Impose the quadratic residuals

$$F_{a,i+1} = D_{a,i}F_{a,i} + H_{a,i}(1 - F_{a,i}), \quad (79)$$

$$\sum_{a \in A} z_{i,a} F_{a,i} = 0 \quad (0 \leq i < T). \quad (80)$$

For a selected word, the recurrence forces the unique memory evolution of (72). Its values are bits by induction, so separate Boolean constraints on these variables are unnecessary. Equation (80) is exactly the rejection test for the selected letter. Even without using its one-hot interpretation, all its summands are nonnegative and therefore cannot cancel.

Let $P_T(m, n, z, \eta)$ be the sum of the squares of all residuals in this construction, where z denotes the word selectors and η all other witnesses.

Theorem 18.1 (Bounded trace-class root bijection; manuscript 06, second route to Theorem 11.2). *For fixed $m, n \in \mathbb{N}^p$ and $T \geq 1$, the natural solutions of $P_T = 0$ are in bijection with the I -trace classes of words w satisfying*

$$|w| = T, \quad m \xrightarrow{w} n.$$

The polynomial has degree at most four and exactly

$$q(2T + 1) + p(7T - 4) \quad (81)$$

witness variables in the allocation described above. All witnesses are at most $\max(1, WT, \|m\|_\infty)$.

Proof. A root determines a unique word by (77). The resource equations force its exact summaries and endpoint slack, and the memory equations force its exact recognition run. Therefore the selected word is executable with the required endpoints and is normal. Conversely, any such normal word gives one selector assignment and, by the uniqueness of the resource and memory computations, exactly one full root.

Every executable trace class has a unique normal word. Adjacent swaps in I preserve the entire partial action by (30); hence that normal word is executable from m to n , whether or not it was the original representative used to specify the class. Thus normal executable words are in bijection with precisely the stated trace classes.

For the variable count, selectors contribute qT . A full binary tree on T leaves has $2T - 1$ nodes, each with $2p$ summary coordinates, and $T - 1$ internal nodes, each with $3p$ cancellation witnesses. The endpoint contributes p slacks and memory contributes $q(T + 1)$. The total is

$$qT + 2p(2T - 1) + 3p(T - 1) + p + q(T + 1) = q(2T + 1) + p(7T - 4).$$

Every residual is quadratic or linear, proving the quartic bound. The resource height proof with subwords of length at most T , together with Boolean selectors and memory, gives the height assertion. $\square$

For $T = 0$, no word-choice or internal witnesses are needed. The polynomial $\sum_{j=1}^p (m_j - n_j)^2$ has the single empty witness tuple when $m = n$ and none otherwise, exactly matching the empty trace.

Corollary 18.2 (Prescribing the Parikh vector; compare Corollary 12.1). *Given $v \in \mathbb{N}^q$, add the linear equations*

$$v_a = \sum_{i=0}^{T-1} z_{i,a} \quad (a \in A).$$

The same construction, without additional witnesses, counts exactly the executable trace classes with that Parikh vector.

A Parikh vector is generally coarser than a trace: dependent labels retain order information. Conversely, two different traces may induce the same resource summary. The three quotients—letter counts, trace classes, and partial actions—should not be substituted for one another in a counting claim.

18.2 Why the length parameter is not free here

The polynomials P_T form a family. Their number of variables grows with T . There is no assertion that one fixed polynomial receives T as a free input and has exactly one root per trace for all T . Passing from the family to a single unbounded trace representation would require an additional coding construction. Ordinary Diophantineness of an encoded trace does not guarantee that the additional coding witnesses are unique or finite in number.

For each fixed T the root set is finite and has at most q^T members. The height bound gives a finite search box, although enumerating that entire box is not proposed as an efficient algorithm. The value of the theorem is its exact combinatorial meaning, not a speedup over direct word enumeration. The compressed compiler addresses a different efficiency question: avoiding expansion of one highly structured schedule.

19 Causal traces and canonical layers [01]

19.1 The quotient to be represented

Fix a symmetric irreflexive relation $I \subseteq A \times A$ consisting of commuting action pairs. Put $D = (A \times A) \setminus I$; in particular $(a, a) \in D$. Two words are trace-equivalent when one can be obtained from the other by swapping adjacent I -independent letters. Write $\text{Tr}(A, I)$ for this quotient monoid.

By equality of partial maps, equivalent words have the same execution domain and endpoint. Accordingly, a trace is executable from x when any, equivalently every, representative is executable. This equivalence is why we required equality of domains in the independence test.

A trace is not the same thing as an endpoint. Distinct labels with identical effects remain distinct labels, and dependent orders remain distinct traces even when their particular endpoints happen to coincide. The quotient removes precisely the swaps specified by I , no more.

19.2 A self-contained normal-form lemma

The underlying normal form is classical Cartier–Foata theory [16, 17]. We prove the exact version needed by the polynomial compiler to make the padding and predecessor conditions explicit.

For a word with individually distinguished occurrences, generate a partial order by putting occurrence i below occurrence j whenever $i < j$ in the word and their labels are D -dependent. Take the transitive closure. The resulting labelled dependence poset has an intrinsic order on repeated occurrences of each letter because self-dependence is included.

Lemma 19.1 (Posets and swaps). *Two words represent the same trace exactly when their labelled dependence posets agree, with the k th occurrence of each label matched to the k th occurrence. The representatives of a trace are exactly the label words of the linear extensions of its dependence poset.*

Proof. Swapping adjacent independent occurrences changes the order of no dependent pair and so preserves all generating comparisons and their transitive closure. Conversely, fix a dependence poset and two linear extensions. Locate the first occurrence in the second extension within the first extension. All occurrences before it in the first extension are incomparable with it: a predecessor would also have to precede it in the second extension, while a successor could not precede it in the first. Incomparable occurrences must have independent labels, since every dependent pair was made comparable. Move the selected occurrence left by adjacent allowed swaps. Remove this common first occurrence and repeat inductively. Thus any two linear extensions are connected by allowed swaps.

The original word is a linear extension. A linear extension orders every dependent pair in the prescribed direction, so reconstructs the same dependence poset. This proves both assertions. $\square$

Give each occurrence the level

$$h(e) = 1 + \max\{h(f) : f \text{ is an earlier occurrence with } (\lambda(f), \lambda(e)) \in D\}, \quad (82)$$

with the maximum of an empty set equal to zero. Let C_i be the set of labels of level i .

Theorem 19.2 (Cartier–Foata layer characterization). *Every trace has a unique sequence of nonempty layers $C_1, \dots, C_h$ satisfying:*

- (a) *each C_i is a clique for I , so distinct members commute and no label is repeated;*
- (b) *every $a \in C_{i+1}$ has a predecessor $b \in C_i$ with $(b, a) \in D$.*

Concatenating the layers, in any order within each layer, represents the trace. The number h is the length of a longest chain in the dependence poset.

Proof. The level of an occurrence is its longest-chain rank in the dependence poset. Indeed, every generating predecessor has a lower rank, and any longest chain can be refined to use generating comparisons between successive levels; the recurrence (82) computes exactly these ranks in a topological order. Thus the levels are intrinsic to the trace by Lemma 19.1.

Two dependent occurrences cannot have the same level, because the earlier is a generating predecessor of the later. Hence each layer has distinct pairwise independent labels. If an occurrence has level $i + 1$, the finite maximum in (82) is attained by a dependent earlier occurrence of level i . This is condition (b). Sorting occurrences by level respects the poset, so concatenating the layers yields a linear extension and hence the same trace.

Conversely, start from a sequence satisfying (a) and (b), and concatenate its layers in arbitrary internal orders. Induct on i . All letters of the first layer have level one, since no two of them are dependent. A letter in layer $i + 1$ is independent of its same-layer predecessors, has no predecessor from a later layer, and has a dependent predecessor in layer i . By induction the maximum predecessor level is exactly i , so its computed level is $i + 1$. Thus the proposed layers are precisely the intrinsic levels of the resulting trace. This also proves uniqueness. Finally, the maximum rank equals the longest-chain length by the rank recurrence. $\square$

19.3 Unique trailing padding

For a fixed bound H , append empty layers to a normal form until it has H layers. The local conditions above characterize these padded forms if empty layers are allowed: if $C_i = \emptyset$, condition (b) forces $C_{i+1} = \emptyset$, and hence all later layers are empty. No choice of a padding length needs to be encoded. The all-empty sequence represents the empty trace.

This small observation is important for fibre preservation. A separate “finished” flag with unconstrained transitions, or arbitrary idle letters, can create several certificates for the same finite history. Trailing empty layers with the predecessor condition do not.

20 An explicit quartic for causal histories [01]

20.1 Parameters and witnesses

Fix the action alphabet A , a valid independence relation I , and a natural causal-height bound H . Only $x, y \in \mathbb{N}^P$ are endpoint parameters. For $H \geq 1$ introduce Boolean-intended variables z_{ha} for $1 \leq h \leq H$ and $a \in A$; they indicate membership in the h th layer.

Let e be the number of unordered *distinct* dependent pairs and set

$$D(a) = \{b : (b, a) \in D\}, \quad d = \sum_a |D(a)| = m + 2e.$$

Fix an ordering $b_{a1}, \dots, b_{a,d_a}$ of each $D(a)$, where $d_a = |D(a)| \geq 1$.

The witnesses are the layer bits, internal states $M_h \in \mathbb{N}^P$ for $1 \leq h < H$, totals L_{hj}, G_{hj} , lower slacks α_{haj} , finite-upper slacks β_{haj} , and blocker products U_{hak} for $1 \leq h < H$, $1 \leq k \leq d_a$. Put $M_0 = x$, $M_H = y$, and use $U_{ha0} = 1$ as a constant, not a witness.

20.2 The complete residual catalogue

Use the following residuals, for all indices in their specified ranges:

$$z_{ha}(z_{ha} - 1), \tag{83}$$

$$z_{ha}z_{hb} \quad \text{for each unordered distinct dependent pair } \{a, b\}, \tag{84}$$

$$L_{hj} - \sum_a z_{ha}v_{aj}^-, \quad G_{hj} - \sum_a z_{ha}v_{aj}^+, \tag{85}$$

$$M_{hj} + L_{hj} - M_{h-1,j} - G_{hj}, \tag{86}$$

$$z_{ha}(M_{h-1,j} - L_{hj} - r_{aj} - \alpha_{haj}), \quad (1 - z_{ha})\alpha_{haj}, \tag{87}$$

$$z_{ha}(s_{aj} - G_{hj} - M_{h-1,j} - \beta_{haj}), \quad (1 - z_{ha})\beta_{haj} \quad (u_{aj} < \infty), \tag{88}$$

$$U_{hak} - U_{ha,k-1}(1 - z_{h,b_{ak}}) \quad (h < H), \tag{89}$$

$$z_{h+1,a}U_{ha,d_a} \quad (h < H). \tag{90}$$

Let $Q_{A,I,H}$ be the sum of their squares. All residuals have degree at most two, including the blocker recurrences; no high-degree conjunction is left unlifted.

Theorem 20.1 (Parsimonious causal-history compiler; bounded causal height, not length). *For fixed $A, I, H \geq 1$, the nonnegative roots in all witness coordinates of*

$$Q_{A,I,H}(x, y; w) = 0$$

are in an explicit bijection with the executable I -traces from x to y whose Cartier–Foata height is at most H . The polynomial has degree at most four, with

$$N_H = H(m(1 + p) + f + 2p) + (H - 1)(d + p), \tag{91}$$

$$E_H = Hm + He + 3Hp + 2H(pm + f) + (H - 1)(d + m) \tag{92}$$

witnesses and squared residuals, respectively. In particular, every such trace gives exactly one complete natural solution, including all auxiliary coordinates.

Proof. Extraction. A zero of Q makes all residuals vanish. Equation (83) makes each z_{ha} Boolean. The corresponding subsets $C_h = \{a : z_{ha} = 1\}$ are cliques by (84).

Induction on k in (89) gives

$$U_{hak} = \prod_{t=1}^k (1 - z_{h,b_{at}}) \in \{0, 1\}. \tag{93}$$

Therefore (90) says that an action present in layer $h + 1$ has at least one dependent action in layer h . Section 19 implies that these are the uniquely padded layers of a trace of height at most H .

The totals (85) are the losses and gains for the layer multiplicities z_h . The guard slacks (87)–(88) assert its envelope at M_{h-1} . Within a layer all actions are pairwise independent and hence commute, so Corollary 6.2 makes every internal ordering executable. Equation (86) sets the endpoint to M_h . Concatenating all layers yields an execution from x to y . Its trace is independent of the chosen internal order.

Construction. Take an executable trace of height at most H . Its unique padded normal form fixes z . Any representative has the same partial map, so the layer-concatenated representative is executable. Start at $M_0 = x$ and set the boundary states by the cumulative displacements. Set L, G by their aggregate formulas. Set each active slack to its exact nonnegative guard margin and every inactive slack to zero. Define U by the product (93). The resource theorem and normal-form characterization verify every residual.

Uniqueness. The trace fixes its padded normal form, hence all z . From x and z , the displacement recurrence fixes all states; the aggregate equations fix all totals. The complementary active/inactive equations fix every slack, and the blocker recurrences fix every U . Thus there is exactly one full root over that trace. Extraction and construction are inverse because they use these same formulas.

Counts. There are Hm bits, $(H - 1)p$ internal-state entries, $2Hp$ totals, Hpm lower slacks, Hf upper slacks, and $(H - 1)d$ blockers. Summing gives (91). The residual families contribute, in order, Hm , He , $2Hp$, Hp , $2Hpm$, $2Hf$, $(H - 1)d$, and $(H - 1)m$, which gives (92). Squaring residuals of degree at most two proves the degree bound. $\square$

For $H = 0$ there are no witnesses and the separate polynomial

$$Q_{A,I,0}(x, y) = \sum_{j=1}^p (y_j - x_j)^2$$

represents the single empty trace when $x = y$, and no trace otherwise.

20.3 Counting, costs, and finite fibres

Corollary 20.2 (Exact trace counts; compare Corollary 12.1). *For each fixed x, y, H , the number of natural roots of $Q_{A,I,H}$ is the number of executable trace classes between these endpoints with height at most H , and is at most 2^{Hm} . If prescribed letter counts n_a are desired, adjoining the squared linear residuals*

$$\sum_{h=1}^H z_{ha} - n_a$$

preserves degree at most four and selects precisely those traces with these counts. No new witness is required.

Proof. Use the bijection in Theorem 20.1. There are at most 2^{Hm} Boolean layer matrices, each with at most one completion. The additional residuals record exactly the number of occurrences of each action in that matrix. $\square$

The same construction records exact total length or any fixed additive integer action cost by one further linear equation. It does not encode a scheduling-dependent cost that varies between equivalent words; such a cost is not a function on the chosen quotient and would require a different semantic contract.

Proposition 20.3 (An explicit bound on all history witnesses). *Suppose $W \geq 1$ bounds every finite guard constant and every $|v_{aj}|$, and $X = \max_j x_j$. Every root of $Q_{A,I,H}$ has all witness coordinates at most*

$$B = X + (H + 1)mW + 2W + 1. \quad (94)$$

Proof. A layer contains at most m actions, so every executable boundary state is at most $X + HmW$ in each coordinate. Each loss or gain total is at most mW . An active lower slack is $M_{h-1,j} - L_{hj} - r_{aj}$, which is nonnegative and no greater than its state coordinate because $r_{aj} \geq 0$. An active upper slack is at most $s_{aj} \leq 2W$. Inactive slacks are zero; all bits and blockers are zero or one. These bounds are each no greater than (94). $\square$

Consequently the bounded-height construction permits complete finite enumeration in principle, with witness bit length logarithmic in B . This is a representation and size statement, not a claim that enumerating B^{NH} points is a practical solver.

20.4 A parameter that must not be hidden

The bound H determines the number of variables and equations. Thus $Q_{A,I,H}$ is an effectively generated family of ordinary polynomials, not one fixed-arity polynomial with H as an input variable. Taking the union over all H yields an effective reachability relation, and MRDP can represent that union existentially. Nothing in that observation preserves the preceding root bijections or finite-foldness.

Part III

Compressed repetition and accelerators

Source and scope. [write] This Part collects the fixed-arity constructions whose witness count does not grow with repetition counts. Manuscript 01's Sections 4–5 give the single-fold quartic accelerator for a multiset of box-guarded actions (Theorem 21.1), fixed block programs, exact powers of a fixed word (Proposition 22.2) and rank and progress criteria for endpoint-level single- and finite-foldness; its Section 8 works the examples (eleven witnesses for multiplicities 10^{100} , 10^{80} , 10^{90}). Manuscript 06's Sections 5–6 prove that nested normality only sees repetition counts 0, 1 and at least 2 (Theorems 24.1 and 24.2) and build the single-fold quartic compiler for parametrized nested repetition expressions (Theorem 25.1); its Section 8 locates the expressiveness boundary (flat schemes are Presburger-definable, separated depth-two schemes represent arbitrary quadratic systems).

Relation between the two. [write] Neither construction contains the other. Manuscript 06's nested expressions generalize manuscript 01's single-level word power to arbitrarily nested repetitions and add the normality test; manuscript 01's actions carry upper guards, and its accelerator characterizes *every* serialization of a multiset, and *some* serialization under commutation. The single-word case appears in both forms, Proposition 22.2 and Theorem 7.5 (Part I), and both are credited.

Setting and hypotheses. [write] Manuscript 01: a fixed finite alphabet of box-guarded translations on $\mathbb{N}^p$; multiplicities, endpoints and phase counts are parameters of the polynomial, not witnesses, unless a corollary says otherwise. Manuscript 06: consume/produce transitions on p counters with lower guards only, q labels, and a fixed repetition expression with N syntax nodes whose repetition parameters are inputs.

21 A fixed-arity single-fold quartic accelerator [01]

21.1 The complete polynomial system

Fix the action descriptions once and for all. Treat $x, y \in \mathbb{N}^p$ and $n \in \mathbb{N}^m$ as *parameters*. Let

$$f = \#\{(a, j) : u_{aj} < \infty\}.$$

Introduce natural witnesses z_a, k_a , two totals L_j, G_j , lower slacks α_{aj} for all a, j , and upper slacks β_{aj} exactly for the finite upper guards. Define the following residuals:

$$z_a(z_a - 1), \quad n_a - z_a(1 + k_a), \quad (1 - z_a)k_a; \quad (95)$$

$$L_j - \sum_a n_a v_{aj}^-, \quad G_j - \sum_a n_a v_{aj}^+, \quad y_j + L_j - x_j - G_j; \quad (96)$$

$$z_a(x_j - L_j - r_{aj} - \alpha_{aj}), \quad (1 - z_a)\alpha_{aj}; \quad (97)$$

$$z_a(s_{aj} - G_j - x_j - \beta_{aj}), \quad (1 - z_a)\beta_{aj} \quad (u_{aj} < \infty). \quad (98)$$

Here and throughout the residuals are *integer* expressions evaluated on natural variables. Subtraction is not truncated. Let P_A be the sum of the squares of all listed residuals.

Theorem 21.1 (Single-fold quartic accelerator). *For every fixed action alphabet A , P_A has total degree at most four and*

$$N_{\text{acc}} = 2m + 2p + pm + f \quad \text{witnesses}, \quad (99)$$

$$E_{\text{acc}} = 3m + 3p + 2pm + 2f \quad \text{squared residuals}. \quad (100)$$

For every fixed (x, y, n) , its natural zero set has exactly one point if every serialization of n executes from x to y , and is empty otherwise. If the active actions commute, “every” can be replaced by “some.” Neither arity nor degree depends on the magnitudes of the multiplicities.

Proof. A sum of integer squares vanishes exactly when every residual vanishes. The first residual in (95) forces $z_a \in \{0, 1\}$. If $n_a = 0$, the second forces $z_a = 0$, and the third then forces $k_a = 0$. If $n_a > 0$, the second forces $z_a = 1$ and $k_a = n_a - 1$, after which the third is automatic. Thus the support encoding has one and only one solution for every natural n_a .

Equations (96) uniquely set $L = S^-$ and $G = S^+$. If $z_a = 1$, the lower residual forces $\alpha_{aj} = x_j - S_j^- - r_{aj}$; the naturalness of this uniquely determined slack is exactly the relevant lower bound. If $z_a = 0$, the inactive equation uniquely sets $\alpha_{aj} = 0$, and no guard is imposed. The same argument for the finite upper guard gives $\beta_{aj} = s_{aj} - S_j^+ - x_j$ when active and zero otherwise.

The existence of all these natural slacks is therefore exactly the envelope box. The remaining endpoint residual is (19). Theorem 6.1 gives soundness and completeness; Corollary 6.2 gives the existential interpretation under commutation. Every witness was uniquely determined in the preceding argument, proving single-foldness.

There are $2m$ support witnesses, $2p$ totals, pm lower slacks, and f upper slacks. The three support residuals per action, three total/endpoint residuals per coordinate, and two residuals per slack give the stated counts. Every residual has degree at most two, including degrees in the parameter variables, so P_A has degree at most four. $\square$

21.2 Why the inactive equations are indispensable

The equation $z\alpha = 0$ by itself is not a canonical conditional constraint: when $z = 0$ it permits every natural α . In our construction the active residual imposes the inequality and the complementary residual

$(1 - z)\alpha = 0$ fixes unused storage to zero. A single omitted inactive equation can turn a singleton fibre into an infinite one.

Similarly, $n = z(1 + k)$ does not determine k when $n = z = 0$. The third residual in (95) is not an optimization detail; it is part of the uniqueness proof. These are small examples of a general rule: a semantic proof that ignores inactive branches need not be a fibre-preserving proof.

21.3 What is compressed and what is not

The construction compresses arbitrarily many repetitions into fixed-arity arithmetic. It does not promise small *numbers*: the witness $k_a = n_a - 1$ has essentially the same bit length as n_a , as it should. The totals are computed by linear forms in the multiplicities. No powers, factorials, prime coding, or CRT witnesses occur.

The sparse residual presentation has $O(pm + f)$ terms for a fixed action description, counting the terms in all aggregate sums. Expanding the sum of squares can produce $O(pm^2)$ terms from those sums, before cancellation. Thus a short constraint list, a short arithmetic circuit, and a small expanded monomial list are different size measures. The attachment exposes both representations for the worked examples, rather than silently equating them.

Remark 21.2 (Not a new universal variable bound). The arity in (99) is a bound for the specific envelope relation of a fixed finite alphabet. It is not a universal Diophantine-equation bound, and it does not improve ProveIt's independently reported universal certificate operation count [1].

22 Fixed block programs with unbounded repetition [01]

A single commuting block is not the only unbounded relation obtained without sequence coding. Fix an ordered list of $b \geq 1$ phases. In phase s , a fixed alphabet A_s of m_s actions is pairwise commuting. Multiplicities $n^{(s)}$ may be arbitrarily large. Only the number and order of the phases are fixed.

Theorem 22.1 (Single-fold composition with explicit counts). *The relation asserting that the successive phases with multiplicities $n^{(1)}, \dots, n^{(b)}$ carry x to y has a single-fold quartic representation. It has*

$$(b - 1)p + \sum_{s=1}^b (2m_s + 2p + pm_s + f_s) \quad (101)$$

witnesses, in addition to the endpoint and multiplicity parameters. Its arity is independent of every repetition count.

Proof. Introduce natural boundary states $X_1, \dots, X_{b-1}$, set $X_0 = x$ and $X_b = y$, and use the residual system of Section 21 for each phase between X_{s-1} and X_s , with disjoint local witnesses. The sum of all residual squares is still a quartic.

For fixed input and all phase counts, the endpoint of each phase is forced by its net displacement. Consequently every intermediate state is unique. Once those states are fixed, Theorem 21.1 gives unique local witnesses whenever the phase executes. Conversely, a zero provides a legal execution of each phase; their concatenation is a legal block program. Counting the boundary states and local witnesses gives (101). $\square$

The phase counts are semantic data in this theorem. If the same label can occur in adjacent phases, one untagged trace could have more than one decomposition into phase counts. The theorem does not claim to eliminate that ambiguity. One may retain phase tags as part of the semantic object, require counts as input, or establish a separate unique-decomposition result.

22.1 Fixed-word macros: noncommuting loop bodies

The same model also compresses repetition of a fixed word whose individual actions need not commute. This is a closure consequence, not a new universality claim.

Proposition 22.2 (Exact powers of a fixed word). *Let $w = a_1 \cdots a_t$ be fixed, put $\delta_i = \sum_{k=1}^i v_{a_k}$ with $\delta_0 = 0$, and set $\Delta = \delta_t$. Its partial map has domain given by*

$$\ell_{w,j} = \max(0, \max_{1 \leq i \leq t} (\ell_{a_i j} - \delta_{i-1,j})), \quad (102)$$

$$u_{w,j} = \min_{1 \leq i \leq t} (u_{a_i j} - \delta_{i-1,j}). \quad (103)$$

If this box is nonempty, it defines a valid guarded-translation macro with displacement Δ . For $n > 0$, the word w^n executes precisely when

$$x_j \geq \ell_{w,j} + (n-1)\Delta_j^-, \quad x_j \leq u_{w,j} - (n-1)\Delta_j^+ \quad (104)$$

for every coordinate, with infinite upper bounds omitted. Its endpoint is $y = x + n\Delta$. This relation, including $n = 0$, has a single-fold quartic representation with $2 + 3p + f_w$ witnesses, where f_w counts the finite macro upper bounds. The arity is independent of both n and the length of the already compiled fixed word.

Proof. Before letter a_i , the formal state is $x + \delta_{i-1}$. Pulling every guard back to x gives (102)–(103). The same induction on formal prefix states as in Theorem 6.1 proves that their intersection is exactly the execution domain. If nonempty, its lower corner itself lies in the domain. Its final state is natural, so $\ell_{w,j} + \Delta_j \geq 0$, proving the required validity condition for the macro.

Apply Theorem 6.1 to this one macro with multiplicity n . Its lower envelope is $n\Delta_j^- + \ell_{w,j} - \Delta_j^-$, and its upper envelope is $u_{w,j} + \Delta_j^+ - n\Delta_j^+$, giving (104). Theorem 21.1 with $m = 1$ supplies the stated witness count, includes the zero-repetition case, and proves unique completion.

If the macro domain is empty, only $n = 0$ can execute; that special case is instead represented without witnesses by $n^2 + \sum_j (y_j - x_j)^2$. For the empty word, use the identity macro with lower bound zero and no upper bound. $\square$

This extends the fixed-phase construction to expressions such as $w_1^{n_1} \cdots w_b^{n_b}$ for fixed words w_s , even if the letters within a loop body are highly dependent. The macro description is computed in time linear in the total word length times the state dimension; its guard and displacement constants can grow with that length. Thus constant witness arity does not mean that the source word has disappeared from the coefficient description. An empty-domain macro simply forces its repetition count to zero. Multiplicities remain specified parameters unless a separate argument such as the next two corollaries controls their fibres.

[write] Priority. Part XII treats fixed words under first-enabled-rule priority. For a fixed program its block relation is an instance of Theorem 31.2; the new content there is uniformity in the program entries, the capacity formula and the sharp threshold $2H + 1$ (Theorems 96.4 and 97.1).

22.2 Endpoint-level single-foldness by displacement rigidity

There is a useful sufficient criterion for making the counts existential without creating extra roots. List all phase-labelled action occurrences as $q = \sum_s m_s$ action types. Let $V \in \mathbb{Z}^{p \times q}$ have their displacements as columns.

Corollary 22.3 (Rigid endpoint certificates). *If V has full column rank over $\mathbb{Q}$, then the endpoint relation*

$$R(x, y) \iff \text{some phase multiplicities carry } x \text{ to } y$$

has a single-fold quartic representation obtained from Theorem 22.1 by turning the q count parameters into witnesses.

Proof. Every root satisfies $y - x = Vn$. Full column rank makes n unique over $\mathbb{Q}$, hence at most one natural count vector can occur. For that vector, Theorem 22.1 makes every remaining witness unique. Existence is unchanged by treating the counts existentially. $\square$

Full column rank is sufficient, not necessary. It is deliberately easy to verify and identifies the source of uniqueness: the endpoint recovers the semantic counts, while the compiler recovers their auxiliary certificate.

22.3 Endpoint-level finite-foldness from progress

Corollary 22.4 (Progress-bounded fibres). *Suppose there is a vector $w \in \mathbb{Q}^P$ and $\varepsilon > 0$ such that $w \cdot v_i \geq \varepsilon$ for every phase-labelled action type. The same existential-count quartic is finite-fold at each fixed pair (x, y) .*

Proof. Every root obeys

$$w \cdot (y - x) = \sum_i n_i (w \cdot v_i) \geq \varepsilon \sum_i n_i.$$

If the left side is negative there are no roots. Otherwise all count vectors lie in the finite set $\{n \in \mathbb{N}^q : \sum_i n_i \leq w \cdot (y - x) / \varepsilon\}$. For each such vector the remaining fibre is empty or a singleton by Theorem 22.1. Hence the complete fibre is finite. $\square$

These corollaries are restricted positive results, not consequences of a general finite-fold theorem. Their hypotheses are explicit linear-algebraic or progress conditions. In particular, the number of phases is fixed in the polynomial, and arbitrary branching between an unbounded number of phases is outside their scope.

22.4 A directly linear description of each support case

For a fixed support set of each phase, all active-action tests in the envelope become finitely many linear inequalities in the endpoints, boundary states, and multiplicities. Inactive counts are zero and active counts are positive. Thus these block relations have a finite union of such integer linear descriptions, with boundary states eliminable by the displacement equations. The polynomial construction is valuable because it supplies an explicit unique completion and a uniform degree bound, not because it proves previously unavailable decidability for these restricted relations.

23 Worked examples [01]

23.1 Eleven witnesses for an enormous shared-resource block

Consider three one-place Petri actions:

$$a : (c, d) = (2, 1), \quad b : (c, d) = (3, 1), \quad r : (c, d) = (1, 1). \quad (105)$$

The first two decrease the marking by one and two respectively; the third reads the shared retained token. By Proposition 5.3, all three commute. For a, b , both final production thresholds equal one; the read threshold is at most both retained production thresholds.

Put $L = n_a + 2n_b$. If at least one multiplicity is positive, the exact condition is

$$x \geq L + 1, \quad y = x - L. \quad (106)$$

If all multiplicities vanish, the condition is instead $y = x$, including $x = 0$. The empty-support case cannot be silently replaced by the positive-support inequality.

In this example the eleven witnesses are

$$z_a, z_b, z_r, \quad k_a, k_b, k_r, \quad L, G, \quad \alpha_a, \alpha_b, \alpha_r.$$

For $i \in \{a, b, r\}$ the support residuals are

$$z_i(z_i - 1), \quad n_i - z_i(1 + k_i), \quad (1 - z_i)k_i.$$

The remaining nine residuals are

$$\begin{aligned} &L - n_a - 2n_b, \quad G, \quad y + L - x - G, \\ &z_i(x - L - 1 - \alpha_i), \quad (1 - z_i)\alpha_i \quad (i = a, b, r). \end{aligned}$$

Their eighteen squares define one quartic. The generic construction retains the forced-zero gain witness G for a uniform interface; it could be eliminated in this particular example, so eleven is not asserted to be minimal.

Set

$$n_a = 10^{100}, \quad n_b = 10^{80}, \quad n_r = 10^{90}, \quad x = 10^{100} + 2 \cdot 10^{80} + 1, \quad y = 1.$$

The unique witness is $z_i = 1$, $k_i = n_i - 1$, $L = n_a + 2n_b$, $G = 0$, and all three lower slacks zero. These formulas evaluate directly, without traversing the astronomical number of events. The attached expanded polynomial has total degree four and 76 nonzero monomials before specializing the parameters. Its JSON file includes all eighteen residuals, the full expanded coefficients, and the exact natural certificate.

23.2 Many schedules, one trace, one polynomial root

For the same three actions, take the normal form

$$C_1 = \{a, b, r\}, \quad C_2 = \{a, r\}, \quad C_3 = \{a\}.$$

It has counts $(3, 1, 2)$. Since all distinct labels are independent, all

$$\frac{6!}{3! 1! 2!} = 60$$

words with these counts are one trace. Its height is three, not six, because repeated copies of a label remain dependent.

From $x = 9$, the boundary markings are 9, 6, 5, 4, so $y = 4$. The compiler with $m = 3$, $p = 1$, $H = 3$, $f = 0$, $e = 0$, and $d = 3$ produces

$$N_H = 32, \quad E_H = 48.$$

There is exactly one root for this trace. There may be other roots for other traces with the same endpoints and height bound; adding the three count equations isolates this particular trace. This qualification is essential: endpoint equality alone is not a specification of the history being counted.

23.3 Upper guards and an identity block

Let a be the increment on $0 \leq x \leq 4$ and let r be a read on $0 \leq x \leq 5$. They commute: the domain of either two-action order is $0 \leq x \leq 4$. For multiplicities n_a, n_r , the upper envelope includes $x \leq 5 - n_a$ whenever $n_a > 0$, and the same inequality from the read whenever it is active; the lower envelope is $x \geq 0$. Thus with three increments and any positive number of reads, the allowed starts are exactly 0, 1, 2, and the endpoint is $x + 3$.

If no increment occurs but the read does occur, the bound is $x \leq 5$. If neither occurs, there is no upper bound at all. The support variables in the quartic distinguish all three cases without an arbitrary branch-choice witness.

23.4 A diagnostic for spurious concurrency

Let a, b be distinct labels, each consuming one token and producing none. At $x = 1$, the naive Boolean layer conditions “both selected actions are individually enabled” would accept $\{a, b\}$. The exact layer envelope has loss two and retained margin zero, so rejects it. At $x = 2$, both sequential orders execute and the layer is valid. This is precisely the operational interpretation of Remark 5.2; the compiler does not confuse a clique with a feasibility certificate.

24 Saturation: nested normality only sees zero, one, and two [06]

Theorem 24.1 (Square saturation). *Every signature $S = (C, R, \alpha, \beta)$ satisfies*

$$S^{\odot 2} = \left(C \prod_{a \in A} (1 - R_a \beta_a), R, \alpha, \beta \right), \quad S^{\odot 3} = S^{\odot 2}. \quad (107)$$

Consequently $S^{\odot k} = S^{\odot 2}$ for every $k \geq 2$.

Proof. Apply (76) with both factors equal to S . Since the entries are bits, $\alpha_a^2 = \alpha_a$ and $R_a \vee (R_a \alpha_a) = R_a$. Disjointness gives $\alpha_a \beta_a + \beta_a = \beta_a$. The scalar coordinate is the one shown in (107). Multiplication by S once more changes only the scalar coordinate and multiplies it by the same Boolean factor $C \prod_a (1 - R_a \beta_a)$. That factor is idempotent, so nothing changes. Induction proves the assertion for every larger power. $\square$

There are two separate stabilization phenomena. A block’s final-memory map is already idempotent after one application. A second copy can nevertheless expose a forbidden pattern straddling the boundary between the copies. Once that possibility has been checked, a third copy cannot expose a new kind of failure. The signature calculation records both facts, including rejecting blocks.

Define $\text{cap}_2(k) = \min(k, 2)$ and apply it componentwise to parameter tuples.

Theorem 24.2 (Cap-at-two theorem for nested schemes). *For every expression E from Definition 2.16 and every parameter tuple $\mathbf{k} \in \mathbb{N}^r$,*

$$\text{sig}(\llbracket E \rrbracket_{\mathbf{k}}) = \text{sig}(\llbracket E \rrbracket_{\text{cap}_2(\mathbf{k})}). \quad (108)$$

In particular, the two expanded words are either both normal or both nonnormal. Parameters may be reused, and repetitions may be nested to any fixed finite depth.

Proof. Proceed by structural induction. The empty and letter cases do not involve parameters. For concatenation, the induction hypotheses give equal child signatures, and the product formula gives equal

parent signatures. At a repetition node F^{k_j} , the induction hypothesis first identifies the signature S of the body under the two parameter assignments. The two parent signatures are then $S^{\odot k_j}$ and $S^{\odot \text{cap}_2(k_j)}$, which agree by Theorem 24.1, including the identity case $k_j = 0$. Reuse of a parameter causes no difficulty because the same componentwise capping is applied everywhere. Finally, acceptance from initial memory zero is exactly the scalar condition $C = 1$. $\square$

Corollary 24.3 (Finite parameter cells for normality). *For a fixed expression with r distinct parameters, its normality predicate is a union of some of the 3^r cells*

$$X_1 \times \cdots \times X_r, \quad X_j \in \{\{0\}, \{1\}, \{2, 3, 4, \dots\}\}.$$

It can be evaluated from an $O(qN)$ -size circuit over the three-way tests on parameters, without constructing an expanded word or enumerating all cells.

Proof. The theorem shows constancy on each cell. Its inductive proof supplies the circuit: one signature product per concatenation, and at a repetition choose the identity, the child signature, or its square according to the three-way parameter test. Each signature product has $O(q)$ gates. $\square$

Example 24.4 (Why the threshold two is sharp). Take $a < b$ and $(a, b) \in I$. The word ab is normal, but $(ab)^2 = abab$ is not: its middle ba can be swapped. Thus capping all positive powers at one would be false, even for one unnested repetition node. The theorem does not claim that normality is unchanged when zero is identified with one, either.

Remark 24.5 (What saturation does not preserve). The words a^2 and a^{1000} need not have the same resources, output, length, Parikh vector, or trace class. The signature is a homomorphic image of the free word monoid, not of the trace monoid: the independent words ab and ba can have different normality signatures. By contrast, the resource-summary map factors through the trace relation. Cap-at-two is a theorem about the canonicity test only, not a compression of the actual resource action to bounded powers.

25 A single-fold quartic compiler [06]

We now compile the two summaries together. The construction is elementary: it uses ordinary polynomial equations, nonnegative unknowns, and no exponentiation predicate, bounded universal quantifier, or invocation of MRDP.

25.1 Quadratic gates with forced values

A Boolean wire b is constrained by $b(b-1) = 0$. With Boolean inputs, the following equations uniquely determine their Boolean outputs:

$$t = xy \quad (\text{AND}), \quad t = x + y - xy \quad (\text{OR}), \quad t = 1 - x \quad (\text{NOT}). \quad (109)$$

One may add the output bit equation explicitly, although its value already follows from the inputs. Long conjunctions are implemented by a chain or a balanced tree of binary AND gates. Expressions such as $R_a \vee ((S_0)_a \alpha_a)$ must use an intermediate wire for the inner product; substituting it directly into the OR equation would create a cubic residual. Likewise the product defining C' in (76) is not left as a single high-degree residual.

To share a repetition switch with both branches of the compiler, introduce $z_0, z_1, z_2, h \in \mathbb{N}$ and impose

$$\begin{aligned} z_i(z_i - 1) &= 0 \quad (0 \leq i \leq 2), & z_0 + z_1 + z_2 &= 1, \\ k &= z_1 + 2z_2 + h, & (1 - z_2)h &= 0. \end{aligned} \quad (110)$$

Exactly one selector is one. If it is z_0 , then $h = 0$ and $k = 0$. If it is z_1 , then $h = 0$ and $k = 1$. If it is z_2 , then $k = 2 + h \geq 2$. Conversely, each k chooses exactly one case and a unique h . Thus the selector gadget has no surplus solutions.

For resources the active bit is $e = z_1 + z_2$, used in (28). For a signature, compute the child's square and then select, coordinate by coordinate, among the identity, child, and square:

$$s' = z_0 s_{\text{id}} + z_1 s + z_2 s_{\text{square}}. \quad (111)$$

The three terms are at most quadratic, since each s is a wire rather than an expanded polynomial. Exactly one term is active. Both the child and its square remain fully constrained in inactive cases.

25.2 The polynomial and its correctness contract

At each syntax node allocate its $2p$ resource coordinates. For an empty node they are zero; for a letter node they are the transition weights. At a concatenation node impose the minimum gadget coordinatewise and the two output equations following Lemma 7.2. At a repetition node impose the child minimum, the selector gadget, and the power output equations. For the canonical version, also allocate the signature at each node and the intermediate Boolean wires described above.

At the root impose, with a new slack vector $r \in \mathbb{N}^p$,

$$m = U_E + r, \quad n = V_E + r. \quad (112)$$

In the canonical version also impose $C_E = 1$. List all integer residuals as $r_1, \dots, r_s$. Every residual has total degree at most two in all free inputs and witnesses. Define

$$Q_E(m, n, \mathbf{k}, z) = \sum_{i=1}^s r_i(m, n, \mathbf{k}, z)^2. \quad (113)$$

This is one ordinary polynomial in $\mathbb{Z}[m, n, \mathbf{k}, z]$ of degree at most four. Its sum-of-squares presentation is simply a convenient exact way to store it; it is not an extension of polynomial syntax.

Theorem 25.1 (Compressed single-fold compilation). *For every fixed expression E , the construction above effectively produces polynomials Q_E^{res} and Q_E^{can} of degree at most four. For every fixed $m, n \in \mathbb{N}^p$ and $\mathbf{k} \in \mathbb{N}^r$,*

$$\#\{z : Q_E^{\text{res}}(m, n, \mathbf{k}, z) = 0\} = \mathbf{1}_{m \xrightarrow{\llbracket E \rrbracket_{\mathbf{k}}} n}, \quad (114)$$

$$\#\{z : Q_E^{\text{can}}(m, n, \mathbf{k}, z) = 0\} = \mathbf{1}_{m \xrightarrow{\llbracket E \rrbracket_{\mathbf{k}}} n \wedge \llbracket E \rrbracket_{\mathbf{k}} \text{ normal}}. \quad (115)$$

The resource-only compiler uses $O(pN)$ witnesses and equations. The canonical compiler uses $O((p+q)N)$ witnesses and equations. These bounds are independent of the expanded execution length and of the numerical parameter values.

Proof. All residuals are integer-valued. Therefore $Q_E = 0$ is equivalent to every residual being zero: a sum of nonnegative integer squares vanishes only when each square vanishes.

We first show that all internal witnesses are uniquely determined by the parameters. At an empty or letter node the summary coordinates are fixed constants. If the child summaries at a concatenation are fixed, Lemma 7.4 uniquely determines the cancellation variables, and the output equations uniquely determine the parent coordinates. At a repetition, the child minimum is likewise unique, (110) uniquely determines the selectors and remainder, and (28) uniquely determines the parent summary. Structural induction proves uniqueness everywhere in the resource branch, as well as agreement with the actual word summary by Lemma 7.2 and Theorem 7.5.

For the canonical branch, the base signatures are fixed. Each elementary Boolean gate has a unique value for fixed inputs; concatenation therefore computes exactly (76). The same argument computes the child square and the selected power signature, which agrees with the full word power by Theorem 24.1. All inactive branches are still computed, so zero or one repetitions do not release any unconstrained variables. This proves uniqueness of every signature and intermediate wire.

The only remaining witnesses are the root slacks. By Lemma 7.1, they exist exactly for an execution with the stated endpoints and are unique when they exist. In the canonical version, the additional equation $C_E = 1$ holds exactly when the word accepts from zero memory, hence exactly when it is normal. This proves both counting equalities, including nonexistence on rejected inputs.

For the size estimate, each resource node needs a constant number of vectors of dimension p and each signature node needs $O(q)$ wires. A repetition switch has constant size. Thus summing over N syntax nodes gives the stated bounds. The use of intermediate wires ensures the quadratic residual bound, and (113) doubles the degree at most. $\square$

25.3 An explicit resource-only variable bound

The asymptotic estimate does not hide a dependence on the numerical repetition counts. A more precise bound is available for a streamlined resource-only implementation using the two-way switch (27). Let C be the number of concatenation nodes and R the number of repetition nodes. Allocate all $2p$ summary coordinates at every node, three minimum witnesses per coordinate at every internal node, two switch witnesses per repetition, and p endpoint slacks. This gives

$$2pN + 3p(C + R) + 2R + p \leq (5p + 2)N + p \quad (116)$$

witnesses. Some are constant or could be shared, but retaining them makes the inductive proof transparent. The supplied prototype intentionally uses the three-way switch in both modes and some redundant Boolean bit equations; its exact counts therefore need not equal this optimized resource-only count.

25.4 Certificate height and bit complexity

Let W be the largest transition weight, K the largest supplied repetition parameter, and h the repetition depth. Put $K = 0$ when no parameter occurs. The next bound concerns a syntax tree, not a recursively defined program.

Proposition 25.2 (Witness height). *For an accepted input to the compiler in Theorem 25.1, its unique witness can be chosen—and in the displayed construction is forced—with all coordinates at most*

$$H = \max \left(1, K, \|m\|_\infty, WN(1 + K)^h \right). \quad (117)$$

In particular, exponentially long expansions need not require exponentially many witness bits per coordinate.

Proof. For any subexpression, each letter occurrence in its syntax contributes at most a product of h repetition counts to its expanded length. Summing over at most N letter leaves gives length at most $N(1 + K)^h$ for every subexpression, including ones discarded by an outer zero repetition.

For a word of length L , its summary satisfies $U_w, V_w \leq WL$ coordinatewise. One way to see this is to start with the bound W at leaves and use (24), which never exceeds the sum of the child bounds; an equivalent direct bound follows from (23). Minimum witnesses and their residual differences are bounded by the corresponding child coordinates. The intermediate nonnegative terms in a power output are bounded by that output. Selector remainders are at most K , Boolean wires are at most one, and the endpoint slack is at most m . These bounds give (117). $\square$

Thus a safe per-coordinate binary-length bound is

$$O(1 + \log(1 + K) + \log(1 + \|m\|_\infty) + \log(1 + W) + \log(1 + N) + h \log(1 + K)).$$

The root expansion length alone would not provide a correct bound: a zero outer power can erase an enormous child, whose uniquely constrained internal certificate is still present. This is another reason to track all syntax nodes, rather than only the visible final word.

A directed acyclic expression graph can share child summaries and retain the $O((p + q)N)$ variable bound when N counts distinct graph nodes. The tree-height estimate must then be replaced by a bound on expansion of shared nodes. A coarse universal estimate is $2^N (1 + K)^N$ for their lengths, obtained by induction over an ordering of the acyclic graph. Its logarithm is still polynomial in N and the input bit lengths. The prototype in this package uses trees, not graph sharing.

25.5 Worked examples and a scope warning

Example 25.3 (A nested multiplication schedule). Let $+$ increment a counter and $-$ decrement it when positive. The scheme

$$E_{x,y,z} = ((++)^x)^y + + + (-)^z \quad (118)$$

executes from zero to zero exactly when

$$z = 2xy + 3.$$

Its expansion first makes $2xy + 3$ increments and then z decrements. Thus there is no intermediate enabling obstruction when the equality holds. For $x = 3, y = 4, z = 27$, the concrete word has length 54. For $x = 10^{100}, y = 10^{120}$ and $z = 2xy + 3$, the same fixed compiler verifies a word whose length $2z$ has 221 decimal digits, without expanding it. There is no independence between $+$ and $-$ for a single counter, so every word is normal for the resulting empty independence relation.

Example 25.4 (Shared-resource commutation). Suppose a consumes one token and b consumes two tokens from the same counter, producing none, and $a < b$. The global commutation criterion includes (a, b) : either order has summary $(3, 0)$. From marking 3, the words ab and ba are two executable representatives of one trace. The canonical compiler accepts the former and rejects the latter even though their resource certificates describe the same partial action.

Remark 25.5 (A canonicity filter is not a normalizer). For a restricted expression family, adding the canonicity condition can remove every feasible instance. If a fixed expression is ba with $a < b$ and $(a, b) \in I$, its normal representative ab need not occur anywhere in that expression family. Therefore (115) tests whether the *given expansion* is canonical; it does not preserve existential reachability for every restricted schedule language. The trace-class bijection in the next section avoids this problem by ranging over all words of the fixed length, a trace-closed universe.

26 Expressiveness and an exact nesting boundary [06]

The compiler gives short certificates *once the repetition counts are fixed*. The existence of suitable counts is a separate problem. We now show that sharing parameters between nested repetitions is enough to recover Diophantine universality even with exceptionally simple counter commands.

26.1 Flat schemes are Presburger-definable

Theorem 26.1 (Flat-scheme decidability). *For every fixed token system and every expression of repetition depth at most one, the relation*

$$\{(m, n, \mathbf{k}) : m \xrightarrow{\llbracket E \rrbracket_{\mathbf{k}}} n\}$$

is effectively definable in Presburger arithmetic. The same holds with the additional requirement that the expanded word is normal. Consequently, existence of parameters for fixed endpoints is decidable in this fragment.

Proof. A repetition body has no repetition nodes and hence has a constant resource summary (u, v) . In the zero case its power is $(0, 0)$; in the positive case (26) is affine in the repetition parameter, because c, α, β are constants. This case split is expressible with $k = 0$ or $k \geq 1$.

Concatenation combines summaries by addition and coordinatewise minimum. The graph of $c = \min(x, y)$ has the Presburger definition

$$(x \leq y \wedge c = x) \vee (y \leq x \wedge c = y).$$

Thus structural induction yields a Presburger formula for all summaries and endpoints. Reusing a parameter only reuses the same variable in affine expressions; it does not introduce a product of two variable quantities at a depth-one repetition node. Normality is also Presburger-definable, either by the finite cells of Corollary 24.3 or by the finite Boolean circuit and the tests $k = 0, k = 1, k \geq 2$. Existential quantification over parameters preserves Presburger definability, and Presburger arithmetic is decidable. $\square$

This proof deliberately does not use the quadratic equation $st = 0$ to define the minimum inside Presburger arithmetic. It replaces that gadget by a linear case distinction. The nonlinear certificate language and the simpler decision procedure for a restricted fragment are compatible but serve different purposes.

26.2 Separated one-counter schemes and polynomial degree

Consider one counter with commands $+$ and $-$. Call a scheme *separated* if it is a concatenation $E_+ E_-$ where E_+ uses only $+$ and E_- uses only $-$. Empty expressions are allowed. For a fixed parameter tuple, let $A(\mathbf{k})$ and $B(\mathbf{k})$ be their respective lengths.

Lemma 26.2 (Length polynomial). *For any expression using a single letter, its expansion length is a polynomial in its repetition parameters with nonnegative integer coefficients. Its total degree is at most the repetition depth.*

Proof. The length polynomials of ε and a letter are 0 and 1. Concatenation adds length polynomials, while repetition by k_j multiplies the body length by k_j . Induction proves both nonnegativity of coefficients and the degree bound, since repetition increases nesting depth by one and concatenation takes the maximum of child depths. $\square$

Theorem 26.3 (Solution-preserving degree–depth correspondence). *For each fixed $d \geq 0$, the following two problem presentations are effectively interconvertible, preserving the parameter solution set exactly:*

- (i) *one polynomial equation $P(\mathbf{k}) = 0$ over $\mathbb{N}$, with $P \in \mathbb{Z}[\mathbf{k}]$ and $\deg P \leq d$;*
- (ii) *a separated one-counter scheme of repetition depth at most d , required to execute from zero to zero.*

No extra existential parameters are needed in either direction.

Proof. A separated scheme executes first $A(\mathbf{k})$ increments and then $B(\mathbf{k})$ decrements. It executes from zero to zero if and only if $A(\mathbf{k}) = B(\mathbf{k})$: equality supplies enough tokens for every decrement and leaves none. By Lemma 26.2, $P = A - B$ has degree at most d and gives exactly the same parameter solutions.

Conversely split an integer polynomial coefficientwise as $P = P_+ - P_-$, where P_+, P_- have nonnegative coefficients. For a monomial $c \prod_j k_j^{e_j}$, begin with c copies of the appropriate sign command and place them under $\sum_j e_j$ nested repetitions, using each k_j exactly e_j times. Its length is the monomial and its repetition depth is the monomial degree. Concatenate the positive monomial blocks to form E_+ and the negative ones to form E_- . The resulting scheme has depth at most d and accepts exactly when $P = 0$. $\square$

The direct construction writes a coefficient c as c letter occurrences. It is an effective solution-set correspondence, not a claim of polynomial size in a binary encoding of coefficients. Constant blocks could instead be represented by shared straight-line expressions. We do not need that optimization for any undecidability or multiplicity conclusion.

At $d = 1$, this recovers a linear Diophantine fragment. At $d = 2$, the separated *one-counter* problem is exactly natural solvability of one quadratic equation; decision procedures for positive integral solutions of arbitrary quadratic equations, and more general linear-region restrictions, are due to Grunewald and Segal [24]. Translating variables by one interconverts positive and nonnegative domains. At $d = 3$, the theorem states an exact equivalence with cubic Diophantine solvability; we make no additional classification of that problem here. At $d = 4$, undecidability follows from the next degree-reduction lemma and MRDP.

26.3 Unique quadratic systems and quartic equations

Lemma 26.4 (Multiplicity-preserving degree reduction). *For every $P(x, y) \in \mathbb{Z}[x, y]$, one can effectively construct quadratic or linear polynomials $r_1(x, y, g), \dots, r_s(x, y, g)$ such that, for each fixed $(x, y) \in \mathbb{N}^{a+b}$, the system $r_i = 0$ has exactly one natural solution g if $P(x, y) = 0$, and no solution otherwise. Hence*

$$Q(x, y, g) = \sum_{i=1}^s r_i(x, y, g)^2$$

has degree at most four and its roots project bijectively to the roots of P .

Proof. Split $P = P_+ - P_-$ into nonnegative-coefficient polynomials. Evaluate each by a finite arithmetic circuit using only nonnegative integer constants, original input variables, addition, and multiplication. Introduce one new variable for each gate output. An addition gate gives $g - u - v = 0$ and a multiplication gate gives $g - uv = 0$, with u, v either previously available variables or constants. Finally equate the two output wires, or the corresponding constants when a circuit has no gates.

Topological induction on the circuit shows that every gate variable is uniquely forced to a nonnegative integer for any fixed original inputs. The final equation then holds exactly when $P_+ = P_-$. This proves the unique-lifting assertion for the system. A sum of its squares is zero precisely when every gate equation and the final equality are zero, and its degree is at most four. $\square$

Avoiding signed intermediate witnesses is important. Representing each signed integer as an arbitrary difference of two naturals would introduce infinitely many encodings. Splitting the original polynomial into two nonnegative circuits avoids that problem entirely. This is a standard arithmetic-circuit normal form, used here with its multiplicity contract made explicit.

Corollary 26.5 (Depth-four one-counter universality). *Every computably enumerable relation $R \subseteq \mathbb{N}^a$ has a representation*

$$x \in R \iff \exists k \in \mathbb{N}^b \ 0 \xrightarrow{\llbracket E \rrbracket_{(x,k)}} 0$$

by a fixed separated one-counter scheme E of repetition depth at most four. The scheme uses only increment and enabled decrement. Its parameters may be globally reused.

Proof. Apply MRDP to obtain a polynomial representation of R . Apply Lemma 26.4, and then Theorem 26.3 at $d = 4$. Treat the input coordinates x as prescribed repetition parameters and all the other coordinates as existential repetition parameters. $\square$

This is a normal form for *existential schedule parameterization*. It does not say that an ordinary one-counter machine with freely chosen steps can compute an arbitrary partial recursive function. The numerical relationships between globally shared loop counts carry the Diophantine expressiveness.

26.4 Depth two is already universal with several resources

For each counter j , let $+_j$ and $-_j$ increment or decrement that counter and leave all others unchanged. A vector-separated scheme consists of a positive block using only the $+_j$, followed by a negative block using only the $-_j$.

Theorem 26.6 (Quadratic-system substrate and sharp flat/nested boundary). *A finite system of polynomial equations $P_j(\mathbf{k}) = 0$ of degrees at most d can be represented, without new parameters, by a vector-separated scheme of depth at most d on one counter per equation, executing from $\mathbf{0}$ to $\mathbf{0}$. Conversely, every such scheme gives a system of polynomial equations of degrees at most its depth.*

In particular, every computably enumerable relation has a fixed vector-separated representation of depth at most two. Therefore parameter existence for depth-two schemes in this class is undecidable, whereas it is decidable for all depth-one token schemes by Theorem 26.1.

Proof. Write $P_j = A_j - B_j$ coefficientwise with nonnegative polynomials. In the positive phase, use the monomial construction in Theorem 26.3 to perform A_j increments of counter j . Do this for every j . In the negative phase, perform B_j decrements of counter j for every j . Counter j starts and ends at zero exactly when $A_j = B_j$. If all equalities hold, every negative phase is enabled. If the run ends at zero, its net increment in every coordinate is zero, so all equalities hold. The depth bound and the absence of extra parameters follow from the monomial construction. The converse takes the increment and decrement counts in each coordinate and applies the length-polynomial induction.

For a computably enumerable relation, MRDP supplies one polynomial. Replace it by the quadratic system of Lemma 26.4, without first summing its squares. The preceding construction gives a depth-two scheme, with the gate variables among its existential parameters. Taking a computably enumerable undecidable relation proves undecidability. Since a single universal relation has one fixed polynomial and one fixed quadratic system, even the number of counters and the schedule template can be fixed for that relation; only its input parameters vary. The depth-one decision theorem proves the sharp boundary in the stated multi-resource class. $\square$

Corollary 26.7 (Universality can coexist with an already canonical word). *The universal schemes in Theorem 26.6 can be chosen so that every expansion is lexicographically normal for the global commutation relation.*

Proof. In the positive phase, group all occurrences of $+_1$, then $+_2$, and so on. Group the negative phase by $-_1, -_2, \dots$ in the same way. Choose the total alphabet order

$$+_1 < \dots < +_p < -_1 < \dots < -_p.$$

Every expanded word is globally nondecreasing in that order. It is therefore lexicographically least even among all permutations with the same letter multiplicities, hence also among the smaller set of words in its trace class. Grouping does not change the coordinatewise positive and negative counts or the depth of any monomial block. $\square$

The nontrivial search thus does not come from interleaving ambiguity. It persists for sorted, phase-separated schedules whose commands merely change one counter by one.

Part IV

Polynomial trajectories

Source and scope. [write] This Part is manuscript 02, “Canonical Diophantine Certificates for Polynomial Trajectories”: its statement of the problem and of the restricted result (the first two subsections of its introduction), then its Sections 3–9 (the finite-difference sign certificate, the single-fold quartic theorem and its cost, Boolean guards, acceleration of polynomial trajectories, the computational substrates covered, nontermination certificates, and worked examples), and its conclusion. Its canonical signed coordinates and total circuits (its Section 2) are printed with the conventions, in Section 2.3; its remaining introduction is in Section 3; its other substrates are in Parts VII and VIII and its boundary results in Part IX.

Setting and hypotheses. [write] Coefficient parameters are *signed* integers, $\mathbf{c} \in \mathbb{Z}^{d+1}$, with d a nominal degree bound; existential coordinates range over $\mathbb{N}$. T is the horizon of the index range $\{0, \dots, T\}$, not a number of execution steps. The main theorem is a family P_d indexed by the degree bound, uniform in coefficients and horizon.

27 The problem addressed and the precise restricted result [02]

[write] *The first two subsections of manuscript 02’s introduction; the rest of it is in Section 3.3.*

27.1 Existence is not multiplicity preservation

A computational model can be represented arithmetically in several different senses. An existence-preserving representation says that a computation exists exactly when an equation has a solution. A counting-faithful representation must additionally explain which solutions correspond to which computations, and whether auxiliary arithmetic choices create extra solutions. Even a deterministic computation can acquire infinitely many Diophantine witnesses through a noncanonical trace encoding.

The distinction is central to Matiyasevich’s finite-fold and single-fold questions. Classical results give both polynomial Diophantine representations of enumerable sets and single-fold *exponential* Diophantine representations. Combining the two properties is a separate problem, not a consequence of the ordinary MRDP theorem [9]. The present paper does not assume that combination.

Our starting question is more specific:

Can a long, deterministic, polynomial trajectory with nonmonotone guards be replaced by a fixed-dimensional polynomial certificate that has exactly one witness when the claimed run is valid?

Here “fixed-dimensional” means independent of the numerical duration of the run; the dimension may depend on the fixed program and on fixed degree bounds. The answer proved below is affirmative.

27.2 The precise restricted result

For $d \in \mathbb{N}$, put

$$p_{\mathbf{c}}(t) = \sum_{j=0}^d c_j t^j, \quad \mathbf{c} \in \mathbb{Z}^{d+1}, \quad T \in \mathbb{N}.$$

Coefficients may vanish, so d is a *nominal* degree bound, not necessarily the actual degree. Define

$$\text{PolyPos}_d(\mathbf{c}, T) \iff p_{\mathbf{c}}(i) \geq 0 \quad \text{for every integer } 0 \leq i \leq T.$$

The main theorem constructs an effective family of polynomials P_d with

$$\text{PolyPos}_d(\mathbf{c}, T) \iff \exists \mathbf{w} \in \mathbb{N}^{M_d} \quad P_d(\mathbf{c}, T, \mathbf{w}) = 0, \quad (119)$$

where $\deg P_d \leq 4$, $M_d = O((d+1)^3)$, and every true instance has *exactly one* complete witness $\mathbf{w}$.

The family is uniform in coefficients and horizon. It is not a single polynomial accepting an arbitrarily long coefficient list as one encoded input. Varying d changes the finite set of variables and the polynomial itself. This distinction prevents an unjustified leap from our theorem to arbitrary bounded arithmetic.

The main mathematical development proceeds in three stages. First, a finite tower of differences yields a unique short sign table. Second, a completely evaluated arithmetic and Boolean circuit checks this table, including its padding and all comparisons. Third, canonical signed-number and sign relations turn this circuit into one quartic equation. Only after uniqueness of the entire witness has been proved do we project away an execution time, and then only for a first-exit relation where that time is unique.

28 The finite-difference sign certificate [02]

28.1 Differences and sign runs

For $p \in \mathbb{Z}[t]$, define

$$\Delta p(t) = p(t+1) - p(t), \quad q_k = \Delta^k p \quad (0 \leq k \leq d).$$

Then $\deg q_k \leq d - k$, and q_d is constant. In ascending coefficient order, the difference transformation is explicit:

$$[t^j] \Delta p(t) = \sum_{r=j+1}^d \binom{r}{j} c_r. \quad (120)$$

Thus every coefficient of every q_k is a fixed integer linear combination of the input coefficients.

A *sign run* on $\{0, \dots, T\}$ is a maximal nonempty interval of consecutive integers on which the sign is constant. We encode the interval as $[a, b) \cap \mathbb{Z}$, with $0 \leq a < b \leq T+1$. The signs $-1, 0, 1$ are encoded by natural tags $0, 1, 2$, respectively.

Lemma 28.1 (A bound on the number of runs). *A real polynomial of degree at most m has at most $2m+1$ sign runs on any nonempty finite interval of consecutive integers. The zero polynomial has one run.*

Proof. Suppose the polynomial is nonzero. Let Z be the number of distinct integer zeros in the interval. Let C be the number of edges between consecutive sampled integers at which the signs are strictly opposite, with neither value zero. The intermediate value theorem puts a noninteger real root in the open interval of each such edge. These roots are distinct from one another and from the Z integer roots. Therefore $C + Z \leq m$.

Every change of sign tag occurs either at one of these C edges or at an edge incident with an integer zero. There are at most $2Z$ edges of the second kind. Thus the number of tag changes is at most $C + 2Z \leq 2m$. There is one more run than changes. Consecutive zero values cause no difficulty: edges joining two zeros are not changes, and our bound may only overcount them. $\square$

28.2 Canonical padding

Set

$$L_k = 2(d - k) + 1 \quad (0 \leq k \leq d).$$

Level k has L_k slots, with cuts and tags

$$b_{k,0}, \dots, b_{k,L_k}, \quad \tau_{k,0}, \dots, \tau_{k,L_k-1}.$$

The outside cuts are fixed:

$$b_{k,0} = 0, \quad b_{k,L_k} = T + 1.$$

All other cuts and all tags are natural primary witness coordinates. The following finite conditions specify the permitted shape:

$$b_{k,j} \leq b_{k,j+1}, \quad 0 \leq \tau_{k,j} \leq 2, \quad (121)$$

$$b_{k,j} = b_{k,j+1} \implies (b_{k,j} = T + 1 \wedge \tau_{k,j} = 1), \quad (122)$$

$$b_{k,j} < b_{k,j+1} < b_{k,j+2} \implies \tau_{k,j} \neq \tau_{k,j+1}. \quad (123)$$

The last condition is imposed only when the displayed indices exist. A slot is *active* when its two cuts are different. Condition (122) forces all inactive slots to be terminal padding; their tag is the fixed zero-sign tag. Condition (123) prohibits splitting a constant-sign run into two active slots. Every genuine maximal-run table therefore has exactly one padded encoding.

The count of primary coordinates is exact:

$$\sum_{k=0}^d ((L_k - 1) + L_k) = \sum_{m=0}^d (4m + 1) = (d + 1)(2d + 1). \quad (124)$$

The $2(d + 1)$ outside cuts are not counted, because they are fixed expressions rather than witnesses.

28.3 The local endpoint condition

The top level has one slot. Require

$$\tau_{d,0} = 1 + \text{sgn}(q_d(0)). \quad (125)$$

Now let $k < d$. Consider any active parent slot $[a, b) = [b_{k,j}, b_{k,j+1})$ and active child slot $[c, e) = [b_{k+1,\ell}, b_{k+1,\ell+1})$. Define

$$u = \max(a, c), \quad v = \min(b - 1, e). \quad (126)$$

Whenever $u \leq v$, require

$$1 + \text{sgn}(q_k(u)) = \tau_{k,j} = 1 + \text{sgn}(q_k(v)). \quad (127)$$

The child right endpoint in (126) is e , not $e - 1$. This is intentional: information about Δq_k on $[c, e)$ makes q_k monotone on the integer interval $[c, e]$. Using $e - 1$ would also cover the domain if handled consistently, but (126) gives a convenient closed monotonicity interval and is the convention used throughout the implementation.

All pairs of slots are considered. An implication disables the endpoint assertion for an inactive slot or empty intersection. Its arithmetic computations are nevertheless total, as in Lemma 2.8. Maxima and minima are implemented by comparison bits and selection gates, so these conditions are a fixed quantifier-free polynomial formula for fixed d .

Lemma 28.2 (Endpoint propagation). *Assume every child tag at level $k + 1$ is correct. If a parent slot satisfies (127) for every active child slot, then its tag is the actual sign tag of q_k at every integer point in that parent slot.*

Proof. On an active child interval $[c, e)$, the difference $q_k(i + 1) - q_k(i) = q_{k+1}(i)$ has a constant sign. Hence q_k is nondecreasing, nonincreasing, or constant on $[c, e] \cap \mathbb{Z}$. Its restriction to the intersection

$$[a, b - 1] \cap [c, e] \cap \mathbb{Z} = [u, v] \cap \mathbb{Z}$$

is also monotone. If both endpoint values are positive, every intervening value is positive; the analogous statement holds for negative values. If both endpoint values are zero, monotonicity forces every intervening value to be zero. Thus the parent tag is correct throughout each nonempty intersection.

Every integer t in the parent slot belongs to an active child slot, because the child slots partition $\{0, \dots, T\}$. For that child, $t \in [c, e - 1] \subseteq [c, e]$, so the relevant intersection is nonempty and contains t . This proves correctness at every point of the parent. $\square$

Theorem 28.3 (Existence and uniqueness of the sign tower). *For every $\mathbf{c} \in \mathbb{Z}^{d+1}$ and $T \in \mathbb{N}$, there is exactly one primary tuple satisfying (121)–(127). It is the canonically padded sequence of maximal sign-run tables of $q_0, \dots, q_d$ on $\{0, \dots, T\}$.*

Proof. For existence, take the actual maximal sign runs of each q_k . Lemma 28.1 provides at most L_k runs, so terminal padding is possible. The shape conditions hold by construction. The top difference is constant, so (125) holds. Each endpoint appearing in (127) is an integer point in the parent slot; its tag is therefore correct.

For soundness, start at the top level, whose unique active slot is correct by (125). Descend through $k = d - 1, \dots, 0$ and apply Lemma 28.2. Every active slot at every level is then a constant-sign interval with its true tag.

Finally, the shape conditions imply that the active slots form an ordered partition of the entire domain, and adjacent active slots have different tags. Such a partition must be the maximal sign-run partition: a boundary inside a constant-sign run is prohibited, and a missing boundary at a sign change would make a slot nonconstant. Terminal empty slots and their tags are prescribed by (122). Consequently every coordinate is uniquely determined. $\square$

Remark 28.4 (Degenerate cases are part of the theorem). The proof includes $T = 0$, the zero polynomial, a nominal degree larger than the actual degree, and runs consisting of one integer zero. No genericity, simple-root, nonzero-leading-coefficient, or sufficiently-large-horizon assumption is used. A zero polynomial at any level has one active zero-sign slot followed by uniquely fixed padding.

29 The single-fold quartic theorem and its cost [02]

Theorem 29.1 (Canonical bounded polynomial positivity). *There is an effective family of integer polynomials P_d , indexed by $d \in \mathbb{N}$, and an absolute constant C such that*

$$\deg P_d \leq 4, \quad M_d \leq C(d+1)^3,$$

and (119) is a single-fold representation for every signed coefficient tuple and natural horizon. Exactly $(d+1)(2d+1)$ primary coordinates suffice before auxiliary arithmetic and Boolean wires are introduced. Both the number of quadratic residuals and a bounded-arity straight-line description length are $O((d+1)^3)$.

Proof. Take the finite sign-tower conditions and add

$$b_{0,j} < b_{0,j+1} \implies \tau_{0,j} \neq 0 \quad (0 \leq j < L_0). \quad (128)$$

By Theorem 28.3, the primary tuple is unique before these conditions are added. They hold precisely when $q_0 = p$ has no negative value in the domain. Apply Lemma 2.8 to the complete quantifier-free checking formula. It gives one quartic equation, and its internal wires have a unique extension for every fixed primary tuple. Thus the complete existential tuple is unique whenever the positivity relation holds, and no tuple exists otherwise.

It remains to justify the cubic size bound, rather than merely asserting that the formula is finite. At a level where the parent degree bound is $m \geq 1$, there are $(2m+1)(2m-1) = 4m^2 - 1$ parent-child pairs. Their intersection and tag comparisons take boundedly many gates per pair. The potentially expensive operations are polynomial evaluations.

Do not evaluate q_k afresh at the selected endpoint of every pair. The possible left endpoints are parent starts and child starts; the possible right endpoints are parent stops minus one and child stops. There are $O(m)$ such candidate expressions. Evaluate q_k once at each candidate by Horner's rule, using $O(m)$ arithmetic gates per evaluation, and cache the sign tags. A comparison then selects the appropriate precomputed endpoint tag in $O(1)$ gates. The total cost at this level is $O(m^2)$.

The difference coefficient tower can be constructed in $O((d+1)^3)$ scalar arithmetic operations using (120). The shape tests use $O((d+1)^2)$ gates. Summing the $O(m^2)$ level costs therefore gives $O((d+1)^3)$ gates overall. The canonical circuit lemma introduces only a constant number of variables and residuals per gate. The exact primary count is (124). $\square$

29.1 An explicit specification, not only an existence proof

The preceding construction completely specifies P_d : unroll the stated slot conditions, evaluate the stated finite circuit, introduce the signed wire and sign equations, and sum the squared residuals. A factored sum-of-squares description is an exact description of an ordinary polynomial; expanding all monomials is unnecessary.

The accompanying compiler makes this description executable. Its exported JSON records every residual as a sparse polynomial with integer coefficients and named variables. The equation represented by the file is

$$P_d = Q_1^2 + \cdots + Q_N^2 = 0.$$

There is no exponentiation primitive, bounded quantifier, floor function, black-box Diophantine predicate, or implicit oracle left in this output. The only powers are the fixed squares and products appearing in the polynomial itself.

29.2 Generating the unique witness without scanning the horizon

Proposition 29.2 (Generation and verification complexity). *The sign tower can be generated using $O((d+1)^3 \log(T+2))$ exact arithmetic operations, and its canonical endpoint checker can be implemented using $O((d+1)^3)$ such operations. With dense binary integer coefficients and binary T , generation and verification have polynomial bit complexity in*

$$d+1, \quad \log(H+2), \quad \log(T+2), \quad H = \max_j |c_j|.$$

The arithmetic auxiliary witnesses also have polynomially bounded bit length in these quantities.

Proof. The constant top row is immediate. Suppose the child row has been constructed. On each child interval $[c, e)$, q_k is monotone. For a nondecreasing sequence, find the first index with value at least zero and the first index with value strictly greater than zero by binary search. These two indices split the interval into its negative, zero, and positive subintervals. Empty subintervals are discarded. For a nonincreasing sequence, apply the same procedure to $-q_k$ and reverse the sign labels. A constant sequence is included in the nondecreasing case. Merge adjacent subintervals with equal tags when passing from one child interval to the next.

There are $O(m)$ child intervals at a level of degree bound m . Each requires at most two binary searches, each with $O(\log(T+2))$ evaluations. Horner evaluation costs $O(m)$ arithmetic operations. This gives the generation bound after summing over the levels. The verification bound follows from the cached endpoint implementation in the proof of Theorem 29.1.

For bit complexity, the explicit identity

$$\Delta^k p(t) = \sum_{\ell=0}^k (-1)^{k-\ell} \binom{k}{\ell} p(t+\ell)$$

shows that the coefficient heights of all differences have bit length $O(\log(H+2) + d \log(d+2))$. One can also obtain this bound by expanding $(t+\ell)^r$ and bounding its binomial coefficients. Evaluation at an integer of magnitude at most $T+1$ adds $O(d \log(T+2))$ bits. All candidate cuts lie in $[0, T+1]$, signs and tags are bounded, and the sign-test slack is at most the magnitude of its argument. Polynomially many additions and multiplications on numbers with these polynomial bit bounds establish the claim using ordinary integer arithmetic. The same estimates apply to stored arithmetic outputs and comparison slacks. No assertion that unit-cost arithmetic is unit-cost *bit* computation is needed. $\square$

Remark 29.3 (Cubic size is an upper bound, not an optimum). The supplied compiler favors transparency and total evaluation over small constants. The theorem does not claim minimal witness count, optimal quartic sparsity, or an improved universal Diophantine equation. The specialized quartics should not be compared numerically with universal witness-count records as if they represented the same class of relations.

30 Boolean guards and empty execution prefixes [02]

Theorem 30.1 (Bounded Boolean polynomial guards). *Fix a Boolean formula Φ in signs of $r \geq 1$ polynomials $p_1(t), \dots, p_r(t)$, with respective fixed degree bounds $d_j \leq D$. The relation*

$$\forall i \in \{0, \dots, T\} \quad \Phi(\text{sgn } p_1(i), \dots, \text{sgn } p_r(i))$$

has an effective single-fold quartic representation, uniform in all coefficients and in T . A witness and circuit size bound is

$$O\left(r(D+1)^3 + r^2(D+1)^2 + |\Phi|r(D+1)\right).$$

A guard with no polynomial atoms is simply a constant Boolean guard and is handled directly.

Proof. Construct the unique difference sign tower for every p_j . Take the union of the starts of all active level-zero runs. It includes zero. Between two consecutive distinct starts in this union, every polynomial has constant sign, hence Φ has constant truth value.

It is unnecessary to existentially sort or deduplicate this union. For every active start of every polynomial, evaluate all r polynomials at that start and require Φ to be true there. Repeated starts merely repeat a true assertion and do not add choices. These tests are sufficient because every common sign cell begins at one of the tested starts; they are necessary because each tested start belongs to the domain.

There are $O(r(D+1))$ candidate starts. Evaluating r polynomials of degree at most D at each uses $O(r^2(D+1)^2)$ gates. Evaluating the Boolean formula at these starts uses $O(|\Phi|r(D+1))$ gates. The sign towers contribute $O(r(D+1)^3)$ gates. Every primary tower is unique and every extra evaluation is total and deterministic, so Lemma 2.8 gives the claimed single-fold quartic. $\square$

Strict comparisons, equations, disequalities, and arbitrary negations are all included. For example, a guard may be

$$(p_1 > 0 \wedge p_2 \neq 0) \vee (p_3 = 0 \wedge \neg(p_4 \geq 0)).$$

There is no monotonicity assumption on this entire guard or on p_j . Monotonicity is used only on the certified difference intervals.

30.1 Why a zero-length run requires explicit treatment

In execution semantics the usual condition is $\forall i < n \ G(i)$. For $n = 0$ this is true, even if $G(0)$ is false. Replacing it by a test on $[0, \max(n-1, 0)]$ without an additional condition would therefore be wrong. Use instead the uniquely determined number

$$h = \max(n-1, 0).$$

Always construct the full sign towers on $[0, h]$, but enable the final universal-guard assertions only when $n > 0$. The tables and their checking wires remain fully determined when $n = 0$; only the demand that the guard be true is disabled. The graph of this maximum is a quantifier-free piecewise-polynomial relation, and the circuit lemma supplies its unique arithmetic implementation. This method preserves both vacuity and single-foldness.

31 Accelerating polynomial trajectories [02]

Definition 31.1 (Polynomially iterable map). A fixed integer-state map $f : \mathbb{Z}^s \rightarrow \mathbb{Z}^s$ is *polynomially iterable* when there is a fixed vector $F(t, \mathbf{x}) \in \mathbb{Q}[t, \mathbf{x}]^s$ such that

$$F(0, \mathbf{x}) = \mathbf{x}, \quad F(t+1, \mathbf{x}) = f(F(t, \mathbf{x}))$$

for every natural t and integer $\mathbf{x}$. In particular, $F(t, \mathbf{x}) = f^t(\mathbf{x})$ is integer-valued at these inputs. The trajectory formula, not just its existence, is supplied as part of the fixed program specification.

Let $G(\mathbf{x})$ be a fixed Boolean combination of polynomial comparisons with integer coefficients. The loop tests G *before* performing each simultaneous update. Define

$$\text{Run}_f^G(\mathbf{x}, n, \mathbf{y}) \iff \mathbf{y} = F(n, \mathbf{x}) \wedge \forall i < n \ G(F(i, \mathbf{x})), \quad (129)$$

$$\text{Exit}_f^G(\mathbf{x}, n, \mathbf{y}) \iff \text{Run}_f^G(\mathbf{x}, n, \mathbf{y}) \wedge \neg G(\mathbf{y}). \quad (130)$$

The run relation allows stopping while the guard is still true; the exit relation does not. This semantic distinction is essential when an existential execution time is introduced.

Theorem 31.2 (Single-fold polynomial acceleration). *For every fixed polynomially iterable f and fixed polynomial Boolean guard G , both (129) and (130) have effective single-fold representations by ordinary integer quartics with finitely many natural witnesses. Their witness dimensions are independent of n and of the values of the initial and final states.*

Proof. Substitute $F(t, \mathbf{x})$ into each polynomial atom of G . The result is a polynomial in t whose coefficients are rational polynomials in $\mathbf{x}$. Multiply each atom's polynomial difference by a fixed positive common denominator. This preserves its sign and gives an integer polynomial in $(t, \mathbf{x})$. Its degree in t is fixed by f and G , not by n or by the state values.

Compute the coefficient values from $\mathbf{x}$ using a fixed arithmetic circuit and canonical signed wires. Apply Theorem 30.1 to these polynomials, with the empty-prefix convention from Section 30. Add the equations $\mathbf{y} = F(n, \mathbf{x})$ after clearing fixed denominators. These equations can be checked by the same canonical circuit lemma, so no polynomial degree increase survives the introduction of gate variables. For exit, add the Boolean test $\neg G(\mathbf{y})$.

Coefficient computations and endpoint computations are unique. The primary sign tables are unique. All their auxiliary wires are unique. Consequently the complete witness is unique for each valid $(\mathbf{x}, n, \mathbf{y})$. Every invalid triple fails at least one of the equivalent checking conditions. Finally sum the squared quadratic residuals. $\square$

[write] Priority words. Part XII treats fixed words under first-enabled-rule priority. For a fixed program its block relation is an instance of Theorem 31.2, with $f(x) = x + D$ and a guard of linear atoms; the new content there is uniformity in the program entries, the capacity formula and the sharp threshold $2H + 1$ (Theorems 97.1 and 98.6).

Corollary 31.3 (Projecting the first-exit time). *The halting domain*

$$H_f^G(\mathbf{x}) \iff \exists n \in \mathbb{N} \exists \mathbf{y} \in \mathbb{Z}^s \text{Exit}_f^G(\mathbf{x}, n, \mathbf{y})$$

and the partial output graph

$$O_f^G(\mathbf{x}, \mathbf{y}) \iff \exists n \in \mathbb{N} \text{Exit}_f^G(\mathbf{x}, n, \mathbf{y})$$

have effective single-fold quartic representations with natural witnesses.

Proof. For a fixed initial state, there is at most one first-exit time. Indeed, if $n < m$ were two such times, the exit at n would give $\neg G(F(n, \mathbf{x}))$, while the valid prefix for m would give $G(F(n, \mathbf{x}))$. The final state is then uniquely $F(n, \mathbf{x})$. Thus existentially projecting these quantities cannot introduce additional choices. Represent every projected signed state coordinate by the normalized pair of Lemma 2.6; merely using an unrestricted difference of two naturals would be insufficient. The existing certificate for the unique exit triple has a unique extension. The added normalization equations are quadratic. $\square$

Remark 31.4 (Reachability is not automatically single-fold). Replacing Exit by Run in this corollary would be unjustified. For the identity update and the guard *true*, the same state is reachable at every duration. Even a perfect single-fold certificate for each fixed n then gives infinitely many witnesses after n is projected away. The theorem does not rule out a different single-fold representation of that trivial relation; it identifies why this particular projection argument would fail.

32 Computational substrates covered by the theorem [02]

32.1 Unipotent affine state transformations

Consider

$$f(\mathbf{x}) = A\mathbf{x} + \mathbf{b}, \quad A \in M_s(\mathbb{Z}), \quad \mathbf{b} \in \mathbb{Z}^s.$$

Introduce the augmented matrix

$$M = \begin{pmatrix} A & \mathbf{b} \\ 0 & 1 \end{pmatrix}, \quad \tilde{\mathbf{x}} = (\mathbf{x}, 1)^\top.$$

Then $M^t \tilde{\mathbf{x}}$ is the augmented state at time t .

Proposition 32.1 (Unipotent affine acceleration). *Suppose $(M - I)^r = 0$ for a fixed $r \geq 1$. Then*

$$M^t = \sum_{j=0}^{r-1} \binom{t}{j} (M - I)^j \quad (t \in \mathbb{N}). \quad (131)$$

Consequently this affine update is polynomially iterable and every fixed polynomial Boolean guard admits the single-fold execution, first-exit, and output representations above.

Proof. Expand $(I + (M - I))^t$ by the finite binomial theorem. All powers of $M - I$ of exponent at least r vanish. For a fixed j , the expression

$$\binom{t}{j} = \frac{t(t-1) \cdots (t-j+1)}{j!}$$

is a rational polynomial in t , with the usual value 1 for $j = 0$. For natural $t < j$ it is zero, so (131) also covers small t . Matrix multiplication by the initial augmented state gives the required polynomial trajectory. The identities are valid over integers after clearing fixed denominators. $\square$

This includes translations, counter transfers with nilpotent coupling, and accumulation chains. State coordinates may increase or decrease; nonnegativity requirements for a natural-counter interpretation can be inserted as guard atoms. There is no assumption that the full state or the guard value is monotone.

32.2 Unitriangular nonlinear updates

Consider simultaneous assignments

$$\begin{aligned} x'_1 &= x_1 + c, \\ x'_i &= x_i + R_i(x_1, \dots, x_{i-1}) \quad (2 \leq i \leq s), \end{aligned} \quad (132)$$

where $c \in \mathbb{Z}$ and every R_i is an integer polynomial. The word *simultaneous* means that all right-hand sides use the old state.

Lemma 32.2 (Discrete antidifferentiation). *For every $q(t) \in K[t]$ over a characteristic-zero field K , there is a unique polynomial $S(t) \in K[t]$ satisfying*

$$S(0) = 0, \quad S(t+1) - S(t) = q(t).$$

It has degree at most $\deg q + 1$ when $q \neq 0$.

Proof. The polynomials $\binom{t}{j}$, $j \geq 0$, form a basis of $K[t]$, since they have distinct degrees and nonzero leading coefficients. Write $q(t) = \sum_j \alpha_j \binom{t}{j}$ and set $S(t) = \sum_j \alpha_j \binom{t}{j+1}$. Pascal's identity proves the difference equation and $S(0) = 0$. If two such polynomials existed, their difference would have zero forward difference; a nonconstant polynomial in characteristic zero cannot have that property, because taking a difference lowers its degree by exactly one. Thus their difference is a constant, and its value at zero is zero. $\square$

Theorem 32.3 (Nonlinear unitriangular substrates). *Every map (132) is polynomially iterable. Therefore its exact guarded execution, first exit, halting domain, and partial output graph admit the single-fold quartic representations of Theorem 31.2 and Corollary 31.3.*

Proof. The first coordinate is $F_1(t, \mathbf{x}) = x_1 + ct$. Suppose the trajectories of the first $i - 1$ coordinates have already been expressed as rational polynomials in $(t, \mathbf{x})$. Then

$$R_i(F_1(t, \mathbf{x}), \dots, F_{i-1}(t, \mathbf{x}))$$

is another such polynomial. Apply discrete antidifferentiation in the variable t , treating the initial coordinates as coefficients, and add x_i to obtain F_i . The resulting polynomial satisfies exactly the update recurrence and the initial condition. Induction on the integer time then shows that its values coincide with the actual integer computation, so no separate integer-valuedness assumption is needed. Continue through all coordinates. $\square$

Degree growth in this construction can be substantial. For example, if $\deg_t F_j \leq D_j$ for $j < i$, the degree in t of F_i is bounded by one plus the largest weighted degree $\sum_{j < i} \alpha_j D_j$ among monomials $x_1^{\alpha_1} \cdots x_{i-1}^{\alpha_{i-1}}$ of R_i , with the initial constant added. The resulting certificate is fixed-dimensional for a fixed program, but the theorem does not promise a small polynomial bound in the source-code length of an arbitrary nested triangular expression.

32.3 A certified eventually quasi-polynomial affine extension

Polynomial trajectories exclude sign-alternating linear modes. A fixed residue decomposition handles a wider class without introducing variable exponentiation.

Theorem 32.4 (Eventually quasi-polynomial affine acceleration). *Let M be the augmented integer matrix of an affine update. Suppose fixed integers $h \geq 0$, $p \geq 1$, and $r \geq 1$ are supplied and satisfy*

$$M^h(M^p - I)^r = 0. \quad (133)$$

For every fixed polynomial Boolean guard, exact guarded execution and first exit have single-fold quartic representations of fixed dimension. Their halting domains and partial output graphs are also single-fold.

Proof. For $0 \leq s < p$ and $k \in \mathbb{N}$, all the matrices involved commute, and the binomial expansion gives

$$M^{h+s+pk} = \sum_{j=0}^{r-1} \binom{k}{j} M^{h+s} (M^p - I)^j. \quad (134)$$

Terms of exponent at least r vanish by (133). Thus the trajectory is a polynomial in k on each of the finitely many residue classes after the fixed prefix of length h .

In a claimed run of length n , the prefix tests at times $i < h$ are required only when $i < n$. In residue class s , the relevant times are $h + s + pk < n$. Their number is the uniquely determined natural number

$$N_s = \left\lceil \frac{\max(n - h - s, 0)}{p} \right\rceil.$$

The graph of this expression has a canonical polynomial certificate: compute the positive part by a sign test, divide its sum with $p - 1$ by the fixed positive integer p , and impose the unique Euclidean remainder condition $0 \leq u < p$. Comparisons are converted using the circuit lemma. This determines N_s and all its auxiliary values uniquely, including when $N_s = 0$.

Substituting (134) into the guard gives a polynomial Boolean guard in k . Apply the bounded guard theorem on the prefix $0 \leq k < N_s$ for each s , with the zero-length convention already proved. The terminal state is selected from the fixed prefix or the unique residue decomposition of $n - h$. These are finite, deterministic piecewise-polynomial computations; every inactive branch is evaluated on uniquely prescribed arguments or explicitly padded by zero. Thus all auxiliaries are unique. The circuit lemma again reduces the finite combined system to a quartic. First exit and its projections follow by the same uniqueness argument as before. $\square$

Condition (133) is an exact integer matrix identity that can be checked directly. It is sufficient; this paper does not require an algorithm for finding optimal h, p, r or a spectral classification. Taking $p = 2$ accommodates appropriate alternating modes, while nilpotent transient modes can be absorbed into h . The period is fixed as part of the substrate certificate, not an unbounded existential choice.

32.4 Composition of deterministic phases

Proposition 32.5 (Single-fold composition of partial functions). *For fixed integer or natural tuple domains A, B, C , if the graphs of partial functions $f : A \rightarrow B$ and $g : B \rightarrow C$ have single-fold polynomial representations, then so does the graph of $g \circ f$. It can be presented by a quartic after introducing canonical circuit variables.*

Proof. Use the conjunction of the two graph representations and existentially quantify the intermediate value b . For fixed input a , functionality of f makes b unique if it exists. The two auxiliary tuples are then unique by hypothesis. Encode signed components of b canonically. Flatten the finite polynomial expressions into arithmetic gates and sum squared quadratic residuals. $\square$

A fixed sequence of terminating phases from our loop classes therefore inherits single-foldness. This is not closure under arbitrary recursive calls or a variable number of phases. A deterministic branch controlled by a polynomial test can also be handled, provided every inactive certificate tuple is explicitly normalized. Unrestricted nondeterministic choice requires a separate multiplicity analysis.

33 Canonical nontermination certificates and input bounds [02]

The exact-run theorem treats a supplied duration. For polynomial trajectories, a canonical stabilization bound also handles all future times without leaving ordinary polynomial arithmetic.

Lemma 33.1 (An elementary canonical root bound). *Let $q(t) = \sum_{j=0}^m a_j t^j$ be an integer polynomial, with zero leading coefficients permitted in the displayed list. Set*

$$B_q = 2 + \sum_{j=0}^m |a_j|.$$

If q is nonzero, it has no real root $t \geq B_q$. Its sign is constant on $[B_q, \infty)$. The zero polynomial has constant zero sign there.

Proof. Constant polynomials require no argument. Otherwise discard zero leading coefficients and let $a_m \neq 0, m \geq 1$. Since a_m is a nonzero integer, $|a_m| \geq 1$. Put $A = \max_{j < m} |a_j/a_m| \leq \sum_j |a_j|$. For a real $t > 1 + A$, the finite geometric sum gives

$$\left| \sum_{j < m} a_j t^j \right| \leq |a_m| A \frac{t^m - 1}{t - 1} < |a_m| t^m.$$

Thus the lower terms cannot cancel the leading term, and $q(t)$ has the sign of a_m . Every $t \geq B_q$ satisfies the strict inequality needed above. The extra constant 2 removes any endpoint ambiguity. $\square$

Theorem 33.2 (Specified-input nontermination is single-fold). *Let f, F, G be as in Theorem 31.2. The relation*

$$\text{Nonterm}_f^G(\mathbf{x}) \iff G(F(t, \mathbf{x})) \text{ holds for every } t \in \mathbb{N}$$

has an effective single-fold quartic representation. Moreover, an explicit canonical bound $B(\mathbf{x})$ can be computed such that every terminating input first exits at some time at most $B(\mathbf{x})$.

Proof. After clearing fixed positive denominators as before, write the polynomials of the guard along the trajectory as

$$q_j(t, \mathbf{x}) = \sum_{\ell=0}^{D_j} a_{j\ell}(\mathbf{x})t^\ell, \quad a_{j\ell} \in \mathbb{Z}[\mathbf{x}].$$

Define the exact, not merely upper-bounded, number

$$B(\mathbf{x}) = 2 + \sum_{j,\ell} |a_{j\ell}(\mathbf{x})|. \quad (135)$$

For each specified input, Lemma 33.1 makes every atom's sign constant from time $B(\mathbf{x})$ onward. The Boolean combination G is therefore constant from that time onward as well. It follows that

$$\text{Nonterm}_f^G(\mathbf{x}) \iff \forall 0 \leq t \leq B(\mathbf{x}) \ G(F(t, \mathbf{x})). \quad (136)$$

Indeed, if the right-hand side holds, its truth at B extends to the entire tail; the converse is immediate. If the loop exits at all, some false guard value must consequently occur no later than B .

The coefficient values and their absolute values are computed uniquely using canonical signed pairs. Equation (135) determines B uniquely. Apply the bounded Boolean guard theorem to (136). Its primary tables and all auxiliary arithmetic values are unique, so the complete nontermination certificate is single-fold. Flattening the finite combined circuit gives a quartic. $\square$

Corollary 33.3 (Bit-efficient specified-input classification). *For a fixed polynomial trajectory and fixed polynomial Boolean guard, termination or nontermination from a binary-encoded integer initial state can be decided in polynomial time in that state's bit length. The corresponding canonical certificates have polynomial bit length. When the loop terminates, its first-exit time and output can be produced within the same polynomial-time bound.*

Proof. Let $H = \max(1, \|\mathbf{x}\|_\infty)$. Since the program and the coefficient polynomials are fixed, there are fixed constants C, K with $B(\mathbf{x}) \leq C(H+1)^K$. Coefficient computation therefore uses polynomial bit time and yields $O(\log(H+2))$ -bit values, with constants allowed to depend on the program.

Generate the fixed-degree sign towers on $[0, B]$ by Proposition 29.2. Their running time is polynomial in $\log(H+2)$. Inspect the finitely many common sign-cell starts used in Theorem 30.1. If all guard values are true, the loop does not terminate. Otherwise the smallest false cell start is exactly the first exit time. Evaluate the fixed polynomial trajectory there to obtain the output. The same magnitude estimates bound the complete certificate's bit length. $\square$

This is an acceleration statement about binary arithmetic, not a bound on the literal number of loop iterations. A loop can run for a very large number of steps and still have an efficiently produced compressed certificate. The constants in the complexity bound depend on the fixed trajectory and its degrees; this corollary is not a polynomial bound for arbitrary programs presented as input.

33.1 Why this does not decide global termination

The quantifier over the initial state is crucial. For an arbitrary integer polynomial $Q(x)$, consider the identity update with guard $Q(x) = 0$. There is a nonterminating initial state exactly when Q has an integer zero. With natural initial states, the same example expresses natural solvability. Hence a uniform procedure deciding whether *every* initial state terminates for all such supplied loops would decide a Hilbert-tenth solvability problem. Ordinary MRDP supplies the undecidability background [14].

Our theorem assigns a finite single-fold certificate to each specified initial state. It does not decide the existential assertion that some state has a nontermination certificate. That assertion is itself a Diophantine solvability problem. Thus the polynomial-time specified-input corollary is compatible with the broader termination distinctions in the triangular-loop literature [25, 26].

[write] Batch 63, 30 September 2026: the quantifier over infinite behaviour is the subject of the neighbouring report *liveness-beyond-halting* of this category (Report C, written in commit 2a34b1740 from batch-62 manuscripts 10 and 03 and batch-63 manuscript 01, its sources 10, 03 and 11). Nontermination from one specified input is the existence of an infinite run, a Π_1^0 property of a deterministic system in general; the certificate above makes it finitely certifiable for fixed polynomial trajectories, while Report C shows which recurrence and liveness properties of its fixed universal systems admit no finite certificate (its quantifier-sensitive no-go theorem, 1bh : thm : nogo).

34 Worked examples [02]

34.1 A guard that fails only in the interior

Let

$$p(t) = (t - 2)(t - 4) = t^2 - 6t + 8, \quad T = 6.$$

Its values are

$$(p(0), \dots, p(6)) = (8, 3, 0, -1, 0, 3, 8).$$

Both outer endpoint values are positive, but bounded nonnegativity is false. The difference tower is

$$q_0 = t^2 - 6t + 8, \quad q_1 = 2t - 5, \quad q_2 = 2.$$

The actual active runs are shown in Table 3. Their tags are one plus the signs shown in the table.

Level	Active half-open sign intervals on $\{0, \dots, 6\}$
q_0	$[0, 2) : +, [2, 3) : 0, [3, 4) : -, [4, 5) : 0, [5, 7) : +$
q_1	$[0, 3) : -, [3, 7) : +$
q_2	$[0, 7) : +$

Table 3: A unique short certificate exposing a negative interior value.

The nominal level capacities are 5, 3, 1. Thus q_1 gets one inactive slot $[7, 7)$ with tag 1, while the other levels need no padding. The sign-table certificate is valid, but the additional positivity assertion rejects the negative level-zero run.

For the loop $x' = x + 1$ with guard $(x - 2)(x - 4) \geq 0$, started at $x = 0$, the first exit is $n = 3$, $x = 3$. A procedure checking only the guard at the start and at time 6 would incorrectly infer that the entire prefix was valid. The sign-table method does not make that inference.

34.2 A nonlinear accumulation chain

Consider the simultaneous update

$$x' = x + 1, \quad y' = y + x, \quad z' = z + y^2, \quad (137)$$

with initial state $(a, b, c) \in \mathbb{Z}^3$. Its exact trajectory is

$$\begin{aligned} x_t &= a + t, \\ y_t &= b + at + \binom{t}{2}, \\ z_t &= c + b^2t + (2ab + a^2)\binom{t}{2} \\ &\quad + (2a^2 + 2b + 4a + 1)\binom{t}{3} + (6a + 6)\binom{t}{4} + 6\binom{t}{5}. \end{aligned} \quad (138)$$

To verify the last formula, write $y_j = b + aj + \binom{j}{2}$ and use

$$\begin{aligned} j^2 &= \binom{j}{1} + 2\binom{j}{2}, \\ j\binom{j}{2} &= 2\binom{j}{2} + 3\binom{j}{3}, \\ \binom{j}{2}^2 &= \binom{j}{2} + 6\binom{j}{3} + 6\binom{j}{4}. \end{aligned}$$

Expanding y_j^2 in this basis gives the coefficients in (138) after summation with $\sum_{j=0}^{t-1} \binom{j}{k} = \binom{t}{k+1}$. This is a symbolic proof; the accompanying tests also compare the formula with 2,500 explicitly iterated states.

An example guard is

$$(z^2 - 3xy + \lambda \geq 0) \wedge (y \neq \mu),$$

where λ, μ are fixed parameters or additional unchanged input coordinates. Along the trajectory, the first atom has degree at most ten in t , and the second has degree at most two. The bounded Boolean theorem therefore supplies a fixed-dimensional certificate for any duration. The result does not depend on the first atom being monotone; it may pass through several sign intervals.

34.3 An enormous horizon with a small certificate

The test suite includes

$$A = 10^{50}, \quad T = 10^{100}, \quad p(t) = (t - A)^2.$$

The three difference levels have respectively three, two, and one active sign runs. Their total active run count is six. The supplied generator used 832 distinct exact polynomial evaluations to obtain this certificate; the horizon was not enumerated. The degree-two quartic compiler then verified its generated natural witness against every quadratic residual.

The numbers 10^{50} and 10^{100} are simply concrete test inputs, not new primitive operations in the output equation. The compiled polynomial is the same degree-two template used for small coefficients and horizons; only its free parameter assignment changes.

35 Manuscript 02: Conclusion [02]

A long computation need not be represented by a long explicit trace. For polynomial trajectories, the signs of a guard can be certified by a unique hierarchy of maximal sign intervals of successive finite differences. The hierarchy is finite because degree decreases; its correctness is local because differences control monotonicity; and its uniqueness is global because all runs are maximal and all padding is fixed.

Combined with total canonical arithmetic circuits, this yields an ordinary quartic equation with a unique natural witness and a dimension independent of the run duration. It applies to genuinely nonlinear unitriangular dynamics, not just monotone linear counters. First-exit semantics permits safe projection of time, and a canonical polynomial root bound yields unique specified-input nontermination certificates.

The construction gives a concrete addition to a research program in which ordinary Diophantine reachability is already established: a multiplicity-preserving accelerator with an executable polynomial output. Its exact scope, proof obligations, and exponential boundary are explicit.

Part V

Memory logs, RAM and graph evaluation

Source and scope. [write] This Part is manuscript 03, “Linear-Size Canonical Diophantine Certificates for Random-Access and Graph Computation”: its Sections 2–9 (canonical comparison and zero-test gadgets, a sorting network as a canonical polynomial circuit, the canonical random-access memory theorem, size and witness height, the linear-size compiler by exact large-base products, complete RAM executions, applications to counters, cellular automata, stacks, tapes and an immutable SK/SKI/Iota graph evaluator, and exact counting and first-halting probabilities), and its conclusion. Its conventions are in Section 2, its introduction in Section 3, its boundary theorem in Part IX.

Setting and hypotheses. [write] A log of length L records reads and writes of a random-access memory with natural addresses and values; N is the number of padded records and C the number of comparators of the sorting network. “Canonical certificate” means a single-fold representation (Section 2); “witness height” bounds the magnitude of witness coordinates. The radix packing of Section 40 is the device that Part VI uses for queue words (Lemma 45.1).

36 Canonical arithmetic with quadratic residuals [03]

36.1 Comparison in two equations

The elementary observation doing most of the work is a signed-gap identity.

Lemma 36.1 (Canonical comparison). *For every $x, y \in \mathbb{N}$, the system in $b, d \in \mathbb{N}$*

$$b(b-1) = 0, \tag{139}$$

$$y - x - (2b-1)d - b + 1 = 0 \tag{140}$$

has exactly one solution. It is

$$(b, d) = \begin{cases} (1, y-x), & x \leq y, \\ (0, x-y-1), & x > y. \end{cases} \tag{141}$$

Thus b is the indicator of $x \leq y$, with equality assigned to the first branch, and $0 \leq d \leq \max(x, y)$.

Proof. Equation (139) forces $b = 0$ or $b = 1$. If $b = 1$, (140) becomes $y - x = d$, possible precisely when $x \leq y$, and then $d = y - x$ is forced. If $b = 0$, it becomes $x - y = d + 1$, possible precisely when $x > y$, and then $d = x - y - 1$ is forced. The two cases are disjoint and exhaustive. The stated bound follows in either case. $\square$

The strict offset 1 is important. Without it, a tie could choose either branch. Equally importantly, there is only one slack coordinate: no unused branch slack survives as an arbitrary witness. A naive disjunctive encoding with two unnormalised slacks would generally have many auxiliary solutions.

Example 36.2. For $(x, y) = (3, 7)$, the unique witness is $(b, d) = (1, 4)$. For $(7, 3)$ it is $(0, 3)$, not $(0, 4)$, because the reverse branch uses a strict inequality. For $(3, 3)$ it is $(1, 0)$. The formulas include zero without a separate convention.

36.2 Zero tests and conditional choice

Putting $(x, y) = (\delta, 0)$ in Lemma 36.1 gives the following useful form.

Corollary 36.3 (Canonical zero test). *For $\delta \in \mathbb{N}$, the two equations*

$$z(z - 1) = 0, \quad \delta + (2z - 1)h + z - 1 = 0 \quad (142)$$

have the unique natural solution $(z, h) = (1, 0)$ when $\delta = 0$, and $(z, h) = (0, \delta - 1)$ when $\delta > 0$.

The nonnegativity assumption on δ is part of this corollary. When we use it on an address difference, that assumption will follow from the sorting proof, not from an unproved premise in the polynomial system.

For a Boolean b and natural inputs x, y , the residual

$$u - bx - (1 - b)y = 0 \quad (143)$$

uniquely specifies a natural output. Addition, multiplication, copying, and fixed constants are likewise represented by their defining equations. Boolean AND is multiplication, NOT is $1 - b$, and OR can be represented by $u - b - c + bc = 0$. The latter is nonnegative for Boolean inputs.

Natural truncated subtraction is also canonical without allowing a negative intermediate. Compare x with y using (b, d) , and set

$$u - (1 - b)(d + 1) = 0. \quad (144)$$

Then $u = \max(x - y, 0)$. For fixed $B \geq 2$, Euclidean division is specified by

$$x - Bq - r = 0, \quad r + h + 1 - B = 0, \quad (145)$$

with $q, r, h \in \mathbb{N}$. The second equation says $r < B$ and fixes h ; division with remainder proves uniqueness. No multiplicity is introduced by the bound.

36.3 Composing canonical gadgets

Lemma 36.4 (Acyclic composition). *Suppose a finite directed acyclic circuit is built from gadgets whose output and auxiliary coordinates are uniquely determined by their inputs. Use fresh coordinates for different gadgets and identify only prescribed connecting wires. Then the combined residual system has a unique assignment to every internal coordinate for each external input. Adding predicates on the final outputs either keeps that assignment or removes it; it never creates another.*

Proof. Order the gadgets topologically. The first gadget has fixed inputs, so its output and auxiliaries are forced. The same statement then holds for the next gadget, and induction forces the entire assignment. Conversely, computing the gadgets in that order constructs an assignment satisfying their residuals. Additional final predicates merely test the already determined outputs. $\square$

This elementary lemma is a stronger contract than closure of Diophantine relations under existential quantification. It retains the size of each fiber. It applies to acyclic certificate circuits, not to an arbitrary cyclic system of equations. In the RAM theorem below, temporal memory correctness supplies a separate induction for the apparent dependence of future states on read values.

37 Deterministic sorting as a canonical polynomial circuit [03]

37.1 One four-field compare-exchange

A record has four natural coordinates

$$X = (\alpha, \kappa, w, v), \quad (146)$$

where κ is the sorting key. Given two records X, Y , apply Lemma 36.1 to their keys, obtaining b, d . For each field $f \in \{\alpha, \kappa, w, v\}$ introduce outputs ℓ_f, u_f and require

$$\ell_f - bX_f - (1 - b)Y_f = 0, \quad (147)$$

$$u_f - X_f - Y_f + \ell_f = 0. \quad (148)$$

The first output record has the smaller key, with the left record first at a tie; the second contains the other record. Descending comparators simply interchange the two output ports. No additional variable is necessary for that change of wiring.

Proposition 37.1 (Comparator cost). *The four-field comparator introduces exactly ten natural variables and ten residual equations, all of degree at most two. Its complete auxiliary and output assignment is unique for every pair of input records. Its outputs are exactly those records, not merely records with the same keys.*

Proof. There are two variables and two equations for (b, d) , and eight output variables with eight equations for the fields. The key comparison forces (b, d) ; (147) forces each lower field, and (148) forces each upper field. In either Boolean case these are the two original records in the required order. $\square$

Consequently, a fixed sorting network with C comparators becomes a canonical arithmetic circuit with exactly $10C$ new coordinates and $10C$ residuals. Unlike existential permutation routing, there is no choice of internal switch settings once the input records are fixed.

37.2 A completely explicit network

For $N = 2^k$, use the iterative bitonic network whose complete generator is

```

for block in 2, 4, ..., N:
  for stride in block/2, block/4, ..., 1:
    for i in 0, ..., N-1:
      j = i xor stride
      if j > i:
        compare wires i and j
        ascending exactly when (i bitwise-and block) == 0

```

All operations in this generator act on fixed wire indices at compilation time. No bitwise arithmetic is needed inside the Diophantine equation.

For completeness, here is the mathematical sorting argument behind the network. A finite sequence is bitonic when some cyclic rotation is nondecreasing and then nonincreasing. Comparing positions separated by half the length, and sending the smaller entries to the first half, is a bitonic split. Both resulting halves are bitonic, and every entry of the first half is at most every entry of the second half.

To verify this split property rigorously, threshold all entries at an arbitrary value. A bitonic zero–one sequence has a single circular interval of ones. If that interval has length at most half the total length, no opposite pair contains two ones, so the lower half is all zero; the upper half has a circular interval of ones. If the interval is longer, every opposite pair contains a one, so the upper half is all one and the lower half again has a circular interval of ones. This proves the split property for every threshold. Comparison commutes with thresholding. Any failure of separation or bitonicity in the original total order would appear at a threshold between offending values. Thus the split property holds in the original order as well.

Recursively splitting each half sorts a bitonic sequence. To sort an arbitrary sequence, sort one half ascending and the other descending, producing a bitonic sequence, and merge. The displayed iterative generator performs precisely these merges on successively doubled blocks, with the alternating block orientations needed for the next merge. Its final block is ascending.

Lemma 37.2 (Exact bitonic count). *For $N = 2^k$, the displayed network sorts arbitrary totally ordered keys and has*

$$C_N = \frac{N}{4}k(k+1) \quad (149)$$

comparators, with $C_1 = 0$.

Proof. Correctness was established above. At block size 2^r there are r stride layers, each containing $N/2$ disjoint comparisons. Therefore $C_N = (N/2) \sum_{r=1}^k r = Nk(k+1)/4$. $\square$

The proof also explains why duplicate keys cause no problem. The comparator’s left-biased tie convention makes its internal variables unique, while sorting correctness only requires a nondecreasing output. In the memory application, timestamps will in fact make all keys distinct.

38 The canonical random-access memory theorem [03]

38.1 The semantic relation

A finite chronological log is

$$E = ((a_t, w_t, v_t))_{0 \leq t < L}, \quad a_t, v_t \in \mathbb{N}, \quad w_t \in \{0, 1\}. \quad (150)$$

The memory is initially zero at every address. A write event ($w_t = 1$) stores v_t at address a_t . A read event ($w_t = 0$) asserts that the current value at a_t is v_t and does not change memory. Write values are unrestricted natural numbers; reads before any write must return zero. Let $\text{Valid}_L(E)$ mean that all events obey this semantics. Non-Boolean write flags make the log invalid.

We now construct one polynomial for each L , treating the $3L$ event coordinates as free parameters. The polynomial’s coefficients do not depend on the numerical values of those parameters.

38.2 Padding and chronological keys

Set

$$N = 2^{\lceil \log_2 \max(1, L) \rceil}, \quad B = N + 1. \quad (151)$$

For each genuine event introduce

$$\alpha_t = a_t + 1, \quad \kappa_t = B\alpha_t + t, \quad (152)$$

and the record $(\alpha_t, \kappa_t, w_t, v_t)$. The two definitions are enforced by linear residuals; also include $w_t(w_t - 1) = 0$. For each $L \leq t < N$, append the fixed dummy record

$$(\alpha_t, \kappa_t, w_t, v_t) = (0, t, 1, 0). \quad (153)$$

These records are constants, not additional existential choices. They are zero writes at a reserved shifted address. No genuine event has that shifted address, so padding cannot affect the original memory.

Lemma 38.1 (Key order). *The keys of all N records are distinct. Sorting by κ first sorts by shifted address α and, within an address, by original timestamp t . All dummy records precede all genuine records.*

Proof. All timestamps lie in $[0, N - 1]$. If $\alpha < \alpha'$, then

$$B\alpha + t \leq B\alpha + N - 1 < B(\alpha + 1) \leq B\alpha' + t'.$$

At equal addresses, the key comparison is the timestamp comparison. Distinct events have distinct timestamps, proving distinctness. Dummy addresses are zero and genuine shifted addresses are positive. $\square$

Apply any fixed sorting network on N wires using the four-field comparator. Write the sorted records as

$$(\alpha_j^*, \kappa_j^*, w_j^*, v_j^*) \quad (0 \leq j < N). \quad (154)$$

By the preceding lemmas, all these records and all internal sorting coordinates are uniquely determined by E .

38.3 Four equations per adjacent pair

For the first sorted record require

$$(1 - w_0^*)v_0^* = 0. \quad (155)$$

For each $1 \leq j < N$, the difference $\delta_j = \alpha_j^* - \alpha_{j-1}^*$ is nonnegative by Lemma 38.1. Introduce three natural variables z_j, h_j, u_j , and the following four residuals:

$$z_j(z_j - 1) = 0, \quad (156)$$

$$\alpha_j^* - \alpha_{j-1}^* + (2z_j - 1)h_j + z_j - 1 = 0, \quad (157)$$

$$u_j - z_j v_{j-1}^* = 0, \quad (158)$$

$$(1 - w_j^*)(v_j^* - u_j) = 0. \quad (159)$$

The first two are the canonical zero test of δ_j . Consequently $z_j = 1$ exactly when the addresses agree; h_j is the unique corresponding gap witness. The third equation supplies the previous value when the address is unchanged, and zero otherwise. The fourth checks a read but puts no condition on the value of a write.

Introducing u_j is necessary for our degree bound: writing $(1 - w_j^*)(v_j^* - z_j v_{j-1}^*) = 0$ directly would create a cubic residual and a sixth-degree square. The new coordinate is uniquely defined and therefore does not spoil parsimony.

Lemma 38.2 (Sorted memory criterion). *For sorted records arising from (152)–(153), (155)–(159) have a natural solution exactly when the original log is valid. When a solution exists, all their auxiliary coordinates are unique.*

Proof. Consider a maximal consecutive block of one shifted address. At its first record, the value supplied as the predecessor is zero: this follows either from (155), or from $z_j = 0$ and (158). Thus its first read must return zero, while its first write may set any value.

At every subsequent record of that block, $z_j = 1$ and $u_j = v_{j-1}^*$. A read is therefore required to agree with the preceding record's value; a write may replace it. Inductively, the value in every record is the memory contents just after that event. A read leaves that contents unchanged, and a write changes it to its stated value. Because timestamp order is preserved within the block, this is precisely the chronological memory semantics at that address.

Different addresses do not interact, and dummy zero writes are harmless. This proves equivalence with the original log. For uniqueness, the sorted addresses and values fix each z_j, h_j by Corollary 36.3, then fix u_j by (158). The read equations are only tests of those forced values. $\square$

38.4 The main theorem and its exact constants

Theorem 38.3 (Canonical quartic memory compiler). *Fix $L \geq 0$ and N as in (151). Let a sorting network on N wires have C comparators. One can explicitly construct*

$$P_L \in \mathbb{Z}[a_0, w_0, v_0, \dots, a_{L-1}, w_{L-1}, v_{L-1}, \boldsymbol{\eta}]$$

of degree at most four such that, for every natural event-parameter tuple E ,

$$\#\{\boldsymbol{\eta} \in \mathbb{N}^W : P_L(E, \boldsymbol{\eta}) = 0\} = \begin{cases} 1, & \text{Valid}_L(E), \\ 0, & \text{otherwise.} \end{cases} \quad (160)$$

The construction has exactly

$$W = 10C + 2L + 3(N - 1), \quad (161)$$

$$R = 10C + 3L + 4(N - 1) + 1 \quad (162)$$

auxiliary variables and residuals, before omission of identically zero residuals. It uses no exponentiation, divisibility predicate, bounded universal quantifier, or appeal to MRDP.

Proof. Form the input residuals in (152) and the write-bit residuals, the C comparator systems, and the final memory residuals. Let P_L be their sum of squares. Every residual is quadratic or linear, proving the degree bound.

If $P_L = 0$, every residual vanishes. Write flags are Boolean. The input coordinates force the shifted addresses and keys; the comparator lemma and network correctness force the sorted permutation of complete records. The sorted memory criterion then proves $\text{Valid}_L(E)$. This proves soundness.

If the log is valid, compute the shifted records, run the deterministic network, and choose every comparator and adjacent-pair variable by its explicit formula. All residuals vanish by Lemma 38.2, giving a root. This proves completeness. The same sequence of forcing arguments shows that any root must be the one just constructed, proving uniqueness.

There are $2L$ input auxiliaries and $3L$ input residuals. Each comparator contributes ten of each. Each of the $N - 1$ adjacent pairs contributes three auxiliaries and four residuals, while the first-record check contributes one residual and no auxiliary. Adding these quantities gives (161)–(162). $\square$

[write] A second compiler. Part X (Theorem 76.2, manuscript 08) gives a second compiler for the same semantics with $10C + 4N - 3$ witnesses and $10C + 7N - 4$ residuals. It has $2L - N$ fewer witnesses, and $3N - 3L - 1$ more residuals unless $L = N$, when it has one fewer; neither dominates. The exact comparison follows that theorem.

Remark 38.4 (The empty log). For $L = 0$, take $N = 1$. The sole record is the constant dummy record $(0, 0, 1, 0)$. There are no witnesses and just the identically zero first-record residual. The empty tuple is the unique witness in $\mathbb{N}^0$. Thus the theorem includes the empty computation without an exceptional existential parameter.

Remark 38.5 (Natural versus unrestricted integer witnesses). The theorem does not assert uniqueness, or even the same relation, when the witnesses are allowed to be arbitrary integers. A negative signed-gap witness would destroy the meaning of comparison. Replacing natural variables by sums of four integer squares restores existential representability, but generally creates many representations and destroys the stated bijection. A formal implementation must retain the natural-number domain explicitly.

39 Size, height, and the cost of canonical routing [03]

39.1 Explicit and asymptotic size

Substituting (149) into Theorem 38.3 gives

$$W_L = \frac{5N}{2}k(k+1) + 2L + 3(N-1), \quad (163)$$

$$R_L = \frac{5N}{2}k(k+1) + 3L + 4(N-1) + 1. \quad (164)$$

These are scalar-coordinate and scalar-equation counts. For $L \geq 1$ we have $L \leq N < 2L$ unless $L = 1$, where $N = L$. Thus the explicit construction is $O(L \log^2(L+2))$. For any sorting network with C comparators the simpler bounds $W_L \leq 10C + 5N$ and $R_L \leq 10C + 7N$ hold.

The theorem of Ajtai, Komlós, and Szemerédi supplies sorting networks of size $C = O(N \log N)$ [29]. Substituting those networks in the *same* canonical comparator construction proves the following statement.

Corollary 39.1. *There is an effectively constructible family of quartic natural-number polynomials representing valid length- L memory logs with exactly one auxiliary witness and $O(L \log(L+2))$ auxiliary coordinates and residuals.*

This is an asymptotic existence/construction result using a known sorting network theorem, not a practical improvement implemented in the companion code. The simple bitonic network has transparent constants and a short checker.

Every displayed memory residual has at most eight monomials after collecting terms. Therefore the expansion of P_L has at most $64R_L$ monomial *contributions*, and hence at most that many distinct nonzero monomials. The generous constant includes repeated or constant input wires. Expanding the sum of squares does not entail an exponential expression-size explosion. The residual coefficients have absolute value at most $2(N+1)$; a safe bound for the absolute coefficients of the expanded polynomial is $256R_L(N+1)^2$. Variable names or indices themselves require logarithmically many bits in a serialized description. None of these statements identifies a scalar count with the bit length of a complete file.

39.2 Witness height

Theorem 39.2 (Height bound). *For a valid log let*

$$A = \max(\{a_t : 0 \leq t < L\} \cup \{0\}), \quad V = \max(\{v_t : 0 \leq t < L\} \cup \{0\}).$$

Every coordinate of its unique auxiliary witness is at most

$$H = \max\{1, V, (N + 1)(A + 1) + N - 1\}. \quad (165)$$

Consequently, if the addresses and values have at most b binary digits, every auxiliary coordinate has $b + O(\log(N + 2))$ digits, with an absolute implied constant.

Proof. Shifted addresses are at most $A + 1$ and keys at most $(N + 1)(A + 1) + N - 1$. Every sorting output is a field of an input record, so it obeys the same bound, as do Boolean flags. The comparator gap is bounded by the larger input key by Lemma 36.1. In an adjacent-pair test, $\alpha_j^* - \alpha_{j-1}^* \leq A + 1$; its gap-minus-one coordinate is at most A , and its previous-value coordinate is either zero or a log value. These cover all auxiliary coordinates. Taking binary lengths proves the last claim, also when the maximum input value is zero. $\square$

The witness can be generated by the very comparisons and substitutions used in the proof. In particular, its generation and checking are polynomial-time in the input log's bit length and the displayed certificate size. Address *magnitude* does not enter the number of coordinates; address *bit length* necessarily enters the cost of reading and manipulating a coordinate.

This distinction becomes essential when certificates describe arithmetic programs. The T -step program

$$x_0 = 2, \quad x_{t+1} = x_t^2$$

has $x_T = 2^{2^T}$. Its local quadratic description has $O(T)$ coordinates, but the last coordinate needs $2^T + 1$ binary digits. Thus a small arithmetic certificate, even a unique one, does not by itself supply an NP witness of polynomial bit length. Memory-log height bounds and instruction-level growth bounds must be stated separately.

39.3 What is optimal, and what is not

Proposition 39.3 (Comparison-network lower bound). *An oblivious comparison network that sorts arbitrary N distinct keys has at least $\lceil \log_2(N!) \rceil = \Omega(N \log N)$ comparators. In particular, the comparison order in Corollary 39.1 is optimal within this sorting-network architecture.*

Proof. Give the input positions every possible permutation of N distinct ranks. The sorting process must route those positions into $N!$ different output permutations. A sequence of C binary comparison outcomes determines at most 2^C routing patterns. Therefore $2^C \geq N!$. For example, $N! \geq (N/2)^{N/2}$ for $N \geq 2$, giving the asymptotic bound. The memory-key format does not evade this argument: choosing distinct addresses in an arbitrary order realizes every rank permutation of the keys. $\square$

This is not a lower bound for arbitrary Diophantine encodings. A different certificate architecture could avoid explicit comparison sorting. Nor does it say that Batcher's extra logarithmic factor is necessary.

40 A linear-size alternative: exact large-base products [03]

The comparison lower bound is not a lower bound on Diophantine certificates. We now remove the sorting network entirely. The key is to certify a proposed sorted list by one exact product identity, at a uniquely determined integer base large enough to prevent every possible carry collision. Unlike a random fingerprint over a finite field, this check has no soundness error. It exchanges small witness height for a linear number of scalar coordinates.

40.1 A deterministic multiset lemma

Lemma 40.1 (Carry-free product fingerprint). *Let $C \geq 2$, $L \geq 1$, $\beta = C^L + 1$, and let $c_1, \dots, c_L, d_1, \dots, d_L$ belong to $\{0, \dots, C-1\}$. Then*

$$\prod_{i=1}^L (\beta + c_i) = \prod_{i=1}^L (\beta + d_i) \quad (166)$$

holds if and only if the two lists have the same multiset of entries.

Proof. Form $f(X) = \prod_i (X + c_i)$ and $g(X) = \prod_i (X + d_i)$ in $\mathbb{Z}[X]$. Their coefficients are nonnegative, and the sum of the coefficients of f is

$$f(1) = \prod_i (1 + c_i) \leq C^L < \beta.$$

Thus every coefficient of f is a base- β digit; the same holds for g . Equality $f(\beta) = g(\beta)$ forces equality of all digits, by repeated Euclidean division by β . Hence $f = g$ as polynomials.

To recover the multiset, evaluate at $X = -c_1$. Some factor $-c_1 + d_j$ must be zero, since a product of integers is zero only if a factor is zero. Cancel $X + c_1$ from the two polynomial factorizations and proceed by induction. Cancellation is valid in the integral domain $\mathbb{Z}[X]$. This argument includes repeated entries and zero. Conversely, equal multisets plainly give equal products. $\square$

The carry bound, rather than any unproved collision-resistance assumption, is the entire reason this evaluation at one point is exact. Evaluating at a small fixed integer without the bound would not justify the conclusion. The elementary base-expansion argument is not itself claimed to be a new algebraic principle; the point is its fully canonical use in the memory compiler.

40.2 Packing complete records

For $L \geq 1$, use the genuine records

$$X_t = (\alpha_t, \kappa_t, w_t, v_t), \quad \alpha_t = a_t + 1, \quad \kappa_t = (L+1)\alpha_t + t \quad (0 \leq t < L).$$

No padding is needed. Define uniquely

$$p_0 = 0, \quad p_{t+1} = p_t + \alpha_t + \kappa_t + w_t + v_t, \quad D = p_L + 2. \quad (167)$$

Each input field is strictly less than D . Introduce L proposed output records $X'_j = (\alpha'_j, \kappa'_j, w'_j, v'_j)$ as witness coordinates. For each of their four fields f add a natural slack s and the equation

$$f + s + 1 - D = 0. \quad (168)$$

This both asserts $f < D$ and fixes the bound witness uniquely.

The radix- D code of a record is

$$\rho(X) = \alpha + D\kappa + D^2w + D^3v. \quad (169)$$

For four digits in $[0, D-1]$, the code belongs to $[0, D^4-1]$ and is injective in the complete tuple, by uniqueness of base- D expansion. Represent it with three quadratic gates, not the high-degree expression in (169):

$$h_1 - vD - w = 0, \quad h_2 - h_1D - \kappa = 0, \quad \rho - h_2D - \alpha = 0. \quad (170)$$

Do this for both input and proposed output records.

Introduce $D_2, C, q_1, \dots, q_L, \beta$ and impose

$$D_2 = D^2, \quad C = D_2^2, \quad q_0 = 1, \quad q_i = q_{i-1}C \quad (1 \leq i \leq L), \quad \beta = q_L + 1. \quad (171)$$

These quadratic equations force $C = D^4$ and $\beta = C^L + 1$ without putting exponentiation into the polynomial language. The chain length depends on the fixed parameter L , not an exponent variable. Binary powering could shorten the chain, but the displayed version gives particularly simple exact counts.

Let $A_0 = B_0 = 1$ be constants. The two product chains are

$$A_{i+1} = A_i(\beta + \rho(X_i)), \quad B_{i+1} = B_i(\beta + \rho(X'_i)) \quad (0 \leq i < L). \quad (172)$$

Impose $A_L - B_L = 0$. Every equation remains quadratic. Finally require the proposed keys to be nondecreasing by introducing one natural gap per adjacent pair:

$$\kappa'_j - \kappa'_{j-1} - g_j = 0 \quad (1 \leq j < L). \quad (173)$$

The product lemma and radix injectivity force the proposed records to be a permutation of the complete input records. The keys are distinct, so the nondecreasing order fixes that permutation uniquely. All gap variables are then uniquely determined.

Attach the first-record and adjacent-record memory equations from Section 38, now on these L sorted records. Their address differences are nonnegative by the same key-order argument, and the previous proof applies unchanged.

40.3 The linear compiler theorem

Theorem 40.2 (Linear-size canonical quartic memory). *For every $L \geq 0$ there is an explicit polynomial*

$$P_L^{\text{lin}} \in \mathbb{Z}[X_1, \dots, X_{3L}, Y_1, \dots, Y_{24L}]$$

of degree at most four such that each valid length- L memory log has exactly one natural witness, and each invalid log has none. It is the sum of squares of exactly $22L + 1$ quadratic residuals, each with at most eight monomials. Its expanded sparse polynomial therefore has $O(L)$ monomial contributions. The polynomial and its coefficients depend on L , not on the numerical log.

Proof. For $L = 0$, take no witnesses and the single zero residual. Suppose $L \geq 1$. The input definitions and write-bit constraints force precisely the declared records and Boolean write flags. The prefix chain uniquely fixes D , so the input fields satisfy the digit bounds. The explicit output bound equations force every proposed output field into the same digit range. The Horner and power chains fix all codes and β . The product chains and their final equality, by Lemma 40.1, force equality of code multisets. By radix injectivity this is equality of complete record multisets. Distinct keys and (173) therefore force the single correctly sorted output list. The sorted memory criterion proves soundness.

For a valid log, take that sorted list, use its uniquely determined digit slacks, and compute every chain as displayed. Multiset equality verifies the product identity, and memory validity verifies the final checks. This gives a root. Conversely, in any root the preceding argument forces the output list; all remaining coordinates are then forced by their defining equations, bound slacks, key gaps, and canonical memory tests. Thus there is exactly one root.

The complete accounting is as follows. “Output fields” are existential coordinates whose semantics are forced globally by the product and order checks; they do not need separate defining residuals.

Component	Witnesses	Residuals
Input shifted addresses, keys, write bits	$2L$	$3L$
Prefix sums and D	$L + 1$	$L + 1$
Proposed output fields	$4L$	0
Output digit-bound slacks	$4L$	$4L$
Input and output Horner chains	$6L$	$6L$
$D_2, C, q_1, \dots, q_L, \beta$	$L + 3$	$L + 3$
Two product chains and final equality	$2L$	$2L + 1$
Adjacent key gaps	$L - 1$	$L - 1$
Memory checks	$3(L - 1)$	$4(L - 1) + 1$
Total	$24L$	$22L + 1$

Every residual has degree at most two and a bounded number of terms. The sum-of-squares argument gives the degree and sparse-size claims. All generated constants are fixed small integers or functions of L such as timestamps and $L + 1$; the large base itself is a witness fixed by quadratic equations. $\square$

[write] Batch 78, 2 October 2026: manuscripts 13 and 14 re-derive the product lemma (Lemma 40.1) and prove two further linear-size memory certificates for different relations: manuscript 14's for a zero-initialized log with external address and value caps, with exactly $17L - 4$ witnesses and residuals (Theorem 144.6), and manuscript 13's for a log with initial and final arrays, with the base computed by binary powering (Theorem 144.4). Neither improves this theorem, whose relation has no caps; the three ledgers are compared in Section 144.4.

Theorem 40.3 (Height cost of linear size). *For a valid nonempty log, let D have the value in (167). Every witness of the linear compiler is at most*

$$H_{\text{lin}} = 2^L D^{4L^2}. \quad (174)$$

If the input addresses and values have at most b bits, each witness has $O(L^2(b + \log(L + 2)))$ bits. Generation and verification of the full certificate are polynomial-time in the bit length of the finite log and in L .

Proof. All fields, digit slacks, prefix sums, key gaps, and local memory coordinates are at most D . Horner coordinates are below $C = D^4$; the powers are at most C^L , and $\beta = C^L + 1$. Each factor in a product chain is at most $C^L + C \leq 2C^L$. Every prefix product is therefore at most $(2C^L)^L = 2^L D^{4L^2}$, which also dominates the other coordinates. Further, $D \leq 2 + 4LM$, where M is the largest input-record field, and $\log_2 M = O(b + \log(L + 2))$. Taking logarithms gives the bit-length bound. The chains have linear length, and all their integer operands have the polynomial bit lengths just bounded. Exact addition, multiplication, comparison, and division by fixed bases can thus generate and verify the witness in polynomial time. $\square$

There is a genuine tradeoff, not a free improvement in every resource:

Backend	Scalar witnesses	Maximum witness bit length
Explicit bitonic network	$O(L \log^2(L + 2))$	$O(b + \log(L + 2))$
Optimal sorting network	$O(L \log(L + 2))$	$O(b + \log(L + 2))$
Exact large-base products	$24L$	$O(L^2(b + \log(L + 2)))$

The linear construction is not a bounded-word online memory checker or a finite-field proof system, and makes no claim to evade lower bounds for those different models. It is an offline exact natural-number

certificate whose large coordinates encode the entire multiset check without carries. For $L = 4$ the simple linear version has 96 witnesses versus the network's 77; for $L = 16$ it has 384 versus 877. Thus even the scalar improvement is asymptotic, not a claim of the smallest constant for every short trace.

Corollary 40.4 (Two-backend compilation principle). *Any composition argument using only the memory checker's existence, uniqueness, and decoding contracts may substitute the linear backend for the network backend. The semantic bijection is unchanged. Only the coordinate count and the height estimate change.*

Proof. Both theorems characterize the identical relation on the same chronological log, with a singleton fiber on every valid log. Replace the unique witness of one checker by the unique witness of the other, leaving the semantic log and all other coordinates unchanged. $\square$

41 From memory logs to complete RAM executions [03]

41.1 A precise local interface

A *normalized RAM* has a fixed finite tuple of natural registers and a finite control state. One time step has an explicit choice label c from a fixed finite alphabet (a singleton for a deterministic machine). It computes, from its current local state s and c , a request

$$(a, w, d) = \text{request}(s, c), \quad w \in \{0, 1\}.$$

The request is computed *before* the response is used. Here $d = 0$ on a read and d is the value to store on a write. Let r be the response, canonically zero on a write. The event submitted to the memory checker is

$$(a, w, v), \quad v = d + r, \tag{175}$$

and the next local state is $\text{update}(s, c, r)$. Include the residuals

$$(1 - w)d = 0, \quad wr = 0, \quad v - d - r = 0. \tag{176}$$

On a read the memory checker forces r to be the old value; on a write the local normalization forces $r = 0$. This convention removes an otherwise unused response variable on write steps.

The functions `request` and `update` must have canonical acyclic quadratic implementations, with a bounded number of coordinates per step for the fixed program. Addition, multiplication, comparison, truncated subtraction, Boolean logic, and division by a fixed positive base satisfy this requirement by Section 36. A fixed finite control table is compiled by equality tests and Boolean selection among these operations. Compute every branch using total operations and select the chosen outputs; then even an inactive branch has uniquely determined internal data. An alternative guarded encoding must explicitly zero or otherwise normalize inactive coordinates. Merely multiplying all its constraints by a branch bit would leave arbitrary witnesses.

A fixed finite tuple of possible branch labels can be validated by comparing c with its allowed labels and testing the resulting Boolean disjunction. Exactly one label is retained after halting. Conditional branches of a deterministic program are computed by comparisons, not represented by free choice labels.

We reserve physical address zero for a no-operation event $(0, 1, 0)$. All user addresses are shifted by one before they enter this physical interface. A local step not needing memory uses that event. This reserve is distinct from, and compatible with, the memory compiler's own shifted dummy records. A finitely supported initial memory is supplied by I prescribed initialization writes. Their addresses and values are fixed by the input, not arbitrary witness choices. The initial local state is likewise fixed by the input.

41.2 Halting without padding multiplicity

After the first halt, leave the local state unchanged, force the choice label to zero, set the response to zero, and submit the reserved no-operation write. Thus a run that halts before the horizon has one, not many, padded extensions. The accepting and rejecting halted states are absorbing. Acceptance by time T is tested on the final control state after T steps. A run already halted at time zero is included.

Theorem 41.1 (Parsimonious RAM compilation). *Fix a normalized RAM program as above and a horizon T . There is an effectively constructed integer polynomial $Q_{P,T}(x, Y)$ of degree at most four whose natural-number solutions Y , for each fixed well-formed input x , are in bijection with the labelled executions of P that accept by time T . The bijection is projection to the chronological local states and choice labels; its inverse is canonical certificate generation.*

If the initialization has I writes, the linear backend gives $O_P(T + I)$ coordinates and residuals. The bitonic construction uses $O_P((T + I) \log^2(T + I + 2))$ coordinates and quadratic residuals, counting the trace coordinates as witnesses. An optimal sorting network improves this to $O_P((T + I) \log(T + I + 2))$. In particular, a deterministic program has either no root or exactly one root at each fixed input and horizon.

Proof. Introduce local state, label, request, and response coordinates for each step. Add the canonical request and update circuits, normalization (176), the initial-state constraints, and the final acceptance constraint. Join the I initialization events and the T step events into one chronological log, and attach the memory checker of Theorem 38.3. Use fresh internal coordinates for every circuit and for the checker. The sum of the squares of all residuals has degree at most four.

Consider any root. The local circuits describe valid instruction steps for the values supplied to them. The memory theorem proves that the entire event log obeys sequential zero-initialized memory semantics, including initialization. Start at the fixed initial state and proceed in chronological order. The choice label and current state determine the request before any read. On a read, sequential memory fixes the response uniquely; on a write, the response is zero by (176). The update circuit then fixes the next state. This proves that the projected data form a genuine labelled run, not a self-consistent but causally impossible circular solution. Its final control state is accepting.

Conversely, take any labelled accepting run. Extend it by the prescribed absorbing steps to length T . Its chronological events form a valid memory log. Assign every local auxiliary using its canonical circuit, then the unique memory witness. All residuals vanish.

Finally, fix the semantic run. Its local coordinates and event values are fixed. Lemma 36.4 fixes each local circuit's auxiliaries, and Theorem 38.3 fixes the memory auxiliaries. Therefore the constructed root is the only one projecting to that run. Distinct labelled runs project differently, proving the bijection. There are $O_P(T + I)$ local coordinates and residuals, in addition to the memory checker on $T + I$ events. The size estimates follow; substituting Theorem 40.2 gives the linear bound. A deterministic program has a unique length- T padded run, so its accepting root set has size zero or one. $\square$

[write] Inactive branches and fresh names. Part X normalizes inactive branches canonically, without cubic residuals (Theorem 75.6), and adds heaps with fresh names modulo renaming (Theorems 78.1 and 79.2).

The instruction interface is a hypothesis with a concrete implementation method, not an appeal to MRDP for arbitrary local predicates. For a family of programs the constants depend on their finite instruction circuits. A fixed interpreter can instead make program data part of the initialized memory, but one must count that data and its interpreted runtime.

Corollary 41.2 (Guarded resource restrictions). *Imposing additional Boolean tests on the uniquely decoded trace preserves parsimony. Imposing a coordinate bound $y \leq H$ using $y + z - H = 0$, $z \in \mathbb{N}$,*

also preserves it, because the new slack is forced.

Proof. The tests can only remove decoded runs, and the bound slack is necessarily $z = H - y$. Neither can split a surviving root into several roots. $\square$

42 Applications to computational substrates [03]

42.1 Counter machines and bounded cellular computation

For a fixed counter machine, increment is linear and a zero/decrement branch uses the canonical zero test and truncated subtraction. Control is finite. There is no random-access memory consistency problem if all counters are explicit local registers. Unrolling T steps therefore yields $O_P(T)$ quadratic residuals and a parsimonious quartic equation. The memory theorem is valuable precisely when replacing a large register array by actual indirect addressing; it is not a necessary overhead for every substrate.

A finite-alphabet cellular automaton has a similarly direct description. Fix a finite space–time region, its boundary convention (for example a periodic finite lattice), and an initial configuration. Use one-hot Boolean coordinates for each cell state. Enforce their sum to be one. For each output symbol, express its local transition rule as the sum of the products of the appropriate input indicators. The neighborhood size and alphabet are fixed, so each product has a fixed-length chain of quadratic multiplication gates. The products and sums are unique on valid one-hot inputs. Temporal induction fixes the next layer. Thus a region with V cell updates has an $O(V)$ canonical quartic certificate. For a nondeterministic rule, retain an explicit permitted local choice label; solutions then count labelled space–time histories, not only distinct final configurations.

This construction does not represent an infinite configuration by one finite list without specifying a coding. Infinite-lattice claims require a finite causal cone or an explicit finite-support convention.

42.2 Stacks and Turing tapes

Let the stack alphabet have b symbols and set $B = b + 1$. Encode symbols by $1, \dots, b$, with zero representing the empty stack. Pushing digit d onto stack code n produces $Bn + d$. Popping a nonempty valid stack uses the unique quotient and remainder in (145); its remainder is the top symbol and its quotient the residual stack. A zero test handles the empty stack with a fixed blank-symbol convention. Nonzero digits avoid leading-zero ambiguity. When checking externally supplied stack codes, validate their finite symbol sequence or assume a well-formed initial encoding; a random natural number need not encode a valid stack in this scheme.

Two stacks and a scanned symbol describe a Turing tape: moving right pushes the old scanned symbol onto the left stack and pops the right stack, with blank on an empty stack; moving left is symmetric. Therefore a fixed Turing machine has a canonical bounded-horizon quartic description by the local-gadget method. After T steps only $O(T)$ symbols beyond the input can have been visited, so these radix codes have $O(T + n)$ digits for input length n . Alternatively, representing stacks and tape cells in RAM uses the memory theorem without large radix codes. These are two resource tradeoffs, not distinct proofs of computability equivalence.

42.3 An immutable graph evaluator for SK, SKI, and Iota

The following concrete substrate is close to ProveIt’s combinatory-logic work, but its evaluation strategy must be specified. An immutable directed acyclic graph has leaf nodes S, K, I, ι and binary application nodes $\text{app}(p, q)$. A fresh application node points only to already allocated nodes. Number the initial

nodes densely, including the four reserved leaf nodes, and allocate subsequent nodes at successive fresh identifiers. This is the input representation used in the bit bounds below. A runtime state consists of a focus pointer and a stack of argument pointers. The stack's first item is the next argument to apply.

If the focus is $\text{app}(p, q)$, push q and set the focus to p . For a leaf with sufficient arguments, perform one of the following actions, leaving any remaining argument stack unchanged:

$$\begin{aligned} I; [x, \dots] &\mapsto x; [\dots], \\ K; [x, y, \dots] &\mapsto x; [\dots], \\ S; [x, y, z, \dots] &\mapsto \text{app}(\text{app}(x, z), \text{app}(y, z)); [\dots], \\ \iota; [x, \dots] &\mapsto \text{app}(\text{app}(x, S), K); [\dots]. \end{aligned}$$

The S action allocates three application nodes, and the Iota action two. If there are too few arguments, the machine is in weak-head normal form and halts. There are no reductions inside arguments before they become a focus.

Proposition 42.1 (Unfolding and immutable sharing). *Unfold the finite acyclic heap into ordinary combinator terms and apply the focus term to the argument stack in order. An application-navigation step leaves this term unchanged. Each leaf action realizes exactly the corresponding head contraction of SKI/Iota. The evaluator is deterministic. If the initial heap has n nodes, after m navigation or leaf actions it has at most $n + 3m$ nodes and can be implemented by $O(n + m)$ fixed-size RAM microsteps, including initialization.*

Proof. Acyclicity gives an unambiguous recursively defined unfolding of each pointer. For navigation, unfolding $\text{app}(p, q)$ with residual arguments is precisely unfolding p with q pushed in front. The I, K, S, ι identities are exactly the displayed rules after replacing every pointer by its unfolding. In the S case both occurrences of z refer to the same immutable node, but their unfoldings are identical copies of the same term. No update through one occurrence changes the other. Fresh nodes point to older nodes, preserving acyclicity and the meaning of every old pointer.

Node tags and two pointer fields occupy a fixed number of memory words. Push/pop and each leaf action require a bounded number of field accesses, register operations, and allocations. Distinct affine address ranges encode node fields and stack slots without collision. The stack length grows by at most one per action. Initialization is linear in n . The node count follows because no action allocates more than three nodes. Determinism follows from inspection of the focus tag and the available stack length. $\square$

Corollary 42.2 (Canonical graph-evaluation certificates). *For a well-formed size- n initial combinator DAG and a budget of m spine actions, reaching weak-head normal form within that budget has a quartic natural-number certificate with a unique witness and $O((n + m) \log^2(n + m + 2))$ coordinates. Using optimal sorting networks replaces the squared logarithm by one logarithm. For this pointer-only evaluator, the certificate coordinates have $O(\log(n + m + 2))$ bits. Alternatively, the linear backend uses $O(n + m)$ coordinates with $O((n + m)^2 \log(n + m + 2))$ bits per coordinate.*

Proof. Choose the fixed microstep implementation just described, with a bounded number of phases per spine action. Insert an action counter so that the budget is tested on completed spine actions, rather than accepting a partial contraction. Pad halted states canonically. Apply Theorem 41.1 with a constant-factor microstep horizon large enough for the initialization and m actions. Pointer identifiers, stack positions, control values, allocation counters, and addresses are bounded by $O(n + m + 1)$ when nodes are given dense identifiers. Its local canonical circuits have the same order of magnitude for intermediate values. Theorem 39.2 bounds the memory-checker coordinates by $O((n + m + 1)^2)$, which gives the claimed bit length. Substituting the linear backend and Theorem 40.3 gives the alternative bounds. $\square$

Here m counts navigation *and* contraction actions. It is not merely the number of source reductions. Well-formedness can be checked separately in linear-size canonical circuits by verifying tags and that each edge points to an earlier identifier. The certificate does not expand shared subterms: the acyclic family $D_0 = K, D_{j+1} = \text{app}(D_j, D_j)$ has only $j + 1$ graph nodes but 2^j leaves in its unfolded term. Pointer sharing can therefore be meaningful even before any reduction occurs.

ProveIt's existing development uses context-closed weak beta and combinator relations, while its cited Coq equivalence uses a different call-by-value lambda model [4]. The strategy above is a particular deterministic weak-head graph evaluator. The proof establishes its unfolding semantics; it does not silently identify those different evaluation relations, prove strong normalization, or complete the repository's outstanding exact-semantics bridge. Likewise, injectivity of a source compiler and forward simulation alone do not imply a bijection on complete target execution histories.

43 Exact counting and first-halting probabilities [03]

43.1 Counting executions, not certificate accidents

Let $\text{AccPaths}_{P,T}(x)$ be the finite set of labelled accepting runs of the normalized machine, with the unique halted padding included. The RAM theorem states the exact equality

$$\#\{Y \in \mathbb{N}^{s(P,T)} : Q_{P,T}(x, Y) = 0\} = \#\text{AccPaths}_{P,T}(x). \quad (177)$$

Finiteness follows from the finite branch alphabet and the fixed horizon, although the coordinates themselves may be large. This equality would generally fail for an existential encoding with arbitrary inactive slacks, arbitrary permutation routes, or unconstrained choices after halting.

The same idea yields a useful bounded counting statement. Define the *bounded quartic counting problem* to take a sparse integer polynomial of degree at most four and a binary-encoded bound for each natural variable, and to count its roots inside that box.

Proposition 43.1 (Bounded counting completeness). *Bounded quartic counting is #P-complete under parsimonious polynomial-time reductions. Hardness already holds for polynomials that are sums of squares of quadratic residuals with finite natural root sets.*

Proof. Membership is immediate from the definition of #P: guess the fixed binary representation of each bounded coordinate, reject out-of-range strings, and evaluate the sparse polynomial using exact integer arithmetic. The total number of guessed bits is the sum of the input bound lengths. Degree four ensures that evaluation and its intermediate bit lengths are polynomial in the instance size. Fixed-length binary representations with leading zeros represent each coordinate exactly once.

For hardness, let a #P function be the number of accepting labelled paths of a polynomial-time nondeterministic Turing machine. The stack encoding in Section 42, or its RAM implementation, gives a canonical quartic certificate for every path within its polynomial horizon. Pad after halting canonically. Stack codes have polynomially many bits, because only a polynomial number of fixed-alphabet symbols can occur; the local arithmetic and the memory height bound therefore give a polynomial-length binary bound for every certificate coordinate. The certificate's sparse description and these bounds are computable in polynomial time. Equation (177) proves that the number of bounded roots is precisely the required accepting-path count. The compiler already has finitely many roots for the fixed horizon, so one may choose a box containing all of them. $\square$

This proposition is a consequence of the parsimonious construction and the definition of #P, not a claim that bounded counting hardness is new. In particular, it does not place unrestricted quartic root counting in #P; unrestricted equations can have infinitely many roots.

43.2 Canonical stopping versus random padding

Suppose each live step consumes exactly one independent fair bit. At a halted state no bit is consumed; the recorded padding label is forced to zero. If an accepting path first halts after $\tau \leq T$ live steps, its probability is $2^{-\tau}$, not 2^{-T} . Merely counting canonically padded roots and dividing by 2^T would therefore be incorrect.

Let $e_t \in \{0, 1\}$ indicate that the machine is live at the start of step t . It is computed canonically from the control state. Attach coordinates $u_0, \dots, u_T$ and quadratic residuals

$$u_0 - 1 = 0, \quad u_{t+1} - u_t(2 - e_t) = 0 \quad (0 \leq t < T). \quad (178)$$

These coordinates do not change the number of solutions: their values are forced by the run.

Theorem 43.2 (Exact path weights). *For the extended quartic certificate, the first-halting accepting probability by horizon T is*

$$\Pr[\text{accept by } T] = 2^{-T} \sum_{Y: \tilde{Q}_{P,T}(x,Y)=0} u_T(Y). \quad (179)$$

Every accepting root whose first halt occurs at τ has $u_T = 2^{T-\tau}$. An input already accepting at time zero has one root of weight 2^T , contributing probability one.

Proof. Before the first halt, $e_t = 1$ and the multiplier in (178) is one. After the halt, $e_t = 0$ and the multiplier is two. There are $T - \tau$ such padding steps, so $u_T = 2^{T-\tau}$. The root's contribution to (179) is $2^{-\tau}$, the probability of its live choice sequence. Distinct first-halting labelled paths describe disjoint cylinders in the space of infinite fair-bit streams: no already halted live path is a proper prefix of another live path. Summing their cylinder probabilities gives the probability of acceptance by T . $\square$

More generally, replace e_t by a canonical bit indicating whether a random bit is *consumed* at that step. The same proof uses the number of consumed bits, provided every consumed bit is retained in the labelled path, even when both choices lead to the same next state. Non-random steps have one label. The fixed common denominator remains 2^T when at most one bit is consumed per step. This formulation covers deterministic navigation between probabilistic instructions.

Example 43.3 (Stop on the first one). At each live step, toss a fair bit; accept and halt when it is one, and otherwise continue. There are exactly T accepting canonical paths by time T , namely $1, 01, \dots, 0^{T-1}1$. Their weights are $2^{T-1}, 2^{T-2}, \dots, 1$. Hence

$$\Pr[\text{accept by } T] = \frac{2^{T-1} + \dots + 1}{2^T} = 1 - 2^{-T}.$$

Counting the T roots without weights would instead produce $T/2^T$, illustrating the error caused by conflating canonical halted padding with random padding.

44 Manuscript 03: Conclusion [03]

The main constructive result is an exact linear-size Diophantine memory compiler: $24L$ natural witnesses, $22L + 1$ quadratic residuals, and one sparse quartic equation, with a singleton fiber for every valid log. The central nonlocal check is deterministic carry-free product equality. The complementary comparison-network compiler retains much smaller coordinate heights, exposing a useful quantitative distinction between arithmetic size and bit resources.

Both backends preserve the same semantic object. Their canonical fibers allow complete RAM executions to be compiled parsimoniously, support immutable graph evaluation without unfolding shared terms, and make exact path counting and first-halting probability formulas possible. Those statements are stronger than bare existential representability, but remain finite-horizon statements. The article gives complete mathematical proofs, executable memory compilers, and a precise boundary between what has been obtained and the further fixed-variable compression problem. Establishing literature-wide novelty and formalizing the new proofs are separate tasks, not facts inferred from the successful computations.

Part VI

Queues: tag systems and FIFO networks

Source and scope. [write] This Part is manuscript 07, “Causality Without Tableaux: Compact Diophantine Certificates for Queue Universality”, delivered in batch 61. It prints the manuscript’s little-endian word codes (its Section 2.3), then its Sections 4–10 in their own order: cyclic tag systems without intermediate queues (Theorem 46.1), a real computation and a self-starting fiction, the variable/degree/witness-height tradeoffs, general deletion-tag systems (Theorem 49.1), the exact dimension of polynomial word memory (Theorem 50.4), a quartic certificate from two-coordinate memory (Theorem 51.1), and FIFO networks and switched arithmetic dynamics (Theorem 52.2); it closes with the manuscript’s conclusion. Its first-failure product, on which every certificate here depends, is Theorem 8.1 in Part I; its conventions are in Section 2; its fixed-arity boundary section is in Part IX. The word-memory section is kept before the quartic certificate, as in the manuscript, because the quartic stores each queue in the two coordinates of Proposition 50.1.

Relation to the other Parts. [write] Manuscript 07 names manuscripts 04 and 06 as its companions and treats what they leave out: FIFO word content and the temporal availability of produced symbols. Manuscript 06 says that its counts cannot encode FIFO order and asks which queue restrictions can be added (Research question 71.33); this Part answers that question for bounded runs of fixed-read FIFO networks, while data-dependent read counts stay open. The bounded executions here have one witness each, as in Parts II and III, but on a different substrate: an ordered queue instead of a multiset of tokens, so none of the constructions below is a second route to Theorem 11.2. Where Part II spends $(2T + 1)(d + m)$ or more coordinates on per-step markings, the stream certificate spends $T + 1$ (degree $\max\{4, 2T\}$), the quartic $3T - 2$, the deletion-tag certificate $dT + 1$, and a fixed-read network $D + 1$. The radix word code is the device of Part V’s linear-size compiler (Section 40).

Setting and hypotheses. [write] Queues are first-in first-out. Symbols lie in $\{0, \dots, B - 1\}$; the first written symbol is the head and the *least* significant digit. Formal lengths are signed integers and may become negative after an illegal step. The program, the input word, the terminal word and the horizon $T \geq 1$ are data of the construction, not unknowns; for $T = 0$ the certificate is a constant polynomial. In fixed-read networks the number of symbols read in each phase does not depend on the data. Inside this Part, p is the period of a cyclic-tag program or the number of queues, m_i, a_i, c_i are the length, value and $2^{m_i} - 1$ of an appendant, d is the deletion number, $K = (B - 1)!$, $J = ((B - 1)!)^C$, D is the total number of consumed symbols, G the guard with slack u , and H an upper bound on queue length (Section 2).

45 Little-endian words and faithful concatenation [07]

[write] Section 2.3 of manuscript 07; its Sections 2.1–2.2 are in Section 2 and its Section 3, the first-failure product Theorem 8.1 used below, is in Part I.

Let $B \geq 2$ and let words use digits $0, \dots, B-1$. The first written symbol is the queue head. Define

$$\text{val}_B(a_0 \cdots a_{n-1}) = \sum_{i=0}^{n-1} a_i B^i, \quad \text{val}_B(\varepsilon) = 0.$$

When $B = 2$ the subscript is omitted. The first symbol is thus the *least* significant digit. Written terminal zero symbols still contribute to length.

Lemma 45.1 (Word-code identities). *For finite words u, v ,*

$$\text{val}_B(uv) = \text{val}_B(u) + B^{|u|} \text{val}_B(v).$$

For each fixed n , the value map is injective on words of length n . In particular, equality of length and value is equivalent to equality of words.

Proof. Split the defining sum for uv at position $|u|$. For injectivity, reduce an equality of values modulo B to recover the first digit, subtract that digit, divide by B , and repeat n times. The length condition is essential: ε , 0 , and 00 all have value zero. $\square$

A candidate execution will be represented by its sequence of consumed symbols. A global concatenation identity checks the content of this sequence. The main difficulty is to verify that each of those symbols existed *before* it was consumed.

46 Cyclic tag systems without intermediate queues [07]

46.1 Operational semantics

A cyclic-tag program is a nonempty finite list of binary words

$$\mathcal{A} = (A_0, A_1, \dots, A_{p-1}).$$

The phase at time i is $i \bmod p$, with initial phase zero. From a nonempty queue bq , where $b \in \{0, 1\}$, one step produces

$$qA_{i \bmod p} \quad \text{if } b = 1, \quad q \quad \text{if } b = 0.$$

The phase advances in both cases. There is no transition from an empty queue. Empty appendants are allowed. This is Cook's cyclic-tag semantics with the two symbols renamed 0 and 1 [31].

Fix an input word w , a terminal word v , and $T \geq 1$. Set

$$n = |w|, \quad x = \text{val}(w), \quad h = |v|, \quad y = \text{val}(v),$$

and abbreviate the phase-dependent program constants by

$$m_i = |A_{i \bmod p}|, \quad a_i = \text{val}(A_{i \bmod p}), \quad c_i = 2^{m_i} - 1.$$

These constants depend on i , but do not depend on the candidate run.

46.2 The complete polynomial

Introduce precisely $T + 1$ unknowns

$$b_0, \dots, b_{T-1}, u \in \mathbb{N}.$$

The b_i are intended to be the consumed bits. Define the following *polynomial expressions*, not new variables:

$$\ell_i = n - i + \sum_{j < i} m_j b_j, \quad (180)$$

$$D_i = 1 + c_i b_i, \quad R_i = \prod_{j < i} D_j, \quad (181)$$

$$L = \ell_T - h, \quad (182)$$

$$F = x + 2^n \sum_{i < T} a_i b_i R_i - \sum_{i < T} 2^i b_i - 2^T y, \quad (183)$$

$$G = \prod_{i < T} \ell_i. \quad (184)$$

The representing polynomial is

$$P_{\mathcal{A}, w, v, T}(\mathbf{b}, u) = \sum_{i < T} (b_i(b_i - 1))^2 + L^2 + F^2 + (G - u - 1)^2. \quad (185)$$

Every symbol on the right has been defined by integer polynomial operations. In particular, R_i uses products of affine factors, not an exponent whose exponent is an unknown.

Theorem 46.1 (Causal stream certificate). *For the fixed data above, natural zeros of (185) correspond bijectively to legal T -step executions of $\mathcal{A}$ from w to v . The solution records their consumed bits, and its last coordinate is*

$$u = \prod_{i < T} |w_i| - 1,$$

where w_i is the queue immediately before step i .

The polynomial has exactly $T + 1$ named variables, $T + 3$ residuals, degree at most $\max\{4, 2T\}$, and an $O(T)$ arithmetic circuit for a fixed program. Since cyclic-tag execution is deterministic, its natural zero set is either empty or a singleton. When $v = \varepsilon$, a zero exists exactly when first halting occurs at step T .

46.3 Proof: reconstructing the entire queue from one word identity

Proof. Suppose first that $P(\mathbf{b}, u) = 0$. All residuals vanish, so $b_i \in \{0, 1\}$. Write $A_i^{[b_i]}$ for $A_{i \bmod p}$ when $b_i = 1$ and for ε when $b_i = 0$. This notation denotes a choice of words, not a variable-exponent polynomial operation.

On these Boolean values,

$$D_i = 2^{m_i b_i}, \quad R_i = 2^{\sum_{j < i} m_j b_j}.$$

By Lemma 45.1, the right side of

$$b_0 b_1 \cdots b_{T-1} v = w A_0^{[b_0]} A_1^{[b_1]} \cdots A_{T-1}^{[b_{T-1}]} \quad (186)$$

has value $x + 2^n \sum_{i < T} a_i b_i R_i$. Its length is $n + \sum_{i < T} m_i b_i$. The left side has value $\sum_{i < T} 2^i b_i + 2^T y$ and length $T + h$. Consequently $F = 0$ and $L = 0$ prove the actual word equality (186), not merely a congruence.

It remains to show that this static equality describes a causal computation. The equation $G = u + 1$ makes G positive. Theorem 8.1, with initial length n , unit consumptions, and productions $m_i b_i \geq 0$, implies

$$\ell_i \geq 1 \quad (0 \leq i < T). \quad (187)$$

For each i , let

$$U_i = w A_0^{[b_0]} \dots A_{i-1}^{[b_{i-1}]}.$$

It is a prefix of the right side of (186), and

$$|U_i| = n + \sum_{j < i} m_j b_j = i + \ell_i \geq i + 1.$$

Thus the first $i + 1$ symbols of the global word already occur in U_i , before appendant i is produced. In particular the symbol at position i is b_i and is already available.

Inductively, the actual queue before event i is the suffix of U_i obtained by deleting its first i symbols. The assertion holds at $i = 0$, where $U_0 = w$. It is nonempty by the preceding inequality, its head is b_i , and deleting that head and appending $A_i^{[b_i]}$ gives the stated suffix description for U_{i+1} . This proves every transition. After T steps, (186) says that the remaining suffix is exactly v .

Conversely, consider a legal T -step execution with consumed bits b_i . An induction on the operational semantics gives the same suffix description of its queues, and therefore (186). This implies $L = F = 0$. Its queue lengths are the values (180), all positive before the T steps. Set $u = G - 1 \in \mathbb{N}$. All residuals vanish.

For uniqueness, a fixed input and program determine the first consumed bit and the next queue. Repeating this observation determines the entire consumed sequence. The guard equation then determines u . If $v = \varepsilon$, all earlier queues are nonempty while the last is empty, so T is exactly the first halting time.

Finally, ℓ_i is affine, R_i has degree at most i , and $b_i R_i$ has degree at most $i + 1$. Hence $\deg F \leq T$, $\deg L \leq 1$, and $\deg G \leq T - 1$ because $\ell_0 = n$ is constant. Combining with the Boolean residuals gives degree at most $\max\{4, 2T\}$. There are T Boolean residuals and three further residuals. Prefix recurrence $R_{i+1} = R_i D_i$, length recurrence, and running sums and products evaluate all expressions in $O(T)$ arithmetic operations, with constants from the fixed program. $\square$

46.4 What the theorem does not need

The proof does not need a guessed queue at each time, a guessed time-varying power of two, a division witness for each read, or a general-purpose bounded universal quantifier elimination. Once the consumed bits are fixed, all such quantities are polynomial expressions or are reconstructed at the word level.

The condition that v is fixed matters for the variable count. A terminal *word parameter* can be supplied as data to the compiler without adding witnesses. Treating its numerical code and length as arbitrary unknowns would require a separate validity interface and a different resource statement.

For $T = 0$, no step is taken. Use the constant polynomial 0 if $w = v$ and 1 otherwise, with no witness variables. The unique empty tuple is the zero of the former. The executable compilers deliberately restrict their interface to $T \geq 1$ and report this convention rather than manufacture negative variable counts.

47 Two examples: a real computation and a self-starting fiction [07]

47.1 A small polynomial with exactly one natural zero

Take the period-two program

$$\mathcal{A} = (1\mathbf{0}, \varepsilon), \quad w = 1, \quad v = \varepsilon, \quad T = 3.$$

Remember that the first written symbol is the head. The execution is

$$1 \longrightarrow 1\mathbf{0} \longrightarrow \mathbf{0} \longrightarrow \varepsilon.$$

It consumes $(b_0, b_1, b_2) = (1, 1, 0)$ and has pre-step lengths $(1, 2, 1)$. Therefore $G = 2$ and $u = 1$.

The general formulas simplify to

$$\begin{aligned} L &= -2 + 2b_0 + 2b_2, \\ F &= 1 + b_0 - 2b_1 - 2b_2 + 6b_0b_2, \\ G &= 2b_0(2b_0 - 1). \end{aligned}$$

Thus the completely explicit polynomial

$$\begin{aligned} P &= \sum_{i=0}^2 b_i^2 (b_i - 1)^2 + (-2 + 2b_0 + 2b_2)^2 \\ &\quad + (1 + b_0 - 2b_1 - 2b_2 + 6b_0b_2)^2 \\ &\quad + (2b_0(2b_0 - 1) - u - 1)^2 \end{aligned} \tag{188}$$

has the single natural zero $(1, 1, 0, 1)$. Here cancellation lowers the actual degree to four; the general theorem only promises degree at most six when $T = 3$.

For a direct check of uniqueness, $L = 0$ forces $b_0 + b_2 = 1$ after the Boolean conditions. If $b_0 = 0$, the guard is zero and its residual cannot vanish. Hence $b_0 = 1$, $b_2 = 0$, and $F = 0$ gives $b_1 = 1$. The guard then gives $u = 1$. This agrees with the semantic proof and with the exported exact-integer certificate.

47.2 Why global balance alone is false

Now take

$$\mathcal{A} = (\varepsilon, 1), \quad w = \mathbf{0}, \quad T = 2,$$

and propose the consumed word $\mathbf{0}1$. The global word equation says

$$\underbrace{\mathbf{0}1}_{\text{consumed}} = \underbrace{\mathbf{0}}_{\text{initial}} \underbrace{\varepsilon}_{\text{step 0}} \underbrace{1}_{\text{step 1}},$$

which is true. The length equation is also true. Algebraically these two residuals are $L = b_1 - 1$ and $F = -b_0$, so $(b_0, b_1) = (0, 1)$ satisfies both.

But the actual queue evolves as $\mathbf{0} \rightarrow \varepsilon$ and halts. The proposed second read tries to consume the very symbol that would be appended *as a consequence of that read*. This is not a legal cyclic-tag transition. Here $\ell_0 = 1$ and $\ell_1 = 0$, so $G = 0$ and no natural slack can solve $G = u + 1$.

This example is more than an endpoint bookkeeping issue. It exhibits a genuinely circular dependency in a globally consistent production/consumption equation. The product guard breaks precisely that circularity. In the finite test suite, 268 candidate bitstrings satisfy the raw content and length equations but are rejected by causality; this count is evidence for the implementation, while Theorem 46.1 supplies the proof for all horizons.

Remark 47.1 ([write] Relation to the project’s tag-system notes). The suffix induction in the proof of Theorem 46.1, and the observation that a globally balanced proposed stream can read a symbol supplied by its own later appendant, also appear in exploratory notes of ProveIt’s Hilbert’s-tenth-problem project for deletion- β tag systems (Computability/HilbertTenthProblem/Papers/1980/EXPLORATION_TAG_HALTING_WITHOUT_PREFIX_GUARDS.md, Sections 2–3; EXPLORATION_FINITE_UNIVERSAL_HISTORY_VERIFIERS.md), which manuscript 07 did not consult. Those notes record the causal prefix inequalities that an exact execution requires, and prove that the eventual-halting language needs none of them: the sampled-symbol equations and the final length window already force a genuine halt at the first short prefix. There is no conflict. Theorem 46.1 certifies an exact T -step execution with a unique witness, and the guard is what buys the uniqueness: without it, the notes’ own example ($\beta = 2$, $w = 00$, $h(0) = 0$, $h(1) = 100$) is accepted with both selectors 0 and 001. The notes’ scalar FIFO identity $D = I + WA$ for a queue of constant length (EXPLORATION_NATIVE_STREAM_RAW_QUEUE.md, Section 1) is the global word identity of this Part in a case where the guard is automatic, since every queue length there equals the initial length. The project’s Lean route for Jones’s 1980 tag certificate stores each queue as a content and a length marker 3^ℓ (Computability/HilbertTenthProblem/Lean/Diophantine/Paper1980/TagSystem.lean, TagHistory.lean: Jones1980.length_step’, Jones1980.content_step’), the two-coordinate word memory of Proposition 50.1 in base three. These notes count the arithmetic operations of fixed-arity universal certificates; no result of this Part concerns operation counts.

48 Variable count, degree, and witness height [07]

48.1 The single guard witness has controlled bit height

Let $M = \max_j |A_j|$, put $g = \max\{0, M - 1\}$, and define

$$H = \max\{1, n + Tg\}. \quad (189)$$

During any legal T -step run, queue length can increase by at most g per step. Hence $1 \leq \ell_i \leq H$ for every $i < T$.

Proposition 48.1 (Witness size). *Every zero of the causal stream polynomial satisfies*

$$0 \leq u = G - 1 \leq H^T - 1.$$

The full witness has a binary description of length

$$O(T + T \log_2 \max\{2, H\}).$$

Proof. The T consumed-symbol coordinates are bits. Multiplying T positive queue lengths bounded by H gives $1 \leq G \leq H^T$. A nonnegative integer below H^T needs at most $1 + T \log_2 \max\{2, H\}$ bits under any usual convention for encoding zero. $\square$

The product can be enormous as an integer, but its bit length is only logarithmic in that magnitude. This is one reason not to confuse witness-coordinate count with witness-value bounds. The statement is an upper bound, not a proof that this witness height is best possible.

The coefficients 2^n and 2^T have $n + 1$ and $T + 1$ bits, respectively. If every power 2^i is stored as a separate literal, their total textual size is quadratic in T . An arithmetic circuit can instead generate these powers by repeated doubling. The $O(T)$ arithmetic statement therefore does not assert that every literal-based serialization is bit-linear, nor that multiplication of all intermediate values takes unit time.

48.2 Removing the slack by a larger polynomial guard

For the positive integer H of (189), define

$$Z_H(z) = \prod_{r=1}^H (z - r).$$

Its integer roots are exactly $1, \dots, H$. Keep the Boolean, length, and content residuals from Section 46, but replace $G - u - 1$ by T residuals $Z_H(\ell_i)$.

Theorem 48.2 (No-slack certificate). *The polynomial*

$$\tilde{P}(\mathbf{b}) = \sum_{i < T} (b_i(b_i - 1))^2 + L^2 + F^2 + \sum_{i < T} Z_H(\ell_i)^2$$

has natural zeros in bijection with exact T -step executions from w to v . It uses exactly T variables, $2T + 2$ residuals, has degree at most $2 \max\{2, T, H\}$, and admits an $O(T(1 + H))$ arithmetic circuit for a fixed program.

Proof. Every legal execution has all pre-step lengths in $\{1, \dots, H\}$, so it satisfies the added residuals. Conversely, a zero forces every ℓ_i into that positive range. The same global word identity and suffix induction used in Theorem 46.1 then reconstruct the legal execution. No auxiliary choice remains once its consumed bits are fixed.

Since ℓ_i is affine, $Z_H(\ell_i)$ has degree at most H . The other residual degree bounds were already established. Each bounded-root guard uses $O(H)$ operations, and there are T of them. $\square$

For an empty terminal word, $n > T$ is immediately impossible, because a step removes only one symbol and appends a nonnegative number. When $n \leq T$ and the program is fixed, $H \leq T \max\{1, M\}$. Thus the no-slack version still has degree $O(T)$ and a quadratic-size circuit. It improves the number of witnesses without preserving the linear guard circuit.

48.3 Why these are not universal optimality claims

For a fully specified bounded computation, an unconstrained compiler could simulate all T steps and output the constant polynomial 0 or 1. That would use no variables at all, but would not preserve the computation as a certificate and would tell us nothing about a structural reduction. Meaningful comparisons must specify which computational data remain visible in the solution and what the compiler is permitted to do.

Our constructions are symbolic: they are obtained from the program, input, endpoint, and horizon without needing to run the machine. The implementation optionally simulates the run *after constructing the formula*, solely to attach an example witness. The command-line option `-no-witness` disables this extra operation. The theorem's uniqueness is uniqueness of the complete listed witness, not merely uniqueness of the final machine state.

The no-slack construction also warns against claiming that one guard variable is logically unavoidable. It is not. What that variable buys is a particularly small arithmetic circuit, a simple proof, and a compact witness-height estimate. The exact tradeoff among low degree, nearly minimal trace coordinates, and linear circuit size remains a substantive design problem.

49 General deletion-tag systems [07]

49.1 Semantics and interpolation of the production table

Fix an alphabet $\{0, \dots, B-1\}$ with $B \geq 2$, a deletion number $d \geq 1$, and a word $A(a)$ for each alphabet symbol a . A step requires at least d symbols. It consumes the first d of them and appends $A(a)$, where a was the first consumed symbol. A queue of length less than d is halted. The construction applies to every such system; no universality assumption is required for an individual instance. The family of 2-tag systems with suitable finite alphabets is a standard universal substrate [31, 32].

We need polynomial table lookups without one-hot variables. For each $a \in \{0, \dots, B-1\}$ define

$$\delta_a(z) = \prod_{\substack{0 \leq b < B \\ b \neq a}} \frac{z-b}{a-b}.$$

For an alphabet digit z , this equals one when $z = a$ and zero otherwise. Let

$$M(z) = \sum_{a < B} |A(a)| \delta_a(z), \quad (190)$$

$$A(z) = \sum_{a < B} \text{val}_B(A(a)) \delta_a(z), \quad (191)$$

$$D(z) = \sum_{a < B} B^{|A(a)|} \delta_a(z). \quad (192)$$

Here $A(z)$ is a numerical interpolant; $A(a)$ in the operational semantics is a word. The argument or surrounding value notation distinguishes them. All three interpolants lie in $\mathbb{Q}[z]$ and have degree at most $B-1$.

49.2 The residuals and explicit integer clearing

Fix input w , terminal word v , and $T \geq 1$. Write $n = |w|$ and $x = \text{val}_B(w)$. The unknowns are

$$z_0, \dots, z_{dT-1}, u \in \mathbb{N},$$

where $z_0 \dots z_{dT-1}$ is the full consumed word, including the $d-1$ ignored symbols of each step. Put $h_i = z_{di}$ and define

$$E_B(z) = \prod_{a=0}^{B-1} (z-a), \quad (193)$$

$$\ell_i = n - di + \sum_{j < i} M(h_j), \quad R_i = \prod_{j < i} D(h_j), \quad (194)$$

$$L = \ell_T - |v|, \quad (195)$$

$$F = x + B^n \sum_{i < T} A(h_i) R_i - \sum_{j < dT} B^j z_j - B^{dT} \text{val}_B(v), \quad (196)$$

$$G = \prod_{i < T} (\ell_i)_{\underline{d}}. \quad (197)$$

These expressions are rational polynomials, but denominators can be cleared explicitly. Set $K = (B-1)!$. The denominator of δ_a is, up to sign, $a!(B-1-a)!$, which divides K . Hence KM , KA , and KD have integer coefficients. It follows that the residuals

$$E_B(z_j) \ (j < dT), \quad KL, \quad K^T F, \quad K^{d(T-1)} (G - u - 1) \quad (198)$$

all lie in $\mathbb{Z}[\mathbf{z}, u]$. The last exponent suffices because $\ell_0 = n$ is constant, so only $T - 1$ falling factorials have a possible denominator. All clearing multipliers are nonzero integers. No new unknowns are introduced.

Theorem 49.1 (Deletion-tag stream certificate). *The sum of squares of (198) has natural zeros in bijection with exact T -step executions from w to v . It uses $dT + 1$ variables and $dT + 3$ residuals. Its degree is at most*

$$2 \max\{B, T(B - 1), d(T - 1)(B - 1), 1\}. \quad (199)$$

For fixed B, d , and production table, it has an $O(T)$ arithmetic circuit. Each successful instance has a unique witness. If $|v| < d$, it certifies first halting at exactly T .

Proof. The digit residuals force every z_j to be an alphabet symbol. On those symbols, the interpolants have their prescribed natural values. Clearing denominators does not change any zero condition.

The length and content equations, together with Lemma 45.1, give the word equality

$$z_0 \cdots z_{dT-1} v = w A(h_0) A(h_1) \cdots A(h_{T-1}). \quad (200)$$

The guard equation and Theorem 8.1, with consumption d at every step, imply $\ell_i \geq d$ for every $i < T$. Let $U_i = w A(h_0) \cdots A(h_{i-1})$. Its length is $di + \ell_i \geq d(i + 1)$. Thus the whole next consumed block $z_{di} \cdots z_{di+d-1}$ is already in U_i , not in an appendant produced by this or a later step. The suffix induction now deletes d symbols at a time. Its first symbol is h_i , so the selected appendant is correct. The terminal suffix is v .

Conversely, any actual execution satisfies (200), the length equation, the alphabet constraints, and the positive resource product. It therefore gives a zero with $u = G - 1$. Deterministic semantics fixes every consumed symbol, including the ignored ones, and the guard fixes the slack.

For the degree estimate, digit residuals have degree B , length has degree at most $B - 1$, and $A(h_i)R_i$ has degree at most $(i + 1)(B - 1)$. The nonconstant part of the guard has at most $d(T - 1)$ factors of degree at most $B - 1$. The term $-u - 1$ contributes degree at most one. Squaring gives (199). With fixed alphabet and deletion number, each table evaluation and falling factorial uses a fixed number of arithmetic operations; prefix products, lengths, and sums are accumulated in $O(T)$ operations. The clearing multipliers are constant coefficients with bit length linear in T for fixed B, d . $\square$

Integer-only circuit realization. Denominator clearing need not introduce division gates. Put $\widehat{M} = KM$, $\widehat{A} = KA$, $\widehat{D} = KD$, and

$$\lambda_i = K(n - di) + \sum_{j < i} \widehat{M}(h_j), \quad \Pi_i = \prod_{j < i} \widehat{D}(h_j).$$

Then the cleared content and guard residuals can be evaluated as

$$K^T F = K^T \left(x - \sum_{j < dT} B^j z_j - B^{dT} \text{val}_B(v) \right) + B^n \sum_{i < T} K^{T-i-1} \widehat{A}(h_i) \Pi_i,$$

$$K^{d(T-1)}(G - u - 1) = (n)_d \prod_{i=1}^{T-1} \prod_{r=0}^{d-1} (\lambda_i - Kr) - K^{d(T-1)}(u + 1).$$

All exponents in these expressions are fixed nonnegative integers. Precompute the powers of K by multiplication; every other gate is an integer addition, subtraction, or multiplication. This gives the asserted linear arithmetic size without using rational constants internally.

49.3 Halting tails are not necessarily empty

A deletion- d tag machine can halt with $1, \dots, d-1$ symbols still present. Equating its terminal queue with ε would miss those computations. There are

$$N_{B,d} = 1 + B + \dots + B^{d-1}$$

possible halted words. The endpoint theorem handles each of them.

Corollary 49.2 (Unspecified halted endpoint). *For fixed B, d , program, input, and $T \geq 1$, first halting at exactly T has a witness-faithful polynomial with $dT + 1$ variables and degree at most $N_{B,d}$ times (199).*

Proof. For each word v of length less than d , let P_v be the sum-of-squares polynomial of Theorem 49.1, using the same names for all consumed-symbol variables and the slack. Use $\prod_{|v|<d} P_v$. An integer product vanishes exactly when one factor vanishes. Each factor certifies an actual execution to its specified endpoint. Determinism permits at most one such endpoint and one complete witness. The common guard does not create extra choices. The degree of the product is at most the sum of the degrees of its factors. $\square$

The product here is over a fixed finite collection determined by B, d , not over an unbounded family of words. Its size can be large as these parameters grow; the linear-in- T statement keeps them fixed.

49.4 A ternary example with a nonempty halted tail

Let $B = 3, d = 2$, and choose productions

$$A(0) = \varepsilon, \quad A(1) = 20, \quad A(2) = 0.$$

Starting from 10 , the execution is

$$10 \longrightarrow 20 \longrightarrow 0.$$

At $T = 2$ it halts with one symbol left. The consumed digits are $(1, 0, 2, 0)$; both pre-step lengths are two, so

$$G = (2)_2(2)_2 = 4, \quad u = 3.$$

The global identity is $10200 = 10200$. Its value is 19 in base three, with the stated little-endian convention. This illustrates why the ignored symbols and the terminal length must both be retained.

50 The exact dimension of polynomial word memory [07]

50.1 Two affine coordinates suffice

The stream certificate eliminates queue states globally. For a local encoding, a useful state representation is

$$\mathcal{E}_B(w) = (\text{val}_B(w), B^{|w|}) = (v, s) \in \mathbb{N}^2. \quad (201)$$

The second coordinate records scale, not length itself. Define

$$(v, s) \star (w, r) = (v + sw, sr). \quad (202)$$

Lemma 45.1 gives

$$\mathcal{E}_B(uv) = \mathcal{E}_B(u) \star \mathcal{E}_B(v).$$

The operation is associative, as is clear either by direct expansion or by identifying (v, s) with the matrix

$$\begin{pmatrix} s & v \\ 0 & 1 \end{pmatrix}.$$

The empty word has code $(0, 1)$, the identity element. Insertion of a fixed symbol at either end is affine:

$$L_a(v, s) = (Bv + a, Bs), \quad R_b(v, s) = (v + bs, Bs). \quad (203)$$

For every pair of symbols, $L_a R_b = R_b L_a$, reflecting the word identity $a(wb) = (aw)b$.

Proposition 50.1. *The map $\mathcal{E}_B$ is a faithful natural-number encoding of all finite words, with polynomial concatenation and affine insertion at both ends.*

Proof. The scale determines the length because $B^m = B^n$ implies $m = n$ for natural m, n . The value then determines the word by Lemma 45.1. The displayed formulas establish the operation claims. $\square$

The next result shows that two is the minimum number of natural coordinates for this insertion architecture. It does not assume that arbitrary-word concatenation is polynomial; two prepend maps and a single fixed append map already suffice for the obstruction.

50.2 An elementary centralizer lemma

Composition of polynomials is denoted by $\circ$. A polynomial commutes with Q when $F \circ Q = Q \circ F$.

Lemma 50.2 (Uniqueness at a fixed degree and leading coefficient). *Let $Q \in \mathbb{R}[X]$ have degree $d \geq 2$. Two nonconstant polynomials H_1, H_2 that commute with Q , have the same degree m , and have the same leading coefficient must be equal.*

Proof. Suppose $\Delta = H_1 - H_2$ is nonzero and has degree $k < m$. Let $c \neq 0$ be the leading coefficient of Q and $a \neq 0$ the common leading coefficient of H_1, H_2 . Subtract the two commutation identities to obtain

$$Q(H_1) - Q(H_2) = \Delta(Q).$$

The right side has degree dk . On the left, factor the difference of the leading powers of Q . Its highest term has nonzero coefficient

$$dc a^{d-1} \text{lc}(\Delta)$$

and degree $(d-1)m + k$. Lower powers of Q give smaller degrees. Thus equality would imply $(d-1)m + k = dk$, or $(d-1)(m-k) = 0$, a contradiction. $\square$

Lemma 50.3 (Positive-leading centralizers are commutative). *Let Q be a nonconstant real polynomial with positive leading coefficient, and assume $Q \neq X$. Any two nonconstant real polynomials F, G with positive leading coefficients that commute with Q also commute with one another.*

Proof. First suppose $\deg Q = d \geq 2$ and let $c > 0$ be its leading coefficient. If $\deg F = m$ and $a > 0$ is the leading coefficient of F , comparison in $F(Q) = Q(F)$ gives

$$ac^m = ca^d, \quad a^{d-1} = c^{m-1}.$$

The positive solution is $a = c^{(m-1)/(d-1)}$. Similarly, if $\deg G = n$, its leading coefficient is $c^{(n-1)/(d-1)}$.

Both $F \circ G$ and $G \circ F$ commute with Q . They have degree mn , and their leading coefficients both equal $c^{(mn-1)/(d-1)}$. Lemma 50.2 makes them equal.

Now suppose $Q(X) = cX + e$ with $c > 0$. If $c = 1$, then $e \neq 0$ because $Q \neq X$. Commutation gives

$$F(X + e) - F(X) = e.$$

For a nonconstant polynomial the left side has degree $\deg F - 1$, so F has degree one. Comparing its slope shows that $F(X) = X + f$ for a constant f . The same holds for G , and translations commute.

If $c \neq 1$, translate the fixed point $e/(1 - c)$ to zero. In the translated coordinate, the commuting condition is $F(cX) = cF(X)$. For the coefficient of X^k , this requires $c^k = c$ whenever that coefficient is nonzero. Since $c > 0$ and $c \neq 1$, only $k = 1$ is possible, including the exclusion of the constant coefficient. Thus all such maps are scalar multiplications in the same translated coordinate, and they commute. $\square$

This argument is a small positive-real specialization of the subject of commuting polynomials. It is included in full rather than importing a classification theorem or presenting the general centralizer theory as new [33].

50.3 One natural coordinate cannot support both ends

Theorem 50.4 (One-coordinate polynomial obstruction). *There is no injective map*

$$e : \{0, 1\}^* \longrightarrow \mathbb{N}$$

and no real polynomials P_0, P_1, Q satisfying, for every finite binary word w ,

$$P_0(e(w)) = e(0w), \quad P_1(e(w)) = e(1w), \quad Q(e(w)) = e(w0). \quad (204)$$

In particular, a faithful natural-coordinate word representation with polynomial insertion of arbitrary binary symbols at both ends requires at least two coordinates. Proposition 50.1 attains that dimension with affine maps.

Proof. The image of e is an infinite subset of $\mathbb{N}$ and is therefore unbounded. Each of P_0, P_1, Q is nonconstant, because its outputs on that image include infinitely many distinct word codes. Each has positive leading coefficient: a nonconstant real polynomial with negative leading coefficient is eventually negative, contrary to its natural values on an unbounded subset of $\mathbb{N}$.

For $a \in \{0, 1\}$ and every w ,

$$(P_a \circ Q)(e(w)) = e(a(w0)) = e((aw)0) = (Q \circ P_a)(e(w)).$$

Polynomials agreeing on an infinite set are equal, so P_a commutes with Q as a polynomial identity. Moreover $Q \neq X$, since $Q(e(w)) = e(w)$ would imply $w0 = w$ by injectivity, which contradicts word length.

Lemma 50.3 now gives $P_0 \circ P_1 = P_1 \circ P_0$. Evaluating at $e(w)$ yields

$$e(01w) = e(10w).$$

The two words are distinct, contradicting injectivity. $\square$

[write] *Theorem 50.4 is consistent with manuscript 01's injective polynomial pairing $\pi(a, b) = (a + b)^2 + a$ (Lemma 56.1) and with manuscript 05's pairing (241): those are maps of two arguments used to code finite terms and lists, while the theorem concerns one-argument insertion maps on the code of a single word.*

50.4 The boundary is architectural, not computational

The theorem does not say that one integer cannot encode a word. For example,

$$e(w) = 2^{|w|} + \text{val}(w)$$

is a faithful sentinel encoding. Both prepends are polynomial:

$$e(aw) = 2e(w) + a.$$

Appending a zero, however, requires adding $2^{|w|}$, the leading power of two of the code. The theorem proves that no alternative scalar coding can make both prepends and this append simultaneously polynomial on all word codes.

The result does not exclude one-register computation with conditional operations, floor, division, prime-exponent tests, or auxiliary Diophantine witnesses. It does not apply to a finite set of word lengths, where arbitrary finite tables can be interpolated, or to a code whose operations depend on additional hidden control state. Nor does it claim a dimension bound for all possible continuous or rational encodings. Its conclusion is exactly about faithful natural-number word codes with fixed scalar polynomial insertion maps.

51 A quartic certificate from two-coordinate memory [07]

51.1 Encoding a single queue step

For binary words let $(v, s) = \mathcal{E}_2(w)$. A nonempty word has $s = 2^{|w|} \geq 2$. Write its head as $b \in \{0, 1\}$ and the value of its tail as q . Then

$$v = b + 2q.$$

At phase i , the update after the optional appendant is

$$v' = q + \frac{a_i b s}{2}, \quad s' = \frac{s(1 + (2^{m_i} - 1)b)}{2}. \quad (205)$$

The denominator two is exact on a valid nonempty input scale. The updated scale is $2^{|w|-1+m_i b}$, and the updated value is precisely the value of the tail followed by the selected appendant.

51.2 The full quadratic residual system

Fix w and $T \geq 1$, and require the endpoint to be empty. Substitute the constant endpoints

$$(v_0, s_0) = (\text{val}(w), 2^{|w|}), \quad (v_T, s_T) = (0, 1).$$

The natural unknowns are the $2(T-1)$ internal coordinates

$$(v_i, s_i) \quad (1 \leq i < T)$$

and T quotients q_i for $0 \leq i < T$. In the following formulas,

$$b_i = v_i - 2q_i$$

is an abbreviation for an affine expression, not a new variable. For each $i < T$ use the three residuals

$$B_i = b_i(b_i - 1), \quad (206)$$

$$V_i = 2v_{i+1} - 2q_i - a_i b_i s_i, \quad (207)$$

$$S_i = 2s_{i+1} - s_i - (2^{m_i} - 1)b_i s_i. \quad (208)$$

Let $Q_{\mathcal{A}, w, T} = \sum_{i < T} (B_i^2 + V_i^2 + S_i^2)$.

Theorem 51.1 (Local quartic certificate). *The natural zero set of $Q_{\mathcal{A},w,T}$ is a singleton if the cyclic-tag run from w first halts at time T , and is empty otherwise. The polynomial has $3T - 2$ variables, $3T$ quadratic residuals, degree at most four, and $O(T)$ arithmetic-circuit size for a fixed program.*

Proof. Every actual run first halting at T gives the displayed internal word codes and quotients $q_i = \lfloor v_i/2 \rfloor$. Equations (206)–(208) are just the head decomposition and the update (205) with denominators cleared. Thus it gives a zero.

Conversely, suppose there is a zero. We induct from the valid initial word code. Assume (v_i, s_i) is the code of an actual queue. Equation $B_i = 0$ forces the signed integer $b_i = v_i - 2q_i$ to be either zero or one. Since $q_i \in \mathbb{N}$, the decomposition $v_i = 2q_i + b_i$ uniquely determines both q_i and b_i .

If the queue is nonempty, its scale is even and at least two. The other residual equations uniquely force the actual next value and scale from (205); hence they again form a valid word code. Notice that this establishes scale validity by induction. There is no separate existential predicate for being a power of two.

If the queue were empty before a required step, then $v_i = 0$ and $s_i = 1$. The Boolean residual and nonnegativity of q_i force $q_i = b_i = 0$. The scale residual would become $2s_{i+1} = 1$, which has no natural solution. Therefore no zero can pass through an empty queue before step T .

Induction proves all T actual transitions, and the final constants $(0, 1)$ identify the empty word. The run consequently first halts at T . Determinism and quotient uniqueness fix every variable. The count is $2(T - 1) + T = 3T - 2$, and all displayed residuals are quadratic, proving the remaining bounds. $\square$

51.3 The small example in quartic coordinates

For the example $\mathcal{A} = (10, \varepsilon)$, $w = 1$, $T = 3$, the internal states are

$$(v_1, s_1) = (1, 4), \quad (v_2, s_2) = (0, 2),$$

and the quotients are $(q_0, q_1, q_2) = (0, 0, 0)$. Thus the seven-coordinate witness is

$$(v_1, s_1, v_2, s_2, q_0, q_1, q_2) = (1, 4, 0, 2, 0, 0, 0).$$

This is the witness in `data/07-queue-causality-quartic_example.json`. The stream construction uses only four coordinates for the same computation; the quartic construction guarantees degree four for every horizon instead of allowing degree to grow with T .

51.4 Comparing information content

A valid state of length at most $n + T \max\{0, M - 1\}$ has value and scale requiring $O(n + TM + 1)$ bits. The local witness therefore has the elementary upper bound $O(T(n + TM + 1))$ bits. The stream witness instead stores the consumed bits and a product whose bit length is controlled by Proposition 48.1.

These are different certificates for the same semantic object. One maintains a low-degree local recurrence by carrying explicit memory coordinates. The other eliminates those coordinates, retains one global content equation, and pays for the elimination in polynomial degree. The no-slack variant makes the independence of degree, circuit size, and variable count even more visible.

52 FIFO networks and switched arithmetic dynamics [07]

52.1 Causality for several channels

The causal reconstruction argument is not inherently a one-queue argument. Consider p FIFO queues. At event i , a candidate trace consumes the word C_{ij} from queue j and appends the word A_{ij} to it. All reads occur before that event's writes. Local rules, if present, must separately validate the proposed input/output words. Let W_j, V_j be the initial and final queues, and write

$$\ell_{ij} = |W_j| + \sum_{k < i} (|A_{kj}| - |C_{kj}|), \quad G = \prod_{i < T} \prod_{j < p} (\ell_{ij})_{|C_{ij}|}.$$

Theorem 52.1 (FIFO network reconstruction). *The candidate words form a legal FIFO execution between the specified endpoints if and only if $G > 0$ and, for each queue j ,*

$$C_{0j}C_{1j} \cdots C_{T-1,j}V_j = W_jA_{0j}A_{1j} \cdots A_{T-1,j}. \quad (209)$$

Here “legal” refers to queue availability and the stated consumption/production words; satisfaction of any additional local control rule is a separate premise.

Proof. Any legal execution satisfies the word identities by following each queue's FIFO order, and its resource product is positive by Theorem 8.1.

Conversely, the same resource theorem makes every queue long enough for the prescribed consumption at each event. Before event i , the already produced prefix on queue j is $W_jA_{0j} \cdots A_{i-1,j}$. Its length is at least the number of symbols previously consumed on that queue plus $|C_{ij}|$. The corresponding next block in the global identity (209) therefore lies in this already produced prefix and equals C_{ij} . Simultaneously induct on all queues: the current queue is the suffix of its produced prefix after deleting all previous consumptions. Removing the next block and appending the designated output preserves that description. The final suffixes are exactly V_j . $\square$

52.2 A trace-symbol compiler for fixed-read networks

A useful concrete class of networks has a fixed finite cyclic control schedule. Each phase specifies a consumption vector $(c_{ij})_{j < p}$, independent of the data. A finite table maps the tuple of consumed symbols to one output word on each queue. Such a table may branch arbitrarily on the consumed tuple, but the number of symbols read in that phase is fixed. A phase is disabled if some queue is too short.

Fix T , all endpoints, and the program. Let

$$D = \sum_{i < T} \sum_{j < p} c_{ij}, \quad C = \max_{i < T} \sum_{j < p} c_{ij}, \quad h = C(B - 1).$$

For $T \geq 1$, introduce one alphabet-digit variable for each of the D consumed symbols, and one natural slack u . Tensor-product Lagrange interpolation of each finite output table gives its output length, output value, and output scale as polynomials of degree at most h in that event's digits. Give each queue its length and value equation from (209); append prefixes are accumulated by multiplying their scale interpolants. Use the single global resource residual $G - u - 1$.

Theorem 52.2 (Fixed-read network certificate). *The resulting residuals, after fixed integer denominator clearing, give a witness-faithful polynomial with $D + 1$ variables, $D + 2p + 1$ residuals, and degree at most*

$$2 \max\{B, Th, hD, 1\}.$$

For a fixed network program with a fixed number of queues, it has $O(T)$ arithmetic-circuit size. Its natural zeros correspond bijectively to exact T -event executions between the fixed endpoints.

Proof. The D digit residuals force the local table arguments to be alphabet symbols. Table evaluation then returns actual nonnegative lengths and actual finite words. The two residuals per queue imply (209), by equal length and equal radix value. The resource residual makes the global guard positive, so Theorem 52.1 reconstructs the execution. Conversely a legal execution provides all its consumed digits and the unique slack $G - 1$. The fixed schedule and deterministic tables determine every consumed word from the endpoints' initial queues, hence the complete witness is unique.

Each digit residual has degree B . An output value times the product of preceding output scales has degree at most Th . Formal queue lengths have degree at most h . The product guard has D linear-in-length factors and therefore degree at most hD . Including u and squaring gives the stated bound.

For explicit clearing, put $J = ((B - 1)!)^C$. Every tensor-product table interpolant has denominator dividing J . Multiply each length equation by J and each content equation by J^T . Writing $\lambda_{ij} = J\ell_{ij}$, the cleared guard is

$$\prod_{i < T} \prod_{j < p} \prod_{r < c_{ij}} (\lambda_{ij} - Jr) - J^D(u + 1).$$

It is an integer polynomial. Weighted integer prefix products give integer-only content circuits exactly as in Section 49. With the program fixed, table sizes, p , C , and B are fixed, while $D = O(T)$; the arithmetic size follows. $\square$

This class contains cyclic tag systems as the case of one queue, one symbol read per phase, and the two-entry append/skip table. It therefore includes universal computation, but the theorem does not claim that every individual network is universal. It also does not cover arbitrary data-dependent read counts or hidden zero tests without additional residuals. The general network theorem is proved here; the shipped executable compilers instantiate the one-queue cyclic-tag and deletion-tag cases, not the full network-table interface.

52.3 A two-track arithmetic substrate

On valid binary word codes, (205) becomes the following periodically switched affine rule:

$$(v, s) \mapsto \begin{cases} (v/2, s/2), & v \text{ even,} \\ ((v - 1 + a_i s)/2, 2^{m_i - 1} s), & v \text{ odd.} \end{cases} \quad (210)$$

The state domain is

$$\mathcal{W} = \{(v, 2^n) : n \in \mathbb{N}, 0 \leq v < 2^n\},$$

and the process stops at $s = 1$. When $m_i = 0$, the second multiplier is $1/2$, a rational coefficient, not an omitted case.

Corollary 52.3 (Conjugate arithmetic computation). *The map $\mathcal{E}_2$ is a bijection from binary words to $\mathcal{W}$ and conjugates cyclic-tag transitions to (210), including halting. Consequently this family of two-track, parity-switched rational-affine processes inherits cyclic-tag universality and the exact certificates of Theorems 46.1 and 51.1.*

Proof. Each integer $0 \leq v < 2^n$ has a unique length- n base-two digit expansion, including terminal zeros. For a nonempty word, its head is the parity of v , and (205) is precisely the stated branch formula. Scale one corresponds only to the empty word. This establishes a step-for-step bijection, not merely a simulation with unspecified overhead. $\square$

The switching condition is essential. This is not a claim that iteration of one everywhere-polynomial scalar function is universal. Nor does the two-coordinate lower bound apply to arbitrary parity-switched scalar dynamics.

52.4 What transports to spatial computation

Cook’s construction connects cyclic tags to glider interactions and then to Rule 110 [31]. A certified simulation can transport a statement about an encoded cyclic-tag run to the corresponding spatial computation. But the transport has to preserve the chosen event and endpoint semantics. A tag step is not automatically one cellular-automaton time step, and an encoded infinite periodic background is not a finite queue state.

Thus our displayed counts measure tag events or fixed-read network events. They do not establish a direct $T + 1$ -variable polynomial for every arbitrary T -step Rule 110 space-time diagram. A direct glider or cellular-automaton compiler with certified clocks, boundary conditions, and observation conventions is a separate research target. This distinction keeps the unconventional-substrate connection concrete without conflating different notions of time and input.

[write] *The bounded cellular-automaton constructions of the other manuscripts are finite space–time tableaux with a fixed number of cells: manuscript 05’s finite cellular automata and manuscript 04’s cellular automata in a finite box, manuscript 01’s circuit cylinders and manuscript 02’s Boolean spacetime substrates (all in Part VIII), and manuscript 03’s bounded cellular computation (Section 42). The “without tableaux” of manuscript 07’s title contrasts its queue certificates with such constructions; as just said, it does not claim a direct polynomial for arbitrary Rule 110 diagrams.*

53 Manuscript 07: Conclusion [07]

A queue computation has two logically different requirements: its consumed word must agree with what the computation produces, and each consumed symbol must already be present when it is read. A global radix identity addresses the first requirement. A first-failure product addresses the second without quantifying intermediate queues.

Together they give an exact $T + 1$ -variable cyclic-tag certificate, a $dT + 1$ -variable deletion-tag certificate, and a consumed-symbol-plus-one certificate for fixed-read FIFO networks. The no-slack and quartic alternatives show how circuit size, degree, and variable count can be traded without losing witness uniqueness. The polynomial-memory obstruction independently explains why two-coordinate local word memory cannot simply be replaced by one scalar with polynomial insertion at both ends.

These results answer concrete structural questions about Diophantine compilation. Their next useful tests are formalization, sharper resource bounds, and verified transport to genuinely spatial substrates. Their bounded uniqueness is a precise starting point for fixed-arity research, not a substitute for the additional argument that research requires.

Part VII

FRACTRAN, SKI and term rewriting

Source and scope. [write] Manuscript 04’s Sections 6–7 give a linear-size, unique-witness certificate for bounded FRACTRAN runs with first-applicable-fraction priority (Theorem 54.2) and a context-local certificate for SKI reduction along a prescribed schedule (Theorem 55.2), extended to finite first-order rewriting (Corollary 55.6). The last section of the Part collects three shorter treatments: manuscript 02’s local single-fold FRACTRAN step interface, manuscript 01’s local polynomial adapter for combinatory logic and its warning about forward simulations, and manuscript 05’s bounded word rewriting and explicit term graphs.

Relations. [write] For FRACTRAN, manuscript 04’s certificate covers a complete bounded run; manuscript 02’s interface certifies one step and the terminal condition, and is printed after it as the local counterpart. For combinatory logic the three constructions are different and all are kept: manuscript 04 encodes whole subterms by integer codes and follows a fixed schedule, manuscript 01 gives a local head-rule adapter with the injective pairing of Lemma 56.1, and manuscript 03’s immutable graph evaluator is in Part V (Section 42.3). The repository’s combinatory-logic development distinguishes source syntax, runtime syntax and evaluation strategy ([4]); none of these constructions is a formal bridge to it.

Setting and hypotheses. [write] Manuscript 04: an ordered list of r fractions in lowest terms and a horizon of T steps; SKI terms coded by $\text{App}(\ell, r)$ with leaves S, K, I coded 0, 1, 2, a fixed schedule σ of total context depth H . Manuscript 01: the same pairing $\pi(a, b) = (a + b)^2 + a$ with $\text{App}(a, b) = 3 + \pi(a, b)$, validity of the input term as a domain promise. Manuscript 05: a fixed alphabet, substring rules, horizon T and an explicit word-length bound B .

54 A linear-size, unique-witness FRACTRAN priority certificate [04]

54.1 Semantics and the issue with naive disjunction

A FRACTRAN program is a finite ordered list of positive fractions

$$\frac{a_1}{b_1}, \dots, \frac{a_r}{b_r}, \quad r \geq 1,$$

which we first put in lowest terms. At a positive integer n , choose the least j for which $b_j \mid n$ and replace n by $a_j n / b_j$. If there is no such j , execution halts. Reduction to lowest terms is essential: applicability of an unreduced fraction a/b is $b \mid an$, not generally $b \mid n$.

Diophantine treatment of this language is established prior work [30, 13]. Our question is the exact cost of a bounded, priority-correct, unique-witness certificate. An existential disjunction over all applicable fractions would lose priority. Giving every inactive branch an unrestricted remainder witness would also lose uniqueness. The following circuit handles both issues with a linear, rather than all-pairs, priority scan.

54.2 Canonical division and zero detection

Fix a positive denominator b . Introduce $q, \rho, s, u, z \in \mathbb{N}$ and impose

$$n = bq + \rho, \tag{211}$$

$$\rho + s = b - 1, \tag{212}$$

$$z(z - 1) = 0, \tag{213}$$

$$z\rho = 0, \tag{214}$$

$$\rho = (1 - z)(u + 1), \tag{215}$$

$$zu = 0. \tag{216}$$

Lemma 54.1 (Single-valued divisibility gadget). *For every $n \in \mathbb{N}$ and $b \geq 1$, equations (211)–(216) have exactly one natural solution. It has $q = \lfloor n/b \rfloor$, $\rho = n \bmod b$, $s = b - 1 - \rho$, and*

$$z = \mathbf{1}_{b \mid n}, \quad u = \begin{cases} 0, & \rho = 0, \\ \rho - 1, & \rho > 0. \end{cases}$$

Proof. The first two equations imply $0 \leq \rho < b$, so the quotient and remainder theorem forces q, ρ , and then s . If $\rho = 0$, the fifth equation makes $z = 0$ impossible, since it would say $0 = u + 1$. Hence $z = 1$, and the last equation forces $u = 0$. If $\rho > 0$, the fourth equation forces $z = 0$, and the fifth forces $u = \rho - 1$. These values satisfy all six equations. The argument also covers $b = 1$, where the remainder is always zero. $\square$

Equation (216) is not cosmetic. Without it, the divisible case would permit every natural u , producing infinitely many encodings of the same division.

54.3 A cumulative least-applicable-index circuit

At time t , run the preceding gadget for every denominator b_j , obtaining $q_{t,j}, \rho_{t,j}, s_{t,j}, u_{t,j}, z_{t,j}$. Add natural variables $h_{t,0}, \dots, h_{t,r}$ and $x_{t,1}, \dots, x_{t,r}$ satisfying

$$h_{t,0} = 1, \quad (217)$$

$$h_{t,j} = h_{t,j-1}(1 - z_{t,j}), \quad (218)$$

$$x_{t,j} = h_{t,j-1}z_{t,j}, \quad (219)$$

$$\sum_{j=1}^r x_{t,j} = 1, \quad (220)$$

$$n_{t+1} = \sum_{j=1}^r a_j q_{t,j} x_{t,j}. \quad (221)$$

The prefix variables say that no applicable index has yet occurred. No separate Boolean equations for h or x are required: induction forces their values to be bits.

Theorem 54.2 (Exact FRACSTRAN compiler). *Fix a reduced positive r -fraction program, positive integers n_0^{in} and n^{out} , and $T \in \mathbb{N}$. There is an explicitly given polynomial of degree at most four with at most one natural zero; it has a zero if and only if the program executes exactly T steps from n_0^{in} to n^{out} . The construction uses exactly*

$$V_{\text{frac}} = T(7r + 2) + 1, \quad (222)$$

$$E_{\text{frac}} = T(8r + 3) + 2 \quad (223)$$

variables and residuals. With $A = \max_j a_j$ and $B = \max_j b_j$, every solution coordinate is at most

$$\max\{1, A^T n_0^{\text{in}}, B\}. \quad (224)$$

Proof. Use configurations $n_0, \dots, n_T$ and endpoint equations $n_0 = n_0^{\text{in}}, n_T = n^{\text{out}}$. At every $t < T$ impose the division gadgets and (217)–(221). Lemma 54.1 forces all the division data and all z bits from n_t . The prefix recurrence then forces $h_{t,j} = 1$ precisely when none of $1, \dots, j$ is applicable. Consequently $x_{t,j} = 1$ precisely at the first applicable index, if one exists; all other selectors are zero. Equation (220) rules out a halted step. The output equation is therefore exactly the FRACSTRAN update.

Starting from the fixed positive input, this proves by induction that any solution is the actual run and all its auxiliaries are uniquely determined. In particular, each selected quotient is positive and every configuration remains positive. Conversely, an actual run supplies the uniquely determined division data and prefix bits and satisfies all equations. The final endpoint test is exact.

There are $T + 1$ configuration variables. Each step adds $5r$ division variables, $r + 1$ prefix variables, and r selectors, giving $(T + 1) + T(7r + 1)$ variables. Each index contributes six division equations and two prefix/selection equations; each step adds an initial prefix, a nonhalt test, and an output equation.

The two endpoint equations give (223). All residuals have degree at most two, including the output products $q_{t,j}x_{t,j}$.

Finally, each update satisfies $n_{t+1} \leq An_t$, so $n_t \leq A^t n_0^{\text{in}}$. Quotients are at most n_t , remainders and their slacks are below B , and all flags are bits. This proves (224). Sum the residual squares to obtain the polynomial. $\square$

[write] Repeated words. Part XII accelerates a prescribed repeated word of a FRACTRAN program on exponent vectors, priority included, with a certificate whose size is independent of the repetition count (Theorem 98.6); the raw-integer interface of the theorem above is not covered there.

The theorem uses “exactly T steps” to specify the length, not to require halting at time T . The next construction adds that requirement. For $T = 0$, the basic certificate has one variable and two equations and asserts equality of the positive endpoints.

54.4 First-halting certificates

For each denominator b_j , add three natural variables Q_j, U_j, V_j and two equations

$$n_T = b_j Q_j + U_j + 1, \quad (225)$$

$$U_j + V_j = b_j - 2. \quad (226)$$

They assert that division by b_j has a strictly positive remainder. If $b_j = 1$, the second equation has no natural solution, as it should: an integer fraction is always applicable.

Corollary 54.3 (Unique first-halting witness). *Adding (225)–(226) to the exact- T certificate gives a polynomial with a unique natural zero precisely when the first halt occurs at step T with the prescribed output. This adds $3r$ variables and $2r$ residuals, without increasing the degree bound.*

Proof. For $b_j \geq 2$, the equations force the remainder $U_j + 1$ to lie in $\{1, \dots, b_j - 1\}$, so quotient and remainder are unique, as are U_j, V_j . They are solvable precisely when $b_j \nmid n_T$. All denominators must fail. Earlier times were explicitly required to have an applicable fraction by (220), proving first halting rather than mere terminality of an unrelated state. $\square$

Example 54.4 (Priority and a complete halt). For the program $(3/2, 5/3)$ on input 8, the exact run is

$$8 \longrightarrow 12 \longrightarrow 18 \longrightarrow 27 \longrightarrow 45 \longrightarrow 75 \longrightarrow 125.$$

At 12 and 18, both denominators divide the current integer, but the first fraction must be chosen. At 125 neither is applicable. With $r = 2$ and $T = 6$, the basic certificate uses 97 variables and 116 residuals; the first-halting extension uses 103 variables and 120 residuals. Its largest witness coordinate is 125. The exact example is supplied as `fracran_first_halt.json`.

The height estimate gives witness bit lengths

$$O(1 + \log(n_0^{\text{in}} + 1) + T \log(A + 1) + \log(B + 1)).$$

Thus the bounded certificate has polynomial bit-size in the input, the program, and a unary horizon. This quantitative statement is stronger than a variable count alone. It does not provide a bound on the first halting time as a computable function of an arbitrary program and input.

55 Context-local Diophantine compilation of SKI rewriting [04]

55.1 A total natural coding of finite terms

Let SKI terms be the finite binary application trees with leaves S, K, I , under the usual context-closed rules

$$Ix \rightarrow x, \quad Kxy \rightarrow x, \quad Sxyz \rightarrow xz(yz). \quad (227)$$

Application associates to the left. We fix a concrete syntax; semantic equivalence of terms is not used as equality of their codes.

Define

$$\begin{aligned} \lceil S \rceil &= 0, & \lceil K \rceil &= 1, & \lceil I \rceil &= 2, \\ \text{App}(\ell, r) &= 3 + \frac{(\ell + r)(\ell + r + 1)}{2} + r. \end{aligned} \quad (228)$$

For natural n, ℓ, r , let the application residual be

$$C(n, \ell, r) = 2n - 6 - (\ell + r)(\ell + r + 1) - 2r. \quad (229)$$

It has degree two and integer coefficients.

Lemma 55.1 (Unique natural tree decoding). *Every natural number is the code of exactly one finite SKI term. For $n \geq 3$, there is exactly one pair $(\ell, r) \in \mathbb{N}^2$ with $C(n, \ell, r) = 0$, and both children are smaller than n . For $n < 3$, no such pair exists.*

Proof. Cantor's pairing function $\pi(\ell, r) = \frac{1}{2}(\ell + r)(\ell + r + 1) + r$ enumerates the diagonals $\ell + r = k$. On diagonal k , it takes the consecutive values $k(k + 1)/2, \dots, k(k + 1)/2 + k$, in increasing r . The next diagonal starts at the following integer. Thus it is a bijection $\mathbb{N}^2 \rightarrow \mathbb{N}$.

Equation $C = 0$ is exactly $n = 3 + \pi(\ell, r)$, giving the claimed unique pair and excluding $n < 3$. If $k = \ell + r$, then $\pi(\ell, r) \geq k$ for $k \geq 1$, and the assertion is also immediate for $k = 0$. Hence $\ell, r < n$. Strong induction on n now decodes a finite tree, with the three smaller codes interpreted as the leaves. The same induction and pairing uniqueness prove uniqueness. $\square$

There is no separate well-formedness witness. Moreover, $\text{App}(\ell, r) \geq \ell + r + 3$ shows inductively that the decoded number of tree nodes is at most $n + 1$. This is an effective bound, not a claim that decoding is polynomial in the *binary* length of n .

55.2 Prescribed schedules and context equations

A reduction schedule is a finite list

$$\sigma = ((R_0, \gamma_0), \dots, (R_{T-1}, \gamma_{T-1})), \quad R_t \in \{S, K, I\}, \quad \gamma_t \in \{L, R\}^*.$$

The path γ_t selects the redex in the term at time t . The schedule is *external data*, not an existentially chosen variable-length path. Define its total context depth

$$H = \sum_{t < T} |\gamma_t|,$$

and let n_I, n_K, n_S count its three rule types, so $n_I + n_K + n_S = T$.

Use global configuration variables $c_0, \dots, c_T$. During one descent through a context, suppose u, v are the old and new whole-context codes. For a left descent introduce the old selected child d , the sibling s , and the new selected child d' , and impose

$$C(u, d, s) = 0, \quad C(v, d', s) = 0. \quad (230)$$

Continue the descent with the pair (d, d') . For a right descent use

$$C(u, s, d) = 0, \quad C(v, s, d') = 0. \quad (231)$$

Each context level therefore contributes three variables and two residuals. Sharing the sibling variable forces the off-path subtree to be unchanged, without inspecting its interior.

At the selected root, use the following equations, with u, v now denoting old and new redex codes:

$$I : \quad C(u, 2, x) = 0, \quad v - x = 0; \quad (232)$$

$$K : \quad C(u, h, y) = 0, \quad C(h, 1, x) = 0, \quad v - x = 0; \quad (233)$$

$$S : \quad C(u, h, z) = 0, \quad C(h, g, y) = 0, \quad C(g, 0, x) = 0, \\ C(p, x, z) = 0, \quad C(q, y, z) = 0, \quad C(v, p, q) = 0. \quad (234)$$

The respective fresh-variable counts are 1, 3, and 7; their residual counts are 2, 3, and 6. Reusing the same argument code twice in the S rule represents actual syntactic duplication, not an obligation to create two independently encoded copies.

Theorem 55.2 (Context-local scheduled-reduction compiler). *Fix source and target codes $a, b \in \mathbb{N}$ and an external schedule σ . The context equations, root gadgets, and endpoint equations $c_0 = a$, $c_T = b$ have exactly one natural solution if the schedule is valid and transforms a into b , and none otherwise. Their sum of squares has degree at most four. The construction uses exactly*

$$V_{\text{SKI}} = T + 1 + 3H + n_I + 3n_K + 7n_S, \quad (235)$$

$$E_{\text{SKI}} = 2 + 2H + 2n_I + 3n_K + 6n_S. \quad (236)$$

In particular, $V_{\text{SKI}} \leq 8T + 3H + 1$ and $E_{\text{SKI}} \leq 6T + 2H + 2$, independently of the number of nodes inside untouched subtrees.

Proof. Fix a source code for one step. At each level of the scheduled descent, Lemma 55.1 forces the old selected child and its sibling. If that level is a leaf, the corresponding application equation is impossible. At the selected root, the old-side equations force the argument codes and test the required leaf I , K , or S . Thus an incorrect redex kind also makes the system impossible.

For a correct redex, the root output is uniquely determined: it is x for I or K , and $\text{App}(\text{App}(x, z), \text{App}(y, z))$ for S . All intermediate root variables are forced. Going back up the path, each new-side context equation uniquely forms an application with the already determined new child and old sibling. Hence the whole successor v and all context auxiliaries are unique and equal to the scheduled contraction. This is a two-way argument: a valid contraction supplies exactly those values and satisfies all equations.

Induct on the steps beginning at $c_0 = a$. A solution is precisely the prescribed valid reduction; its final code must equal b . Conversely, that reduction supplies its unique full witness. The counts follow by adding $T + 1$ configurations, three variables and two equations per context level, the listed root-gadget costs, and two endpoint equations. Every residual has degree at most two, proving the quartic statement. $\square$

Example 55.3 (A two-step root computation). The term $(SKI)I$ has code 635. The root schedule $(S, \varepsilon), (K, \varepsilon)$ gives

$$(SKI)I \longrightarrow (KI)(II) \longrightarrow I.$$

Here $T = 2, H = 0, n_S = n_K = 1$, so $V = 13$ and $E = 11$. The certificate `ski_SKII.json` has a unique solution and maximum coordinate 635.

Example 55.4 (A nonempty context). The term $I(IK)$ has code 91. Contracting an I at path R and then at the root gives $I(IK) \rightarrow IK \rightarrow K$. Thus $T = 2, H = 1, n_I = 2$, giving $V = 8$ and $E = 8$. The exact file is `ski_context.json`.

55.3 The unavoidable arithmetic-height qualification

A small number of integer coordinates does not guarantee that those integers have small binary representations. The next bound makes the qualification explicit.

Proposition 55.5 (A coarse height bound). *For a valid scheduled reduction with source code a , let $g = H + 3n_S$. Every witness coordinate is at most a number M satisfying*

$$\log_2(M + 1) \leq 2^g (\log_2(a + 1) + 4) - 4. \quad (237)$$

Thus the construction controls the number of coordinates linearly, but its general bit-height guarantee can be exponential in the amount of rebuilt context.

Proof. Decomposition into subterms never increases the current maximum code. An I or K contraction creates no new application at its root. An S contraction creates three, and each context level creates one on reconstruction, so at most g new applications are formed in temporal order.

For children at most M , formula (228) gives

$$\text{App}(\ell, r) + 1 \leq 16(M + 1)^2.$$

This deliberately loose inequality follows directly from $\ell + r \leq 2M$. If $L = \log_2(M + 1)$, one new application increases the running maximum to a value whose L is at most $2L + 4$. Starting from a and iterating that recurrence at most g times gives (237). All witnesses are old subterms, reconstructed subterms, or configurations, so they are covered. $\square$

Accordingly, Theorem 55.2 is not a polynomial-time verification theorem in a measure that charges only for schedule length while treating large integer values as free. The code-based locality is genuine, but it trades structural expansion for potentially large arithmetic values. Better tree or DAG encodings are a concrete research target, not something supplied by the present pairing construction.

55.4 Extension to finite first-order rewriting

Corollary 55.6 (Fixed-schedule finite-pattern rewriting). *Consider a finite first-order term signature with a disjoint, injective natural constructor coding, each constructor having a fixed-size unique-extension quadratic certificate. Fix finitely many rewrite rules in which every variable on the right occurs on the left, and fix a schedule of rule names and context paths. Then the scheduled reduction admits a unique-extension quadratic residual certificate whose size is linear in the total selected-context size plus the total sizes of the selected rule patterns and constructed right sides.*

Proof. Decompose the source along the fixed context and then along the selected left pattern. Constructor injectivity makes all subterm codes unique. Repeated occurrences of a pattern variable use one shared code, thereby testing their equality; incompatible occurrences make the system unsatisfiable. Every right-side variable has now been determined. Construct the right side using the unique-extension constructor gadgets and rebuild the context. This is the same one-step uniqueness argument as in Theorem 55.2; iteration gives the result. Summing constant costs per visited pattern, constructor, and context node gives the stated bound. $\square$

For a finite signature, such constructor codings can be obtained by tagged pairing, with fixed-arity tuples built from a fixed number of pairings. The corollary concerns ordinary first-order substitution at a fixed occurrence. It does not silently extend to binding syntax with capture-avoiding substitution, or identify different evaluation strategies of the lambda calculus.

56 Local FRACTRAN and combinatory-logic interfaces, and bounded rewriting [02, 01, 05]

[write] *Three subsections from the substrate surveys of manuscripts 02 (its Section 10.2), 01 (its Sections 9.3–9.4) and 05 (its Section 9.4). Manuscript 02’s local FRACTRAN interface certifies one step and the terminal condition; the complete bounded run with first-applicable priority is manuscript 04’s Theorem 54.2 above.*

56.1 A local single-fold FRACTRAN interface

For completeness, a second substrate illustrates how to avoid local multiplicity before confronting the difficult global problem. Fix a FRACTRAN list

$$\frac{p_1}{q_1}, \dots, \frac{p_m}{q_m}, \quad p_i, q_i > 0.$$

At positive integer state n , the first fraction producing an integer is applied. For every i , introduce the *unique* Euclidean quotient and remainder

$$p_i n = q_i a_i + r_i, \quad 0 \leq r_i < q_i.$$

All quotients and remainders are evaluated, not just those before or at the selected fraction. The applicable index j , if any, is characterized by $r_j = 0$ and $r_i \neq 0$ for $i < j$; the next state is a_j . The terminal condition is $r_i \neq 0$ for every i .

These finite arithmetic conditions have unique quotient–remainder witnesses and a unique selected index. The circuit construction therefore gives single-fold local step and terminal-state certificates. This does not imply a single-fold certificate for an unbounded FRACTRAN run. FRACTRAN’s role as a useful intermediate model in a full mechanized MRDP proof is established by Larchey-Wendling and Forster [14]; our discussion here isolates the distinct local uniqueness issue.

56.2 A local polynomial adapter for combinatory logic

The repository’s combinatory-logic development makes careful distinctions between source syntax, runtime syntax, and evaluation strategy. In particular, its SKI-to-Iota simulation is not declared reduction-reflecting or fully abstract, and its exact Coq equivalence uses a weak-call-by-value lambda model distinct from the scoped contextual one [4]. A new Diophantine adapter must not erase those distinctions.

There is nevertheless a useful elementary local construction. Define

$$\pi(a, b) = (a + b)^2 + a, \quad \text{App}(a, b) = 3 + \pi(a, b). \quad (238)$$

Give the three SKI leaves codes 0, 1, 2. Codes of valid terms are generated inductively by these leaves and App.

Lemma 56.1 (Injective polynomial application coding). *The map $\pi : \mathbb{N}^2 \rightarrow \mathbb{N}$ is injective. Moreover, $\text{App}(a, b) > \max(a, b)$, so validity of an alleged finite-term code is decidable by finite recursion on smaller codes.*

Proof. For $s = a + b$, the value lies in the interval $[s^2, s^2 + s]$. Consecutive such intervals are disjoint because $(s + 1)^2 > s^2 + s$. Thus the value determines s , then determines $a = \pi(a, b) - s^2$, and then $b = s - a$. This proves injectivity. Since $s^2 \geq s$ for natural s , the application code is greater than both children. To test validity, search finitely for a pair of children with the given application code and recursively test the unique pair if it exists; both recursive arguments are smaller. $\square$

For the head rule $Sxyz \rightarrow xz(yz)$, fixed valid input code u and proposed output code v can be related by the six quadratic equations

$$\begin{array}{lll} t_1 = \text{App}(0, x), & t_2 = \text{App}(t_1, y), & u = \text{App}(t_2, z), \\ l = \text{App}(x, z), & r = \text{App}(y, z), & v = \text{App}(l, r). \end{array}$$

All seven natural witnesses x, y, z, t_1, t_2, l, r are unique when they exist, because successive decompositions of the valid input determine the three arguments and then all constructions are functional. Their squared residuals give a quartic local rule. The same observation applies to the K and I head contractions.

Two restrictions must remain visible. First, the construction as stated has *validity of the input term as a domain promise*. Applied indiscriminately to arbitrary natural codes, it could decompose a numeral with invalid subterms. Adding an arbitrary MRDP representation of validity need not preserve unique fibres. Second, contextual reduction needs a redex position and a surrounding context, and arbitrary-length reduction needs an additional history mechanism. The local rule therefore gives a concrete starting component, not a completed canonical representation of unrestricted SKI or Iota evaluation.

56.3 What cannot be transported from a forward simulation alone

Suppose a compiler maps source steps into nonempty target traces. This proves preservation of source execution, but by itself it does not prove that every target accepting trace is the image of a source trace. Nor does it prove that one source trace has a unique target realization. A parsimonious transport requires a decoding or reflection theorem and control of administrative target steps.

For ProveIt, this means that the existing operational embeddings are valuable reusable components, but a fibre-preserving Diophantine bridge must state the evaluation relation it preserves and prove the missing reflection and canonicalization obligations. The present guarded-translation construction avoids that ambiguity by defining its substrate and its represented histories explicitly.

56.4 Bounded word rewriting and explicit term graphs

Fix a finite alphabet, a finite family of substring replacement rules, a horizon T , and an explicit maximum word length B . Store the word in B cells with a canonical blank suffix and a uniquely encoded length. At each step choose a rule and a position using a fixed one-hot code, check the matched substring and

size bound, and compute the uniquely padded next word. This is a finite deterministic verifier of the candidate choice history. Its circuit can be constructed by finite comparisons, multiplexers, and shifts of the B cells. By Theorem 57.1 it has a quadratic witness-preserving arithmetic encoding.

The objects counted here are *labelled derivations*, including the chosen rule and position, not merely their endpoint words. Distinct rules with identical replacements count separately unless the object definition first quotients them. For length-changing rules, the numerical positions of later redexes move; declaring two static position labels independent without accounting for that movement is unsound.

An analogous finite verifier is available for a specified bounded term-graph representation with deterministic allocation conventions. For tree representations of duplicating combinators, a bound on the number of reduction steps need not be a comparable bound on tree size. A size budget or a fully specified sharing representation is therefore essential. This article does not claim a linear-size arithmetic encoding of arbitrary tree expansion merely from a linear step bound.

Part VIII

Other substrates

Source and scope. [write] Five manuscripts close with a survey of further computational substrates reached by their methods. This Part prints manuscript 05's (Boolean verifiers, counter and register programs with zero tests, finite cellular automata, exact time and padding), manuscript 04's (counter machines, stacks, Boolean circuits and cellular automata in a finite box, a compositional statement), manuscript 01's (counter machines with finite control, finite arithmetic and Boolean circuits), manuscript 02's (fixed-horizon counter machines, finite Boolean spacetime substrates) and manuscript 06's (what transfers without a further theorem, an abstract compiler contract, and why doubling, stacks and rewriting need more work). The rewriting subsections of these surveys are in Part VII, and their boundary subsections in Part IX.

Overlaps. [write] Boolean circuits are treated by four manuscripts; manuscript 05's Theorem 57.1 (affine equalities, convex quadratic, bounded real relaxation) is the strongest statement, and the circuit encodings of manuscripts 01, 02 and 04 are unique-extension encodings of the same kind with quadratic gate equations, hence quartic rather than quadratic polynomials. Counter machines appear in manuscripts 01, 02, 03 (Part V), 04 and 05; manuscript 06 is the main treatment of counter programs (Parts I–III), and the others are cross-referenced. Bounded cellular automata are compiled as finite space–time tableaux by manuscripts 01, 02, 03 (Section 42), 04 and 05; manuscript 07 does not compile Rule 110 diagrams directly and explains why its queue certificates do not transport to a $(T + 1)$ -variable space–time polynomial (the end of Section 52).

57 Other computational substrates [05]

The preceding theorem is most informative when the arithmetic front end is reusable. We give precise extensions and state which parts retain the Petri-net commutation theorem. In every case, size and time bounds refer to an explicitly unrolled finite computation, not a fixed-dimensional universal equation.

57.1 A general Boolean verifier compilation lemma

Theorem 57.1 (Unique extension of a finite verifier). *Let C be a finite deterministic Boolean circuit whose input strings encode candidate computational objects, with exactly one input encoding for each*

object. There is an effectively constructed system of affine integer equalities in natural variables, of size linear in the number of inputs and gates, whose full natural solutions are in bijection with the inputs accepted by C . Its sum-of-squares polynomial is convex quadratic, and its nonnegative real solution set is bounded.

Proof. For each input bit p , introduce its complement $\bar{p}$ and impose $p + \bar{p} = 1$. This gives exactly one complement for each possible bit. Process gates topologically. A NOT gate is $p + y = 1$. For an AND gate, use (2), with output u . For an OR gate, first compute $u = p \wedge q$ by that gadget and impose $y + u = p + q$. Constants have equations $y = 0$ or $y = 1$. Finally set the acceptance output to one.

On natural assignments the gate values are forced Boolean by induction, and all slacks are unique by Lemma 2.3. Thus every accepted input has one full extension and every full solution decodes an accepted input. Over the reals the input bits are in $[0, 1]$. The AND bounds keep the output in $[0, 1]$, the NOT equation does the same, and the lower bound $u \geq p + q - 1$ makes the OR output at most one. Its nonnegativity is part of its domain. All auxiliary slacks are bounded as in Lemma 2.3. Hence the real feasible set is bounded. Scalarization proves the polynomial assertion. $\square$

The unique input-encoding assumption must not be skipped. Padded records with unconstrained unused bits, multiple syntactic descriptions of the same object, or ignored nondeterministic choices would all change the solution count. The lemma is a compiler for a specified verifier and specified coding convention, not a theorem that arbitrary encodings are automatically parsimonious.

57.2 Counter and register programs, including zero tests

A finite counter program with natural registers and finite control can be unrolled using the raw variables of Section 10. Increment and decrement are consumption–production updates. To enforce a control edge with source $s(a)$ and target $t(a)$, add a control variable q_i and equations

$$q_i = \sum_a s(a) e_{i,a}, \quad q_{i+1} = \sum_a t(a) e_{i,a},$$

with prescribed initial and final control values. Equivalent one-hot control places can be used instead.

For a transition a carrying a zero guard on register p , add

$$e_{i,a} x_{i,p} = 0. \tag{239}$$

This imposes the guard exactly on the selected branch. There is no auxiliary witness in the inactive branch, so uniqueness is unchanged. Nonzero/decrement guards are already expressed by consumption of a token. Thus a bounded counter-program execution has a direct parsimonious quartic sum-of-squares encoding, with finite control and every guard explicit.

For fixed numerical initial registers and a fixed horizon, the same extension can be made quadratic. Let $B_p = M_p + T \max_a v_{a,p}$, known from the raw induction. Replace (239) by the affine equation

$$x_{i,p} + s_{i,a,p} = B_p(1 - e_{i,a}), \quad s_{i,a,p} \in \mathbb{N}. \tag{240}$$

When $e_{i,a} = 1$, it forces $x_{i,p} = 0$. Otherwise it only enforces the already valid bound $x_{i,p} \leq B_p$, and its slack is unique. The coefficient B_p is a fixed integer for this instance. When the initial registers are free parameters, $B_p e_{i,a}$ is no longer jointly affine; the uniform-parameter presentation is then quartic unless another representation is supplied.

[write] Batch 63, 30 September 2026: another representation is supplied in the neighbouring report *liveness-beyond-halting* of this category (Report C, written in commit 2a34b1740 from batch-62 manuscripts 10 and 03 and batch-63 manuscript 01, its sources 10, 03 and 11). Its sources 03 and 11

keep a zero guard as the unsquared natural product $e_a x_{j(a)}$ (source 03) or $c_e x_{i(e)}$ (source 11), so that the transition relation of a counter machine stays of degree two uniformly in the registers, at the price of convexity and of a pure sum-of-squares form (its `lbh:qd:thm:compiler` and `lbh:ql:thm:compiler`, compared in its section `lbh:sec:threecompilers`, which quotes this remark). Squaring the guard, as (239) is squared in the sum of squares above, gives a quartic again.

[write] *Manuscript 05 continues here with its exact commutation theorem for interval guards, printed in Part I as Theorem 4.5 (Section 4.3), and its corollary for guarded programs, printed in Part II as Corollary 11.5 (Section 11.5).*

57.3 Finite cellular automata

Consider a one-dimensional deterministic cellular automaton on a ring of W cells, alphabet size q , radius r , and fixed local rule. Give each cell a one-hot alphabet encoding, initially constrained by a sum equal to one. The local rule can be compiled by testing each of its q^{2r+1} neighbourhood patterns with Boolean conjunctions and collecting patterns that produce each output symbol. Thus one time layer has a deterministic Boolean circuit of size

$$O(W(r+1)q^{2r+1}),$$

and T layers have T times this size. Apply Theorem 57.1, treating the initial one-hot rows as the candidate data and their sums as affine constraints. Every valid initial configuration has exactly one space–time history and exactly one arithmetic extension. Constraints on the final pattern select its preimages without introducing extra encodings.

The same construction works on a finite interval with explicitly specified boundary conditions. For an infinite quiescent-background automaton, one must first specify and justify a sufficient finite space–time region; the ring theorem alone does not supply that justification. Also, synchronous local updates are not a sequence of freely swappable asynchronous transitions. A schedule quotient for an asynchronous automaton needs its own partial-map semantics.

[write] *Manuscript 05's subsection on bounded word rewriting and explicit term graphs, which stands here in the manuscript, is printed in Part VII (Section 56).*

57.4 Exact time, first halting, and padding

All previous constructions use an exact horizon. For a deterministic program that halts in $t \leq T$ steps, a unique length- T record can be obtained by a distinguished stopped mode with a single mandatory padding action and frozen data. The first visit to stopped mode is the halting event; every later position is forced. The native time is then a uniquely determined observable.

Allowing arbitrary stuttering at arbitrary positions is different. It introduces multiple records of the same computation and can change the interaction with the commutation relation. Unused bits in a padded record must also be forced. A compiler for “at most T ” needs a canonical padding convention as part of its correctness specification, not just an existential choice of a shorter length.

58 Other computational substrates through the same interface [04]

The three main compilers illustrate separate difficulties: concurrent equivalence, priority, and context locality. The unique-extension viewpoint also gives a disciplined route to further substrates. The following constructions are elementary applications of Section 2.5, not claims of newly discovered universality.

58.1 Counter machines and stacks

An increment of a natural register has equation $R' = R + 1$. A nonzero decrement has $R = R' + 1$, with $R' \in \mathbb{N}$ supplying the guard. A zero branch has $R = 0$ and the appropriate unchanged-register equations. Finite control and labeled instruction choices can be encoded by one-hot selectors and Lemma 2.14. A fixed horizon gives one solution per valid labeled instruction trace. In a deterministic program the guards, rather than an arbitrary disjunction, must force the chosen branch.

For a stack over digits $1, \dots, B - 1$, use zero for the empty stack and encode the top digit as the least significant digit. A push of digit d and its inverse pop are

$$S' = BS + d, \quad S = BS' + d,$$

respectively. The selected digit is part of the labeled operation; it can alternatively be selected from the finite alphabet by a canonical circuit. Emptiness is $S = 0$. Starting with a valid stack, these equations preserve the no-zero-digit representation and hence the intended stack semantics. For a run of T pushes and pops, $S + 1 \leq B^T(S_0 + 1)$, so its code has $O(T \log B + \log(S_0 + 1))$ bits. Several stacks and finite control can be handled simultaneously.

A tape machine encoded by stacks additionally needs a convention for the current cell and implicit blank tails. Without that convention, distinct finite blank extensions would be distinct arithmetic objects representing the same tape. This is precisely the type of representational multiplicity that the compiler interface requires one to resolve explicitly.

58.2 Boolean circuits and cellular automata in a finite box

For Boolean inputs, AND and NOT gates have unique-extension equations

$$z = xy, \quad z + x = 1.$$

Output Booleanity follows from the input Booleanity. Thus an acyclic circuit has one gate assignment per input, and every gate residual is quadratic. A finite-alphabet cellular automaton in a fixed finite box can be represented by a Boolean encoding of each cell and a fixed local update circuit. With L cells, T rounds, and C gates per local update, a direct synchronous tableau uses $O(LTC)$ gate variables, together with the cell variables; all are bits. Boundary conditions and invalid finite-alphabet bit patterns must be specified, not inferred from an informal diagram.

Asynchronous local systems require more care. A proposed independence relation must guarantee that swapping actions preserves enabling and outcome. Disjoint complete read/write footprints are a sufficient condition. Disjoint write sets alone are not: an action may read a location changed by the other. Once a sound swap property is proved and the finite-action guards and updates have unique-extension certificates, the normal-form monitor of Section 14 applies independently of the particular state representation.

58.3 A compositional statement

Proposition 58.1 (Trace-faithful composition). *Suppose a finitely labeled transition system has unique-extension polynomial certificates for each labeled step, and a fixed independence relation with the enabled-swap property. Suppose further that a length- T operational tableau has no duplicate state encodings. Then adjoining the monitor equations yields a certificate whose solutions are in bijection with the independent-swap classes of the length- T endpoint traces. Quadraticization and the sum-of-squares construction preserve that bijection.*

Proof. Use canonical branch activation to form the labeled-step tableau, ensuring that every labeled trace has exactly one auxiliary extension. Append the deterministic flag tableau for its action word. By the normal-form theorem, precisely the least words survive. The enabled-swap property makes the endpoint language trace-closed, so every class still has its least word. Unique-extension quadratization adds no multiplicity. This is the proof of Theorem 15.2 with the specific token equations replaced by the assumed step certificates. $\square$

The hypotheses are substantive. In particular, state-coding injectivity and canonical inactive variables are not automatic consequences of a simulation theorem or of ordinary Diophantine definability.

59 Other computational substrates and safe adapters [01]

59.1 Counter machines with finite control and zero tests

The guarded-translation model can describe a fixed finite counter program directly. Add a program-counter coordinate q to the nonnegative registers. An instruction with source label i and target label j imposes the exact guard $q = i$ and displacement $j - i$ on that coordinate. An increment of register t adds $+1$ to that register with lower guard zero. A positive decrement branch has lower guard one and displacement -1 ; its zero branch uses the exact guard zero and displacement zero. Other registers have zero displacement and unrestricted nonnegative guards.

For the program-counter coordinate, $\ell = i$ and $v = j - i$ satisfy (9), because $j \geq 0$. For a decrement, its lower guard similarly guarantees natural output. Consequently the general envelope and history theorems apply to the resulting finite instruction alphabet.

This adapter is operational: the action labels retain which branch was taken, and each step agrees exactly with the specified counter instruction. It does not assert that all instructions commute. In a sequential program most useful adjacent instructions will be dependent, and the causal height may equal the ordinary execution length. Commuting-block acceleration applies only where the exact pair test certifies it.

To use first-halt semantics, the halt state has no outgoing action. Adding a free halt self-loop would create many finite histories after the same successful computation; neither determinism nor the earlier result would remove those genuinely different words. The classical use of counter or register machines in Diophantine arithmetization supplies the context for this adapter, not a new universality proof [8, 14].

59.2 Finite arithmetic and Boolean circuits

A second elementary adapter concerns deterministic acyclic circuits. A gate equation such as $t = u + v$, $t = uv$, or a Boolean NAND equation $t = 1 - uv$ has degree at most two. Boolean wires can be constrained by $t(t - 1) = 0$. In a topological order the input values determine every wire value uniquely; hence the sum of squared gate residuals has degree at most four and a unique witness for the entire circuit evaluation.

More generally, any acyclic list of quadratic functional constraints with uniquely determined natural outputs can be lifted this way. The statement is proved by induction in the gate order, not by assuming that low degree alone forces uniqueness. A multiplication equation can have many factorizations if its two inputs are not already fixed.

This familiar circuit observation covers finite Boolean state-transition circuits and finite cellular-automaton cylinders with specified boundary conditions. It is a bounded circuit construction, not an unbounded cellular-automaton single-fold theorem. Boundary values, update order, and the exact

cells represented must be part of the input specification. Unspecified boundaries would be additional semantic choices, not harmless auxiliary witnesses.

[write] *The remaining two subsections of this section of manuscript 01, a local polynomial adapter for combinatory logic and what cannot be transported from a forward simulation, are printed in Part VII (Section 56).*

60 Other substrates and the limit of the construction [02]

60.1 Fixed-horizon counter-machine certificates

A fixed finite counter program has configurations consisting of a finite control label and a fixed tuple of natural counters. Increment, decrement with zero test, and control transfer are described by finite Boolean combinations of polynomial equations and comparisons. The control and zero-test branches are mutually exclusive for a deterministic program.

By Lemma 2.8, each one-step relation has a single-fold arithmetic certificate when its endpoint configurations are fixed. For an *externally fixed* horizon T , introduce the $T-1$ intermediate configurations and the one-step certificates. The number of variables is $O(T)$ for a fixed program. A deterministic initial configuration determines every intermediate configuration uniquely, so this unrolled system is single-fold whenever the claimed run exists. For a halting certificate, earlier configurations must also be required not to be halting configurations, unless the chosen semantics already prevents transitions from them.

This is a family of polynomials whose arity grows with T . It is not a fixed polynomial with T as a free parameter. The substantive gain in Theorem 31.2 is precisely the elimination of that duration-dependent arity for a restricted, nontrivial class of updates.

[write] *Manuscript 02's local single-fold FRACTRAN interface, the second subsection of this section, is printed in Part VII (Section 56); its last three subsections, on the exponentiation boundary, universal first-halting computations and witness heights, are in Part IX (Section 64).*

60.2 Finite Boolean spacetime substrates

A finite Boolean circuit has unique internal wire values once its inputs are fixed. The same is true of a finite spacetime diagram obtained by unrolling a deterministic cellular automaton with a fixed local rule, specified boundary data, and an externally fixed time window. Boolean gate equations turn such a diagram into a single-fold quartic system. The witness count scales with the number of explicitly represented cells and time steps.

Again, this elementary fact does not provide a fixed-dimensional polynomial certificate for an unbounded computation. It clarifies a useful design principle across substrates: local single-foldness is often straightforward, whereas unique compression of variable-length execution is the real obstacle.

61 Other computational substrates: transfers and limits [06]

61.1 What transfers without any additional theorem

The same resource compiler applies to a fixed finite multiset-rewriting system: a rule consumes the multiset u_a and produces v_a . A discrete chemical reaction network with stoichiometric integer updates is an instance of these firing semantics. The claim concerns exact discrete firings and token availability, not continuous concentrations, reaction rates, or stochastic kinetics.

Finite control can be included by adding one place for each control state. A transition consumes the token of its source control state and produces the token of its target state. On markings with exactly one control token, this reproduces the finite-state transition condition, while the remaining places hold the original resources. The summary theorem then holds for the expanded Petri system; semantic equivalence on all expanded markings is stronger than one restricted to the one-control-token invariant. Accordingly the globally computed independence relation can be conservative for the intended invariant, but it is sound.

Bounded channels containing indistinguishable messages can also be expressed by resource counts, with a complementary capacity place when needed. A FIFO channel over distinct symbols is different: its state includes order, not just multiplicity. The present proof does not encode that order by pretending that each symbol count is sufficient.

61.2 A reusable abstract compiler contract

The resource argument suggests the following transfer principle.

Proposition 61.1 (Local unique summaries imply global unique compilation). *Suppose a class of word actions has finite-dimensional natural summaries with: constant summaries for letters and the empty word; exact composition and parameter-power relations given by fixed-size quadratic systems having a unique output and unique local auxiliaries for each valid input summary; and an endpoint-action relation with unique local witnesses. Assume the relations preserve validity of summaries. Then every fixed parametrized repetition expression in these actions has a single-fold quartic representation of its endpoint relation, with a number of witnesses linear in its syntax size, up to the fixed local summary dimensions.*

Proof. Allocate all child summaries and local auxiliaries as in Theorem 25.1. Structural induction on the syntax forces each valid summary and every local auxiliary. The endpoint contract gives the final zero-or-one solution condition. Every residual is quadratic, so its sum of squares is quartic. Summing the fixed local allocations proves the size bound. $\square$

This is a useful design contract, not a theorem that arbitrary machine models satisfy its hypotheses. The difficult clause is usually the unique polynomial representation of arbitrary powers, or the existence of a finite-dimensional exact summary in the first place. If local residuals have degree at most D instead, the same argument yields degree at most $2D$.

61.3 Why doubling, stacks, and rewriting require more work

If a primitive command doubles a counter, then its k th power applied to one produces 2^k . A generalization of our arbitrary-power interface to that command would in particular provide the corresponding witness-controlled polynomial specification of exponentiation. Our translation-only resource algebra avoids this issue; multiplication in (28) multiplies an externally supplied repetition count by a summary component, not the runtime counter value by itself at each iteration. These are different operations.

For stacks, a word of pushes and pops has cancellation information akin to a required prefix and a residual suffix, but these summaries are words of unbounded length, not fixed tuples of ordinary counters. Encoding the words as integers then introduces base powers, length constraints, and canonical coding questions. A proof of plain Diophantineness via MRDP would not preserve single-foldness automatically.

Lambda terms and SKI or Iota reductions introduce another issue: sharing and copying of syntax. A selected finite reduction sequence can be checked by an interpreter, but an exact compressed summary of arbitrary iterated rewriting need not have the affine principal-domain structure used here. Such

models are natural continuation targets for ProveIt, but they are not covered by the current compiler. A reliable extension should state its own summary and witness contracts before claiming transfer of our bounds.

Part IX

The boundary of fixed-arity compression

Source and scope. [write] Every construction of Parts I–VIII is either a family of polynomials indexed by a horizon, a height, a length or a schedule, or a fixed-arity polynomial for a restricted relation. Every manuscript ends by explaining why this does not yield one fixed-arity single-fold or finite-fold polynomial for unbounded universal computation. This Part prints manuscript 05’s boundary section first (base), then those of manuscripts 01, 02, 03, 04, 06 and 07.

Merged statements. [write] Four manuscripts prove that a single-fold (finite-fold) polynomial for universal halting exists exactly when every computably enumerable set has a single-fold (finite-fold) Diophantine representation: 05, 01, 03 and 04, with manuscript 05 adding a decidable trace-verifier form, manuscript 04 a degree-four form and manuscript 01 relations of any finite arity; manuscript 02 argues the first-halting form informally. The statement is printed once, as Theorem 62.1, in manuscript 05’s section; each other manuscript’s proof stays in its own section, after a note giving its letters. Five manuscripts prove that no total computable function bounds a witness of a universal (or any undecidable) representation; that is printed once, as Proposition 62.2, in the most general form (manuscript 02’s), with the proofs in place. Manuscript 06’s finite-fold substrate equivalence (Theorem 67.1) and its parameter-bound obstruction (Proposition 67.2) are different statements about separated one-counter schemes and are kept whole. Manuscript 07 proves neither theorem; it separates horizon-indexed from fixed-arity representations and remarks that its witness bound gives no computable bound on the unknown halting time.

What is and is not claimed. [write] The equivalence is a reduction, not a resolution: nothing in this report proves or refutes the existence of a single-fold or finite-fold universal polynomial. The single-fold *exponential* Diophantine representation of every enumerable set (Jones–Matiyasevich 1984) is formalized in ProveIt (Remark 64.2); the ordinary polynomial version is the open problem.

62 Fixed-dimensional compression and the finite-fold boundary [05]

The constructions above are a family of equations P_T , with a number of unknowns growing with T . A single polynomial with a fixed number of unknowns can represent reachability by MRDP or the repository’s exact-iteration machinery, but preservation of witness multiplicity requires a separate argument. We make the precise logical strength of one natural proposed compression explicit.

62.1 A decidable verifier with one successful history

Fix a deterministic universal machine U , a canonical encoding of its programs and inputs, and a canonical encoding of finite configuration lists. Define

$$\text{Trace}_U(e, x, h)$$

to mean that h codes the complete execution of program e on input x , beginning in its specified initial configuration and ending at the *first* halting configuration. There is no padding and no irrelevant annotation. A run halting immediately has the singleton list containing its initial configuration.

This relation can be chosen primitive recursive. For specificity, a useful polynomial pairing injection is

$$\pi(a, b) = (a + b)^2 + a. \quad (241)$$

It is injective: for $s = a + b$, its values lie in $[s^2, s^2 + s]$, and these intervals for distinct s are disjoint. Within an interval, a determines b . Encode the empty list as zero and a nonempty list as

$$\text{cons}(a, h) = 1 + \pi(a, h).$$

The tail is strictly smaller than the code. Decoding is by bounded search, rejecting numbers that are not valid codes, and checking the whole list requires only bounded iteration. Choose a standard machine encoding with primitive-recursive initial-configuration, one-step, and halting tests. The verifier checks all adjacent pairs and verifies that no earlier configuration halted. It is therefore primitive recursive.

Determinism and the absence of padding imply

$$\text{for each } (e, x), \quad \#\{h : \text{Trace}_U(e, x, h)\} \leq 1. \quad (242)$$

Existence of this unique history is equivalent to $U(e, x)$ halting. The history can be enormous; decidability of a proposed finite history says nothing about deciding whether some such history exists.

62.2 An exact equivalence theorem

Theorem 62.1 (Universal halting and the single-fold and finite-fold problems; manuscripts 05, 01, 03 and 04). *Fix a universal programming system and let $K(e, x)$ mean that program e halts on input x . The following statements are equivalent.*

- (a) *Every computably enumerable subset of $\mathbb{N}$ has a finite-fold polynomial Diophantine representation over $\mathbb{N}$.*
- (a') *Every computably enumerable relation of finite arity over $\mathbb{N}$ has a finite-fold polynomial Diophantine representation over $\mathbb{N}$.*
- (b) *There is one integer polynomial $H(e, x, z)$, with a fixed finite tuple z , representing universal halting $K(e, x)$ and having finitely many z -solutions for every fixed (e, x) .*
- (b') *There is such a polynomial H of degree at most four.*
- (c) *There is one integer polynomial $V(e, x, h, y)$, with a fixed finite tuple y , representing the decidable relation Trace_U of Section 62, with finitely many y -solutions for every fixed (e, x, h) .*

The statements remain equivalent when “finite-fold” and “finitely many” are replaced throughout by “single-fold” and “at most one”; in the single-fold form of (b), the natural zero set of $H(e, x, \cdot)$ is a singleton when $K(e, x)$ holds and empty otherwise.

Sources, letters and proofs. [write] Manuscript 05 states $(a) \Leftrightarrow (b) \Leftrightarrow (c)$ for a deterministic universal machine U with the canonical first-halting histories defined above; its proof follows. Manuscript 01 proves $(a') \Leftrightarrow (b)$ for a universal interpreter of register programs with first-halt semantics; its (i) is (a') and its (ii) is (b) (proof in Section 63). Manuscript 03 proves $(a) \Leftrightarrow (b)$ for a universal deterministic interpreter of an acceptable enumeration; its (i) is (a) and its (ii) is (b) (proof in Section 65). Manuscript 04 proves $(a) \Leftrightarrow (b) \Leftrightarrow (b')$ for an effective universal programming system with halting relation $H(e, x)$; its (i), (ii),

(iii) are (a), (b), (b') (proof in Section 66). The implications (a') $\Rightarrow$ (a) and (b') $\Rightarrow$ (b) are immediate, and (b) $\Rightarrow$ (a') is manuscript 01's argument; with the cycles proved by manuscripts 05 and 04, all five statements are equivalent. Manuscript 02 argues the first-halting form informally (Section 64.2), and manuscript 06 proves an equivalence for its own substrate (Theorem 67.1). The theorem is a reduction: it does not say whether the statements are true. The formalized exponential counterpart is recorded in Remark 64.2.

Proof of (a) $\Leftrightarrow$ (b) $\Leftrightarrow$ (c), both versions (manuscript 05). (a) $\Rightarrow$ (c). The set of triple codes

$$S = \{\pi(e, \pi(x, h)) : \text{Trace}_U(e, x, h)\}$$

is computably enumerable, indeed decidable using the chosen bounded decoding. By (a), there is a polynomial $Q(n, y)$ with finite fibres representing S . Set

$$V(e, x, h, y) = Q(\pi(e, \pi(x, h)), y).$$

This is an integer polynomial because the code map is polynomial. Its natural y -fibre at a fixed triple is exactly the corresponding Q -fibre, so it has the required finiteness and represents exactly the trace relation.

(c) $\Rightarrow$ (b). Existentially quantify h as one more witness and use V itself as the halting polynomial. For fixed (e, x) , a nonhalting computation has no possible h , and a halting computation has exactly one possible h by (242). Above this unique value there are only finitely many y by (c). Hence the full (h, y) -fibre is finite and is nonempty exactly for halting inputs.

(b) $\Rightarrow$ (a). For a computably enumerable set A , choose a program e_A that halts on x exactly when $x \in A$. Specializing the program parameter gives the polynomial $H(e_A, x, z)$ with finite fibres representing A .

Every step also preserves “at most one”: polynomial substitution does not add witnesses, specialization does not add witnesses, and in the middle implication there is at most one admissible h as well as at most one admissible y . This proves the single-fold version. $\square$

[write] A one-parameter refinement. Part XIII proves that finite-fold (single-fold) representability of the least-reservoir relation of one fixed reaction network is equivalent to statements (a)–(c) (Theorem 118.1); the canonical witness there is one number instead of a history.

[write] A rank-function form. Part XIV proves that a single-fold (finite-fold) representation of the graph of one total, monotone, Boolean function computable in polynomial time, the rank function of a universal one-router system, is equivalent to statement (a) in its single-fold (finite-fold) version (Theorem 133.1); no history argument remains.

This is a useful obstruction to an overambitious compiler contract. A fixed polynomial with finite fibres for a perfectly decidable first-halting trace verifier is already as strong as the general finite-fold conjecture. Calling the input a “verified finite history” does not make the required *uniform arithmetic representation* harmless. The verifier is fixed while its history parameter is unbounded, and projecting that parameter yields universal halting.

The theorem is an equivalence and a design boundary, not a proof that such a polynomial cannot exist. The finite-fold and single-fold questions are the classical stronger representation problems discussed in [9, 10]. Nothing in our bounded family contradicts a positive resolution of either question. It simply does not furnish that resolution.

62.3 Why obvious compression arguments lose multiplicity

A common informal argument says: encode a long execution in one integer, represent the digit tests Diophantinely, and combine all equations. This can correctly prove *existence*. It does not establish that a fixed history code has only one, or only finitely many, auxiliary arithmetic encodings. A representation of an exponentiation or sequence-decoding relation may have uncontrolled fibres. Each such relation needs its own finite-fold or single-fold theorem before this property can be composed.

Likewise, choosing “the least witness” at the semantic level is not a polynomial construction. Its minimality condition must itself be represented, and that representation can introduce further witnesses. The normal-form selection in Section 9 avoids this particular problem because it is an explicit finite-state recurrence unrolled to a fixed length; the number of variables is allowed to grow.

Proposition 62.2 (No computable witness-height bound; manuscripts 02, 01, 03, 04 and 05). *Let $A \subseteq \mathbb{N}$ be undecidable and suppose*

$$x \in A \iff \exists \mathbf{w} \in \mathbb{N}^m P(x, \mathbf{w}) = 0.$$

There is no total computable function $B(x)$ such that every positive instance has some solution with all coordinates at most $B(x)$. In particular, let $P(e, x, z)$ be any fixed polynomial, with a fixed finite witness arity, representing universal halting, with no assumption on the multiplicity of its witnesses. There is no total computable function $B(e, x)$ with the following property: whenever $U(e, x)$ halts, there is a zero of $P(e, x, \cdot)$ whose every coordinate is at most $B(e, x)$.

Sources. [write] The first statement is manuscript 02’s, for an arbitrary undecidable set; the second is the form stated by manuscripts 05 (whose proof follows), 01, 03 and 04, whose proofs are printed in their own sections below. The same finite-box search proves both. Manuscript 06 proves the corresponding statement for the parameters of a separated one-counter scheme (Proposition 67.2), and manuscript 07 records the point as a remark: its witness bound depends on T and supplies no computable bound on the unknown halting time (Section 68). As manuscripts 01, 02, 03, 04 and 05 each observe, finite-foldness alone gives no computable bound either, and the explicit bounds of the bounded constructions pose no conflict, since the horizon, height or log is given with the instance.

Proof of Proposition 62.2 for universal halting (manuscript 05). Such a function would decide halting. Compute $B(e, x)$, enumerate the finite box $\{0, \dots, B(e, x)\}^{\dim z}$, and evaluate P exactly at each tuple. A zero exists in the box exactly when the machine halts, by the assumed representation and bound. This contradicts the undecidability of universal halting. $\square$

[write] Specializations. Proposition 91.13 (tile assembly) and Theorem 115.4 (reaction reservoirs) are cases of this proposition, and so is Theorem 132.1(iv) (routing odometers, Part XIV).

Even a bound on the *least available* witness would be impossible in that sense; it is not necessary to bound every zero. Finite-foldness does not provide an effective bound of this kind. In contrast, the explicit bounds of Theorem 11.2 depend on the given horizon and therefore pose no conflict.

63 The boundary of unbounded canonicalization [01]

63.1 Infinitely many histories can be semantically unavoidable

Proposition 63.1 (An infinite history fibre). *There is a two-action one-counter system with infinitely many distinct finite trace classes from a fixed state to itself. Therefore an unbounded representation in bijection with all its finite histories cannot be finite-fold at that pair of endpoints.*

Proof. Use an increment a enabled at every natural marking and a decrement b enabled at positive markings. They are dependent by Proposition 5.3. Starting at zero, each word $(ab)^k$ executes and returns to zero. The words for distinct k have different lengths and hence belong to different trace classes. A bijection with all these classes requires infinitely many natural roots at endpoints $(0, 0)$. $\square$

This does not disprove finite-fold representability of the endpoint relation. In this example an endpoint relation can have a very simple existential description. It proves only that “preserve every finite history” and “have finitely many roots per endpoint” are sometimes incompatible specifications.

63.2 Semantic uniqueness does not canonicalize a coding base

The next observation isolates a different source of multiplicity. It is an elementary warning about one coding architecture, not a no-go theorem for all possible codes.

Proposition 63.2 (CRT base multiplicity). *Fix a nonempty finite list $a_0, \dots, a_{t-1} \in \mathbb{N}$. There are infinitely many distinct pairs (B, A_B) which encode that same list by residues modulo*

$$m_i = 1 + (i + 1)B,$$

even if A_B is required to be the least nonnegative simultaneous CRT representative.

Proof. Choose any sufficiently large positive multiple B of $t!$ so that every $a_i < m_i$. For $i < j$, a common divisor g of m_i, m_j divides $(j - i)B$. But $\gcd(g, B) = 1$, since g divides $1 + (i + 1)B$. Thus g divides $j - i$. Every positive divisor of $j - i$ divides $t!$ and hence B , so $g = 1$. The moduli are pairwise coprime.

The Chinese remainder theorem therefore gives a unique A_B with $0 \leq A_B < \prod_i m_i$ and $A_B \equiv a_i \pmod{m_i}$ for all i . Infinitely many choices of B satisfy the stated conditions. They give distinct pairs regardless of possible coincidences among the A_B values. $\square$

A canonical least residue fixes A relative to B , not B itself. The example explains why extracting a unique run from a trace-coding equation is weaker than proving uniqueness of the complete arithmetic certificate. Our finite-height compiler avoids coding bases altogether; the accelerator avoids storing a list of repeated states.

63.3 A sharp universal test for a single-fold compiler

Let U be a fixed deterministic universal interpreter for register programs, with first-halt semantics. Let

$$K(e, x) \iff U \text{ with program code } e \text{ and input } x \text{ eventually halts.}$$

We use only the defining universal property: every computably enumerable relation has a recognizer simulated by some fixed program code. This is standard computability infrastructure, also underpinning the machine-to-Diophantine routes cited earlier [14, 8].

Theorem of manuscript 01: “Universal halting characterizes single-fold MRDP”. [write] Manuscript 01 states here, for the relation K just defined, that (i) every computably enumerable relation of finite arity over $\mathbb{N}$ has a single-fold ordinary Diophantine representation if and only if (ii) one ordinary integer polynomial $P(e, x; w)$ with a fixed finite witness tuple has, at each (e, x) , a singleton natural zero set if $K(e, x)$ and an empty one otherwise; and the same with “finite-fold” and a nonempty finite zero set. This is (a') $\Leftrightarrow$ (b) of Theorem 62.1, where the statement is printed once; manuscript 01’s proof follows.

Proof of (a') $\Leftrightarrow$ (b) of Theorem 62.1, both versions (manuscript 01). The halting relation K is computably enumerable, so (i) gives (ii).

Conversely, let $R \subseteq \mathbb{N}^k$ be computably enumerable. For $k \geq 1$, iterating the polynomial injection π of Lemma 56.1 gives a polynomial injection $C_k : \mathbb{N}^k \rightarrow \mathbb{N}$ with a computable inverse on its image. A program can decode a valid encoded tuple and simulate a recognizer for R ; on an invalid code it can diverge. Let e_R be its fixed code. Then

$$R(a_1, \dots, a_k) \iff K(e_R, C_k(a_1, \dots, a_k)) \iff \exists w P(e_R, C_k(a_1, \dots, a_k); w) = 0.$$

Substituting a constant and a polynomial into an integer polynomial gives an integer polynomial. No new existential variable is introduced, so the entire fibre is exactly the fibre of P at that substituted pair. A singleton remains a singleton and a finite set remains finite. Nullary relations are handled by a fixed dummy input. This proves the reverse implication in both versions. $\square$

The theorem identifies the precise strength of a hoped-for universal fixed-arity canonical halting compiler. It would resolve the single-fold principle discussed by Matiyasevich, or its finite-fold version, rather than being a routine corollary of an ordinary exact-iteration interface [9, 12]. The explicit results of this report establish neither premise (ii). They instead provide fixed-arity compression for commuting blocks and a parsimonious family at bounded causal height.

63.4 Witness bounds are a separate obstruction

Proposition of manuscript 01: “No computable global witness-height bound”. [write] For any integer polynomial $P(e, x; w)$ representing the halting relation K , this is the universal-halting case of Proposition 62.2; manuscript 01’s proof follows.

Proof of Proposition 62.2 for universal halting (manuscript 01). If such B existed, compute it and test P on the finite box $\{0, \dots, B(e, x)\}^{\dim w}$. A root would certify halting, and exhaustion without a root would certify nonhalting, contradicting undecidability. The argument uses only existence representation; it does not assume unique or finite fibres. $\square$

There is no conflict with Proposition 20.3: that proposition concerns a fixed bounded-height instance family, not a representation of unrestricted halting. Equally, finite-foldness by itself would not provide a computable bound on where its finitely many witnesses lie.

64 The exponentiation boundary and universal first halting [02]

[write] *The last three subsections of manuscript 02’s section “Other substrates and the limit of the construction” (Section 60, Part VIII).*

64.1 The exponentiation boundary

The polynomial trajectory hypothesis cannot simply be dropped. Even the single assignment

$$y' = 2y, \quad y_0 = 1$$

has the exact-time graph $y = 2^n$. This trajectory is exponential, not polynomial in n .

Proposition 64.1 (A precise implication of a stronger accelerator). *If a fixed-dimensional single-fold ordinary polynomial accelerator covered this doubling loop uniformly in n , then the graph $y = 2^n$ would have a single-fold ordinary polynomial representation. Combined with the classical single-fold exponential normal form, this would yield single-fold ordinary polynomial representations of all enumerable sets.*

Proof. The first assertion follows by specializing the accelerator’s initial state to 1 and its guard to *true*. For the second, use the classical single-fold normal form in which exponentiation is isolated in one relation $v = 2^u$ [10, p. 588]. Substitute the hypothetical single-fold polynomial graph certificate for that relation. The original outer tuple is unique, and the new graph certificate has a unique extension for its fixed (u, v) . Thus the combined tuple remains unique. Combine the resulting finite polynomial equations by a sum of squares, or first lower their degrees by canonical arithmetic wires. $\square$

The same issue arises for variable-base multiplication $y' = ay$, which contains doubling as a specialization. The proposition is an implication, not a claimed impossibility theorem. It identifies the strength of an extension that the present paper has *not* proved. The literature on finite-fold representations explicitly distinguishes ordinary polynomial and single-fold exponential results [9].

Remark 64.2 ([write] The formalized exponential counterpart). The classical input used here — every recursively enumerable set has a single-fold unary exponential Diophantine representation (Jones–Matiyasevich 1984) — is formalized in ProveIt as `JM1984.RM.re_sfu` and `JM1984.RM.re_single_equation` (`Computability/HilbertTenthProblem/Lean/Diophantine/Paper1984/DPR.lean`), with the single-fold predicate `JM1984.Exp.SFU`. The reductions in this report are not formalized.

[write] Part XIV proves the analogous equivalence for the graph of one monotone Boolean rank function computable in polynomial time, in both the single-fold and the finite-fold version, without using this normal form (Theorem 133.1).

64.2 Universal first-halting computations

A related conditional statement is useful when evaluating proposed “substrate-independent” encodings. Suppose one has a single-fold polynomial representation of the input–time–output relation for the *first halt* of a fixed universal deterministic machine. For each input program and data, there is at most one first-halting time and output. Projecting those quantities, canonically encoding any signed components, and specializing the program therefore gives a single-fold polynomial representation of its halting set. Universality then gives such a representation for every enumerable set.

Replacing single-fold by finite-fold gives the corresponding finite-fold implication: there is at most one first-halting computation, and finitely many auxiliary witnesses for it. Thus a universal finite-fold first-halt compiler would settle a genuinely general representation question, rather than merely repackage ordinary MRDP. Our accelerator is deliberately restricted to supplied polynomial or fixed-residue polynomial trajectories.

64.3 A height bound cannot be silently added to universality

Proposition of manuscript 02: “Witness-height obstruction”. [write] Manuscript 02’s statement, for an arbitrary undecidable set $A \subseteq \mathbb{N}$ represented by $P(x, w)$, is the first statement of Proposition 62.2, printed there once; its proof follows.

Proof of Proposition 62.2 (manuscript 02). Compute $B(x)$ and check the finitely many tuples in $\{0, \dots, B(x)\}^m$. A found zero certifies membership; if no zero is found, the assumed bound certifies nonmembership. This would decide A . $\square$

Finite-foldness alone does not give such a computable height bound. Our height and bit-complexity bounds are possible because we treat a decidable specified-input class, not an arbitrary universal halting predicate.

65 The exact boundary of uniform single-fold compression [03]

65.1 The quantifiers cannot be interchanged

The construction proved so far has the form

$$\forall L \exists P_L \in \mathbb{Z}[X_1, \dots, X_{3L}, Y_1, \dots, Y_{W_L}] \quad \text{with a canonical fiber at every fixed log.} \quad (243)$$

The number of variables grows with L . For a fixed program we similarly have one polynomial per horizon. This is not the assertion that one polynomial with one fixed finite list of witnesses recognizes all halting computations while preserving unique fibers.

For a set $A \subseteq \mathbb{N}$, a *single-fold Diophantine representation* is a polynomial $F(x, y_1, \dots, y_s)$ such that $x \in A$ precisely when there is a natural root in y , and there is at most one such tuple for every x . A *finite-fold representation* requires finitely many such tuples instead of at most one. The general single-fold/finite-fold compression questions are not settled by MRDP or by determinism of the represented computation [9]. In particular, introducing a code for a unique trace does not prove that the auxiliary variables used to decode and check that code are unique.

Theorem of manuscript 03: “A universal substrate is the exact bottleneck”. [write] Manuscript 03 fixes a universal deterministic interpreter $U(e, x)$ for an acceptable enumeration of partial computations and states that (i) every computably enumerable subset of $\mathbb{N}$ has a single-fold natural-number Diophantine representation if and only if (ii) one integer polynomial $F(e, x, Y_1, \dots, Y_s)$, with fixed s , has natural roots in Y that are unique when $U(e, x)$ halts and nonexistent otherwise; and the same with “finite-fold/finitely many”. This is (a) $\Leftrightarrow$ (b) of Theorem 62.1, where the statement is printed once; manuscript 03’s proof follows.

Proof of (a) $\Leftrightarrow$ (b) of Theorem 62.1, both versions (manuscript 03). For (i) $\Rightarrow$ (ii), encode pairs by the injective polynomial

$$\pi(e, x) = (e + x)^2 + e.$$

Its injectivity follows because for $n = e + x$ its possible values form the interval $[n^2, n^2 + n]$, and these intervals for successive n are disjoint; within one interval the value fixes e , then x . The set

$$K_\pi = \{\pi(e, x) : U(e, x) \text{ halts}\}$$

is computably enumerable. Apply (i) to obtain a single-fold polynomial $G(z, Y)$ for that set and substitute $z = \pi(e, x)$. Injectivity ensures that the result recognizes exactly the desired halting relation and preserves each witness fiber.

For (ii) $\Rightarrow$ (i), let A be computably enumerable. By universality there is a fixed program index e_A such that $U(e_A, x)$ halts exactly when $x \in A$. Specializing $e = e_A$ in F gives a single-fold polynomial for A . Both operations preserve finiteness of fibers as well, proving the finite-fold version. $\square$

The theorem is an elementary reduction exposing where an ambitious extension would have to make genuinely new progress. It is not a solution of either conjecture. The source determinism used in Theorem 41.1 eliminates multiple bounded executions, but says nothing by itself about the auxiliary fibers of a fixed-variable arithmetization of arbitrary run length.

65.2 An unavoidable witness-height obstruction

Proposition of manuscript 03: “No computable global witness bound”. [write] For any fixed polynomial $F(e, x, Y)$ representing the universal halting relation, without any assumption on multiplicity, and a candidate bound $h(e, x)$, this is the universal-halting case of Proposition 62.2; manuscript 03’s proof follows.

Proof of Proposition 62.2 for universal halting (manuscript 03). Given (e, x) , compute $h(e, x)$ and test the finitely many natural tuples in $[0, h(e, x)]^s$. Polynomial evaluation is decidable. By the supposed property, the search finds a root exactly when $U(e, x)$ halts, deciding the universal halting problem. This is impossible by the usual diagonal argument: a decider for halting would allow a program that halts precisely when the decider predicts that the same program on its own code does not halt. $\square$

There is no conflict with Theorem 39.2. That theorem takes a fixed finite log as input, including the values already appearing in it. The machine theorem fixes a horizon. Neither supplies a total computable bound on the first halting time of an arbitrary universal computation. Even finite-fold representability, by itself, would not make the maximum root computable.

A concrete formalization target is therefore a hierarchy of interfaces: canonical local gadgets; canonical finite logs; canonical horizon-indexed machine runs; ordinary fixed-variable Diophantine reachability by MRDP; and, separately, the presently unproved finite-fold or single-fold upgrade of that last interface. An implementation should not erase these distinctions behind a common “Diophantine” label.

66 Unbounded computation and the fixed-arity frontier [04]

66.1 What MRDP gives for the present substrates

For any fixed effective substrate, the relation

$$\text{Reach}(a, b) \iff \text{some finite valid execution transforms } a \text{ into } b$$

is computably enumerable: enumerate finite candidate histories and check each one. For SKI, every natural code is a finite term, and both redex checking and a finite contextual step are effective. For FRACSTRAN and Petri nets the finite checks are explicit arithmetic procedures. MRDP therefore supplies a fixed finite-witness polynomial for each such reachability relation; program or net descriptions can also be included as natural inputs to a uniform interpreter.

This implication is classical, and the ProveIt exact-iteration interface is another route when the one-step relation has already been connected to its Diophantine API [3]. Neither route automatically preserves the one-to-one correspondences constructed here.

66.2 The difference between a family and one polynomial

Definition 66.1 (Finite-fold and single-fold). A fixed polynomial $P(\mathbf{a}, \mathbf{y}) \in \mathbb{Z}[\mathbf{a}, \mathbf{y}]$ represents a relation R if

$$R(\mathbf{a}) \iff \exists \mathbf{y} \in \mathbb{N}^m P(\mathbf{a}, \mathbf{y}) = 0.$$

It is finite-fold if the set of such $\mathbf{y}$ is finite for every fixed $\mathbf{a}$, and single-fold if that set has at most one element. The number m is fixed independently of $\mathbf{a}$.

The present FRACTRAN and scheduled-SKI polynomials are single-fold *for each fixed horizon or schedule*. But their number of variables grows with that external data. They do not produce one fixed polynomial $P(T, a, b, y)$ with fixed m and the same uniqueness guarantee. The Petri-net polynomial intentionally permits multiple solutions, one for each distinct class, while giving exactly one solution per class.

The fixed-arity requirement is the essential frontier. Single-fold specification and its connection with eliminating exponentiation are discussed as substantive research questions by Cantone, Cuzziol, and Omodeo [10]. Nothing in the bounded constructions here supplies that missing elimination or settles the general finite-fold or single-fold conjectures.

66.3 Three tempting but invalid shortcuts

First, a deterministic machine has only one run, but a number-theoretic representation of that run may have many witnesses. The omitted equation $zu = 0$ in the divisibility gadget gives a concrete example: the semantic branch remains unique, while the arithmetic fiber becomes infinite.

Second, representing a finite list by a variable base often introduces many choices of sufficiently large base and padding. One may still prove existential equivalence, but a uniqueness argument must specify and enforce a canonical base, length, and padding. Merely saying “choose the least encoding” is not a Diophantine construction: minimality introduces a statement excluding all smaller candidates, whose compilation and fiber behavior must themselves be justified.

Third, a forward operational simulation is not a reduction-reflecting equivalence. Extra target reductions can create arithmetic witnesses not corresponding to source computations. This matters for compiler chains involving different lambda-calculus evaluation strategies, or the distinction between a pure Iota source and a runtime syntax containing auxiliary combinators. The inspected repository explicitly distinguishes several of these semantics [4]; our scheduled local theorem does not bypass those obligations.

66.4 A precise universal reduction of the remaining problem

Theorem of manuscript 04: “Universal-halting scope equivalence”. [write] Manuscript 04 fixes an effective universal programming system with halting relation $H(e, x)$ and states, for either choice of “finite-fold” or “single-fold”, the equivalence of (i) every computably enumerable subset of $\mathbb{N}$ has a representation of that kind; (ii) $H(e, x)$ has one by one fixed polynomial; (iii) $H(e, x)$ has one by a fixed polynomial of degree at most four. These are (a), (b) and (b') of Theorem 62.1, where the statement is printed once; manuscript 04's proof follows.

Proof of (a) $\Leftrightarrow$ (b) $\Leftrightarrow$ (b') of Theorem 62.1 (manuscript 04). Assume (i). Encode pairs by the Cantor pairing function π from Section 55. The set $\{\pi(e, x) : H(e, x)\}$ is computably enumerable, so it has the assumed representation. Adjoin a variable z and the equation

$$2z - (e + x)(e + x + 1) - 2x = 0.$$

For fixed e, x , this determines exactly one natural z . Combining this equation with the representation of the paired set preserves its fiber cardinality and gives (ii).

Assume (ii). For a computably enumerable set S , choose the index e_S of a program halting exactly on S . Substitute the fixed integer e_S for the program coordinate. The remaining polynomial represents S and has exactly the same fibers, proving (i).

Finally, (ii) implies (iii) by unique-extension quadratization and summing squares, while (iii) trivially implies (ii). $\square$

This is a scope theorem, not a resolution. It identifies why a successful fixed-arity witness-faithful compiler for a universal substrate would be much stronger than any of the horizon-indexed families above. Restricting degree to four does not remove the main obstruction once an arbitrary finite number of variables is allowed.

66.5 No computable global witness-height bound for halting

Theorem of manuscript 04: “Witness-height obstruction”. [write] For $P(e, x, y)$ of fixed finite witness arity representing the halting relation of a universal programming system, this is the universal-halting case of Proposition 62.2 (printed there as a proposition); manuscript 04’s proof follows.

Proof of Proposition 62.2 for universal halting (manuscript 04). If such B existed, compute it on (e, x) and enumerate the finite box $\{0, \dots, B(e, x)\}^m$. Evaluate the integer polynomial on every tuple. A zero would certify halting; if none existed, the assumed bound would certify nonhalting. This would decide the universal halting problem, a contradiction. $\square$

Notice that the hypothesis asks only for *one* small witness per positive instance, not for a bound on every witness. Finite-foldness does not imply such a computable bound: a finite set can have an extremely large maximum with no computably uniform control. Conversely, the bounded-horizon height estimates in this article are entirely compatible with the obstruction because the horizon is supplied as part of the compiled instance, and no computable universal bound on halting horizons is provided.

67 The exact finite-fold obstruction [06]

Single-foldness for fixed schedule parameters is strong enough to prevent compiler-generated redundancy, but it cannot settle the general finite-fold problem. The preceding normal forms turn this limitation into an exact logical equivalence rather than a vague warning.

Theorem 67.1 (Finite-fold substrate equivalence). *For a relation $R \subseteq \mathbb{N}^a$, the following are equivalent:*

- (i) *R has a finite-fold Diophantine representation over $\mathbb{N}$.*
- (ii) *There is a fixed separated one-counter scheme E of depth at most four, with input parameters x and witness parameters k , such that*

$$x \in R \iff \exists k \in \mathbb{N}^b \ 0 \xrightarrow{\llbracket E \rrbracket_{(x,k)}} 0,$$

and for each fixed x there are only finitely many successful parameter tuples k .

The same equivalence holds with “finite-fold” replaced by “single-fold” and “finitely many” replaced by “at most one.”

Proof. From (i), apply Lemma 26.4 to a witnessing polynomial. For each original witness tuple there is exactly one tuple of circuit-gate values, so finite or singleton fibers are preserved. Apply Theorem 26.3 to the resulting quartic. It preserves its parameter solution set with no additional existential coordinates. This gives (ii) with exactly the same fiber cardinalities.

Conversely, a separated depth-four scheme has acceptance polynomial $A(x, k) - B(x, k)$ of degree at most four by Theorem 26.3. Its roots are exactly the successful parameter tuples, with no coding variables to eliminate. The assumed fiber cardinality therefore gives precisely the required finite-fold or single-fold Diophantine representation. $\square$

The result is a normal-form equivalence, not a proof that every computably enumerable relation satisfies these conditions. The general finite-fold program and the role of single-fold specifications are discussed in [9, 10]. We make no claim to settle either conjecture. Instead, the theorem places their full difficulty inside a very small substrate: one nonnegative counter, a block of increments followed by a block of decrements, and four levels of globally shared repetition.

There is an analogous formulation using vector-separated depth-two schemes. Use the quadratic-system version of Lemma 26.4 in the forward direction; in the reverse direction, sum the squares of the coordinate balance equations. The same parameter tuples are preserved. Thus moving from one counter to several trades nesting depth for simultaneous equations, not for an automatic improvement in witness multiplicity.

67.1 An exact accounting identity

For fixed endpoints and input parameters, split the schedule parameters into a prescribed tuple x and a quantified tuple k . The compressed compiler satisfies, with cardinalities understood in $\mathbb{N} \cup \{\infty\}$,

$$\#\{(k, z) : Q_E(m, n, x, k, z) = 0\} = \#\{k : \llbracket E \rrbracket_{(x,k)} \text{ meets the compiler's acceptance condition}\}. \quad (244)$$

This follows by summing the zero-or-one fiber equality of Theorem 25.1. There is no extra factor from arithmetic witnesses. But the right side can still be infinite, and it counts parameter tuples, not distinct words or traces. This is the exact point at which finite-fold information must enter from somewhere other than the local compiler.

67.2 No computable bound for a universal parameter search

Proposition 67.2 (Parameter-bound obstruction). *Let E be a fixed separated scheme whose existential parameter acceptance set is an undecidable computably enumerable set of inputs. There is no total computable function $B(x)$ such that every accepted input x has a successful witness parameter tuple all of whose coordinates are at most $B(x)$.*

Proof. If such a function existed, compute $B(x)$ and test the finitely many witness tuples in its box. For a separated scheme each test is an equality between explicit integer polynomial values, hence decidable. An accepted input would have a successful tuple in the box, and a rejected input would not. This would decide the chosen undecidable set, a contradiction. $\square$

The explicit height bound (117) does not conflict with this proposition. It bounds *auxiliary coordinates after the schedule parameters are supplied*; it does not bound a successful choice of those parameters from the original input. Even hypothetical single-fold representations would not automatically provide a computable bound on their unique witness.

68 The fixed-arity and finite-fold boundary [07]

68.1 Three quantifiers that must not be interchanged

For a fixed program and input, Theorem 46.1 has the form

for each $T \geq 1$, there is a polynomial P_T in $T + 1$ variables.

Unbounded halting is the existence of some successful horizon. This is not the same as one polynomial of fixed arity with T as an ordinary input coordinate. Writing $\prod_{i < T} \ell_i$ when T is an unknown does not perform that conversion; it merely writes a variable-length expression.

A fixed polynomial representation of a set $S \subseteq \mathbb{N}$ has the form

$$n \in S \iff \exists \mathbf{y} \in \mathbb{N}^k P(n, \mathbf{y}) = 0,$$

where both P and k are chosen before n . It is finite-fold if each input has finitely many witnesses, and single-fold if each input has at most one. Research on obtaining finite-fold representations goes substantially beyond constructing a unique finite trace; Cantone, Cuzziol, and Omodeo give a concrete sufficient-condition approach to that problem [11].

The constructions here solve the horizon-indexed witness problem. They do not provide the missing fixed-arity compression with controlled fibers. This is a statement of their exact logical strength, not a reason to abandon the stronger problem.

68.2 What MRDP supplies and what uniqueness still requires

An effective code for a finite queue computation has an effectively checkable transition relation. The set of successful codes, and the projection to halting inputs, can therefore be handled by the existing MRDP interface [5]. That yields a fixed finite polynomial representation. It does not, by its stated interface alone, assert unique or finitely many auxiliary solutions.

A compression may introduce arithmetic witnesses for coding a list, extracting digits, evaluating powers, or eliminating bounded quantifiers. Even when the machine has one exact halting trace, these new witnesses must be separately controlled. Replacing a variable-length trace by one natural code does not prove that its polynomial validity predicate has unique witnesses.

Our treatment of first halting does remove one possible source of multiplicity: a halted run cannot be padded with fictitious empty-queue steps. The product guard and, independently, the local parity-of-scale obstruction both enforce that fact. This is useful but does not settle the multiplicity of a later fixed-arity arithmetization.

68.3 No contradiction with bounded simulation

All formulas are effectively constructible, and for a specified finite T the actual deterministic run can also be simulated. Their purpose is not to outperform bounded simulation on this elementary decision problem. Their purpose is to preserve the run in a small, explicit algebraic witness and to support composition, formal verification, and comparison of computational substrates.

The witness bound depends on T . It supplies no computable bound on the unknown halting time from the input alone. Likewise, a circuit of size $O(T)$ is not necessarily small in the binary encoding length $\log T$. These distinctions are necessary when moving from exact finite certificates to unbounded computational questions.

[write] A fixed-arity degree threshold. Part XI's degree classification (Theorem 90.2) is a fixed-arity boundary theorem for the orthant-nonnegative format: polynomials of degree at most three that are nonnegative on the whole real orthant represent exactly the semilinear sets, so no such cubic is a universal certificate (Corollary 90.17), while degree four represents every computably enumerable set.

69 Implementation and validation

[write] This section covers manuscripts 01–07. Manuscripts 08–11 (batch 62) and 12 (batch 63) print their implementation and validation records inside Parts X–XIV.

[write] Every manuscript ships Python programs that import only the standard library, together with recorded outputs. Each subsection below is one manuscript’s own account of what it implemented, what it tested and what the tests do not establish. The programs and data are shipped in `code/` and `data/` with the manuscript’s prefix (`01-causal-traces-`, `02-poly-trajectories-`, `03-linear-memory-`, `04-witness-faithful-`, `05-trace-polytopes-`, `06-counter-schedules-`, `07-queue-causality-`); in the prose below the shipped names are printed. The displayed command lines are the manuscripts’ own and use the delivered layout (for example `code/verify.py`, `examples/`, `tests/`), because the programs import one another and write their outputs by those names; the README gives rerun recipes that copy the shipped files into that layout, and warns which programs overwrite recorded outputs. The manuscripts’ instructions for rebuilding their own PDFs do not apply to this report, whose build is described in the README. None of the finite checks is a proof; each manuscript says so, and its theorems rest on the proofs printed in Parts I–IX.

69.1 Manuscript 05: Implementation and reproducible verification

69.1.1 Reference implementation

The companion package contains a Python implementation using only the standard library. It emits sparse residual polynomials and regards the final polynomial as their squared sum. Every natural assignment is checked using exact integer arithmetic. The one fractional-relaxation example uses exact rational numbers, not numerical tolerances.

The primary entry point is

```
compile_certificate(net, initial, final, horizon,
                  form="quadratic", independence=None,
                  upper=None)
```

Here `form` can be `raw`, `quartic`, or `quadratic`. With no supplied independence relation the compiler computes the maximal sound relation. A supplied relation must be symmetric and contained in that maximal relation. The optional upper-guard matrix uses `None` for infinity and finite natural entries for upper bounds. One-step empty domains are rejected rather than silently retained; removing such transitions is a straightforward preprocessing step.

The returned certificate has methods for constructing the unique witness of a valid word, checking a whole natural assignment, decoding a zero, exporting the residual system as JSON, and expanding the polynomial for small instances. The code does not invoke an integer-programming solver or pretend to solve general Diophantine equations. It is a compiler and exact checker.

The source also implements the graph-realization construction of Theorem 4.4, the flag recognizer, a literal guarded-run simulator, and the circuit-to-safe-net reduction used in Theorem 13.1. The cellular-automaton and rewriting front ends in Section 57 are mathematical constructions; complete standalone translators for those front ends are not included.

69.1.2 What was tested

The full run of `python code/verify.py` produced PASS. The machine-readable record is `results/verification.json` [write] (shipped as `data/05-trace-polytopes-verification.json`; the program is `code/05-trace-polytopes-verify.py`). The following are the exact finite scopes of the checks.

Check	Scope and result
Petri demand criterion	256 scalar transition pairs, arc weights 0–3; 2,048 starting-marking comparisons.
Trace normal forms	All 75 independence graphs on ordered alphabets of sizes 1–4; all words of length at most 6: 358,509 words and 124,494 classes.
Boolean memory family	All 254 subset prefixes for $k = 1, \dots, 7$, including distinguishing one-letter continuations.
Core certificates	286 constructed full witnesses in the three presentations; 576 single-coordinate perturbations rejected.
Exhaustive zero search	All $2^{16} = 65,536$ assignments in the full forced-height box of a tiny quadratic instance; exactly its two predicted zeros.
Real relaxation	Exact rational witness for Example 11.4, together with absence of an enabled integer step.
Circuit reduction	AND, OR, XOR, and a repeated-input example; accepting-run counts 1, 3, 2, 1, with safety and distinct Parikh vectors checked.
Ballot formula	160 endpoint–horizon instances, with horizons through 7.
Interval guards	729 scalar transition pairs and 6,561 marking comparisons; 45 bounded execution instances and 216 constructed witnesses.

The normal-form oracle is independent of the flag recurrence: it constructs the graph of legal adjacent swaps, computes its connected components with a disjoint-set data structure, and compares accepted words with the actual minimum of each component. The guarded commutation check compares literal two-step execution domains against the closed-form box criterion. The ballot formula provides a separate combinatorial count for the producer–idle–consumer examples.

For the core three-transition net, the computed endpoint examples are:

M	N	T	Raw runs	Classes	Quadratic (V_2, R_2)
0	0	0	1	1	(4, 5)
0	0	4	9	4	(96, 77)
2	2	6	140	28	(142, 113)
1	2	5	44	13	(119, 95)

A finite test does not prove the universal theorems. In particular, changing one coordinate of a constructed witness is not an exhaustive uniqueness proof, and the full box search concerns only its explicitly stated tiny instance. Universal correctness and uniqueness are supplied by the preceding mathematical arguments. The tests serve as independent checks of indexing, signs, degree accounting, boundary cases, and the executable realization of those arguments.

69.1.3 Reproduction and polynomial format

From the package directory [write] (in the delivered layout; see the README, which also explains why the shipped code/05-trace-polytopes-build.py must not be run in the report directory), run

```
python code/verify.py
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

The verification program should be run without Python’s optimization flag, since its test assertions are part of the checks. The ordinary compiler’s witness validation is an explicit runtime check. Python 3.10 or later is required; no third-party Python package or network access is needed for verification. A standard TeX installation with the packages listed in the source is needed to rebuild the PDF.

Each exported JSON residual is a list of coefficient–monomial pairs. A monomial is a list of variable names, repeated when a power is present. The represented polynomial is the sum of the squares of the residuals, not the sum of the residuals. The domain field explicitly says “nonnegative integers.” A separate small text file contains a fully expanded quadratic polynomial as a format sanity check.

69.2 Manuscript 01: Implementation, validation, and repository integration

69.2.1 The reference compiler

The accompanying code/`01-causal-traces-causal_diophantine.py` uses only the Python standard library. It contains an exact integer operational semantics, the composition-box test, the envelope, Cartier–Foata normalization, sparse integer polynomials, and four compiler entry points:

Entry point	Mathematical contract
<code>compile_accelerator</code>	Theorem 21.1; all serializations, also some under active commutation.
<code>compile_history</code>	Theorem 20.1; one root for each bounded-height executable trace.
<code>compile_blocks</code>	Theorem 22.1; fixed ordered commuting phases, optionally existential counts.
<code>compile_word_power</code>	Proposition 22.2; arbitrary repetition of one fixed, possibly noncommuting word.

A separate code/`01-causal-traces-verify_exports.py` checker reads the exported JSON without importing the compiler. It validates both natural certificates and compares the expanded polynomial with the residual-square expression on fifty seeded assignments per file; these comparisons are finite checks, not a symbolic identity proof.

The matching `complete_*` functions compute the canonical witness when a given semantic object is valid. They do not search for arbitrary polynomial roots. A `System` can check an assignment, evaluate the sum of squares, expand it, or export all coefficients and an optional verified certificate to JSON. The exporter refuses to attach an invalid certificate.

Natural-number domains are part of the specification. Polynomial evaluation uses ordinary signed integer arithmetic, and all numerical work is exact. The implementation deliberately uses no floating-point feasibility test, relaxed solver, modular substitute for integer equality, or fixed-width machine counter.

69.2.2 What was actually tested

Run

```
python code/verify.py
```

from a machine with Python 3.10 or later. The recorded run used Python 3.13.5 and pseudorandom seed 20260930. The tests are reproducible and fail by raising an exception; they write their exact counts and examples to `data/` and `examples/`. [write] (Shipped as `data/01-causal-traces-*`; the command runs `code/01-causal-traces-verify.py` in a copy with the delivered layout, as the README describes.)

The domain-box tests use lower guards from zero through three, every displacement from the negative lower guard through two, and four upper-bound choices; direct execution is tested from zero through 25.

Finite check	Count
Pairs of scalar guarded actions, box test versus direct execution	5,184
Petri pairs, closed sign classification versus box criterion	1,296
State-multiset instances for the exact envelope	9,600
Individual serialized executions within those instances	102,768
Words checked against adjacent-swap connected components	8,744
Boolean three-layer matrices across all eight independence graphs	4,096
Accepted canonical matrices in that enumeration	443
Random finite-history instances	2,400
Compiler arity/equation-count instances	200
Changed auxiliary coordinates correctly rejected	43,804
Tiny history assignments exhaustively tested	1,041
Tiny accelerator assignments exhaustively tested	6,561
Fixed commuting-phase composition instances	512
Serializations of the worked six-event trace	60
Fixed-word power instances, including empty macro domains	16,380

Table 4: Exact finite implementation checks. Counts are not a claim of formal verification.

The Petri classification uses every consumption and production value from zero through five. Random envelope instances have one through three coordinates, one through four action labels, multiplicities at most two, and total event count at most six. Every multiset serialization in each selected instance is executed independently of the envelope formula.

For the normal form, all eight independence graphs on three labels and every word of length at most six are checked. An independent union-find calculation of adjacent-swap components is compared with the computed layers. The tests also enumerate all 2^9 Boolean three-layer matrices for each graph, using read-only actions so that every chosen independence subrelation is operationally valid.

The mutation checks change auxiliary witnesses while holding the semantic layer bits fixed. A different layer matrix could represent a different valid trace, so it would be incorrect to demand rejection of every change to a layer bit. In the accelerator, all witness coordinates are auxiliary relative to fixed (x, y, n) and are tested accordingly.

The tiny exhaustive natural-root examples are a single read action at endpoint one with height bounds zero, one, and two. Their root counts are respectively one, two, and three. Every possible witness lies in the tested Boolean box for these examples, as follows directly from the residual formulas. A single decrement accelerator is similarly exhaustively checked for $x, y, n \leq 2$, with witness values zero through two. These finite tests exercise root multiplicities, not only production of successful certificates.

For fixed-word powers, every word of length at most five over a guarded increment, a decrement, and a zero test is checked at nine initial markings and five repetition counts. This includes impossible loop bodies and the zero-repetition exception.

The large worked accelerator is also evaluated as an expanded polynomial: its 76-monomial quartic evaluates to exactly zero at the supplied eleven-witness assignment. The 32-witness history example passes all 48 residuals. Full results, including the test ranges encoded in the script, accompany the paper.

69.2.3 What the tests do not establish

The tests are not a proof for all dimensions, counts, guards, or height bounds; those claims rest on the mathematical arguments above. They do not constitute a Lean kernel check, an independent source-level verification of the Python implementation, or a proof that the arity bounds are optimal. This report did not rebuild the ProveIt repository or rerun its published axiom audits. Repository facts are taken from

the pinned source and documentation inspected for this study.

69.3 Manuscript 02: Executable artifacts and validation

69.3.1 What is implemented

The supplementary files implement the core bounded polynomial theorem [write] (shipped as code /02-poly-trajectories-*.py and data/02-poly-trajectories-*.json under the delivered names below):

certificates.py

Exact integer forward differences, binary-search generation of the canonical sign tower, terminal padding, and an independent local endpoint checker. A brute-force sign-run routine is used only in small regression tests, never by the main generator or checker.

quartic_compiler.py

A reference compiler for the fixed-degree sign-table conditions and the optional nonnegativity requirement. It exports every quadratic residual, all variable names, and a sample complete assignment. Its symbolic structure depends only on the nominal degree and the chosen final assertion, not on coefficient values or the horizon.

run_tests.py and test_report.json

A deterministic regression suite and its actual output, with seed 20260930. The output includes operation counts, finite uniqueness tests, and compiler structure fingerprints.

quartic_example.json

The exact degree-two quartic template in sum-of-squares form, together with a satisfying assignment for $p(t) = (t - 2)^2$, $T = 6$.

verify_export.py

A separate sparse-polynomial evaluator that reads the exported JSON, checks natural witness domains and quadratic residual degrees, and calculates every residual without importing the compiler.

The higher-level Boolean-guard, arbitrary trajectory, and quasi-polynomial front ends are proved mathematically but are not packaged as automatic source-language compilers in this release. This distinction matters: working core code is not the same as a complete program-analysis tool.

69.3.2 Tests actually performed

The included run produced the following results, with all assertions passing:

Check	Count
Random integer polynomials, nominal degrees 0–8	1,620
Additional explicit edge cases	8
Degree-two coefficient triples in $\{-1, 0, 1\}^3$	27
Candidate sign towers exhaustively checked on $[0, 2]$	59,049
Incorrect individual tag mutations rejected	68
Compiled polynomial instances, degrees 0–4	50
Negative positivity instances among those compiled	33
Nonlinear closed-form state comparisons	2,500

For each of the 27 small coefficient triples, all possible sign strings at the first two levels and all three constant top tags were considered. The checker accepted exactly one tower, equal to the generated tower. This tests uniqueness in a finite domain; the general uniqueness proof is Theorem 28.3.

For each compiled instance, the checker confirmed that every exported residual has degree at most two, every sample existential coordinate is natural, and the squared-sum equation vanishes exactly on the positive instances among the tested assignments. Ten differently instantiated examples at each nominal degree produced identical symbolic polynomial structures. False instances fail the required positivity residuals even though their full sign tables remain valid.

69.3.3 Concrete compiler size

Table 5 reports the unoptimized reference implementation. It is not a lower bound or a record claim.

d	Primary coordinates	Auxiliary coordinates	Total witnesses	Residuals
0	1	24	25	42
1	6	202	208	312
2	15	834	849	1218
3	28	2192	2220	3132
4	45	4540	4585	6414

Table 5: Actual variable and residual counts for the supplied compiler. The final equation is the sum of the squares of all residuals.

These constants are intentionally conservative. Repeated comparisons, zero-test slack variables, and straightforward Boolean wiring create many auxiliaries. Eliminating a redundant gate is safe only when it preserves both logical equivalence and uniqueness of the remaining witness.

69.3.4 Validation limits

The numerical tests do not prove soundness on all integer assignments, completeness for arbitrary horizons, or historical novelty. Those are separate questions: soundness and completeness are addressed by the mathematical proofs, and novelty requires further literature comparison. No Lean or Coq formalization of the new construction was compiled here. The ProveIt repository was inspected as a source, not independently rebuilt.

The independent JSON checker certifies an exported *assignment*'s arithmetic validity. It is not a solver and does not verify that the compiler correctly represents the intended relation for all inputs. That higher-level correspondence is exactly the theorem a future proof-assistant implementation should establish.

69.4 Manuscript 03: A hand-checkable example and executable validation

69.4.1 Four events

Consider the zero-initialized memory log

t	a_t	w_t	v_t	Interpretation
0	7	0	0	Read an untouched address.
1	2	1	9	Write 9 at address 2.
2	7	1	4	Write 4 at address 7.
3	2	0	9	Read back 9 at address 2.

Here $N = 4$ and $B = 5$. The chronological keys are 40, 16, 42, 18. The sorted records (α, κ, w, v) are

$$(3, 16, 1, 9), \quad (3, 18, 0, 9), \quad (8, 40, 0, 0), \quad (8, 42, 1, 4).$$

The adjacent address differences are 0, 5, 0. Their zero-test pairs (z, h) are $(1, 0)$, $(0, 4)$, $(1, 0)$, and their previous-value coordinates are 9, 0, 0. The first sorted record is a write. The next two records are reads returning 9 and 0 respectively; the final write can store any natural value. All memory checks pass.

There are $C_4 = 6$ comparators, $W_4 = 77$ auxiliaries, and $R_4 = 85$ residuals, besides the 12 free event parameters. The supplied export contains the actual expanded polynomial: degree four, with 587 distinct nonzero monomials. Changing the final read value from 9 to 8 makes the appropriate read residual nonzero. Because the preceding circuit has a unique assignment, no alternate routing or slack assignment can repair that invalid log.

69.4.2 What the implementation does

The file `code/03-linear-memory-canonical_memory.py` is a dependency-free Python implementation of the memory theorem, using exact integers and sparse polynomials. Its representation stores each monomial as a sorted tuple of variable indices. It rejects residuals of degree greater than two. The compiler creates symbolic event coordinates, so its polynomial structure is determined by L , not by the particular numerical log used to generate the witness.

The JSON export includes the natural-domain convention, every parameter and witness name, the generated assignment, and every quadratic residual. With `-expanded`, it also exports the actual sum-of-squares polynomial. Canonical values are serialized as decimal strings to avoid accidental loss of precision in consumers that treat JSON numbers as floating point. Polynomial coefficients in this backend are small ordinary JSON integers. The independent semantic checker uses a dictionary as memory and compares reads with its current contents.

Both memory backends are implemented in this archive; `code/03-linear-memory-linear_memory.py` provides the large-base alternative. The RAM, graph, counting, and probability results are mathematical constructions proved in the preceding sections; a complete machine-language frontend is not supplied. The probability test checks the stated weight formula, not an implemented probabilistic RAM compiler. No new Lean or Rocq proof artifact is claimed.

69.4.3 Executed checks

The complete test driver is `code/03-linear-memory-test_certificates.py`; its recorded results are in `data/03-linear-memory-test_results.json`. The following tests were actually executed in this environment:

Test family	Result
Comparison and zero tests	Exhaustive candidate witnesses for 81 ordered input pairs and 21 zero-test inputs; exactly the stated witnesses survived.
Sorting networks	105,883 cases: every permutation at widths 1, 2, 4, and 8, and all binary inputs at width 16.
Small memory logs	All 585 logs of lengths 0 through 3 over addresses, write flags, and values in $\{0, 1\}$: 259 valid and 326 invalid, in exact agreement with the independent semantic checker.
Large integers	150 generated valid logs, with addresses up to 160 bits and values up to 200 bits; 143 deliberately corrupted read logs were rejected.
Auxiliary corruption	All 273 single-coordinate increments of the auxiliary witness for a six-event valid log were rejected.
Expansion	The four-event quartic has 587 monomials and degree four; 20 additional assignments confirmed equality with the residual sum of squares.
Path weights	For each horizon 1 through 12, the stop-on-first-one weight sum was exactly $2^T - 1$.

The six-event test has $N = 8$, 24 comparators, 273 witnesses, and 287 residuals, in agreement with the exact formulas. All recorded tests passed.

Finite testing is not a proof that arbitrary natural assignments have no unexpected roots. That conclusion rests on the forcing argument in Theorem 38.3. The tests instead check correspondence between the written compiler and the equations, exercise edge cases, and make regressions observable. The mutation test in particular is not presented as an exhaustive search over all possible witness tuples.

69.4.4 Reproduction

From the archive's root, run [write] (the delivered layout; see the README)

```
python3 code/test_certificates.py
python3 code/canonical_memory.py artifacts/example/events.json \
    artifacts/rebuilt_example --expanded
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

The test script writes its result file relative to its own location. The Makefile supplies equivalent test, example, and pdf targets. The Python files use only the standard library; Python 3.10 or later is required for their type-annotation syntax. The article uses standard pdfL^AT_EX packages. The included PDF is already compiled. [write] (The PDF is not shipped in ProveIt, and the shipped Makefile, code/03-linear-memory-Makefile, names delivered paths; see the README.)

69.5 Manuscript 04: Executable artifacts and exact validation

69.5.1 Contents and mathematical interpretation

The package contains code/diophantine_compiler.py [write] (shipped as code/04-witness-faithful-diophantine_compiler.py, with the test driver code/04-witness-faithful-run_tests.py, the results data/04-witness-faithful-results.json and the four examples data/04-witness-faithful-*.json), an exact sparse-integer-polynomial implementation; tests/run_tests.py; a generated tests/results.json; and four JSON examples. Each example

lists all natural variables, all polynomial residuals as integer coefficient/monomial data, and a complete witness assignment. The JSON convention is

combination = sum of squared residuals.

Thus the residual list defines one explicit ordinary polynomial without printing an unnecessarily expanded quartic. No oracle, numerical tolerance, floating-point root test, or general Diophantine solver is used. The separate code/verify_certificate.py verifier does not import the compiler; it independently parses and evaluates the exported JSON residuals and checks the supplied witness and degree bound.

From the package directory [write] (in the delivered layout; see the README), run

```
python tests/run_tests.py
```

Python 3.10 or later is sufficient; only its standard library is used. To rebuild this article, run `pdflatex article.tex` twice, or `latexmk -pdf article.tex`. The bibliography is included in the source, so a separate bibliography tool is unnecessary.

69.5.2 What was checked

The normal-form check enumerates every symmetric irreflexive independence relation on alphabets of sizes one through four and all words of lengths zero through five. It constructs commutation classes independently using union-find on adjacent allowed swaps, then compares the class minima with the monitor. This checks 75 graphs, 90,404 words, and 39,093 classes. The lower-bound family is checked through $k = 7$, including all 254 prefixes in those instances and their distinguishing one-letter suffixes.

For Petri nets, 24 seeded small nets with three places and three transitions are tested on 2,904 candidate words. The test evaluates 1,417 enabled-run certificates, checks that zero energy coincides with normality, verifies swap closure, and checks exact variable and residual counts. A separate smallest example exhausts 729 natural assignments in a finite test box and finds one zero.

For FRACTRAN, the tests enumerate 132 one- or two-fraction programs whose positive reduced numerators and denominators are at most four, use inputs 1 through 12, and horizons zero through three. Together with explicit normalization and denominator-one cases, they evaluate 4,914 valid certificates and check their sizes and degree bounds. These checks test the constructive direction and bookkeeping; they do not exhaust all possible false assignments to the larger systems.

For SKI, 1,997 application codes are inverted and rebuilt, with both children checked to be smaller. The root rules and several context paths are exercised in 1,875 valid certificates. The proof of Theorem 55.2, rather than this finite test, establishes completeness and absence of arbitrarily large spurious witnesses.

Example	Variables	Residuals	Largest coordinate	Rejected mutations
Petri fork/join	56	67	1	69
FRACTRAN first halt	103	120	125	161
SKI root schedule	13	11	635	26
SKI contextual schedule	8	8	91	16

Every listed witness has exactly zero sum-of-squares value. The mutation column counts all permissible one-coordinate increments and decrements by one, each rejected by a nonzero residual. These local mutation checks are useful for catching disconnected variables but are not a proof of global uniqueness.

69.5.3 Validation status and reproducibility limitations

The reported suite completed successfully in the working environment. Its mathematical checks are deterministic apart from the seeded net generator, whose seed is fixed in the source; the elapsed-time field in the JSON report is not a mathematical invariant. The implementation mirrors the displayed residual systems and does not implement a fixed-arity MRDP elimination.

No new Lean or Rocq/Coq theorem was compiled as part of this package, and the repository's existing build was not rerun. The article supplies conventional proofs, executable constructions, and finite exact checks. A proof-assistant version of the new witness-level theorems remains a proposed next artifact, not an accomplished part of the present result.

69.6 Manuscript 06: Executable artifacts and verification

The accompanying implementation uses Python's exact integers and the standard library. It is an executable reference model of the displayed constructions, not a verified extraction from a proof assistant. Its sparse polynomials store integer coefficients and monomials explicitly. A certificate file stores the quadratic residual list and declares the represented polynomial to be their sum of squares. This gives an exact ordinary polynomial without forcing an unhelpful expansion of every square.

69.6.1 What the files implement

The module `code/compiler.py` [write] (shipped as `code/06-counter-schedules-compiler.py`) defines the expression syntax, transition semantics, resource product and powers, the globally sound commutation relation, the Boolean signatures, and both versions of the compiler. `Builder.construct` computes the deterministic auxiliary assignment from free inputs. It does not make a rejected instance satisfiable: the returned assignment has nonzero sum-of-squares value when endpoint or normality constraints fail. Witness-construction recipes involving a minimum or a case test are *not* extra symbols in the polynomial. The exported residuals are the sole algebraic constraints, and the proofs above establish that they force the recipe values on every root.

The module `code/substrates.py` [write] (shipped as `code/06-counter-schedules-substrates.py`) implements coefficientwise sign splitting, nonnegative arithmetic-circuit lifting, and the separated-schedule frontend. It compiles a polynomial directly to one counter at its original degree, compiles its quadratic circuit system to several counters at depth two, and compiles the sum-of-squares quartic to one counter at depth four. Its current coefficient encoding is unary, as specified in the proof of Theorem 26.3.

The bounded-length construction of Theorem 18.1 is proved in the article but is not a separate polynomial-export frontend in the supplied prototype. The tests do independently enumerate bounded trace classes to check the normality component. No claim of complete implementation coverage should be inferred from the scope of the mathematical results.

69.6.2 Finite checks and their limits

The main suite exhaustively checks small resource pairs and powers, every independence graph on a three-letter alphabet, and every word of length at most six over that alphabet. For the trace tests, connected components under actual adjacent independent swaps provide an independent source of true lexicographic representatives. The suite also checks signature composition, power saturation, and 160 seeded expression instances. Both resource-only and canonical compiled systems are tested on valid and invalid endpoints. A focused catalyst test catches the zero-power error discussed above.

Mutation tests change one auxiliary coordinate of a valid certificate and verify rejection. These are useful implementation checks, but they are not a proof that no coordinated multi-variable mutation could satisfy the system. The structural uniqueness proof in Theorem 25.1 supplies that theorem. Similarly, random expression tests do not establish cap-at-two for all expressions; Theorem 24.2 does.

Verification group	Checks passed
Exact resource product, powers, and global commutation	8,140
Trace cross-sections, memory signatures, composition, saturation	39,032
Seeded compressed evaluation and cap-at-two instances	480
Compiler acceptance, degree checks, invalid inputs, zero repetition	1,180
Valid mutation baselines and rejected one-coordinate mutations	569
A certificate for an execution with a 221-digit length	1
Polynomial/circuit/separated-substrate frontend checks	320
Total	49,722

Table 6: Executed finite checks. The main suite contains 49,402 checks; the substrate frontend adds 320. A check is a tested identity or predicate, not an independent mathematical theorem.

The substrate test uses

$$P(x, y, z) = xyz + 2z - x^2 - 3$$

on the grid $\{0, 1, 2, 3\}^3$. Its nonnegative circuit introduces six uniquely forced gate parameters and seven quadratic equations. The corresponding multi-counter scheme has seven counters and depth at most two. Summing the seven residual squares gives a quartic and hence a one-counter scheme of depth at most four. The test compares all three presentations with direct polynomial evaluation and checks rejection after changing a gate value.

For the multiplication schedule in Example 25.3, the resource-only prototype allocates 79 witnesses and 86 equations. Its canonical version allocates 309 witnesses and 547 equations, even though the independence relation is empty and could be optimized away. These deliberately transparent counts are not claimed to be minimal. The large-number test uses the same canonical compiled expression; its longest witness has 221 decimal digits.

69.6.3 Reproduction

From the package directory [write] (in the delivered layout; see the README), run:

```
python code/test_compiler.py
python code/substrates.py
python code/compiler.py --export examples/multiplication_certificate.json
```

The first two commands regenerate the reports in `verification/` [write] (shipped as `data/06-counter-schedules-*.json`). The random seed in the main suite is 20260930. The compiler has no network requirements or third-party Python dependencies. Timing fields are machine-dependent; the reported counts and mathematical comparisons are the reproducible content.

Rebuild the article with:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

Two or more direct `pdflatex` passes can be used instead. The bibliography is contained in this source, so no external bibliography database is needed. The PDF uses standard installed TeX fonts; font files are not included in the package.

69.7 Manuscript 07: Executable artifacts and exact validation

69.7.1 Contents and reproduction

The package includes `code/queue_certificates.py` [write] (shipped, like every file named here, with the prefix `07-queue-causality-` in `code/` or `data/`), with three cyclic-tag modes and the general deletion-tag compiler; `code/compile_tag.py`, a JSON-specification front end; and `code/verify_certificate.py`, an independent evaluator that does not import the compiler. All use only the Python standard library. Python 3.10 or later is sufficient.

From the package directory [write] (in the delivered layout; see the README), run

```
python tests/run_tests.py
python code/verify_certificate.py examples/stream_example.json
python code/verify_certificate.py examples/ternary_tag_example.json
```

To construct the small cyclic-tag example without simulating it, run

```
python code/queue_certificates.py --appendants 10 "" \
  --initial 1 --time 3 --mode stream --no-witness \
  --output examples/symbolic_only.json
```

The empty quoted argument is the second appendant. The general tag front end reads arrays of integer digits, so it is not limited to single-character decimal alphabets:

```
python code/compile_tag.py --spec examples/ternary_tag_spec.json \
  --output examples/ternary_tag_example.json
```

Compile the article with two runs of `pdflatex article.tex`. Its bibliography is included in this source file.

69.7.2 The certificate format

Each positive example stores the variable list, topologically ordered arithmetic nodes, residual-root indices, metadata, a conservative degree upper bound, and a complete natural witness. Constants and witness values are serialized as decimal strings to avoid fixed-width integer limits in other software. The convention `combination = sum_of_squares` defines one polynomial from the residual list.

The independent verifier validates the node operations and references, evaluates exact integers, checks the degree bound, and evaluates every residual. It has no floating-point tolerance and calls no Diophantine solver. It proves computationally that the supplied witness is a zero of the exported polynomial. Semantic equivalence and uniqueness rely on the mathematical proofs, not on that single evaluation.

The circuit's degree analysis is deliberately conservative: it may not detect cancellation or multiplication by zero. Thus `stream_example.json` reports degree at most six, while the simplified explicit polynomial (188) visibly has degree four. This is not a discrepancy in the represented equation.

69.7.3 What was checked

The reproducible results are stored in `tests/results.json`. The principal counts are:

Check	Cases
Cyclic-tag candidate consumed bitstrings	92,610
Raw word-balance solutions rejected by causality	268
Exact halting instances in that enumeration	370
Positive cyclic-tag certificates checked in two modes	740
First-failure resource-product checks	5,916
General deletion-tag candidate consumed words	194,940
No-slack cyclic-tag candidate bitstrings	840
Natural witness assignments exhausted for quartics	79,631
Word-monoid concatenation pairs	3,969
Integer table-interpolation evaluations	423
General integer-polynomial tag compiler instances	120
Candidate words evaluated in those compiled polynomials	4,200

The cyclic-tag enumeration uses all 49 period-two programs whose appendants have length at most two, all 15 inputs of length at most three, horizons one through six, and every candidate consumed bitstring. It compares the word-and-guard formulas with a direct string simulator. Every positive instance is compiled in stream and quartic modes; single-coordinate witness increments are rejected.

The resource tests include all scalar three-event histories with initial resource at most three and consumptions and productions at most two, followed by 3,000 seeded multi-resource cases. They explicitly allow the formal history to become negative after failure.

The general-tag checks compare the formulas with direct two-symbol deletion, including nonempty halted tails, over exhaustive small binary instances and seeded ternary instances. Separate compiler checks use alphabets of sizes two through four, deletion numbers one and two, and horizons one and two. They evaluate the cleared *integer-polynomial circuits*, not merely the interpolation tables. They also verify that disabling witness attachment changes no polynomial nodes or residuals.

The quartic tests exhaust actual finite boxes of natural assignments, not only generated semantic witnesses. Nevertheless no finite box proves global uniqueness. That is why Theorem 51.1 proves that every zero propagates a valid word code and cannot pass through an empty queue. The lower bound on polynomial memory dimension is likewise a mathematical proof, not an inference from the finite word-monoid test.

69.7.4 Limitations of the validation

The tests do not certify every implementation behavior for arbitrarily large integers, replace formal proof of the compiler, or establish historical novelty. They do provide independently reproducible checks of the stated formulas, exact resource counts in tested instances, symbolic table clearing, and several easily missed edge cases. The full fixed-read network compiler and a Lean development are proposed extensions, not unreported completed artifacts.

70 Formalization targets

[write] *This section covers manuscripts 01–07. The formalization plans of manuscripts 08–12 are printed inside Parts X–XIV; they too are proposals, and no module or declaration they propose exists.*

[write] No theorem of this report is formalized in Lean or Rocq, and no manuscript ships a proof-assistant file; the subsections below are the manuscripts’ proposals, and every module name in them is proposed, not asserted to exist. The report sits beside ProveIt’s Lean project `Computability/HilbertTenthProblem`, which proves MRDP in both directions. The manuscripts use its existence interfaces `Diophantine.boundedForall_dioph`, `Diophantine.exactIter_dioph` and `Diop`

`hantine.existsExactIter_dioph` (Lean/Diophantine/Common/DiophantineTrace.lean) and `Diophantine.mrdp` as context only: those theorems assert that representations exist and say nothing about how many witnesses a representation has. The project also formalizes the single-fold unary exponential representation of Jones and Matiyasevich (Remark 64.2), and the queue content and length-marker identities of its tag-system route (Remark 47.1). Placement beside a formal development confers no formal status on anything here.

70.1 Manuscript 05: A formalization route that preserves the stronger specification

The new proofs are not currently Lean-checked. A useful formalization should preserve the fibre information rather than immediately discarding it through an existential predicate. The repository's existing `Dioph` interfaces can remain unchanged; the stronger results can imply them as corollaries.

70.1.1 Separate semantic and arithmetic layers

The semantic layer should define a finite labelled family of partial maps, exact finite runs, a proved sound independence relation, and its generated word equivalence. The arithmetic layer should define a fixed finite set of variables and residual polynomials. A decoder reads the selected word from a natural zero. A witness constructor produces all variables from a canonical run. The central obligations are

$$\text{decode}(\text{encode}(w)) = w, \quad \text{encode}(\text{decode}(z)) = z.$$

The second identity is what excludes free arithmetic witnesses. Proving only the first would establish completeness but not injectivity of the representation.

An appropriate result object is an equivalence between a finite execution-class type and a subtype of natural assignments satisfying the polynomial equation. Cardinality and weighted-enumerator statements then follow from that equivalence rather than from separate ad hoc proofs. The quotient can be replaced by the subtype of normal runs, using the unique-normal-form theorem to build an equivalence with classes.

70.1.2 Suggested proof decomposition

A practical decomposition has six modules, whose names below are proposed rather than asserted to exist.

PetriDemand. Formalize Theorem 4.1, including both implications and the empty-coordinate case. Keep the distinction between integer vector updates and natural state domains explicit. Prove equality of partial functions, not merely equality of results when both are defined.

IntervalGuardCommutation. Represent upper endpoints by a type with an infinity element. Prove the composed-box formula, canonicalize empty boxes, and derive Theorem 4.5. Equality of nonempty boxes must include unbounded-coordinate cases.

TraceNormalForm. Prove the forbidden-factor characterization and the flag invariant. Establish uniqueness of the lexicographically least representative using finite word classes. Prove that the recognizer's natural recurrence forces Boolean values by induction, rather than adding unnecessary Boolean polynomial constraints.

UniqueAffineExtension. Formalize unique slack, the affine AND gadget, and deterministic circuit extension. A reusable combinator should compose witness equivalences over fixed inputs. Sum-of-squares scalarization should be stated over an ordered ring, then specialized to integer polynomials evaluated at natural assignments.

CanonicalTraceCertificate. Define the variable index type as a tagged finite union of the arrays in the construction. Prove decoding, existence, and coordinatewise uniqueness. This layer also carries the exact variable and residual counts, including $T = 0$ and $d = 0$.

TracePolytope. Reuse the affine system over the reals, prove each coordinate bound, and package boundedness and convexity. Relate the natural assignment subtype to lattice points without changing domains by an uncontrolled integer encoding.

70.1.3 Integration with the current repository

The existing `exactIter_dioph` theorem can provide the unbounded existence-level consequence once a substrate's one-step relation is Diophantine [3]. It should not be renamed or reinterpreted as a multiplicity theorem. Instead, add a clearly separate bounded compiler interface carrying unique extension, and document precisely what is forgotten when projecting to `Dioph`.

The most valuable first milestone is the raw affine certificate plus unique extension; it is small, reusable, and independent of trace-monoid infrastructure. The second is the exact flag recognizer, and the third is the combined bijection. Only after those should cardinality and the full real-polytope statements be layered on. No project axiom or placeholder proof is needed in the mathematical design; an actual implementation must still pass the repository's normal kernel and transitive-axiom checks.

70.2 Manuscript 01: A proposed Lean integration boundary

A useful formalization should keep the new proof obligations separate from existing ordinary-Diophantine closure. One possible layering is:

1. Define box-guarded partial translations, word execution, and equality of partial maps. Prove Proposition 5.1 and the natural-state invariant without importing the MRDP machinery.
2. Prove the envelope theorem using counts of positive and negative displacements in prefixes. State the theorem over arbitrary finite action and coordinate types; prove the Petri formula by simplification.
3. Introduce a representation structure containing a polynomial, a semantic decoder, and an equivalence between its natural zero set and the specified semantic type. Ordinary Diophantineness is then a projection, not the entire contract.
4. Formalize conditional slack canonicalization and the support gadget. Assemble the accelerator polynomial with finite indexed residual families and a sum-of-squares lemma.
5. Develop the dependence-poset normal form and its padded layer characterization, then prove the full-root bijection of the history compiler. Only after that should an adapter export ordinary reachability through the existing Diophantine interfaces.

The key extra structure is the *equivalence of fibres*. A theorem that a decoder is surjective onto semantic histories proves completeness but not uniqueness. Injectivity of that decoder on the complete zero set must also be retained. No uncompiled Lean skeleton is presented as a completed formalization in this package.

70.3 Manuscript 02: Integration with ProveIt and further research

70.3.1 An interface worth adding

A useful extension of the repository would distinguish three levels of representation explicitly. An ordinary Diophantine interface stores an existence equivalence. A finite-fold interface additionally proves finite fibers. A single-fold interface additionally proves subsingleton fibers. Conversions should retain these properties in their types or theorem statements rather than discard them in an undifferentiated existence predicate.

For the present construction, the clean decomposition is a theorem about unique sign tables, a theorem about canonical circuit evaluation, and a composition theorem relating the compiled quartic to the checking formula. The ordinary existence result can then be exported to the current Diophantine interfaces without making the stronger witness theorem depend on a noncanonical general MRDP backend.

The key invariants for a formal implementation are: ordered cuts cover the entire integer domain; empty slots form a terminal suffix; their tags are fixed; adjacent active tags differ; every parent point is covered by a child monotonicity interval; all internal arithmetic computations have a unique normal form. These invariants are small enough to isolate into reusable modules, but no claim of completed formalization is made here.

70.4 Manuscript 03: A proposed ProveIt formalization architecture

The repository already exposes ordinary Diophantine closure interfaces, including `Diophantine.boundedForall_dioph`, `Diophantine.exactIter_dioph`, and `Diophantine.existsExactIter_dioph`, according to the inspected status register [2]. These are useful downstream interfaces, but their existential statements should not be treated as uniqueness theorems. A new compiler should retain the stronger specification until an application explicitly chooses to forget it.

A natural representation is a finite list of polynomials over $\mathbb{Z}$, evaluated on natural-number assignments cast into $\mathbb{Z}$. A semantic certificate object should record a decoder from satisfying assignments to source objects and a canonical encoder in the reverse direction, with both inverse laws on their specified domains. Counting is then a consequence of an equivalence of types or finite sets, rather than a separate combinatorial assertion.

The following module names are *proposals*, not files claimed to exist in the inspected repository:

Proposed module	Main proof obligation
CanonicalQuadratic	Natural-domain residual semantics, sum-of-squares combination, fresh-variable composition, and fiber equivalences.
CanonicalCompare	The two-equation signed-gap theorem, zero tests, unique bound slacks, multiplexing, and exact gadget counts.
CanonicalSorting	Record-preserving comparators, fixed network semantics, the bitonic generator, and its closed count formula.
ExactProductFingerprint	Base-expansion coefficient recovery, multiset equality from product identity, and injective record packing.
CanonicalMemory	Both backend theorems, chronological semantics, exact counts, and distinct height bounds.
CanonicalRAM	The request-before-response interface, temporal soundness induction, normalization of inactive instructions, and halted padding.
CanonicalGraphRuntime	Immutable heap unfolding, spine evaluation, microstep accounting, and the explicitly stated strategy correspondence.
CertificateAudit	Emitted-polynomial reflection, checked examples, domain conventions, and transitive assumption reports.

The linear backend offers an especially short conceptual trusted boundary. Its nonlocal step is the multiset lemma, reducible to elementary base expansion and polynomial cancellation; it requires no asymptotically optimal sorting network formalization. The low-height backend remains valuable when later applications need tight bit bounds.

An untrusted compiler may emit a residual syntax tree, but the kernel should check that the emitted object is the polynomial claimed by the compiler correctness theorem. Testing the value at one supplied witness checks only that instance. The complete theorem must quantify over every natural assignment to prove that there are no spurious roots, and over every source execution to prove that none are lost. A degree or variable-count audit should inspect the actual emitted object, not just a separately maintained formula.

A sensible order of work is to formalize the common residual semantics, then the product and comparison lemmas independently, then the two memory backends, and finally one small RAM instruction language. The combinator frontend should come after the precise runtime strategy and heap invariants have been fixed. Existing ProveIt MRDP interfaces can then export an ordinary Diophantine reachability statement, with an explicit note when that export forgets the canonical-fiber contract.

The inspected status register lists further work on the 1984 paper’s specific NP characterizations (53) and (54) [2]. The bounded-counting corollary here is not a formal proof of those numbered statements or a replacement for their required bounded-exponentiation theory. It should be integrated as a distinct result unless an exact implication is separately proved.

70.5 Manuscript 04: How these results extend the inspected ProveIt development

The mathematical extension point is a *witness-level* interface. An ordinary Diophantine theorem records an existential equivalence. A stronger reusable interface would record a semantic object, a decoder from every zero, a canonical encoder, and proofs that the encoder and decoder are inverse. Section 2.5 shows how circuit compilation can preserve that interface, and the three substrate constructions instantiate it in substantially different ways.

The repository snapshot used for comparison is

e8bb0931d67f80d9fce87a8cddb0f661ff19f956.

The inspection was focused on its Hilbert-tenth and combinatory-logic branches; it was not a full audit of every repository file. Its existing exact-iteration development already proves the existential closure we need [3]. The present paper should therefore be integrated as a refinement tracking fibers, not as a duplicate theorem that finite reachability is Diophantine.

There are three natural mathematical modules. The first is an injective-state-coding and unique-extension layer, including canonical slacks and circuit wires. The second is a semantic quotient layer: an independence relation, enabled-swap proof, normal-form monitor, and class-to-zero bijection. The third is a resource layer measuring variables, residuals, degree, coefficient bit lengths, and witness heights separately. A verified composition theorem should carry all three layers rather than only a proposition about nonempty solution sets.

The numerical bounds in this article do not compete with the repository's universal polynomial pairs or its optimized straight-line universal certificates [1]. Here the horizon and often the schedule are external compilation data, whereas a universal equation encodes unbounded computation with fixed arity. Comparing these counts without recording that distinction would be misleading.

No new repository proof-assistant build is reported in this package. The executable artifacts are standard-library Python implementations of the explicitly displayed bounded compilers. A separate source-audit note records the inspected paths and the claims relied upon, without treating repository documentation as an independent verification of the new results.

70.6 Manuscript 06: Proof dependencies and precise completion status

Result	Essential ingredients	Status in this package
Exact resource summaries	Partial translations, coordinatewise minimum, word induction	Full proof; implemented and tested
Global swap criterion	Exact two-letter resource products	Full proof; implemented and tested
Compact normality signature	Classical forbidden factors; Boolean affine coordinate maps	Full proof; implemented and tested
Cap-at-two saturation	Signature product; $\alpha_a \beta_a = 0$	Full proof; implemented and tested
Compressed quartic compiler	Unique local gates, syntax induction, sum of squares	Full proof; implemented and tested
Bounded trace-class bijection	One-hot words, deterministic memory, trace-invariant resource action	Full proof; separate export frontend not implemented
Flat-scheme decidability	Affine positive-power formula; Presburger case splitting	Full reduction proof; decision frontend not implemented
Degree–depth correspondence	Length polynomials; signed monomial splitting	Full proof; frontend tested
Depth-two multi-counter universality	MRDP; unique arithmetic-circuit lift	Full reduction proof using classical MRDP; frontend tested on a sample
Finite-fold equivalence	Unique quartic lift; exact separated length balance	Full equivalence proof; no solution of the general conjectures

Result	Essential ingredients	Status in this package
Formal verification	A proof-assistant implementation of all new definitions and compiler semantics	Proposed; not supplied or claimed

The only deep external theorem used for the universality conclusions is MRDP. The main compressed compiler, its cap-at-two theorem, and the bounded trace-counting construction do not depend on it. The quadratic one-counter decidability observation cites a separate classical decision theorem; it is not used to prove the flat/nested multi-counter boundary.

70.7 Manuscript 07: Integration with ProveIt and the formal trust boundary

70.7.1 What was inspected

The repository context was pinned at commit

725d2ebb6909fe11a13a92354c0f47367a3cbbf5

The inspected Hilbert-tenth-problem README and MRDP guide describe `Diophantine.mrdp`, `Diophantine.mrdp_iff`, and the finite-polynomial and trace interfaces [5, 1]. The guide reports an axiom-audited development and distinguishes the current forward proof from the older shared trace route. Those are repository reports, not build results independently reproduced for this package.

No theorem about the queues in this article is inferred from an unrelated research claim elsewhere in the repository. The explicit queue and word-memory proofs are independent mathematical arguments. Existing MRDP infrastructure is relevant to eventual fixed-arity representability, not needed for the bounded constructions themselves.

70.7.2 A proposed formalization sequence

A manageable formalization should separate word semantics from polynomial syntax. First define the cyclic-tag step as a partial function on finite Boolean lists with a phase parameter, stopping on the empty list. Prove the radix concatenation identity and fixed-length injectivity, then prove the suffix reconstruction theorem at the word level. These proofs need no Diophantine machinery.

Next formalize Theorem 8.1 over integer-valued formal resource histories. The invariant before the first disabled event is nonnegativity. This is the crucial point where natural-number truncation must not enter the formal recurrence. The first disabled event supplies a specific zero factor in the finite product.

Only then define the representing polynomial in

$$\text{MvPolynomial } (\text{Fin } (T+1)) \mathbb{Z}.$$

The first T coordinates are consumed symbols and the last is the guard slack. Prove that evaluating each symbolic residual gives its semantic integer expression; combine this with the sum-of-squares zero criterion. Soundness, completeness, and uniqueness should be separate theorem endpoints, followed by degree and variable counts. The quartic compiler should have its own endpoint because its preservation-of-valid-scale argument is different.

Finally, a reified arithmetic-DAG checker can connect exported data to these polynomial definitions. JSON parsing and Python generation should remain outside the trusted boundary. The checker must prove that accepted nodes define the intended polynomial and that the displayed witness evaluates to zero. Checking one witness alone must not be presented as a proof that no other witness exists.

70.7.3 What is shipped now

The package supplies human-readable proofs and tested Python compilers, including integer-only finite-table interpolation for deletion-tag systems. It supplies neither a Lean proof file nor an assertion that the new theorems have passed a kernel audit. The file `notes/lean_integration.md` [write] (shipped as `07-queue-causality-lean_integration.md`) records proposed modules and obligations, so a later formalization can begin without confusing an interface plan with completed code.

71 Research questions

[write] This section prints every research question of the seven manuscripts: ten from manuscript 01 (its Section 12), ten from manuscript 02 (its Section 12.2), ten from manuscript 03 (its Section 13), ten from manuscript 04 (its Section 11), twelve from manuscript 05 (its Section 13), twelve from manuscript 06 (its Section 12), and ten from manuscript 07 (its Section 13): seventy-four questions in all. Manuscripts 03, 04 and 06 posed their questions under subsection headings; those headings are the titles here, and 06's question numbers are kept in the titles. Where several manuscripts ask the same question, the questions are merged into one `question` environment whose title names every source; inside it each source's question text is printed verbatim under *Source NN*, and each source's follow-up paragraphs are printed verbatim after the environment under the same heading. Seven merges reduce the seventy-four questions to fifty environments. Listing a question here does not claim that it is answered, by this report or elsewhere. Two questions are partly answered inside this report, and a [write] note at each says how far: manuscript 01's question on sharp degree, arity and domain comparisons (Research question 71.16), by the degree-two certificate of Part II (manuscript 05); and manuscript 06's Question 9 on new storage monoids (Research question 71.33), for bounded runs of fixed-read FIFO networks, by Part VI (manuscript 07). Manuscript 02's closing paragraph on what would constitute a stronger result is printed as the last subsection.

Questions of manuscripts 08–11. [write] Manuscripts 08–11 (batch 62) pose thirty-nine further questions: twelve from 08, ten from 09, ten from 10 and seven from 11. They are printed at the end of Parts X–XIII, one `question` environment each, under the manuscripts' own titles (manuscript 11's subsection headings) and with their follow-up paragraphs. Where one coincides with a question of this section, a [write] line after it names that question; they are not merged, because each was written without knowledge of this report. Six questions of this section are answered in part or re-scoped by Parts X–XIII, and a [write] note at each says how far: Research question 71.16 (Part XI), Research questions 71.27 and 71.43 (Part X), Research questions 71.3 and 71.49 (Part XII, with Part XIII on the second) and Research question 71.50 (Part XIII). Manuscript 08's question on certificates without explicit sorting is answered in part by Part V (see the note after it in Part X). Manuscript 12 (batch 63) poses eight more, printed at the end of Part XIV in the same way. Part XIV answers in part Research question 71.49 and an analogue of the question on variable periods (Research question 71.37), and re-scopes Research questions 71.48 and 71.50; a dated [write] note at each says how far. Manuscripts 13–15 (batch 78) pose twenty-five more, printed at the end of Part XV (Section 149) in the same way. Part XV answers Part X's question on an interaction-combinator frontend for bounded scheduled histories (Research question 85.7), answers Research question 71.24 in part for capped relations, and bears on Research question 71.43; a dated [write] note at each says how far.

71.1 How the manuscripts frame their questions

Source 01. The following questions separate tractable extensions of the present construction from the genuinely global compression problem. They are proposed investigations, not claims of solutions.

Source 03. The following questions separate tractable implementation work, quantitative mathematics, and the much stronger fixed-variable frontier. They are proposals from this investigation; calling them questions does not assert that each is unanswered in all existing literature.

Source 04. The following questions range from bounded construction problems to genuinely stronger fixed-arity goals. They are proposed research targets, not all asserted to be previously named open problems. Each is stated so that a success criterion and a likely failure mode are visible.

Source 05. The following agenda separates finite, testable development tasks from the genuinely stronger uniform arithmetic questions. None of the questions is claimed to have a new answer merely

because it is listed here.

Source 06. The following questions are proposed continuations, not a claim that every one is a newly identified open problem in the literature. They are organized to separate tasks that build directly on the proved compiler from those that confront the finite-fold frontier.

Source 07. The questions in this section are unresolved by this article. Except for the explicitly identified finite-fold topic, they are proposed construction and optimization problems, not assertions that a literature-wide open-problem status has been established.

Source 07, closing paragraph of its question section. The most immediate projects are the first-failure and FIFO formalizations, a complete fixed-read network compiler, and optimization of the quartic variable count. They have precise interfaces and can be validated independently. The fixed-arity problem is a separate, more demanding line of investigation, for which the present explicit unique traces identify exactly what information a successful compression must preserve.

[write] Manuscript 02 printed its ten questions under the heading “Research questions with concrete targets” without an introductory paragraph.

71.2 Verified compilers and fibre-preserving interfaces

Research question 71.1 (A verified single-fold compiler interface; sources 01, 02, 03, 04, 05, 06, 07). *Source 01 (Formalizing the exact resource envelope).* Can ProveIt obtain a reusable finite-type theorem for box-guarded translations whose specialization automatically produces Petri and finite-control counter adapters? A useful first milestone is a kernel-checked equivalence between all multiset serializations and (17)–(18), including empty support, finite upper bounds, and empty composition domains. The elementary finite combinatorics make this a focused formalization target.

Source 02 (A verified single-fold compiler). Formalize the total sign gadget, signed normalization, and the endpoint propagation theorem, then extract a compiler whose output includes a proof of both equivalence and unique extension. A useful first milestone is the complete degree-two template, including zero polynomials and $T = 0$, rather than a generic compiler with unproved boundary cases.

Source 03 (What is the right verified compiler interface?). Build an actual Lean polynomial emitter whose contract contains soundness, completeness, inverse decoding, degree, sparsity, and height bounds. The two backends provide a test of whether the abstraction separates semantics from resource properties cleanly. A useful first milestone is formal verification of both four-event exports in this archive, followed by the symbolic theorem for arbitrary L ; finite examples alone are not the endpoint.

Source 04 (Canonicity-preserving arithmetic library design). Develop a library of polynomial gadgets certified not only by existential equivalences but by explicit solution-set bijections, together with exact cost and height annotations.

Source 05 (Kernel-checked fibre preservation). Can the full equivalence of Theorem 11.2, including every slack coordinate, be formalized as a reusable compiler theorem in ProveIt? A concrete first target is to reproduce the raw, quartic, and quadratic variable counts with a finite-indexed polynomial definition and an executable witness constructor.

Source 06, Question 10 (A small verified compiler and a multiplicity audit). Formalize the resource pair action, the minimum and selector gadgets, the Boolean signature product, and the structural single-fold theorem as a compact independent layer. The decisive endpoint should state a bijection between the specified semantic witnesses and polynomial roots, not merely an implication from a generated certificate to acceptance. A subsequent integration with ProveIt’s existing Diophantine interfaces could preserve that stronger contract. The audit should explicitly check natural versus integer subtraction, zero-length traces, inactive branches, and every projection that turns a parameter into a witness.

Source 07 (Formal arithmetic-DAG compilation). Implement the proposed ProveIt modules for first-failure products, word reconstruction, and certificate evaluation. The first deliverable should be the full equivalence and uniqueness theorem for the cyclic-tag stream compiler, not just a theorem that a supplied example evaluates to zero. Degree-bound reflection can follow independently.

Source 04. Priority selection, canonical remainder, fixed-pattern matching, context reconstruction, and unique-extension quadratization are concrete initial modules. The important regression tests are inactive-variable freedom, integer-versus-natural subtraction, duplicate state encodings, and mismatched semantics between compiler stages. Formalizing the Petri-net theorem is an especially contained first milestone because its state and monitor equations are only quadratic.

Source 05. The decisive acceptance test is the identity $\text{encode} \circ \text{decode} = \text{id}$ on the full solution subtype. A proof only of solvability equivalence is not sufficient for the counting applications.

Research question 71.2 (Which normal forms preserve canonical fibers?; source 03). Investigate small coefficient sets, bounded variable occurrence, and more restrictive quadratic gate types. Introducing a uniquely defined copy of a coordinate preserves parsimony, but replacing natural coordinates by ordinary integer sum-of-squares representations generally does not. Identify exactly which normal-form transformations preserve existence only, finite fibers, or singleton fibers. This could become a reusable theorem library independent of memory checking.

Research question 71.3 (Substrate interfaces with explicit witness contracts; source 02). Build counter-machine and FRACSTRAN libraries whose one-step arithmetic certificates are single-fold and whose composition interfaces state exactly which property survives. Use the polynomial accelerator only when a checked trajectory certificate is available; otherwise fall back to an ordinary existence-preserving backend. The resulting mixed system would make the boundary between established local uniqueness and unresolved unbounded compression visible to users.

[write] Batch 62, 30 September 2026: answered in part by Part XII (manuscript 10): a fixed repeated word of a FRACSTRAN or ordered multiset program, priority included, has a single-fold quartic certificate uniform in the program entries (Theorem 98.6), with an exact capacity formula and a checked interval accelerator (Theorems 95.2 and 96.4). A library with composition interfaces, and the fallback to an existence-only backend, remain to be built.

71.3 Commutation, independence and canonical representatives

Research question 71.4 (State-dependent independence and commutation beyond a fixed relation; sources 01, 04, 05, 06, 07). *Source 01 (Dynamic rather than global independence).* Can state-dependent commuting diamonds support a canonical arithmetic trace quotient? Global partial-map equality gives a congruence for free. A rule that swaps actions only when both orders happen to work at the current state may lose a simple global normal form. A successful extension needs explicit coherence conditions and a proof of canonicity, not merely local diamond checks. Small counterexamples should be sought before designing the polynomial layer.

Source 04 (Canonical certificates for dynamically independent actions). Can the quadratic monitor be generalized to a sound state-dependent independence relation while retaining one solution per chosen execution-equivalence class and a polynomial-size bounded certificate?

Source 04 (Beyond disjoint supports). Find effectively checkable families of transition pairs with overlapping supports that satisfy the enabled-swap property for all reachable markings, and derive more aggressive trace quotients without losing witness-faithfulness.

Source 05 (State-sensitive commutation). Can the static relation be replaced by a state-dependent diamond test while retaining a unique polynomial-size normal-form verifier? What precise consistency conditions replace the trace-monoid forbidden-factor language?

Source 06, Question 5 (Invariant-sensitive and state-dependent swaps). Replace equality of partial actions on all markings by equality on a prescribed invariant, such as exactly one control token. This could admit more commutations than (29). Which classes of invariants permit an effective exact test, and when do the resulting swaps support a sound canonical cross-section? If the permitted swap depends on the current marking, a fixed trace monoid may no longer be the right object; confluence and uniqueness then require new proofs, not just a larger pair relation.

Source 07 (Concurrent traces rather than schedules). For communicating FIFO processes, can solutions represent equivalence classes of independent event schedules while preserving channel order? A swap can change which symbols an event observes, so a static independence relation is not automatically sufficient. A semantic commutation criterion should be proved before importing schedule-normalization techniques.

Source 04 (Canonical certificates for dynamically independent actions). The current proof relies on a fixed trace congruence. If independence changes with state, a swap can change later independence tests and equivalence need not be described by a fixed graph. A meaningful first target is a finite-state system with a certified local diamond property, together with a canonicalization theorem that explicitly proves class preservation. Simply evaluating a different I at every row of (57) is not justified by Section 14.

Source 04 (Beyond disjoint supports). For example, shared read-only resources can permit commutation beyond literal support disjointness. The right hypothesis is preservation of both enabling and outcome, not merely commutativity of the net incidence vectors. A certificate for an invariant region of markings may justify additional swaps. Any added pair must preserve the complete endpoint language used in Theorem 15.2.

Source 05. A local diamond at one marking is insufficient to justify arbitrary sequences of swaps. A satisfactory answer must show invariance of enabledness across the whole generated equivalence and uniqueness of the chosen representative, not just local endpoint equality.

[write] The hypothesis every part of this question asks to relax is the one fixed in Theorem 11.2: a single symmetric irreflexive relation $I \subseteq I_{\max}$, where $I_{\max}$ of (8) contains exactly the pairs whose two-step partial actions agree at every marking. Under that hypothesis a swap is sound in every context; the six texts above ask what replaces it when commutation is allowed to depend on the current state, on a prescribed invariant, or, for FIFO channels, on the symbols an event observes.

Research question 71.5 (Structure of independence graphs; source 05). Which graph parameters permit smaller flag systems or tractable exact class counting? Candidates include neighbourhood inclusion, modular decomposition, and widths of a graph encoding the residual interactions.

A useful project would report both coefficient sparsity and actual enumeration behaviour on a library of graphs. Theorem 4.4 rules out assuming that Petri-net semantic commutation graphs belong to a narrow graph class in general.

Research question 71.6 (Minimal signatures and algebraic structure; source 06, Question 1). For a given independence graph and alphabet order, what is the smallest exact block signature for normality? Our upper bound of $2 \cdot 6^q$ possible records is often redundant, especially when the scalar acceptance bit is zero. Determine the reachable signature submonoid, compare it with the syntactic monoid of the normal-form language, and characterize when every reachable signature is already idempotent. The general exponent two is sharp, but its dependence on graph structure can be finer. A useful outcome would be a smaller circuit representation together with a proof that it preserves the $S^2 = S^3$ law.

Research question 71.7 (Compressed normalization rather than testing; source 06, Question 2). Given a parametrized repetition expression, can one produce a comparably small expression for its lexicographic normal word, uniformly in the parameters? The present result only tests whether the original expansion is normal. In the fully commuting case, sorting is controlled by letter counts, but general dependencies

retain nontrivial ordering information. The key issues are the output representation, whether new parameter arithmetic is allowed, and whether polynomial-size output can always exist. A useful first target is bounded dependence width or a fixed small alphabet.

Research question 71.8 (Trace-injective parameterizations; source 06, Question 3). Which repetition templates have at most one parameter tuple per trace, or finitely many tuples per trace? The example $(a^{k_1})^{k_2}$ shows why empty powers and factorization can destroy injectivity. Can redundant parameters be removed in a useful decidable fragment while retaining a compact expression and a witness-preserving compiler? A classification for flat schemes would connect the Presburger execution relation with trace equality, but execution equality alone is not a sufficient substitute for the latter.

Research question 71.9 (Unique acceleration of repeated causal patterns; source 01). The accelerator compresses one commuting multiset, and Proposition 22.2 already handles a prescribed fixed word repeated an arbitrary number of times. Can a general causal trace be assigned a unique compressed segmentation into such patterns? The unresolved step is to infer the decomposition without multiple period or boundary choices representing the same trace. A useful restricted target is a class with a canonical primitive-period factorization, together with a proof that the arithmetic witnesses preserve that factorization uniquely.

Research question 71.10 (Canonicity inside restricted schedule families; source 06, Question 4). Find effective syntactic conditions ensuring that a schedule family contains the normal representative of every trace it contains. Under such a trace-closure condition, the canonical compiler would preserve existential reachability within the family, avoiding the failure in Remark 25.5. One may alternatively seek a certified expansion of the family that is trace-closed but has controlled size. The cap-at-two theorem makes testing a proposed representative cheap; it does not by itself construct a family containing that representative.

71.4 Counting, weights and relaxations

Research question 71.11 (Counting trace classes, histories and certificates; sources 01, 03, 04, 05, 06). *Source 01 (Counting and generating functions)*. For a finite invariant marking region, can the canonical-layer transition system yield useful exact rational generating functions for trace counts, weighted by event number or cost? A finite-state construction is plausible, but extracting small formulas and comparing different valid independence relations remain worthwhile tasks. The quartic roots give a direct arithmetic interpretation of those counts and a way to test candidate recurrences on finite instances.

Source 03 (Can concurrency retain meaningful counting semantics?). A sequential log gives one total order of accesses. For a concurrent execution, different legal linearizations may describe the same partially ordered history. An existential timestamp certificate would count linearizations, not necessarily semantic histories. Develop canonical linearization conventions or explicitly weighted quotient semantics, distinguishing sequential consistency from weaker memory models. The local memory checker is a backend, not a solution to those semantic choices.

Source 04 (Counting directly from structured residual systems). For which structural restrictions on a net or independence graph can the quartic certificates be counted or sampled efficiently without expanding the entire reachable-state automaton?

Source 05 (Counting algorithms under explicit parameters). Which parameters yield fixed-parameter or output-sensitive algorithms for counting the lattice points of these particular trace polytopes? Parameters should be stated precisely: fixed horizon, small active transition set, bounded residual-graph width, or a restricted control architecture give different problems.

Source 06, Question 11 (Counting beyond bounded words). Extend the bounded trace-class bijection to compressed families with a known trace-injective parameterization, and then to weighted trace

counting. A weight assigned to an execution must be invariant under the chosen commutations before it can be interpreted as a trace weight. The main obstruction for unbounded families is not the local quartic encoding; it is controlling the number of parameterizations of a behavior and the witnesses introduced by any unbounded coding mechanism.

Source 04. Potential parameters include the interaction-graph treewidth, the size of conflict sets, and the number of shared resource places. Theorem 16.2 rules out a general polynomial-time exact algorithm under standard complexity expectations, but does not rule out parameterized algorithms. The natural comparison point is current trace counting and sampling for regular languages [21]; input representation and trace closure must remain explicit.

Source 05. An empirical benchmark should compare direct canonical-word generation, lattice-point methods, and dynamic programming. The #P-completeness result gives a worst-case boundary, not a prohibition on useful structured algorithms.

Research question 71.12 (Weight-preserving quotients; source 05). For which state-dependent costs, probabilities, or resource observations does a commutation quotient preserve the intended observable? Can the necessary observation data be augmented without destroying a useful independence relation?

Label-additive weights are fully handled by Corollary 12.1. Peak space, scheduler probabilities, and costs depending on the current marking require additional analysis. The correct target may be a weighted sum over linear extensions rather than one unit per class.

Research question 71.13 (What extends to weighted, probabilistic, or algebraic computation?; source 03). The fair-bit formula handles exact positive path weights with canonical stopped padding. Extend the construction to bounded-denominator rational transition probabilities and track a uniquely defined common denominator. For quantum circuits, path counting alone is insufficient because amplitudes can cancel; one must specify exact algebraic coefficient representations and certify the amplitude arithmetic and measurement rule. The distinction between counting paths and summing amplitudes is fundamental.

Research question 71.14 (Nondeterminism and path multiplicities; source 02). For a finite family of polynomial phases, can a certificate have one solution per distinct execution, rather than one solution per endpoint? Canonical encodings of branch sequences and phase boundaries are needed, especially when two phase descriptions overlap. A first tractable case is a fixed acyclic control graph with locally deterministic phases and explicitly labeled nondeterministic edges.

Research question 71.15 (Integral or ideal relaxations; source 05). For which subclasses of nets and independence relations is the canonical trace polytope integral? Can one characterize subclasses admitting a polynomial-size ideal extension, meaning an extension whose real projection is the convex hull of the relevant discrete encodings?

Example 11.4 supplies a small explicit obstruction to unrestricted integrality. Total unimodularity of a suitable residual matrix would be a sufficient condition, but the canonicalization constraints need not preserve it. The general counting hardness theorem also prevents treating a compact formulation as a general counting algorithm.

71.5 Size, degree and height of certificates

Research question 71.16 (Sharp degree, arity, domain and variable-count comparisons; sources 01, 07). *Source 01 (Sharp degree, arity, and domain comparisons).* How much of the quartic construction is essential? Totals, forced-zero gains, and some blocker products can be eliminated in special cases. A systematic optimization pass should preserve the complete-root bijection, rather than only satisfiability. Extending the result from natural witnesses to unrestricted integer witnesses is also not automatic: a

four-squares conversion can introduce many witnesses for one natural value. A separate canonical integer-domain encoding is needed for a parsimonious statement over $\mathbb{Z}$.

Source 07 (Nearly trace-minimal quartics). Can a structural cyclic-tag compiler retain the actual T consumed bits, use only $T + O(1)$ total natural variables, have degree at most four, and have polynomial, preferably linear, arithmetic-circuit size? The present stream construction achieves the variable count with growing degree; the local construction achieves quartic degree with $3T - 2$ coordinates.

Source 07. A useful first milestone is to improve the coefficient of T in the quartic construction while retaining complete witness uniqueness. Any lower-bound attempt must exclude the trivial compiler that first simulates the bounded run and outputs a constant.

[write] Manuscript 05's convex quadratic certificate (Theorem 11.2, degree two, Part II) answers part of manuscript 01's question: for length- T trace classes of Petri nets and of interval-guarded translations (Corollary 11.5), degree four is not needed. The height-bounded quartic of Theorem 20.1 and the integer-domain part remain open, and so does manuscript 07's variable-count question for cyclic-tag compilers, which the degree-two certificate does not address.

[write] Batch 62, 30 September 2026: Part XI (manuscript 09) answers a further part for fixed-arity representations by polynomials that are nonnegative on the whole real orthant: degree three adds nothing to degree two, which represents exactly the semilinear sets, single-fold; degree four is needed for every nonsemilinear set, even the squares, and suffices for every computably enumerable set (Theorems 90.2 and 90.14 and Corollary 90.16). The threshold depends on the positivity hypothesis: without it, $x - y^2 = 0$ represents the squares in degree two. The integer-domain part of manuscript 01's question and manuscript 07's variable-count question stay open. Part XI's assembly compiler (Theorem 91.7) is a further degree-two instance for bounded families.

Research question 71.17 (Smaller canonical predecessor circuits; source 01). Can the $(H - 1)d$ blocker witnesses be reduced by sharing common subproducts among the dependence neighborhoods $D(a)$? The objective is a smaller *functional* Boolean circuit, so every introduced gate remains uniquely determined. Graph classes with repeated neighborhoods, bounded treewidth, or a small neighborhood basis offer concrete cases. The benchmark should report witness count, residual count, and expanded polynomial size separately.

Research question 71.18 (Smaller uniquely extended affine systems; source 05). Can the five local auxiliary variables per time-label pair in (45)–(49) be reduced uniformly while retaining a convex quadratic sum of affine squares, bounded real relaxation, and unique natural extension?

The displayed constants are not optimality claims. Eliminating variables symbolically, sharing complementary quantities, or exploiting implication relations among flags may help. The automaton lower bound concerns Boolean memory and does not obstruct such arithmetic improvements by itself.

Research question 71.19 (Witness and coefficient optimization; source 06, Question 8). How far can the explicit variable bounds be reduced while preserving unique lifting and manageable coefficient heights? Potential improvements include constant propagation, shared exponent selectors, common-subexpression elimination, suppression of determined bit equations, and direct compilation of the square-saturation formula rather than a full signature square. The relevant objective is a tuple of costs: witness count, residual count, degree, sparsity, coefficient bit length, and verification depth. Optimizing only an expanded monomial count can make the actual certificate worse. Comparisons with universal-equation operation counts must use the same charging conventions and witness domains.

Research question 71.20 (Reduce the cubic arithmetic overhead; source 02). Can the full certificate be reduced from $O(d^3)$ to $O(d^2)$ natural coordinates while retaining a quartic equation and a polynomial-time witness generator? The geometric coordinates are already quadratic. A promising target is a canonical merge of neighboring sign partitions, avoiding all parent-child pairs, together with shared

multipoint polynomial evaluation. Both the merge and its arithmetic intermediates must have unique encodings; an arbitrary permutation witness would not do.

Research question 71.21 (Quantitative lower bounds; source 02). What lower bounds can be proved for canonical sign-table certificates under a specified gate model? A bound on our particular interval layout would not be a lower bound on all Diophantine representations. Separating geometric information, arithmetic circuit size, and bit height is necessary before any optimality statement is meaningful.

Research question 71.22 (Certificate height and practical checking; source 02). Derive tighter coefficient-sensitive bounds for every intermediate wire, not only asymptotic bit bounds. These can guide bounded exact search, serialization, and independently checked resource estimates. Compare the canonical root cutoff with a minimal stabilization point; a smaller cutoff is useful only when its certification does not cost more than the saved sign-table work.

Research question 71.23 (Can the size–height tradeoff be improved substantially?; source 03). The linear backend has $O(L)$ scalar witnesses but $O(L^2(b + \log L))$ maximum bit length, whereas the comparison backend has near-linear scalar size and essentially input-sized witnesses. Is there a canonical $O(L)$ -coordinate memory certificate with $O(L(b + \log L))$, or even $O(b + \log L)$, bits per coordinate? A first concrete target is an exact multiset certificate that avoids a base of size D^{4L} while introducing no choice of primes, hash challenges, or noncanonical factorization witnesses. The comparison lower bound does not settle this question.

Research question 71.24 (What is the best explicit scalar constant?; source 03). The count $24L$ deliberately uses a simple length- L power chain. Binary powering reduces the power stage to $O(\log L)$ coordinates. Eliminating redundant output fields, optimizing record packing, and sharing certified subexpressions may improve the leading constant further. Every proposed optimization must preserve injectivity of the packed record and uniqueness of all discarded or shared witnesses. Publish the exact generated residuals and count proofs, not only an operation-count estimate.

[write] Batch 78, 2 October 2026: answered in part, for capped relations, by Part XV. For a zero-initialized log with external address and value caps fixed with L , manuscript 14’s certificate (Theorem 144.6) has exactly $17L - 4$ witnesses and $17L - 4$ residuals ($16L - 4$ with a precomputed base, Corollary 144.7), with its residuals exported for an example and its counts proved; manuscript 13’s (Theorem 144.4) computes the base by binary powering, as suggested here, and the review [65] removes one residual per adjacent pair from its example. Both relations differ from Part V’s, which has no caps (Section 144.4), so the question stays open for Part V’s relation.

Research question 71.25 (Can one minimize total witness bit volume?; source 03). Maximum coordinate height is not the only meaningful metric. Balanced product trees can reduce the sum of intermediate bit lengths while retaining linear many product nodes. A systematic comparison of sequential products, balanced products, carry-free block packing, and sorting networks could yield an explicit Pareto frontier for scalar count, total certificate bits, arithmetic work, and parallel depth. The current implementation uses sequential products for a transparent exact count; it is not claimed optimal in those other metrics.

Research question 71.26 (Can canonical routing be made practical at optimal comparison order?; source 03). The AKS theorem supplies the asymptotic bound but not a small practical backend here. A routing-based memory checker may have fewer constraints, yet often admits multiple internal routes for one permutation. Can those routes be canonically normalized with low overhead and a short proof? A deterministic routing algorithm is not enough unless its own computation is certified without reintroducing the original overhead or unbounded auxiliary choices.

Research question 71.27 (Locality with polynomial bit-height; source 04). Can a canonical DAG or balanced-tree coding retain context-local certificate size and also provide polynomial witness bit lengths in the size of a succinct input representation and the performed updates?

The pairing code pays for repeated constructor nesting by repeated squaring. A DAG can share repeated subterms, but arbitrary node numbering introduces enormous witness multiplicity. One needs canonical numbering or a quotient theorem for node-renaming equivalence, plus unique proofs of pointer validity and acyclicity. A first useful milestone is a linear-height representation of purely applicative terms with a canonical topological order.

[write] Batch 62, 30 September 2026: answered in part by Part X (manuscript 08) for bounded-local heaps: birth-order normalization numbers dynamically created nodes canonically modulo renaming (Theorem 78.1), the heap and graph compilers are bijective for it (Theorems 79.2 and 81.1), and for finite-tag pointer programs every witness coordinate has $O(\log(S + T + 2))$ bits (Proposition 79.4). Context-local certificates for arbitrary term DAGs, with unique proofs of pointer validity and acyclicity, remain open.

Research question 71.28 (Sharp arithmetic-memory tradeoffs; source 04). Can the r flags in the monitor be replaced by fewer natural variables without hiding exponential bit-height, high degree, or a costly digit-extraction gadget?

Theorem 14.3 gives a streaming bit lower bound, not a variable lower bound. Packing all flags into one integer is easy at the data level but makes bit access a new arithmetic obligation. A useful theorem would state a tradeoff among witness arity, witness height, residual degree, and circuit description size, with an explicit computational model rather than an informal “compression” claim.

Research question 71.29 (A linear-size zero-slack guard; source 07). Can the no-slack construction be made linear-size in T for a fixed program, while retaining degree $O(T)$ and precisely the consumed-symbol coordinates? The bounded-root guard uses quadratic size in the relevant emptying regime. The question concerns circuit structure, not existence of some polynomial with the correct finite zero set.

Research question 71.30 (Smaller uniquely determined guard values; source 07). Can a guard with similarly small circuitry replace the product of all queue lengths by a substantially smaller canonical natural witness? A useful objective is a bound linear in the trace length plus the logarithm of the maximum queue length, rather than their product, without introducing many separate slacks.

71.6 Richer actions, storage and trajectories

Research question 71.31 (Beyond translations; source 01). Which reset, affine, stack, or indirect-addressing primitives admit exact fixed-arity unique-witness acceleration? The present extremal-prefix argument works because displacement sums are independent of the current value. Reset and multiplication destroy that property. One concrete target is diagonal affine actions on invariant regions where products and guards have a controlled closed form; an answer must separately account for witnesses used to represent any exponentiation.

Research question 71.32 (Reset, copy, and richer guarded updates; source 05). Can Theorem 4.5 be extended to partial affine maps with reset or copy operations, with a similarly explicit exact commutation criterion?

Unlike translations, different affine expressions can agree on a lower-dimensional guarded domain without having equal coefficient matrices. Any exact criterion must account for those domain equalities as well as for empty compositions. A bounded compiler can still use finite Boolean verification, but that does not supply the desired symbolic all-state criterion.

Research question 71.33 (New storage monoids and genuine exponential growth; source 06, Question 9). Identify further action algebras satisfying the local unique-summary contract of Proposition 61.1. Promising structured cases include finite-state products, selected queue restrictions, and stack fragments

with bounded unmatched contexts. The essential test is whether arbitrary powers retain an exact finite summary with controlled polynomial witnesses. Affine multiplication of a runtime counter and unbounded string coding should be treated as separate exponential-growth problems, not added as innocuous gates.

[write] The queue case is partly answered by Part VI (manuscript 07): Theorem 52.2 gives a witness-faithful polynomial whose natural zeros are in bijection with the exact T -event executions of a fixed-read FIFO network program. That is a bounded-run certificate for a fixed schedule; it does not supply the exact finite summary of arbitrary powers that the contract of Proposition 61.1 asks for. Networks whose read counts depend on data or on emptiness tests stay open (Research question 71.34).

Research question 71.34 (Data-dependent read counts and zero tests; source 07). Extend Theorem 52.2 from fixed phase-dependent read counts to finite-control networks whose consumption sizes depend on data or on emptiness tests. Specify exact witness counts and ensure that local branch selection and global availability do not admit circular justifications. The fixed-read theorem gives a controlled starting point rather than a completed compiler for every actor language.

Research question 71.35 (Rational and piecewise-polynomial memory; source 07). Which extensions of the scalar insertion architecture invalidate Theorem 50.4? Study rational insertion maps with poles excluded from the code image, or a bounded number of polynomial branches. The corresponding centralizer restrictions and the natural-number image condition both matter. A classification should separate genuine scalar memory from hidden finite-state coordinates.

Research question 71.36 (Automatic trajectory discovery; source 02). Develop a front end recognizing polynomially iterable updates and producing the trajectory identities as checked certificates. For affine maps, a nilpotence witness or an identity of the form $M^h(M^p - I)^r = 0$ is an immediate target. For nonlinear maps, triangular coordinate changes and verified discrete antidifferentiation could enlarge the recognized class. The synthesis procedure may be incomplete while its accepted certificates remain sound.

Research question 71.37 (Variable periods and variable degrees; source 02). What uniformity survives when the degree bound or residue period is part of the numerical input rather than fixed externally? A family of polynomials with growing arity does not answer this question. Any proposed single fixed polynomial must account for the encoding and checking of variable-length coefficient data without silently using an ordinary MRDP representation that loses single-foldness.

[write] Batch 63, 30 September 2026: an analogue is answered in part by Part XIV (manuscript 12) for routing networks. There the run lengths, and hence the period lengths D_v , are free parameters of one single-fold quartic for each routing topology (Theorem 128.1), so the period is part of the numerical input; the number of blocks stays fixed. For periods described by Boolean circuits, uniformly produced bounded-degree certificates with short witnesses would give $\text{NP} = \text{coNP}$ (Theorem 131.2). For polynomial trajectories, and for variable degrees, the question stays open.

Research question 71.38 (Nonlinear guards on fixed-residue trajectories; source 02). Implement the residue-class front end of Theorem 32.4, with canonical Euclidean division and empty-class normalization. Extend its nontermination theorem using one canonical root bound per residue class. This is a concrete development target within the proved framework, not a claim about arbitrary exponential recurrences.

Research question 71.39 (Parameter sharing, depth, and resource tradeoffs; source 06, Question 6). Determine which restrictions on global parameter reuse restore decidability at nesting depth two. The universality proof reuses numerical variables across several monomial blocks and circuit equations. Restrictions such as one syntactic occurrence per parameter, bounded occurrence count, or an acyclic

parameter-incidence pattern may separate tractable subclasses. Independently, determine quantitative tradeoffs between the number of resources and repetition depth. One counter at depth four and finitely many counters at depth two suffice; exact small-resource bounds would require an explicit, audited construction rather than a qualitative MRDP invocation.

Research question 71.40 (The separated cubic interface; source 06, Question 7). Use Theorem 26.3 as an exact interface for studying separated one-counter schemes of depth three. Every theorem about this fragment translates to a theorem about natural solutions of one cubic polynomial, and conversely. Useful restricted targets include bounded numbers of parameters, sparse support, prescribed sign patterns, or limited parameter reuse. The correspondence prevents one from mistaking an unproved general cubic decision assertion for a routine consequence of counter simplicity.

Research question 71.41 (Unbounded but structurally restricted finite-foldness; source 01). What larger classes of fixed-skeleton counter programs admit endpoint-level finite-fold certificates? Corollaries 22.3 and 22.4 give simple sufficient conditions. Potential extensions use guard-induced count bounds even when the displacement matrix has a nonnegative kernel, or unique solutions of a lower-triangular count system when full column rank fails. The target should be a checkable structural criterion with a proved fibre bound.

71.7 Rewriting, graph evaluation and comparisons across substrates

Research question 71.42 (Bridges between combinatory or graph evaluation and certificates; sources 01, 03, 04). *Source 01 (A fully specified SKI/Iota bridge)*. Can the local quadratic application-code equations be assembled into a parsimonious representation of a chosen bounded evaluation semantics? The immediate milestones are a unique-witness term-validity checker under a stated size bound, a canonical redex-position encoding, and an evaluation-strategy-specific reflection theorem. The repository's distinction between scoped contextual lambda reduction and weak call-by-value semantics must remain explicit.

Source 03 (How should graph strategies connect to the existing calculi?). Establish the exact relation between the immutable spine evaluator and the repository's contextual combinator reductions, then between the particular lambda semantics already present in its separate developments. Which source observables are preserved and reflected? Which administrative steps should be counted, erased, or quotiented? For probabilistic or nondeterministic evaluators, a quotient on histories can change path multiplicity and therefore requires a measure-preserving argument, not just a reachability simulation.

Source 04 (Existential SKI schedules without path duplication). For horizon T and a maximum context depth D , construct a certificate with $\text{poly}(T, D)$ local variables whose solutions are in bijection with valid scheduled SKI reductions, while the redex paths themselves are chosen existentially.

Source 04. The fixed-schedule theorem is not this result. A direct disjunction over all paths of length at most D has exponentially many branches. A prospective construction should choose one direction bit per active depth, enforce one canonical stopping point, and force all unused suffix variables to zero. It must also guarantee that a shorter path does not acquire many padded encodings. An even stronger version would choose a canonical reduction strategy, rather than count arbitrary schedules.

Research question 71.43 (Faithful graph-rewriting front ends; source 05). Can bounded term-graph or interaction-net histories be compiled with a canonical node-allocation discipline and a proved commutation quotient that respects shared substructure?

The important distinction is between counting graph-rewrite histories, tree interpretations, and observable normal forms. Allocation symmetries and accidental free names can destroy parsimony even when the underlying reduction relation is well understood.

[write] Batch 62, 30 September 2026: answered in part by Part X (manuscript 08): a canonical node-allocation discipline, birth-order normalization, with a bijection between natural zeros and bounded-local graph histories modulo fresh names (Theorems 78.1 and 81.1). The commutation quotient, which would identify independent rewrites, is not supplied; manuscript 08 poses it again (its question on quotienting independent schedules).

[write] Batch 78, 2 October 2026: for interaction nets, Part XV (manuscripts 13–15) supplies faithful bounded front ends for Lafont’s interaction combinators, with a fixed fresh-name allocation and exact cyclic-wire counts: one natural zero per legal scheduled execution (Theorems 145.3 and 146.7). The commutation quotient is still not supplied; all three manuscripts pose it again (Section 149).

Research question 71.44 (Cost-preserving compilation between substrates; source 04). Build translations among counter machines, FRACTRAN, and a deterministic combinatory evaluator that preserve a canonical execution object and give explicit arithmetic-certificate overheads.

Forward simulation is not enough. The target must not introduce multiple admissible administrative schedules for one source step, unless those schedules are canonically quotiented. A useful statement would identify a unique target macrotrace for each source step, prove reflection at designated boundaries, and bound variables, time expansion, and bit-height. This would make cross-substrate comparisons quantitative and semantically trustworthy.

Research question 71.45 (Spatial certificate transport with explicit clocks; source 07). Build a formally checked bridge from a concrete cyclic-tag program to its glider or Rule 110 realization, recording exact macro-step boundaries, periodic background conventions, and observable halting events. Then compare a transported queue certificate with a direct space-time polynomial, including bit costs as well as variable counts.

Research question 71.46 (Quantitative universal compilation; source 07). Compare universal simulation routes by the final arithmetic resource vector: number of consumed symbols, polynomial degree, circuit size, coefficient bit size, and witness bit height. Simulation time alone does not determine that vector. A useful experiment is to compile the same family of small algorithms through both deletion-tag and cyclic-tag routes, with formal clock correspondence.

[write] This question compares horizon-indexed compilation routes by their resource vectors; it asks nothing about fixed arity or fibres, so it is printed here and not merged into Research question 71.48.

Research question 71.47 (Representation-size versus operation-count optimization; source 05). Can ProveIt’s compact universal-equation and straight-line-certificate work be given a semantic interface that records which optimizations preserve arithmetic fibres, and which preserve only solvability?

Operation counts for fixed universal certificates and variable counts for a time-unrolled compiler measure different resources. A useful comparative study should report both without conflating them, and should test whether elimination or relation-combining steps introduce multiple witnesses.

[write] This question stays a question. It concerns ProveIt’s operation-counted straight-line certificates (the Hilbert’s-tenth-problem project’s 75-operation universal certificate [1]), which measure a different resource from the witness counts of this report, and no work in that project is opened here.

71.8 The fixed-arity frontier

Research question 71.48 (Fixed-arity compression with finite or single fibres; sources 01, 02, 03, 04, 06, 07). *Source 01 (The universal compression problem)*. Can a fixed-arity ordinary polynomial represent first-halt computations of a universal interpreter with finite, or single, witness fibres? Theorem 62.1 identifies this as the full finite-fold or single-fold MRDP principle, not an omitted routine assembly

lemma. A useful contribution would isolate a genuinely canonical unbounded coding primitive and prove that every helper variable has controlled multiplicity. Merely choosing one semantic run or its least numerical code does not discharge that obligation.

Source 02 (The first genuinely exponential extension). Identify a meaningful subclass of exponential-polynomial trajectories that admits finite-fold or single-fold ordinary polynomial certificates without assuming such a representation for 2^n . Any extension including the doubling graph already has the strength established in Proposition 64.1. A conditional theorem that exposes this dependency is useful; hiding it inside an exponentiation predicate is not.

Source 03 (Can one make genuine progress on fixed-variable compression?). Theorem 62.1 identifies the strongest frontier precisely. A single-fold polynomial for one universal halting relation would settle the general single-fold question; a finite-fold one would settle its finite-fold counterpart. Investigate restricted trace families whose coding and all decoding relations have independently proved finite fibers. Any attempted extension to universal traces must explicitly discharge the new uniqueness or finiteness obligations. The fact that the source machine is deterministic is not such a proof, and no total computable global root bound can be supplied by Proposition 62.2.

Source 04 (The universal fixed-arity challenge). Can one preserve the first-halting uniqueness of Corollary 54.3 while eliminating the horizon-dependent number of variables for a universal interpreter?

Source 06, Question 12 (Finite-foldness in its minimal substrate). Study finite or singleton successful-parameter fibers directly in the separated normal forms of Theorem 67.1. The theorem shows that an unrestricted positive result would have the full force of the corresponding Diophantine conjecture. More focused questions concern special computably enumerable relations or special polynomial support patterns whose schedule fibers can be controlled constructively. An answer must constrain the *parameter tuples*; uniqueness of the auxiliary arithmetic certificate is already supplied here and is not the missing step.

Source 07 (Fixed-arity compression with controlled fibers). Can one compress these canonical exact-halting certificates into a fixed number of Diophantine coordinates while preserving finite or unique fibers? An approach must identify and control every auxiliary introduced by word coding and bounded verification. This is the link to the published finite-fold research program [11], not a consequence already contained in the present theorems.

Source 04. Theorem 62.1 explains the strength of a positive answer. This is the genuinely global compression problem, rather than a consequence of the existing deterministic gadgets. Any proposal should identify the exact number-theoretic lemma controlling the fibers of trace encoding, bounded universal elimination, or exponentiation replacement. The witness-height obstruction also shows that no computably uniform bound on positive witnesses can accompany a universal solution.

[write] The source theorems cited in this question are merged in Part IX as Theorem 62.1 (the universal-halting equivalence) and Proposition 62.2 (no computable witness-height bound).

[write] Batch 63, 30 September 2026: re-scoped by Part XIV (manuscript 12). The fixed-arity target need not involve histories at all: a single-fold (finite-fold) representation of the graph of one total, monotone, Boolean function computable in polynomial time is already equivalent to the single-fold (finite-fold) principle for all computably enumerable sets, with $2r + 3$ witnesses and degree at most $\max\{4, 2d\}$ from an r -witness, degree- d representation of the graph (Theorem 133.1; compare Theorem 118.1 and Proposition 64.1). A proposed compression theorem for total polynomial-time functions must therefore apply to that one function, as manuscript 12's question on the monotone rank bottleneck says.

Research question 71.49 (Restricted unbounded classes with fixed-arity finite-fold or single-fold compression; sources 04, 05). *Source 04 (Finite-fold compression for restricted loop patterns).* Identify nontrivial unbounded program classes whose canonical bounded tableaux can be compressed into one fixed-arity finite-fold polynomial, with an explicit proof controlling every auxiliary fiber.

Source 05 (Restricted fixed-dimensional compression). Which explicitly described nonuniversal families admit fixed-dimensional single-fold arithmetic representations of their histories? Can a theorem state a sufficient structural hypothesis on the transition system together with the exact arithmetic primitives needed for compression?

Source 04. A tractable starting point is flat control flow with a fixed number of loops and affine or multiplicative updates. The challenge is not merely to summarize a loop existentially, but to show that loop counts, intermediate summaries, and all number-theoretic auxiliaries admit finitely many choices. One should state precisely which restrictions prevent the construction from immediately covering a universal halting relation.

Source 05. A computable time bound alone is not a proof that its graph, or the associated sequence coding, has a single-fold polynomial representation. A successful restricted result must audit those arithmetic fibres. Theorem 62.1 provides a sharp warning whenever the family contains canonical universal first-halting histories.

[write] Batch 62, 30 September 2026: answered in part by Part XII (manuscript 10): for fixed words $w_1, \dots, w_s$ of a priority program, the relation that the trace begins with $w_1^{k_1} \dots w_s^{k_s}$ and ends at y has a single-fold fixed-arity quartic, with the program entries and the counts as free inputs (Corollary 99.1). With the counts existentially quantified, distinct factorizations may give several witnesses, so the fixed-endpoint version stays open. Part XIII (manuscript 11) gives negative guidance: its minimum-fuel relation is single-fold for each resource cutoff (Theorem 117.1), but removing the cutoff is as hard as the finite-fold problem (Theorem 118.1).

[write] Batch 63, 30 September 2026: answered in part by Part XIV (manuscript 12) for routing networks. For every finite run-length or grammar routing topology, one quartic of fixed dimension ($4n + 4m + s$, respectively $4n + 4g + 4c + s$ witnesses), uniform in unbounded initial loads and run lengths, has exactly one natural zero for each terminating input and none otherwise (Theorems 128.1 and 129.1). It represents the outcome, the odometer and sink outputs, rather than the history, which routing makes order-independent and which a canonical last-exit forest replaces. Circuit-defined periods mark the limit (Theorem 131.2), and the finite periodic networks form a decidable class, so no universal family is covered.

Research question 71.50 (Finite-fold arithmetic primitives; source 05). Can one establish finite-fold or single-fold polynomial representations for the sequence or growth relations actually used in a uniform history compressor, rather than for an unspecified black-box decoder?

This connects directly to the arithmetic questions in [9, 10]. The computational front end developed here isolates what such a result would need to preserve. It neither supplies nor assumes the missing number-theoretic theorem.

[write] Batch 62, 30 September 2026: re-scoped by Part XIII (manuscript 11): even a primitive-recursive, single-valued relation decidable in logarithmic space on unary inputs, the least-reservoir relation of one fixed 61-species reaction network, has a finite-fold (single-fold) representation exactly when every computably enumerable set has one (Theorem 118.1). Low complexity of a primitive is therefore not enough. Part XI shows that every semilinear relation has a single-fold quadratic representation (Theorem 90.14).

[write] Batch 63, 30 September 2026: re-scoped further by Part XIV (manuscript 12): the graph of one total, monotone, Boolean function computable in polynomial time on binary inputs, the rank function of a universal one-router system, has a finite-fold (single-fold) representation exactly when every computably enumerable set has one (Theorem 133.1). Monotonicity and a Boolean range do not help either.

71.9 What would constitute a stronger result (manuscript 02)

A successful optimization of the present compiler, or its formal verification, would strengthen a restricted theorem already proved here. A genuine extension to universal first-halting computations, or to a single-fold polynomial graph of 2^n , would cross a different boundary and have general representability consequences. Keeping those two kinds of progress separate prevents a modest but useful construction from being mistaken for a resolution of the finite-fold program.

72 Conclusions of manuscripts 05, 01, 04 and 06

[write] *The conclusions of manuscripts 02, 03 and 07 close Parts IV, V and VI, and those of manuscripts 08–12 close Parts X–XIV.*

72.1 Manuscript 05: Conclusion

The existence of a Diophantine representation and the preservation of a computation's witnesses are different mathematical requirements. For bounded executions, a fully explicit solution to the stronger requirement is available: compute exact partial-map commutations, select lexicographic representatives with a small Boolean state, and express the resulting verifier by affine equations with unique natural slacks. The final polynomial is convex quadratic, while its natural zeros still carry the full discrete computational content.

The resulting theorem package does more than certify reachability. It preserves class counts and label-additive weights, gives a bounded rational polytope with explicit coordinate bounds, handles interval-guarded counter programs, and explains both an automaton-size obstruction and a counting-complexity obstruction. The elementary ballot family and the exhaustive checks provide independent reference cases for an eventual formal implementation.

At the same time, a growing family of bounded certificates is not one fixed-dimensional universal equation. The exact equivalence in Theorem 62.1 identifies where a proposed uniform finite-fold compressor reaches the full strength of the classical finite-fold problem. This separation makes the construction useful without overstating it: the bounded theorem is proved and executable, and the stronger arithmetic frontier is stated precisely enough to guide further research.

72.2 Manuscript 01: Conclusion

For box-guarded translations, the exact resources needed for all serializations of a repeated block admit a surprisingly compact description: total coordinatewise losses plus the largest retained lower margin, and dually the smallest upper margin minus total gains. Global commutation turns this universal condition into exact existential acceleration. The resulting ordinary quartic has a unique complete natural witness and constant arity with respect to the repetition counts.

Combining that resource calculation with canonical causal layers yields an explicit quartic whose roots correspond one-for-one to executable trace classes of bounded height. This preserves a stronger semantic object than reachability alone, and it exposes the otherwise easy-to-miss roles of inactive slacks, padding, and auxiliary circuit variables. Fixed block programs provide additional unbounded-repetition cases, with rank and progress criteria for single-fold or finite-fold endpoint fibres.

The general unbounded problem remains sharply separated. Infinitely many real histories cannot all be encoded by finitely many roots at one endpoint pair, and unique semantic execution does not by itself canonicalize arithmetic sequence codes. A universal fixed-arity single-fold halting polynomial would settle the corresponding full MRDP principle. The present constructions provide explicit, testable

components on the tractable side of that boundary, together with a proof-oriented route for extending ProveIt.

72.3 Manuscript 04: Conclusion

The informative unit of Diophantine compilation is not merely a yes/no equivalence. It is a controlled fiber: the collection of arithmetic witnesses attached to a computational object. Canonical slacks handle guards, prefix circuits handle priority, symbolic flags remove redundant schedules, and local constructor equations handle rewriting without inspecting untouched subterms.

The resulting theorems give explicit quartic certificates with exact variable and residual counts for three different substrates. For concurrent Petri executions, the zeros represent trace classes rather than their many linearizations. For FRACSTRAN, every bounded deterministic run has one arithmetic witness, including its first-applicable-fraction proof. For fixed SKI schedules, a certificate follows the changed context rather than the full tree. These guarantees are compatible with substantial complexity and with large witness heights.

The unbounded question remains sharply separated: fixed arity with finite or unique fibers is not supplied by finite-horizon determinism. The practical research contribution is therefore both constructive and diagnostic—a set of proved, executable witness-faithful compilers, and a precise statement of what additional compression would have to accomplish.

72.4 Manuscript 06: Conclusion

There is a substantial middle ground between a black-box MRDP invocation and an explicitly expanded computation trace. For token systems, exact resource summaries collapse nested repetition into syntax-sized arithmetic. For lexicographic trace normality, a compact Boolean signature collapses every power to the three cases zero, one, and at least two. These two compressions are compatible with unique quadratic witnesses and hence with a single quartic polynomial.

The resulting guarantees are precise. A fixed schedule with fixed parameters has one auxiliary certificate. All words of a fixed length admit a separate polynomial whose roots count executable trace classes exactly. Neither fact implies that arbitrary existential schedule parameters have finite fibers. Indeed, separated increment/decrement schedules already contain the full Diophantine finite-fold question, while their flat fragment remains Presburger-definable. This combination of constructive positive results and sharp boundary statements provides a concrete extension of ProveIt's Diophantine program without attributing to the compiler a solution of problems it does not solve.

Part X

Dynamic heaps, fresh names and local graph computation

Source and scope. [write] This Part is manuscript 08, “Names to Numbers: Exact Quartic Certificates for Dynamic Heaps and Local Graph Computation” (batch 62, manuscript 01; archive `Names_to_Numbers_Diophantine_Heaps.zip`; shipped files prefixed `08-dynamic-heaps-`; labels `cdc:nh:`). It prints the manuscript’s Sections 2–14 in their own order: the repository interface and prior work; witness-faithful gadgets and canonical gating (Theorem 75.6); a complete memory-log compiler (Theorem 76.2); sorting costs; birth-order normalization of fresh names (Theorem 78.1); the local-heap compilation theorem (Theorem 79.2); a pointer-only universal substrate (Theorem 80.1); bounded-local graph transfer (Theorem 81.1); a worked certificate and the executable evidence; the safety boundary (Theorem 83.1); the formalization plan; the twelve research questions; and the conclusion. Its two appendices follow. Its title, abstract, status statement and introduction are in Section 3.8; the conventions of this Part are Section 73.

Relation to the other Parts. [write] Manuscript 08 inspected ProveIt at 4e128356d, before this report existed, and does not cite it. Its memory layer re-derives the design of Part V (manuscript 03): address–time sorting by a deterministic compare-exchange network, the same bitonic network and comparator count (Proposition 77.1, compare Lemma 37.2), and the same comparison lower bound (Proposition 77.3, compare Proposition 39.3). The two memory theorems are different variants of one design and neither dominates the other: Theorem 76.2 uses $10C + 4N - 3$ witnesses and $10C + 7N - 4$ residuals, Theorem 38.3 uses $10C + 2L + 3(N - 1)$ and $10C + 3L + 4(N - 1) + 1$. Both are printed with their own proofs; the exact comparison follows Theorem 76.2. Part V also has a sort-free backend with $24L$ witnesses (Theorem 40.2), which answers the size part of this manuscript’s question on certificates without explicit sorting (see the note after it). New relative to Parts I–IX: the orbit bijection for fresh names, a proof that inactive branches can be normalized canonically without cubic residuals (Part V’s RAM compiler asks for such a normalization, Theorem 41.1), the nominal form of the heap theorem, structural two-stack universality, the graph-transfer theorem and the safety boundary. The Part partly answers two questions of the report: locality with polynomial bit-height (Research question 71.27), for bounded-local heaps, and faithful graph-rewriting front ends (Research question 71.43), for the node-allocation half; see the notes there. Its safety theorem is the Π_1^0 base of the recurrence and liveness hierarchy of the neighbouring report *liveness-beyond-halting* of this category.

Setting and hypotheses. [write] Every equation is over $\mathbb{N}$; a unique-extension certificate is single-fold over the source coordinates (Definition 75.1). Every construction is a family indexed by the log length, the number S of initial objects and the number T of macrosteps, so its number of variables grows with these parameters; no fixed-arity claim is made. Heap programs are fixed, bounded-local, with ordered birth slots and globally fresh names (conditions (N1)–(N4) of Section 78). The quotient is by fresh-name permutations only: schedules, selected occurrences and rule labels are not identified.

73 Conventions of Part X

[write] The letters are manuscript 08’s. The following differ from Part V, which certifies the same kind of memory log, or from other Parts; the tempting false reading is to carry Part V’s formula across.

Symbol	Meaning in Part X, and the clash
m, N	m is the log length and $N = 2^{\lceil \log_2 m \rceil}$ the padded length ($m \geq 1$). Part V writes L for the log length and $N = 2^{\lceil \log_2 \max(1, L) \rceil}$. The letter m is kept because manuscript 08 already uses L for the left stack register (Theorem 80.1) and L_f for a routed low record.
k_t, κ_i	The key is $k_t = Na_t + t$ on the raw address a_t , and κ_i is the key of the i th sorted record. Part V's key is $\kappa_t = (N + 1)(a_t + 1) + t$ on the shifted address $a_t + 1$. The two key formulas are not interchangeable, and neither are the padding conventions: here padding events are writes $(0, 1, 0)$ after all real events, in Part V dummy records $(0, t, 1, 0)$ with shifted address 0.
κ comparator	also the largest number of scalar memory accesses per macrostep (Section 79). strict: the system (75.1)–(75.2) has <i>no</i> solution at a tie (Lemma 75.3), and keys are distinct by construction. Part V's comparator (Lemma 36.1) is non-strict and sends a tie to its first branch.
zero test	three residuals (Lemma 75.4). Part V's has two (Corollary 36.3) and Part XIII's four (Lemma 116.1); each Part keeps its own gadget.
d C, A, V, H	record width. In Part V, d is the slack of the comparison gadget. C is the number of comparators, A and V bound addresses and values, and $H = \max\{V, N(A + 1), 1\}$ bounds the witness (Proposition 76.4), as in Part V, where the bound is $\max\{1, V, (N + 1)(A + 1) + N - 1\}$ (Theorem 39.2). A_{mem} and E_{mem} count witnesses and residuals; A_i is the address of the i th sorted record. H, H_0, H' also denote finite heap histories and the initial heap.
NF_{birth}	the birth-order normal form of a history (Theorem 78.1); manuscript 08 wrote NF. It is not Part II's lexicographic or Cartier–Foata normal form.
S, T, r, B_t, b_t	initial objects, macrosteps, births per macrostep, the allocator counter and the births of step t . B is also the number of branches of a selector, and $\{0, 1\} = \{0, 1\}$.
Safe_U	perpetual memory safety of the fixed program U (Theorem 83.1); a Π_1^0 property, not the noisy safety predicate of the neighbouring report <i>probabilistic-quantum-and-continuous-computation</i> .

The manuscript's `\Bool` is printed as this report's `\B`, and its sans-serif `NF` as this report's `NF` with the new subscript; its unused macros are dropped. No other symbol of manuscript 08 is renamed.

74 Repository interface and relevant prior work [08]

74.1 The inspected ProveIt snapshot

The repository was inspected at commit

4e128356d0ef75308be8ed405d89aea2ffdb8a57.

The most directly relevant source is

Computability/HilbertTenthProblem/Lean/
Diophantine/Common/DiophantineTrace.lean.

It contains the declarations `boundedForall_dioph`, `exactIter_dioph`, and `existsExactIter_dioph`. The first handles bounded universal quantification; the second handles exact finite iteration under Diophantine substitutions; the third existentially quantifies the iteration length. Their source discharges cipher-closure hypotheses through the `PAListCoding` infrastructure [3].

Those declarations establish Diophantineness, not the auxiliary-witness bijections or the explicit variable counts proved here. Our construction therefore complements them: the low-level compiler

records quantitative and multiplicity information before any global list-coding compression. This article inspects the source; it does not independently rebuild the repository or reproduce its axiom audit.

The combinatory-logic project provides a separate motivation. Its README carefully distinguishes forward simulations, injectivity, different lambda reduction conventions, and exact recursive-function equivalences [4]. We use the same discipline: a heap representation of a rewriting system does not automatically constitute a fully abstract translation of the repository’s particular lambda semantics.

74.2 Sorting memory histories is an established idea

The time-order/memory-order transformation is not new. Ben-Sasson, Chiesa, Genkin, and Tromer explicitly organize memory accesses by address and time, then check local consistency and route between the two views [38]. Their work belongs to a substantial tradition of efficient RAM-to-constraint reductions. The later TinyRAM implementation demonstrates this approach in a concrete verification system [27].

Here the target is different: exact equations over $\mathbb{N}$, unbounded numeric addresses, no probabilistic fingerprints, and a theorem that every arithmetic helper is uniquely determined. A deterministic compare-exchange network is used rather than an existential routing network whose internal routing can have more than one realization. The distinction matters for counting zeros.

[write] Within this report, Part V prints the same address–time sorting compiler from manuscript 03 (Theorem 38.3). Writing $L = m$ for the log length, the present counts use $2L - N$ fewer witnesses and $3N - 3L - 1$ more residuals, which is one *fewer* residual when $L = N$; see the comparison after Theorem 76.2.

Batcher’s bitonic sorting construction supplies a simple implemented network [28]. Linearithmic comparator networks are also established; for example, Goodrich gives a deterministic data-oblivious $O(N \log N)$ network [39]. We use their sorting guarantees, not a claim to have improved sorting theory.

74.3 The multiplicity boundary

The distinction between ordinary Diophantine representability and finite-fold or single-fold representability is substantial. Surveys and subsequent reduction work explicitly separate these questions [40, 11]. Our results concern a family whose number of variables grows with the time bound. Turning that family into one fixed-arity polynomial while retaining its multiplicity would require an additional theorem; no such theorem is assumed implicitly.

75 Witness-faithful polynomial systems [08]

75.1 Source and auxiliary coordinates

Definition 75.1. Let $R \subseteq \mathbb{N}^d$ be a relation. A polynomial $P(x, y) \in \mathbb{Z}[x, y]$ is a *unique-extension certificate* for R if, for every $x \in \mathbb{N}^d$,

$$\#\{y \in \mathbb{N}^a : P(x, y) = 0\} = \begin{cases} 1, & x \in R, \\ 0, & x \notin R. \end{cases}$$

A *run-faithful certificate* has its complete natural zero set in bijection with a specified set of computational runs, possibly after an explicitly stated quotient.

These notions should not be conflated. For a memory log, the source coordinates are the log entries. For a program, most log entries become existential coordinates constrained by the instruction semantics.

Unique extension of the memory layer then preserves, rather than collapses, the multiplicity of source computations.

Lemma 75.2 (Sum of squares). *Let $R_1, \dots, R_e \in \mathbb{Z}[z]$ have degree at most two. Then*

$$P(z) = \sum_{j=1}^e R_j(z)^2$$

has degree at most four, is nonnegative on $\mathbb{R}^{\#z}$, and vanishes exactly where every R_j vanishes. At an integer assignment that violates any residual, $P \geq 1$.

Proof. Each summand is a nonnegative real square. A sum of such squares is zero exactly when all summands are zero. A nonzero integer residual has square at least one. The degree bound follows by squaring. $\square$

Nonnegativity of P on all real assignments does not extend the semantic interpretation to all real assignments. In particular, several order gadgets below use the nonnegativity and integrality of their individual variables.

75.2 A compact strict comparator

Suppose $x, y \in \mathbb{N}$ and $x \neq y$. Introduce $b, h \in \mathbb{N}$ and impose

$$b(b-1) = 0, \tag{75.1}$$

$$(2b-1)(y-x) = h+1. \tag{75.2}$$

Lemma 75.3. *Equations (75.1)–(75.2) have exactly one natural solution:*

$$b = \mathbf{1}_{x < y}, \quad h = |y - x| - 1.$$

For $x = y$ they have no natural solution.

Proof. The first equation forces $b = 0$ or $b = 1$. The right side of the second is strictly positive. Thus $b = 1$ is possible exactly when $y - x > 0$, while $b = 0$ is possible exactly when $x - y > 0$. The positive difference uniquely determines h . At equality the second equation would say $0 = h + 1$. $\square$

For two records $X, Y \in \mathbb{N}^d$ with distinct keys X_0, Y_0 , the same b routes every field. For $0 \leq f < d$, impose

$$L_f = bX_f + (1-b)Y_f, \tag{75.3}$$

$$U_f = (1-b)X_f + bY_f. \tag{75.4}$$

There are $2 + 2d$ new variables and $2 + 2d$ quadratic residuals. The low and high records are exact permutations of the inputs, and every new variable is uniquely determined. A descending comparator reverses the two output wires; it does not change these equations.

75.3 A normalized zero test

For a quantity δ known to be nonnegative, use $z, h \in \mathbb{N}$:

$$z(z-1) = 0, \quad \delta = (1-z)(h+1), \quad zh = 0. \tag{75.5}$$

Lemma 75.4. *The unique solution is $(z, h) = (1, 0)$ if $\delta = 0$ and $(z, h) = (0, \delta - 1)$ if $\delta > 0$.*

Proof. If $z = 1$, the middle equation gives $\delta = 0$ and the last gives $h = 0$. If $z = 0$, the middle equation gives $\delta = h + 1 > 0$ and determines h . These are all Boolean cases. $\square$

The last equation is crucial. Omitting it would leave every $h \in \mathbb{N}$ available when $\delta = 0$, creating infinitely many witnesses for the same comparison.

75.4 Canonical gating without cubic residuals

A common mistake is to guard a quadratic equation $R = 0$ by a selector s and write $sR = 0$. The residual becomes cubic. A second mistake is to leave all private variables of a disabled instruction branch unconstrained. Both problems admit a uniform repair.

Lemma 75.5 (Canonical signed split). *For every integer r , the equations*

$$u - v = r, \quad uv = 0, \quad u, v \in \mathbb{N}$$

have exactly one solution: $u = \max(r, 0)$ and $v = \max(-r, 0)$.

Proof. If both were positive their product would be positive. Hence at least one is zero, and their difference fixes the other. The displayed values give a solution for all signs of r . $\square$

Theorem 75.6 (Unique guarded quadratization). *Let $s \in \{0, 1\}$ and let $R(\mathbf{x}, \mathbf{y})$ be an integer polynomial of degree at most two. The implication $s = 1 \Rightarrow R = 0$ can be represented using two new natural variables u, v and four quadratic residuals:*

$$u - v - R = 0, \quad uv = 0, \quad su = 0, \quad sv = 0. \quad (75.6)$$

For fixed $(s, \mathbf{x}, \mathbf{y})$, this representation has exactly one extension if the implication holds and none otherwise.

For a branch with private coordinates $y_1, \dots, y_a$, adjoining

$$(1 - s)y_j = 0 \quad (1 \leq j \leq a) \quad (75.7)$$

forces all private coordinates to their canonical zero values when the branch is disabled. If an enabled branch has unique private witnesses, the resulting guarded branch retains that uniqueness.

Proof. The first two equations of (75.6) uniquely split the integer value of R by Lemma 75.5. If $s = 0$, the remaining two impose nothing and the split still exists uniquely. If $s = 1$, they force $u = v = 0$, which is equivalent to $R = 0$. When $s = 0$, (75.7) forces every private coordinate to zero. Residuals in that disabled branch may evaluate to nonzero integers, but their signed-split coordinates are still uniquely determined and satisfy (75.6). Thus the construction never requires an inactive partial operation to be defined. When $s = 1$, (75.7) imposes nothing, and uniqueness follows from the active branch's contract. $\square$

For B finite branches, select exactly one with $s_j(s_j - 1) = 0$ and $\sum_j s_j = 1$. The labels are part of the source semantics; if two labels describe the same observable transition, they still represent two labelled choices unless the source explicitly identifies them.

76 A complete memory-log compiler [08]

76.1 Semantics

A source log has m triples

$$E_t = (a_t, w_t, v_t), \quad 0 \leq t < m,$$

where $a_t, v_t \in \mathbb{N}$ and $w_t \in \{0, 1\}$. The flag $w_t = 1$ denotes a write of v_t to address a_t ; $w_t = 0$ denotes a read claiming to return v_t . Memory is initially zero at every address. The log is valid when each read returns the latest preceding write to its address, or zero when none exists. A read does not change memory.

A finite nonzero initial memory map is handled by prepending one fixed write for every explicitly initialized address. A public, fixed order for these writes is used. This avoids treating an arbitrary ordering of initialization as another computational witness.

76.2 Padding and timestamp keys

Assume $m \geq 1$, and let $N = 2^{\lceil \log_2 m \rceil}$. Append $N - m$ events

$$E_t = (0, 1, 0), \quad m \leq t < N.$$

These padding writes occur after every real event. They cannot change whether an earlier read was correct, including an earlier read from address zero. Thus the padded log is valid exactly when the original log is valid.

For each event introduce a key coordinate k_t and the residual

$$k_t - Na_t - t = 0. \tag{76.1}$$

Also emit $w_t(w_t - 1) = 0$, retaining the trivial slot even for a constant padding flag. The records to be sorted are

$$X_t = (k_t, a_t, w_t, v_t) \in \mathbb{N}^4. \tag{76.2}$$

Lemma 76.1. *The N keys are pairwise distinct, and increasing key order is exactly lexicographic order by address and then timestamp.*

Proof. If $Na + t = Nb + s$ with $0 \leq s, t < N$, then $N(a - b) = s - t$. The only multiple of N strictly between $-N$ and N is zero, so $a = b$ and $s = t$. Different events have different timestamps. If $a < b$, then $Na + t \leq Na + N - 1 < N(a + 1) \leq Nb + s$; at equal addresses the comparison reduces to $t < s$. $\square$

76.3 Deterministic sorting

Fix a sorting network on N wires with C comparators. Replace each comparator by (75.1)–(75.4) with $d = 4$. Allocate two comparison variables and eight output-field variables per comparator. Do not allocate variables for unchanged wires: they retain references to their existing coordinates.

By induction over the comparators, every record remains one of the original records and every internal variable has a unique value. In particular, the strict-comparison precondition remains valid. Write the final records as

$$(\kappa_i, A_i, W_i, V_i), \quad 0 \leq i < N.$$

They have strictly increasing keys and nondecreasing addresses.

76.4 The adjacent-record memory test

For the first sorted record emit

$$(1 - W_0)V_0 = 0. \quad (76.3)$$

For each $1 \leq i < N$, introduce $z_i, h_i, c_i \in \mathbb{N}$ and emit

$$z_i(z_i - 1) = 0, \quad (76.4)$$

$$A_i - A_{i-1} - (1 - z_i)(h_i + 1) = 0, \quad (76.5)$$

$$z_i h_i = 0, \quad (76.6)$$

$$c_i - z_i V_{i-1} = 0, \quad (76.7)$$

$$(1 - W_i)(V_i - c_i) = 0. \quad (76.8)$$

The address gap is nonnegative, so z_i uniquely indicates that the two addresses agree. The carry c_i is the preceding access value for the same address, and is zero at the beginning of a new address group. The final equation checks reads and imposes no restriction on a write's new value.

Why is the *preceding access value* enough, rather than the *preceding write value*? Because a correct read repeats the stored value. Within one address group, induction shows that every access value equals the stored value immediately after that access, whether it is a read or a write.

Theorem 76.2 (Exact memory certificate). *Let $m \geq 1$, $N = 2^{\lceil \log_2 m \rceil}$, and let a fixed sorting network on N wires have C comparators. The residuals above have the following properties.*

- (a) *Each residual has total degree at most two in source and auxiliary coordinates, with integer coefficients.*
- (b) *There are exactly $10C + 4N - 3$ auxiliary coordinates and $10C + 7N - 4$ emitted residuals.*
- (c) *For a fixed natural source log, there is exactly one natural auxiliary solution if the log is valid, and none if it is invalid.*
- (d) *The sum of squared residuals is a unique-extension certificate of degree at most four, nonnegative on all real assignments.*

Proof. The explicit expressions show (a). Source flags are Boolean by their residuals. The N key equations uniquely determine the keys. By Lemma 76.1, all comparisons are strict; Lemma 75.3 and the routing equations then uniquely determine every comparator variable. The assumed sorting property puts all accesses to a given address together, in time order.

At a new group, (76.3) or (76.8) requires a first read to return zero. A first write is unrestricted and establishes the group's stored value. For subsequent records, suppose V_{i-1} is the stored value after the preceding access. If $W_i = 0$, the final equation requires $V_i = V_{i-1}$ and the read is correct. If $W_i = 1$, the value V_i becomes the new stored value. This proves equivalence to memory semantics by induction within each group. There is no interaction between different addresses. Because padding cannot affect earlier reads, the equivalence is also valid for the original unpadded log.

Completeness is constructive: calculate the keys, run the sorting network, set z_i and h_i by Lemma 75.4, and set $c_i = z_i V_{i-1}$. For a valid log every residual vanishes. Conversely, any satisfying assignment has those same keys, comparison decisions, routed fields, zero tests, and carries, so it encodes a valid log and is unique. This proves (c).

For the count, keys contribute N auxiliary coordinates, comparators contribute $10C$, and the scan contributes $3(N - 1)$. Residuals consist of N key equations, N flag equations, $10C$ comparator equations, one first-read equation, and $5(N - 1)$ scan equations. Hence

$$A_{\text{mem}} = N + 10C + 3(N - 1) = 10C + 4N - 3,$$

$$E_{\text{mem}} = 2N + 10C + 1 + 5(N - 1) = 10C + 7N - 4.$$

The sum-of-squares lemma proves (d). □

Remark 76.3 (The empty log). For $m = 0$ use $P = 0$ with no auxiliary coordinates. The empty tuple is the unique witness. The displayed count formulas are intended only for $m \geq 1$.

[write] Comparison with Part V. Part V’s memory theorem (Theorem 38.3, manuscript 03) certifies the same logs with the same sorting design. Write $L = m$ for the log length; for $L \geq 1$ both use $N = 2^{\lceil \log_2 L \rceil}$ and any sorting network with C comparators. The theorem above needs $10C + 4N - 3$ witnesses and $10C + 7N - 4$ residuals; Part V’s needs $10C + 2L + 3(N - 1)$ and $10C + 3L + 4(N - 1) + 1$. So this compiler has $2L - N$ fewer witnesses, between 1 and L : it stores keys and no shifted-address coordinate, although it also keys the padding events. It has $3N - 3L - 1$ more residuals, which is one fewer when $L = N$ and at least two more otherwise: its zero test has three residuals against two, and it emits constant slots for padding keys and flags. Neither theorem dominates the other, and both have the leading term $10C$. For the worked example of Section 82 ($L = 5$, $N = 8$, $C = 24$) the counts are 269 witnesses and 292 residuals here, 271 and 284 in Part V. The comparators differ as well: this one is strict and has no witness at a tie, Part V’s is not (Section 73). Part V also bounds the witness height (Theorem 39.2) and has a sort-free backend with $24L$ witnesses at a larger bit height (Theorem 40.2).

76.5 Bounds on witness magnitude

Proposition 76.4. *Suppose all real source addresses are at most A and source values at most V . In the unique satisfying memory witness, every source or auxiliary coordinate is at most*

$$H = \max\{V, N(A + 1), 1\}.$$

The emitted quadratic-residual coefficients have bit length $O(\log(N + 2))$.

Proof. Keys are at most $NA + N - 1$. Comparator outputs are routed source records, so inherit the bounds on their fields. A comparator slack is an absolute key difference minus one. Address-gap slacks are at most A and carries are zero or a previous value. All control bits are at most one. The only growing coefficients in the displayed memory residuals are N and the timestamps $t < N$; the remaining coefficients belong to a fixed bounded set. □

This is an arithmetic-coordinate bound, not a claim of constant bit cost for an unbounded integer. An address with a million bits still has a million-bit representation. The advantage is that one does not allocate a Boolean wire for each address bit merely to enforce order over $\mathbb{N}$.

77 Sorting costs and a restricted optimality statement [08]

77.1 The implemented bitonic network

For $N = 2^k$, the following fixed network is used by the code:

```
for span in 2, 4, ..., N:
    for stride in span/2, span/4, ..., 1:
        for i in 0, ..., N-1:
            j = i xor stride
            if j > i:
                compare-exchange wires i,j
                ascending iff (i bitwise-and span) == 0
```

All bit operations here generate a finite network at compile time. They are not operations required inside the Diophantine representation.

Proposition 77.1. *The network sorts and contains exactly*

$$C_{\text{bit}}(N) = \frac{N}{4}k(k+1)$$

comparators. Consequently the explicit memory layer has

$$A_{\text{bit}} = \frac{5}{2}Nk(k+1) + 4N - 3, \quad (77.1)$$

$$E_{\text{bit}} = \frac{5}{2}Nk(k+1) + 7N - 4. \quad (77.2)$$

Proof. A bitonic merge compares opposite positions in a bitonic sequence of length 2^j , separates its lower and upper halves, and recursively merges both halves. One way to see the separation is to threshold all entries at an arbitrary value. A thresholded bitonic sequence has a single circular block of ones. If this circular block has at most half the positions, no opposite pair contains two ones: all minima are zero, and the maxima form a circular block in the half-length cycle. If it has at least half the positions, no opposite pair contains two zeros: all maxima are one, and the minima form a circular block in the half-length cycle. Thus every entry in the lower half is at most every entry in the upper half, and both halves remain bitonic. Induction sorts each half. Since compare-exchange commutes with thresholding, a nonsorted numerical output would give a nonsorted binary output at some threshold, which is impossible. Combining an ascending sorted half and a descending sorted half gives the bitonic input for the next merge. This is the standard bitonic construction [28].

There are j merge layers at stage j , each with $N/2$ comparators. Summing $(N/2)j$ for $j = 1, \dots, k$ gives the stated count. Substitute that count into Theorem 76.2. $\square$

77.2 Linearithmic existence and its exact dependency

Corollary 77.2. *There is an effective family of unique-extension quartic memory certificates using $O(m \log(m+2))$ source-plus-auxiliary coordinates and residuals.*

Proof. Use an established deterministic $O(N \log N)$ sorting network [39] in Theorem 76.2. Since $N < 2m$ for $m > 1$, the bound follows. $\square$

The archive implements the bitonic family, not Goodrich's or AKS-style networks. The linearithmic statement is a mathematical corollary with an explicit external dependency. It is not a benchmark result for the included program.

[write] Part V states the same existence result with the network of Ajtai, Komlós and Szemerédi (Corollary 39.1, [29]); either established deterministic $O(N \log N)$ network gives the bound.

Proposition 77.3 (Comparator-architecture lower bound). *Any binary-comparison architecture that explicitly sorts arbitrary distinct memory keys requires at least $\lceil \log_2(N!) \rceil$ comparisons in the worst case. Thus $O(N \log N)$ is asymptotically optimal for the sorting stage within this architecture.*

Proof. Choose the N addresses to be the numbers $1, \dots, N$ in any input permutation. The resulting keys have all $N!$ possible address orderings. A binary comparison decision path has at most two successors per comparison. Different input orderings require different sorting permutations, hence at least $N!$ leaves. Depth is at least $\log_2(N!) = \Omega(N \log N)$. A sorting network with C comparators is a special case with at most 2^C comparison outcome patterns. $\square$

This is not a lower bound on all Diophantine representations, all arithmetic circuits, or all memory certificates. A representation that does not explicitly sort records is outside the proposition's hypotheses.

78 Fresh names and canonical finite histories [08]

78.1 A nominal heap model

Let $\mathcal{A}$ be a countably infinite set of object names, disjoint from scalar data. A null pointer $\perp$ is not an allocated object. An initial heap has a specified ordered set $I = \{\iota_1, \dots, \iota_S\}$ of objects. Each object has a fixed finite list of typed fields: some contain pointers, others contain tags or scalar data. A configuration also has finitely many registers and a finite control state. Finite histories record their rule choices, object selections, field accesses, and allocation events.

The following conventions define the precise class being studied.

- (N1) Names can be copied, tested for equality, and followed as pointers. Source operations cannot order names, perform arithmetic on them, or turn an arbitrary scalar into a pointer. Rules are equivariant under permutations of $\mathcal{A}$ fixing I .
- (N2) Allocation returns a name never used earlier in the history, including names of deleted objects. A finite run has only finitely many allocations. Deletion may leave a tombstone; it does not authorize name reuse.
- (N3) Births have a total order: time orders macrosteps, and a rule specifies ordered birth slots within a macrostep. Rule choices and selected occurrences remain part of the history.
- (N4) Initially present objects and all pointer-valued inputs belong to the specified initial structure. Subsequently appearing non-null pointers name previously allocated objects or objects born in the current ordered slots.

The last condition prevents an unexplained external name from acquiring a numeric identity for free. External allocation can instead be represented by an explicit birth event. Rules with nondeterministic selection of an existing object are allowed; that selection is retained in the run.

A permutation acts only on names, not on scalar values, field labels, times, rule labels, or birth-slot indices. Two histories are *name-equivalent* when a finite-support permutation fixing I sends one to the other.

78.2 The normalization theorem

Number the initial objects $1, \dots, S$. Number the j th subsequent birth $S + j$, and encode null by 0. Replace every occurrence of an object name in the history by its number, including occurrences in garbage, registers, pointer fields, selected roots, and tombstones. Scalar values are unchanged. Write the resulting numerical history as $\text{NF}_{\text{birth}}(H)$.

Theorem 78.1 (Birth-order normalization). *For finite histories satisfying (N1)–(N4):*

- (i) $\text{NF}_{\text{birth}}(H)$ is a legal history in the canonically numbered interpretation;
- (ii) $\text{NF}_{\text{birth}}(H) = \text{NF}_{\text{birth}}(H')$ if and only if H and H' are name-equivalent;
- (iii) every name-equivalence class has exactly one canonical numerical history.

The correspondence preserves length, rule choices, field values, and the ordered pattern of births, accesses, and deletions.

Proof. The finitely many names occurring in H are exactly initial names together with some of its born names; counting all births also includes names that later become unreachable. Distinct birth events have distinct names by (N2). The map from these names to their assigned numbers is therefore injective

and preserves null and all equality tests. Apply this map to each configuration and each transition. A fresh name remains distinct from every earlier name, and field lookup and update commute with the map. By equivariance, each transformed transition is legal. This proves (i).

A permutation fixing I preserves the time and slot of every birth. It must therefore preserve its birth number, so equivalent histories normalize to the same history. Conversely, suppose the normalizations coincide. Match the name born at slot j in H to the name born at slot j in H' , fixing the initial names. This is a bijection between two finite sets of names and sends all name occurrences of H to the corresponding occurrences of H' .

Such a finite partial bijection extends to a finite-support permutation of $\mathcal{A}$. For completeness, regard its specified arrows as a finite directed graph on the union of domain and range. Every vertex has at most one specified incoming and one specified outgoing edge. Its components are cycles and directed paths. Keep cycles and close each nontrivial path by an arrow from its last vertex to its first; fix all remaining names. Initial fixed points remain fixed. The resulting permutation has the required action. This proves (ii). Existence in (iii) follows from (i), and uniqueness follows from the deterministic numbering rule and (ii). $\square$

Example 78.2 (Names are not schedules). A rule chooses to rewrite an existing object p or an existing object q . Even if the resulting heaps happen to be isomorphic, these are different histories when p and q are distinct fixed initial objects. Likewise, performing two independent rewrites in opposite orders normally produces two canonical histories: birth-order normalization does not commute their time indices. An unordered right-hand side with two indistinguishable new objects also needs an additional quotient argument; choosing two ordered birth slots is a semantic refinement, not a proof that the unordered quotient is solved.

The theorem avoids a graph-isomorphism oracle because it preserves the entire ordered history. It does not provide canonical labeling of an arbitrary final unrooted graph. Numerical pointer comparisons used internally by the memory compiler are permissible: the source cannot observe them, and the compiler has already selected a canonical representative.

79 The local-heap compilation theorem [08]

79.1 A precise local instruction interface

Fix a program Π , a record width $d \geq 1$, a constant number of registers, a maximum r of births per macrostep, and a maximum κ of scalar memory accesses per macrostep. A macrostep is one of finitely many rule templates. Within a template, a fixed finite sequence of primitive operations can: read or write a field; copy a register; inspect a finite tag; test pointer equality or nullity; select an existing object; allocate objects in ordered slots; and update finite control. Optional scalar arithmetic must have a fixed-size, unique-extension quadratic certificate. Addition and multiplication over $\mathbb{N}$ satisfy this requirement directly.

A template has no unbounded inner loop and traverses no unbounded list in a single macrostep. Such traversals must be decomposed into multiple macrosteps. Different successful rule choices or selected objects count as different runs unless the source explicitly supplies a deterministic selection convention. An accepting or fault state is an explicit finite control state, not a silently assumed global graph property.

For programs with deletion, give each object a live field. Allocation writes all fields, including live = 1, in a specified order; deletion writes live = 0 and changes any other fields specified by the rule. A dereference checks that its object exists and is live. This convention admits unreachable objects but forbids a dangling pointer from being dereferenced without a fault transition. A program without deletion can omit the live field.

Number objects canonically as in Theorem 78.1. Field j of object $p > 0$ has scalar memory address

$$\text{addr}(p, j) = dp + j, \quad 0 \leq j < d. \quad (79.1)$$

This encoding is injective in (p, j) , and address 0 is reserved for dummy operations. It belongs to the compiler, not the pointer-only source language. At each step the allocator counter B_t is the number of objects ever born, including the S initial objects. If the selected rule creates b_t objects,

$$B_0 = S, \quad B_{t+1} = B_t + b_t, \quad p_{t,j} = B_t + j \quad (1 \leq j \leq b_t). \quad (79.2)$$

All b_t and allocation-slot choices are forced by the selected rule. There is no existential search for a fresh numerical name.

Each macrostep occupies exactly κ memory slots. An unused slot is the constant operation $(0, 1, 0)$; its values are not arbitrary witnesses. Fields of the initial heap are written in a fixed order before the macrostep log. Consequently the complete log has

$$m = dS + \kappa T \quad (79.3)$$

events, after enlarging d or κ once to include the chosen fixed metadata operations. Initial zero fields may still be explicitly written; this keeps the shape uniform. A final acceptance test may inspect a bounded number of registers or fields, which can be included in the final macrostep.

To represent acceptance within a bound, an accepting state has exactly one padding rule: keep control, registers, and B_t unchanged, and emit only dummy operations. Padding is compulsory after the first accepting state. An exact-length version simply omits this absorbing convention.

79.2 Local arithmetic without hidden multiplicity

The following construction explains the local certificate assumption in the pointer-only case, rather than leaving it as an unspecified computability oracle. Copies, fixed tags, addresses (79.1), and the allocator (79.2) use linear equations. A selected pointer p is positive and at most a current allocation counter B exactly when there are unique $u, v \in \mathbb{N}$ with

$$p = 1 + u, \quad B = p + v. \quad (79.4)$$

At a macrostep boundary use $B = B_t$; after reserving new birth slots, use the corresponding updated local counter. All newborn records are initialized before their fields are followed. A live-field read followed by equality to 1 tests liveness. If null is an allowed value, use the disjoint null and positive cases.

Equality tests are also canonical. Write an integer difference $x - y$ uniquely as $u - v$ with $uv = 0$, by Lemma 75.5. Apply the zero-test equations (75.5) to $u + v$. Its Boolean output is 1 exactly when $x = y$. The same construction distinguishes equal and unequal pointer values without requiring a noncanonical witness to nonzeroness. Ordered scalar comparisons, when included in a richer source, have the analogous signed-difference construction.

Give the finitely many rule templates Boolean selectors $s_1, \dots, s_q$ with $\sum_i s_i = 1$. A chosen rule's ordinary equations and tests must hold; use Theorem 75.6 to guard every degree-at-most-two equation. Force every original template-private variable of an inactive template to zero by (75.7). The new signed-split pairs for guarded residuals are not among these zero-clamped original variables. Shared registers and shared event coordinates are not private variables and are constrained by the selected template. Signed splitting of a residual still has a unique solution when a rule is inactive. Thus there is no assumption that an inactive partial operation was successfully evaluated.

Within the active template, primitive operations occur in a fixed acyclic order. The source state, retained rule choice, retained object selections, and returned read values determine its scratch variables

and output registers. Each guard helper is then uniquely determined. Halt padding has a single selector and constant events. Because Π is fixed, this produces only a constant number of quadratic residuals and helpers per macrostep.

Lemma 79.1 (Local certificate construction). *For the finite pointer-only instruction interface above, there is an effective fixed-size family of quadratic residual systems that checks one macrostep. For every fixed legal step, including its selected rule, selected objects and memory return values, it has exactly one assignment to private arithmetic helpers. An illegal step has none. The construction extends to additional scalar primitives with the stated unique-extension interface.*

Proof. Construct the equations operation by operation as described above. The range slacks in (79.4) are forced. Copy, arithmetic gate, and address equations force their outputs. Signed parts and zero tests have unique helpers by Lemmas 75.4 and 75.5; Boolean selectors and their guards encode exactly the specified finite cases. The selected rule is part of the step being checked, not an arbitrary extra witness. Inactive original template helpers are forced to zero, and every residual's signed split is unique even in this inactive case. Induction through the finite primitive sequence proves existence and uniqueness for a legal step. Conversely, each zero residual forces the corresponding primitive condition, and the selected rule's guarded residuals force its entire sequence, proving soundness. The fixed template size gives the size assertion. $\square$

The lemma does not claim a single canonical proof of an arbitrary semantic predicate supplied by a user. Such a predicate must be reduced to this interface, or accompanied by its own witness-faithful certificate proof.

79.3 A bijection, not merely a projection

A length- T run can be recorded by its finite control and registers at each boundary, its rule choices and selected roots, its scalar event log, and its allocation counts. Full heap snapshots are not stored. Given this record, chronological execution reconstructs all heap snapshots, including unchanged fields and garbage. The size of the record is $O(S + T)$ for fixed Π .

Theorem 79.2 (Local heaps to one quartic polynomial). *For fixed Π as above, a fixed well-formed initial heap H_0 of S objects, and $T \in \mathbb{N}$, one can effectively construct an integer polynomial*

$$P_{\Pi, H_0, T}(\mathbf{z})$$

of degree at most four whose natural zero set is in bijection with the accepting T -macrostep runs of Π from H_0 , modulo finite fresh-name permutations fixing H_0 . Use the stated padding convention for acceptance within T instead of at exactly T .

With bitonic sorting, the number of coordinates and residuals is $O((S + T) \log^2(S + T + 2))$. With an established deterministic linearithmic sorting network it is $O((S + T) \log(S + T + 2))$. The constants depend on the fixed local program, not on S, T or the names chosen during allocation.

Proof. First normalize each nominal run by Theorem 78.1. Introduce the $O(S + T)$ local record coordinates and helper variables from Lemma 79.1. Fix its initial registers and control to the encoding of H_0 , enforce (79.2), and impose the selected accepting-control condition at the end. Keep all field accesses as common event variables. Add the memory residual system of Theorem 76.2 for the log (79.3). These two sets of residuals share the event coordinates but otherwise have disjoint auxiliary variables. Sum the squares of *all quadratic residuals once*. Do not square an already quartic sum-of-squares polynomial.

For completeness, take a legal nominal run. Normalize its names and fill in its chronological local record. Initial writes and all subsequent read/write events are memory-valid. The local lemmas provide

their unique helpers, and Theorem 76.2 provides the memory helpers. Every residual vanishes, hence so does the resulting polynomial.

For soundness, suppose a natural assignment makes the polynomial zero. Each residual vanishes by Lemma 75.2. The log is memory-valid by Theorem 76.2. Reconstruct the heap chronologically after its fixed initialization writes. At a macrostep, the selected local rule is valid by Lemma 79.1; every returned value is the value of the most recent write to its field, or its initialized value, by the memory theorem. Updates affect only explicitly written addresses. Thus all untouched fields retain their values without separate frame equations. The allocator assigns distinct ordered fresh objects, and pointer-range and live checks exclude illegal dereferences. Induction over macrosteps reconstructs a legal canonical run. Its final control is accepting by construction.

For the bijection, a canonical run uniquely determines its local record, constant padding, allocator sequence and local helpers. It uniquely determines the memory helpers as well. Conversely, the reconstruction above uniquely determines the run from any zero. Consequently these maps are inverses. Finally compose with the orbit bijection of Theorem 78.1. Distinct rule choices or selected roots have distinct record coordinates, as required.

There are $O(S + T)$ local coordinates and equations, and $m = O(S + T)$ memory events. Apply Proposition 77.1 and Corollary 77.2. The constant-size empty-log case is handled without a sorting network. Every residual has degree at most two, including guarded ones, so the final degree is at most four. $\square$

Corollary 79.3 (Counting retained runs). *Whenever the set of length- T runs is finite, its number of name-equivalence classes equals the number of natural zeros of $P_{\Pi, H_0, T}$. In particular, a deterministic program under the fixed padding convention has either zero or one such zero. No factor for fresh-name choices or arithmetic helper choices occurs.*

Proof. This is the bijection in Theorem 79.2, with cardinalities taken on its two sides. In a deterministic run, all operational choices other than fresh names are forced, and those names have been quotiented. $\square$

For finite-tag pointer programs the set is finite because at each step there are finitely many rules and at most $S + rT$ selectable objects. A source primitive that nondeterministically chooses an arbitrary natural scalar may instead have infinitely many length- T runs; the bijection still holds, but the finite counting assertion should not be applied.

79.4 Height and numerical cost

Proposition 79.4 (Polynomial heights for finite-tag pointers). *For a fixed finite-tag pointer program, every coordinate of a zero constructed in Theorem 79.2 is bounded by $(S + T + 2)^{c_\Pi}$ times a program-dependent constant. Consequently each coordinate uses $O_\Pi(\log(S + T + 2))$ bits. This claim excludes unbounded scalar arithmetic or unbounded scalar input values.*

Proof. Every pointer is at most $S + rT$, every tag belongs to a fixed finite set, and allocator counters and range slacks are $O_\Pi(S + T + 1)$. Scalar field addresses (79.1) have the same bound. There are $N = O_\Pi(S + T + 1)$ padded events, so keys and comparison slacks have quadratic bounds in $S + T + 1$ by Proposition 76.4; routed fields do not increase them. Scan variables have at most the corresponding address or value bounds.

The local compiler adds a fixed acyclic sequence of additions, multiplications, differences, signed parts and slack computations per macrostep. Its depth is bounded by a constant depending on Π . Starting from polynomially bounded local inputs, its helpers therefore have a polynomial bound of some fixed degree c_Π . Inactive residual splitting also has such a bound. These scratch computations do not

feed an unbounded numeric accumulator in a finite-tag pointer program. Increasing c_Π and the constant covers small S, T . $\square$

This is stronger than merely knowing that witnesses exist, but it does not make unbounded halting decidable: both the dimension and the bound depend on T . Exhaustive search has a combinatorial cost even when individual integers are small. With arbitrary scalar values the appropriate statement is the parameterized bound of Proposition 76.4, together with the actual cost and height growth of the chosen scalar primitives.

80 A universal pointer-only substrate [08]

80.1 Two immutable linked stacks

Use two pointer registers L, R , a finite control state, and a finite symbol register. A stack cell has exactly two fields: a symbol from a fixed finite alphabet and a next pointer. Cells are immutable after initialization. A stack root is either null or a pointer to a previously allocated cell.

The instruction $\text{push}(X, a)$ allocates a fresh cell p , writes its symbol field to a , writes its next field to the old root X , and sets $X := p$. The instruction $\text{pop}(X)$ has two disjoint cases. If $X = \perp$, it returns an empty indication and leaves X null. Otherwise it reads the symbol and next fields of the root, returns the symbol, and sets X to the next pointer. A removed cell is retained as unreachable garbage; no name is reused. Empty pops emit the fixed dummy slots when placed in a fixed-shape certificate.

These operations use only finite control, pointer equality to null, field reads and writes, and fresh allocation. Push has two real field writes; nonempty pop has two real reads. No source operation interprets a name as an integer. The numerical allocator and field-address arithmetic appear only in the compiler.

Theorem 80.1 (Direct Turing-machine simulation). *Finite-control programs using the two immutable linked stacks are Turing complete. A transition of a fixed ordinary one-tape Turing machine can be simulated with at most one allocated cell and four real scalar field accesses, with a fixed finite amount of additional register/control work.*

Proof. Let the machine's tape alphabet be Γ with blank symbol $\square$. Represent a tape configuration by (q, c, L, R) , where q is the control state, c is the currently scanned symbol, L contains the symbols strictly to the left of the head in nearest-first order, and R contains those strictly to the right in nearest-first order. Missing stack elements denote blanks. It is harmless for a finite stack to end in explicit blank symbols.

Suppose a transition reads c , writes b , changes state to q' , and moves right. Push b onto L . Pop R ; put the returned symbol into c , or put $\square$ into c if the stack was empty. Set $q := q'$. A left move is symmetric: push b onto R and pop L . A stationary transition simply sets $c := b$ and $q := q'$. Each moving transition uses one push and at most one nonempty pop, hence at most one allocation and four field accesses.

The representation invariant holds initially by placing the finite input to the right of the initially scanned cell into R in nearest-first order and setting L null. If necessary, construct R by pushes in reverse order; this needs linear initial space. The cases above preserve the tape contents and head position represented by the invariant, including motion into previously unrepresented blank cells. Induction proves exact simulation of every finite machine computation. Map halting states to the corresponding finite-control terminal states. A universal Turing machine is a particular finite machine, so this fixed instruction substrate is universal. $\square$

Corollary 80.2 (A quartic representation for a pointer-only universal model). *For each fixed Turing machine, its input heap and time bound have the witness-bijective quartic representation of Theorem 79.2. Under deterministic execution and canonical halt padding, its natural zero set is a singleton precisely when the machine accepts within the bound, and is empty otherwise.*

Proof. The simulation satisfies the local interface with fixed record width and fixed access/birth bounds. Apply Theorem 79.2 and Corollary 79.3. $\square$

The constant “four” counts only symbol and next-field accesses in this particular immutable-stack simulation. A general heap language that also logs liveness, reference counts, or other metadata has different constants. No claim is made that four is optimal.

80.2 Why this is not a disguised arithmetic register machine

The source cannot increment a pointer, compare two pointers by magnitude, or factor an address. Its unbounded storage consists of allocated linked cells; its visible data alphabet is finite. The two-stack simulation demonstrates universality entirely at that structural level. Arithmetic is introduced only to certify an already specified source computation. In particular, fresh-name normalization does not assume that the source can choose the next integer address.

The retained immutable garbage is deliberate. Removing it by a tracing collector in one macrostep would violate bounded locality. A collector can instead be implemented as an explicit local program, with its own steps and invariants; name reuse after collection would additionally require the history model to change.

81 Local graph computation and the exact transfer boundary [08]

81.1 An explicit class of graph substrates

A finite rooted directed graph with bounded out-degree can be stored as fixed-width object records: a finite vertex tag, a live bit if needed, and ordered outgoing pointer fields. A local rule selects a bounded tuple of existing roots, follows a bounded number of stored pointers, checks a finite pattern of tags and equalities, creates a bounded number of new vertices, and changes a bounded number of fields. Incoming edges that must be updated must either be explicitly stored and bounded or be discovered through separate local traversal steps. A global search for all predecessors is not one local operation.

To give a precise simultaneous rewrite semantics, specify the order of root roles and birth slots, read the bounded pattern before performing writes, and either prohibit conflicting writes or specify their resolution order. If root roles can alias, the rule must state which equalities and inequalities are required. For a fixed bounded pattern there are only finitely many alias patterns. A compiler can treat them as disjoint equality-test cases using Lemma 79.1. It must not treat aliased field locations as independent.

Theorem 81.1 (Bounded-local graph transfer). *For a fixed finite collection of graph rules with the interface just described, and with globally fresh ordered births, finite accepting occurrence histories have the quartic orbit-bijective representation of Theorem 79.2. The quotient fixes the initial graph identities and removes only subsequently created vertex names. The class of such graph programs is Turing complete.*

Proof. Translate each rule into a fixed finite heap template. The bounded pattern traversal becomes field reads; its finite tag and alias conditions become canonical equality tests; its births and writes become allocator equations and ordered field updates. Case splitting on the finitely many possible alias

patterns is disjoint, so it introduces no alternative arithmetic witness for a fixed occurrence. All bounds depend only on the fixed rule. This gives an occurrence-preserving correspondence between graph steps and heap macrosteps, with the specified write convention. Apply Theorem 79.2. Two linked stacks are graphs of this form, so Theorem 80.1 proves the last assertion. $\square$

This result is independent of whether the graph is drawn in a plane, used to represent a circuit, or interpreted as a symbolic term with shared subgraphs. Its hypotheses concern the operational rule, not the drawing. Sharing matters: a pointer update to a shared object is not automatically the same computation as rewriting one copy in a fully expanded syntax tree.

81.2 Interaction systems: an adapter, not a blanket identification

Interaction combinators are a natural next target: Lafont’s paper studies a small universal interaction system [41]. A bounded-port implementation appears compatible with the interface above, but a complete instance requires a specific syntax and implementation to be fixed. In particular, one must prove that principal-port selection, port matching, interface reconnection, alias cases, and creation order reproduce the chosen source semantics. A port graph may have reciprocal-link invariants that a plain directed heap does not automatically possess.

The theorem here applies once that bounded-local, occurrence-preserving adapter is proved. This archive does not implement Lafont’s complete rule set or claim a checked reduction from the repository’s Iota or lambda semantics to it. The direct two-stack construction already provides a fully specified universal instance; interaction systems are a concrete research extension, not an unproved premise of our main result.

[write] Batch 78, 2 October 2026: Part XV (manuscripts 13–15) fixes such a syntax for Lafont’s interaction combinators and proves the bounded adapter directly, for them and for fixed interaction systems with bounded interfaces (Theorem 145.7): principal-port selection, port matching, interface reconnection with exact cyclic-wire counts, and creation order. It does not check a reduction from the repository’s Iota or lambda semantics.

81.3 A useful division of labor

The source adapter proves *which local computation is intended*. The memory compiler proves *that the recorded reads and writes fit one chronological heap*. Birth normalization proves *which name choices should be identified*. These obligations are independent enough to be reused separately. A correct memory trace cannot repair a wrong graph rule, and a correct graph rule cannot repair an arithmetic certificate that accepts a stale read.

82 A worked certificate and executable evidence [08]

82.1 Five real events, eight sorted records

Consider the following zero-initialized memory log. A write has $w = 1$ and a read has $w = 0$.

Time t	Address a_t	Flag w_t	Value v_t	Meaning
0	7	1	4	Write 4 to address 7
1	2	0	0	Read untouched address 2
2	7	0	4	Read the value written at time 0
3	7	1	9	Overwrite address 7
4	7	0	9	Read the new value

Pad with $(0, 1, 0)$ at times 5, 6, 7. For $N = 8$, the five real keys are 56, 17, 58, 59, 60, and the padding keys are 5, 6, 7. Sorting gives:

Sorted position i	κ_i	A_i	W_i	V_i	z_i	h_i	c_i
0	5	0	1	0	—	—	—
1	6	0	1	0	1	0	0
2	7	0	1	0	1	0	0
3	17	2	0	0	0	1	0
4	56	7	1	4	0	4	0
5	58	7	0	4	1	0	4
6	59	7	1	9	1	0	4
7	60	7	0	9	1	0	9

At position 6, the carry is 4 but the event is a write of 9, so no read-equality constraint applies. At position 7, the carry is therefore 9. This illustrates why consecutive accesses, not just consecutive writes, can be used in the scan.

The bitonic network has $C = 24$ comparators. Its certificate has 15 source coordinates, 269 auxiliary coordinates, and 292 quadratic residuals. The single quartic polynomial is specified exactly by their sum of squares. The archive’s `data/08-dynamic-heaps-memory_certificate.json` records every variable, coefficient, residual and value of the unique valid witness; the polynomial need not be expanded into a much longer monomial list.

Changing the read at time 2 from 4 to 5 gives an invalid log. Its canonical candidate assignment violates the position-5 read residual by one, and the emitted sum of squares equals 1. The theorem says more than this particular failed candidate: *no* alternative natural helper assignment can make that invalid source log satisfy the polynomial.

82.2 What the implementation does

`code/08-dynamic-heaps-memory_quartic.py` implements sparse integer polynomials, bitonic network generation, the complete memory residual construction, canonical candidate-witness generation, and exact residual evaluation. Its JSON export describes $P = \sum_j R_j^2$ through the quadratic residuals. Equal input lengths produce the same symbolic polynomial regardless of the input values. Source values affect the proposed assignment, not the polynomial’s coefficients or network shape.

`code/08-dynamic-heaps-pointer_machine.py` implements two stack heaps, their memory logs, and birth-order normalization of action records. Its diagnostic log omits empty-pop dummy slots; each such omission is a fixed two-slot expansion when compiling a fixed-shape source program. The example code is not the full local-program compiler of Theorem 79.2, and the archive does not claim an implemented Turing-machine frontend.

`code/08-dynamic-heaps-run_tests.py` compares the memory compiler with an independent chronological dictionary simulation. It also checks sorting permutations, small arithmetic gadgets, invalid reads, and fresh-name renamings. The seed and exact coverage are recorded in `data/08-dynamic-heaps-receipt.json`.

Check	Cases
Distinct comparator inputs from $\{0, \dots, 9\}^2$	90
Zero-test inputs $0, \dots, 19$	20
Guarded residuals $s \in \{0, 1\}, R \in \{-20, \dots, 20\}$	82
All sorting permutations for sizes 1, 2, 4, 8	40,347
All logs of lengths 0 through 4 over two addresses, two flags and two values	4,681
Valid logs among those exhaustive cases	1,555
Additional random valid logs, including 71-digit addresses	24
Corrupted reads rejected	80
Single-coordinate auxiliary increments rejected	342
Pairs of independently fresh-named pointer histories	12

Every listed check passed. The mutation count refers to increasing each auxiliary coordinate by one in the first two random test logs, not to all possible simultaneous mutations. Small gadget checks enumerate a finite helper range sufficient for those input values. Their unbounded correctness and uniqueness follow from the proofs, not from extrapolating test coverage.

The JSON verifier checks that the supplied natural assignment satisfies the supplied residuals and that their degree is at most two. By itself it does not authenticate an untrusted compiler’s claim that those residuals encode a particular log. That semantic link is the compiler-correctness theorem and its future formalization obligation.

83 Unbounded properties and two necessary obstructions [08]

83.1 Ordinary Diophantineness survives; multiplicity need not

For the programs above, a finite run is effectively checkable. Natural parameters can encode a program and finite input heap. Acceptance is recursively enumerable by enumerating finite candidate runs, or by dovetailing executions in the nondeterministic case. MRDP consequently gives an ordinary fixed-arity Diophantine representation [1]. Equivalently, once a coded one-step relation has been proved Diophantine, the repository’s exact iteration interface provides a compositional route to finite reachability [3].

These statements do not preserve all the additional information in Theorem 79.2. The polynomial there depends on T in its syntax and in the number of variables. The expression

$$\exists T \exists \mathbf{z} P_{\Pi, H_0, T}(\mathbf{z}) = 0$$

is a convenient description of a computably generated family, not literally one polynomial formula: the length of $\mathbf{z}$ is not fixed. Encoding that family into a fixed tuple is a separate arithmetization step.

Furthermore, a deterministic accepting run padded to different bounds produces different bounded witnesses. Recording the first accepting time removes this elementary duplication, but does not prove that every list code, exponentiation representation, or compression helper has unique or finitely many realizations. The finite-fold and single-fold literature concerns precisely a stronger fixed-arity requirement [40, 11]. This article neither assumes nor settles that requirement.

83.2 Finite faults versus perpetual safety

A memory fault can be made an explicit terminal control state. The operation “dereference null” enters that state instead of making an unconstrained read. For other invalid dereferences, a finite fault

certificate includes the local failing condition and the preceding valid prefix. Finite conditions such as nullity or a tombstone read are decidable local tests.

Theorem 83.1 (The existential safety boundary). *For effectively specified local heap programs, the parameterized property “some finite execution reaches a memory fault” is recursively enumerable and hence Diophantine. There is a fixed deterministic finite-tag pointer program U for which the input property*

$$\text{Safe}_U(x) = \text{“the execution from input } x \text{ never faults”}$$

is not recursively enumerable and has no existential polynomial representation over $\mathbb{N}$.

Proof. A finite fault is witnessed by a finite sequence ending in a designated fault transition. Enumerate such sequences and verify each one. This proves recursive enumerability, and MRDP gives the first assertion. A bounded version can instead be represented directly by Theorem 79.2, with its accepting state interpreted as the fault state.

For the negative assertion, choose a fixed universal Turing machine and simulate it by the two-stack program in Theorem 80.1. The simulation itself performs only safe stack operations: empty pops are explicitly handled, allocated cells are never reused, and every non-null root names an initialized cell. Modify the simulator so that, when the simulated machine halts, it performs one deliberate null dereference. For a natural code x of a machine/input pair, the resulting fixed program U never faults exactly when the encoded machine does not halt. Malformed codes can be assigned a fixed harmless behavior; well-formed codes suffice for the reduction.

The halting set is recursively enumerable but not decidable. To recall the last fact, a hypothetical halting decider would let a program on input e loop exactly when the decider says that program e halts on input e , and halt otherwise. Applying this program to its own code contradicts the decider. If the complement of the halting set were recursively enumerable, dovetailing its enumerator with the halting-set enumerator would decide halting. Thus that complement is not recursively enumerable. The exact reduction above implies the same for Safe_U .

Finally, any set of the form $\{x : \exists \mathbf{y} \in \mathbb{N}^a, Q(x, \mathbf{y}) = 0\}$ for a fixed integer polynomial Q is recursively enumerable: enumerate $\mathbf{y}$ and evaluate Q exactly. Therefore Safe_U cannot have such a representation. $\square$

[write] In the neighbouring report *liveness-beyond-halting* of this category, perpetual properties of this kind are studied systematically; the safety property above is the Π_1^0 base of its recurrence and liveness hierarchy.

[write] Batch 63, 30 September 2026: that report has since been written (commit 2a34b1740, from batch-62 manuscripts 10 and 03 and batch-63 manuscript 01); its one hierarchy table for its three sources (its section `lbh:sec:onetable`) names this theorem as the Π_1^0 level for heap programs.

This obstruction concerns an unbounded universal property, not the usefulness of finite certificates. For a fixed bound, either a safe prefix or a faulting prefix can be checked by finite methods. What fails is the inference from “all finite faults have existential certificates” to “absence of all future faults has an existential certificate of the same kind.” Similar care is required for fairness, perpetual productivity, and all-schedule termination.

83.3 Other limits that should remain explicit

Our equations are interpreted over $\mathbb{N}$, with zero included. Replacing that domain by arbitrary integers changes the order gadgets, and replacing it by reals changes the discrete semantics. A standard existential domain conversion may preserve existence without preserving uniqueness. Neither conversion is part of the claimed bijection.

Similarly, $P \geq 1$ on violated integer assignments is an exact integrality statement, not a robustness theorem for noisy real measurements. Nearby real points may have small residuals; an analog physical implementation would need an additional analysis of integrality enforcement, perturbations, and energy barriers.

84 A formalization and integration plan [08]

The following order isolates the genuinely new proof obligations from the already available repository infrastructure. It is a proposed implementation plan, not a claim that the corresponding Lean files have been created.

84.1 Arithmetic layer

Start with a structure for a finite list of integer quadratic residuals over natural variable assignments. Define its sum-of-squares polynomial and prove that evaluation is zero exactly when every residual evaluates to zero. Keep variable allocation explicit so that counts are theorems about the syntax, not estimates extracted from a pretty-printer.

Next prove the strict-comparator, normalized-zero and signed-split lemmas. The comparator proof should include its precise precondition: equal keys have no witness. Prove that routing preserves the multiset of complete records, not merely their key sequence. A generic deterministic network correctness theorem can then separate numerical sort correctness from the polynomial implementation of each comparator.

84.2 Memory semantics

Define chronological evaluation by a finitely supported map with default zero. The central semantic lemma should say that, after sorting distinct address/time keys, read validity is equivalent to the adjacent-access scan. Prove its two directions independently of polynomial syntax. Then compose it with the comparator and scan certificates to obtain an actual equivalence of valid logs and auxiliary zero assignments. A separate uniqueness theorem should quantify over *all* natural helper assignments, not just the canonical witness generator's output.

The explicit formulas $10C + 4N - 3$ and $10C + 7N - 4$ can be proved by induction over the list of comparators, then by counting scan positions. This is a useful place to catch accidental free coordinates or unnoticed extra branch helpers.

84.3 Names and source languages

Represent a finite history together with its ordered allocation list. Prove that its normalization is invariant under name permutations and reflects orbit equivalence. The finite-partial-bijection extension in Theorem 78.1 is a small independent combinatorial lemma.

Formalize one source instance first: the immutable two-stack machine. Then prove its local instruction certificate, its heap-log correspondence, and its Turing simulation. Only after those foundations are checked should a generic graph-rule frontend be generalized. The repository's combinatory semantics should be connected by an explicit operational adapter, preserving the chosen reduction convention rather than silently changing it.

84.4 Interface to existing Diophantine closure

The repository's `Diophantine.exactIter_dioph` and `Diophantine.existsExactIter_dioph` consume a Diophantine relation on coded natural states and Diophantine endpoint substitutions [3]. A heap frontend still needs to supply a state coding and a theorem for its one-step relation; those are not discharged merely by naming the closure theorem. The bounded compiler can meanwhile be developed without committing to a global coding.

A productive API would keep two distinct interfaces: ordinary `Dioph` for existence, and a stronger relation carrying a specified witness equivalence and a resource-count contract. Forgetting the stronger information should be an explicit map. That design would make it much harder to infer single-foldness accidentally from an ordinary closure result.

85 Further research questions and proposed topics [08]

The questions below are deliberately sharper than asking only whether another universal substrate is Diophantine. Some seek an improved theorem; others identify a concrete formalization or implementation target.

Research question 85.1 (Reduce the exact memory constant; source 08). Can one reduce the 10C auxiliary and residual contribution while preserving unique extension for arbitrary natural addresses and values? One route is to recover an event's address from its key and the fixed range of its timestamp, but doing so may simply move the cost into quotient/remainder helpers. Another is to transport fewer fields and reconstruct the rest. The comparison must count every added helper, including those of disabled branches.

[write] Compare Research question 71.24.

Research question 85.2 (A practical linearithmic implementation; source 08). Implement a deterministic $O(N \log N)$ network behind the same memory interface and determine the crossover against bitonic sorting. The asymptotic corollary is established here using known sorting theory; a compact, verified implementation with acceptable constants remains an engineering and formalization target, not an open sorting-existence question.

[write] Compare Research question 71.26.

Research question 85.3 (Certificates without explicit sorting; source 08). Is there a unique-extension natural quadratic memory certificate with $o(N \log N)$ total coordinates and residuals? The restricted comparison lower bound does not exclude one. Any proposal based on products, hashing, permutation equations or integer encodings must prove exact equality of multisets and auxiliary uniqueness without a probabilistic soundness error.

[write] Answered in part inside this report. Part V's linear backend (Theorem 40.2) is a unique-extension quartic memory certificate with $24L$ witnesses and $22L + 1$ residuals, with no sorting: $o(N \log N)$ coordinates and residuals are therefore achievable. Its witnesses have $O(L^2(b + \log L))$ bits (Theorem 40.3). What stays open is $o(N \log N)$ coordinates with $O(b + \log L)$ -bit witnesses, which is Part V's size–height question (Research question 71.23).

Research question 85.4 (Name reuse and allocation epochs; source 08). Extend birth normalization to allocators that recycle deallocated names. A plausible representation uses an object identity consisting of a physical slot and a generation number. The difficulty is specifying which identity the source can observe and preserving stale-pointer faults. A generation encoding that changes pointer equality would not be semantics-preserving.

Research question 85.5 (Unordered simultaneous allocation; source 08). For graph rules whose newly created objects have no ordered birth slots, construct a certificate for the true quotient rather than for an ordered refinement. Can bounded rule automorphisms be handled by canonical local choices without introducing global graph-isomorphism obligations? Repeated interactions of local symmetries may prevent a purely local answer.

Research question 85.6 (Quotient independent schedules; source 08). Birth normalization preserves scheduling. Can one additionally quotient independent local rewrites and retain a small unique-extension certificate? The target equivalence must specify conflicts caused by shared reads, writes, allocation and deletion. A canonical schedule should be proved to exist and be unique for each equivalence class, not selected by an arbitrary existential ordering witness.

[write] Compare Research questions 71.4 and 71.43.

[write] Batch 78, 2 October 2026: not answered by Part XV, whose bijections are with scheduled executions (see the subsection after Theorem 145.3 on what the bijection does not quotient); manuscripts 13–15 pose the question again for interaction nets (Section 149).

Research question 85.7 (A complete interaction-combinator frontend; source 08). Fix a port-level presentation of Lafont’s interaction combinators and prove the bounded-local adapter discussed in Section 81. Deliver the rule interpreter, semantic simulation, polynomial compiler and tests. Explicitly handle reciprocal ports, self-connections, parallel edges, alias patterns, interface splicing and creation slots. This would give a particularly small unconventional universal source with a fully exposed Diophantine compilation path.

[write] Compare Research question 71.43.

[write] Batch 78, 2 October 2026: answered for bounded scheduled histories by Part XV (manuscripts 13–15). A port-level presentation with Lafont’s numbered conventions is fixed (Section 140). Rule interpreters and semantic simulation are implemented by all three manuscripts, with reciprocal ports, self-connections, parallel temporary edges, interface splicing and creation slots (Section 141); the alias cases are the internal boundary patterns of Theorem 143.1. Quartic compilers with one witness per legal schedule are proved, at linear size on paper (Theorem 145.3, with manuscript 13’s Remark 145.5), and implemented for fixed schedules at size $O(T(C_0 + T + F)^2)$ (Theorem 146.7); the tests are in Section 148. Still open: an exporter for the unknown-schedule compiler, fixed arity, an ordinary-integer universal loader, and a quotient of independent schedules.

Research question 85.8 (Link to the repository’s exact operational models; source 08). Connect one of the repository’s SKI or Iota reduction conventions to a specified graph implementation. Determine whether the adapter preserves only termination, individual reduction occurrences, or a stronger observational semantics. Shared graph reduction and textual duplication should not be identified without a proof of the desired equivalence.

[write] Compare Research question 71.42.

Research question 85.9 (Canonical bounded-history compression; source 08). Under additional restrictions, can $O(T)$ named record coordinates be compressed into substantially fewer natural variables while preserving a bijection? Useful restricted classes might bound address growth, recurrence order or branching. Record the height cost and every coding helper. A successful bounded compression is not automatically a fixed-arity finite-fold MRDP theorem.

[write] Compare Research question 71.48.

Research question 85.10 (Preserve counting under composition; source 08). Develop a formal category or library of witness-bijective reductions, with composition, disjoint sums, deterministic branching

and explicitly tracked quotients. Which existing Diophantine closure proofs can be upgraded to this interface, and where do they necessarily lose multiplicity information? The guarded quadratization theorem supplies one small compositional building block.

[write] Compare Research question 71.1.

Research question 85.11 (Verified resource and bit-height accounting; source 08). Mechanize both syntactic counts and the polynomial-height theorem, including coefficient sizes. Extend the latter to selected scalar primitives with controlled growth. This would distinguish a small number of enormous integer coordinates from an actually short binary certificate.

[write] Compare Research question 71.1.

Research question 85.12 (Beyond exact natural semantics; source 08). For physical or analog computational substrates, identify a finite discrete trace relation that faithfully captures the intended computation, then analyze what perturbation tolerance the resulting polynomial has. Exact natural zeros alone do not establish that a laboratory system reliably realizes the same computation. The safety obstruction also warns against claiming existential certificates for all unbounded behavioral properties.

[write] Compare $\text{pqc} : \text{q} : \text{continuous}$, the question on continuous-time and robust physical semantics in the neighbouring report *probabilistic-quantum-and-continuous-computation*.

86 Conclusion [08]

The useful research target is not another isolated invocation of MRDP. It is a compiler with visible operational semantics, exact arithmetic constraints, controlled witness multiplicity, and auditable cost. For dynamic heaps, the construction here provides that target in three separable layers: a unique-extension memory certificate with explicit constants, canonical birth-order names, and finite local instruction certificates.

The result is a quartic natural-zero representation in bijection with bounded computations of a pointer-only universal substrate and its stated local graph generalization. The implemented memory layer realizes the explicit bitonic bound and supplies reproducible exact checks. The broader source compiler and its Lean integration remain clearly identified proof engineering tasks. Unbounded fixed-arity multiplicity, unordered graph symmetries, and perpetual safety are not silently absorbed into the bounded theorem; they mark distinct mathematical boundaries and productive research directions.

87 Reproduction and artifact map [08]

[write] *Appendix A of manuscript 08.*

The code uses the Python standard library and syntax supported by Python 3.10 and later. From the archive directory:

```
python tests/run_tests.py
python code/memory_quartic.py \
  --events examples/memory_events.json \
  --output examples/regenerated_certificate.json
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

The final two or three LaTeX passes resolve the table of contents, theorem references and bibliography links. A POSIX build script is included. No external Python package, solver, internet connection or repository checkout is required for the memory checks. The LaTeX build requires the packages named in the preamble, including `newtx`.

[write] *This command listing is the manuscript's own and keeps its delivered layout, in which the programs import one another by their delivered names; the shipped copies carry the prefix `08-dynamic-heaps-` and must be copied back to that layout, outside the collection, before they are run. Where `python` does not resolve, use `py`. The README of this report gives rerun recipes that write only into the copy. The manuscript's own `article.tex` is not shipped; its text is this Part.*

The principal files are `article.tex` and `article.pdf`; `code/` contains the implementation; `tests/` contains checks and the recorded receipt; `examples/` contains the worked input and complete certificate. `PROVENANCE.md` records the inspected repository snapshot, source dependencies and verification limitations. A checksum manifest permits accidental file changes to be detected. No Lean certificate is included.

[write] *Shipped here: `code/08-dynamic-heaps-memory_quartic.py`, `code/08-dynamic-heaps-pointer_machine.py`, `code/08-dynamic-heaps-run_tests.py` and the POSIX build script `code/08-dynamic-heaps-build.sh` (delivered `code/` and `tests/`); `data/08-dynamic-heaps-memory_events.json`, `data/08-dynamic-heaps-memory_certificate.json` and `data/08-dynamic-heaps-receipt.json` (delivered `examples/` and `tests/`); and `08-dynamic-heaps-PROVENANCE.md` at the report root. Not shipped: the manuscript's `article.tex` and `article.pdf`, whose text is this Part and Section 3.8, and the checksum manifest `SHA256SUMS`, verified and retired at placement.*

88 Dependency and claim ledger [08]

[write] *Appendix B of manuscript 08.*

Statement	Status and dependency
Strict comparator, zero test, signed split, guarded quadratization	Elementary proofs in this article; finite gadget checks included.
Memory uniqueness, soundness and exact counts	Proved here for any correct deterministic sorting network; complete bitonic implementation and finite tests included.
Bitonic sorting and comparator count	Established network construction; proof and implementation supplied.
$O(N \log N)$ memory certificate	Corollary using an established linearithmic network; that network is not implemented in this archive.
Birth-order orbit theorem	Proved here under ordered births and global freshness.
Local heap and graph compiler	Mathematical construction and proofs supplied; full frontend not implemented or Lean-checked.
Two-stack universality	Direct Turing simulation proved here; pointer-log demonstration implemented.
Ordinary unbounded Diophantineness	Uses MRDP or a separately verified coded-step adapter plus repository closure.
No existential perpetual-safety representation	Halting-complement reduction proved here.
Global historical novelty or single-fold MRDP	Not established or claimed.

Part XI

Order-free self-assembly and a positivity-restricted degree threshold

Source and scope. [write] This Part is manuscript 09, “Order-Free Diophantine Self-Assembly and a Sharp Positivity-Restricted Degree Threshold” (batch 62, manuscript 02; archive `order_free_diophantine.zip`; shipped files prefixed `09-order-free-`; labels `cdc:of:`). The manuscript is in three parts, printed here as three sections, with its sections demoted to subsections: (I) the degree classification for polynomials nonnegative on the real orthant (Section 90, main theorem Theorem 90.2); (II) the order-free compiler for seeded irreversible tile assembly (Section 91, main theorem Theorem 91.7); (III) its implementation, formalization plan, ten research questions and conclusion (Section 92). Its title, abstract, status statement and introduction are in Section 3.9; the conventions of this Part are Section 89.

Relation to the other Parts. [write] Manuscript 09 inspected ProveIt at 4e128356d, before this report existed, and does not cite it. Its two halves attach to different Parts. The degree classification is a fixed-arity boundary theorem of the kind collected in Part IX: no polynomial of degree at most three that is nonnegative on the whole real orthant is a universal certificate (Corollary 90.17). It partly answers the question on sharp degree, arity and domain comparisons (Research question 71.16); see the note there. It is compatible with Part III’s depth hierarchy (flat schemes Presburger, Theorem 26.1; separated one-counter schemes of depth d and equations of degree d), whose degree- d polynomials change sign on the orthant and so lie outside the classes $\mathcal{D}_d^+$. The assembly compiler is a companion of Part II’s base theorem (Theorem 11.2): both give a convex quadratic whose natural zeros are in bijection with equivalence classes of bounded executions, but here the classes are the legal orders of a

monotone accretion, which are not the commutation classes of any fixed independence relation (the two-by-two example below is not the set of linear extensions of a poset). The absence of a computable spatial cutoff (Proposition 91.13) is the tile case of Proposition 62.2.

Setting and hypotheses. [write] In the first half, positivity is *joint*: the polynomial is nonnegative at every real point of the nonnegative orthant, inputs and witnesses together. In the second half, the tile model is the seeded, irreversible abstract Tile Assembly Model with nonnegative glue strengths, a positive temperature, a finite candidate domain supplied as compiler data, and labelled targets; the number of coordinates grows with the domain, so the compiler is not a fixed-arity representation of an unbounded predicate.

89 Conventions of Part XI

[write] The letters are manuscript 09's, with one renaming: its tile palette $T = \{1, \dots, q\}$ is written $\mathcal{T}$ here, because elsewhere in this report T is a horizon or a number of steps. The tempting false reading of the main theorem is that it bounds the degree of Diophantine representations in general. It does not: $\mathcal{D}_d^+$ requires nonnegativity on the whole real orthant, and without that hypothesis the squares have the degree-two representation $x - y^2 = 0$, which changes sign on the orthant. $\mathcal{D}_3^+ = \text{SL}$ is a statement about orthant-nonnegative polynomials only; natural solvability of arbitrary cubics is untouched.

Symbol	Meaning in Part XI, and the clash
$\mathcal{D}_d^+(k), \text{SL}(k), \text{CE}(k)$	sets of k -tuples represented by integer polynomials of degree at most d that are nonnegative on the real orthant $\mathbb{R}_{\geq 0}^{k+m}$; semilinear sets; computably enumerable sets.
d, k, m	degree bound, input arity and witness arity in the first half; in the second half m is the number of variables of the compiler ((91.17)). Part II uses d for places and m for transitions.
$\mathcal{T}, q, \tau$	tile palette (manuscript 09: T), number of tile types, temperature.
D, N, s, S	finite candidate domain, $N = D $, seed size $s = S $, seed support S . Part V and Part X use N for a padded log length; Part II for a final marking.
K, J, ℓ	ordered neighbour pairs, compatibility tuples, number of affine equations.
$C(x, y; \dots)$	the unbounded comparison penalty (90.5); Parts V and X use C for a number of comparators.
h, H_D, H	size of the exterior halo H_D ; also a slack of the comparison penalty and the rank slacks h_v . H is a Hessian in the first half.
L_I, F_I°	affine zero space and relative interior of a coordinate face; not Part V's log length L .
E_a	the Euclidean-division penalty (90.6); not the events E of Parts V and X.
$\mathbf{1}_{\{\cdot\}}$	indicator of a condition; manuscript 09's macro <code>\ind</code> is renamed <code>\indset</code> , because this report's <code>\ind</code> is the constant 1 .

Polynomials in the first half may have rational or real coefficients where a lemma says so (Lemma 90.3 and Theorem 90.7); the classes $\mathcal{D}_d^+$ use integer polynomials. The comparison penalty accepts arbitrary integers x, y as inputs.

90 A sharp degree threshold under real-orthant nonnegativity [09]

[write] *Part I of manuscript 09.*

90.1 Representation classes and semilinear sets

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$, and every polynomial equality is an ordinary equality over $\mathbb{Z}$ or $\mathbb{R}$, not an equality using truncated natural subtraction.

Definition 90.1. For $k \geq 1$ and $d \geq 0$, let $\mathcal{D}_d^+(k)$ consist of sets $S \subseteq \mathbb{N}^k$ for which some $m \geq 0$ and $P \in \mathbb{Z}[X_1, \dots, X_k, Y_1, \dots, Y_m]$ satisfy

$$\deg P \leq d, \quad (90.1)$$

$$P(z) \geq 0 \quad (z \in \mathbb{R}_{\geq 0}^{k+m}), \quad (90.2)$$

$$x \in S \iff \exists y \in \mathbb{N}^m P(x, y) = 0. \quad (90.3)$$

We write $\text{SL}(k)$ for the semilinear subsets of $\mathbb{N}^k$, and $\text{CE}(k)$ for the computably enumerable subsets of $\mathbb{N}^k$.

A linear set has the form

$$b + \mathbb{N}v_1 + \dots + \mathbb{N}v_t, \quad b, v_j \in \mathbb{N}^k;$$

a semilinear set is a finite union of linear sets. Zero generators and zero generators in number are allowed. The empty union is the empty set. Coordinate projections of semilinear sets are semilinear, simply by projecting their bases and generators. We will prove directly the specific linear-equation facts needed below.

Theorem 90.2 (Positivity-restricted degree classification). *For every $k \geq 1$,*

$$\boxed{\mathcal{D}_2^+(k) = \mathcal{D}_3^+(k) = \text{SL}(k), \quad \mathcal{D}_4^+(k) = \text{CE}(k).}$$

Every semilinear set admits a single-fold degree-two representation in this class. Moreover, the full natural zero set of any rational polynomial of degree at most three that is nonnegative on the real nonnegative orthant is a finite union of natural solutions of rational affine systems. Such a union, and hence a Presburger definition, is effectively computable.

The theorem is stronger than the observation that a globally nonnegative polynomial cannot have odd highest degree. We allow nonnegativity only on the orthant, where genuine cubics such as x^2z are nonnegative. We also impose no sum-of-squares or convexity hypothesis on the cubic.

90.2 Facewise rigidity of nonnegative cubics

Lemma 90.3 (Affine zero slice). *Let $O \subseteq \mathbb{R}^r$ be an open convex set. Let p be a real polynomial of degree at most three with $p \geq 0$ on O , and suppose $a \in O$ and $p(a) = 0$. With $H = \nabla^2 p(a)$,*

$$H \succeq 0, \quad \nabla p(a) = 0, \quad \{x \in O : p(x) = 0\} = (a + \ker H) \cap O.$$

In addition, p vanishes identically on the entire affine space $a + \ker H$, including points outside O .

Proof. The point a is a local minimum. More elementarily, for every direction h , the polynomial $p(a + th)$ is nonnegative for sufficiently small positive and negative t , and has zero constant term. Its linear coefficient is therefore zero and its quadratic coefficient is nonnegative. Varying h gives $\nabla p(a) = 0$ and $H \succeq 0$.

Suppose first that $b \in O$ is another zero, and put $h = b - a$. The univariate polynomial $g(t) = p(a + th)$ has a zero at both 0 and 1. Each is a local minimum because both a and b are interior points of the region of nonnegativity. Unless g is identically zero, each of these two distinct roots has multiplicity

at least two. That is impossible for a polynomial of degree at most three. Consequently $g \equiv 0$, so $h^T H h = 0$. A positive-semidefinite symmetric matrix has $h^T H h = 0$ exactly when $H h = 0$. Thus $b - a \in \ker H$.

Conversely, let $h \in \ker H$. Exact Taylor expansion, with no remainder, gives

$$p(a + th) = t^3 C(h)$$

for some cubic homogeneous expression $C(h)$, since the constant, linear, and quadratic terms vanish. Nonnegativity for small t of both signs forces $C(h) = 0$. Hence $p(a + th) = 0$ for every real t , in particular at $t = 1$. This works for every $h \in \ker H$, which proves both assertions. $\square$

Remark 90.4 (Why this is a special zero-set statement). The set of all local minima of an arbitrary cubic need not itself be a polyhedron; the spectrahedral descriptions in [43] are more general. Here a prescribed zero level is a lower bound *throughout the chosen region*. Inside that region, every point of the affine zero space remains at the lower bound. It is this additional nonnegativity hypothesis that permits the stronger affine-slice conclusion.

For a subset $I \subseteq \{1, \dots, r\}$, write

$$F_I^\circ = \{z \in \mathbb{R}_{\geq 0}^r : z_i > 0 \text{ exactly when } i \in I\}.$$

It is the relative interior of a coordinate face. Restricting a polynomial to this face means setting all coordinates outside I to zero; its active coordinates range over $\mathbb{R}_{> 0}^I$. The Hessian in Lemma 90.3 is the Hessian of this *restricted* polynomial. The full Hessian need not be positive semidefinite at a boundary zero: $p(x, y) = xy$ is an elementary example.

Proposition 90.5 (At most one affine zero space per face). *Let $p \in \mathbb{R}[Z_1, \dots, Z_r]$ have degree at most three and be nonnegative on $\mathbb{R}_{\geq 0}^r$. For each coordinate face containing a zero in its relative interior, choose such a point a^I . There is an affine space L_I , contained in the coordinate subspace supported on I , such that*

$$\{z \in \mathbb{R}_{\geq 0}^r : p(z) = 0\} = \bigcup_I (L_I \cap \mathbb{R}_{\geq 0}^r).$$

There are at most 2^r spaces. If a^I and the coefficients of p are rational, then L_I is defined by rational affine equations.

Proof. For a nonempty face set

$$L_I = \{z : z_j = 0 \ (j \notin I), \quad H_I(a^I)(z_I - a^I) = 0\},$$

where H_I is the restricted Hessian. Lemma 90.3 identifies the zeros in the relative interior of this face and says that p vanishes on all of L_I . Thus it is legitimate to intersect L_I with the *closed* orthant, possibly including zeros of smaller support. Every zero is accounted for by its own support face, and every included point is still a zero. The empty face contributes $\{0\}$ precisely when $p(0) = 0$. Rationality follows by evaluating the rational Hessian at a rational point. $\square$

90.3 From affine zero spaces to semilinear arithmetic

90.3.1 An elementary finite-generation lemma

We use the componentwise order on $\mathbb{N}^r$. Dickson's lemma says that every subset of $\mathbb{N}^r$ has only finitely many componentwise-minimal members. A short justification is useful: every infinite sequence in $\mathbb{N}$ has an infinite nondecreasing subsequence, either by selecting an infinitely repeated value or by recursively selecting increasing values. Applying this successively to the r coordinates gives two distinct terms in componentwise nondecreasing order. An infinite antichain is therefore impossible. The minimal members of any set form an antichain.

Lemma 90.6 (Natural solutions of affine systems). *For an integer matrix A and integer vector b ,*

$$V_b = \{z \in \mathbb{N}^r : Az = b\}$$

is semilinear.

Proof. Let $H = \{h \in \mathbb{N}^r : Ah = 0\}$, a commutative additive monoid. By Dickson's lemma, the set G of minimal nonzero elements of H is finite. It generates H : for a nonzero $h \in H$, choose a minimal nonzero $g \leq h$. Then $h - g \in H$, and induction on $\sum_i h_i$ completes the decomposition. If $H = \{0\}$, the empty set of generators suffices.

Let M be the finite set of minimal elements of V_b . For each $z \in V_b$, the finite nonempty set of solutions bounded above by z contains a minimal member m ; it is also globally minimal. Then $z - m \in H$. Conversely $m + h \in V_b$ whenever $m \in M$ and $h \in H$. Consequently

$$V_b = \bigcup_{m \in M} \left(m + \sum_{g \in G} \mathbb{N}g \right).$$

If V_b is empty, this is the empty union. □

90.3.2 The arithmetic collapse in degree three

Theorem 90.7 (Semilinearity of the full natural zero set). *Suppose $p \in \mathbb{Q}[Z_1, \dots, Z_r]$, $\deg p \leq 3$, and $p \geq 0$ on $\mathbb{R}_{\geq 0}^r$. Then*

$$Z_{\mathbb{N}}(p) = \{z \in \mathbb{N}^r : p(z) = 0\}$$

is a union of at most 2^r sets of natural solutions of rational affine systems. In particular it is semilinear.

Proof. For each support I for which there is a natural zero of exact support I , choose one such zero a^I . Unlike an arbitrary real zero, this is rational. Define L_I as in Proposition 90.5. Every natural zero belongs to the space selected for its support. Conversely the polynomial vanishes on every selected space, so all its natural points are zeros. Hence

$$Z_{\mathbb{N}}(p) = \bigcup_I (L_I \cap \mathbb{N}^r).$$

Each L_I has rational affine equations. Clearing denominators and applying Lemma 90.6 proves the result. □

Projecting away witness coordinates now gives $\mathcal{D}_3^+(k) \subseteq \text{SL}(k)$. This proof is elementary but does not yet give a terminating procedure for selecting the a^I : enumerating natural candidates on a face with no natural zero could run forever. The next subsection removes exactly that gap.

90.3.3 Effectivity without assuming rational real zeros

Proposition 90.8 (Effective affine decomposition). *Given a rational polynomial of degree at most three satisfying the orthant-nonnegativity promise, one can effectively compute a finite union of rational affine systems whose natural solutions are exactly its zeros. Thus natural solvability and every existential projection are effectively reducible to Presburger arithmetic.*

Proof. For every support I , use effective real-algebraic decision and sampling to decide whether

$$\exists z_I > 0 \quad p(z_I, 0) = 0$$

holds and, if so, obtain a real algebraic sample a^I . These are standard capabilities of algorithms over real closed fields; see [45]. The empty face is checked by direct evaluation.

The Hessian equations defining L_I now have real algebraic coefficients. Put their finitely many coefficients, including the constant terms, in one finite algebraic extension $K/\mathbb{Q}$, and choose a $\mathbb{Q}$ -basis $\theta_1, \dots, \theta_d$. Exact arithmetic with real algebraic numbers and such a basis is effective. For rational unknowns z , expand a typical affine equation as

$$\sum_j c_j z_j = c_0, \quad c_j = \sum_{h=1}^d c_{jh} \theta_h, \quad c_{jh} \in \mathbb{Q}.$$

It is equivalent to the d rational equations

$$\sum_j c_{jh} z_j = c_{0h} \quad (1 \leq h \leq d),$$

by linear independence of the basis. Clear denominators, retain the inactive-coordinate equations, and intersect with $\mathbb{N}^r$. Proposition 90.5 proves that the union so obtained is exact. All steps terminate. The final existential affine formula is a Presburger formula; quantifier elimination supplies an effective decision procedure and semilinear description [42]. $\square$

Example 90.9 (An algebraic sample need not be rational). The polynomial

$$p(x, y) = 2x^3 + 4y^3 + 1 - 6xy$$

is nonnegative on the orthant by the arithmetic–geometric mean inequality applied to $2x^3, 4y^3, 1$. Equality holds exactly when all three terms are equal, so its unique nonnegative real zero is

$$(x, y) = (2^{-1/3}, 2^{-2/3}).$$

There is no rational zero. A method that silently replaces real-algebraic sampling by rational sampling would fail even in this two-variable case. The field-basis step above correctly turns the affine description of this singleton into an inconsistent system for rational unknowns.

Remark 90.10 (What the effectiveness assertion does not say). No polynomial-time complexity bound is asserted. The number of support faces is exponential in the number of variables, and real-algebraic sampling can be expensive. The orthant-nonnegativity promise itself is a first-order real sentence and is decidable in principle, but the included Python prototype does not implement real quantifier elimination.

90.3.4 A polyhedral extension

Corollary 90.11 (Nonnegative cubics on rational polyhedra). *Let $C \subseteq \mathbb{R}^r$ be a rational polyhedron, and let $p \in \mathbb{Q}[Z_1, \dots, Z_r]$ have degree at most three and satisfy $p \geq 0$ on C . Then*

$$\{z \in C \cap \mathbb{N}^r : p(z) = 0\}$$

is semilinear, with an effective Presburger definition.

Proof. Decompose C into relative interiors of its finitely many faces. Each face has a rational affine hull and therefore admits a rational affine parametrization. On its relative interior, the restriction of p is nonnegative and Lemma 90.3 applies. For a face containing a natural zero, choosing that zero

makes the affine kernel space rational in the original coordinates. Its intersection with the face and the natural lattice is described by rational affine equalities and inequalities. After clearing denominators, each non-strict inequality is equivalent to an equality with one natural slack. The natural solution set is semilinear by Lemma 90.6, followed by projection. There are only finitely many faces.

For an effective version, compute the faces, choose real algebraic samples where zeros exist in their relative interiors, and use the field-basis conversion from Proposition 90.8. The affine zero space may be intersected with the closed face, because the polynomial vanishes on the whole space. This avoids any difficulty with strict inequalities at face boundaries. $\square$

90.4 Quadratic completeness for semilinearity, quartic completeness for computation

90.4.1 Every semilinear set has an orthant-nonnegative quadratic

A finite disjunction of unbounded linear sets cannot be encoded by a bounded big- M trick without further care: the coefficients of a selected linear set may be arbitrarily large. Nonnegative product penalties provide a direct solution.

Proposition 90.12 (An explicit quadratic for a semilinear set). *Suppose*

$$S = \bigcup_{i=1}^s \left(b_i + \sum_{j=1}^{m_i} \mathbb{N} v_{ij} \right) \subseteq \mathbb{N}^k.$$

Then $S \in \mathcal{D}_2^+(k)$.

Proof. For $s = 0$, use the constant polynomial 1. Otherwise introduce natural variables e_i, f_i, c_{ij} and define

$$\begin{aligned} Q(x, e, f, c) = & \left(\sum_i e_i - 1 \right)^2 + \sum_i (e_i + f_i - 1)^2 \\ & + \sum_{\ell=1}^k \left(x_\ell - \sum_i b_{i\ell} e_i - \sum_{i,j} v_{ij,\ell} c_{ij} \right)^2 + \sum_{i,j} f_i c_{ij}. \end{aligned} \quad (90.4)$$

Every term is nonnegative on the entire real nonnegative orthant, and Q has degree at most two.

At a natural zero, $\sum e_i = 1$ forces exactly one selector, say e_{i_0} , to equal one; every other selector equals zero. The equations $e_i + f_i = 1$ then give $f_{i_0} = 0$ and $f_i = 1$ for $i \neq i_0$. The product penalties force all inactive coefficients c_{ij} to be zero. The remaining coordinate equations say $x = b_{i_0} + \sum_j c_{i_0 j} v_{i_0 j}$, as required. Conversely, any expression of x in one selected linear set supplies these variables and makes every term zero. $\square$

The polynomial in (90.4) need not be convex. Its orthant-nonnegative cross terms $f_i c_{ij}$ are not squares. Nor is this representation claimed to be single-fold: distinct linear-set expressions may represent the same point.

Combining Proposition 90.12 with Theorem 90.7 and the inclusion $\mathcal{D}_2^+ \subseteq \mathcal{D}_3^+$ proves the degree-two/degree-three part of Theorem 90.2.

90.4.2 A stronger result: single-fold quadratics for every semilinear set

The preceding direct union formula is economical, but its witnesses may not be unique. Existence of a unique-witness quadratic is nevertheless true for *every* semilinear set. The key is to evaluate a

quantifier-free Presburger formula deterministically, rather than choose one of its possible linear-set descriptions.

Lemma 90.13 (An unbounded unique comparison penalty). *For arbitrary integers x, y , define*

$$C(x, y; p, r, c, f, h) = (x + p - y - r)^2 + pr + (c + f - 1)^2 + (p - c - h)^2 + fp. \quad (90.5)$$

There is exactly one natural zero in the five witness coordinates. It has $c = \mathbf{1}_{\{x < y\}}$. The polynomial is quadratic and is nonnegative whenever the five witnesses are nonnegative reals, regardless of the real values of x, y .

Proof. Every displayed term is nonnegative on that domain. At a zero, $p - r = y - x$ and $pr = 0$, so

$$p = \max(y - x, 0), \quad r = \max(x - y, 0).$$

The equation $c + f = 1$ makes c, f complementary Boolean variables. If $p = 0$, then $p = c + h$ forces $c = h = 0$, and $f = 1$. If $p > 0$, then $fp = 0$ forces $f = 0, c = 1$, and $h = p - 1 \geq 0$. These possibilities determine all coordinates and satisfy all terms. Thus the zero exists uniquely, with the claimed bit. $\square$

Here no big- M constant and no a priori operand bound is needed. This is possible because pr and fp are nonnegative product penalties, not squared bilinear residuals. Unlike the bounded affine comparator used later for self-assembly, this penalty need not be convex. Replacing x, y by integer affine expressions in the external inputs preserves both the degree and orthant nonnegativity.

For a fixed modulus $a \geq 1$ and any integer affine expression $L(x)$, introduce natural variables u, v, ρ, s and the penalty

$$E_a(L; u, v, \rho, s) = (L - a(u - v) - \rho)^2 + (\rho + s - (a - 1))^2 + uv. \quad (90.6)$$

At a natural zero, $0 \leq \rho < a$, and Euclidean division gives the unique quotient $q = \lfloor L/a \rfloor$ and remainder ρ . The condition $uv = 0$ then forces $u = \max(q, 0)$, $v = \max(-q, 0)$, and $s = a - 1 - \rho$. Thus all four coordinates are unique, including for negative L . Append $C(0, \rho; p, r, c, f, h)$. Its complement bit f equals one precisely when $a \mid L$.

Theorem 90.14 (Single-fold quadratic Presburger representation). *Every semilinear set $S \subseteq \mathbb{N}^k$ has a representation*

$$x \in S \iff \exists y \in \mathbb{N}^m Q(x, y) = 0$$

by an integer polynomial of degree at most two that is jointly nonnegative on the real nonnegative orthant, with exactly one natural witness y for each $x \in S$, and none for $x \notin S$. Such a polynomial is effectively constructible from a Presburger formula for S .

Proof. By Presburger quantifier elimination, obtain an equivalent quantifier-free Boolean combination of affine order tests and congruence tests with fixed integer moduli [42, 35]. Strict comparisons are evaluated by (90.5); non-strict comparisons reduce to strict comparisons by adding one to an integer endpoint. Equality is the conjunction of the negations of the two strict comparisons. Congruence is evaluated by (90.6) followed by the remainder comparison. A zero modulus, if present in the chosen syntax, reduces to equality; a negative modulus can be replaced by its absolute value.

Evaluate the entire Boolean circuit, including both sides of each Boolean connective. Negation of a Boolean input b is enforced by $(b + z - 1)^2$. A conjunction $z = b_1 \wedge b_2$ is enforced by the sum of the squares of

$$z + p_1 - b_1, \quad z + p_2 - b_2, \quad z + 1 - b_1 - b_2 - r,$$

with natural slacks. For Boolean inputs this has exactly one natural extension: if either input is zero, the upper equations force $z = 0$; if both are one, the last equation forces $z = 1$. All slacks are then fixed. Disjunction is expressible through negation and conjunction. Constants cause no difficulty.

Sum all atom penalties and all gate penalties, and add $(b_{\text{out}} - 1)^2$ for the final output bit. This is an integer quadratic; each constituent term is either the square of an affine expression or a product of two nonnegative witness coordinates. Hence it is nonnegative on the whole real nonnegative orthant jointly with the external inputs.

For any fixed natural input, each atom has exactly one natural extension, regardless of whether that atom is true. Induction along the Boolean circuit gives exactly one extension to all gates and their slacks. The final output condition is satisfied exactly when the input belongs to S . Thus there is one witness for a member and no witness for a nonmember. Evaluating both Boolean branches is essential: no inactive branch leaves free arithmetic coordinates. $\square$

[write] ProveIt's Presburger development (`Logic/PresburgerArithmetic`) formalizes Cooper quantifier elimination over the integers and exposes a decision procedure for sentences, `PresburgerArithmetic.Formula.presburgerArithmetic_decidable` (a definition in `Lean/PresburgerArithmetic/Decision.lean`). It does not export a formula-level elimination over $\mathbb{N}$ in the form this proof uses; compiling such an elimination into single-fold quadratics is a formalization obligation, not an existing interface (see Section 92.2).

Consequently the degree-two/degree-three classification remains valid if representations are required to be single-fold. The single-fold existence assertion is restricted to semilinear sets and does not imply a single-fold representation theorem for arbitrary computably enumerable sets. The quartic normalization below has a different witness guarantee.

90.4.3 Degree four retains all computably enumerable relations

Proposition 90.15 (Quartic normalization). *Every computably enumerable subset of $\mathbb{N}^k$ belongs to $\mathcal{D}_4^+(k)$. The representing polynomial can be chosen nonnegative on all of $\mathbb{R}^{k+m}$, not just on the orthant.*

Proof. By MRDP, choose a polynomial $F(x, y) \in \mathbb{Z}[x, y]$ such that $x \in S$ exactly when $F(x, y) = 0$ for some natural tuple y [34]. The usual several-input version follows from the one-input theorem by computable tuple coding. Explicitly, the graph of Cantor pairing is given by

$$2c = (a + b)(a + b + 1) + 2b,$$

so finitely many pairing equations can also be included before the following normalization.

Write $F = F_+ - F_-$ with F_+, F_- having nonnegative integer coefficients. Compute both by acyclic arithmetic circuits using natural constants, additions, and multiplications. Introduce a natural value for each internal gate. An addition gate is expressed by $z - u - v = 0$, a multiplication gate by $z - uv = 0$, and equality of the two final outputs by an affine residual. The sum of the squares of all residuals has degree at most four and is globally nonnegative.

Every solution of $F = 0$ extends to the unique correct set of gate values. Conversely, in any natural zero of the sum of squares every residual vanishes; induction along the circuit recovers its correct values and hence $F = 0$. This proves the desired equivalence. $\square$

Every existential natural polynomial zero set is computably enumerable: enumerate its finite witness tuples and evaluate the integer polynomial. This converse proves $\mathcal{D}_4^+(k) = \text{CE}(k)$ and completes Theorem 90.2.

Corollary 90.16 (Degree four is necessary even for squares). *The set $\{n^2 : n \in \mathbb{N}\}$ has a single-fold globally nonnegative quartic representation, but has no representation in $\mathcal{D}_3^+(1)$.*

Proof. Use $(x - y^2)^2 = 0$. Its natural witness y is unique. An infinite one-dimensional semilinear set contains an infinite arithmetic progression: at least one of its finitely many linear components is infinite and has a positive generator. No infinite arithmetic progression consists only of squares. Indeed, the gaps between consecutive squares eventually exceed its fixed positive step. Theorem 90.2 therefore excludes degree at most three. $\square$

Corollary 90.17 (No nonnegative cubic universal halting certificate). *No computably enumerable undecidable set has a fixed-arity representation by a rational polynomial of degree at most three that is jointly nonnegative on the real nonnegative orthant.*

Proof. Such a set would be semilinear by the classification, and thus decidable. $\square$

90.4.4 Restrictions that must not be dropped in the statement

The results do not decide natural solvability of arbitrary cubics, and do not rule out any representation by a cubic that changes sign on the real orthant. Nonnegativity only on natural assignments is a different hypothesis and is not covered by the Hessian argument. Nonnegativity only after fixing an input does not give the joint semilinear conclusion unless it holds for every real nonnegative input as well. Arbitrary variable coefficients, interpreted outside the fixed polynomial's coordinate system, also change the question.

The quartic construction adds gate values uniquely *relative to an existing MRDP witness*. It does not make the original MRDP witnesses unique or finite in number. Consequently neither this normalization nor the degree threshold settles a finite-fold or single-fold MRDP conjecture.

91 Order-free arithmetic certificates for self-assembly [09]

[write] *Part II of manuscript 09.*

91.1 The seeded, irreversible model

We now give a direct compiler for a spatial substrate rather than passing first through a Turing-machine tableau. The abstraction is mathematical: there are no concentrations, rate constants, resource depletion, detachment events, or finite-precision physical measurements.

91.1.1 Tiles, bonds, seeds, and finite targets

Let $\mathcal{T} = \{1, \dots, q\}$ be a finite palette of square tile types with fixed orientation. Each side carries a glue label with a nonnegative integer strength. Matching glue labels contribute their strength and nonmatching labels contribute zero. Mismatches are permitted and give no bond. Let $\tau \geq 1$ be the attachment threshold, or temperature. For adjacent lattice sites u, v , write

$$g_{uv}(b, a) \in \mathbb{N}$$

for the bond contribution between a tile of type b at u and a tile of type a at v . These contributions agree on opposite orientations as induced by the glue labels. Add a blank label 0, and set $g_{uv}(0, a) = g_{uv}(b, 0) = 0$.

A finite nonempty seed σ with support $S \subseteq \mathbb{Z}^2$ is fixed. Assume it is τ -stable: every cut splitting its occupied sites into two nonempty parts has total bond strength at least τ . A singleton seed is stable vacuously. A tile of type a may be added at an unoccupied site v when the sum of its bonds to existing neighbors is at least τ .

Lemma 91.1 (Stability and capping). *Single-tile additions of total bond strength at least τ preserve τ -stability. Replacing every bond contribution g by $\min(g, \tau)$ leaves all attachment decisions unchanged.*

Proof. For a cut of the enlarged assembly, either both parts contain old sites, in which case the old cut already contributes at least τ , or one part consists only of the new tile, in which case its attachment bonds contribute at least τ . Nonnegative additional bonds cannot lower either sum. For capping, a sum reaching τ either has one term already at least τ , which stays so after capping, or has all terms below τ , which are unchanged. A sum below τ has every term below τ and is unchanged as well. $\square$

From now on assume $0 \leq g \leq \tau$.

Let $D \subseteq \mathbb{Z}^2$ be an explicitly supplied finite domain containing S , and put $N = |D|$, $s = |S|$. A target is a map $\alpha : D \rightarrow \mathcal{T} \cup \{0\}$ extending the seed, with all sites outside D blank. A target is *producible* if some sequence of legal additions produces exactly its occupied labelled sites. Irreversibility ensures that a sequence ending in this target never needs a tile outside it. The domain may contain holes, unused sites, or multiple components; it is not required to be a rectangle.

91.1.2 Monotone parallel closure

For a fixed target, define sets of reached sites by

$$\begin{aligned} C_0 &= S, \\ C_{r+1} &= C_r \cup \left\{ v \in \text{dom } \alpha : \sum_{u \in C_r, u \sim v} g_{uv}(\alpha(u), \alpha(v)) \geq \tau \right\}. \end{aligned} \quad (91.1)$$

Here $\text{dom } \alpha = \{v \in D : \alpha(v) \neq 0\}$. Only the target's prescribed tile type is tested at each site.

Lemma 91.2 (Sequential producibility equals full closure). *A target is producible if and only if $\bigcup_r C_r = \text{dom } \alpha$. If this holds, every nonseed site first appears in a round between 1 and $N - s$.*

Proof. If the closure is full, append each finite batch $C_{r+1} \setminus C_r$ in any order. Each tile was already enabled by C_r ; nonnegative bonds added within the batch cannot disable it. Thus the batches give a legal sequential construction.

Conversely, induction along any legal construction shows that all its tiles belong to the closure. When its next tile is added, its finitely many earlier neighbors all occur in some common closure round. It is then added by the next closure round. If a closure round adds nothing, all later rounds are equal, so a full finite closure has at most $N - s$ nonempty rounds. $\square$

91.2 Two local cuts force the unique earliest ranks

For a producible target, assign seed sites rank zero and each occupied nonseed site its first-arrival round in (91.1); blank sites are also assigned zero. It is not enough merely to require a site to have enough earlier neighbors: that permits arbitrarily delayed ranks. A second test rules out such delays.

Theorem 91.3 (Two-cut characterization). *Fix a target α . There is a rank assignment $t : D \rightarrow \mathbb{N}$ satisfying the following conditions if and only if α is producible. When it exists, the assignment is*

unique and is the first-arrival assignment:

$$t_v = 0 \quad \text{for seed and blank sites,} \quad (91.2)$$

$$1 \leq t_v \leq N \quad \text{for occupied nonseed sites,} \quad (91.3)$$

$$A_v := \sum_{u \sim v, t_u < t_v} g_{uv}(\alpha(u), \alpha(v)) \geq \tau \quad \text{for occupied nonseed sites,} \quad (91.4)$$

$$B_v := \sum_{u \sim v, t_u + 1 < t_v} g_{uv}(\alpha(u), \alpha(v)) \leq \tau - 1 \quad \text{for occupied nonseed sites.} \quad (91.5)$$

Only neighbors in D matter; all others are blank.

Proof. For first-arrival ranks, a site of rank $r \geq 1$ is enabled by C_{r-1} . Those occupied sites have ranks strictly below r , which proves (91.4). For $r \geq 2$ it was not enabled by C_{r-2} ; these are exactly the occupied sites with rank plus one strictly below r . The integer support is at most $\tau - 1$. For $r = 1$, the second sum is empty and the inequality holds since $\tau \geq 1$. The other conditions follow from Lemma 91.2.

Conversely, suppose the displayed conditions hold. Define

$$L_r = \{v \in \text{dom } \alpha : t_v \leq r\} \quad (r \geq 0).$$

We prove $L_r = C_r$ by induction. At zero both equal S . Assume equality for $r - 1$, where $r \geq 1$. If $t_v \leq r$, then either $v \in S$, or all sites contributing to (91.4) have rank at most $r - 1$ and lie in C_{r-1} . Monotonicity shows $v \in C_r$. Thus $L_r \subseteq C_r$.

For the reverse inclusion take $v \in C_r \setminus C_{r-1}$. It has support at least τ from $C_{r-1} = L_{r-1}$. If $t_v > r$, then for each $u \in L_{r-1}$ one has $t_u + 1 \leq r < t_v$. All those support contributions occur in B_v , contradicting (91.5). Hence $t_v \leq r$, and $v \in L_r$.

All occupied sites have rank at most N , so the closure is full. The identities $L_r = C_r$ also determine every rank uniquely. $\square$

Remark 91.4 (No circular activation). An unseeded island cannot create its own justification. Its minimum-positive-rank sites would need sufficient bonds to still earlier occupied sites, which are absent. More generally, the theorem rejects all self-supporting directed cycles unless they are enabled from the seed. The earliest-rank condition is a certificate of actual reachability, not a condition of static bond stability alone.

91.3 The explicit affine-to-quadratic compiler

All unknowns introduced in this section range over $\mathbb{N}$. Every constraint is affine over $\mathbb{Z}$. The tile palette, threshold, coordinates, and domain size are constants fixed before the polynomial is constructed.

91.3.1 Two unique-slack gadgets

Lemma 91.5 (A bounded strict comparator). *Suppose $0 \leq x \leq N + 1$, $0 \leq y \leq N$, and $M = N + 2$. The affine system*

$$\begin{aligned} c + \bar{c} &= 1, \\ x + 1 + p &= y + M\bar{c}, \\ y + r &= x + Mc \end{aligned} \quad (91.6)$$

has exactly one solution in $(c, \bar{c}, p, r) \in \mathbb{N}^4$. Its first coordinate is $c = \mathbf{1}_{\{x < y\}}$.

Proof. The first equation makes c Boolean. If $c = 1$, the second requires $x + 1 \leq y$, while the third has the unique slack $r = x + M - y \geq 0$. If $c = 0$, the third requires $y \leq x$, while the second has unique slack $p = y + M - x - 1 \geq 0$. The respective bounds on x, y make both stated slack inequalities valid. The two cases are disjoint and exhaustive, and all slacks are forced. $\square$

Lemma 91.6 (Boolean conjunction with unique slacks). *For given Boolean inputs $b_1, \dots, b_k$, the affine equations*

$$\begin{aligned} z + p_i &= b_i & (1 \leq i \leq k), \\ z + k - 1 &= \sum_{i=1}^k b_i + r \end{aligned} \tag{91.7}$$

have exactly one natural solution in $z, p_1, \dots, p_k, r$, with $z = \prod_i b_i$.

Proof. The upper equations give $0 \leq z \leq b_i \leq 1$. If any input is zero, $z = 0$, and the last slack is $r = k - 1 - \sum b_i \geq 0$. If every input is one, the last equation requires $z \geq 1$, so $z = 1$ and $r = 0$. All other slacks are $p_i = b_i - z$, uniquely determined. $\square$

This uses $k + 2$ variables and $k + 1$ equations. We will use $k = 3$ for realized bonds and $k = 2$ for hypothetical terminality bonds. These gadgets exploit integrality; their real relaxations need not have the same semantics.

91.3.2 Labels and ranks

Introduce variables $e_{v,a}$ for $v \in D$ and $0 \leq a \leq q$, with equations

$$\sum_{a=0}^q e_{v,a} = 1 \quad (v \in D), \tag{91.8}$$

$$e_{v,\sigma(v)} = 1 \quad (v \in S). \tag{91.9}$$

Because the variables are natural, each site has exactly one label. Let $\delta_v = 1 - e_{v,0}$ be an affine expression, not a new variable. It is the Boolean occupancy indicator.

Introduce t_v, u_v for every site and h_v for each nonseed site:

$$t_v + u_v = N\delta_v \quad (v \in D), \tag{91.10}$$

$$t_v = 0 \quad (v \in S), \tag{91.11}$$

$$t_v = \delta_v + h_v \quad (v \in D \setminus S). \tag{91.12}$$

These enforce (91.2)–(91.3). In particular, blank ranks and their slacks are all zero. The seed upper slack is N , uniquely determined.

91.3.3 Neighbor comparisons and realized support

Let E_D be the ordered neighboring pairs (u, v) in D whose destination v is not a seed site. Put $K = |E_D|$, so $K \leq 4(N - s)$. For each such edge make two copies of the comparator:

$$c_{uv}^0 = \mathbf{1}_{\{t_u < t_v\}}, \quad c_{uv}^1 = \mathbf{1}_{\{t_u + 1 < t_v\}}.$$

The comparator hypotheses hold because $0 \leq t_u, t_v \leq N$.

Let $\mathcal{J}$ consist of the nonzero compatibility tuples

$$(u, v, b, a), \quad (u, v) \in E_D, \quad 1 \leq a, b \leq q, \quad g_{uv}(b, a) > 0,$$

and write $J = |\mathcal{J}| \leq q^2 K$. For each tuple and $j \in \{0, 1\}$, use Lemma 91.6 with three inputs to enforce

$$z_{uvba}^j = e_{u,b} \wedge e_{v,a} \wedge c_{uv}^j.$$

Define the affine support expressions

$$\begin{aligned} A_v &= \sum_{(u,v,b,a) \in \mathcal{J}} g_{uv}(b, a) z_{uvba}^0, \\ B_v &= \sum_{(u,v,b,a) \in \mathcal{J}} g_{uv}(b, a) z_{uvba}^1. \end{aligned} \tag{91.13}$$

No support-value variable is needed. One-hot labels ensure that for each neighbor at most one compatibility term contributes.

For every $v \notin S$, introduce two natural slacks λ_v, μ_v , and impose

$$A_v = \tau \delta_v + \lambda_v, \tag{91.14}$$

$$B_v + \mu_v = (\tau - 1) \delta_v. \tag{91.15}$$

At an occupied site these are exactly the two threshold inequalities. At a blank site, all bond conjunctions targeting the site are zero, so both support expressions and both slacks are zero. There is no hidden choice in the inactive branch.

91.3.4 One polynomial, exact correspondence, exact counts

Let $L_1, \dots, L_\ell$ be the integer affine residuals of all the equations above, with all variables collected as w . Define

$$P_{\mathcal{T}, \sigma, D}(w) = \sum_{i=1}^{\ell} L_i(w)^2. \tag{91.16}$$

Theorem 91.7 (Assembly-faithful convex quadratic compiler). *The polynomial (91.16) is a globally nonnegative convex quadratic. Projection to the label coordinates is a bijection from its natural zero set to the producible labelled targets in D extending σ . The construction uses exactly*

$$m = N(q + 6) - 3s + 8K + 10J \tag{91.17}$$

$$\ell = 5N - s + 6K + 8J \tag{91.18}$$

natural variables and affine equations, respectively. For fixed palette, it has linear sparse description size in N , without an additional time-unrolling factor.

Proof. A real sum of squares vanishes exactly when every residual vanishes. At a natural zero, equations (91.8)–(91.9) decode a unique target. The rank equations impose the required bounds. The comparator and conjunction lemmas force all indicator meanings, so the support equations are exactly those in Theorem 91.3. Consequently the decoded target is producible and its rank assignment is the unique first-arrival assignment.

Conversely, a producible target supplies its one-hot labels and first-arrival ranks. The gadgets then give their unique values and slacks. The two activation slacks exist by the rank theorem and are also unique. All equations are satisfied. This proves both surjectivity and injectivity over label assignments.

The resource ledger is

Block	Variables	Equations
One-hot labels and seed	$N(q + 1)$	$N + s$
Ranks and rank slacks	$3N - s$	$2N$
Two comparisons per ordered edge	$8K$	$6K$
Two three-input conjunctions per compatibility	$10J$	$8J$
Two threshold slacks per nonseed site	$2(N - s)$	$2(N - s)$

Adding these rows proves (91.17)–(91.18). The row system has the form $Aw + b = 0$, and $\nabla^2 P = 2A^T A \succeq 0$, proving convexity.

Finally the total number of nonzero entries in the sparse affine system is $O(Nq + K + J)$: all gadget rows have bounded support and the support sums contain $O(J)$ terms in total. Since $K \leq 4N$ and $J \leq q^2 K$, this is $O(N)$ for fixed q . $\square$

Corollary 91.8 (Witness height and finite counting). *The natural zero set of (91.16) is finite, with at most $(q + 1)^{N-s}$ points. Every zero coordinate is $O(N + \tau + 1)$, so a full witness has $O(m \log(N + \tau + 2))$ bits. Counting natural zeros counts final labelled assemblies, not assembly orders.*

Proof. There are at most $(q + 1)^{N-s}$ label assignments and at most one zero above each. Ranks and their slacks are at most N ; comparator slacks are bounded by a constant multiple of $N + 1$; Boolean-gate slacks are at most two. Each realized support is at most 4τ , so activation slacks are bounded by 4τ . These cover every variable block. $\square$

Remark 91.9 (Sparse size versus expanded size). The stated sparse size refers to the affine rows and their sum-of-squares presentation. Squaring a long support row can create many cross terms; when the palette is part of the varying input, no linear-in- J bound on the fully expanded monomial list is asserted. With a fixed palette and bounded grid degree, each row has bounded support, and the expanded monomial list is linear in N as well.

91.4 Terminality in the whole infinite grid

Checking only unoccupied sites inside D is insufficient: a target might still grow outside the chosen domain. Checking only exterior sites is also insufficient: an interior hole may still be fillable.

Let the exterior halo be

$$H_D = \{v \notin D : \text{some } u \in D \text{ is adjacent to } v\}, \quad h = |H_D| \leq 4N.$$

Every site outside $D \cup H_D$ has no occupied neighbor and cannot receive a tile at positive temperature. Thus a target is terminal in the whole grid exactly when no tile type can attach to a blank site of D or to a site of H_D .

For each $(u, v, b, a) \in \mathcal{J}$, add a two-input conjunction

$$f_{uvba} = e_{v,0} \wedge e_{u,b}.$$

For each nonseed $v \in D$ and each real tile type a , define

$$W_{v,a} = \sum_{u \sim v, b: g_{uv}(b,a) > 0} g_{uv}(b,a) f_{uvba}$$

and impose, with a new natural slack $\eta_{v,a}$,

$$W_{v,a} + \eta_{v,a} = (\tau - 1)e_{v,0}. \tag{91.19}$$

For $v \in H_D$ use the direct affine expression

$$W_{v,a} = \sum_{u \in D, u \sim v} \sum_{b=1}^q g_{uv}(b, a) e_{u,b}$$

and impose

$$W_{v,a} + \eta_{v,a} = \tau - 1. \quad (91.20)$$

No halo label or rank variables are introduced.

Theorem 91.10 (Terminal assembly compiler). *Adding the terminality rows to (91.16) gives a globally nonnegative convex quadratic whose natural zeros are in bijection with the producible targets in D that are terminal in the whole grid. The exact additional resources are*

$$\Delta m = 4J + q(N - s + h), \quad \Delta \ell = 3J + q(N - s + h).$$

All added witnesses are uniquely determined by the target, and the witness-height bound of Corollary 91.8 is preserved.

Proof. At a blank interior site v , each f_{uvba} equals the neighbor-type indicator and $W_{v,a}$ is the attachment support for hypothetical tile type a . Equation (91.19) is solvable exactly when it is below τ . At an occupied site, all f 's and W 's are zero, and the equation forces its slack to zero. Seed sites need no vacancy test. On the halo, the corresponding statement is (91.20). These conditions are precisely whole-grid terminality by the preceding halo argument.

A two-input conjunction uses four variables and three equations; there are J of them. There is one final inequality slack and equation for each of the $q(N - s + h)$ site/type tests. This gives the ledger. All conjunctions and slacks are unique. Potential support at any site is at most 4τ , so the same height estimate applies. $\square$

91.5 Why the witnesses describe objects rather than histories

91.5.1 Feasible subassemblies form an antimatroid

Proposition 91.11 (Accessible union closure). *Fix a producible target α , and let $E = \text{dom } \alpha \setminus S$. Let $\mathcal{F} \subseteq 2^E$ consist of those X for which the restriction of α to $S \cup X$ is producible. Then $\emptyset \in \mathcal{F}$, $\mathcal{F}$ is union-closed, every nonempty feasible set has a feasible one-element deletion, and $\bigcup \mathcal{F} = E$. Thus $\mathcal{F}$ is an antimatroid.*

Proof. The seed gives the empty set, and producibility of α gives $E \in \mathcal{F}$. For accessibility, remove the last tile of a construction of a nonempty feasible set. For union closure, first build X , then follow a construction of Y , skipping sites already present. Every tile added in the second phase has at least the neighbors it had in the original construction of Y , with the same types. The two targets are compatible restrictions of α , and nonnegative support cannot decrease. Thus $X \cup Y$ is producible. $\square$

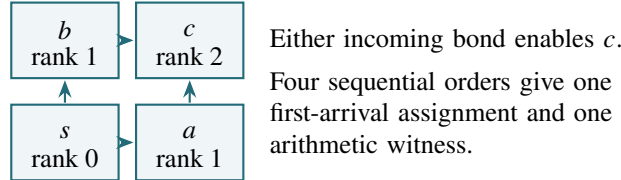
For a nonproducible target, the same statement holds after restricting the ground set to the sites that its closure actually reaches.

91.5.2 A two-by-two example that is not a fixed partial order

At temperature one, take one tile type with strength-one glue on every side. The seed is the lower-left site s of a two-by-two square; write a for the lower-right, b for the upper-left, and c for the upper-right site. The full target has exactly four legal orders:

$$abc, \quad acb, \quad bac, \quad bca.$$

Tile c needs either a or b , not a predetermined one. The orders cab, cba are illegal.



These four orders are not the linear extensions of any fixed poset on $\{a, b, c\}$. Every pair occurs in both possible orders among them, so no pairwise comparison could be required by such a poset. The poset would be an antichain and would allow all six permutations, a contradiction. The first-arrival ranks are nevertheless uniquely $t_a = t_b = 1, t_c = 2$. Among all eight possible occupied subsets of $\{a, b, c\}$, exactly seven are producible; the lone impossible subset is $\{c\}$.

91.5.3 A capped star and genuine nondeterminism

Place a seed at the center of a four-arm cross. Give the seed four orientation-specific strength-one glues. For each arm provide a tile with only its inward-facing matching glue; its other three sides are inert. At temperature one, all sixteen subsets of the arms are producible. The full cross is terminal in the entire grid and has $4! = 24$ legal assembly orders. The terminal polynomial has exactly one natural zero.

Now supply a second, distinctly labelled north-arm tile with the same glues. There are twenty-four producible labelled partial assemblies and two terminal full assemblies. Each full assembly still has twenty-four orders, but the terminal polynomial has exactly two zeros. This is intentional: the compiler does not quotient distinct physical tile labels merely because they have identical bond behavior. It is assembly-faithful, not unlabelled-shape-faithful.

91.5.4 Cooperation is retained

A temperature-two square can be arranged with a singleton lower-left seed, two strength-two attachments leading to a and b , and strength-one bonds from both a, b to c . All other exposed sides are inert. Then c requires *both* predecessors. The feasible nonseed subsets are

$$\emptyset, \quad \{a\}, \quad \{b\}, \quad \{a, b\}, \quad \{a, b, c\}.$$

The full square has two schedules and first-arrival ranks 1, 1, 2. The same arithmetic compiler handles this cooperative behavior without changing its logic or choosing a causal predecessor.

91.5.5 Negative interactions break the argument

The nonnegativity assumption cannot be suppressed. Consider adjacent seed sites at $(0, 0), (1, 0)$, with a stable positive bond between them. Potential tiles at $(0, 1), (1, 1)$ each receive a strength- τ bond from the seed site below. Give their mutual bond strength $-\tau$. Both potential tiles individually attach to the

seed, so both would receive rank one in a parallel enabling test. They satisfy the formal analogues of the two cut inequalities: each sees the seed at rank zero, while their mutual negative bond is excluded by equal ranks. But after either is added, the other's net support is zero; no sequential order builds both.

Thus signed interactions invalidate the batch-serialization step and union closure. Detachment systems and resource-sensitive chemical networks require a different semantics and a different certificate. Our theorem applies to nonnegative irreversible accretion, not to those models by analogy.

91.6 Unbounded computation and the fixed-arity obstruction

91.6.1 Marker production is computably enumerable

Let $n \mapsto \sigma_n$ be a computable family of finite seeds for a fixed finite tile palette and threshold. Fix a distinguished marker tile type, and let $A \subseteq \mathbb{N}$ consist of inputs for which some finite producible assembly contains the marker. Legal finite assembly sequences can be enumerated effectively. At each finite stage, only finitely many frontier sites and finitely many tile types need be tried, since $\tau > 0$. Checking a sequence uses exact finite integer sums. Therefore A is computably enumerable.

Theorem 91.12 (From tile events to quartic Diophantine equations). *For every such effective marker-production family, there is a fixed finite number of natural witnesses and a globally nonnegative quartic polynomial $U(n, y)$ with*

$$n \in A \iff \exists y \in \mathbb{N}^m U(n, y) = 0.$$

If A is nonsemilinear, no jointly orthant-nonnegative polynomial of degree at most three gives such a fixed-arity representation.

Proof. Apply Proposition 90.15 to the enumerated event set. The obstruction is Theorem 90.2. $\square$

In a marker-universal family, the marker event is arranged to coincide with halting of a universal computation on the seed input. Theorem 91.12 then applies to a nonsemilinear undecidable set. The availability of computation in seeded aTAM is established background [46, 47]; our theorem does not supply or optimize a new universal tile palette. The exact implication above is stated for any effective marker-production family, so it does not depend on an unstated choice of simulation layout or halting marker convention.

91.6.2 There cannot be a computable universal assembly-size bound

[write] The following proposition is the tile case of Proposition 62.2 (Part IX), which is stated for an arbitrary undecidable set; the proof is manuscript 09's.

Proposition 91.13 (No effective complete spatial cutoff). *If the marker-production event set A is undecidable, there is no total computable $B : \mathbb{N} \rightarrow \mathbb{N}$ such that whenever $n \in A$, some marker-containing assembly extending σ_n has at most $B(n)$ occupied sites.*

Proof. Suppose such a bound exists. Any newly added tile has a positive bond to some earlier tile, since the temperature is positive and all strengths are nonnegative. Iterating this predecessor relation reaches the seed. An assembly of at most $B(n)$ sites therefore lies within lattice distance $B(n)$ of the finite seed support. This gives an effectively computable finite region containing every relevant bounded witness.

Enumerate all labelled targets with at most $B(n)$ sites in that region, and test their producibility by parallel closure and whether they contain the marker. This finite search decides $n \in A$, contrary to undecidability. Equivalently, the local compiler gives exact finite integer certificates for the same bounded search, but cannot supply a computable bound that is complete for all positive inputs. $\square$

91.6.3 Why the two polynomial degrees do not conflict

For each D , the assembly polynomial is a finite convex quadratic. Its coefficients contain the constants N, τ , and its coordinate list contains one block per site and neighboring compatibility. It represents a finite set of labelled objects. Increasing D gives a *new* polynomial with more variables.

By contrast, a fixed-arity MRDP polynomial chooses its complete finite coordinate list before all inputs and all possible execution sizes. Theorem 90.2 says that a single orthant-nonnegative cubic with that quantifier order can never represent a nonsemilinear set. A family of increasingly large quadratic certificates does not contradict this. Compressing the family must change some feature: allow higher degree, allow sign changes, change the coefficient domain, or abandon the fixed-arity existential-polynomial format.

There are also two distinct notions of uniqueness. The local compiler has one full arithmetic witness for each fixed labelled target. One halting input may have many marker-containing targets, and a fixed-arity encoding of those targets may introduce many additional arithmetic witnesses. Neither multiplicity is eliminated by the first-arrival theorem.

91.7 Extensions and algorithmic consequences

91.7.1 Finite graph activation

The lattice geometry was used only to bound the number of neighbors and to compute the finite terminality halo. On any finite graph, let vertices have finitely many possible labels, a fixed seed set, nonnegative label-dependent edge weights, and a positive activation threshold. An occupied vertex may be added once, and its addition cannot diminish support for other vertices with fixed target labels. The parallel closure, the two-cut theorem, and the unique-slack compiler carry over verbatim.

On a graph of maximum degree Δ , replace $K \leq 4(N - s)$ by $K \leq \Delta(N - s)$, and support bounds 4τ by $\Delta\tau$. For fixed palette and fixed Δ , the compiler remains linear-size. Different fixed positive thresholds at vertices can be accommodated by using each vertex's threshold in its support equations. This gives the same object-level arithmetic semantics for a class of irreversible threshold-growth systems, not only square tiles.

The extension does not automatically cover arbitrary cellular automata: a general cellular automaton overwrites states and may disable future events. It does not automatically cover hierarchical assemblies that attach as multi-tile units either. Those are separate transition models.

91.7.2 A bounded-width counting corollary

Proposition 91.14 (An explicit width-dependent counting bound). *Suppose the domain-neighbor graph has maximum degree Δ and is supplied with a tree decomposition of width w having $O(N)$ bags. The number of producible labelled targets can be computed in*

$$N ((q + 1)(N + 1))^{(\Delta+1)(w+1)} \text{poly}(q, \Delta, \log(\tau + 2))$$

arithmetic operations, up to polynomial factors for bag manipulation. The stored counts have $O(N \log(q + 1))$ bits, so bit complexity incurs the corresponding polynomial arithmetic factor. This is a polynomial bound for fixed q, Δ, w , not a claim of fixed-parameter tractability in w .

Proof. Keep only each site's label and rank as a state. There are at most $L = (q + 1)(N + 1)$ states per site. The local conditions involve a site and its neighbors, not the auxiliary arithmetic coordinates, whose values are forced once these states are fixed.

Enlarge each bag by adding all neighbors of its vertices. Each enlarged bag has at most $(\Delta + 1)(w + 1)$ vertices and contains the full closed neighborhood of every original vertex in that bag. The enlarged bags still satisfy the running-intersection property. Indeed, the bags now containing a vertex v are the union of the original bag subtree for v and the original subtrees for its neighbors. Each neighbor's subtree intersects that of v , since an original bag covers their edge. This union is connected.

Assign each local constraint to a bag containing its scope. Standard bottom-up table combination over this decomposition counts satisfying label/rank assignments, with table size at most $L^{(\Delta+1)(w+1)}$. One-hot semantics, rank bounds, and the two support cuts are tested exactly. The rank theorem ensures that counting satisfying state assignments counts assemblies, without an extra factor for rank choices. Evaluating a support requires only bounded-degree integer arithmetic, giving the remaining polynomial factor. $\square$

For terminal counting, use the graph on $D \cup H_D$, fixing all halo sites blank, and include the local hypothetical-tile tests. The same argument applies with a decomposition of that graph. No claim is made that this elementary width bound is optimal.

91.7.3 Boolean observables and shape projections

A specified marker at a fixed site can be required by fixing its label indicator. A fixed finite Boolean circuit on site-label bits can be encoded with unique affine Boolean gates and slacks, preserving one arithmetic extension per assembly. This is useful for accepting a particular output pattern or requiring a prescribed set of sites to be occupied.

Projection from labelled assemblies to unlabelled supports can merge solutions. The two-north-tile star is already a counterexample to shape-faithfulness. A separate canonical labelling selection would be needed to obtain one witness per shape, and such a selection is not supplied by simply dropping label coordinates from the statement.

92 Reproduction, formalization, and further research [09]

[write] *Part III of manuscript 09.*

92.1 Executable compiler and exact finite checks

92.1.1 What the package implements

The standard-library Python module `code/09-order-free-assembly_compiler.py` constructs the literal affine rows of the base and terminal compilers. Rows are stored as sparse integer coefficient maps plus a constant. The polynomial is specified exactly as their sum of squares, so no symbolic expansion is required. The module also contains a canonical witness constructor, an arithmetic row checker, and a JSON exporter.

Two separate finite dynamics implementations are used: a parallel-closure routine computes arrival ranks, while an asynchronous search enumerates all reachable labelled states by legal single-tile additions. Their agreement is tested for every seed-extending labelling in four fixtures. The exported certificate checker evaluates the saved rows and witness; it does not call the compiler or reconstruct a dynamics trace.

The implementation checks basic palette, threshold, and seed-label validity. General multi-site seed stability is a mathematical precondition, not a property automatically proved by the prototype. All four executable fixtures use singleton seeds and meet it vacuously.

92.1.2 Results of the reproduced tests

Running `code/09-order-free-verify.py` produced the following exact counts:

Fixture	Labelings	Producible	Terminal	Full-target orders
Two-by-two diamond	8	7	0	4
Capped four-arm star	1296	16	1	24
Star with two north labels	2401	24	2	24
Cooperative square	125	5	1	2

The diamond has no whole-grid terminal assembly in this domain because its exposed strength-one glues permit growth beyond the domain. The capped fixtures correctly remove that possibility. All labelings counted in the second column extend the seed, including many impossible assignments of tile types to other sites.

The exact base and terminal resource counts are:

Fixture	K	J	Base m	Base ℓ	Terminal m	Terminal ℓ
Diamond	6	6	133	103	168	132
Capped star	4	8	164	112	256	196
Alternative star	4	10	189	128	301	230
Cooperative square	6	12	205	151	297	231

Here the compatibility count J includes every nonzero type pair on an eligible oriented edge, even if a particular pair cannot occur in a producible target. This explains why it can exceed the number of bonds in the illustrated full assembly.

Additional exact checks covered 568 comparator input cases, all 12 Boolean input cases for the two- and three-input conjunctions, and 3,882 rank assignments across all producible targets of the fixtures. Each producible target admitted precisely its canonical rank assignment. A separate exhaustive test of all 125 relevant rank assignments for all eight diamond targets also checked the rejected isolated tile. All 1,304 tested natural single-coordinate perturbations of the four exported witnesses were rejected by at least one arithmetic row.

For the positivity constructions, the test suite evaluated 512 natural grid points of $(x - y)^2(1 + z)$ and 476,656 assignments to the explicit quadratic for $2\mathbb{N} \cup (1 + 3\mathbb{N})$, confirming its projected behavior for inputs from zero to thirty. These checks are illustrations and implementation tests. The universal cubic classification rests on the proofs in Section 90, not on these finite experiments.

92.1.3 Reproduction commands and trust boundary

From the extracted package directory:

```
python code/verify.py
pdflatex -interaction=nonstopmode -halt-on-error order_free_diophantine.tex
pdflatex -interaction=nonstopmode -halt-on-error order_free_diophantine.tex
```

[write] *This command listing is the manuscript's own and keeps its delivered layout, in which the programs import one another by their delivered names; the shipped copies carry the prefix `09-order-free-` and must be copied back to that layout, outside the collection, before they are run. Where `python` does not resolve, use `py`. The *README* of this report gives rerun recipes that write only into the copy. The manuscript's own `article.tex` is not shipped; its text is this Part.*

The additional module `code/09-order-free-presburger_gadgets.py` implements the unbounded comparison and signed-divisibility penalties. The tests check uniqueness for 121 signed comparison operand pairs and 186 signed-operand, positive-modulus pairs. This is a gadget library, not an implementation of full Presburger quantifier elimination.

The test script writes `data/09-order-free-verification_report.json` and four literal certificate files (shipped as `data/09-order-free-*_certificate.json`). The JSON format stores the integer rows, variable names, input tile system, selected target, and witness. A row has semantics

$$\text{constant} + \sum_{(i,c) \in \text{terms}} c w_i = 0.$$

The prototype verifies these identities with exact Python integers. It is not an SMT proof, a Lean proof, or a proof that the source compiler is free of all defects. The mathematical proof establishes general soundness and completeness; the finite tests exercise its implementation and edge cases.

92.2 A concrete route into the repository’s formal development

No new Lean file in this package is labelled as verified. The following is an implementation plan with explicit proof obligations, not a claim that those obligations are already discharged.

92.2.1 The low-degree classification

The most self-contained starting point is the affine-zero lemma. A formal proof can avoid general multivariate Taylor analysis by expanding $p(a + th)$ as a univariate polynomial of degree at most three. One then proves the small-positive-and-negative sign lemmas, the double-root argument, and the positive-semidefinite kernel equivalence. Restricting variables to a coordinate face is a polynomial substitution.

The elementary natural-zero decomposition can use a finite choice of one natural zero per occupied support face. Formalizing the semilinearity of $Az = b$ requires the finite minimal-element theorem for $\mathbb{N}^r$ and the monoid decomposition in Lemma 90.6. The constructive quadratic (90.4) then reduces to elementary arithmetic and nonnegativity. For the stronger single-fold theorem, formalize the unbounded comparison and signed Euclidean-division penalties, then compose them with the existing Presburger quantifier eliminator and a unique Boolean-circuit evaluator. This gives the mathematical classification without first formalizing a real-algebraic sampling engine.

The effective strengthening is a separate development. It needs certified algebraic samples and a certified number-field basis conversion before feeding a Presburger formula to the existing decision procedure [35]. Treating sampling as an unproved oracle would not establish an unconditional executable decision theorem.

92.2.2 The assembly compiler

A finite graph formulation is preferable as the core abstraction. Define finite labels, support sums, monotone closure, sequential additions, and first-arrival ranks. Prove Lemma 91.2 and Theorem 91.3 before introducing any polynomial syntax. Then formalize each unique-slack gadget and establish that the generated finite list of affine equations is equivalent to the semantic rank conditions. Sum-of-squares reflection supplies the final polynomial.

The principal endpoint should be a bijection, not merely an existential soundness theorem. Its inverse is the canonical witness constructor. This permits exact counting theorems to be transported without another uniqueness proof. The terminal extension should explicitly quantify over the interior blanks and one exterior neighbor halo.

92.2.3 The MRDP bridge and its limits

The repository's `Diophantine.mrdp` already has the required fixed-finite-witness shape [34]. To use it for marker production, the remaining obligation is to prove the chosen encoded event predicate recursively enumerable. The bridge does not require claiming that this article's explicit spatial compiler itself has fixed arity.

A circuit-to-quartic normalization can be proved once as a general library construction on polynomials, independently of tile semantics. Its theorem should state unique extension only for gate variables given the original input and witness. That wording prevents an accidental strengthening to single-fold MRDP.

92.3 Further research questions and proposed projects

The following questions arise directly from the proved statements. They are proposed problems; their being listed here is not a claim that all are previously unstudied.

Research question 92.1 (Single-fold representations of semilinear sets; source 09). Theorem 90.14 settles existence of a single-fold orthant-nonnegative quadratic. What are the optimal witness-dimension and coefficient-height bounds in terms of a semilinear presentation or an already quantifier-free Presburger formula? How large an improvement is possible over one unique-evaluation block per atom and Boolean gate?

One alternative to deterministic formula evaluation is a disjoint canonical decomposition into linear pieces with uniquely representable generator coefficients. Comparing the dimension costs of these routes could yield sharper arithmetic certificates than the general construction.

Research question 92.2 (Effective cubic decomposition without full real quantifier elimination; source 09). Can the facewise affine zero spaces be recovered by a simpler exact algorithm using the special geometry of cubics, perhaps the semidefinite descriptions of [43], followed by rational linear algebra? Can one bound the bit complexity of the resulting Presburger presentation in useful restricted classes?

The cubic classification already proves decidability; the unresolved algorithmic target is a practical certified reduction. Exact feasibility and algebraic sampling must remain distinct from numerical semidefinite optimization.

Research question 92.3 (Sharp positivity hypotheses; source 09). How far can full real-orthant nonnegativity be weakened while preserving semilinearity of cubic natural zero sets? What happens under a finite collection of rational-polyhedral positivity promises, or positivity only on natural assignments, or positivity on nonlinear semialgebraic regions?

The finite-polyhedral version follows by applying Corollary 90.11 to each piece. More general promises need care: a proof that uses local minima must still justify a neighborhood of nonnegativity around every relevant point within its face.

Research question 92.4 (Fewer variables in the order-free compiler; source 09). Can the two comparisons per edge and the two three-input conjunctions per compatibility be shared or replaced to reduce the exact coefficients in (91.17), while preserving convexity and unique witnesses? Can a lower bound distinguish one-rank-per-site encodings from explicit round-by-round encodings?

The present ledger is explicit and validated, but not advertised as optimal. Any optimization should retain a local uniqueness theorem for every replacement gadget and should count all slacks.

Research question 92.5 (Canonical certificates for hierarchical self-assembly; source 09). Can a producible hierarchical assembly be assigned a canonical, polynomial-size arithmetic certificate that does not enumerate all possible binary merge trees? Is there a counterpart of earliest ranks whose local conditions enforce a unique hierarchy?

Doty’s greedy producibility results [48] are a natural starting point. The obstruction is that merges combine objects, not single sites, so the two-cut theorem is not immediately applicable.

Research question 92.6 (Negative glues and reversible growth; source 09). Which signed or reversible systems still admit an efficiently computable canonical witness of their final object? Can a restricted dependency condition restore serializability of parallel layers?

The two-tile signed example proves that unrestricted negative bonds do not permit a naive extension. A useful result would isolate exact semantic conditions under which batches remain serializable, rather than merely adding signed terms to the support polynomial.

[write] Compare Research question 71.4.

Research question 92.7 (Shape-faithful rather than label-faithful witnesses; source 09). For nondeterministic tile systems, can one choose a canonical producible labelling of each attainable shape using a polynomial-size direct certificate? What is the counting complexity of this quotient problem for natural restricted tile families?

First-arrival ranks remove schedule multiplicity, not label multiplicity. The alternative-star fixture provides a small regression test for any proposed quotient construction.

Research question 92.8 (Width dependence and arithmetic compression; source 09). Can the counting bound in Proposition 91.14 be made fixed-parameter tractable in treewidth, avoiding the factor $N^{O(w)}$ caused by ranks? Are only relative rank orders and a bounded amount of threshold information needed at a separator?

A compressed boundary state must preserve both support at the preceding round and non-support one round earlier. Losing that second distinction would count delayed rank assignments again.

[write] Compare Research question 71.11.

Research question 92.9 (Fixed-arity assembly-faithful encodings; source 09). Can finite labelled assemblies and their earliest ranks be encoded into a fixed tuple of integers, with explicit Diophantine decoding that retains a controlled number of arithmetic witnesses? Which parts can be made single-fold without assuming a general finite-fold MRDP result?

Theorem 90.2 excludes jointly orthant-nonnegative cubics for any nonsemilinear unbounded event. A successful construction must therefore respect this degree obstruction. The distinction between unique object coding and unique witnesses for arithmetic relations such as exponentiation must be tracked explicitly.

[write] Compare Research question 71.48.

Research question 92.10 (A minimal verified vertical slice; source 09). Can the repository first certify the entire diamond example—semantics, rank uniqueness, affine compiler, 133-variable quadratic, and exact seven-zero count—and then replace its finite domain by an abstract graph theorem without changing the interfaces?

This is a concrete integration target with a small enough independent reference model to expose off-by-one comparisons, blank-site ranks, seed conventions, and missing exterior terminality checks.

[write] Compare Research question 71.1.

92.4 Conclusion

There are two different ways in which an arithmetic certificate can be simple. It can be low-degree for each finite spatial instance, or it can have one fixed finite coordinate list for every execution size. The first does not imply the second.

For irreversible nonnegative self-assembly, earliest-arrival ranks make finite producibility order-free. Two neighboring rank cuts suffice, and unique-slack affine gadgets yield an explicit convex quadratic with one natural zero per labelled assembly. The same correspondence survives a careful whole-grid terminality test.

For fixed-arity representations subject to joint real-orthant nonnegativity, degree at most three has an exact arithmetic limitation: all projected natural zero sets are semilinear. Every semilinear set already has a single-fold quadratic representation, while degree four reaches all computably enumerable sets. This answers the precise degree question, explains why the local compiler cannot itself be a cubic universal compression, and provides two well-separated formalization targets for ProveIt: canonical finite objects on the one hand, and a sharp Presburger-versus-MRDP distinction on the other.

Part XII

Priority repetition: pumping bounds, program-uniform certificates and periodic tails

Source and scope. [write] This Part is manuscript 10, “Sharp Pumping Bounds and Canonical Diophantine Certificates for Priority Arithmetic Computation” (batch 62, manuscript 04; archive `ProveIt_Priority_Pumping_Diophantine.zip`, inner directory `priority_fractran_research`; shipped files prefixed `10-priority-pumping-`; labels `cdc:pf:`). It prints the manuscript’s Sections 2–14 in their own order: ordered arithmetic computation and the exact FRACTRAN embedding; the priority-sensitive interval checker (Theorem 95.2); least seeds and the exact repetition capacity (Theorem 96.4); the sharp pumping bound (Theorems 97.1 and 97.3); a canonical quartic uniform in the program entries (Theorem 98.6); variants and flat composition (Corollary 99.1); infinite periodic tails (Theorem 100.1); computably enumerable completeness of eventual periodicity (Theorem 101.1); affine macro certificates (Theorem 102.2); validation; the formalization roadmap; the ten research questions; and the conclusion. Its three appendices follow. Its title, abstract, status statement and introduction are in Section 3.10; the conventions of this Part are Section 93.

Relation to the other Parts. [write] Manuscript 10 inspected ProveIt at 4e128356d, before this report existed, and does not cite it. Its central object is Part III’s: a fixed word repeated k times, with k a parameter and single-fold witnesses. It adds what Part III’s guarded translations lack, first-enabled-rule priority, under which an earlier rule must be *disabled*, so that interior collision windows appear (Example 95.3). For a fixed program the block relation $\text{Block}_{\mathcal{P},w}(x, k, y)$ is an instance of Part IV’s run relation, and a single-fold quartic for it already follows from Theorem 31.2; the priority-free fixed-word power is Proposition 22.2. The new content is uniformity in the program entries (they are free inputs of one polynomial), the exact capacity formula, the sharp threshold $2H + 1$, polynomial witness height, and the theory of infinite tails, eventual periodicity and affine macros. Part VII’s FRACTRAN certificate (Theorem 54.2) certifies a bounded raw-integer run step by step; this Part accelerates a prescribed repeated word on exponent vectors, with size independent of the repetition count, and leaves the raw-integer interface open (its question on direct raw-integer interfaces). The flat-schema corollary partly answers the question on restricted unbounded classes (Research question 71.49), and

fixed-word acceleration with a single-fold certificate partly answers the question on substrate interfaces (Research question 71.3); see the notes there. The results on eventual periodicity and affine macros are nontermination certificates, the Σ_1^0 fragment of the recurrence and liveness hierarchy of the neighbouring report *liveness-beyond-halting*; they stay here with the pumping theory they use.

Setting and hypotheses. [write] An ordered multiset program on $\mathbb{N}^d$ fires its first enabled rule; reduced FRACTRAN embeds by prime valuations. The instruction word w and the dimensions m, d are compiler data, while the program entries, the seed, the repetition count and the endpoint are free inputs of one polynomial. Neither universality nor a universal single-fold theorem is claimed.

93 Conventions of Part XII

[write] The letters are manuscript 10's; none is renamed. Its script letter $\mathcal{P}_{m,d,w}$ (the polynomial) and calligraphic $\mathcal{P}$ (the program) are kept apart as in the manuscript.

Symbol	Meaning in Part XII, and the clash
A, B	production and consumption matrices in $\mathbb{N}^{m \times d}$, with rows A_i, B_i . In Part VII, A and B are the largest numerator and denominator of a FRACTRAN program.
H	the largest consumption exponent $\max_{i,p} B_{ip}$, the parameter of the pumping threshold $2H + 1$. It is not Part II's causal height, Part VI's queue bound or a witness height.
w, ℓ, j_r, k	the instruction word, its length, its one-based letters, and the repetition count. Part III writes n for a multiplicity and t for a word length.
$D, \beta_r, \lambda, \lambda(k)$	drift, prefix displacement, least one-copy seed and least k -copy seed. Part III's word power writes Δ for the displacement and $\ell_{w,j}$ for the lower corner of the same kind of box; D is used in other senses in Parts I, II, V and VI.
L_{ri}, R_{ri}	left and right waiting margins of an earlier rule; not Part III's loss totals L_j .
R, S	$R = \sum_r j_r$ and $S = dR$, the numbers of phase-rule rows and coordinate interval computations; not residual counts. The residual count of this Part is n_f , the auxiliary count n_z .
$P_{r,p}, Q_{r,p}$	production and consumption prefix sums; P, Q their totals.
$\text{Cap}(w), M_w(x), \text{EP}$	repetition capacity, maximal number of complete copies from x , and the eventually periodic instances.

94 Ordered arithmetic computation [10]

94.1 States, rules, and operational semantics

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$ and vector inequalities are coordinatewise. All polynomial unknowns are natural numbers; polynomial coefficients may be negative integers. We assume $m, d \geq 1$.

Definition 94.1 (Ordered multiset program). An ordered multiset program is a list $\mathcal{P} = ((A_1, B_1), \dots, (A_m, B_m))$ with $A_i, B_i \in \mathbb{N}^d$. At state $x \in \mathbb{N}^d$, rule i is enabled when $x \geq B_i$. If an enabled rule exists, let j be the smallest enabled index and perform

$$x \xrightarrow{j}_{\mathcal{P}} x - B_j + A_j. \quad (94.1)$$

Otherwise the program halts. The instruction trace records the chosen indices.

The term “ordered” is essential. A later enabled rule is not a legal choice when an earlier rule is also enabled. The model can be viewed as multiset rewriting with a deterministic priority policy: B_i is consumed and A_i is produced. No stochastic or parallel scheduling semantics are being imposed. In particular, the results below do not silently extend to arbitrary chemical-reaction semantics.

94.2 The exact FRACTRAN embedding

Let $(a_i/b_i)_{i=1}^m$ be positive fractions in lowest terms. Let $p_1, \dots, p_d$ contain every prime dividing any numerator or denominator, and set

$$A_{ip} = v_{p_p}(a_i), \quad B_{ip} = v_{p_p}(b_i). \quad (94.2)$$

Every positive integer has a unique decomposition

$$N = c \prod_{p=1}^d p_p^{x_p}, \quad \gcd\left(c, \prod_{p=1}^d p_p\right) = 1.$$

Because the fractions are reduced, $a_i N / b_i$ is an integer exactly when $x \geq B_i$. Its valuation vector is $x - B_i + A_i$, and the coprime factor c stays unchanged. Thus (94.1) is an exact conjugate of FRACTRAN on each fixed inert-factor component.

Remark 94.2 (Reduction is not optional). In the FRACTRAN subcase, $A_{ip}B_{ip} = 0$ for every i, p . For a general multiset rule, a coordinate may be both consumed and produced. One must not replace such a rule by the fraction $\prod p_p^{A_{ip}} / \prod p_p^{B_{ip}}$ and cancel it: cancellation can remove a genuine enabling condition. The general multiset model and reduced FRACTRAN are related but not identical syntactic classes.

94.3 Why priority permits universal computation

For orientation, the conventional counter-machine simulation can be made explicit. Give counter r a prime p_r and control state s a distinct prime q_s . A well-formed machine configuration is encoded by $q_s \prod_r p_r^{x_r}$, with exactly one control prime present. An increment transition $s \rightarrow t$ on counter r is represented by $p_r q_t / q_s$. A positive decrement transition uses $q_t / (q_s p_r)$; place it before the default zero branch q_u / q_s . All rules belonging to another control state are disabled. A halting state has no outgoing fraction. Direct self-targets can first be split through fresh control states, so cancellation does not destroy the control test. This preserves the selected branch and produces a finite, effective simulation. The universality conclusion is classical [30, 14]; we use it later only for a computability reduction.

94.4 The compression interface

Fix a nonempty word

$$w = (j_0, \dots, j_{\ell-1}) \in \{1, \dots, m\}^\ell.$$

Let $\text{Block}_{\varphi, w}(x, k, y)$ mean that the actual deterministic execution from x begins with w^k and reaches y immediately after those $k\ell$ steps. Here $k = 0$ is allowed, in which case the relation is exactly $y = x$. No halting condition at y is implicit.

The word is part of the compiled schema, not an unbounded input hidden in a fixed polynomial. The program entries, x, k, y , will all be free inputs of one polynomial for that schema. This distinction prevents a false inference from the later unique-witness theorem to a universal single-fold theorem.

95 Exact finite-block acceleration [10]

95.1 Affine phase trajectories

Put

$$d_i = A_i - B_i, \quad \beta_r = \sum_{s < r} d_{j_s}, \quad D = \sum_{s < \ell} d_{j_s}. \quad (95.1)$$

Before phase r of the h th copy, the only possible state consistent with the prescribed word is

$$z_{h,r} = x + \beta_r + hD. \quad (95.2)$$

These are hypothetical integer vectors until validity has been proved. It would be circular to assume in advance that every such vector is a legal state.

Lemma 95.1 (Exact local criterion). *For $k \in \mathbb{N}$, the relation $\text{Block}_{\mathcal{P},w}(x, k, y)$ holds if and only if $y = x + kD$ and, for every $0 \leq h < k$ and $0 \leq r < \ell$,*

$$x + \beta_r + hD \geq B_{j_r}, \quad (95.3)$$

$$x + \beta_r + hD \not\geq B_i \quad (1 \leq i < j_r). \quad (95.4)$$

Proof. Necessity follows by summing the updates of the actual run. Conversely, induct on the lexicographically ordered pairs (h, r) . The initial state is x . At the next prescribed phase, (95.3) makes j_r enabled and (95.4) disables every earlier rule. Hence the deterministic semantics chooses j_r , and the resulting state is the next vector in (95.2). It is nonnegative because it is obtained from a state at least B_{j_r} by subtracting B_{j_r} and adding A_{j_r} . The last state is $x + kD$. For $k = 0$ the phase conditions are empty and the same conclusion reduces to $y = x$. $\square$

95.2 A single coordinate contributes an interval

For $a, s \in \mathbb{Z}$ and $k \in \mathbb{N}$, define

$$I(a, s, k) = \{h \in \mathbb{Z} : 0 \leq h < k, a + sh \geq 0\}.$$

We represent this as a half-open integer interval $[L, U)$; $L \geq U$ represents the empty set. The following formulas choose canonical endpoints:

condition	L	U
$s > 0$	$\min\{k, \max\{0, \lceil -a/s \rceil\}\}$	k
$s < 0, a \geq 0$	0	$\min\{k, \lfloor a/(-s) \rfloor + 1\}$
$s < 0, a < 0$	0	0
$s = 0, a \geq 0$	0	k
$s = 0, a < 0$	0	0

(95.5)

Each row follows by solving a linear inequality and clipping to $[0, k)$. In particular, all returned endpoints are natural numbers at most k .

At phase r , rule i is enabled on the intersection of the d intervals

$$I(x_p + \beta_{r,p} - B_{ip}, D_p, k), \quad 1 \leq p \leq d. \quad (95.6)$$

Its lower endpoint is the maximum of the coordinate lower endpoints; its upper endpoint is the minimum of their upper endpoints.

Theorem 95.2 (Priority-sensitive interval checker). *The relation $\text{Block}_{\mathcal{P},w}(x, k, y)$ can be decided without unrolling k copies. At every phase, require the chosen rule's interval to be $[0, k)$ and every earlier rule's interval to be empty, and finally check $y = x + kD$. The number of coordinate interval calculations is S from (3.10.1).*

Proof. A rule is enabled exactly when all its coordinate inequalities hold, so its enabling times are precisely the intersection (95.6). An earlier rule is disabled at every prescribed time exactly when the intersection is empty, equivalently $L \geq U$. Since each coordinate endpoint lies in $[0, k]$, the chosen rule is enabled throughout exactly when $L = 0$ and $U = k$. Apply Lemma 95.1. Phase r checks the j_r rules with index at most j_r , giving $d \sum_r j_r = S$ calculations. $\square$

Example 95.3 (An interior-only priority collision). Consider

$$\mathcal{F} = (1/72, 3/2), \quad N_0 = 32, \quad w = (2).$$

In the $(2, 3)$ valuation coordinates, $x = (5, 0)$, $D = (-1, 1)$, and the earlier rule consumes $(3, 2)$. A proposed five-copy block has pre-states

$$(5, 0), (4, 1), (3, 2), (2, 3), (1, 4).$$

The second rule is enabled at all five pre-states. The first rule is disabled at both ends but enabled at the middle state $(3, 2)$. Its enabling interval is exactly $[2, 3)$. Consequently the proposed block is invalid; the actual raw-integer run starts $32 \rightarrow 48 \rightarrow 72 \rightarrow 1$, with labels 2, 2, 1. An endpoint-only priority test accepts a false computation here.

95.3 Maximal repetition from a given seed

Let $M_w(x)$ be the largest number of complete copies of w executable from x , or ∞ if all numbers of copies are executable. Block validity is prefix-closed in k . Thus, once infinitude has been excluded by Theorem 100.1 below, doubling followed by binary search computes $M_w(x)$ using the interval checker. No intermediate trace of length $M_w(x)\ell$ has to be generated.

The phrase “complete copies” is important. If $M_w(x) = k$, the next copy may execute some proper prefix before it fails. The certificate is not claiming that the very next instruction already differs.

96 Least seeds and the exact repetition capacity [10]

The interval theorem handles a supplied seed. A stronger structural result emerges when the seed is existentially quantified. There is no search over all starting vectors: one canonical vector suffices.

96.1 The resource-minimal seed

Define

$$\lambda_p = \max_{0 \leq r < \ell} (B_{j_r, p} - \beta_{r, p}), \quad D_p^- = \max\{0, -D_p\}. \quad (96.1)$$

The term $r = 0$ is $B_{j_0, p} \geq 0$, so $\lambda \in \mathbb{N}^d$. For $k \geq 1$ put

$$\lambda(k) = \lambda + (k - 1)D_p^-. \quad (96.2)$$

Lemma 96.1 (Resource lower bound). *All selected-rule inequalities (95.3) hold for w^k if and only if $x \geq \lambda(k)$.*

Proof. For a fixed coordinate, all the selected inequalities are equivalent to

$$x_p \geq \max_{r, h} (B_{j_r, p} - \beta_{r, p} - hD_p).$$

The two indices vary independently. The maximum over r is λ_p ; the maximum of $-hD_p$ over $0 \leq h < k$ is $(k - 1)D_p^-$. This is exactly (96.2). $\square$

Lemma 96.2 (Lowering cannot create a priority conflict). *Suppose x executes w^k and $\lambda(k) \leq x' \leq x$. Then x' executes w^k . In particular,*

$$\exists x \in \mathbb{N}^d \exists y \text{ Block}_{\mathcal{P},w}(x, k, y) \iff \lambda(k) \text{ executes } w^k. \quad (96.3)$$

Whenever the set of valid seeds is nonempty, $\lambda(k)$ is its unique least element.

Proof. The resource lower bound preserves all selected-rule inequalities at x' . At any fixed phase and repetition index, an earlier rule's enabling predicate is upward closed in the seed. If that rule were enabled at x' , it would also be enabled at x , a contradiction. Thus all prior rules remain disabled and Lemma 95.1 applies. Every valid seed is at least $\lambda(k)$ by Lemma 96.1, proving the last assertions. $\square$

This monotonicity is deliberately restricted. The entire deterministic transition system is not monotone in the sense that raising a seed preserves its selected trace. Raising a seed can enable an earlier instruction and change the computation. The lemma uses lowering within a resource-safe box.

96.2 The collision window at the least seed

For every phase r and earlier rule $i < j_r$, set

$$c_{ri,p} = \lambda_p + \beta_{r,p} - B_{ip}. \quad (96.4)$$

Call (r, i) *permanently blocked* if some coordinate satisfies

$$D_p = 0 \quad \text{and} \quad c_{ri,p} < 0. \quad (96.5)$$

For a pair that is not permanently blocked, define two nonnegative integers

$$L_{ri} = \max \left(\{0\} \cup \left\{ \left\lceil \frac{(-c_{ri,p})_+}{D_p} \right\rceil : D_p > 0 \right\} \right), \quad (96.6)$$

$$R_{ri} = \max \left(\{0\} \cup \left\{ \left\lceil \frac{(-c_{ri,p})_+}{-D_p} \right\rceil : D_p < 0 \right\} \right). \quad (96.7)$$

Here $(u)_+ = \max\{u, 0\}$, and an empty family contributes only the displayed zero. The use of R_{ri} as a right-margin quantity is distinct from the aggregate count R in (3.10.1).

Lemma 96.3 (Two-sided collision window). *At seed $\lambda(k)$, a non-permanently-blocked earlier rule i is enabled before phase r precisely at the integers*

$$L_{ri} \leq h \leq k - 1 - R_{ri}. \quad (96.8)$$

A permanently blocked pair is never enabled there. Consequently the first repetition count for which this pair creates a conflict is $L_{ri} + R_{ri} + 1$.

Proof. At the least seed, its coordinate enabling inequality is

$$c_{ri,p} + (k - 1)D_p^- + hD_p \geq 0.$$

If $D_p > 0$, this says $h \geq \lceil (-c_{ri,p})_+ / D_p \rceil$, once $h \geq 0$ is included. If $D_p < 0$, it says $k - 1 - h \geq \lceil (-c_{ri,p})_+ / (-D_p) \rceil$. If $D_p = 0$, it is either identically true or identically false; the latter is exactly a permanent blocker. Taking the strongest left and right bounds gives (96.8). This interval contains an integer exactly when $k \geq L_{ri} + R_{ri} + 1$. $\square$

Theorem 96.4 (Closed repetition-capacity formula). *Define*

$$\text{Cap}(w) = \min_{\substack{0 \leq r < \ell, \ 1 \leq i < j_r \\ (r,i) \text{ not permanently blocked}}} (L_{ri} + R_{ri}), \quad \min \emptyset = \infty. \quad (96.9)$$

Then for every $k \geq 1$,

$$\exists x, y \in \mathbb{N}^d \text{ Block}_{\mathcal{P},w}(x, k, y) \iff k \leq \text{Cap}(w). \quad (96.10)$$

The capacity and the least valid seed for any permitted k are computable by the formulas above, using $O(d\ell + d \sum_r (j_r - 1))$ elementary integer operations, including divisions.

Proof. At seed $\lambda(k)$, every selected-rule inequality already holds. By Lemma 96.3, a non-permanently-blocked earlier rule remains inactive for the whole block exactly when $k \leq L_{ri} + R_{ri}$. All permanently blocked pairs impose no further restriction. Intersect these conditions and use Lemma 96.2. The computation forms prefixes and coordinatewise maxima, then processes one d -tuple for every earlier-rule pair. The stated operation bound follows. Integer bit costs depend on the sizes of the input entries and the word, not on an enumeration of k steps. $\square$

Corollary 96.5 (Structure of the seed region). *For each $k \geq 1$, the valid seeds form either the empty set or a set with least element $\lambda(k)$ that is downward closed relative to the orthant $\lambda(k) + \mathbb{N}^d$. For fixed program and word, this set is Presburger-definable.*

Proof. The order assertion is Lemma 96.2. For fixed program and word, Lemma 95.1 is a Presburger formula: the coefficients D_p and $\beta_{r,p}$ are constants, and the quantifier over h is a bounded first-order quantifier over a linear formula. Fixing k gives the claimed seed region. This observation does not require quantifier elimination to compute the least seed or capacity. $\square$

97 A sharp pumping bound [10]

Theorem 97.1 (Threshold $2H + 1$). *Let $H = \max_{i,p} B_{ip}$. For every nonempty instruction word w ,*

$$\text{Cap}(w) < \infty \implies \text{Cap}(w) \leq 2H. \quad (97.1)$$

Equivalently, if some seed executes w^{2H+1} , then for every $k \in \mathbb{N}$ there is a seed executing w^k . The threshold is independent of the word length, the number of rules, the number of coordinates, and the sizes of the production entries.

Proof. By definition of λ , for every phase and coordinate, $\lambda_p + \beta_{r,p} \geq B_{jr,p} \geq 0$. Thus

$$(-c_{ri,p})_+ = \max\{B_{ip} - \lambda_p - \beta_{r,p}, 0\} \leq B_{ip} \leq H.$$

Every nonzero $|D_p|$ is a positive integer. Therefore every ceiling in (96.6) or (96.7) is at most H , whence $L_{ri} + R_{ri} \leq 2H$. If the capacity is finite, it is a minimum of such numbers by Theorem 96.4. The equivalent pumping statement follows immediately. For $k = 0$, every seed works. $\square$

Remark 97.2 (What the parameter measures). For FRACTRAN, H is the largest prime exponent in a reduced denominator, not the largest denominator itself, its numerical value, or its bit length. For a squarefree-denominator program, $H = 1$, so any word executable three times from some seed is executable arbitrarily many times from suitable seeds. If $H = 0$, the same theorem allows only capacities 0 and ∞ .

Theorem 97.3 (Sharpness within reduced FRACTRAN). *For every $H \geq 1$, the reduced three-fraction program*

$$\mathcal{F}_H = \left(\frac{1}{7 \cdot 6^H}, \frac{7}{10}, \frac{15}{7} \right) \quad (97.2)$$

and word $w = (2, 3)$ have $\text{Cap}(w) = 2H$. Hence $2H + 1$ is the smallest universal pumping threshold as a function of H alone.

Proof. Use coordinates for primes 2, 3, 5, 7. The two selected rules transport one unit from the 2-counter to the 3-counter, using 5 and 7 as alternating control tokens. The one-copy least seed and drift are

$$\lambda = (1, 0, 1, 0), \quad D = (-1, 1, 0, 0).$$

Thus the least seed for k copies is $(k, 0, 1, 0)$, corresponding to the raw integer $5 \cdot 2^k$. Before the second fraction in copy h the state is

$$(k - h, h, 1, 0),$$

and before the third it is

$$(k - 1 - h, h, 0, 1).$$

The first fraction is disabled at the former state by the missing 7. At the latter it is enabled exactly when

$$h \geq H \quad \text{and} \quad k - 1 - h \geq H.$$

The earlier second fraction is disabled at the latter state by the missing 5. All selected rules themselves are enabled for $0 \leq h < k$. Consequently the prescribed word works at the least seed precisely when $k \leq 2H$. Lemma 96.2 excludes rescuing a larger k by choosing a different seed. The largest denominator exponent in (97.2) is indeed H . $\square$

97.1 The quantifier distinction

The pumping theorem proves

$$\forall k \exists x \quad w^k \text{ is executable from } x, \quad (97.3)$$

not necessarily

$$\exists x \forall k \quad w^k \text{ is executable from } x. \quad (97.4)$$

For the one-fraction program $(3/2)$ and word (1) , the first statement holds by choosing a seed divisible by 2^k . Every fixed seed has only finitely many factors of 2, so the second statement is false. This example is also why the least seed grows with k in negative-drift coordinates. Section 100 gives the exact condition under which the quantifiers can be exchanged.

97.2 A refined, program-dependent bound

The proof also gives more detailed bounds. For any nonblocked pair, retain its actual coordinate deficits and drift magnitudes in (96.6)–(96.7) instead of replacing them by H . Large drift magnitudes reduce the waiting margins, and one-sided drift can remove one margin altogether. For example, if $D \geq 0$, every $R_{ri} = 0$ and a finite capacity is at most H . These refinements follow directly from the exact formula; they need not be approximated by a universal threshold in an implementation.

98 A canonical quartic Diophantine compiler [10]

This section replaces the interval algorithm by an explicit polynomial construction. The central issue is not merely expressing the required inequalities. It is doing so with a *unique* tuple of auxiliary values, including values associated with cases that do not apply.

98.1 Single-fold representations and total gates

Definition 98.1 (Single-fold relative to the free inputs). A polynomial $P(u, z) \in \mathbb{Z}[u, z]$ is a single-fold Diophantine representation of a relation $\mathcal{R}(u)$ over $\mathbb{N}$ when

$$\mathcal{R}(u) \iff \exists z \in \mathbb{N}^n P(u, z) = 0,$$

and for every u there is at most one such z .

The ambient domain matters. Replacing natural auxiliary variables by four squares to quantify over all integers can introduce many choices and does not preserve this uniqueness assertion. We work directly over $\mathbb{N}$.

Lemma 98.2 (Canonical comparison gate). For $u, v \in \mathbb{N}$, the equations

$$b(b-1) = 0, \quad b(v-u-1-s) = 0, \quad (1-b)(u-v-s) = 0 \quad (98.1)$$

have a unique solution $(b, s) \in \mathbb{N}^2$. It is

$$(b, s) = \begin{cases} (1, v-u-1), & u < v, \\ (0, u-v), & u \geq v. \end{cases}$$

In particular, $b = \llbracket u < v \rrbracket$.

Proof. The first equation forces $b \in \{0, 1\}$. If $b = 1$, the second equation forces $s = v - u - 1$, which is natural exactly when $u < v$. If $b = 0$, the third forces $s = u - v$, which is natural exactly when $u \geq v$. These alternatives partition all input pairs and determine the slack uniquely, including the tie case. $\square$

Lemma 98.3 (Canonical division gate). For $a \in \mathbb{N}$ and $d \geq 1$, the equations

$$a - dq - r = 0, \quad d - r - 1 - t = 0 \quad (98.2)$$

have a unique solution $(q, r, t) \in \mathbb{N}^3$. Here $q = \lfloor a/d \rfloor$, r is the Euclidean remainder, and $t = d - r - 1$.

Proof. The second equation is equivalent to $0 \leq r < d$ together with the uniquely determined slack t . The first is Euclidean division, whose quotient and remainder are unique under that bound. $\square$

A straight-line assignment $z = E$ contributes one auxiliary and one residual $z - E$. We use it only when E has degree at most two in already available variables and is nonnegative on the uniquely determined earlier values. Comparison followed by one assignment gives

$$\min(u, v) = v - b(v - u), \quad \max(u, v) = u + b(v - u), \quad b = \llbracket u < v \rrbracket. \quad (98.3)$$

Thus each min or max uses three auxiliaries and four residuals. We keep separate comparison gates in the stated count even when sharing them could save variables.

98.2 Removing signed data without hiding arithmetic

The mathematical interval formulas use signed drifts and intercepts. For the natural-number circuit, form separate nonnegative prefix sums

$$P_{r,p} = \sum_{s < r} A_{js,p}, \quad Q_{r,p} = \sum_{s < r} B_{js,p}. \quad (98.4)$$

Then $\beta_r = P_r - Q_r$. Write $P = P_\ell$ and $Q = Q_\ell$. For each coordinate, compute

$$e_p^- = \llbracket P_p < Q_p \rrbracket, \quad e_p^+ = \llbracket Q_p < P_p \rrbracket, \quad e_p^0 = 1 - e_p^- - e_p^+.$$

The comparison of P_p, Q_p supplies a slack s_p . Define

$$D_p^+ = (1 - e_p^-)s_p, \quad D_p^- = e_p^-(s_p + 1), \quad (98.5)$$

$$d_p^+ = D_p^+ + 1 - e_p^+, \quad d_p^- = D_p^- + 1 - e_p^-. \quad (98.6)$$

The first two values are $\max(D_p, 0)$ and $\max(-D_p, 0)$. Both denominators in (98.6) are at least one: on the relevant nonzero-sign branch they equal the true drift magnitude, and otherwise they equal one. Also materialize $g_p = e_p^+k$ and $f_p = e_p^0k$.

The safe denominators are not a cosmetic detail. If a division gate were introduced only under a conditional multiplier, its inactive branch could leave quotient or remainder variables unconstrained. Here all gates are always evaluated and all their outputs are uniquely fixed.

98.3 A natural-number interval circuit

For one phase-rule-coordinate triple, set

$$u = x_p + P_{r,p}, \quad v = B_{i,p} + Q_{r,p}. \quad (98.7)$$

Thus $u - v = x_p + \beta_{r,p} - B_{i,p}$. Apply the comparison gate to u, v , obtaining $b = \llbracket u < v \rrbracket$ and slack s . In the following display, every assignment to a named output is materialized as a gate; min and division have the implementations just given:

$$a^- = b(s + 1), \quad a^+ = (1 - b)s, \quad (98.8)$$

$$n^+ = a^- + d_p^+ - 1, \quad q^+ = \lfloor n^+ / d_p^+ \rfloor,$$

$$\ell_p = e_p^+ \min(k, q^+), \quad q^- = \lfloor a^+ / d_p^- \rfloor,$$

$$t_p = (1 - b) \min(k, q^- + 1), \quad u_p^- = e_p^- t_p,$$

$$u_p^0 = f_p(1 - b), \quad u_p = g_p + u_p^- + u_p^0. \quad (98.9)$$

No product of three auxiliary variables is used: the intermediate outputs $t_p, u_p^-, u_p^0, g_p, f_p$ are materialized separately.

Lemma 98.4 (Correctness and totality of the interval circuit). *The circuit (98.8)–(98.9) has unique natural outputs for every natural input tuple. Its pair (ℓ_p, u_p) is exactly the canonical interval in (95.5) with $a = u - v$, $s = D_p$.*

Proof. The comparison gives $a^- = \max(v - u, 0)$ and $a^+ = \max(u - v, 0)$. All dividend and denominator values are natural, and both denominators are positive. All remaining products and sums are nonnegative.

When $D_p > 0$, $d_p^+ = D_p$ and $q^+ = \lceil a^- / D_p \rceil$. The upper endpoint is k because $g_p = k$ and the other sign contributions vanish. This is the first row of (95.5). When $D_p < 0$, the lower endpoint is zero, $d_p^- = -D_p$, and the upper endpoint is $\min(k, \lfloor a^+ / (-D_p) \rfloor + 1)$ if $u \geq v$, and zero otherwise. When $D_p = 0$, only $u_p^0 = k(1 - b)$ survives. These are exactly the remaining rows. Uniqueness follows successively from the comparison, assignments, and division lemmas. $\square$

98.4 From coordinate intervals to one Boolean

For each phase and each $i \leq j_r$, initialize $L = 0, U = k$ and fold

$$L \leftarrow \max(L, \ell_p), \quad U \leftarrow \min(U, u_p)$$

over the coordinates. For an earlier rule use the Boolean $1 - \llbracket L < U \rrbracket$. For the selected rule use

$$\llbracket L < 1 \rrbracket (1 - \llbracket U < k \rrbracket). \quad (98.10)$$

Since $L, U \in [0, k]$, this selected flag is one exactly when $L = 0$ and $U = k$. Multiply the row flags in a materialized chain, starting at one, to produce an acceptance bit a_w . This product is a conjunction of already canonical Boolean values; it does not choose which failed guard to witness.

Proposition 98.5 (Canonical accelerated decision circuit). *For every fixed m, d, w , the preceding circuit has exactly one natural auxiliary evaluation on every natural input (A, B, x, k) . Its output satisfies*

$$a_w = 1 \iff \exists y \text{ Block}_{\mathcal{P},w}(x, k, y).$$

All circuit residuals have total degree at most two, even when the entries of A and B are treated as variables rather than constants.

Proof. The prefix sums and nonnegative input sums have canonical values. The slope and interval constructions are total by the preceding lemmas. Min, max, comparison, and product assignments are also canonical. Induction over the finite gate order gives totality and uniqueness. The acceptance conditions are those of Theorem 95.2. Every residual is an assignment of a linear or quadratic expression, one of (98.1), or one of (98.2); the materialization of intermediate products ensures that no residual exceeds degree two. $\square$

98.5 The polynomial and its proof

Adjoin the obligation $a_w - 1 = 0$. For each coordinate, materialize $q_p^{\text{out}} = kQ_p$ and $p_p^{\text{out}} = kP_p$, and add

$$x_p + p_p^{\text{out}} - y_p - q_p^{\text{out}} = 0. \quad (98.11)$$

Let $F_1, \dots, F_{n_f}$ be all gate residuals and obligations, and set

$$\mathcal{P}_{m,d,w}(A, B, x, k, y, z) = \sum_{a=1}^{n_f} F_a(A, B, x, k, y, z)^2. \quad (98.12)$$

Equation (98.12), together with the displayed gate rules and their finite ordering, is an explicit polynomial construction. Expanding its monomials is optional and gives the same integer polynomial. The supplied JSON represents the residuals exactly, so there is no unimplemented MRDP oracle inside this definition.

Theorem 98.6 (Uniform canonical quartic representation). *For fixed $m, d \geq 1$ and nonempty $w \in \{1, \dots, m\}^\ell$, the polynomial (98.12) has integer coefficients and degree at most four. For every $A, B \in \mathbb{N}^{m \times d}$, $x, y \in \mathbb{N}^d$, and $k \in \mathbb{N}$,*

$$\text{Block}_{\mathcal{P},w}(x, k, y) \iff \exists !z \in \mathbb{N}^{n_z} \mathcal{P}_{m,d,w}(A, B, x, k, y, z) = 0. \quad (98.13)$$

When the block relation is false, the polynomial has no natural auxiliary zero. The construction has the exact counts (3.10.2) and (3.10.3), with $2md + 2d + 1$ free input coordinates.

Proof. A sum of squares of integers is zero if and only if each summand is zero. Thus a zero of (98.12) satisfies every gate and final obligation. Proposition 98.5 forces the unique canonical evaluation of all decision auxiliaries. The acceptance obligation says precisely that the word is executable, and (98.11) says that its endpoint is $y = x + k(P - Q)$. Conversely, if the block relation holds, the canonical gate evaluation is natural and satisfies both kinds of obligations, giving a zero. Endpoint multiplication outputs are also unique, establishing the single-fold claim. Each F_a has degree at most two, so their squared sum has degree at most four. The count is verified in the next subsection. $\square$

98.6 Audited size accounting

Table 11 specifies the count at the level of the actual compiler. Prefix sums are not optimized away, initial max/min folds are not simplified, and duplicate comparisons are not shared.

Table 11: Exact contributions to the endpoint compiler.

Component	Auxiliaries	Residuals
Separate P, Q prefix sums	$2d\ell$	$2d\ell$
Shared $u = x + P_r$ per phase	$d\ell$	$d\ell$
Slope signs, magnitudes, safe denominators	$10d$	$12d$
Coordinate row, including v and interval folds	$29S$	$32S$
Row flags and conjunction chain	$3R + 3\ell$	$4R + 4\ell$
Endpoint products and equations	$2d$	$3d$
Acceptance obligation	0	1

Here is a finer check of the largest row. One coordinate interval uses 22 auxiliaries and 23 residuals. Constructing v costs one of each. Folding its lower and upper endpoint into L, U uses two min/max gates, costing six auxiliaries and eight residuals. The total is 29 and 32. An earlier-rule row uses a comparison and one acceptance-product assignment, costing three and four. A selected-rule row uses two comparisons, one selected-flag assignment, and one acceptance-product assignment, costing six and eight. There are $R - \ell$ earlier rows and ℓ selected rows. Summing the table yields (3.10.2)–(3.10.3).

For a polynomial whose inputs must themselves encode reduced FRACTRAN fractions, append md residuals $A_{ip}B_{ip} = 0$. This changes neither the auxiliary count nor the degree bound. Alternatively, reduction can remain a stated input promise, as in the sample endpoint compiler.

98.7 Magnitude and bit complexity

Proposition 98.7 (Small canonical witnesses). *Suppose all free input entries other than k are at most M , with $M \geq 1$. Every auxiliary in the endpoint compiler's canonical evaluation is at most*

$$16(k + 1)(\ell + 1)(M + 1). \quad (98.14)$$

In particular, the witness bit length per coordinate is $O(\log(k + 1) + \log(\ell + 1) + \log(M + 1))$.

Proof. The prefix sums are at most ℓM , and u, v are at most $(\ell + 1)M$. Slope magnitudes are at most ℓM , while the safe denominators are at most $\ell M + 1$. The signed-intercept deficit and surplus are at most $(\ell + 1)M$. The ceiling dividend is at most $(2\ell + 1)M + 1$. A nonnegative quotient with denominator at least one is bounded by its dividend; its remainder and division slack are bounded by the denominator. Clipped endpoints lie in $[0, k]$, and all their comparison slacks are bounded by the largest compared value. Sign bits, row flags, and acceptance products are at most one. Finally, endpoint products are at most $k\ell M$. These bounds, including the harmless additions of one in the gate definitions, are all smaller than (98.14). $\square$

For a fixed schema, computing the canonical witness and checking the residuals therefore take time polynomial in the bit lengths of the natural inputs. A repetition count with a thousand decimal digits does not require a thousand-digit number of simulation steps. This claim is about the valuation-vector interface, not about expanding a raw integer such as 2^k .

99 Variants, composition, and what is not free [10]

99.1 Failure and exact maximality

Because the decision circuit is total, changing only its final obligation from $a_w = 1$ to $a_w = 0$ gives a single-fold quartic representation of failure of the prescribed block. This version has inputs (A, B, x, k) and no endpoint obligation. Merely retaining a guessed endpoint while switching the bit would not represent the complement of the full endpoint relation; failure of the word and failure of a proposed endpoint are different predicates.

For exact maximality of complete copies, make two decision circuits with inputs k and $k + 1$ sharing only the free inputs. Require their outputs to be one and zero respectively, and append the endpoint equations for k . The result represents

$$M_w(x) = k \quad \text{and} \quad y = x + kD$$

with exactly one witness whenever true. In terms of the counts for the ordinary endpoint compiler, the duplicated construction uses

$$n_z^{\max} = 2n_z - 2d, \quad n_f^{\max} = 2n_f - 3d. \quad (99.1)$$

Indeed, each ordinary circuit consists of a decision core, $2d$ endpoint product auxiliaries, $3d$ endpoint residuals, and one acceptance obligation. Two cores, one endpoint, and two acceptance obligations give (99.1).

99.2 Fixed concatenations of repeated words

Let $w_1, \dots, w_s$ be fixed nonempty words and $k_1, \dots, k_s$ supplied repetition counts. Compile each block, giving it a fresh set of internal auxiliaries. Introduce boundary states $x_1, \dots, x_{s-1}$, with $x_0 = x$ and $x_s = y$, and add all block residuals to one sum of squares.

Corollary 99.1 (Single-fold flat-schema composition). *For fixed $w_1, \dots, w_s$, the relation that the actual trace begins with $w_1^{k_1} \cdots w_s^{k_s}$ and ends at y has a single-fold quartic representation relative to the free tuple $(A, B, x, k_1, \dots, k_s, y)$. Its auxiliary count is the sum of the block auxiliary counts plus $d(s - 1)$ for the internal boundaries.*

Proof. A zero is exactly a sequence of valid block certificates with matching boundaries. If all blocks are valid, their endpoints are forced recursively by the previous boundary and the known update $k_i D_i$. Thus every internal boundary is unique, and each block's auxiliary tuple is unique by Theorem 98.6. The sum of all squared residuals remains quartic. $\square$

Making the counts k_i existential may yield multiple witnesses: distinct factorizations can describe the same overall endpoint, especially when a word has zero drift. The corollary's single-fold property fixes the count tuple as part of the free input. This is a schema-relative statement.

99.3 Raw integers and variable prime powers

For a reduced FRACTRAN program, the compiler currently accepts exponent vectors. The relation between a raw initial integer and these vectors is

$$N = c \prod_{p=1}^d p_p^{x_p}, \quad \gcd\left(c, \prod_{p=1}^d p_p\right) = 1. \quad (99.2)$$

For a given concrete input, this can be computed externally; the supplied small examples use exact integer factorization. Uniformly adjoining (99.2) as an existential polynomial relation requires a genuine representation of variable exponentiation and the inert-factor condition. The repository already treats exponential input encodings in its FRACTRAN Diophantine development [37], but its witness costs and uniqueness are not inherited for free by the present compiler.

The same caution applies to the raw output integer. An endpoint of the form $2^{10^{1000}}$ cannot be explicitly printed using a certificate whose entire bit size is only thousands of bits. The succinct exponent interface is an explicit part of the theorem, not an output-size miracle.

99.4 Why this is not a universal single-fold result

A fixed word schema describes a restricted, efficiently checkable relation. A universal computation may require unboundedly many distinct macro words or boundaries. Encoding an arbitrary word into one natural-number input and verifying all its phases with a fixed number of polynomial variables is an additional problem. Our size grows with ℓ , and the compiler does not remove that dependence.

Furthermore, one equation is not the same as one auxiliary solution. The vendored name `FRACTRAN_HALTING_dio_single` refers to a single equation representation, not automatically to a single-fold witness fiber. The general single-fold problem belongs to a much broader research program [51]; no conclusion about it follows from a family of schema-indexed canonical verifiers.

99.5 Why this is not a new universal operation record

The numbers n_z, n_f count witnesses and residual constraints for a specified macro. They are not counts of additions and multiplications in a complete universal halting certificate. Priority checks have been included in this macro compiler, but arbitrary history structure, input conversion, terminal conditions, and universal control remain separate. The repository's reported 75-operation certificate and its older 90-operation research baseline concern a different complete task [1, 36]. No numerical comparison that omits these differences is justified.

100 Infinite periodic instruction tails [10]

100.1 An exact finite criterion

An infinite repetition of labels need not return to the same numerical state. A word with positive drift can repeat forever while its state grows. Accordingly, periodicity below always refers to the *instruction trace*, not to equality of consecutive block-boundary states.

Theorem 100.1 (Infinite repetition criterion). *The execution from a supplied state x begins with w^ω if and only if all three conditions hold:*

- (i) $D_p \geq 0$ for every coordinate p ;

(ii) $x + \beta_r \geq B_{j_r}$ for every phase r ;

(iii) for every phase r and earlier rule $i < j_r$, there is a coordinate p with $D_p = 0$ and $x_p + \beta_{r,p} < B_{i_p}$.

Proof. If the infinite repetition occurs, the boundary states $x + hD$ are natural for every h . A negative coordinate of D would eventually make one of them negative, proving (i). The first copy proves (ii). If (iii) failed for a pair (r, i) , every zero-drift coordinate would already meet that earlier rule's threshold. Every positive-drift coordinate would meet its threshold for all sufficiently large h . There are finitely many coordinates, so some common h would enable rule i at phase r , contrary to the selected trace.

Conversely, by (i) and (ii), every chosen rule is enabled at every vector $x + \beta_r + hD$. For each earlier rule, the coordinate provided by (iii) stays below its consumption threshold forever. Thus all earlier rules are disabled at every phase. Lemma 95.1 gives every finite prefix w^k , and determinism makes these prefixes one infinite execution. $\square$

Corollary 100.2 (Existence of a periodic seed). *There exists a state executing w^ω exactly when $D \geq 0$ and $\text{Cap}(w) = \infty$. When such a state exists, λ is the least one.*

Proof. If an infinite seed exists, all finite capacities are possible and the drift is nonnegative. Conversely, when $D \geq 0$, formula (96.2) gives $\lambda(k) = \lambda$ for all $k \geq 1$. Infinite capacity therefore makes the same seed execute every finite prefix. Leastness follows from the one-copy resource lower bound. $\square$

Corollary 100.3 (Sharp one-sided pumping). *When $D \geq 0$, existence of a seed executing w^{H+1} implies existence of a seed executing w^ω . The threshold $H + 1$ is sharp for $H \geq 1$.*

Proof. With no negative drift, every right margin in (96.7) is zero. A finite capacity is therefore at most H , and the preceding corollary applies. For sharpness use the reduced FRACTRAN list $(1/2^H, 2)$ and word (2) . Its least seed has 2-valuation zero, its drift is $+1$, and the earlier rule first becomes enabled at valuation H . Hence its capacity is exactly H . $\square$

100.2 A canonical polynomial for a supplied infinite tail

For fixed m, d, w , the three conditions in Theorem 100.1 can also be compiled into a single-fold quartic polynomial, with natural inputs A, B, x . This requires no unbounded search: compare P_p and Q_p for the drift sign and zero tests; use $x_p + P_{r,p}$ versus $B_{i_p} + Q_{r,p}$ for the phase inequalities; and combine the resulting Boolean values.

For completeness, a conjunction of Boolean values uses the gate $z = ab$. A disjunction uses $z = a + b - ab$, which is again Boolean and nonnegative. Each stationary-blocker flag is the product of the zero-drift flag and a strict-deficit flag. Fold disjunction over coordinates and conjunction over phase-rule pairs. Every flag is computed, so the polynomial does not guess one of several possible blocking coordinates. The proofs of totality, uniqueness, and quartic degree are the same gate induction as before, and the size is $O(d\ell + d \sum_r j_r)$. This extension is a proved construction; the delivered code implements its decision criterion, not a separate infinite-tail polynomial exporter.

101 Discovering periodic tails is still undecidable [10]

Define EP to be the set of finite encodings $(\mathcal{F}, N)$ of positive reduced FRACTRAN programs and positive initial integers whose actual run has an eventually periodic infinite instruction trace. A halted finite trace is not in EP.

Theorem 101.1 (Computably enumerable completeness). *The set EP is computably enumerable and is complete for computably enumerable sets under computable many-one reductions. Hardness persists on a computably constructed family of inputs whose runs are all nonhalting. Consequently, some finite descriptions certify nontermination effectively, but discovering a periodic-tail certificate is not decidable in general.*

Proof. For membership, enumerate a finite prefix length t and a nonempty word w . Simulate exactly t steps; discard the pair if the run halts earlier. Factor the finitely many program numerators, denominators, and the reached state to obtain its relevant valuation vector. Apply Theorem 100.1. This is a finite decision for each pair. Dovetailing all pairs accepts precisely the inputs in EP.

For hardness, start with a deterministic counter-machine halting instance (M, c) . Its halting problem is computably enumerable complete, and the control-prime compilation in Section 94 is effective. We describe an effective counter-machine transformation before applying that compilation.

Add two fresh clock counters C, E , initially zero. At the beginning of each round, test whether the simulated control is halting; if so, enter a fresh two-control-state loop forever. Otherwise simulate one instruction of M and then run a delay routine: emit one distinguished marker, increment C , transfer all units of C to E , and then transfer all units of E back to C . The clock is thus restored, E is again zero, and the next simulation round begins. A transfer is implemented with the usual positive-decrement/zero-test branches and an increment on the other counter. Every delay terminates because each of its two transfer loops decreases its source counter at every iteration.

The continuation after a delay can be handled by finitely many copies of the routine, one for each simulated control state. In that implementation there is a finite set of distinguished marker labels, exactly one of which occurs per nonhalting simulation round, and none occurs inside a delay or the final two-state loop. This avoids assuming an implicit return stack. The state-splitting convention removes any self-target that could cause cancellation in the FRACTRAN compilation.

If M halts, the transformed run eventually enters the fresh two-state loop, so its instruction trace is eventually periodic. If M never halts, markers occur infinitely often, but their successive gaps are unbounded: the clock grows by one per round and each delay performs at least C ordinary steps. Compilation replaces each instruction by a bounded finite number of labeled arithmetic steps and preserves designated marker occurrences. Hence the gaps between the finite set of marker labels remain unbounded in the FRACTRAN trace.

In any eventually periodic infinite word, a finite set of labels that occurs infinitely often has bounded gaps after the periodic prefix: some such label occurs in the period, so every sufficiently long interval of the period length contains one. Thus the nonhalting simulation case cannot have an eventually periodic label trace. The transformed program itself never halts in either case: it either keeps simulating with finite delays or reaches the fresh loop. We have an effective reduction

$$M \text{ halts on } c \iff (\mathcal{F}_{M,c}, N_{M,c}) \in \text{EP},$$

with the announced nonhalting promise, proving completeness. $\square$

Corollary 101.2 (One fixed squarefree-denominator program). *There is a fixed reduced FRACTRAN program $\mathcal{F}_*$, all of whose denominators are squarefree, such that*

$$\{N \geq 1 : (\mathcal{F}_*, N) \in \text{EP}\}$$

is computably enumerable complete. The hardness reduction maps every input to a nonhalting run of this same program.

Proof. Choose a fixed universal deterministic counter machine U . Encoding a machine and its input into an initial configuration of U is an effective reduction of the general halting problem to halting of

U on its encoded configurations. Apply the clock transformation in the proof of Theorem 101.1 to U once. The transformed control graph and its FRACTRAN fraction list depend on U , not on its initial counter values. Only the encoded initial integer varies. Therefore the same proof gives the claimed fixed-program reduction.

Use the increment and decrement/zero-test instruction basis for U and for the clock. After splitting self-targets through fresh states, the control-prime compilation uses denominators q_s or $q_s p_r$, where a control prime and a counter prime are distinct. Each denominator is squarefree, and numerators do not cancel the control or decrement test. Thus the resulting program has $H = 1$. All constructed inputs still follow either the infinite simulation-with-delays or the final two-state loop. Membership is semidecidable by the same prefix-and-word enumeration as before. This is an effective existence construction, not a claim that a minimal or literal fraction list has been printed here. $\square$

Corollary 101.3 (An existential arithmetic class of divergent runs). *For any effective natural-number encoding of FRACTRAN instances, EP has a Diophantine representation and is undecidable.*

Proof. Apply MRDP to Theorem 101.1; the required equivalence between computable enumerability and Diophantine representability is established in the sources discussed in Section 3.10.1 [14, 1]. $\square$

This corollary is qualitative. No variable bound, canonical witness, or single-fold claim is supplied for the existential choice of an arbitrary prefix and arbitrary word. The constructive quartic theorem concerns the verification of a *supplied fixed schema*, not this global search problem.

Proposition 101.4 (No complete existential divergence certificates). *There is no effective certificate relation, decidable on each finite certificate, that is both sound and complete for nontermination of all FRACTRAN inputs. In particular, no single existential Diophantine relation represents their full nontermination set under an effective encoding.*

Proof. If such certificates existed, enumerate them to semidecide nontermination. Halting is already semidecidable by simulation. Running the two procedures in parallel would decide halting on every input, contradicting universality and halting undecidability. The zero relation of a fixed integer polynomial is decidable on each natural tuple, so a purported Diophantine representation would provide just such finite certificates. $\square$

[write] Eventual periodicity and the affine macro certificates below are finite certificates of non-termination: the Σ_1^0 fragment of the recurrence and liveness hierarchy of the neighbouring report *liveness-beyond-halting* of this category.

[write] Batch 63, 30 September 2026: that report has since been written (commit 2a34b1740, from batch-62 manuscripts 10 and 03 and batch-63 manuscript 01); its one hierarchy table for its three sources (its section 1bh:sec:onetable) names the completeness of eventual periodicity (Theorem 101.1) as the Σ_1^0 level for priority programs.

102 Beyond periodic instruction words [10]

102.1 An explicit six-fraction counterexample to completeness

Consider the program

$$\mathcal{D} = \left(\frac{33}{10}, \frac{5}{11}, \frac{7}{5}, \frac{52}{21}, \frac{7}{13}, \frac{5}{7} \right). \quad (102.1)$$

Use primes 2, 3 as data counters and 5, 7, 11, 13 as control tokens. All reduced denominators are squarefree, so this example already lies in the threshold- $H = 1$ class.

Proposition 102.1 (Exact doubling macro). *For every $a \geq 1$, the actual execution of (102.1) from $5 2^a$ begins with the word*

$$(1, 2)^a (3) (4, 5)^a (6) \quad (102.2)$$

and reaches $5 2^{2a}$. Starting at 10, the program never halts and its instruction trace is not eventually periodic.

Proof. At a state $5 2^u 3^v$ with $u > 0$, fraction 1 consumes one factor of 2 and the control 5, producing one factor of 3 and the control 11. Fraction 2 then restores the control 5. No earlier rule is enabled at the intermediate control-11 state. Hence $(1, 2)^a$ transports all a units from the 2-counter to the 3-counter and reaches $5 3^a$. Here fraction 1 is disabled and fraction 3 changes control 5 to 7.

At a state $7 2^u 3^v$ with $v > 0$, fraction 4 consumes one factor of 3 and the control 7, producing two factors of 2 and control 13. Fraction 5 restores control 7. Earlier fractions are disabled by their missing control primes. Thus $(4, 5)^a$ reaches $7 2^{2a}$. Fraction 4 is now disabled by the absence of 3, and fraction 6 changes the control back to 5. This proves the macro, including all priority choices.

Iterate with $a = 1, 2, 4, 8, \dots$. Every round is finite and has $4a + 2$ steps, so the actual run is infinite. Label 3 occurs exactly once in each round. The gap between its occurrence in the a round and its next occurrence in the $2a$ round is $6a + 2$. These gaps are unbounded. An eventually periodic word in which label 3 occurs infinitely often has bounded gaps, so this instruction trace is not eventually periodic. $\square$

The example shows concretely that even correct fixed-period certificates miss some transparent arithmetic nontermination. It also shows why the sharp finite-word pumping theorem does not imply eventual periodicity of all runs, even at the smallest positive threshold.

102.2 Finite affine macro certificates

A modest extension captures this example and many similar nested schedules. Fix a program, words $w_1, \dots, w_s$, and a parameter dimension q . Specify integer-affine maps

$$g : \mathbb{Z}^q \rightarrow \mathbb{Z}^d, \quad T : \mathbb{Z}^q \rightarrow \mathbb{Z}^q, \quad \rho_i : \mathbb{Z}^q \rightarrow \mathbb{Z},$$

and a Presburger-definable invariant $V \subseteq \mathbb{N}^q$. The intended macro starts at $g(v)$, executes $w_1^{\rho_1(v)} \dots w_s^{\rho_s(v)}$, and ends at $g(T(v))$. The word drifts D_i are fixed integer vectors.

Define intermediate affine boundaries

$$b_0(v) = g(v), \quad b_i(v) = g(v) + \sum_{t=1}^i \rho_t(v) D_t. \quad (102.3)$$

They are affine because the D_t are constants, not variable parameters. A certificate asserts, for every $v \in V$, that all $\rho_i(v)$ and $b_i(v)$ are natural, every block $(b_{i-1}(v), \rho_i(v), b_i(v))$ is valid,

$$T(v) \in V, \quad b_s(v) = g(T(v)), \quad \sum_{i=1}^s |w_i| \rho_i(v) \geq 1. \quad (102.4)$$

It also supplies $v_0 \in V$ and an actual finite prefix reaching $g(v_0)$.

Theorem 102.2 (Soundness and decidable checking of affine macro certificates). *Every such certificate proves nontermination of its supplied initial configuration. For a fixed finitely described program and proposed affine certificate data, all universal obligations just listed are decidable by Presburger arithmetic. Restricted to FRACTRAN instances, the resulting class of finitely certifiable nontermination strictly contains EP, but is not all nontermination instances.*

Proof. By induction, $v_n = T^n(v_0)$ stays in V . The certified macro at v_n is an actual finite execution from $g(v_n)$ to $g(v_{n+1})$ and contains at least one step. Concatenating these macros after the supplied reachable prefix gives arbitrarily long prefixes of one deterministic run, hence nontermination.

For decidability, insert the local criterion of Lemma 95.1 for each block. The program and the words are fixed. Every prefix vector and every drift is therefore constant, and the expressions in v and the bounded repetition index h are linear. Conditions (102.3)–(102.4), naturalness, and the Presburger invariant add only Presburger formulas. Their universal closure is a sentence in decidable Presburger arithmetic. No product of two independently varying parameters is introduced.

To include EP, take a supplied reachable periodic seed x , its valid word w , parameter $n \in \mathbb{N}$, invariant $V = \mathbb{N}$, embedding $g(n) = x + nD$, update $T(n) = n + 1$, and repetition count one. The infinite-tail theorem ensures these obligations. The example in Proposition 102.1 lies outside EP but has an affine certificate as displayed below, proving strictness. Finally, these finite certificate descriptions can be enumerated and their correctness decided. If they covered every nonterminating input, they would contradict Proposition 101.4. $\square$

For (102.1), in coordinate order (2, 3, 5, 7, 11, 13), use

$$V = \{a \in \mathbb{N} : a \geq 1\}, \quad g(a) = (a, 0, 1, 0, 0, 0), \quad T(a) = 2a.$$

Take words (1, 2), (3), (4, 5), (6) with respective counts $a, 1, a, 1$. Their successive boundaries are

$$\begin{aligned} (a, 0, 1, 0, 0, 0), & \quad (0, a, 1, 0, 0, 0), & \quad (0, a, 0, 1, 0, 0), \\ (2a, 0, 0, 1, 0, 0), & \quad (2a, 0, 1, 0, 0, 0). \end{aligned}$$

Proposition 102.1 proves the block obligations for all $a \geq 1$; closure and positivity are immediate. Starting at the raw integer 10, which encodes $g(1)$, requires no initial prefix.

Remark 102.3 (Scope of the affine extension). Fixed-path and invariant-based verification are established methods, not novel in themselves [49, 50]. The contribution here is their precise connection to the priority interval compiler and the explicit strict separation example. The delivered executable includes the six-fraction example and the fixed-word checkers; it is not a general Presburger solver or an implemented synthesis engine for affine invariants.

103 Reproducible artifacts and finite validation [10]

103.1 What the implementation actually contains

The archive includes a standard-library Python implementation requiring Python 3.10 or later. Its central object is a finite directed acyclic circuit of assignment, comparison, and division gates. Each residual is stored as an exact sparse integer polynomial, with monomials represented by tuples of variable indices. No floating-point arithmetic, symbolic simplification oracle, or external theorem prover is used.

The principal entry point is

```
compile_block(m, d, word, endpoint=True,
              reject=False, maximal=False, reduced=False)
```

In code, instruction indices are *zero-based*. Thus the mathematical word (2, 3) is supplied as [1, 2]. The dimensions and the word are compile-time data; all program entries are free inputs. The exporter writes the exact list of quadratic residuals defining their sum-of-squares polynomial. The witness method executes the canonical gates and the value method evaluates the polynomial exactly.

Separate functions implement direct one-step simulation, direct finite unrolling, the interval checker, infinite-tail recognition, exact repetition capacity, least seeds, and maximal full-copy counts. Keeping both the unrolled interpreter and interval checker makes it possible to cross-check different implementations on small instances.

103.2 What was tested

The delivered run used pseudorandom seed 20260930 and recorded **114,875 counted finite checks**. The word “counted” is literal: the total includes program instances, elementary gate cases, corrupted witness checks, and individual steps in the native-integer comparison. It is not a count of independent theorems or an exhaustive enumeration of all program inputs.

Table 12: Principal recorded validation categories.

Check category	Count
Exhaustive comparison and division input cases	159
Exhaustive scalar interval cases	6,975
Exhaustive one-counter block cases	12,150
Random multicoordinate block cases	12,000
Actual quartic-certificate instances	640
Single-auxiliary corruption rejections	3,200
Interior-priority and maximality checks	5
Giant-horizon certificates	4
Native-integer doubling steps	522
Capacity, least-seed, and pumping instances	79,172
Sharp-family parameter instances	12
Exact size-formula shapes	36
Total	114,875

The gate tests enumerate candidate natural witnesses in finite ranges and check that the only zero is the intended comparison or quotient tuple. The direct block tests compare the accelerated predicate with step-by-step simulation. Polynomial tests construct the full residual system, evaluate its canonical witness, and reject selected single-coordinate mutations of auxiliary values. The pumping tests compare the closed capacity formula with direct execution from the least seed and from sampled larger seeds. The proofs, rather than these samples, establish general correctness.

103.3 Concrete compiler sizes

Table 13 reports endpoint-compilation sizes from the actual exporter. Words are shown in the mathematical one-based convention.

Table 13: Generated schemas; all residual degrees are at most two.

m	d	Word	Free inputs	Auxiliaries	Residuals
1	1	(1)	5	50	59
2	2	(2)	13	155	177
3	2	(2, 3, 1)	17	417	469
4	3	(4, 2)	31	600	672

The sample JSON is the second schema. It is uniform in A, B , not a specialized polynomial with the example’s fraction coefficients hard-coded. The residual representation fully specifies the polynomial; an independent checker can evaluate it without invoking the circuit builder.

For the one-fraction program $(3/2)$, initial valuation vector $(k, 7)$, and endpoint $(0, k + 7)$, the

maximal-copy compiler has 184 auxiliaries and 212 residuals, independent of k . Tests at

$$k = 10, \quad 10^{10}, \quad 10^{100}, \quad 10^{1000}$$

all produced exact zero certificates. The maximum bit lengths among the computed value vectors were 5, 34, 333, and 3322 respectively. These runs constructed and checked arithmetic certificates; they did *not* unroll 10^{1000} simulated transitions.

The direct-integer test of (102.1) covered seven rounds, with $a = 1, 2, 4, 8, 16, 32, 64$. It checked each of the 522 chosen fractions against both raw-integer divisibility and valuation-vector semantics, ending with 2-exponent 128.

103.4 Reproduction commands and artifact boundary

From the archive root:

```
python code/test_priority_certificates.py
python code/priority_certificates.py \
  --export results/example_certificate.json
python code/generate_instances.py
python code/verify_export.py \
  results/example_certificate.json results/example_instance.json
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

[write] *This command listing is the manuscript's own and keeps its delivered layout, in which the programs import one another by their delivered names; the shipped copies carry the prefix 10-priority-pumping- and must be copied back to that layout, outside the collection, before they are run. Where python does not resolve, use py. The README of this report gives rerun recipes that write only into the copy. The manuscript's own article.tex is not shipped; its text is this Part.*

The first command refreshes the structured verification report. The exported polynomial and a concrete witness can be checked independently by the fourth command. The article is self-contained LaTeX with an internal bibliography and does not need a downloaded bibliography database.

The mathematical proof-to-code correspondence has been inspected and tested, but not mechanically established. The code's sparse-polynomial arithmetic, the JSON serialization, Python's runtime, and the test harness are outside any proof-assistant kernel. The formalization plan below is intended to close that boundary rather than conceal it.

104 A formalization roadmap for ProveIt [10]

A clean implementation should separate a semantic development from certificate generation. The recommended first target is the pumping argument, because it uses finite maxima, integer linear arithmetic, and coordinatewise order, without invoking the full MRDP machinery.

104.1 Semantic layer

Define an ordered rule as two vectors in $\mathbb{N}^d$ and selection as the least enabled list index. Prove determinism, nonnegativity of successors, and the prefix-state identity. Then formalize Lemma 95.1. This layer should not mention fraction factorization or Diophantine polynomials.

For the FRACTRAN bridge, add a separate theorem connecting reduced positive fractions to prime valuations, explicitly retaining the inert coprime factor. A proof must not use unreduced numerator and denominator exponent vectors as if cancellation were harmless.

104.2 Capacity and pumping layer

The core milestones are the resource maximum (96.2), lowering within the safe orthant, the two-sided interval (96.8), and the finite-or-infinite capacity formula. The bound on each deficit by H then makes Theorem 97.1 short. The sharp family requires only checking four-coordinate rules and the two inequalities $h \geq H$ and $k - 1 - h \geq H$. This proof order keeps the genuinely important monotonicity claim visible and avoids proving the wrong global monotonicity property.

104.3 Verified arithmetic circuit layer

Give gates a syntax and an evaluation relation over naturals. Prove the comparison and Euclidean-division uniqueness lemmas first. A gate-DAG induction should establish a generic theorem: a total canonical circuit with quadratic residuals has exactly one auxiliary tuple satisfying its gate equations. A second theorem appends an output obligation and forms a sum of squares in an integer multivariate polynomial ring.

The interval circuit then needs a sign-case proof, including $k = 0$ and $D_p = 0$. Next prove its acceptance bit equivalent to the semantic block predicate. Finally, make variable and residual counts functions of the syntax and prove the accounting formulas structurally. The certificate producer should return both a sparse polynomial and a theorem connecting its evaluation to that syntax, rather than trusting an external JSON file.

104.4 Where existing repository results can be reused

The inspected Rocq file already exposes `dio_rel_frastran_step`, `dio_rel_frastran_rt`, `dio_rel_frastran_stop`, and `dio_rel_frastran_eval` [37]. These are useful targets for an extensional correctness bridge from the accelerated semantics. The new compiler should not be advertised as replacing their unbounded reachability theorem: it verifies one prescribed macro family.

For the qualitative EP corollary, the existing MRDP interface is an appropriate endpoint after formalizing the effective semidecider and the clocked reduction. For quantitative canonical certificates, however, invoking a generic Diophantine closure theorem may erase the witness count and uniqueness information. Those properties need their own strengthened interface and proof.

No Lean or Rocq source is included as a purported checked implementation. The roadmap names proof obligations rather than inventing successful builds or unverified theorem APIs.

105 Further research questions [10]

The following are concrete next questions arising from this manuscript. They are research proposals, not claims that each is an independently established longstanding open problem. The first priorities are an external novelty audit and formal verification of the exact statements proved here.

Research question 105.1 (A sharp structural hierarchy beyond H ; source 10). Can the exact capacity be bounded sharply by a smaller invariant combining which coordinates occur in earlier guards, the signs of D , and the priority dependency graph? The formula $\min(L_{ri} + R_{ri})$ provides a starting point. A useful result would characterize the programs in which all finite capacities are at most H , even when mixed drift is permitted.

Research question 105.2 (Presburger guards instead of consumption orthants; source 10). Which extensions of the enabling predicates preserve a word-length-independent pumping threshold? The least-seed argument depends on upward-closed consumption guards. Arbitrary Boolean combinations of

linear inequalities need not have a least resource-safe seed. A sharp positive theorem for a specific guard class, or a small counterexample showing unbounded finite capacity with bounded guard constants, would locate the boundary precisely.

Research question 105.3 (A substantially smaller canonical interval circuit; source 10). How much can the constants 29 and 32 per coordinate row be reduced while retaining totality and unique witnesses? Natural opportunities are shared min/max comparisons, elimination of initially trivial folds, constant propagation for fixed programs, and shared sign tests. The target should be a proved count for a complete macro verifier, including priority and endpoint obligations, rather than a transport-only operation estimate.

Research question 105.4 (Direct raw-integer interfaces; source 10). Can useful classes of FRACTRAN macros be certified directly on N and its raw endpoint without first exposing all valuations, and with a controlled single-fold witness cost? Factoring externally is an implementation choice, not a uniform polynomial interface. Any proposed improvement must account for variable powers and output size explicitly.

Research question 105.5 (Canonical macro decompositions; source 10). For which deterministic priority programs is there a computable normal form for finite traces as concatenations of repeated words, with uniqueness of the repetition counts and a bounded number of boundaries? This would extend Corollary 99.1 from fixed counts to canonical existential counts. A general affirmative answer cannot simply be assumed from deterministic small-step execution, because the same trace can admit many macro factorizations.

[write] Compare Research question 71.9.

Research question 105.6 (Certificate synthesis with completeness on a subclass; source 10). Can periodic-tail or affine-macro certificates be synthesized completely for a natural syntactic class of ordered programs? Possibilities include bounded control alternation, a fixed number of phase changes, or a well-founded priority dependency graph. A useful theorem would combine sound synthesis with an explicit completeness hypothesis, avoiding the impossibility of a complete method for all nontermination.

[write] Compare Research question 71.49.

Research question 105.7 (Nonlinear parameter updates with linear phase checking; source 10). The doubling example uses the affine update $a \mapsto 2a$. Can a broader certificate class handle nonlinear updates while keeping phase-validity obligations decidable or directly Diophantine with manageable witness growth? The point where a variable repetition count multiplies a variable drift should be isolated: it is exactly where the elementary Presburger argument in Theorem 102.2 no longer applies.

[write] Compare Research question 71.48.

Research question 105.8 (A small explicit fixed interpreter; source 10). Corollary 101.2 establishes fixed-program completeness already at $H = 1$, but does not print an optimized fraction list. What quantitative tradeoffs between fraction count, number of primes, and input encoding can be achieved by an explicit clocked universal interpreter? The reduction must preserve its unbounded-gap marker invariant at the level of actual selected fraction labels, and any claimed small certificate must include that control machinery rather than count only the simulation instructions.

Research question 105.9 (Proof-carrying sparse polynomial export; source 10). Can the Python exporter be replaced by a verified Lean or Rocq exporter whose output carries a checkable coefficient certificate, a semantics proof, and a structural proof of the variable count? The modest size of individual gates makes a small independent residual checker attractive. The trust boundary should include the exact polynomial, not merely its high-level circuit specification.

[write] Compare Research question 71.1.

Research question 105.10 (Use inside the universal arithmetic-certificate project; source 10). Can priority-exact macro compression remove enough repeated history structure from a fixed universal interpreter to improve a *complete* universal certificate? This would require measuring input conversion, macro decomposition, all control obligations, halting, and arithmetic operations together. The present results supply a sound local component, not a numerical answer to that global optimization problem.

106 Conclusion [10]

The most useful additional structure in this problem is not merely that FRACTRAN computes arbitrary partial recursive functions. It is the geometry of one prescribed priority trace. At a phase of a repeated word, every coordinate evolves affinely. Earlier-rule enabling therefore occupies an exact interval of repetition indices, and the least resource-safe seed turns that interval into a left waiting margin and a right waiting margin. Their sum is the exact repetition capacity. Bounding each margin by the largest consumption threshold gives a sharp $2H + 1$ pumping theorem.

The same interval geometry admits a completely explicit quartic polynomial with a unique natural auxiliary solution, provided the word schema is fixed. This answers the priority obligation for a meaningful compressed class without pretending to solve arbitrary history compression. Infinite periodic tails have an equally exact finite criterion, while clocked universality and the doubling example show why certificate verification, certificate discovery, and full nontermination must remain distinct.

The next decisive steps are independent scrutiny of the claimed contribution boundary, machine verification of the capacity and gate theorems, and experiments that integrate the macro checker into a complete interpreter certificate while preserving honest end-to-end cost accounting.

107 An executable specification of the interval algorithm [10]

[write] Appendix A of manuscript 10.

The following pseudocode uses signed integers for clarity. Its canonical natural-number implementation is the gate circuit in Section 98. It returns only word validity; an endpoint check is then appended.

```
valid_block(A, B, x, word, k):
  drift := sum(A[j] - B[j] for j in word)
  phase := x
  for selected in word:
    for i in 1 .. selected:
      low, high := 0, k
      for coordinate p:
        l, u := affine_interval(phase[p] - B[i,p],
                                drift[p], k)
        low := max(low, l)
        high := min(high, u)
      if i < selected and low < high:
        return false
      if i = selected and (low != 0 or high != k):
        return false
    phase := phase + A[selected] - B[selected]
  return true
```

A hypothetical phase may have a negative coordinate before validity is established; this is allowed in the mathematical pseudocode. The actual polynomial circuit does not introduce negative natural

variables: it keeps production and consumption prefixes separate as in (98.4).

108 An executable specification of the capacity formula [10]

[write] *Appendix B of manuscript 10.*

```
capacity(A, B, word):
  prefixes := cumulative sums of A[j] - B[j] along word
  drift := final prefix
  least[p] := max(B[word[r],p] - prefixes[r,p] for r)
  cap := infinity
  for phase r, selected := word[r]:
    for i in 1 .. selected-1:
      c[p] := least[p] + prefixes[r,p] - B[i,p]
      if exists p: drift[p] = 0 and c[p] < 0:
        continue
      left := max(0, ceil(max(0, -c[p])/drift[p]))
      for p with drift[p] > 0)
      right := max(0, ceil(max(0, -c[p])/(-drift[p])))
      for p with drift[p] < 0)
      cap := min(cap, left + right)
  return cap, least, drift
```

For $k \geq 1$, the candidate seed is $\text{least} + (k - 1) \max(0, -\text{drift})$ coordinatewise. The value ∞ returned by this algorithm means arbitrary *finite* repetition from suitable seeds. For infinite repetition from one seed, the additional nonnegative-drift condition must be checked.

109 Claims and non-claims ledger [10]

[write] *Appendix C of manuscript 10.*

Statement	Evidential status
Exact capacity formula and sharp $2H + 1$ threshold	Proved in this manuscript; finite cross-checks supplied; not proof-assistant verified; literature-wide priority not established.
Uniform fixed-word quartic with unique witnesses	Explicit gate construction and proof; implemented sparse residual exporter; size formulas cross-checked.
FRACTRAN halting is Diophantine	Established prior work and existing repository development; not a new claim.
Eventual label periodicity is c.e.-complete	Full clocked reduction and semidecision proof here; no numerical bound for its MRDP polynomial.
Affine macro certificates beyond periodic tails	Soundness and decidable checking proved; explicit non-periodic example checked; no general synthesis implementation supplied.
Universal single-fold MRDP	Not proved or claimed. Fixed-word and fixed-count interfaces do not imply it.
An improved complete universal operation count	Not proved or claimed. Residual counts and universal arithmetic-operation counts are different.
Lean/Rocq formal verification of the new results	Not performed. Existing repository theorem names describe inspected prior code, not builds run here.

Statement	Evidential status
Global novelty or resolution of a named longstanding conjecture	Not claimed. The sharp statements are proposed research contributions requiring external literature and mathematical review.

Part XIII

Conservative priority reaction networks and a canonical-fuel form of the finite-fold problem

Source and scope. [write] This Part is manuscript 11, “One Shared Fallback: Conservative Reaction Computation, Unique Quartic Certificates, and a Canonical Fuel Form of the Finite-Fold Problem” (batch 62, manuscript 05; archive `one_shared_fallback_research.zip`, inner directory `one_shared_fallback`; shipped files prefixed `11-reaction-fallback-`; labels `cdc:rx:`). It prints the manuscript’s Sections 2–14 in their own order: models and multiplicity; the one-fallback conservative compiler (Theorem 112.3); reservoir thresholds and the fixed universal instance with 61 species and 62 reactions (Theorem 113.1 and Corollary 113.2); why priority cannot be erased (Theorem 114.1 and Proposition 114.2); decidable bounded resources (Theorems 115.2 and 115.4); single-fold trace quartics (Theorem 116.2); bounded minimum-fuel quartics (Theorem 117.1); the canonical-fuel form of the finite-fold problem (Theorem 118.1); Pareto-minimal resources (Theorem 119.1); the worked example and executed tests; the integration plan; the seven research questions; and the conclusion. Its two appendices follow: the universal source machine with a discrepancy in the source literature, and the manifest. Its title, abstract and introduction are in Section 3.11; the conventions of this Part are Section 110.

Relation to the other Parts. [write] Manuscript 11 inspected ProveIt at 4e128356d (and names a later commit ccfb084ad, at which the cited files are unchanged), before this report existed, and does not cite it. It adds a substrate to Part VIII: Part VIII already treats counter and register programs with zero tests, and discrete reaction networks as instances of multiset firing *without* priority; here a conservative two-to-two network with one shared low-priority fallback implements every zero test, with an exact reservoir threshold and a proof that one priority level does not suffice. Its trace quartic is the priority analogue of Part II’s raw-run certificates, without a commutation quotient (the compiled networks are deterministic). Its arithmetic content is Part IX’s reduction pattern with a smaller canonical witness: Theorem 118.1 is the one-parameter refinement of Theorem 62.1(c), with the least reservoir, a single number decidable in unary logarithmic space, in place of the first-halting history; and Theorem 115.4 is the reservoir case of Proposition 62.2. The canonical-fuel theorem re-scopes the question on finite-fold arithmetic primitives (Research question 71.50); see the note there. The finite-rate obstruction (Proposition 114.2) concerns the physical semantics asked about in the neighbouring report *probabilistic-quantum-and-continuous-computation*; no probabilistic law is computed here.

Setting and hypotheses. [write] Deterministic increment/decrement register machines are compiled to priority reaction systems with two priority levels and two-to-two reactions with distinct unit reactants. Closed finite-reservoir instances are finite-state and decidable; universality holds only after existentially quantifying the reservoir, or in the open-reservoir semantics. Priority is an ideal execution rule, not a claim about physical reaction rates. The quartic certificates are families indexed by the horizon T or the resource cutoff L .

110 Conventions of Part XIII

[write] The letters are manuscript 11's; none is renamed. Its macro for the encoding map is printed as this report's $\mathcal{E}$ (the same glyph), and its names FF and SF for the finite-fold and single-fold assertions are used in this Part only; elsewhere the report writes them out.

Symbol	Meaning in Part XIII, and the clash
R	the number of reactions of a compiled network (62 for the universal one). It is not a residual count, as R is in Parts II, V and VII; this Part counts residuals by $E(T)$. R_q is also an internal control species.
d, k, q, a, b	species, registers, labels, increment labels and test labels ($q = a + b + 1$). d counts places in Part II, in a compatible sense.
$A, B, F, Z, C_j, Q_q, S_q$	species: the mode tokens A, B, Z , the reservoir F , register j , and control states. In Part XII, A and B are matrices.
$\mathcal{E}, E(T), V(T)$	the encoding of a configuration as a marking; the numbers of residuals and witnesses of the trace quartic. Parts V and X write E for memory events.
L	a public resource cutoff in Theorem 117.1, and the label set of a register machine. Part V's L is a log length.
f, ρ_t, m	the initial reservoir, the peak-population threshold and the storage total.
H, G, FF, SF	the bounded acceptance predicate, the least-reservoir relation, and the finite-fold and single-fold assertions.
$B_M(m), \mathcal{B}(N)$	the configuration bound and the maximal least reservoir; not Part V's witness-height bounds.
zero gadget	four equations (Lemma 116.1), including $xz = 0$; compare Part V's two (Corollary 36.3) and Part X's three (Lemma 75.4).

111 Models and basic notation [11]

111.1 Register machines

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. A deterministic k -register machine has a finite label set L , a start label q_0 , and one halt label h . Each other label carries exactly one instruction of one of the forms

$$\text{INC}(j, r) : (q, \mathbf{c}) \mapsto (r, \mathbf{c} + \mathbf{e}_j), \quad (111.1)$$

$$\text{DEC}(j, r, s) : (q, \mathbf{c}) \mapsto \begin{cases} (r, \mathbf{c} - \mathbf{e}_j), & c_j > 0, \\ (s, \mathbf{c}), & c_j = 0. \end{cases} \quad (111.2)$$

There is no successor at h . Let a be the number of increment labels and b the number of decrement/test labels, so that

$$q := |L| = a + b + 1. \quad (111.3)$$

The notation b here counts source instructions, not arithmetic witnesses.

Such machines can simulate general computation. We use the eight-register, 23-label instance specified in Section 124; its source is the universal-register-machine construction of Korec and its increment/conditional-decrement presentation in Alhazov and Verlan [52, 53]. We need the following precise consequence: there is a fixed machine U such that, for every c.e. set $S \subseteq \mathbb{N}$, some fixed natural number e_S satisfies

$$x \in S \iff U \text{ halts from registers } (0, e_S, x, 0, \dots, 0). \quad (111.4)$$

A computable translation from an external program numbering to e_S is allowed. Since e_S is a constant when representing S , we never need to assert that this translation has a single-fold Diophantine graph.

111.2 Priority reaction systems

A marking over a finite species set Σ is a vector $X \in \mathbb{N}^\Sigma$. A reaction $u \rightarrow v$ consists of two multisets $u, v \in \mathbb{N}^\Sigma$. It is *enabled* at X if $X \geq u$ coordinatewise, and its firing produces $X - u + v$. Each reaction has priority 0 or 1. A reaction is *permitted* if it is enabled and no enabled reaction has higher priority. One permitted reaction fires at a time.

Our compiled systems have one priority-0 reaction and otherwise only priority-1 reactions. Moreover, each reactant multiset consists of two *distinct* species, each with multiplicity one. This last condition simplifies the polynomial enabling test; products also have total multiplicity two. We do not impose a maximal-parallel-step interpretation.

A marking is accepting if it contains the distinguished species Q_h . On reachable legal markings its count is either zero or one. Acceptance means finite reachability, not fairness, almost-sure termination, or an asymptotic concentration property.

111.3 Diophantine multiplicity

A relation $R \subseteq \mathbb{N}^m$ is Diophantine if a polynomial $P \in \mathbb{Z}[X_1, \dots, X_m, W_1, \dots, W_s]$ satisfies

$$R(\mathbf{x}) \iff \exists \mathbf{w} \in \mathbb{N}^s P(\mathbf{x}, \mathbf{w}) = 0.$$

The number s and the polynomial must be fixed before $\mathbf{x}$ is chosen. The representation is *finite-fold* if each parameter tuple has only finitely many witness tuples, and *single-fold* if it has at most one. Thus a true instance of a single-fold representation has exactly one witness.

A family of polynomials whose number of variables depends on a resource cutoff is not, merely by being a family, a single fixed Diophantine representation. This distinction is essential in Sections 117 and 118. The historical finite-fold and single-fold issues are reviewed in [40, 10].

112 The one-fallback conservative compiler [11]

112.1 Species and initialization

For a source machine M , introduce the following species:

$$Q_q \ (q \in L), \quad S_q, R_q \ (q \text{ a test label}), \quad C_0, \dots, C_{k-1}, F, Z, A, B.$$

Here C_j stores register j , F is the reusable reservoir, Q_q is a normal control state, and S_q, R_q are internal phases. The three mode species are Z, A, B ; no separate permanent catalyst is necessary.

For a source configuration $(q, \mathbf{c})$ and reservoir f , set

$$\mathcal{E}(q, \mathbf{c}; f) = Q_q + 2Z + fF + \sum_{j=0}^{k-1} c_j C_j. \quad (112.1)$$

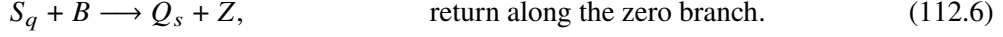
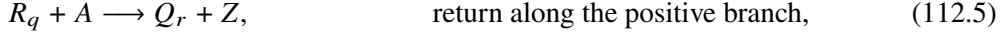
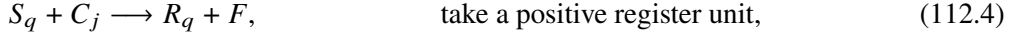
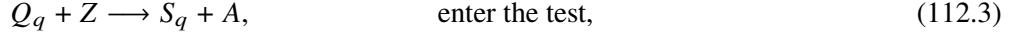
Multiset addition and natural scalar multiplication are understood. There are exactly two Z molecules initially. One becomes an active mode token while the other remains available to the shared fallback.

112.2 The reactions

An increment instruction $q : \text{INC}(j, r)$ contributes the single high-priority reaction



A test instruction $q : \text{DEC}(j, r, s)$ contributes four high-priority reactions:



There is just one low-priority reaction in the whole network:



There is no reaction attached to the halt label.

The fallback does not explicitly inspect any register. It is suppressed when a high-priority reaction is enabled. In the pending-test phase, that fact is exactly what detects whether the tested register is positive. The intermediate R_q phase ensures that the fallback cannot fire between consuming a positive register unit and returning to the next instruction.

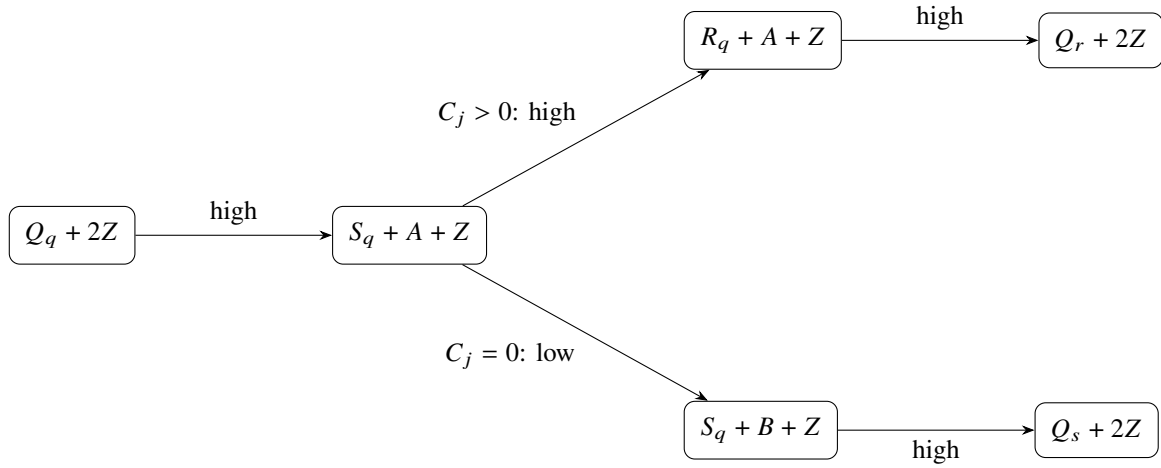


Figure 1: The test macro. Storage species are omitted from the node labels. The positive middle step exchanges one C_j for one F . The low arrow is the same reaction for every test label.

112.3 Invariants and determinism

Lemma 112.1 (Phase and conservation invariant). *Let $m = \|\mathbf{c}\|_1 + f$. Every marking reachable from $\mathcal{E}(q_0, \mathbf{c}; f)$ has storage sum*

$$X(F) + \sum_j X(C_j) = m, \quad (112.8)$$

and its nonstorage part has exactly one of the forms

$$Q_q + 2Z, \quad S_q + A + Z, \quad R_q + A + Z, \quad S_q + B + Z. \quad (112.9)$$

In the last three forms q is a test label. In particular, total molecule count is $m + 3$ and all control coordinates are zero or one.

Proof. The initial marking has the first form. Reaction (112.2) exchanges one reservoir unit for one register unit and changes only the normal control label. Reaction (112.3) changes the first phase to the second. Reaction (112.4) exchanges one register unit for one reservoir unit and changes the second

phase to the third. Reaction (112.5) changes the third phase back to the first. The fallback changes the second phase to the fourth; it is blocked in the third phase by (112.5). Reaction (112.6) changes the fourth phase to the first. All these statements are direct multiset calculations. The control token excludes reactions belonging to other labels. Induction on the number of firings proves the claim. Every displayed reaction is two-to-two, which also proves total molecule conservation directly. $\square$

Lemma 112.2 (Unique permitted reaction). *Every reachable legal marking has either exactly one permitted reaction or none. No permitted reaction exists precisely at a halt marking or at a normal increment label with reservoir zero.*

Proof. At $Q_q + 2Z$, an increment has exactly the reaction (112.2), which is enabled precisely when $F > 0$; a test has exactly its entry reaction; a halt has none. The fallback is disabled because $A = 0$.

At $S_q + A + Z$, only (112.4) can be high-priority enabled, and it is enabled precisely when $C_j > 0$. If so it suppresses the enabled fallback; otherwise the fallback is the sole permitted reaction. At $R_q + A + Z$, reaction (112.5) is high-priority enabled and suppresses the fallback. At $S_q + B + Z$, only (112.6) is enabled. This exhausts the phase invariant. $\square$

112.4 Exact simulation and exact size

Theorem 112.3 (Conservative one-fallback simulation). *The construction is effective. A machine with k registers, q labels and b test instructions compiles to a network with*

$$d = q + 2b + k + 4 \quad \text{species}, \quad R = q + 3b \quad \text{reactions.} \quad (112.10)$$

Exactly one reaction has low priority. Every reaction is two-to-two.

A source increment is simulated in one firing whenever $f > 0$. A positive or zero branch of a source test is simulated in exactly three firings, without any lower bound on f . At normal phase boundaries the simulated register vector is exact. A source computation of t instructions, of which b_t are tests, takes exactly $t + 2b_t$ reaction firings whenever sufficient reservoir is available. Running out of reservoir cannot create a false zero branch or a spurious halt.

Proof. There are q normal control species, $2b$ internal control species, k register species, and the four species F, Z, A, B . Each increment contributes one reaction, each test four, and the common fallback one. Thus $R = a + 4b + 1 = q + 3b$ by (111.3).

For a positive test, the unique sequence is enter, take, return; its net storage effect is $C_j \mapsto C_j - 1$ and $F \mapsto F + 1$. For a zero test, the unique sequence is enter, fallback, zero-return, with no storage change. These statements follow from Lemma 112.2; in particular the remaining Z molecule makes the fallback possible even with no reservoir. The increment effect follows from (112.2). Concatenate the macros and count one firing for an increment and three for a test.

A reservoir shortage can occur only at an increment phase. That phase has no enabled fallback and cannot leave by a different instruction. Therefore it stops rather than changing the source program's meaning. $\square$

113 Reservoir thresholds and two notions of universality [11]

113.1 Peak register population is the exact threshold

For the first t source steps from $(q_0, \mathbf{c}_0)$, write the successive register vectors as $\mathbf{c}_0, \dots, \mathbf{c}_t$, and define

$$\rho_t = \max_{0 \leq i \leq t} \|\mathbf{c}_i\|_1 - \|\mathbf{c}_0\|_1. \quad (113.1)$$

The initial vector participates in the maximum, so $\rho_t \geq 0$.

Theorem 113.1 (Exact resource threshold). *The closed network completes those t source instructions if and only if $f \geq \rho_t$. If the source halts after t instructions, the set of initial reservoirs from which the compiled network accepts is exactly*

$$\{\rho_t, \rho_t + 1, \rho_t + 2, \dots\}. \quad (113.2)$$

If the source does not halt, no finite initial reservoir produces acceptance.

Proof. At each normal phase boundary, conservation forces the reservoir to equal

$$f_i = f + \|\mathbf{c}_0\|_1 - \|\mathbf{c}_i\|_1.$$

A test macro needs no reservoir and never increases register population. An increment is possible exactly when the subsequent value f_{i+1} is nonnegative: its precondition $f_i \geq 1$ is equivalent to that inequality. Thus all instructions through t are executable exactly when each f_i is nonnegative, which is (113.1).

Now suppose the source halts after t instructions. If $f < \rho_t$, take the first source increment that would cross the storage capacity. The compiled network stops at that increment; it cannot escape through another rule. If $f \geq \rho_t$, the exact simulation reaches Q_h . Finally, any compiled acceptance occurs at a normal phase boundary and projects to an actual source halt, by Theorem 112.3. $\square$

113.2 The fixed universal instance

For the universal machine of Section 124,

$$k = 8, \quad q = 23, \quad a = 9, \quad b = 13.$$

Therefore the compiled network $\mathcal{N}_U$ has

$$\boxed{d = 61, \quad R = 62.} \quad (113.3)$$

Define $H(e, x, f)$ to mean that $\mathcal{N}_U$ reaches Q_h from the normal start marking with $C_1 = e$, $C_2 = x$, all other registers zero, and reservoir f . Then

$$U(e, x) \downarrow \iff \exists f \in \mathbb{N} H(e, x, f). \quad (113.4)$$

Here $U(e, x)$ denotes that specified register initialization, not an unexplained external coding convention.

Corollary 113.2 (Reservoir-existential universality). *Every c.e. subset $S \subseteq \mathbb{N}$ has a constant e_S such that*

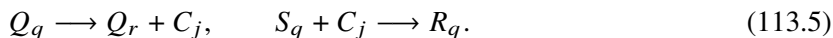
$$x \in S \iff \exists f H(e_S, x, f).$$

The fixed network uses 61 species, 62 two-to-two reactions, and one shared low-priority reaction.

Proof. Combine (111.4) with Theorem 113.1. Every halting source computation has a finite maximum register population, so its threshold is a natural number. $\square$

113.3 The genuinely open-reservoir machine

There is also a conventional unbounded computation semantics. Erase the reservoir bookkeeping by replacing just the two storage-changing reaction types with



Keep all other reaction types, all priorities, and the initial two Z molecules. The same phase proof gives an exact simulation of every source step, with no possibility of reservoir shortage. This open variant is a Turing-complete reaction substrate in the ordinary sense. It has 60 species and the same 62 rules, but it is not molecule-conserving and not uniformly two-to-two.

Thus the closed construction is an exact finite-resource realization of a Turing-complete open substrate. The existential reservoir in Corollary 113.2 is mathematically necessary to bridge them; it is not a device that permits one fixed finite chemical sample to have an infinite state space.

114 Why priority cannot simply be erased [11]

114.1 Ordinary coverability with freely chosen reservoir

The following theorem is a structural boundary, not a claim that ordinary reaction networks lack all useful computation. It concerns the exact acceptance semantics just defined.

Theorem 114.1 (No-priority obstruction). *For a fixed finite ordinary reaction network without priorities, a computable initial marking $I(\mathbf{a})$, a reservoir species F , and a fixed upward-closed target with a finite basis, the predicate*

$$\exists f \in \mathbb{N} \quad I(\mathbf{a}) + fF \text{ can reach the target} \quad (114.1)$$

is decidable. In particular, acceptance by the presence of a halt species cannot give a universal c.e. family of the form above without the additional priority semantics.

Proof. Adjoin the ordinary source reaction $0 \rightarrow F$. Acceptance from $I(\mathbf{a})$ in this augmented net is equivalent to (114.1). One direction fires $0 \rightarrow F$ initially f times. Conversely, a finite augmented accepting path uses the source reaction some finite number f of times. Move those productions to the beginning and delete the corresponding source firings from the path. Ordinary reaction enabling is monotone under addition of molecules, so each remaining firing is still possible. The final marking is unchanged.

For completeness, coverability in the augmented ordinary net has a finite backward procedure. If $r : u \rightarrow v$ is a reaction and m a target lower bound, then

$$\text{Pre}_r(\uparrow m) = \uparrow(u + (m - v)^+), \quad (114.2)$$

where positive part is taken coordinatewise. Indeed, a predecessor X must satisfy both $X \geq u$ and $X - u + v \geq m$, which are exactly the displayed lower bound. Starting with the finite target basis, repeatedly add all such predecessor bounds and remove dominated vectors.

This saturation stops. Dickson's lemma says that every infinite sequence in $\mathbb{N}^d$ contains indices $i < j$ with $x_i \leq x_j$. One elementary proof takes an infinite nondecreasing subsequence of the first coordinates and applies induction to the remaining coordinates. If the increasing sequence of upward-closed backward sets never stabilized, choose x_i newly added at the i th strict increase. For $i < j$, one cannot have $x_i \leq x_j$, because x_i already belongs to the previous upward-closed set containing all old points, while x_j does not. This contradicts Dickson's lemma.

At stabilization the set is exactly the states with a finite path to the target. Membership of $I(\mathbf{a})$ is a finite basis comparison, yielding the decision procedure. $\square$

This proof is the elementary backward-coverability form of the classical well-quasi-order approach to ordinary Petri nets and vector-addition systems [56]. The priorities in our construction invalidate the monotonicity used when moving reservoir production to the beginning in a general priority system: adding molecules can enable a high-priority reaction and suppress a previously permitted low-priority reaction. Our particular compiler separately proves reservoir monotonicity by exact simulation; it does not inherit it from general priority-net semantics.

Together with Corollary 113.2, the theorem gives a sharp *priority-level boundary* for this acceptance format: one priority level is insufficient, while two levels with just one low-priority reaction suffice. It does not give a lower bound of 61 species or 62 reactions.

114.2 A concrete false zero branch

Consider the source program

$$q_0 : \text{DEC}(0, q_1, h), \quad q_1 : \text{INC}(0, q_1).$$

From $C_0 = 1$ it never halts. A correctly prioritized finite-reservoir simulation either continues the increment loop or eventually blocks. After entering the first test, however, the marking contains $S_{q_0} + A + Z + C_0$. If priorities are ignored, one may fire the fallback and then the zero-return reaction, reaching Q_h incorrectly.

The supplied checker constructs exactly this three-reaction spurious trace. All its enabling and update equations are satisfied. Its quartic certificate fails solely at one priority residual, and the polynomial value is exactly one. This is a targeted test of the negative information encoded by priority, not just a successful-run demonstration.

114.3 Finite rates do not implement exact priority

Proposition 114.2 (Exponential-race obstruction). *Suppose the fallback and positive-test reactions in the pending-test phase are assigned strictly positive finite stochastic rates. With $c > 0$ tested register molecules, assume their instantaneous hazards are respectively $\lambda_0 > 0$ and $\lambda_1 c > 0$. Under the independent exponential-clock model, the probability that the fallback fires first is*

$$\frac{\lambda_0}{\lambda_0 + \lambda_1 c} > 0. \quad (114.3)$$

Hence no finite choice of these rates implements the exact deterministic priority zero test on the unchanged reaction graph.

Proof. If T_0, T_1 are independent exponential clocks of these rates, then

$$\Pr(T_0 < T_1) = \int_0^\infty \lambda_0 e^{-\lambda_0 t} e^{-\lambda_1 c t} dt = \frac{\lambda_0}{\lambda_0 + \lambda_1 c}.$$

That event is an incorrect zero branch because $c > 0$. Setting $\lambda_0 = 0$ would prevent the genuine zero branch as well. $\square$

This proposition concerns the stated stochastic model, not every conceivable physical implementation. An implementation with additional sensing, compartments, control hardware, or error correction is a different substrate and requires its own correctness analysis.

115 Decidable bounded resources and unbounded thresholds [11]

115.1 A finite-state bound

The phase invariant has $q + 3b$ possibilities, and the k registers together with the reservoir are $k + 1$ nonnegative counts summing to m . Stars and bars therefore gives the following bound.

Lemma 115.1 (Uniform configuration bound). *A legal closed computation with storage total m has at most*

$$B_M(m) = (q + 3b) \binom{m + k}{k} \quad (115.1)$$

different markings. If it reaches Q_h , it does so in fewer than $B_M(m)$ reaction firings unless the initial marking is already accepting, in which case zero firings suffice. For the universal instance,

$$B_U(m) = 62 \binom{m + 8}{8}. \quad (115.2)$$

Proof. Multiply the number of phase patterns by the number of weak compositions of m into $k + 1$ parts. This counts a superset of the reachable markings, which is sufficient. A deterministic path cannot repeat a marking before its first acceptance: after a repeated marking its future repeats as well. Thus the number of pre-acceptance firings is less than the stated state bound. $\square$

It is convenient to *totalize* the transition function: if no reaction is permitted, repeat the current marking. This is canonical padding, not an extra chemical reaction. It pads both halts and reservoir-blocked states, without turning a blocked nonhalt into an accepting state. After exactly $B_M(m)$ totalized steps, presence of Q_h is equivalent to eventual acceptance.

Theorem 115.2 (Low-complexity bounded-resource predicates). *For the fixed universal network, $H(e, x, f)$ is primitive recursive. On unary encodings of (e, x, f) , it is decidable in deterministic logarithmic space. On binary encodings it is decidable in space linear in input length and time $2^{O(n)}$, where n is binary input length.*

Proof. The storage total is $m = e + x + f$. Compute (115.2) and iterate the totalized transition that many times. Each step uses only bounded integer arithmetic, zero tests, and the fixed finite list of reactions. A primitive-recursive iteration bound gives a primitive-recursive characteristic function.

There are only 61 counts. Storage counts are at most m , mode counts at most two, and control counts at most one. The configuration and a time counter bounded by $B_U(m) = O((m + 1)^8)$ require $O(\log(m + 2))$ bits in total. Binomial evaluation uses a fixed number of arithmetic operations on numbers of that same asymptotic bit length. The deterministic simulation uses polynomially many steps in $m + 1$, with polynomial logarithmic bit overhead. For unary inputs this space is logarithmic in input length. For binary inputs $\log(m + 2) = O(n)$, and the numeric polynomial time bound becomes $2^{O(n)}$. $\square$

115.2 Canonical minimum fuel

Define

$$G(e, x, f) \iff H(e, x, f) \wedge [f = 0 \vee \neg H(e, x, f - 1)], \quad (115.3)$$

where the expression $f - 1$ occurs only in the positive- f branch.

Corollary 115.3 (Decidable graph of a partial resource function). *The relation $\mathbf{G}$ is primitive recursive and unary-logspace decidable. For each (e, x) it holds for at most one f . It holds for some f exactly when $U(e, x)$ halts, and then that f is the peak-population threshold of (113.1).*

Proof. By Theorem 113.1, the accepting reservoir set is either empty or an upper ray. Its least member is precisely the one accepted reservoir whose immediate predecessor is not accepted. Two bounded-resource decisions and a zero test compute (115.3); the simulations can reuse their work space. $\square$

Thus the characteristic function of the graph is total and easy to compute relative to the numeric resource. The corresponding partial function $\rho(e, x)$ is not a total primitive-recursive function: its domain is the universal halting relation. It is partial computable by searching increasing reservoirs and deciding each bounded instance.

[write] The following theorem is the reservoir case of Proposition 62.2 (Part IX); the proof is manuscript 11's.

Theorem 115.4 (No recursive universal reservoir bound). *There is no total computable function $g(e, x)$ such that $\rho(e, x) \leq g(e, x)$ for every halting input. Moreover, the finite maximum*

$$\mathcal{B}(N) = \max(\{\rho(e, x) : e + x \leq N, U(e, x) \downarrow\} \cup \{0\})$$

exceeds every total computable function of N infinitely often.

Proof. Such a bound would decide halting by evaluating the decidable predicate $H(e, x, g(e, x))$, using the upper-ray property. This contradicts universality. If $\mathcal{B}(N) \leq g(N)$ eventually, finitely many exceptional values could be bounded by a fixed natural constant C ; then $\max\{g(N), C\}$ would be a total computable bound everywhere. Substitution $N = e + x$ would contradict the first assertion. $\square$

We assert infinitely-often domination, not eventual domination, which would require further care about the chosen indexing and size measure.

116 Explicit quartic certificates with unique witnesses [11]

116.1 A canonical zero gadget

The following elementary gadget is responsible for keeping negative enabling information polynomial and unambiguous.

Lemma 116.1 (Unique zero indicator). *For every $x \in \mathbb{N}$, the equations*

$$z(z - 1) = 0, \quad xz = 0, \quad x - (1 - z)(b + 1) = 0, \quad zb = 0 \quad (116.1)$$

have exactly one solution $(z, b) \in \mathbb{N}^2$, namely

$$z = \begin{cases} 1, & x = 0, \\ 0, & x > 0, \end{cases} \quad b = \max(x - 1, 0).$$

Proof. The first equation makes z Boolean. If $x > 0$, the second forces $z = 0$, and the third gives $b = x - 1$. If $x = 0$, the third rules out $z = 0$ because $b + 1 > 0$; hence $z = 1$, and the fourth gives $b = 0$. Direct substitution checks both cases. The fourth equation is important: without it, the positive-part witness would be arbitrary in the zero case. $\square$

116.2 Variables and residuals

Fix a compiled network with d species and R reactions, and a horizon $T \in \mathbb{N}$. Index the unique low-priority reaction by ℓ . Let u_j, v_j be the distinct reactant species of reaction j , and let Δ_{ij} be its signed change in species i .

The witness variables are the marking counts $x_{t,i}$ for $0 \leq t \leq T$; zero flags $z_{t,i}$ and canonical positive-part witnesses $b_{t,i}$ for $t < T$; enabling flags $e_{t,j}$ for $j < R$; and selectors $s_{t,j}$ for $j \leq R$. The extra selector $s_{t,R}$ is the canonical idle option. All variables range over $\mathbb{N}$.

Let $I_i(\mathbf{a})$ be the affine initial marking coordinates, with input parameters $\mathbf{a}$. First impose

$$x_{0,i} - I_i(\mathbf{a}) = 0 \quad (i < d). \quad (116.2)$$

For every $t < T$, impose the following finite list of residual equations:

$$z_{t,i}(z_{t,i} - 1) = 0, \quad x_{t,i}z_{t,i} = 0, \quad (116.3)$$

$$x_{t,i} - (1 - z_{t,i})(b_{t,i} + 1) = 0, \quad z_{t,i}b_{t,i} = 0, \quad (116.4)$$

$$e_{t,j} - (1 - z_{t,u_j})(1 - z_{t,v_j}) = 0, \quad (116.5)$$

$$s_{t,j}(s_{t,j} - 1) = 0 \quad (j \leq R), \quad \sum_{j=0}^R s_{t,j} - 1 = 0, \quad (116.6)$$

$$s_{t,j}(1 - e_{t,j}) = 0 \quad (j < R), \quad (116.7)$$

$$s_{t,\ell}e_{t,j} = 0 \quad (j \neq \ell), \quad (116.8)$$

$$s_{t,R}e_{t,j} = 0 \quad (j < R), \quad (116.9)$$

$$x_{t+1,i} - x_{t,i} - \sum_{j=0}^{R-1} \Delta_{ij}s_{t,j} = 0. \quad (116.10)$$

The unqualified species equations range over $i < d$, and enabling equations over $j < R$. No higher-degree product over all enabled reactions is needed. The pairwise products in (116.8) and (116.9) suffice. For an endpoint bit $\eta \in \{0, 1\}$, append

$$x_{T,Q_h} - \eta = 0. \quad (116.11)$$

An endpoint may alternatively be another Boolean variable constrained elsewhere, as in Section 117.

Let $\mathcal{R}_T$ denote the complete residual list. Define the one ordinary polynomial

$$P_T(\mathbf{a}, \mathbf{w}; \eta) = \sum_{r \in \mathcal{R}_T} r(\mathbf{a}, \mathbf{w}; \eta)^2. \quad (116.12)$$

Every residual has degree at most two, so $\deg P_T \leq 4$. This is a literal finite polynomial for every fixed T , not an exponential Diophantine expression and not a bounded quantifier left unevaluated.

116.3 Soundness, completeness, and multiplicity

Theorem 116.2 (Single-fold bounded trace compiler). *For a legal initial marking and fixed horizon T , natural solutions of $P_T = 0$ are in bijection with totalized length- T reaction traces whose endpoint has halt count η . In the deterministic compiled networks, there is exactly one solution if the endpoint condition is correct, and no solution otherwise. The witness count and residual count are respectively*

$$V(T) = (3d + 2R + 1)T + d, \quad (116.13)$$

$$E(T) = (5d + 5R + 1)T + d + 1. \quad (116.14)$$

Proof. Over natural assignments, the residuals are integers. A sum of their squares is zero exactly when every residual is zero. Thus it is enough to analyze the system of residual equations.

The initial row is fixed by (116.2). Given a row, the zero flags and b variables are uniquely determined by Lemma 116.1. Equation (116.5) then fixes every enabling flag to its actual Boolean enabling value; distinct unit reactants are used here. The selectors are Boolean and sum to one, so exactly one is selected.

If an ordinary reaction is selected, (116.7) makes it enabled. If it is the low reaction, (116.8) rules out every enabled high reaction. If the idle option is selected, (116.9) says that there is no enabled reaction at all. Conversely, these equations allow every permitted reaction and allow idle exactly at a deadlock. Existence of an enabled reaction is equivalent to existence of a permitted one because there are only two priority levels.

The update equations give precisely the selected stoichiometric change, or no change for idle. Starting from a legal initial marking, induction therefore produces an actual totalized trace and preserves the phase invariant. On that invariant, Lemma 112.2 makes the selector unique at each step. Induction fixes every later marking and every auxiliary variable. Finally (116.11) accepts exactly the stated endpoint.

Conversely, take an actual totalized trace. Assign each zero flag and positive-part witness its canonical value, enabling flags their actual values, and the single selector its fired reaction or idle. Every residual vanishes, proving completeness.

There are $d(T + 1)$ marking variables, $2dT$ zero-gadget variables, RT enabling flags, and $(R + 1)T$ selectors, proving (116.13). A time row has $4d$ zero equations, R enabling equations, $R + 1$ selector Boolean equations, one selector-sum equation, R eligibility equations, $R - 1$ priority equations, R idle equations, and d update equations. Their sum is $5d + 5R + 1$. Add d initialization equations and one endpoint equation to obtain (116.14). $\square$

Remark 116.3 (Which assertions depend on determinism). The residual list also expresses allowed traces from nondeterministic markings, but then there may be several selectors and several traces. Uniqueness is proved on the compiler's invariant, not on arbitrary markings in $\mathbb{N}^d$. An arithmetic certificate cannot manufacture uniqueness of a semantics that actually has several executions.

For the universal instance, the exact counts specialize to

$$\boxed{V(T) = 308T + 61, \quad E(T) = 616T + 62.} \quad (116.15)$$

These are transparent, deliberately unoptimized counts. Removing redundant Boolean checks or exploiting one-hot control could lower them, but is not needed for the theorem.

117 Single-fold quartics for bounded minimum fuel [11]

Fix a public cutoff $L \in \mathbb{N}$ and consider triples with $e + x + f \leq L$. Put $T = B_U(L)$. We now construct a quartic $Q_L(e, x, f; \mathbf{w})$ that represents the restriction of $\mathbf{G}$ to this finite resource region. The polynomial depends on L , but its three input parameters do not get substituted until it is used.

Introduce two canonical fuel-gadget variables d_f, b_f satisfying

$$d_f(d_f - 1) = 0, \quad f d_f = 0, \quad f - (1 - d_f)(b_f + 1) = 0, \quad d_f b_f = 0. \quad (117.1)$$

Thus d_f is the indicator of $f = 0$ and $b_f = \max(f - 1, 0)$. Use two disjoint copies of the trace system of Section 116:

	initial reservoir	endpoint halt count
first trace	f	1
second trace	b_f	d_f .

(117.2)

Both have input registers $C_1 = e, C_2 = x$, the same start control, and two Z molecules. Both have the fixed horizon $T = B_U(L)$. Add one slack variable $g \in \mathbb{N}$ and the residual

$$e + x + f + g - L = 0. \quad (117.3)$$

Define Q_L as the sum of squares of all residuals in these two trace systems, the four fuel-gadget residuals, and (117.3).

Theorem 117.1 (Explicit bounded canonical-fuel quartics). *For every L , Q_L is an effectively constructible integer polynomial of degree at most four, and*

$$\exists \mathbf{w} \in \mathbb{N}^{v_L} \quad Q_L(e, x, f; \mathbf{w}) = 0 \iff e + x + f \leq L \wedge \mathbf{G}(e, x, f). \quad (117.4)$$

For every parameter triple there is at most one witness tuple. More precisely, with $T = 62 \binom{L+8}{8}$,

$$v_L = 616T + 125, \quad (117.5)$$

$$\# \text{quadratic residuals} = 1232T + 129. \quad (117.6)$$

Every coordinate of a satisfying witness is at most $\max(L, 2)$.

Proof. The slack equation enforces the cutoff and uniquely determines g . The fuel gadget uniquely determines d_f, b_f . Both actual storage totals are at most L , and the state bound is nondecreasing in storage total. By Lemma 115.1, a trace padded to $B_U(L)$ ends at Q_h if and only if that reservoir permits eventual acceptance.

If $f > 0$, then $d_f = 0$ and $b_f = f - 1$. The first trace certifies acceptance at f and the second certifies nonacceptance at $f - 1$, exactly the minimum-fuel condition. If $f = 0$, both reservoirs are zero and $d_f = 1$. Both traces are copies of the same accepting zero-reservoir computation. This redundant second trace is intentional: it fixes all of that branch's variables without introducing an unconstrained inactive certificate.

The trace uniqueness theorem now fixes all variables of both copies. All residuals are quadratic, including the endpoint equation whose right side is d_f . Thus the sum of squares is quartic and single-fold. Two copies of (116.15), three extra variables, and five extra residuals give the stated counts.

Register and reservoir counts are at most L , control counts at most one, and mode counts at most two. Positive-part witnesses are no larger than the corresponding counts, and all flags and selectors are Boolean. Finally $b_f \leq f \leq L$ and $g \leq L$. This proves the height bound. $\square$

117.1 What this does not imply

The number of variables grows as $O((L + 1)^8)$. It is therefore invalid to replace L by an existential variable in the subscript of Q_L and call the result a fixed polynomial. The syntax of a polynomial has one fixed finite list of variables.

One might instead allow an arbitrary sufficiently large cutoff and try to compress the trace. That introduces infinitely many choices of cutoff for a true instance. Choosing $L = e + x + f$ removes this particular ambiguity, but still leaves a variable-sized row system to encode. The difficulty is not a missing instruction to set the cutoff minimally; it is the global finite-fold elimination of the unbounded trace representation.

The height bound is also local. It bounds witnesses after the resource parameter and cutoff are fixed. It does not provide a computable bound on the least successful resource as a function of (e, x) alone; Theorem 115.4 forbids the latter.

118 A canonical-fuel normal form for the finite-fold problem [11]

118.1 The conjectures and the exact equivalence

Write FF for the assertion that every c.e. relation over $\mathbb{N}$ has a finite-fold Diophantine representation, and SF for the corresponding single-fold assertion. One can formulate these using unary sets only: iterated Cantor pairing is polynomially expressible with uniquely determined auxiliaries, so this change of arity does not affect either assertion. For example, the graph of a pairing is

$$2z = (x + y)(x + y + 1) + 2y,$$

with unique z for each (x, y) .

These are the polynomial witness-multiplicity questions discussed in the literature [40, 10]. The following equivalence is an unconditional theorem about those assertions; it does not assume that either is true.

[write] The following theorem is the one-parameter refinement of Theorem 62.1, statement (c) (Part IX): the least reservoir, one number whose relation is decidable in unary logarithmic space, replaces the first-halting history as the canonical witness, and the proof is the pattern $(c) \Rightarrow (b) \Rightarrow (a)$ of that theorem. It is manuscript 11's statement and proof.

Theorem 118.1 (Canonical-fuel equivalence). *For the single fixed 61-species network constructed above,*

$$\text{FF} \iff G(e, x, f) \text{ has a finite-fold Diophantine representation,} \quad (118.1)$$

$$\text{SF} \iff G(e, x, f) \text{ has a single-fold Diophantine representation.} \quad (118.2)$$

In each right-hand statement, “Diophantine representation” may be replaced by “representation by one polynomial of degree at most four.”

Proof. If FF holds, it applies to G , which is primitive recursive and hence c.e. The same argument gives the forward implication for SF.

Conversely, suppose a polynomial $D(e, x, f, \mathbf{w})$ represents G finite-fold, where multiplicity is measured in $\mathbf{w}$ after (e, x, f) are fixed. Let $S \subseteq \mathbb{N}$ be any c.e. set, and choose the fixed program input e_S of (111.4). Then

$$x \in S \iff \exists f \exists \mathbf{w} D(e_S, x, f, \mathbf{w}) = 0. \quad (118.3)$$

For each fixed x , there is at most one f with $G(e_S, x, f)$. If there is none, the equation has no witnesses. If there is one, the assumed representation allows only finitely many $\mathbf{w}$. Thus the combined witness tuple $(f, \mathbf{w})$ is finite in number. This proves FF. Under the single-fold hypothesis the same argument gives at most one combined tuple, proving SF. The only operation on the program index is substitution of a natural constant.

For the quartic qualification, split any integer polynomial D into $D^+ - D^-$, with D^+, D^- having nonnegative integer coefficients. Evaluate these two expressions by finite addition/multiplication circuits. Introduce a natural gate variable for each intermediate value and impose its gate equation, of degree at most two, along with equality of the two output gates. The sum of squares of these residuals is quartic. For any original parameter and witness tuple, every gate value is uniquely specified. Therefore this transformation preserves both finite-foldness and single-foldness. The reverse implication from quartic representations is immediate. $\square$

118.2 Why this is a substantive but limited reduction

The right-hand relation in Theorem 118.1 is not a generic halting predicate. It is a concrete decidable relation about the least storage reserve in a fixed finite-state family, with a unary-logspace algorithm and explicit bounded single-fold quartics. Thus the theorem concentrates the multiplicity problem in a narrowly specified resource boundary.

The logical mechanism—projecting a canonical witness for a universal c.e. relation—is a general normal-form principle, not a newly invented replacement for MRDP. What the compiler adds is a restricted operational realization and an explicit bounded algebraic realization of that canonical witness relation. This is the appropriate level of novelty claimed here.

118.3 Unconditional global Diophantine representations

MRDP unconditionally supplies a fixed polynomial representation of G , because its graph is decidable. Quadratic circuit lifting as in the proof above makes that representation quartic. Likewise $H(e, x, f)$ and the projected universal halting relation have fixed ordinary Diophantine representations. These existence statements follow from established machinery, including ProveIt’s MRDP and primitive-recursive interfaces [5, 3].

What is not supplied unconditionally is a bound on the number of auxiliary solutions of the *global* representation of G . The package contains the explicit bounded compiler, not an expanded fixed-arity global MRDP polynomial. A large unspecified global polynomial is not being presented as if it had been generated or checked.

Proposition 118.2 (Unnormalized fuel forces infinitely many witnesses). *Suppose $D(e, x, f, \mathbf{w}) = 0$ represents $H(e, x, f)$, with any multiplicity at fixed (e, x, f) . For every halting input (e, x) , the representation*

$$\exists f \exists \mathbf{w} D(e, x, f, \mathbf{w}) = 0$$

has infinitely many combined witness tuples $(f, \mathbf{w})$.

Proof. Every $f \geq \rho(e, x)$ satisfies H , so each such distinct f has at least one corresponding witness tuple. Distinct first coordinates give distinct combined tuples. $\square$

This is a concrete reason why deterministic computation by itself does not produce finite-fold Diophantine representations. Even before considering auxiliary number-theoretic witnesses, arbitrary excess resource creates an infinite fiber. Minimum-fuel normalization removes that source of multiplicity, but Theorem 118.1 shows why it cannot automatically remove all global arithmetic multiplicity.

118.4 A further parameter distinction

Because G is decidable, any particular correct polynomial representation of it admits *some* total computable search bound on a least witness as a function of the full triple (e, x, f) . Decide the relation first; on false triples return zero, and on true triples enumerate witnesses until one is found. This observation supplies neither a useful explicit bound nor a finite-fold representation.

It also does not contradict Theorem 115.4. In that theorem f is the unknown resource to be found, not part of the supplied parameter triple. Quantifier placement, rather than a vague appeal to “large witnesses,” explains the difference.

119 Pareto-minimal resources: a general extension [11]

The scalar construction has a unique least successful resource. A useful extension applies to several resource dimensions, even when no single least vector exists. It isolates which parts of the finite-fold argument need uniqueness and which need only finiteness.

Let $A(\mathbf{a}, \mathbf{r})$ be a decidable predicate on parameters $\mathbf{a}$ and resources $\mathbf{r} \in \mathbb{N}^s$, with fixed finite $s \geq 1$. Assume it is upward closed in the resource coordinates:

$$A(\mathbf{a}, \mathbf{r}), \mathbf{r} \leq \mathbf{r}' \implies A(\mathbf{a}, \mathbf{r}').$$

No assertion is being made that arbitrary priority systems satisfy this monotonicity. It must be proved for the particular substrate.

Theorem 119.1 (Local certification of Pareto minimality). *The Pareto-minimal successful vectors are characterized by*

$$A(\mathbf{a}, \mathbf{r}) \wedge \bigwedge_{i=1}^s [r_i = 0 \vee \neg A(\mathbf{a}, \mathbf{r} - \mathbf{e}_i)]. \quad (119.1)$$

There are finitely many such vectors for each fixed $\mathbf{a}$, and at least one whenever any successful vector exists. Their relation is decidable; it is primitive recursive when A is primitive recursive.

Proof. A successful immediate predecessor makes $\mathbf{r}$ nonminimal. Conversely, if a successful vector $\mathbf{u} \leq \mathbf{r}$ with $\mathbf{u} \neq \mathbf{r}$ exists, choose an index i with $u_i < r_i$. Then $\mathbf{u} \leq \mathbf{r} - \mathbf{e}_i$, so upward closure makes that immediate predecessor successful. This proves (119.1).

Every successful vector has a minimal successful vector below it because its lower coordinate box is finite. Distinct minimal vectors form an antichain in $\mathbb{N}^s$, which must be finite by Dickson's lemma. The displayed criterion uses only $s + 1$ decisions of A and fixed arithmetic tests, proving the computability assertions. $\square$

Corollary 119.2 (Finite-fold resource-frontier normal form). *Suppose the projection $\exists \mathbf{r} A(e, x, \mathbf{r})$ is universal for c.e. sets by specialization of e . Then the assertion that its Pareto-minimum relation has a finite-fold Diophantine representation is equivalent to FF.*

Proof. The forward implication to FF specializes a program parameter and existentially quantifies the resource vector. Only finitely many minimum vectors exist, and each has finitely many arithmetic witnesses; their finite union is finite. The other direction applies FF to the decidable Pareto-minimum relation. $\square$

The analogous argument does *not* automatically prove a single-fold equivalence: there may be several incomparable minima. The scalar reserve in our main construction is therefore especially useful. It provides a unique minimum, so Theorem 118.1 covers single-foldness as well.

For deterministic source computations with registers assigned to several separate reusable reservoirs, one obtains an even simpler special case: each reservoir needs the maximum population increase in its assigned register group. The vector of those coordinatewise maxima is a unique least successful vector. The interesting nonprincipal Pareto case arises when there are genuinely alternative successful computations or resource tradeoffs; those semantics require an additional compilation argument.

120 Worked example and executable verification [11]

120.1 A doubling machine with exact fuel spectrum

Consider the two-register program

$$\begin{aligned}
 0 : & \text{ DEC}(0, 1, H), \\
 1 : & \text{ INC}(1, 2), \\
 2 : & \text{ INC}(1, 0), \\
 H : & \text{ HALT}.
 \end{aligned} \tag{120.1}$$

From $(C_0, C_1) = (n, 0)$, it repeatedly consumes one unit of the first register and produces two in the second. It halts at $(0, 2n)$. There are $3n + 1$ source instructions, $n + 1$ of them tests, hence $5n + 3$ reaction firings. The peak total register population is $2n$, including the initial value n , so the exact least reservoir is n .

Here $q = 4, b = 1, k = 2$, giving 12 species and 7 reactions. In the notation of Section 112, the complete reaction list is



Only the last rule has low priority. The theorem gives the exact spectrum

$$H_{\text{double}}(n, f) \iff f \geq n, \quad G_{\text{double}}(n, f) \iff f = n.$$

These conclusions also follow directly by induction on the number of completed loop iterations: the normal-phase registers are $(n - i, 2i)$ and the reservoir is $f - i$.

For $n = f = 1$ the network halts in eight reaction firings. The generated $T = 8$ certificate has 420 witness variables and 781 quadratic residuals. Its expanded quartic has 3,202 nonzero monomials. The package contains that polynomial, its residual presentation, and a full natural witness whose polynomial value was checked to be zero.

120.2 Tests actually executed

The verification program uses the deterministic seed 20260930. The following counts are taken from the executed output, not from a planned test suite.

Check	Count or result
Zero-gadget natural assignments inspected	832
Exhaustive local instruction/input/reservoir cases	600
Random source-machine/input/reservoir cases	1,000
Universal-machine input prefixes	125
Reaction microsteps compared with source semantics	67,943
Doubling threshold and endpoint certificate cases	63
Single-coordinate witness mutations rejected	420
Expanded quartic versus residual-sum random evaluations	10
Bounded minimum-fuel cases, including zero fuel	20
Spurious zero-branch certificate value	1
Overall test result	PASS

The source/network comparison checks the phase invariant, exact register values, exact reservoir value, total conservation, absence of alternative permitted reactions, and the one/three-firing macro lengths. Random source machines include cycles and blocked runs; they are not all terminating toy programs.

The 420 mutation tests independently increase each coordinate of the successful eight-step witness by one and confirm that the polynomial no longer vanishes. This is useful corruption detection, but it is not an exhaustive uniqueness proof. Uniqueness is established by Theorem 116.2. Similarly, bounded universal-machine prefixes do not prove the inherited universality theorem.

120.3 Reproduction and artifact formats

From the extracted package directory, run

```
python code/reaction_compiler.py --output examples
python code/verify.py --output verification/results.json
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

[write] *This command listing is the manuscript's own and keeps its delivered layout, in which the programs import one another by their delivered names; the shipped copies carry the prefix 11-reaction-fallback- and must be copied back to that layout, outside the collection, before they are run. Where python does not resolve, use py. The README of this report gives rerun recipes that write only into the copy. The manuscript's own article.tex is not shipped; its text is this Part.*

The Python programs require Python 3.10 or later and only the standard library. They do not call a computer algebra system, an SMT solver, or a network service. Integer arithmetic is exact.

The JSON polynomial format stores each monomial as an integer coefficient and a list of variable names, with repetition recording powers. Thus a record with variables $[x, x, y]$ denotes its coefficient times x^2y . The residual file stores the quadratic expressions separately; the quartic file stores their fully combined sum of squares. For larger horizons the residual presentation is preferable because it avoids unnecessary expansion.

The complete fixed universal reaction list is supplied both as readable text and as JSON. The minimum-certificate constructor is executable; the large universal Q_L instances are specified by the generator rather than materialized as enormous expanded files.

121 A concrete integration plan for ProveIt [11]

The useful formalization boundary is between semantic correctness, finite-horizon witness uniqueness, and unbounded existence. These should be separate declarations, so an existence theorem cannot be mistaken for a multiplicity theorem.

Operational layer. Represent markings as finite vectors of naturals and reactions by their consumption and production vectors. Define enabling first and priority permission second. The compiled initial invariant should include the four phase forms, not merely total molecule conservation. Prove the local one/three-step simulation theorem and the exact threshold theorem by induction on source execution.

Bounded decision layer. The explicit configuration bound supplies a totalized bounded evaluator. Its primitive recursiveness gives H and G as primitive-recursive predicates. This is a direct client of the primitive-recursive-to-Diophantine route described by the repository's MRDP guide [5]. It avoids

an unnecessary appeal to informal “Turing completeness” when the bounded predicates are actually decidable.

Polynomial layer. Formalize the zero gadget with uniqueness, the enabling flags, and the row-update polynomial. A finite sum-of-squares lemma gives the quartic combination. The key new theorem should identify the solution set with the set of totalized traces and then specialize that bijection to deterministic legal starts. This explicitly tracks variables that otherwise become unconstrained on inactive branches.

Unbounded existence layer. The inspected `DiophantineTrace.lean` provides `exactIter_dioph` and `existsExactIter_dioph`, together with bounded universal closure [3]. These interfaces establish Diophantineness under their hypotheses; their statements do not promise unique or finite fibers. They can be used for an independent existence proof of reachability after the reaction relation is encoded.

Multiplicity layer. Theorem 118.1 should be formalized as an equivalence between explicit propositions. A declaration asserting global finite-foldness of G would be a major additional theorem, not routine compiler glue. The local Q_L theorem does not justify such a declaration. In particular, one must not introduce an unproved finite-fold bounded-quantifier eliminator as if it were a harmless coding lemma.

A plausible module split is reaction semantics, compiler correctness, resource bounds, quartic trace certificates, and finite-fold normal forms. These are proposed modules; no corresponding new Lean files or successful Lean build are claimed in the delivered package.

122 Further research questions and priorities [11]

[write] *Manuscript 11 posed its seven questions under subsection headings; those headings are the titles here.*

Research question 122.1 (A verified compiler with a solution-set bijection; source 11). The most concrete next target is a Lean formalization of Theorems 112.3, 113.1, and 116.2, including executable extraction of the reaction list and the quadratic residuals. The deliverable should be stronger than a reachability equivalence: it should preserve the exact source trace and provide a bijection between canonical traces and polynomial witnesses. The local zero-gadget lemma and the phase invariant make this a modular project.

[write] Compare Research question 71.1.

Research question 122.2 (Optimize under the exact restrictions, not a different model; source 11). Can the 61-species, 62-reaction bound be lowered while retaining one shared low-priority reaction, two distinct unit reactants, total two-to-two conservation, deterministic legal executions, and the exact peak-reserve formula? Removing a source-machine state may save several internal species and reactions, but combining source instructions can change the time and resource correspondence. Any claimed improvement should state which properties survive. Existing small universal multiset systems use different execution conventions and cannot be compared by rule count alone [53].

[write] Compare Research question 71.47.

Research question 122.3 (Sharper state bounds and sparse certificates; source 11). The bound $62 \binom{m+8}{8}$ counts many unreachable storage/phase pairs. One can exploit source-control invariants, permanently zero registers in specialized programs, or a verified reachable-state enumeration to reduce it. On the polynomial side, one-hot control and known phase constants can remove many zero flags. The goal is an explicit size improvement together with a proof that no formerly unique witness becomes unconstrained.

Research question 122.4 (Global finite-fold compression; source 11). Theorem 118.1 identifies the ambitious arithmetic target: a fixed finite-fold polynomial for the canonical fuel graph. A viable strategy must control the witnesses introduced when a variable-length trace is packed and its digitwise conditions are eliminated. A canonical trace code alone is insufficient if the arithmetic representation of digit operations or exponentiation has uncontrolled fibers. The exponential single-fold constructions discussed in [10] provide a comparison point, but replacing their exponential component while preserving multiplicity is precisely part of the unresolved issue.

[write] Compare Research questions 71.48 and 71.50.

Research question 122.5 (Multiple resources and alternative successful computations; source 11). For nondeterministic substrates, resource tradeoffs can produce several Pareto-minimal successful vectors. Theorem 119.1 gives a decidable local minimality test under upward closure, and the finite-fold implication survives because the frontier is finite. A useful next result would combine that argument with an explicit compiler for a nondeterministic substrate, tracking trace multiplicity separately from the number of minimal resource vectors. Single-fold normalization would require an additional canonical choice mechanism.

Research question 122.6 (Exact priority versus robust physical computation; source 11). An actual molecular implementation needs a different correctness statement from the ideal priority model. One research direction is a probabilistic compiler with a finite error bound for a specified time and resource horizon, including both opportunities for premature fallback in each positive-test macro. Another is an externally controlled implementation whose priority mechanism is part of the formally modeled hardware. Neither should be advertised as an exact autonomous realization of the present reactions merely by assigning widely separated finite rates.

[write] Compare $\text{ppc} : \text{q} : \text{continuous}$, the question on continuous-time and robust physical semantics in the neighbouring report *probabilistic-quantum-and-continuous-computation*.

Research question 122.7 (Substrate-transfer theorems; source 11). The same arithmetical pipeline can be applied to a tag system, a combinatory-logic evaluator, a membrane system, or another unconventional machine once a precise effective translation to register instructions is provided. A stronger transfer theorem would track resource overhead: for example, a verified upper and lower relation between source tape span and the compiled reservoir threshold. Such a theorem would turn the reaction normal form into a quantitative comparison tool, rather than just another proof that both models enumerate the same sets.

123 Conclusion [11]

A single shared low-priority reaction is enough to implement all the zero tests of a universal register machine, while every reaction in the closed realization conserves molecule count and has exactly two reactants and two products. The explicit construction has 61 species and 62 rules, and its least initial reserve is exactly a peak-population statistic of the source computation. Ordinary priority-free coverability cannot realize the same universal acceptance format.

At bounded horizons the semantics admit fully explicit single-fold quartic certificates; at bounded resource cutoffs, two canonical traces certify minimum fuel with unique, explicitly bounded witnesses. The minimum-fuel graph remains a low-complexity decidable relation. Yet a fixed finite-fold polynomial representation of that graph is equivalent to the full finite-fold conjecture. This separation between easy local certification and difficult global multiplicity is the main arithmetic lesson of the construction.

The package supplies the proofs, the full fixed reaction list, a standard-library compiler, an expanded example quartic and witness, and executed verification results. It provides a concrete research

contribution and a precise route for formalization, while leaving the genuinely open global multiplicity problem explicitly open.

124 The universal source machine and a source-table discrepancy [11]

[write] *Appendix A of manuscript 11.*

The machine used here has registers $0, \dots, 7$, start label 0 , halt label H , and the following instructions. In a DEC row, the final two columns are the positive and zero destinations. In an INC row, only the first destination is used.

Label	Instruction	Register	Positive/next	Zero
0	DEC	1	1	2
1	INC	7	0	–
2	INC	6	3	–
3	DEC	5	2	4
4	DEC	6	5	3
5	INC	5	6	–
6	DEC	7	7	8
7	INC	1	4	–
8	DEC	6	9	0
9	INC	6	10	–
10	DEC	4	0	11
11	DEC	5	12	13
12	DEC	5	14	15
13	DEC	2	18	19
14	DEC	5	16	17
15	DEC	3	18	20
16	INC	4	11	–
17	INC	2	21	–
18	DEC	4	0	H
19	DEC	0	0	18
20	INC	0	0	–
21	INC	3	18	–
H	HALT	–	–	–

The initial registers are $(0, e, x, 0, 0, 0, 0, 0)$; register 0 is the output. The paper uses only the halting property. This table follows the flowchart in Figure 1 of [53], with the following relabeling of its states:

here	0	1	2	3	4	5	6	7	8	9	10
source	1	3	6	4	7	9	10	12	13	33	14
here	11	12	13	14	15	16	17	18	19	20	21
source	16	18	23	20	27	22	30	32	25	29	31

The source halt q_f is relabeled H .

There is a relevant discrepancy in the inspected arXiv PDF. Its flowchart on printed page 4 routes the zero outcome at q_{27} through q_{29} , which increments register 0 before returning to q_1 . The adjacent

printed instruction list on page 5 instead gives q_1 directly as that zero destination. We inspected both page images and follow the flowchart. Thus our label 15 has zero target 20, not 0. Merely copying the adjacent table would bypass the register-0 increment and leave its listed state q_{29} unreachable from the start. The inherited universality dependency is the flowchart construction, not an assertion that the conflicting literal table is correct.

[write] The source discrepancy was not re-inspected for this report and is printed as manuscript 11’s statement. The table above corroborates it structurally: label 20, the relabelled q_{29} and the only increment of register 0, has a single incoming edge, the zero branch of label 15, so copying the adjacent printed list would leave it unreachable.

The earlier eight-register input/output convention is also documented in later work using the 23-label version [54]. The 61/62 compiler bounds depend on 9 increments and 13 conditional decrements, which are counted directly in the displayed table. We do not claim to reprove Korec’s coding-based universality theorem in this appendix.

125 Manifest and assurance levels [11]

[write] *Appendix B of manuscript 11. The shipped names are printed; the manuscript’s `article.tex`, `article.pdf` and `README.md` are not shipped (its text is this Part and Section 3.11; this report has its own `README`).*

File	Purpose
<code>article.tex</code> , <code>article.pdf</code>	Full mathematical article and compiled PDF.
<code>code/11-reaction-fallback-reaction_compiler.py</code>	Typed standard-library compiler, simulator, sparse polynomial arithmetic, bounded and minimum-resource certificate generators.
<code>code/11-reaction-fallback-verify.py</code>	Deterministic tests, including exact source/network comparisons and adversarial certificate checks.
<code>data/11-reaction-fallback-universal_reactions.txt</code>	All 62 compiled reaction rules with priority labels.
<code>data/11-reaction-fallback-universal_61_species_62_reactions.json</code>	Machine-readable species and reaction data.
<code>data/11-reaction-fallback-doubling_T8_quartic.json</code>	Expanded quartic in the two input parameters and 420 natural witness variables.
<code>data/11-reaction-fallback-doubling_T8_residuals.json</code>	The 781 quadratic residuals whose squared sum is that quartic.
<code>data/11-reaction-fallback-doubling_T8_witness.json</code>	Complete successful assignment for input $n = f = 1$.
<code>data/11-reaction-fallback-example_summary.json</code>	Generated counts and degree check.
<code>data/11-reaction-fallback-results.json</code>	Executed test counts and pass status.
<code>README.md</code>	Build commands, semantics, provenance, and limitations.

The assurance levels are intentionally separate. The algebraic and operational arguments in the article are proofs. The Python checks verify specific finite computations and implementation properties. Universality of the chosen source program is an inherited published result, with the source discrepancy

disclosed. The global MRDP representation is an existence consequence, not a generated artifact. The finite-fold and single-fold conjectures are not claimed solved. No new Lean verification is claimed.

Part XIV

History-free routing certificates: last exits, compressed periods and a rank-function bottleneck

Source and scope. [write] This Part is manuscript 12, “Diophantine Certificates Without Execution Histories: Single-fold quartics for succinct routing, and a universal one-router boundary” (batch 63, manuscript 02; archive `Diophantine_Certificates_Without_Histories.zip`, inner directory `diophantine_certificates`; arrival commit `a4268e78e`; pinned at `e18718e83`; shipped files prefixed `12-rle-routing-`; labels `cdc:rt:`). It prints the manuscript’s Sections 2–11 in their own order: the zero-retention stack model, least action, and the exact odometer criterion by balance and last exits (Theorem 127.3), with canonical heights (Lemma 127.4); the parametric run-length quartic with exactly $4n + 4m + s$ witnesses (Theorem 128.1); the grammar quartic (Theorem 129.1); certificate height, canonical certificates for both outcomes (Theorem 130.2), the Presburger description of fixed periods and a transport identity; the circuit counting barrier (Theorem 131.2); the universal one-router system (Theorem 132.1); the rank-function bottleneck (Theorem 133.1); the executable artifacts and a Lean integration plan; the eight research questions; and the conclusion. Its two appendices follow: reproduction instructions and a dependency map. Its title, abstract and introduction are in Section 3.12; the conventions of this Part are Section 126.

Relation to the other Parts. [write] Manuscript 12 inspected ProveIt at `e18718e83`, when this report had been placed but not yet written, and does not cite it. It proves the report’s kind of theorem for a new substrate, finite chip-routing (rotor or switching) networks with absorbing sinks: an explicit quartic with an exact witness count whose natural zero set above each parameter tuple is empty or a singleton, the root carrying the exact firing counts and sink outputs. Two differences from Parts I–XIII matter. First, the certified object is an outcome, not a history: the root holds the odometer (the number of firings at each vertex) and a canonical last-exit forest, and no schedule, trace or step-indexed row appears. Order independence of routing (Corollary 127.2) makes this possible, where Part II instead quotients schedules by commutation. Second, the polynomial is uniform in unbounded parameters: for a fixed routing topology the initial loads and all run lengths are free inputs, so the arity is fixed while the number of firings is unbounded. It is therefore a sibling of the fixed-arity accelerators of Part III (Theorems 21.1 and 25.1) and of Part XII’s flat corollary (Corollary 99.1), in which repetition counts are also parameters; neither contains the other, since there the schedule is supplied and here the firing counts are witnesses. The Presburger description of fixed periods (Proposition 130.3) parallels that of flat schemes (Theorem 26.1); the circuit counting barrier (Theorem 131.2) sits beside Part II’s counting completeness (Theorem 13.1); and the certificates for both outcomes (Theorem 130.2) are a second route to the $UP \cap coUP$ bound that the manuscript credits to the ARRIVAL literature. The boundary half continues Part IX: Theorem 132.1(iv) is the routing form of Proposition 62.2, and Theorem 133.1 is a history-free form of Theorem 62.1(c), with the graph of one total, monotone, Boolean, polynomial-time rank function in place of the first-halting trace verifier; Theorem 118.1 (Part XIII) and Proposition 64.1 (the graph of 2^n) prove the same pattern for other relations (see the paragraph after Theorem 133.1). Its observation that being eventually periodic and being supplied with a periodic description are different hypotheses parallels Part XII’s completeness of eventual periodicity (Theorem 101.1). The Part partly answers the question on restricted unbounded classes (Research question 71.49) and an analogue of the question on variable periods (Research question 71.37), and re-scopes the questions on fixed-arity compression (Research question 71.48) and on finite-fold arithmetic primitives (Research question 71.50); see the notes there. The neighbouring report *liveness-beyond-halting* of this category shares only the word “clock” with this Part (see the conventions): termination of a finite periodic routing network is decidable (the remark after Proposition 130.3), far below that report’s hierarchy of infinite

behaviour.

Setting and hypotheses. [write] Finite sets of active vertices and of absorbing sinks; every active vertex has an infinite destination stack; a firing removes one chip from an occupied vertex and sends it to the next destination of that vertex's stack; sinks never fire, and no chip is retained at an active vertex. All witnesses are natural, and the single-fold statements are not claimed for integer witnesses (Section 128). The compilers need explicitly supplied periods, as run-length blocks (Theorem 128.1) or one straight-line grammar per vertex (Theorem 129.1); circuit-defined periods (Section 131) and the algorithmic stacks of the universal clock (Section 132) are the boundary. Each quartic has fixed arity for one topology, and its size grows with the topology description; it is not one polynomial for all networks. The bottleneck theorem is an equivalence between open statements, not a proof of either.

126 Conventions of Part XIV

[write] The letters are manuscript 12's, with one renaming: its Cantor pairing, written π in the manuscript, is written π_C here, because Part IX's π (Equation (241)) is the different injection $(a + b)^2 + a$. Its names SF and FF are Part XIII's and mean the same statements. The tempting false readings are to carry Part IX's history h or a horizon T into this Part: here h_v is a forest height and T is determined by the input.

Symbol	Meaning in Part XIV, and the clash
π_C	Cantor's pairing $\pi_C(e, n) = \frac{1}{2}(e + n)(e + n + 1) + n$, (132.1); the manuscript writes π . It is Part VII's pairing $\pi(\ell, r) = \frac{1}{2}(\ell + r)(\ell + r + 1) + r$ (Section 55) with $(\ell, r) = (e, n)$, and not Part IX's $\pi(a, b) = (a + b)^2 + a$ (Equation (241)).
$\mathbf{u}, \mathbf{U}, \mathcal{U}$	a candidate odometer, the genuine odometer (firings per active vertex), and a fixed universal Turing machine. Part IX writes U for its universal machine; in Part I, u_a is an upper guard.
h_v	the canonical height of v in the last-exit forest (Lemma 127.4), at most n . Not the history h of Part IX's $\text{Trace}_U(e, x, h)$, nor Part II's causal height H .
$T, T(c)$	the total number of firings of an execution (Lemma 130.1), and the first halting time of $\mathcal{U}$ on input c . Both are determined by the input; neither is a horizon given with the instance, as T is in Parts II, IV, VI–VIII, X and XIII.
P, P_{vj} $R_{vw}(t), \mathcal{R}, R$	the fixed rank function (132.2) and the block offsets (128.2); neither is a polynomial. prefix routing counts; the list of residuals; and, in the trap collapse, the set of active vertices that reach a sink. Not Part II's residual counts R and R_2 , nor Part XIII's number of reactions.
$L_u(v), L_A, \mathcal{L}, L$	the last exit of v ; the length of the word of grammar node A ; the matrix $D - C^T$ of the transport identity; a run length in Example 128.3. Part V's L is a log length.
D_v, C_{vw}, B_{vjw}, D	the period length, the routing counts per period and before block j , and $D = \text{diag}(D_v)$. Part I's D is a dependence relation.
n, s, S, m	the numbers of active vertices, sinks and run-length blocks, and the sink set; s also names one sink.
c	the input code of $\mathcal{U}$ (Sections 132 and 133), and the number of concatenation nodes of the grammars (Section 129); the two uses do not meet.
SF, FF	every computably enumerable subset of $\mathbb{N}$ has a single-fold (finite-fold) Diophantine representation: statement (a) of Theorem 62.1 in its single-fold (finite-fold) version, as in Part XIII.
G, H_e	a hypothetical representation of the graph Γ_P , and the polynomial (133.1) built from it. H_e is of the kind of $H(e, x, z)$ in Theorem 62.1(b); G is not Part XIII's least-reservoir relation G .
clock	the exponentially dilated universal clock of Section 132; not the Clock predicate or the deadline classes of the neighbouring report <i>liveness-beyond-halting</i> .

Symbol	Meaning in Part XIV, and the clash
routing	chip routing through destination stacks; not the comparator routing of Part V's question Research question 71.26.

No other symbol of manuscript 12 is renamed. Its macros for bits, supp and \code are this report's; P, UP, coUP, NP and coNP are its own. Its equations are numbered within sections, as in the manuscript and in Parts X–XIII.

127 A history-free characterization of exact routing [12]

127.1 The zero-retention stack model

Fix disjoint finite sets V of active vertices and S of absorbing sinks, with $|V| = n$ and $|S| = s$. Each active vertex has an infinite destination stack

$$\rho_v : \{1, 2, 3, \dots\} \longrightarrow V \sqcup S.$$

An initial configuration is $\mathbf{x} = (x_v)_{v \in V} \in \mathbb{N}^V$. Sinks initially contain no chips, except in the explicitly stated trap-collapse construction later. Each active vertex also has a firing counter, initially zero. A legal firing at v is allowed when v contains at least one chip. It removes one chip, increments the counter to j , and sends that chip to $\rho_v(j)$. A sink never fires. The rule retains no chip permanently at an active vertex.

A finite execution is *complete* when all active vertices are empty. The number of firings at each active vertex is its *odometer* $\mathbf{u} \in \mathbb{N}^V$. We say the input terminates when a finite complete legal execution exists. We will prove that in that case every maximal legal execution is finite and has the same odometer. No fairness hypothesis is needed for this statement.

For $t \in \mathbb{N}$, define the prefix-routing counts

$$R_{vw}(t) = \#\{j : 1 \leq j \leq t, \rho_v(j) = w\}, \quad w \in V \sqcup S. \quad (127.1)$$

They are nondecreasing in t , vanish at zero, and satisfy $\sum_w R_{vw}(t) = t$. A candidate odometer $\mathbf{u}$ is *balanced* when

$$u_v = x_v + \sum_{a \in V} R_{av}(u_a) \quad (v \in V). \quad (127.2)$$

This is conservation of chips with final active occupancy zero. The associated sink count is

$$y_s = \sum_{v \in V} R_{vs}(u_v), \quad s \in S. \quad (127.3)$$

When $u_v > 0$, define the last exit $L_{\mathbf{u}}(v) = \rho_v(u_v)$. The selected last-exit graph has one outgoing edge at every vertex in $\text{supp}(\mathbf{u})$ and no outgoing edge at the other active vertices or sinks. Acyclicity means that this selected graph contains no directed cycle.

127.2 Least action, existence, and uniqueness

Lemma 127.1 (Least action for balanced candidates). *If $\mathbf{u}$ satisfies (127.2), the firing-count vector of every finite legal execution prefix is componentwise at most $\mathbf{u}$. Consequently a finite complete legal execution exists and has odometer at most $\mathbf{u}$.*

Proof. Suppose a legal sequence is about to exceed $\mathbf{u}$ for the first time, at vertex v . Just before that firing its count vector $\mathbf{a}$ satisfies $a_v = u_v$ and $a_w \leq u_w$ for all w . The number of chips currently at v is

$$x_v + \sum_w R_{vw}(a_w) - a_v \leq x_v + \sum_w R_{vw}(u_w) - u_v = 0.$$

But a legal firing needs a positive number of chips there, a contradiction. Thus no legal sequence can exceed $\sum_v u_v$ firings. Repeatedly choose any occupied active vertex, for example the least one in a fixed order. The procedure must stop after finitely many firings and can stop only when all active vertices are empty. $\square$

Corollary 127.2 (Order independence). *If there is a complete legal execution, its odometer $\mathbf{U}$ and all sink outputs are independent of the legal firing order. Every maximal legal execution is finite.*

Proof. A complete execution gives a balanced vector. Lemma 127.1 bounds all legal executions by that vector. If $\mathbf{U}$ and $\mathbf{U}'$ are two complete odometers, applying the lemma in both directions gives $\mathbf{U} \leq \mathbf{U}'$ and $\mathbf{U}' \leq \mathbf{U}$. The sink formula then fixes every output. The same finite bound excludes an infinite execution. $\square$

Balance alone proves the existence of *some* termination, but need not make a proposed count vector exact. This distinction is crucial: a balanced flow may add an artificial circulation to the genuine odometer.

Theorem 127.3 (Exact odometer from balance and last exits). *For arbitrary stacks on a finite active set, a vector $\mathbf{u} \in \mathbb{N}^V$ is the odometer of a complete legal execution if and only if it is balanced and its selected last-exit graph is acyclic.*

Proof. First suppose $\mathbf{u}$ comes from a complete execution. Balance is conservation. If the last departure from v sends a chip to an active vertex w , there must be a departure from w strictly later: otherwise that newly arrived chip would remain when the execution ends. Hence the times of last departures strictly increase along selected active edges. They cannot strictly increase around a directed cycle.

Conversely, suppose $\mathbf{u}$ is balanced with acyclic last exits. Lemma 127.1 produces the genuine odometer $\mathbf{U} \leq \mathbf{u}$. Write $d_v = u_v - U_v$ and $W = \{v : d_v > 0\}$. Define residual routing counts

$$E_{vw} = R_{vw}(u_v) - R_{vw}(U_v) \geq 0.$$

Subtracting the two balance equations gives

$$d_w = \sum_{v \in V} E_{vw} \quad (w \in V), \quad \sum_{w \in V \cup S} E_{vw} = d_v. \quad (127.4)$$

If $w \notin W$, its incoming residual sum is zero, so every E_{vw} into it is zero. Summing (127.4) over the active vertices shows also that the total residual flow to sinks is zero. Finally, when $v \in W$, the last symbol $\rho_v(u_v)$ belongs to the nonempty residual suffix $U_v + 1, \dots, u_v$. It therefore cannot point to a sink or outside W . If W were nonempty, every vertex of the finite set W would have its selected successor inside W . Iterating successors would produce a directed cycle. Acyclicity excludes this possibility, so W is empty and $\mathbf{u} = \mathbf{U}$. $\square$

This proof is a sink-routing version of the classical least-action and last-exit methods [57, 59, 60]. Its advantage for compilation is that it refers to neither a schedule nor a temporal tableau.

127.3 Canonical heights rather than arbitrary rankings

An acyclic graph can be certified by a topological ordering, but that ordering usually has many valid choices. To preserve single-foldness we instead use exact distances along the selected functional graph.

Lemma 127.4 (Canonical forest certificate). *For a candidate $\mathbf{u}$, its last-exit graph is acyclic if and only if there exist natural heights h_v satisfying*

$$h_v = \begin{cases} 0, & u_v = 0, \\ 1, & u_v > 0, L_{\mathbf{u}}(v) \in S, \\ 1 + h_w, & u_v > 0, L_{\mathbf{u}}(v) = w \in V. \end{cases} \quad (127.5)$$

When they exist, the heights are unique and at most n .

Proof. Adding the height differences around a cycle of length $r > 0$ would give $0 = r$, which is impossible. Conversely, an acyclic selected graph has a unique finite path from each active vertex until it reaches either a sink or a vertex with no selected outgoing edge. Set its height to that path length. This satisfies the equations and is forced by following the path backwards. A path visits at most n active vertices. $\square$

For balanced vectors, a selected edge cannot end at an inactive active vertex: such an arrival would give that vertex strictly positive incoming flow but zero odometer. Hence the nonempty paths of a valid complete certificate actually end at sinks.

128 A parametric single-fold quartic for run-length routing [12]

128.1 Topology, parameters, and the nonzero remainder convention

Fix the destination lists of a finite routing topology. At vertex v , let its period consist of $m_v \geq 1$ blocks

$$W_v = w_{v1}^{\ell_{v1}} w_{v2}^{\ell_{v2}} \cdots w_{vm_v}^{\ell_{vm_v}}, \quad \ell_{vj} \geq 1. \quad (128.1)$$

Adjacent blocks are allowed to have the same destination; the supplied block boundaries are part of the description. Set $m = \sum_v m_v$. The stack is the infinite repetition of W_v , starting with the first symbol.

To make every parameter natural, write $\ell_{vj} = \lambda_{vj} + 1$ with $\lambda_{vj} \in \mathbb{N}$. The initial loads $\mathbf{x}$ and the m values λ will be *free parameters*, not witnesses. Define

$$\begin{aligned} D_v &= \sum_j \ell_{vj}, & P_{vj} &= \sum_{i < j} \ell_{vi}, \\ C_{vw} &= \sum_{j: w_{vj}=w} \ell_{vj}, & B_{vjw} &= \sum_{i < j: w_{vi}=w} \ell_{vi}. \end{aligned} \quad (128.2)$$

These are linear polynomials in the run-length parameters. When lengths are fixed, they are simply nonnegative integer coefficients.

For $u_v > 0$ we use the unique decomposition

$$u_v = D_v q_v + r_v, \quad q_v \geq 0, \quad 1 \leq r_v \leq D_v. \quad (128.3)$$

Thus $q_v = \lfloor (u_v - 1)/D_v \rfloor$. In particular, at a period boundary $r_v = D_v$, not zero. This convention identifies the actual last symbol without a special wraparound case.

128.2 The complete residual system

The natural witness variables are

$$\begin{aligned} u_v, q_v, z_v, h_v \quad (v \in V), \\ b_{vj}, t_{vj}, \alpha_{vj}, \beta_{vj} \quad (1 \leq j \leq m_v), \\ y_s \quad (s \in S). \end{aligned} \tag{128.4}$$

Here z_v marks a nonzero odometer, b_{vj} selects the block containing the last symbol, and t_{vj} is the one-based position within that selected block. Unselected blocks have all four coordinates zero.

Impose the following equations at every vertex:

$$z_v(z_v - 1) = 0, \tag{128.5}$$

$$\sum_j b_{vj} - z_v = 0, \tag{128.6}$$

$$q_v(1 - z_v) = 0, \tag{128.7}$$

$$u_v - D_v q_v - \sum_j (P_{vj} b_{vj} + t_{vj}) = 0. \tag{128.8}$$

At each block impose

$$t_{vj} - b_{vj} - \alpha_{vj} = 0, \tag{128.9}$$

$$t_{vj} + \beta_{vj} - \ell_{vj} b_{vj} = 0. \tag{128.10}$$

Because all witnesses are natural, (128.5)–(128.6) force $b_{vj} \in \{0, 1\}$ without separate Boolean equations. If $b_{vj} = 0$, (128.9)–(128.10) force $t_{vj} = \alpha_{vj} = \beta_{vj} = 0$. If $b_{vj} = 1$, they give exactly

$$1 \leq t_{vj} \leq \ell_{vj}, \quad \alpha_{vj} = t_{vj} - 1, \quad \beta_{vj} = \ell_{vj} - t_{vj}.$$

The slacks are uniquely determined, including in the inactive case.

Define, without adding new witness variables, the routing expressions

$$F_{vw} = C_{vw} q_v + \sum_j B_{vjw} b_{vj} + \sum_{j: w_{vj}=w} t_{vj}. \tag{128.11}$$

They count complete periods, complete blocks before the selected block, and the selected partial block. These expressions have degree at most two when run lengths are parameters, and are linear when lengths are fixed. The remaining equations are

$$h_v - z_v - \sum_{j: w_{vj} \in V} b_{vj} h_{w_{vj}} = 0 \quad (v \in V), \tag{128.12}$$

$$u_v - x_v - \sum_{a \in V} F_{av} = 0 \quad (v \in V), \tag{128.13}$$

$$y_s - \sum_{v \in V} F_{vs} = 0 \quad (s \in S). \tag{128.14}$$

Let $\mathcal{R}$ be the list of all left-hand sides in (128.5)–(128.14), with their stated multiplicities. Define

$$Q_{\mathcal{T}}(\mathbf{x}, \lambda; \mathbf{w}) = \sum_{R \in \mathcal{R}} R^2. \tag{128.15}$$

The subscript $\mathcal{T}$ denotes the fixed destination-list topology. No floor, divisibility predicate, quantifier, or machine simulator remains in this definition: it is an ordinary integer polynomial.

Theorem 128.1 (Parametric canonical RLE quartic). *For every finite destination-list topology with n active vertices, s sinks, and m blocks, the polynomial (128.15) has the following properties.*

- (i) *Its total degree in both parameters and witnesses is at most four. There are exactly $4n + 4m + s$ natural witness coordinates, $n + m$ natural parameters, and $6n + 2m + s$ residuals of degree at most two.*
- (ii) *For every initial load $\mathbf{x}$ and every run-length tuple $\ell = \lambda + \mathbf{1}$, the natural zero set of $Q_{\mathcal{T}}$ in all witness coordinates is empty if the input does not terminate, and is a singleton if it terminates.*
- (iii) *In that unique root, $\mathbf{u}$ is the exact odometer and $\mathbf{y}$ is the complete vector of sink outputs.*
- (iv) *The residual list, and also the expanded polynomial, have descriptions of size polynomial in the finite topology description. Specializing binary run lengths preserves polynomial description size in the specialized input length.*

Proof. Every residual displayed above has total degree at most two. The quantities D, P, C, B are linear in the free λ parameters, and every product involving them has only one further witness factor. Squaring establishes the degree bound. There are four coordinates per active vertex, four per block, and one per sink. There are six residuals per vertex, two per block, and one per sink, giving the stated counts.

A sum of integer squares is zero exactly when every summand is zero. Suppose first that all residuals vanish. If $z_v = 0$, the nonnegative sum of selectors in (128.6) is zero. The block equations force their offsets and slacks to vanish, the gate (128.7) forces $q_v = 0$, and (128.8) forces $u_v = 0$. Equation (128.12) then forces $h_v = 0$. If $z_v = 1$, exactly one selector is one. Its offset lies in $[1, \ell_{vj}]$, so (128.8) gives precisely the positive-remainder decomposition (128.3). In particular $u_v > 0$. The selected block and its offset are unique for that remainder, since the block intervals partition $\{1, \dots, D_v\}$.

Consequently $F_{vw} = R_{vw}(u_v)$ and the selected destination is exactly $\rho_v(u_v)$. Equation (128.13) is genuine balance, while (128.12) is the canonical height condition of Lemma 127.4. Theorem 127.3 therefore identifies $\mathbf{u}$ with the true complete odometer. The sink equations identify $\mathbf{y}$.

Conversely, a terminating execution has a unique odometer. Use its positive remainders to set q, z, b, t , set the two slacks to their forced differences, and take the unique forest heights. The exact prefix formula and conservation make every residual zero. For fixed parameters, every coordinate has now been shown to be uniquely forced by the odometer. This proves both existence and single-foldness.

Finally, the expressions in (128.2) and (128.11) are finite sums with at most polynomially many terms in the topology size. Squaring an explicitly listed polynomial-size residual multiplies its term count by at most a polynomial factor. Substitution of binary lengths only requires additions and multiplications of integers of polynomial bit length. There is no expansion of a word of length D_v , and no unrolling of $\sum_v u_v$ firing steps. $\square$

Remark 128.2 (What is and is not linear in the description). The exact variable and residual counts are linear in $n + m + s$. The number of terms inside a residual need not be bounded by a constant, and the expanded quartic need not have only $O(n + m + s)$ monomials. The polynomial-size claim is deliberately weaker and is the one proved above.

128.3 Why natural witnesses and canonical gates are essential

The construction uses nonnegativity, not just equations over a field. For example, $\sum_j b_{vj} = 0$ forces every selector to vanish only because the selectors are natural. Similarly, two linear slack equations express an interval only with nonnegative slack variables. The sum of squared residuals can be evaluated over $\mathbb{Z}$, but its intended witness domain is $\mathbb{N}$. A routine replacement of natural variables by sums of four integer squares preserves existence and generally destroys uniqueness: the same natural number

can have several square decompositions. We make no corresponding single-fold claim over unrestricted integer witnesses.

The inactivity gate $q_v(1 - z_v) = 0$ is indispensable. Without it, take an initially empty one-vertex self-routing network with period length D . The values $z = b = t = h = 0$, $q = 1$, and $u = D$ satisfy the other displayed local conditions and balance, inventing a complete cycle of firings without a chip. The gate eliminates this fake execution. Likewise, replacing the exact height equation by the inequality $h_v > h_w$ would typically admit infinitely many rankings for the same true odometer, defeating single-foldness.

128.4 A large exact example with thirteen unknowns

Example 128.3. Take one active vertex v , one sink s , and period $v^L s$ with $L \geq 1$. Put M chips at v . Each chip is absorbed after $L + 1$ firings, so

$$U = M(L + 1), \quad y = M.$$

For $M > 0$ the unique witness is

$$q = M - 1, \quad z = h = 1, \quad (b_1, b_2) = (0, 1), \quad (t_1, t_2) = (0, 1),$$

with all four block slacks zero. For $M = 0$, every witness coordinate is zero. There are $4 + 8 + 1 = 13$ witness variables and $6 + 4 + 1 = 11$ residuals.

The supplied checker evaluates this certificate with

$$L = M = 2^{4096}, \quad U = 2^{8192} + 2^{4096}.$$

It obtains exactly zero for the sum of the eleven squared residuals, using ordinary arbitrary-precision integer arithmetic. It does not simulate the U firings. The integer U has 8193 binary digits, which is a manageable certificate size despite the infeasible execution length.

Example 128.4 (A spurious balanced circulation). Let a have period (b, s) and b have period (a, s) , with no initial chips. Both active vertices have paths to the sink. The proposed odometer $(u_a, u_b) = (1, 1)$ is balanced: each alleged firing supplies the other one. Its last-exit graph is the cycle $a \rightarrow b \rightarrow a$, so no heights can satisfy $h_a = 1 + h_b$ and $h_b = 1 + h_a$. The genuine odometer is $(0, 0)$. Thus even global accessibility of a sink does not make balance alone an exact execution certificate.

129 Shared grammar periods without word expansion [12]

129.1 The representation and its useful statistics

Run lengths compress long constant blocks. A straight-line grammar can compress a different kind of repetition, such as a word obtained by repeated doubling of a nonconstant pattern. For each active vertex v , let its period be specified by a finite acyclic grammar whose nodes have one of two forms:

$$A \longrightarrow w \quad (w \in V \sqcup S), \quad A \longrightarrow BC.$$

Every node defines exactly one nonempty word. Children occur before parents in a topological order, and one distinguished root A_v defines the period. Nodes may be shared within the grammar: the same subword can occur as a child of many different concatenations, including twice in the same concatenation. We count g_v nodes and c_v concatenation nodes at v , and set $g = \sum_v g_v$, $c = \sum_v c_v$. Thus the accounting includes a separate certificate coordinate system for each active vertex's grammar.

It does not claim that several independent prefix queries can share the same witness coordinates for a globally shared grammar.

Precompute the length L_A and symbol-count vector $(C_{Aw})_w$ by

$$L_w = 1, \quad C_{wz} = [w = z], \quad L_A = L_B + L_C, \quad C_{Az} = C_{Bz} + C_{Cz} \quad (A \rightarrow BC).$$

These calculations are polynomial in the grammar size and alphabet size. In a grammar with g_v nodes, every word has length at most 2^{g_v} ; therefore its lengths and counts have $O(g_v)$ bits. No word expansion is needed. Let $D_v = L_{A_v}$. [write] [The bound can be halved: a terminal defines a word of length one, a concatenation defines a word at most twice as long as the longer of its children's, and a descending path meets at most $c_v \leq g_v - 1$ concatenation nodes, so every word has length at most 2^{g_v-1} . The doubling example below, with $k + 3$ nodes and period 2^{k+1} , is within this bound.]

The key observation is that a positive prefix of one grammar word needs only one root-to-terminal descent path. Every right turn contributes the whole left subword to the prefix count. A prefix certificate should describe this path, not a derivation tree containing all repeated copies of the word.

129.2 Canonical path variables and equations

For each grammar node A , introduce natural variables

$$a_A, k_A, \eta_A, \theta_A.$$

Here a_A records whether the selected descent visits that node and k_A is its one-based local position. Impose

$$a_A(a_A - 1) = 0, \tag{129.1}$$

$$k_A - a_A - \eta_A = 0, \tag{129.2}$$

$$k_A + \theta_A - L_A a_A = 0. \tag{129.3}$$

An inactive node therefore has all four coordinates zero; an active node has $1 \leq k_A \leq L_A$ and unique interval slacks.

At a concatenation $A \rightarrow BC$, introduce four additional natural variables $l_A, r_A, \sigma_A, \tau_A$, and impose

$$l_A + r_A - a_A = 0, \tag{129.4}$$

$$\sigma_A - l_A(L_B - k_A) = 0, \tag{129.5}$$

$$\tau_A - r_A(k_A - L_B - 1) = 0. \tag{129.6}$$

Thus l_A, r_A are automatically Boolean. At an active node the left branch is possible exactly when $k_A \leq L_B$, and the right branch exactly when $k_A \geq L_B + 1$. Only one is possible, and the corresponding slack is its forced nonnegative difference. Every unused comparison slack is zero.

For every node X other than the root of its grammar, impose the incoming activation and position equations

$$a_X = \sum_{A \rightarrow XC} l_A + \sum_{A \rightarrow BX} r_A, \tag{129.7}$$

$$k_X = \sum_{A \rightarrow XC} l_A k_A + \sum_{A \rightarrow BX} r_A (k_A - L_B). \tag{129.8}$$

Each occurrence of X as a left or right child contributes in the appropriate sum. In particular, if a rule is $A \rightarrow XX$, the two occurrences are handled separately and only the selected occurrence is active.

At each active routing vertex retain the four variables u_v, q_v, z_v, h_v and impose

$$z_v(z_v - 1) = 0, \quad q_v(1 - z_v) = 0, \quad (129.9)$$

$$a_{A_v} - z_v = 0, \quad u_v - D_v q_v - k_{A_v} = 0. \quad (129.10)$$

The root offset is again the positive remainder; it is zero exactly for an inactive vertex. These equations do not impose a freely chosen path at each shared occurrence. The single activated root, unique branching, and acyclic node order force one path, with all other nodes inactive.

129.3 Prefix counts, last symbols, and global balance

Define

$$F_{vw} = q_v C_{A_v w} + \sum_{A \rightarrow BC} r_A C_{Bw} + \sum_{A \rightarrow w} a_A, \quad (129.11)$$

where both sums range over the grammar at v . A selected right turn adds the whole left word; the final selected terminal adds the last symbol. This accounts exactly for the selected prefix, in addition to the complete periods. The selected terminal is also the last-exit destination, so impose

$$h_v - z_v - \sum_{\substack{A \rightarrow w \\ w \in V}} a_A h_w = 0. \quad (129.12)$$

The active balance equations and sink equations are the same as (128.13)–(128.14), using (129.11). Once again, the final polynomial is the sum of the squares of all residuals.

Theorem 129.1 (Canonical grammar quartic). *For a network whose nonempty periodic words are specified by the grammars above, there is an explicitly constructible integer polynomial of total degree at most four, with*

$$4n + 4g + 4c + s \quad \text{natural witness coordinates}$$

and

$$4n + 5g + 3c + s \quad \text{residuals of degree at most two.}$$

For every natural initial load, the polynomial has exactly one natural root if the execution terminates and no natural root otherwise. Its root contains the exact odometer and every sink output. Construction time and polynomial description size are polynomial in the grammar description and alphabet size, not in the lengths of the expanded periods.

Proof. Start with the local path system. When the root is inactive, an induction from parents to children shows that every incoming activation sum is zero; all nodes are inactive and all their coordinates vanish. When the root is active, (129.1)–(129.6) force exactly one child at each active concatenation, with the uniquely appropriate translated offset. Acyclicity of the grammar ensures this process reaches a terminal without revisiting a node. All activations outside this path are zero: process the nodes from parents to children, noting that no node can become active without an active incoming selected edge. Sharing therefore causes no ambiguity.

Conversely, the usual descent to a position in the root word gives a solution to all local equations, and the same argument proves that this solution is unique. At a left descent the prefix lies entirely in the left child. At a right descent it consists of the entire left word followed by the selected prefix in the right child. Induction along the descent proves (129.11). Exactly one selected terminal supplies the final symbol.

The global equations now give genuine balance and the canonical last-exit heights. Theorem 127.3 proves soundness and identifies the odometer. Conversely, the true odometer determines a unique

positive remainder at every active vertex, a unique grammar path, and all its unique slacks. This proves completeness and single-foldness.

There are four variables per routing vertex, four per grammar node, four per concatenation, and one per sink. For the residual count, the node equations contribute $3g$, the concatenation equations contribute $3c$, and the nonroot incoming equations contribute $2(g - n)$. Root activation contributes n ; Boolean z , the quotient gate, the odometer decomposition, the height, and the balance contribute another $5n$; sinks contribute s . Their sum is $4n + 5g + 3c + s$. Every product contains at most two variables. The metadata have polynomial bit length, which also proves the description-size and construction bounds. $\square$

Example 129.2 (Shared repeated doubling). Start with a terminal a , a terminal b , and the word $A_0 \rightarrow ab$. Define $A_{i+1} \rightarrow A_i A_i$. The final period has length 2^{k+1} after k doublings, but the grammar has only $k + 3$ nodes. For a prefix of length t , its counts are $\lceil t/2 \rceil$ copies of a and $\lfloor t/2 \rfloor$ copies of b . The supplied checker uses 1024 doublings, giving a period of length 2^{1025} , and checks six prefix certificates without expanding that word. This tests the case where both children of every doubling node are the very same grammar node.

130 Certificate height, both outcomes, and fixed-period structure [12]

130.1 A binary height bound

Assume first that every active vertex has a directed path to a sink in the underlying graph of destinations that occur in its period. Put $M = \sum_v x_v$.

Lemma 130.1 (Absorption and a finite-state bound). *Under this accessibility assumption, every maximal legal execution terminates. For a deterministic rule that always chooses the least occupied active vertex, the total number T of firings satisfies*

$$T < (M + 1)^n \prod_{v \in V} D_v. \quad (130.1)$$

Proof. If a legal execution were infinite, some vertex would fire infinitely often. Let I be the nonempty set of such vertices. A periodic stack used infinitely often sends infinitely many chips to every destination occurring in its period. No sink can receive infinitely many chips, since the total number of chips is the fixed finite number M . Likewise, an active vertex receiving infinitely many chips must fire infinitely often; otherwise it would eventually hold more than M chips. Therefore I is closed under all underlying destination edges and has no path to a sink, a contradiction.

For the quantitative bound, a state relevant to future behavior consists of the active chip vector and each counter modulo D_v . There are at most $(M + 1)^n \prod_v D_v$ such states. Accumulated sink counts are not needed to determine future active dynamics. A deterministic run reaching termination cannot repeat a nonterminal state, or its future would repeat forever. Including the final state yields (130.1). $\square$

The bound can be crude in numerical value and still be useful for certificates. Taking binary logarithms gives

$$\log_2(T + 1) = O\left(1 + n \log_2(M + 1) + \sum_v \log_2 D_v\right). \quad (130.2)$$

Every odometer coordinate and quotient is at most T . RLE offsets and slacks are at most the corresponding period length; grammar offsets and slacks are at most a node-word length. Heights are at most n and sink outputs at most M . Thus the complete natural root has polynomial binary length in a binary RLE or grammar input. This is a bound on the length of a certificate, not a polynomial bound on execution time or a polynomial-time method for finding it.

130.2 Canonical nontermination certificates by trap collapse

For a general finite periodic network, let $R \subseteq V$ be the set of active vertices that can reach a sink in the underlying destination graph. Let $Z = V \setminus R$. No destination from Z can be in R or in a real sink; otherwise that source would itself reach a sink.

Construct a modified network with active set R and a new sink $\perp$. Every edge from R into Z is redirected to $\perp$. Retain the original sinks, and add the fixed initial quantity $\sum_{v \in Z} x_v$ to the final output at $\perp$. Every remaining active vertex reaches a real sink, so the modified system always terminates. Its output equation at the new sink is

$$y_{\perp} = \sum_{v \in Z} x_v + \sum_{a \in R} F_{a\perp}. \quad (130.3)$$

The case $R = \emptyset$ has no routing variables and is handled directly by these fixed output counts.

Theorem 130.2 (A unique finite certificate for either answer). *For RLE or grammar-period networks, the modified instance has exactly one natural certificate. The original input terminates if and only if $y_{\perp} = 0$. Consequently the binary-input decision problem “are all chips eventually absorbed at real sinks?” is in $\text{UP} \cap \text{coUP}$ for both representations.*

Proof. The existence, uniqueness, and polynomial bit length of the modified certificate follow from the previous theorems and Lemma 130.1. Firing inside Z never affects R . Projecting an original execution to its firings in R therefore preserves exactly the chip dynamics of the modified network until the corresponding chips have been absorbed or entered Z . A chip initially in Z , or entering Z , can never reach a real sink and can never disappear; hence a positive value of $y_{\perp}$ excludes a complete original execution. If $y_{\perp} = 0$, there were no initial chips in Z and the modified execution sends none there. That complete execution in R is then also a complete original execution.

For the complexity assertion, construct the modified instance deterministically and guess its unique certificate in a fixed coordinate order with prescribed polynomial-width binary fields. Verify the residual equations and require $y_{\perp} = 0$ for the yes verifier, or $y_{\perp} \geq 1$ for the no verifier. Integer arithmetic on the established bit bounds is polynomial time. There is at most one accepting certificate string for either verifier, since every integer coordinate is unique and the binary encoding is fixed. There is one for exactly the correct answer. This is the defining certificate condition for $\text{UP} \cap \text{coUP}$. $\square$

The one-token explicit-switching upper bound and related succinct certificates are established in the ARRIVAL literature [59, 60]. The theorem above supplies a direct proof for the present multi-chip, grammar-aware, fully canonical algebraic formulation; it is not asserted as a new upper bound for ordinary ARRIVAL.

130.3 Fixed periods give an existential Presburger description

Proposition 130.3. *Fix all periods, not only the destination topology. The graph of the partial map taking initial loads to complete odometers and sink outputs is definable by an existential Presburger formula: a finite disjunction of existential systems of integer-linear equalities and inequalities over natural variables.*

Proof. In the RLE residual system, first split into the finitely many possibilities for z_v and the one-hot block selectors. All products of two witness variables are then either a variable times zero or a variable times one. The periods and their metadata are constants. What remains is a linear system with nonnegative unknowns. The finite disjunction over selectors, with the auxiliary coordinates existentially quantified, defines exactly the desired graph by Theorem 128.1. For the grammar compiler, similarly split over the Boolean node activations and branch choices; all remaining products become linear. $\square$

This observation helps prevent a false universality conclusion from the unbounded chip counts. With periods fixed, even the nonlinear-looking height conditions have only a finite number of selector cases. With run lengths left as parameters, products such as ℓq remain and this particular Presburger argument does not apply. Nevertheless, the whole class of finite explicitly described periodic networks remains decidable by finite-state analysis, with an effectively computable bound for each instance.

130.4 A phase-preserving transport identity

The positive-remainder convention also yields a useful exact relation between different inputs. Fix the periods and form the diagonal matrix $D = \text{diag}(D_v)$ and the active routing-count matrix $C = (C_{vw})_{v,w \in V}$, with source vertices indexing rows. Put $\mathcal{L} = D - C^\top$.

Proposition 130.4 (Full-period input transport). *Suppose the input $\mathbf{x}$ terminates with $U_v > 0$ for every active vertex. Let $\mathbf{k} \in \mathbb{N}^V$ and assume $\mathbf{x}' = \mathbf{x} + \mathcal{L}\mathbf{k} \in \mathbb{N}^V$. Then $\mathbf{x}'$ terminates and*

$$U(\mathbf{x}') = U(\mathbf{x}) + D\mathbf{k}. \quad (130.4)$$

The sink-output increase at s is $\sum_v C_{vs}k_v$.

Proof. Adding D_vk_v firings at vertex v adds k_v whole periods. It changes its routing count to w by $C_{vw}k_v$ and leaves its nonzero last phase, last destination, and support unchanged. The new candidate is balanced because its extra departure vector is $D\mathbf{k}$, its extra active arrival vector is $C^\top\mathbf{k}$, and the input change is their difference. Its last-exit graph remains acyclic. Theorem 127.3 therefore proves (130.4), and the sink formula gives the output change. The strict positivity hypothesis avoids activating previously unused vertices, whose new last exits would require a separate analysis. $\square$

[write] [The positivity hypothesis is used only at vertices with $k_v > 0$. At a vertex with $k_v = 0$ the candidate count and last exit are unchanged, and a vertex with $U_v = 0$ and $k_v = 0$ keeps count zero and has no selected edge; adding k_a whole periods at a vertex a with $U_a = 0$ also adds exactly $C_{aw}k_a$ to each routing count, so balance holds, but its new last exit $\rho_a(D_ak_a)$ may close a cycle. Hence the proposition holds whenever $U_v > 0$ at every v with $k_v > 0$.]

131 A circuit-defined period already hides a counting problem [12]

The preceding compilers exploit particular representations of prefix counts. It is not enough that the destination at a given position can be computed quickly. Random access and prefix counting can have very different costs.

Let C be a Boolean circuit on k input bits. Use one active vertex v and two sinks s_0, s_1 . Its period has length 2^k , with the destinations specified in lexicographic assignment order:

$$\rho_v(j) = s_{C(\text{bin}_k(j-1))}, \quad 1 \leq j \leq 2^k, \quad (131.1)$$

and the period repeats. Give v exactly 2^k chips. Every firing goes directly to a sink, so the execution is particularly simple.

Proposition 131.1 (Counting in a one-vertex acyclic routing system). *For (131.1), the odometer and sink outputs are*

$$U_v = 2^k, \quad y_{s_1} = \#\{a \in \{0, 1\}^k : C(a) = 1\}, \quad y_{s_0} = 2^k - y_{s_1}.$$

Thus computing the exact output on these succinct instances computes the number of satisfying assignments of an arbitrary Boolean circuit.

Proof. Each of the 2^k input chips causes exactly one firing, and the first full period queries each assignment once. The displayed counts follow directly. $\square$

Theorem 131.2 (Obstruction to uniformly small bounded-height certificates). *Suppose a polynomial-time procedure, on a circuit C , produces a polynomial-size bounded-degree integer polynomial $Q_C(\mathbf{w})$ with the following properties: it has a natural root exactly when the routing output y_{s_1} above is zero, and whenever it has a root it has one whose total binary encoding length is polynomial in $|C| + k$. Then $\text{NP} = \text{coNP}$. The same conclusion holds for polynomial-size systems of quadratic residuals. Uniqueness of their witnesses is not needed.*

Proof. By Proposition 131.1, zero output is exactly circuit unsatisfiability. Guess a root of the guaranteed polynomial bit length and evaluate Q_C using exact integer arithmetic. Bounded degree, polynomial coefficient bit length, and polynomially many explicitly described monomials make this a polynomial-time check. Thus circuit unsatisfiability has polynomially verifiable existential certificates.

For completeness, the usual complexity consequence follows directly from verifier circuits. If $A \in \text{coNP}$, membership in its complement is expressed by some polynomial-time verifier $V(x, z)$ with a polynomial-length guessed string z . On fixed input x , unrolling that bounded deterministic verifier gives a Boolean circuit C_x whose satisfying assignments are precisely its accepting guesses. Therefore $x \in A$ exactly when C_x is unsatisfiable. The assumed certificates put every such A in NP , so $\text{coNP} \subseteq \text{NP}$. Taking complements gives the reverse inclusion. $\square$

This theorem does not rule out Diophantine representations of circuit-defined routing. It isolates a conjunction of requirements: a uniform polynomial-time compiler, polynomial description size, bounded degree, and short witnesses. Allowing much larger witnesses removes the stated complexity consequence. Nor does the obstruction concern firing-order ambiguity: the example has no active-to-active edges at all. The difficulty is the prefix-count information compressed into a circuit.

132 A universal one-router system with easy prefix ranks [12]

There is a second, logically independent boundary. Even a polynomial-time *prefix-rank* function does not guarantee that a finite semantic certificate has any computable size bound in the input length. An exponentially dilated universal clock makes the distinction explicit.

132.1 One fixed interpreter and one total rank routine

Fix a deterministic universal Turing machine $\mathcal{U}$ taking a natural input code c in binary. Choose its interpreter convention so that for every computably enumerable set $A \subseteq \mathbb{N}$ there is a program code e with

$$n \in A \iff \mathcal{U}(\pi_C(e, n)) \text{ halts,} \quad \pi_C(e, n) = \frac{(e + n)(e + n + 1)}{2} + n. \quad (132.1)$$

This is ordinary effective universality with an explicit polynomial pairing of the program and its input. Let $T(c)$ be the first halting time, when it exists; time zero is allowed.

Define one total Boolean function on natural arguments:

$$P(c, 0) = 0, \quad P(c, k) = \begin{cases} 1, & \mathcal{U}(c) \text{ halts within } \lfloor \log_2 k \rfloor \text{ steps,} \\ 0, & \text{otherwise,} \end{cases} \quad k \geq 1. \quad (132.2)$$

This definition is an executable bounded simulation, not a halting oracle. Given binary c, k , compute the bit length of k , simulate the fixed machine for that many steps minus one, and return the indicated

bit. A standard direct simulation uses time polynomial in $\text{bits}(c) + \text{bits}(k)$: only $O(\text{bits}(k))$ transitions and a polynomial-size portion of the input and tape can be involved. Initial halting is checked before the first transition.

The function is nondecreasing in k . More precisely,

$$P(c, k) = \begin{cases} [k \geq 2^{T(c)}], & \mathcal{U}(c) \text{ halts,} \\ 0, & \mathcal{U}(c) \text{ does not halt.} \end{cases} \quad (132.3)$$

Thus it is an especially simple function pointwise: either identically zero or a single threshold step.

132.2 The routing rule and its universality

Use one active vertex v , one absorbing sink s , one initial chip, and a read-only input code c . The destination of firing $j \geq 1$ is determined by

$$\rho_c(j) = \begin{cases} s, & P(c, j) - P(c, j-1) = 1, \\ v, & P(c, j) - P(c, j-1) = 0. \end{cases} \quad (132.4)$$

The difference is always zero or one because of monotonicity. The full sink prefix rank is exactly $R_{vs}(k) = P(c, k)$, by telescoping, and the self prefix rank is $R_{vv}(k) = k - P(c, k)$.

Theorem 132.1 (Universal halting with polynomial-time ranks). *The family (132.4) has all the following properties.*

- (i) *The graph has one active vertex and one sink, independently of c . A single fixed total algorithm computes both random access to its stack and all prefix counts in time polynomial in the binary input lengths.*
- (ii) *For every c , the stack is eventually constant. It contains at most one sink symbol. If $\mathcal{U}(c)$ first halts at time T , that symbol occurs exactly at position 2^T ; otherwise every symbol is v .*
- (iii) *The router terminates if and only if $\mathcal{U}(c)$ halts, and in that case its exact odometer is $u = 2^{T(c)}$. Its halting set is computably enumerable complete. With the terminal counter as an observable output, arbitrary partial-computable outputs can be recovered effectively.*
- (iv) *There is no computable function of $\text{bits}(c)$ bounding the bit length of the terminating odometer on all halting inputs. There is likewise no computable uniform bound on the position of the last exceptional stack symbol.*

Proof. The bounded-simulation algorithm above proves the rank bound; two rank queries decide (132.4). Equation (132.3) says that the rank changes only at 2^T when a first halt exists. This proves the stack description. The lone chip returns to v until that unique position and then leaves for s , or returns forever when no such position exists. The reduction (132.1) proves computably enumerable completeness; simulation of the router proves its halting set is computably enumerable. After observing a terminal counter u , compute $T = \log_2 u$ and run $\mathcal{U}(c)$ for T transitions to recover its final output. This is a computable decoding of any simulated partial function's output.

Finally, suppose a computable function $f(a)$ bounded $\text{bits}(u)$ on every halting input c of bit length a . Since $\text{bits}(2^T) = T + 1$, simulating $\mathcal{U}(c)$ for $f(a)$ transitions would decide whether it ever halts. This contradicts universal undecidability. A computable bound on the exceptional position would yield a computable bound on its bit length and hence the same contradiction. $\square$

[write] Item (iv) is the routing form of Proposition 62.2, with the same finite-search proof: a computable bound on the odometer, or on some witness of any Diophantine representation of this undecidable halting set, would decide halting.

Remark 132.2 (Where the computational power resides). The theorem is not a claim that a one-state finite automaton has become universal. The routing apparatus has an unbounded firing counter, a read-only input, and a fixed polynomial-time algorithm allowed to inspect the counter's binary digits. That algorithm performs bounded universal simulation. There is no oracle, but the model is algorithmic rather than a finite-state rotor with a supplied finite period. Universality here is a precise time-dilated simulation, not an assertion of a new physically minimal universal device.

If the input halts, its stack has the finite semantic description

$$v^{2^{T(c)}-1} s v^\omega;$$

if it does not, it has description v^ω . These descriptions exist pointwise, but no effective procedure can always produce a correct finite preperiod-plus-period description from c . Such a procedure would make the existence of a sink symbol decidable by examining the finite prefix and period. Thus *being* eventually periodic and *being supplied with* an effective finite periodic description are different hypotheses.

For comparison, universality in an abelian physical-style substrate is not merely a hypothetical possibility: Cairns constructs universal behavior and undecidable halting questions for three-dimensional sandpiles with suitable infinite backgrounds [61]. That model is not the finite conservative routing model of Sections 128–130. We do not transfer its universality to finite periodic rotors.

132.3 Its unique semantic certificate

Apply the general balance theorem to the one-router system. With one input chip, balance reads

$$u = 1 + u - P(c, u),$$

so it requires $P(c, u) = 1$. Exactness also requires that the last exit be a sink; otherwise it is a selected self-loop. Equivalently, the exact semantic certificate is

$$u = v' + 1, \quad P(c, u) = 1, \quad P(c, v') = 0, \quad u, v' \in \mathbb{N}. \quad (132.5)$$

When it exists, this pair is uniquely $u = 2^{T(c)}$, $v' = u - 1$. Checking it takes time polynomial in its binary length and the input length. Its length, however, has no computable input-only upper bound. Uniqueness plus efficient verification in the certificate length therefore does not place this universal halting problem in UP; the missing hypothesis is a polynomial *input-length* certificate bound.

133 An exact finite-fold and single-fold bottleneck [12]

The previous construction lets us turn a caution about witnesses into a precise equivalence theorem. It singles out one fixed, total, monotone, polynomial-time Boolean function whose multiplicity-preserving Diophantine representation already captures the full general problem. [write] [Compare Proposition 64.1, where the graph of 2^n plays the same role through the single-fold exponential normal form, which ProveIt formalizes (Remark 64.2), and Theorem 118.1, where the least-reservoir relation of one reaction network does.]

133.1 The two representation principles

Let SF be the principle that every computably enumerable subset of $\mathbb{N}$ has a single-fold Diophantine representation over natural witnesses. Let FF be the analogous finite-fold principle. We use these names as mathematical statements; no assertion that either is proved is being made. The corresponding statements for relations of finitely many natural variables are equivalent, since iterated Cantor pairing

has unique natural values and is expressed by integer polynomial equations after clearing the factor two. [write] [SF and FF are statement (a) of Theorem 62.1 in its single-fold and finite-fold versions, and the remark on relations of finitely many variables is (a) $\Leftrightarrow$ (a') there; Part XIII uses the same names (Section 110).]

The graph of the fixed rank function is

$$\Gamma_P = \{(c, k, b) \in \mathbb{N}^3 : b = P(c, k)\}.$$

It is a recursive relation, indeed decidable in polynomial time on binary inputs. Ordinary MRDP supplies a polynomial representation of this graph, but its auxiliary multiplicity is not controlled by that theorem [5]. The distinction is exactly what matters next.

Theorem 133.1 (One monotone rank function is a complete multiplicity bottleneck). *For the fixed function P in (132.2):*

- (i) Γ_P has a single-fold Diophantine representation if and only if SF holds.
- (ii) Γ_P has a finite-fold Diophantine representation if and only if FF holds.
- (iii) More quantitatively, suppose a polynomial $G(c, k, b, \mathbf{z})$ of degree d , with r natural auxiliary coordinates, represents Γ_P with either of these multiplicity properties. Then every computably enumerable subset of $\mathbb{N}$ has a representation with the same property, at most $2r + 3$ natural witnesses, and degree at most $\max\{4, 2d\}$.

Proof. The forward implications require a construction. Fix a computably enumerable set A and choose its interpreter code e from (132.1). Use disjoint r -tuples $\mathbf{a}, \mathbf{b}$ and three additional natural witnesses c, u, v' . Define the explicit polynomial schema

$$\begin{aligned} H_e(n; c, u, v', \mathbf{a}, \mathbf{b}) = & (2c - (e + n)(e + n + 1) - 2n)^2 \\ & + (u - v' - 1)^2 \\ & + G(c, u, 1, \mathbf{a})^2 + G(c, v', 0, \mathbf{b})^2. \end{aligned} \quad (133.1)$$

At a natural zero, the first squared residual forces the unique value $c = \pi_C(e, n)$. The remaining residuals say exactly (132.5), with witnesses for its two rank queries. Thus a root exists exactly when $\mathcal{U}(\pi_C(e, n))$ halts, equivalently when $n \in A$.

For a fixed $n \in A$, the triple (c, u, v') is unique. If G is single-fold, there is also exactly one $\mathbf{a}$ and one $\mathbf{b}$, so the complete witness tuple for H_e is unique. If G is finite-fold, there are finitely many choices of each of these two tuples, and their Cartesian product is finite. Hence H_e has the corresponding multiplicity property. There are $2r + 3$ witnesses. Its first residual is quadratic, its second is linear, and substituting constants for the third argument of G does not increase its degree. Squaring therefore gives the bound $\max\{4, 2d\}$. The use of the extra variable c , rather than substituting the quadratic pairing polynomial directly inside G , is what preserves this degree bound.

For the reverse implications, Γ_P is computably enumerable, so the asserted general principle supplies its representation after a unique triple encoding. More explicitly, encode (c, k, b) by $\pi_C(\pi_C(c, k), b)$. Apply the unary principle to the set of such codes in the graph, and adjoin the two unique polynomial pairing equations. A sum of squares combines the equations without creating choices in the pairing coordinates. This preserves single-foldness or finite-foldness, respectively. $\square$

The theorem is an equivalence, not a solution of either representation principle. Its content is that even a very benign-looking local arithmetic routine cannot be assumed to admit a fixed-arity multiplicity-preserving polynomial graph for free. The routine is total, monotone in its second argument,

Boolean-valued, and polynomial time on binary inputs; those properties do not remove the global representation issue.

There is also an unconditional statement: if one chooses any ordinary MRDP polynomial G for the graph, (133.1) is an ordinary Diophantine representation of every A . What is missing in that unconditional construction is precisely control of the G -witnesses. The paper does not supply the coefficients of such a global rank polynomial; its explicit coefficient-level compilers are the finite RLE and grammar constructions, not a claimed new constructive proof of MRDP.

[write] Relation to Part IX and Part XIII. Theorem 133.1 is the history-free form of (c) in Theorem 62.1. There the decidable relation is the first-halting trace verifier $\text{Trace}_U(e, x, h)$, whose history argument h is unbounded; here it is the graph of one total, monotone, Boolean function of two arguments, computable in polynomial time on binary inputs, and the proof follows the same cycle through a canonical witness, here the pair (u, v') of (132.5). Theorem 118.1 (Part XIII) proves the same pattern for the least-reservoir relation of a fixed reaction network, and Proposition 64.1 for the graph of 2^n , in the single-fold direction, through the single-fold exponential normal form that ProveIt formalizes (Remark 64.2). New in Theorem 133.1: a Boolean threshold function, an elementary proof that does not use the exponential normal form, the finite-fold version beside the single-fold one, and the explicit accounting of (iii), $2r + 3$ witnesses and degree at most $\max\{4, 2d\}$.

133.2 Why bounded circuit quartics do not close the gap

Proposition 133.2 (Canonical quartics at every external bit budget). *For every pair of positive integers a, b , there is an effectively constructible quartic polynomial $Q_{a,b}(c; \mathbf{w})$, with a number of natural witness coordinates polynomial in $a + b$, such that it has a unique natural root exactly when*

$$0 \leq c < 2^a \quad \text{and} \quad T(c) = b - 1.$$

The root may include the uniquely forced routing odometer $u = 2^{b-1}$.

Proof. Use a Boolean witness bits to encode c , with the linear equality $c = \sum_{i=0}^{a-1} 2^i c_i$. They give a unique encoding exactly in the specified range. Construct a Boolean circuit that simulates the fixed machine on this input for $b - 1$ transitions and tests that its first halting time is exactly $b - 1$. This is a deterministic bounded computation on a tape segment of polynomial size in $a + b$, so the circuit has polynomially many gates. It also handles $b = 1$ by checking the initial configuration.

Take gates of types AND and NOT, introducing one variable for each wire. Impose Boolean equations $t(t - 1) = 0$ on the wire values, equations $z - xy = 0$ for AND gates, and $z + x - 1 = 0$ for NOT gates. Fix the initial input and constant wires, require the final acceptance wire to equal one, and add $u - 2^{b-1} = 0$. All equations have degree at most two. Their squared sum is quartic. The circuit is deterministic and acyclic, so for fixed input bits all wire values are uniquely determined. The input bits and u are also unique. This proves the claim. $\square$

A different b produces a different polynomial with potentially a different number of coordinates. The assertion

$$\exists b (Q_{a,b}(c; \mathbf{w}_b) = 0 \text{ for some } \mathbf{w}_b)$$

is not yet one ordinary fixed-arity Diophantine equation. Turning this externally indexed family into a fixed polynomial while preserving all auxiliary uniqueness is exactly the issue that must not be skipped. The one-router example makes the gap visible even though there is only one semantic exit count and one accepting time.

134 Executable artifacts and the formalization boundary [12]

134.1 What the supplied programs actually do

The archive contains two dependency-free Python programs. The file `code/12-rle-routing-verify_certificates.py` supplies direct legal execution, analytic prefix counts, canonical RLE witnesses, residual evaluation, canonical grammar-path witnesses, and deterministic regression tests. Its integer arithmetic is exact. The test instantiation of the universal-clock routine uses small countdown and looping transition systems; it tests the rank mechanism, not the universality of a particular encoded Turing machine.

The file `code/12-rle-routing-compile_quartic.py` is a sparse integer-polynomial compiler for the RLE residual system. It supports both fixed numeric run lengths and free natural parameters λ_{vj} with $\ell_{vj} = \lambda_{vj} + 1$. It constructs and expands the sum of squared residuals. Its output is a coefficient-and-monomial representation of an actual polynomial, not a call to an MRDP oracle or a numerical equation solver.

For the one-vertex period v^3s , the expanded fixed-length polynomial has 50 nonzero monomials, total degree four, and thirteen witness variables. The same destination topology with both block lengths left free produces 81 nonzero monomials and still has degree four and thirteen witnesses. These are illustrative counts for the supplied compiler, not minimality claims. Both versions have eleven quadratic-or-linear residuals.

The human-readable residuals and expanded polynomials are included as `data/12-rle-routing-example_quartic.txt` and `data/12-rle-routing-parametric_quartic.txt`; their JSON counterparts list every integer coefficient and monomial exponent. The sample root uses two input chips, giving eight firings and two sink arrivals. This small example makes the coefficient-level construction inspectable without printing a very long multivariate polynomial in the main text.

134.2 Exact test results and their scope

The following results were obtained from the supplied suite. The detailed machine-readable receipt is `data/12-rle-routing-test_results.json`.

Check	Count or value
Two-active-vertex, one-sink, length-two networks	81
Network/input pairs, with each initial count in $\{0, 1, 2\}$	729
Candidate odometers in $\{0, \dots, 12\}^2$ examined	123,201
Balanced candidates among these	2,037
Accepted exact odometers among these	535
Single-coordinate +1 mutations of true RLE roots rejected	13,375
Additional random halting RLE networks	200
Analytic-versus-expanded RLE prefix comparisons	13,236
Small shared grammars / prefix comparisons	150 / 712
Single-coordinate +1 grammar-witness mutations rejected	35,952
Large unexpanded grammar period length	2^{1025}
Universal-clock mechanism checks on toy systems	205

The bounded odometer enumeration is not an enumeration of every auxiliary tuple of every polynomial. Arithmetic uniqueness of the local fields reduces the comparison to candidate odometers; the general completeness and single-foldness claims rest on the proofs. Likewise, mutation tests are good at finding missing constraints but cannot establish a universal theorem.

For every tested halting finite network, the suite compares smallest-index and largest-index legal scheduling and obtains the same full odometer and sink outputs. The largest observed total firing count in the exhaustive tiny network experiment is 20. The enormous example of Example 128.3 is a separate direct certificate evaluation and does not involve a simulation of those firings.

Each illustrative symbolic polynomial is additionally compared with the independent residual evaluator at 500 arbitrary natural assignments, not only at its true root. At those assignments, the expanded polynomial exactly equals the sum of the directly evaluated squared residuals. These checks exercise coefficient aggregation and cancellation in the sparse compiler.

134.3 A concrete Lean integration plan

The repository snapshot used for this paper is `e18718e837d43e162252f9a314e8cb797fbd1a1f`. Its documented MRDP endpoint works with one free natural input and a finite tuple of natural witnesses, with integer polynomial evaluation. The guide also identifies `MRDP.boundedForall_dioph`, `MRDP.primrec_diophFn`, and the older `DiophantineTrace.lean` reachability interfaces [5]. [write] [The two declarations are in `Computability/HilbertTenthProblem/Lean/Diophantine/Common/MRDPCore.lean`, namespace `MRDP`; this report elsewhere cites the separate `Diophantine.boundedForall_dioph` of `DiophantineTrace.lean` [3].] These are relevant integration points, but none of them supplies the new single-fold routing theorem merely by being imported.

A suitable new formal development would separate five layers.

1. **Operational semantics.** Use finite active and sink types, natural chip vectors and firing counters, arbitrary stack functions, and finite legal execution lists. Prove the conservation formula first. The least-action proof then needs only prefix monotonicity and a first attempted excess, not an external theorem about infinite abelian networks.
2. **Exactness.** Define the selected last-exit function on positive odometer support. Prove that complete legal executions induce strictly increasing last-departure times along selected active edges. For the reverse direction, formalize the residual-flow argument of Theorem 127.3, including the finite functional-graph cycle lemma.
3. **Canonical local arithmetic.** Prove the unique positive-remainder and block-selection theorem, with inactive coordinates forced to zero. For grammars, prove the one-path theorem by induction through the acyclic node order. Keep the unique-witness statements explicit in these interfaces.
4. **Polynomial extraction.** Build a finite list of multivariate integer polynomials, one per residual. Prove that evaluating their squared sum at natural assignments gives zero exactly when every residual vanishes. Then establish a bijection between roots and complete odometers, rather than only an existence equivalence. Variable-count and degree bounds are separate syntactic obligations.
5. **Effective universality and reductions.** Formalize the bounded rank evaluator and the time-dilation lemma for the chosen universal machine. Only the ordinary unbounded rank-graph representation needs MRDP. The single-fold and finite-fold implications should remain explicitly conditional on the corresponding local rank-graph hypothesis.

The integer/natural interface merits particular care. Polynomial residuals use signed subtraction in $\mathbb{Z}$; semantic chip counts and all witnesses are natural. One should not translate a signed residual such as $u - x - F$ into truncated subtraction on $\mathbb{N}$. The most convenient semantic invariant is usually an equality of natural sums, followed by an explicit coercion into $\mathbb{Z}$ when connecting it to the polynomial.

None of these new layers has been kernel-checked in this package. The repository's existing audit reports concern its existing declarations, not this manuscript. This distinction is consistent with the broader practice of developing mathematical constructions and their formalization in parallel, as discussed in recent work on universal Diophantine bounds [62].

135 Further research questions and concrete targets [12]

The following is a proposed research program. Items described as questions are questions for this construction or project; the list is not a claim that every formulation is an established open problem in the literature.

[write] *Manuscript 12 posed its eight items under numbered headings of unnumbered subsections; the headings, without their numbers, are the titles here, and the text is unchanged. A [write] line after an item names the question of Section 71 it overlaps; the items are not merged with them.*

Research question 135.1 (A kernel-checked single-fold compiler, end to end; source 12). The first target is the exact statement of Theorem 128.1, including free run-length parameters, complete witness uniqueness, degree four, and the precise coordinate count. A useful deliverable would be a Lean function returning the residual polynomials together with proofs of its semantic correctness and of a unique-root theorem. The residual-flow converse is a natural independent mathematical milestone. A successful compiler should emit a checkable theorem about its output polynomial, not only validate examples.

[write] Compare Research question 71.1, and Part XIII's question on a verified compiler with a solution-set bijection.

Research question 135.2 (Smaller canonical certificates without hidden multiplicity; source 12). Can the $4n + 4m + s$ coordinate count be reduced while retaining an explicit quadratic-residual presentation and natural single-foldness? Some equations are deliberately redundant or chosen for clarity. Savings should be measured against coefficient size and polynomial sparsity, not only variable count. Eliminating a height coordinate by substituting a long path expression, for example, may raise degree or expression size. A rigorous lower-bound question would first fix a compiler model and an allowed local gadget vocabulary; there is no unrestricted minimality assertion in this paper.

[write] Compare Research question 71.16.

Research question 135.3 (Multiple prefix queries on a globally shared grammar; source 12). The grammar theorem shares subwords within each local period but gives each independent prefix query its own activation and position coordinates. Networks with many roots in one enormous shared grammar may waste these duplicated certificates. Can several paths be certified canonically using a compressed query structure whose size follows the actual overlap of the paths? A useful result would specify a sharp parameter, such as the number of distinct selected node/offset pairs, and prove both degree control and uniqueness.

[write] Compare Research question 71.27, which asks for canonical DAG codings with polynomial bit height; here the DAG is given and the descent path is canonical (Theorem 129.1).

Research question 135.4 (Algorithms exploiting the forest certificate; source 12). The polynomial certificate describes an answer without efficiently finding it. Can its canonical last-exit forest guide a practical solver on networks with small feedback vertex sets, bounded treewidth, or few distinct period blocks? The phase-transport identity in Proposition 130.4 suggests reusing solutions for related loads. A meaningful algorithmic result must bound the number of selector or forest changes rather than simply invoke uniqueness of the final certificate. Comparisons with existing ARRIVAL methods should be made on the same representation and output task.

Research question 135.5 (The intermediate territory between grammars and circuits; source 12). Theorem 131.2 shows why arbitrary circuit random access is too broad for the same short-certificate guarantee without a major complexity consequence. Which intermediate descriptions admit efficient prefix counting and compact canonical algebraic certificates? Candidates include restricted branching programs, decomposable circuit descriptions, and descriptions with an explicit small dynamic-programming interface. The task is to prove a representation-sensitive theorem, with the rank routine and its certificate size exposed, rather than to call every description merely “succinct.”

Research question 135.6 (The monotone rank bottleneck as a test for general claims; source 12). Theorem 133.1 gives a specific stress test for any proposed single-fold or finite-fold compilation theorem for total polynomial-time functions. Does the proposed theorem apply to (132.2)? If it does, the explicit polynomial (133.1) must be included in the claimed consequences. A constructive attack could look for special arithmetic structure in the threshold-time relation, while a limitation result could identify exactly which local-certificate hypotheses exclude it. This is a substantial global target, not a consequence already delivered by bounded Boolean-circuit encodings.

[write] Compare Research questions 71.48 and 71.50, and the notes added to them in batch 63.

Research question 135.7 (Infinite-background abelian computation; source 12). Cairns’s universal sandpile construction [61] offers a physically more local setting in which to seek arithmetic certificates. For a specified finite toppling event, what finite causal region can be certified without unrolling every toppling? Which global halting questions require information outside every effectively bounded region? The functional last-exit proof here uses one outgoing chip per firing; sandpile topplings send several chips and need a different residual-flow or causal-forest invariant. Establishing that invariant, or proving an obstruction to a proposed one, would be a meaningful extension rather than a black-box application of MRDP.

Research question 135.8 (Explicit preperiods and more general abelian processors; source 12). There is a useful gap between explicitly supplied ultimately periodic stacks and the non-effectively bounded eventual constancy of the universal clock. A next compiler can handle supplied finite prefixes followed by RLE or grammar periods, with a canonical selector for whether the odometer ends before or after the cutoff. Beyond routing, one can ask which finite-state abelian processors retain a canonical completion certificate when firings consume or produce several messages. The conservation and minimality hypotheses should be stated first; replacing the functional last-exit forest by an unjustified ranking condition would risk accepting false circulations.

[write] Compare Research question 71.31.

136 Conclusion [12]

The paper develops two complementary conclusions. For explicitly compressed finite routing, the entire execution history can be replaced by a canonical spatial object: prefix counts together with exact last-exit distances. This produces a genuine fixed-arity, parametric, single-fold quartic for each routing topology, and extends to shared grammar descriptions of periods. Its size follows the description of the local instructions, not the number of firings.

For universally generated instructions, even strikingly simple pointwise stacks do not provide the same guarantee. The universal clock has one chip, one active vertex, at most one sink event, eventual constancy, and polynomial-time prefix ranks. Nevertheless its exact exit count has no computable input-length bound. Its local rank graph carries the entire single-fold and finite-fold representation issue, as witnessed by an explicit conditional polynomial construction with $2r + 3$ auxiliaries.

The useful frontier is therefore not simply “classical machine versus unconventional substrate.” It is the interaction between effective instruction descriptions, exact local prefix arithmetic, canonical global causality, and control of every arithmetic witness. The constructions and reductions here make those requirements separate, testable, and suitable for a formal continuation in ProveIt.

137 Reproduction instructions and archive contents [12]

[write] *Appendix A of manuscript 12.* The listings keep the delivered layout, in which `article.tex`, `verify_certificates.py` and `compile_quartic.py` sit in one directory and the second imports the first by that name. Manuscript 12’s `article.tex`, `article.pdf`, `README.md` and `SHA256SUMS` are not shipped (its text is this Part and Section 3.12); the programs are `code/12-rle-routing-*.py`, the build file is `code/12-rle-routing-Makefile` and the provenance note is `12-rle-routing-SOURCES.md`. This report’s `README` gives a rerun recipe for the shipped names.

The source is a self-contained L^AT_EX document with an embedded bibliography. No external images, proprietary assets, or font files are required. Standard TeX packages are used. Compile with

```
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

Run the exact checks and regenerate the two illustrative polynomials with

```
python verify_certificates.py --output test_results.json
python compile_quartic.py --output example_quartic.json
python compile_quartic.py --variable-lengths \
    --output parametric_quartic.json
```

The Python code requires Python 3.10 or later and no third-party packages. The programs can also be imported. For a custom topology, instantiate `Network` with its active-vertex periods and sink count, then call `compile_rle(network, variable_lengths=True)`. Destinations $0, \dots, n-1$ are active; destinations $n, \dots, n+s-1$ are sinks. Every supplied block is a pair (`destination`, `positive_length`). In the parametric mode the numeric lengths only specify a sample instance; each actual length in the returned polynomial is a free natural parameter plus one.

The archive includes the PDF and L^AT_EX source, both Python programs, JSON test results, fixed and parametric polynomial exports in JSON and text, a `README`, a build file, and a provenance note. These exported polynomials are examples. The general RLE theorem applies to the compiler for any validated finite topology; the grammar compiler is given by its full mathematical schema and executable local-path checks, not by a separately exported global symbolic grammar compiler in this package.

138 A compact dependency map [12]

[write] *Appendix B of manuscript 12.*

Claim	Dependencies and verification status
Exact odometer criterion	Prefix monotonicity, conservation, finite graph argument; complete proof in Section 127. Classical mechanism.
Parametric RLE quartic	Exact odometer criterion and canonical interval slacks; explicit compiler and numerical regression tests supplied.
Grammar quartic	Exact odometer criterion and unique DAG descent; complete equations and proof, local-path tests supplied.
Both-outcome certificates	Finite-state bound and trap collapse; mathematical proof, no claim of a new standard ARRIVAL upper bound.
Circuit counting barrier	A direct satisfying-assignment count and a polynomial-verifier reduction; proof supplied.
Universal one-router boundary	Fixed universal machine and logarithmically bounded simulation; full reduction proof, toy mechanism tests only.
Single-/finite-fold equivalence	Explicit polynomial (133.1); conditional local rank-graph assumption is retained.
Ordinary unbounded Diophantine representation	Existing MRDP theorem; no auxiliary uniqueness inferred from it.

Part XV

Interaction combinators: exact wiring, loop-exact gluing and bounded quartic frontends

Source and scope. [write] This Part merges three manuscripts delivered together in batch 78 of ProveIt’s incoming reports (arrival commit 1977e6ea6), all dated 2 October 2026 and all written in answer to the research note *A finite wiring obstruction for interaction combinators* [63]. Manuscript 13 is “Exact Wiring in Diophantine Computation: Sharp loop-parity separation and a cycle-preserving compiler for interaction combinators” (batch 78, manuscript 01; archive `Exact_Wiring_Diophantine_Report.zip`; shipped files prefixed `13-exact-wiring-`; labels `cdc:ew:`). Manuscript 14 is “Topology Is Not Free: Loop-Sensitive Diophantine Certificates for Interaction Nets” (batch 78, manuscript 04; `Topology_Is_Not_Free_Interaction_Nets.zip`; `14-net-topology-`; `cdc:nt:`). Manuscript 15 is “No Ghost Wires: Loop-Exact Quartic Certificates for Interaction Combinators” (batch 78, manuscript 05; `no_ghost_wires.zip`; `15-no-ghost-wires-`; `cdc:ng:`). Unlike Parts X–XIV, which print one manuscript each in its own order, this Part is arranged by subject, because the three share their target, the rule table, the gluing theorem and the bounded compiler: port-labelled nets and the six rules (Section 140); exact gluing and loop accounting (Section 141); contextual information, with the sharp $2/5$ – $2/3$ loop-parity theorem (Section 142); the local annihilation kernel (Section 143); memory logs, as second routes to Part V (Section 144); the bounded six-rule compiler (Section 145); the permutation-orbit route and its implemented scheduled compiler (Section 146); the boundaries and the MRDP interface (Section 147); implementation, validation and formalization proposals (Section 148); the twenty-five research questions and the three conclusions (Section 149); and, as the last seven sections of the Part, the manuscripts’ appendices. Every section or subsection taken from a manuscript carries its number in brackets. The title pages, abstracts, status statements and introductions are Sections 3.13 to 3.15; the source audits of manuscripts 13 and 14 are in the provenance appendix (Appendix K). The conventions of this Part are Section 139.

What is printed once. [write] The three manuscripts were written independently, at three pins of the same morning, without knowledge of one another. A result proved by two or three of them is printed once and credits all; a genuinely different proof or construction is kept as a marked second route. The six-rule table and the repository’s witness pair are printed once (Table 20; Section 140.4), with the other manuscripts’ definitions and readings beside them. The storage and loop bounds are Corollary 140.2. Order-independent gluing is Theorem 141.2; manuscript 15’s division-free matrix form (Theorem 141.4 and Corollary 141.5) and manuscript 14’s locality lemma and suppression proposition (Lemma 141.7 and Proposition 141.8) follow it as the further statements they are. Manuscript 15’s contextual completeness (Theorem 142.7) is the loop-count form of manuscript 13’s injectivity consequence (Corollary 142.3) and is kept with its own two-closure proof, which also proves sufficiency. The bounded six-rule compiler is stated once, as manuscript 14’s Theorem 145.3 with its explicit ledger; manuscript 13’s statement follows as a remark (Remark 145.5) with its proof as a second presentation, and likewise its transfer corollary after manuscript 14’s Theorem 145.7. The carry-free product lemma of manuscripts 13 and 14 is Part V’s (see below). Manuscript 15 reaches the scheduled relation by a different route, a canonical permutation-orbit certificate, and is printed whole in Section 146.

Relation to the other Parts. [write] Manuscripts 13, 14 and 15 inspected ProveIt at 439c0a2d9, 6914ccca6 and 928ea9701, on the morning of 2 October 2026, when Parts I–XIV had been written. Manuscript 14 read this report’s README and cites it for the memory and heap compilers of manuscripts 03 and 08 [66]; manuscripts 13 and 15 do not cite the report. Four relations matter.

- *Part X.* The Part answers, for bounded scheduled histories, Part X’s question on a complete interaction-combinator frontend (Research question 85.7): rule interpreters and semantic simulation (all three manuscripts); reciprocal ports, self-connections, parallel temporary edges, interface splicing and creation slots (Sections 140 and 141); polynomial compilers (Theorem 145.3, on paper, and Theorem 146.7, implemented for fixed schedules); and tests (Section 148). The transfer theorem Theorem 145.7 supplies, for port graphs with reciprocal links and bare loops, the adapter that Part X’s subsection on interaction systems asks for before its graph-transfer theorem (Theorem 81.1) can be applied. It is proved directly, through this Part’s memory certificates, and no novelty is claimed for it beside Theorem 81.1. Manuscript 15’s finite schedule disjunction gates inactive branches as Part X’s guarded quadratization does (Theorem 75.6). The commutation quotient of independent schedules, fixed arity and an ordinary-integer universal loader stay open (Research question 71.43 and Part X’s question on quotienting independent schedules, Section 85).
- *Part V.* The carry-free product lemma of manuscripts 13 and 14 (Lemmas 144.1 and 144.5) is Part V’s Lemma 40.1, proved there first; manuscripts 13 and 15 were written without knowledge of this report, and manuscript 14 cites only its README. No novelty is claimed for the lemma. The memory theorems Theorems 144.4 and 144.6 are variants of Theorem 40.2 for different relations. Manuscript 14’s count $17L - 4$ assumes external address and value caps and zero-initialized memory, and is not an improvement on Part V’s $24L$; manuscript 13 audits initial and final arrays. The comparison is at the end of Section 144.
- *Part II.* The bijections of this Part are with *scheduled* executions: choosing a different active pair gives a different zero, even when the two reductions commute (Theorem 145.3 and the paragraphs after it). Part II’s quotient of executions by a commutation relation has no counterpart here.
- *Parts XII and XIV, and the neighbouring reports.* Manuscript 15’s lasso up to renaming (Proposition 146.11) is a finite certificate of nontermination for another substrate; it is related to Part XII’s infinite periodic tails (Theorem 100.1), not a case of them. Infinite behaviour is the subject of the neighbouring report *liveness-beyond-halting*; its question on clock-faithful translations into interaction nets is not answered here: the Part supplies a local polynomial compiler for the target’s own finite steps, not the clock-faithful simulation theorem. Part XIV’s history-free certificates have no analogue here: every compiler of this Part certifies a history.

Two notes of ProveIt’s Hilbert’s-tenth-problem research tree, maintained elsewhere, bear on the Part. The ten-pattern loop formula of manuscript 14 (Theorem 143.1) was derived independently in [64], which was written after manuscript 14’s pin; and the tree’s review of the three archives [65] finds no mathematical error in them and derives three natural-domain reductions of their exported systems. Dated notes after Theorem 143.1, Theorem 143.3, Theorem 144.4 and manuscript 15’s worked example say what these are. Neither note is a source of this Part, and priority is claimed for neither the manuscripts nor the notes.

Setting and hypotheses. [write] All witnesses and parameters range over $\mathbb{N}$; no uniqueness statement is claimed over $\mathbb{Z}$ (Section 147). Nets are explicitly labelled: cells carry identifiers, auxiliary ports are ordered by Lafont’s numbering ($\delta\delta$ keeps the auxiliary labels, $\gamma\gamma$ exchanges them), free ports are labelled, and port-free cyclic wires are counted by a separate natural number. Every construction is a family indexed by an external horizon T and the input dimensions; none is one polynomial of fixed arity. In manuscripts 13 and 14 the schedule of active pairs is a tuple of source coordinates of the polynomial, and the auxiliary witness is unique for each legal schedule; in manuscript 15 the lifetime schedule is structural data of the compiler, and the endpoint matchings and loop counts are parameters. Fresh cells are named by a fixed allocation rule; there is no quotient by schedule, graph isomorphism or observational equivalence. Manuscript 13’s contextual theorems concern bare perfect

matchings, without agents. The six-rule compilers of manuscripts 13 and 14 are paper constructions; manuscript 15's fixed-schedule compiler and the memory and kernel components of manuscripts 13 and 14 are implemented (Section 148). Nothing in this Part is formalized in Lean or Rocq.

139 Conventions of Part XV

[write] The letters are the manuscripts'. Each section of this Part says which manuscript it prints, and a letter means what that manuscript says. The following letters clash between manuscripts 13, 14 and 15, or with other Parts; the tempting false reading is to carry a meaning from one manuscript, or from Part V, into another.

Symbol	Meaning in Part XV, and the clash
L	In manuscript 13's memory theorem, $L = 2A + m$ is the number of records, counting the A initial and A final audit records (Section 144.2). In manuscript 14's, L is the number of logged accesses, as in Part V. In manuscript 15, L is the number of newly created cyclic wires (Theorem 146.1). In manuscript 14's comparison gadget (Section 151), L is the "less than" indicator.
B, β	In manuscripts 13 and 14, B is the carry-free base: $B = U^L + 1$ (13) and $B = (M + 1)^L$ (14); Part V writes $\beta = C^L + 1$ (Lemma 40.1). Manuscript 14 also writes $B = (u1, u2, v1, v2)$ for the ordered boundary of an annihilation, and manuscript 15 writes B for the set of surviving boundary ports, β for its gluing involution, and $\mathcal{B}$ for the second net of the witness pair. Not Part XIV's B_{vjw} .
$M, \mathcal{M}_n$	A perfect matching (13, 15); $\mathcal{M}_n$ is the set of perfect matchings of $2n$ ports (13). In manuscript 14, M is a partial matching of four boundary positions (Theorem 143.1) and also the packing bound $M = 2ALV$ of the memory records; in the product lemma, entries are $< M$.
n	Half the number of ports (13); the number of vertices of a permuted set $[n]$ (15).
D_n, D	$D_n = (2n - 1)!! = \mathcal{M}_n $ (13), not Part V's radix D . Manuscript 14's loader radix is $D = 3C_0 + f + 4$ (Section 145.5); D_1, D_2 are the two fresh δ cells of the mixed rule in all three.
C_0, C_t, C, C_*	Initial cells, cells after t steps (13, 14, 15); $C = C_0 + 4T$ reserved cells (14) and $C_* = C_0 + 4T$ cell slots (13). Part V's $C = D^4$ is a radix power; in manuscript 15, C also denotes an orbit.
F, f	The number of free ports: F in manuscripts 13 and 15, f in manuscript 14. Manuscript 13 also writes $F(\lambda)$ for a signed sum of closure parities (Lemma 142.2); manuscript 15 writes f_j for residuals and $\bar{f}_i$ for complement bits.
ℓ, ℓ_t	The number of port-free cyclic wires, in all three; ℓ_0 is supplied with the input.
c	$c(M, Q)$ is the number of components of the doubled multigraph $M \cup Q$, hence of cyclic wires after closing M by Q (13, 15); in manuscript 14, $c_J(M)$ and c_t count new loops and c also names a cell.
U, H, S	In manuscript 13: the packing range $U = AHV^2$, the time radix $H = m + 2$, and the size $S = 1 + C_0 + F + T$. In manuscript 14, H and G count the private residuals and coordinates of a full assembler (Equation (145.9)) and S_i is a product chain; in manuscript 15, S is a successor (permutation) matrix. Not Part X's H or Part XIV's U .
A, V	The number of addresses and the value bound of a memory log (13, 14); manuscript 14's caps are fixed parameters, and in its machine $A = 4C + f + 1$, $V = 3C + f + 4$ (Equation (145.1)). In manuscript 15, $A^{(t)}$ is the matrix of a temporary matching, V a temporary vertex set and, in Corollary 146.8, the number of existential variables.
h, e	In manuscript 14: h_t is the next fresh cell identifier, h_{ij} a helper product, and e_i the external-position indicator. In manuscript 15: h_i is the strict-gap slack and e_i the root bit of an orbit certificate. In manuscript 13, $h(\lambda)$ is a signed correlation and (l, e, g) a three-way comparison.

Symbol	Meaning in Part XV, and the clash
T	An external horizon (number of sequential interactions), as in Parts II, VI–VIII, X and XIII; a family index, not a variable.
P, Q, K, R	P is a sum of squares (13), $P_{N,T}$ the compiled family (14) and P_t the number of ports of configuration t (15); $\mathcal{P}$ is the set of the three perfect matchings of four positions (14). Q is a closing matching (13) or the compiled polynomial Q_S (15). K is a number of probes or a sorting key (13), the kernel K_J or $K = 4C_0 + f$ (14). R counts residuals (13, 15) or is a product chain (14), not Part II’s residual count.
ports	Manuscript 13 numbers ports $3c + p$ from zero in its example and $3c + p + 1$ in its code; manuscript 14 uses $p(c, j) = 3c + j + 1$; manuscript 15 writes (a, i) or a_i . Manuscript 14 numbers the four boundary positions $1, \dots, 4$; the research note [64] numbers them $0, \dots, 3$.
ϵ, ε	The eraser: ϵ in manuscript 13, ε in manuscripts 14 and 15.

Lafont’s numbered conventions are used throughout: in the annihilation $\delta\delta$ the auxiliary port i of one cell is joined to the auxiliary port i of the other, and in $\gamma\gamma$ to the other one; all three manuscripts state this, read from the numbered figures of [41], and agree with [63]. Renamed macros only: manuscript 13’s `\M`, `\PP` and `\multiset`, and manuscript 15’s `\cyc` and `\id`, are defined in this report’s preamble with the manuscripts’ meanings; manuscript 14’s `pin` macro is printed as the literal `commit` and its `\repo` URL macro as the URL; the manuscripts’ unused macros are dropped. No mathematical symbol of manuscripts 13–15 is renamed. The equations of Part XV are numbered within sections; manuscript 14’s hand-set numbers, such as “(3.3)”, are replaced by these numbers.

140 Port-labelled nets and the six interaction rules

[write] *All three manuscripts define port-labelled nets, the six rules of Lafont’s interaction combinators in numbered-port form, and the repository’s witness pair. The definitions are printed from each, because each fixes conventions that its later sections use (manuscript 14’s predicates “normal”, “agent-free” and “completely empty”, and its fresh-name rule; manuscript 15’s treatment of the removed principal edge and its gluing data). The rule table is printed once, from manuscript 13; manuscript 14’s equations (140.2)–(140.3) and manuscript 15’s (140.4) state the same rules, and all three agree with each other, with Lafont’s numbered figures [41] and with the research note [63]. The witness pair is printed from manuscript 13, with manuscript 15’s table and argument after it and manuscript 14’s reading through its kernel in Section 143.*

140.1 All six interaction-combinator rules as literal wiring [13]

140.1.1 Representation and ordered ports

An agent has type ϵ , δ or γ , with respective arities 0, 2, 2. It has one principal port, numbered 0, and its auxiliary ports are ordered 1, 2. A finite net consists of a finite typed cell set, a finite labelled set of free ports, a matching of all these ports, and a natural number of isolated cyclic wires. Self-wires between different ports of one cell are permitted. A principal–principal wire between two cells is an *active pair*.

The numerical rules below follow Lafont’s clockwise convention. In particular, $\gamma\gamma$ exchanges numbered auxiliaries, whereas $\delta\delta$ does not. This can look reversed when an unnumbered picture is read as if both cells had the same orientation; the numbered picture on journal page 82 removes the ambiguity [41].

140.1.2 The complete right-hand-side table

The principal wire of the selected pair is removed as part of the interaction. It is *not* counted as an isolated cyclic wire. The rule then reconnects the old auxiliary interface as follows.

Pair	Replacement, in numbered-port notation
$\epsilon\epsilon$	No agents and no interface.
$\delta\delta$	For old cells u, v , join $u1$ to $v1$ and $u2$ to $v2$.
$\gamma\gamma$	Join $u1$ to $v2$ and $u2$ to $v1$.
$\epsilon\delta$	Two fresh ϵ agents; connect their principal ports to the old δ auxiliary ports 1, 2, respectively.
$\epsilon\gamma$	Two fresh ϵ agents; connect their principal ports to the old γ auxiliary ports 1, 2, respectively.
$\delta\gamma$	Fresh γ cells G_1, G_2 and δ cells D_1, D_2 . Old δi attaches to $G_i 0$, old γj to $D_j 0$, and $G_i j$ attaches to $D_j i$ for every $i, j \in \{1, 2\}$.

Table 20: The literal six-rule controller. The last rule specifies all eight right-hand-side wires: four boundary wires and four internal wires.

To use a single gluing algorithm, create a fresh temporary boundary port β_a for every old auxiliary port a . In the right-hand side, use β_a wherever the table mentions the old interface port a . This gives a matching on fresh agents and fresh boundary ports, disjoint from the old matching after removal of the principal wire. Finally splice every a with β_a . There are at most four such splices.

Proposition 140.1 (Exact constant-size interaction macro). *The boundary-port construction implements every rule of Table 20 in every valid surrounding net. Its number of memory accesses is bounded by an absolute constant. It preserves existing cyclic wires and creates exactly the cyclic wires of the rule’s contextual gluing.*

Proof. The right-hand side is a finite literal matching with at most four new agents and four boundary ports. Removing the active principal edge removes both its endpoints, so it has no incident surviving wire. Adding the right-hand-side matching is a disjoint union. Each remaining interface connection is a splice, and Theorem 141.2 proves that their composition is exactly the contextual rule application. Each splice uses boundedly many accesses, as do the bounded lists of type updates, port initializations and retirements. The bound is uniform over all input nets and all six rules. $\square$

[write] The storage bounds $C_t \leq C_0 + 2t$ and “at most $C_0 + 4t$ allocated identifiers” are those of the research note [63], which derives them from the rule shapes; the loop bound $\ell_0 \leq \ell_t \leq \ell_0 + 2t$ is new to the repository. All three manuscripts prove them: Corollary 140.2 below; manuscript 14’s (145.4) in Proposition 145.2, which adds the fixed log length of its machine; and manuscript 15’s Proposition 146.9, which adds port and temporary-vertex bounds. They are printed once, here; the other two statements keep their additions.

Corollary 140.2 (Storage and loop bounds). *After t interactions from a net with C_0 cells and ℓ_0 cyclic wires,*

$$C_t \leq C_0 + 2t, \quad \ell_0 \leq \ell_t \leq \ell_0 + 2t. \quad (140.1)$$

A fresh-identifier implementation needs at most $C_0 + 4t$ allocated cell identifiers.

Proof. Every interaction deletes two agents and creates at most four. Only the mixed binary rule creates four; either erasing rule creates two; the annihilation rules create none. At most four new identifiers are allocated per step.

Existing isolated wires never participate in a rule. Only the binary annihilation rules can create new isolated wires: their four interface ports can make at most two closed components. In the mixed and erasing rules, every boundary wire reaches a principal port of a fresh surviving agent, so its gluing component is a path, not a port-free cycle. The $\epsilon\epsilon$ rule has no boundary. Thus each step adds at most two loops and removes none. $\square$

[write] Manuscript 14’s definitions, rule templates and fresh-name convention. Its templates (140.2)–(140.3) are the rules of Table 20; its three terminal predicates are used again in Section 145.5.

140.2 Exact nets and a fully specified rule convention [14]

140.2.1 Data and equality

We use $\mathbb{N} = \{0, 1, 2, \dots\}$. A net has a finite set of cells, each labelled ε , γ , or δ ; a fixed finite ordered set of free ports; a fixed-point-free involution on its incident ports; and a natural number ℓ of agent-free cyclic wires. The arities are 0, 2, 2. A cell c has principal port $c0$ and, when binary, ordered auxiliary ports $c1, c2$. A self-wire may join two *different* ports of one cell. There is no edge from a port to itself. An agent-free cycle is represented separately by ℓ , not by a fictitious self-pair of an incident port.

Two cells form an active pair exactly when their principal ports are matched. Distinct active pairs have disjoint cell sets. Port ordering is structural: forgetting it changes the semantics. We follow Lafont’s numbered-port convention, including the different $\gamma\gamma$ and $\delta\delta$ annihilations [41, pp. 71,81–82].

A *normal* net has no active pair. An *agent-free* net has no cells; it may still have free-port wires and bare loops. A *completely empty* closed net has no cells and $\ell = 0$. These are different predicates.

140.2.2 The six local templates

The following is the literal convention used by the implementation. For a pair of like binary cells u, v , write

$$B = (u1, u2, v1, v2), \quad J_\delta = \{13, 24\}, \quad J_\gamma = \{14, 23\}, \quad (140.2)$$

where ij denotes an unordered pair of positions in B . Delete the two cells and their principal wire; insert the two joins specified by J ; then suppress the old auxiliary ports, counting every closed component. An $\varepsilon\varepsilon$ interaction simply removes its two cells and principal wire.

An $\varepsilon\alpha$ interaction, with α binary, creates two fresh erasers, whose principal ports replace the two old auxiliary endpoints of α . For a mixed pair $\gamma(u), \delta(v)$, create

$$D_1, D_2 : \delta, \quad G_1, G_2 : \gamma.$$

Replace old endpoint ui by D_i0 and vj by G_j0 , and add the four internal wires

$$D_ij \text{ --- } G_ji \quad (i, j \in \{1, 2\}). \quad (140.3)$$

These equations specify the numbered rule table; they are not schematic instructions to exchange arbitrary auxiliary ports. The new roles are ordered as D_1, D_2, G_1, G_2 , or as the two erasers in auxiliary-port order. No template creates a bare loop on its own.

140.2.3 Fresh names and executions

Initially the cells are labelled $0, \dots, C_0 - 1$. The next unused identifier is $h = C_0$. New agents receive $h, h + 1, \dots$ in template-role order and h is increased accordingly. Identifiers are never reused, and

deleted cells are not renumbered. At step t the scheduler supplies the unordered active pair, represented uniquely as $u_t < v_t$.

An execution records these choices and the successive exact nets. Fixing the choices fixes every state. The fresh-name convention is equivalently a normalization of role-labelled executions under consistent renaming of newly created agents, fixing the original cells and free ports. It does not identify independent reduction schedules or solve graph isomorphism.

[write] Manuscript 15's definition and ordered-port conventions. Its rule table is Table 20 and is not printed again; its mixed rule (140.4) and its gluing data (141.4) (Section 141) are used by its orbit compiler (Section 146).

140.3 Port-labelled nets and the complete rule table [15]

Definition 140.3. A finite net consists of a finite set of labelled agents, a type γ , δ , or ε for each agent, a finite set of labelled free ports, a perfect matching of all agent and free ports, and a number $\ell \in \mathbb{N}$ of cyclic wires that have no ports. A γ or δ agent has ports 0, 1, 2; an ε agent has port 0. Port 0 is principal. A pair of agents is active precisely when their principal ports are matched.

This is the convention of Lafont's original definition, which permits both self-wires between different ports of an agent and cyclic wires with no ports [41, pp. 71–72]. A wire from a port to itself is not permitted: a matching always pairs two distinct ports. Parallel edges in an intermediate gluing graph, on the other hand, are essential.

The principal–principal edge of an active pair belongs to the redex and is removed with that redex. It is *not* automatically converted into a cyclic wire. Thus an isolated $\varepsilon\varepsilon$ pair disappears without creating a loop.

140.3.1 Literal ordered-port conventions

Suppose agents a, b are active. Write a_i, b_i for auxiliary ports before removal. The boundary positions in the following table denote those former auxiliary incidences. Each new agent gets a distinct fresh identity, in a fixed order chosen by the compiler.

The annihilation conventions are different. Lafont explicitly warns that it is $\gamma\gamma$, not $\delta\delta$, that exchanges the numbered auxiliary ports [41, p. 82]. Looking only at whether the two strands cross in an unnumbered drawing is unsafe because rotating an agent reverses their left-to-right placement.

For a mixed pair, orient the notation so that a is the γ agent and b is the δ agent. Create δ agents D_1, D_2 and γ agents G_1, G_2 . The replacement has

$$\begin{aligned} a_i &\longleftrightarrow (D_i)_0 & (i = 1, 2), \\ b_j &\longleftrightarrow (G_j)_0 & (j = 1, 2), \\ (D_i)_j &\longleftrightarrow (G_j)_i & (i, j = 1, 2). \end{aligned} \tag{140.4}$$

This is the numbered-port form of the commutation picture in [41, Fig. 2]. It makes the template independently implementable without inferring port identities from a drawing.

[write] The repository's witness pair [63]: two closed nets of four δ cells with the same labelled underlying graph, the same port-type histogram and the same unique initial active pair, of which one reaches an agent-free net with two cyclic wires and the other a normal form with two agents. All three manuscripts replay it; manuscript 13's text comes first, manuscript 15's table and complete argument follow, and manuscript 14 derives the successors from its kernel formulas (Section 143).

140.4 The repository obstruction, now with exact successors [13]

For comparison with the repository, this example uses its zero-based port code $3c + p$. Let A, B, C, D denote cells 0, 1, 2, 3, all of type δ . The two original matchings are

$$M_A = \{\{0, 3\}, \{1, 7\}, \{2, 9\}, \{4, 10\}, \{5, 6\}, \{8, 11\}\}, \quad (140.5)$$

$$M_B = \{\{0, 3\}, \{1, 7\}, \{2, 9\}, \{4, 10\}, \{5, 8\}, \{6, 11\}\}. \quad (140.6)$$

Both have no isolated cycles. Their common unique initial active pair is AB . Annihilation gives

$$M'_A = \{\{C0, D0\}, \{C1, D1\}, \{C2, D2\}\}, \quad (140.7)$$

$$M'_B = \{\{C0, D2\}, \{C1, D1\}, \{C2, D0\}\}. \quad (140.8)$$

The first successor has active pair CD ; its annihilation leaves no cells and two cyclic wires. The second has no active pair and retains two cells. The original nets have the same labelled K_4 graph and port-pair histogram $(1, 0, 2, 2, 0, 1)$ in the order $(00, 01, 02, 11, 12, 22)$ [63]. Our replay recovers these successors by both gluing algorithms. It does not identify the two-loop target with the empty net.

The code uses positive live-port identifiers $3c + p + 1$, reserving zero for inactive memory. Its saved example traces therefore add one to the original port codes. Both inputs normalize; the distinguishing predicate is *agent-free reachability*, not the existence of a normal form.

140.5 The repository obstruction: the two four-agent nets [15]

Use four δ agents A, B, C, D and zero initial cyclic wires. The source note [63] gives the following exact port matchings:

Underlying edge	Net $\mathcal{A}$	Net $\mathcal{B}$
AB	A_0B_0	A_0B_0
AC	A_1C_1	A_1C_1
AD	A_2D_0	A_2D_0
BC	B_2C_0	B_2C_2
BD	B_1D_1	B_1D_1
CD	C_2D_2	C_0D_2

Both underlying labelled graphs are K_4 . In the port-type order $(00, 01, 02, 11, 12, 22)$, both histograms are $(1, 0, 2, 2, 0, 1)$. In each net the only initial active pair is AB .

Annihilate that pair using the same-index $\delta\delta$ rule. The resulting wires are

$$\mathcal{A}_1 : C_0D_0, C_1D_1, C_2D_2, \quad \ell = 0,$$

$$\mathcal{B}_1 : C_0D_2, C_1D_1, C_2D_0, \quad \ell = 0.$$

The first net has active pair CD . Its annihilation yields two two-edge cycles and no agents. The second has no active pair and retains two agents. Since it had only one possible first step, it cannot reach any agent-free net. This is a complete proof for the two examples, independently of the exhaustive search in either the repository or our archive.

[write] The two readings agree: manuscript 13's M_A, M_B in the zero-based code $3c + p$ are manuscript 15's $\mathcal{A}, \mathcal{B}$ with A, B, C, D the cells 0, 1, 2, 3. The example and its interpretation are the research note's; the three manuscripts add the exact arithmetic interfaces of the following sections.

141 Exact gluing and loop accounting

[write] *Order-independent gluing with exact loop accounting is proved by all three manuscripts: manuscript 13's splice lemma and theorem (Lemma 141.1 and Theorem 141.2), printed first; manuscript 15's division-free matrix identity (Theorem 141.4), which is the splice lemma for natural matching matrices without a case-selection witness, with its corollary (Corollary 141.5) and its sixteen-port support bound (Proposition 141.6); and manuscript 14's locality lemma (Lemma 141.7) and suppression proposition (Proposition 141.8). All three credit the established permutation-based gluing of de Falco [68] and the Brauer-category setting [69], and claim no new associativity theorem.*

141.1 Matching closure and exact local splicing [13]

141.1.1 Matchings and cyclic components

Let X be a finite set of even cardinality. A perfect matching M on X is equivalently a fixed-point-free involution $m : X \rightarrow X$. Write $\mathcal{M}_n$ for the matchings on $[2n] = \{0, \dots, 2n-1\}$ and

$$D_n = |\mathcal{M}_n| = (2n-1)!! = \frac{(2n)!}{2^n n!}, \quad D_0 = 1. \quad (141.1)$$

The recurrence $D_n = (2n-1)D_{n-1}$ follows by choosing the partner of one specified port and then matching the rest.

For two matchings M, Q on the same set, form the *edge-coloured multigraph* $M \cup Q$. It is a disjoint union of alternating even cycles. An edge common to M and Q occurs twice and is a two-edge cycle, not a single edge. Let

$$c(M, Q) = \text{the number of components of this doubled multigraph.} \quad (141.2)$$

Closing the ports of the wiring M by the wiring Q produces exactly $c(M, Q)$ agent-free cyclic wires.

More generally, a state of a wiring evaluator is a matching on live ports, together with a natural number ℓ of already isolated cyclic wires. Those cycles have no incident ports and cannot be rediscovered by traversing the live matching. Consequently, ℓ is essential state data whenever exact loop counts matter.

141.1.2 The splice operation

Suppose a, b are distinct live ports and a new join is to identify their wire ends, after which a, b disappear. Read

$$p = m(a), \quad q = m(b). \quad (141.3)$$

There are exactly two cases.

If $p = b$, involutivity gives $q = a$. The old wire and the new join form a two-edge cycle. Delete a, b and increment ℓ by one.

If $p \neq b$, the four ports a, b, p, q are distinct. Delete a, b and replace the two old wires by $p \leftrightarrow q$. The loop count does not change.

Lemma 141.1 (Exact splice). *The preceding operation produces precisely the matching and cyclic-wire count obtained by adding the join $a-b$ and suppressing its two endpoints. It requires two mate reads and at most four mate writes when deletion is represented by writing zero.*

Proof. If a, b were matched, the component is the pair of parallel edges just described. Otherwise the affected component is the path $p-a-b-q$. Suppressing the two internal vertices leaves the wire $p-q$. All other edges and all previously isolated cycles are unchanged. There is no third case because m is a fixed-point-free involution. $\square$

Theorem 141.2 (Sequential gluing is order independent). *Let J be a set of pairwise disjoint joins on live ports. Add these joins to the old matching, and declare all their endpoints to be suppressed. The result is a matching on the unsuppressed ports plus one isolated wire for each component with no unsuppressed endpoint. Performing the splices in any order gives this same result.*

Proof. In the temporary multigraph, every suppressed port has degree two and every unsuppressed port has degree one. A connected component is therefore a path with two unsuppressed endpoints, or an alternating cycle with none.

At any intermediate stage, maintain the following invariant: each live wire represents the unsuppressed remainder of a path in the original temporary graph; each completed component with no remaining endpoint has been recorded once in the loop counter. Lemma 141.1 contracts one such path or completes one such cycle. It neither merges two original components nor changes their surviving endpoints. At the end, every path has become its endpoint wire and every cycle has been counted exactly once. This description is independent of the splice order. $\square$

Remark 141.3 (Two easy but consequential implementation errors). Parallel edges must not be deduplicated: doing so destroys the first case of Lemma 141.1. Also, the reads for a later splice must use the *updated* mate table. Reading all incident wires once and then applying several stale updates is not equivalent to the theorem. The attached two evaluators intentionally use different algorithms: local pointer splicing and connected components of a multigraph.

[write] Manuscript 15's gluing data and its division-free form of the splice. Its α is the temporary matching of old and replacement wires and its β the involution of the new joins; Theorem 141.4 is Lemma 141.1 written as an identity of natural matching matrices, and Corollary 141.5 is Theorem 141.2.

141.2 Gluing a replacement into its context [15]

For a uniform implementation, do not immediately redirect the old auxiliary wires. Instead, remove just the two principal ports and their common edge. Keep the old auxiliary ports temporarily. Take a disjoint copy of the replacement, with one new interface port for every old auxiliary port. Match the replacement ports according to its rule template. Finally, join each old auxiliary port to its corresponding interface port and suppress these joined ports.

All old and replacement wires together form a perfect matching α of a temporary vertex set V . The new joins form a second matching on a subset $I \subseteq V$. Let $B = V \setminus I$ be the surviving ports. Extending the second matching by the identity on B gives an involution β with

$$\alpha^2 = \text{id}, \quad \alpha(v) \neq v, \quad \beta^2 = \text{id}, \quad \text{Fix}(\beta) = B. \quad (141.4)$$

An internal vertex has degree two, one edge of each kind. A boundary vertex has one actual edge; a fixed point of β is bookkeeping, not an additional self-loop. Every component of the temporary multigraph is therefore a path between two boundary vertices or a cycle with no boundary vertex.

141.3 Division-free elimination and locality [15]

141.3.1 An exact quadratic update

Encode a perfect matching by a symmetric matrix W with zero diagonal and entries in $\mathbb{N}$, subject to

$$\sum_v W_{uv} = 1 \quad \text{for every } u. \quad (141.5)$$

These equations already force every entry to be 0 or 1. Explicit Boolean constraints on the off-diagonal entries are unnecessary.

Theorem 141.4 (Pair-suppression identity). *Let $x \neq y$ be ports that are to be joined and suppressed. For remaining ports $u \neq v$, the resulting matching matrix and cyclic-wire count satisfy*

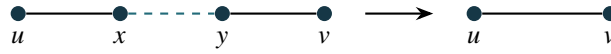
$$W'_{uv} = W_{uv} + W_{ux}W_{yv} + W_{uy}W_{xv}, \quad (141.6)$$

$$\ell' = \ell + W_{xy}. \quad (141.7)$$

The diagonal of W' is zero. These formulas are exact over natural matching matrices and use neither division nor a case-selection witness.

Proof. If $W_{xy} = 1$, x, y are already matched. The old edge and the newly added join form a two-edge cycle. Every other entry in the x and y rows is zero, so (141.6) leaves the surviving matching unchanged and (141.7) adds exactly one loop.

Otherwise let u_0 be the partner of x and v_0 the partner of y . They are distinct surviving ports. The old edges u_0x and yv_0 , together with the join xy , become a single edge u_0v_0 . The two product terms in (141.6) add exactly the symmetric entries for this edge. No surviving edge incident with u_0 or v_0 was already present because their old partners were x, y . All other edges are unchanged. No closed component is created in this case. $\square$



A path is shortened; a closed component is not invented.

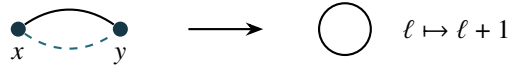


Figure 2: Solid edges are old matching edges; dashed edges are joins. A pair of parallel edges is a genuine cyclic component, even though it has only two vertices.

Corollary 141.5 (Order-independent loop accounting). *Suppress the pairs of the internal matching in any order. The final boundary matching and the total increment of ℓ are independent of the order.*

Proof. Within a path component, each suppression shortens the same path and never closes a component. Its two surviving endpoints are unchanged. Within a cycle component, successive suppressions shorten that cycle until its last join meets an already present edge; that final suppression contributes exactly one to ℓ . Different connected components do not interact. Consequently the result consists of one edge per original path and one loop per original cycle, independently of the suppression order. $\square$

This provides one direct quartic implementation: introduce the intermediate matching matrices and impose (141.6)–(141.7) for each internal join. The compiler in Section 146.4 instead uses a single orbit certificate for the whole gluing. It avoids these additional dense intermediate matrices.

141.3.2 Only sixteen ports can participate in nontrivial surgery

Proposition 141.6 (Support-local gluing). *Let $I = \text{supp}(\beta)$ and $S = I \cup \alpha(I)$. Then S is invariant under both α and β . On $V \setminus S$, the gluing does nothing: its components are already surviving matching edges. In particular, $|S| \leq 2|I|$.*

For one interaction-combinator step, $|I| \leq 8$ and therefore $|S| \leq 16$.

Proof. The identity $\alpha^2 = \text{id}$ gives $\alpha(S) = \alpha(I) \cup I = S$. The involution β permutes I and fixes every point outside I , so it preserves S as well. Outside S , both endpoints of an α edge are outside I , hence are boundary ports, and β fixes them. Each consumed agent has at most two auxiliary ports. There are at most four old auxiliaries and four interface copies, which are precisely the moved points of β . $\square$

The bound does not mean that the current dense arithmetic exporter has constant-size memory access. Locating $\alpha(I)$ inside an unknown global matching still costs something. It identifies a sharp place for future sparse memory work: the actual topological surgery is of bounded size; global addressing and untouched-memory preservation are the remaining expenses.

[write] Manuscript 14's locality lemma and order-independence. Proposition 141.8 is Theorem 141.2 again, with the same component argument; Lemma 141.7 adds the locality of suppression that its kernel uses.

141.4 The local gluing theorem [14]

141.4.1 Why no global path search is necessary

Suppose a rewrite removes two cells. Delete their principal wire, keep every other old wire, and add the right-hand-side template, identifying its boundary ports with the former auxiliary ports of the removed cells. In the resulting temporary multigraph each such auxiliary port has degree two. Every surviving incident port has degree one. Thus every nonempty component is either a path with two surviving endpoints or a cycle with none.

Lemma 141.7 (Locality of suppression). *Every temporary component changed by a fixed interaction rule has all of its internal vertices in that rule's auxiliary boundary. Suppressing these vertices replaces each path by its endpoint wire and each cycle by one bare loop. In particular, for the binary rules all changed paths have at most four internal vertices, independently of the size and topology of the surrounding net.*

Proof. An old wire ends at an incident port. It does not continue through the agent owning that port. Only a boundary port receives a second temporary edge from the replacement. A surviving port therefore terminates a path rather than passing it into the rest of its agent. Degree two gives continuation without branching at an internal vertex; degree one gives a unique endpoint. Finite components with no degree-one vertex are cycles. These statements also hold when two vertices are joined by two parallel edges. Such a component is a two-edge cycle and must not be simplified to one edge. $\square$

The lemma is compatible with the established execution-formula account of port fusion [68]; its purpose here is to expose the bounded arithmetic workload. In particular, an arbitrary graph-connectivity subroutine is unnecessary.

141.4.2 Composition and loop accounting

Proposition 141.8 (Suppression is order-independent). *For a finite temporary wiring with designated degree-two internal vertices, suppressing those vertices in any order gives the same boundary matching and the same total number of newly closed components. The loop count is additive over intermediate suppression stages.*

Proof. A path component retains its two boundary endpoints until it becomes one wire. It cannot produce a closed component during suppression. A cycle component never acquires a boundary endpoint, and its last suppression produces exactly one bare loop. Distinct original components never

merge. Classifying components before suppression therefore determines both the final matching and the sum of all loop increments, independent of order. $\square$

This elementary proof is included to make the loop invariant explicit, not to claim a new associativity theorem for interaction-net gluing.

[write] Manuscript 14’s “at most four internal vertices” (Lemma 141.7) and manuscript 15’s “at most sixteen ports” (Proposition 141.6) are one locality statement with two bookkeepings: the internal vertices of the changed paths are the at most four old auxiliary ports of one step, and manuscript 15’s support adds the at most four interface copies and the old partners of these eight vertices. Neither bound is a constant-cost global lookup; both manuscripts say so.

142 Contextual information: loop parity, injectivity and tomography

[write] Manuscript 13’s Sections 3–4: the sharp loop-parity theorem (Theorem 142.1) and its consequences. They are new to the repository; the research note [63] exhibits one indistinguishable pair of coarse summaries, and Corollary 142.3 shows that every identification of two distinct matchings is detected by loop parity in at least two fifths of closing contexts. Priority for Theorem 142.1 is not certified by manuscript 13 (its Section 1 and its source audit). Manuscript 15’s contextual completeness theorem follows (Theorem 142.7); manuscript 14’s local ten-state result (Theorem 143.2) is printed with its kernel in Section 143.

142.1 A sharp theorem for one-bit wiring observations [13]

142.1.1 Statement

For a closing context Q , define its one-bit observation on M by

$$b_Q(M) = c(M, Q) \bmod 2. \quad (142.1)$$

Although this observable forgets almost the entire closed net, its family over all contexts is unexpectedly discriminating.

Theorem 142.1 (Sharp loop-parity separation). *Let $n \geq 2$ and $M, N \in \mathcal{M}_n$ with $M \neq N$. If Q is uniform on $\mathcal{M}_n$, then*

$$\frac{2}{5} \leq \mathbb{P}(b_Q(M) \neq b_Q(N)) \leq \frac{2}{3}. \quad (142.2)$$

The upper equality holds exactly when $M \cup N$ consists of one alternating four-cycle and any number of common-edge two-cycles. The lower equality holds exactly when it consists of one alternating six-cycle and any number of common-edge two-cycles. In particular the lower bound is attained for every $n \geq 3$, and the upper bound for every $n \geq 2$.

The proof will be elementary. It uses neither an asymptotic estimate nor a spectral assertion about an uncomputed matrix.

142.1.2 Cycle type and signed correlation

The *cycle half-length partition* of $M \cup N$ is the partition $\lambda \vdash n$ whose parts are half the component lengths. Thus a common edge contributes a part 1. Simultaneous relabelling acts transitively on pairs

(M, N) of the same cycle type: enumerate the vertices alternately around each component to construct the relabelling. It follows that

$$F(\lambda) = \sum_{Q \in \mathcal{M}_n} (-1)^{c(M, Q) + c(N, Q)} \quad (142.3)$$

depends only on λ . Set

$$h(\lambda) = \frac{F(\lambda)}{D_n}. \quad (142.4)$$

Then

$$\mathbb{P}(b_Q(M) \neq b_Q(N)) = \frac{1 - h(\lambda)}{2}. \quad (142.5)$$

The identity case $M = N$ is $\lambda = (1^n)$ and has $h = 1$.

142.1.3 The contraction recurrence

For a partition μ , notation such as (k, μ) means the multiset of its parts together with k , with order immaterial. Repeated parts in a sum over $\ell \in \mu$ are counted repeatedly. Set $F(\emptyset) = 1$.

Lemma 142.2 (Signed closure recurrence). *If $|\mu| = n - 1$, then*

$$F(1, \mu) = (2n - 1)F(\mu). \quad (142.6)$$

If $k \geq 2$ and $k + |\mu| = n$, then

$$\begin{aligned} F(k, \mu) &= (k - 3)F(k - 1, \mu) \\ &\quad + \sum_{i=1}^{k-2} F(i, k - 1 - i, \mu) \\ &\quad + \sum_{\ell \in \mu} 2\ell F(k + \ell - 1, \mu \setminus \{\ell\}). \end{aligned} \quad (142.7)$$

Every term on the right has total weight $n - 1$.

Proof. Choose a port a and condition on its partner x in Q . Remove the edge $\{a, x\}$ from Q . Separately in M and N , suppress a, x by Lemma 141.1. A matching containing $\{a, x\}$ contributes one closed component and hence a factor -1 to its signed loop count; otherwise its two incident edges are contracted without this factor. The remaining Q is an arbitrary matching on the remaining $2n - 2$ ports.

First suppose $\{a, b\}$ is common to M, N . If $x = b$, the two signs cancel and the common component disappears. If $x \neq b$, both signs are positive, and the old port b takes the place of x in the residual pair of matchings. In all $2n - 1$ cases the residual type is μ . This proves (142.6).

For a nontrivial component, label its ports $a_0, b_0, \dots, a_{k-1}, b_{k-1}$ so that

$$M : \{a_i, b_i\}, \quad N : \{b_i, a_{i+1}\}, \quad i \pmod{k}. \quad (142.8)$$

Choose $a = a_0$. The complete case distinction for its Q -partner is given in Table 21. The two partners b_0, b_{k-1} give the two negative terms; the $k - 1$ partners a_i give positive terms of the same residual type. Their combined coefficient is $k - 3$.

For $x = b_i$ with $1 \leq i \leq k - 2$, removing the selected ports cuts the alternating cycle into components of half-lengths i and $k - 1 - i$. For a partner in a different component of half-length ℓ , the two components merge into one of half-length $k + \ell - 1$. None of these cases closes an M - or N -edge, so all their signs are positive. They exhaust the $2n - 1$ choices of x , proving (142.7). $\square$

Partner of a_0	Sign	Residual cycle half-lengths
b_0 or b_{k-1}	–	$(k-1, \mu)$ in each case
$a_i, 1 \leq i \leq k-1$	+	$(k-1, \mu)$ in each case
$b_i, 1 \leq i \leq k-2$	+	$(i, k-1-i, \mu)$
A port in another component of half-length ℓ	+	$(k+\ell-1, \mu \setminus \{\ell\})$, with 2ℓ choices

Table 21: Exhaustive local contraction underlying the signed recurrence.

Dividing (142.6) by D_n yields the particularly useful identity

$$h(1, \mu) = h(\mu). \quad (142.9)$$

Adding common wires changes the number of closing contexts but does not change the signed correlation.

142.1.4 Proof of the sharp bounds and equality cases

Proof of Theorem 142.1. By (142.5), it suffices to show

$$-\frac{1}{3} \leq h(\lambda) \leq \frac{1}{5} \quad (142.10)$$

for every nonidentity cycle type, and to characterize equality. Remove all parts 1 using (142.9). Induct on the remaining total weight.

The first two cases follow directly from the recurrence:

$$F(2) = -1, \quad D_2 = 3, \quad F(3) = F(1, 1) = 3, \quad D_3 = 15. \quad (142.11)$$

Thus $h(2) = -1/3$ and $h(3) = 1/5$.

Suppose the partition has a part $k \geq 3$ and total weight $n > 3$. Use that part as the distinguished k in (142.7). Every coefficient is nonnegative, and their sum is

$$(k-3) + (k-2) + 2(n-k) = 2n-5. \quad (142.12)$$

Every term with positive coefficient has a nonidentity residual partition. Indeed, the merged part is at least 4; for $k \geq 4$ the split parts have sum at least 3; and for $k = 3$ the other nontrivial parts prevent the split term from becoming the identity. The sole exception to this reasoning is (3) itself, already treated.

Divide the recurrence by $D_n = (2n-1)D_{n-1}$ and apply the induction hypothesis to every residual correlation. This gives

$$-\frac{2n-5}{3(2n-1)} \leq h(\lambda) \leq \frac{2n-5}{5(2n-1)}. \quad (142.13)$$

Both endpoints are strictly inside the interval in (142.10).

It remains to handle $\lambda = (2^r)$ with $r \geq 2$. The recurrence and (142.9) give

$$h(2^r) = \frac{-h(2^{r-1}) + 4(r-1)h(3, 2^{r-2})}{4r-1}. \quad (142.14)$$

Both correlations on the right have smaller active weight. Consequently

$$\frac{-1/5 - 4(r-1)/3}{4r-1} \leq h(2^r) \leq \frac{1/3 + 4(r-1)/5}{4r-1}. \quad (142.15)$$

The left endpoint exceeds $-1/3$, since the numerator after moving $-(4r-1)/3$ across is $4/5 > 0$. The right endpoint is less than $1/5$, since $1/3 < 3/5$. Thus these cases are also strict.

The only equality cases after removing parts 1 are therefore (2) and (3). Equation (142.5) gives exactly the asserted constants and classification. $\square$

142.1.5 Small values and an independent finite check

Table 22 illustrates that the probability is not a function of the number of differing edges alone. The exact recurrence is implemented in `code/13-exact-wiring-parity.py`. The verification script independently enumerates closing matchings and counts multigraph components for every partition of weight at most six: 29 partition cases and 121,537 closures. It also checks the bounds and equality cases on all 28,598 nonidentity partitions of weight at most thirty. These finite calculations detect implementation and transcription errors; the induction above, not the finite range, proves the theorem.

Nontrivial cycle type	$h(\lambda)$	Parity disagreement probability
(2)	$-1/3$	$2/3$
(3)	$1/5$	$2/5$
(4)	$-1/15$	$8/15$
(2, 2)	$17/105$	$44/105$
(5)	$1/21$	$10/21$
(3, 2)	$-1/15$	$8/15$
(6)	$-13/1155$	$584/1155$
(3, 3)	$17/385$	$184/385$
(4, 2)	$47/1155$	$554/1155$
(2, 2, 2)	$-73/1155$	$614/1155$

Table 22: Exact signed closure correlations. Common-edge parts 1 may be added without changing either numerical column.

142.2 Consequences for abstraction, information, and tomography [13]

142.2.1 A lossless-information obstruction

Corollary 142.3 (One-bit contextual observations force injectivity). *Suppose $s : \mathcal{M}_n \rightarrow \Sigma$ is a summary map and a function g satisfies*

$$g(s(M), Q) = b_Q(M) \quad (M, Q \in \mathcal{M}_n). \quad (142.16)$$

Then s is injective. In particular, $|\Sigma| \geq D_n$.

Proof. If $s(M) = s(N)$ for distinct matchings, the displayed equality gives $b_Q(M) = b_Q(N)$ for every Q . This contradicts the positive lower bound in Theorem 142.1. $\square$

This is stronger than an isolated pair of indistinguishable summaries: it applies to *every* identification of two distinct matchings, and only requires parity of the final loop count. Its assumptions are nevertheless specific. The contexts range over all labelled bare wirings. A summary for one restricted family of contexts, or an overapproximating reachability analysis, is not ruled out.

Corollary 142.4 (State-bit lower bound). *A fixed-width binary summary satisfying Corollary 142.3 requires at least $\lceil \log_2 D_n \rceil$ bits. Moreover,*

$$\log_2 D_n = n \log_2 n + (1 - \log_2 e)n + \frac{1}{2} - \frac{1}{24n \log 2} + O(n^{-3}). \quad (142.17)$$

Thus a mate table using $O(n \log n)$ bits has the correct asymptotic order for contextually exact state information.

Proof. There are at most 2^b states of a b -bit register, so injectivity implies $2^b \geq D_n$. Subtracting the Stirling expansions of $\log(2n)!$ and $\log n!$ in (141.1), and dividing by $\log 2$, gives (142.17). The $1/n$ coefficient is $1/24 - 1/12 = -1/24$ in natural logarithms. A table of $2n$ partner identifiers uses $2n\lceil\log_2(2n)\rceil$ bits, up to a constant convention for labels. $\square$

This is an information lower bound, not a Diophantine variable lower bound. A single unbounded natural number can encode arbitrarily much finite information. Nor does it prove the bit optimality of the large-height history certificate constructed later.

142.2.2 A short distinguishing context for every modulus

The following result supplies a deterministic, very local witness to contextual separation, without the random theorem.

Proposition 142.5 (Two-edge contextual separation). *For distinct $M, N \in \mathcal{M}_n$ and every integer $r \geq 2$, there is a matching Q , equal to M or differing from it in exactly two edges, such that*

$$c(M, Q) \not\equiv c(N, Q) \pmod{r}. \quad (142.18)$$

Proof. Put $d = n - c(M, N) > 0$. If $d \not\equiv 0 \pmod{r}$, use $Q = M$, since $c(M, M) = n$.

Otherwise choose two M -edges in one nontrivial alternating component of $M \cup N$. Removing those edges cuts the component into two paths. Of the two nonoriginal pairings of their four endpoints, one closes the paths separately and one reconnects them into a single cycle. Choose the latter and leave every other M -edge unchanged; call the result Q . Then $c(N, Q) = c(N, M)$, whereas $c(M, Q) = n - 1$: the two changed edges form one four-cycle in $M \cup Q$, and all other edges are common. The difference is $d - 1$, which is nonzero modulo r . $\square$

The attached code checks this construction on every pair of matchings of at most eight ports, for moduli 2, 3, 4, 5, giving 22,272 tests. The proposition also clarifies the role of the random theorem: existence of a separator is much easier than a sharp lower bound on the density of separators.

142.2.3 An order-optimal number of nonadaptive one-bit probes

A *probe family* is a list $Q_1, \dots, Q_K$ fixed before the unknown matching is seen. Its observation vector is

$$\Phi(M) = (b_{Q_1}(M), \dots, b_{Q_K}(M)) \in \{0, 1\}^K. \quad (142.19)$$

Theorem 142.6 (Random simultaneous tomography). *For $n \geq 2$ and $0 < \varepsilon < 1$, choose the probes independently and uniformly from $\mathcal{M}_n$. If*

$$K \geq \left\lceil \frac{\log \binom{D_n}{2} + \log(1/\varepsilon)}{\log(5/3)} \right\rceil, \quad (142.20)$$

then Φ is injective with probability at least $1 - \varepsilon$. Consequently a separating family of $O(n \log n)$ one-bit probes exists. Every separating one-bit family has $K \geq \lceil \log_2 D_n \rceil$.

Proof. For a fixed pair $M \neq N$, a single probe fails to distinguish them with probability at most $3/5$. Independence makes the probability of agreement on all probes at most $(3/5)^K$. A union bound over the $\binom{D_n}{2}$ pairs gives failure probability at most $\binom{D_n}{2} (3/5)^K \leq \varepsilon$. The asymptotic statement follows from (142.17). Finally an injective map to $\{0, 1\}^K$ has at most 2^K distinct inputs. $\square$

This is an existence and sampling theorem, not an efficient decoding algorithm. Exhaustively testing injectivity over D_n matchings is not a polynomial-time construction in n . Random probes are also not used in the Diophantine compiler: its soundness will be exact for every witness tuple.

[write] Manuscript 15's contextual completeness theorem. Its necessity half is a weaker form of Corollary 142.3 and Proposition 142.5: it separates distinct summaries $(M, \ell) \neq (M', \ell')$ by loop counts under the two closures M and M' , where manuscript 13 separates distinct matchings by loop parity under at least two fifths of all closures (Theorem 142.1) and by a two-edge context modulo every $r \geq 2$; manuscript 15's closure by M is the first case of the proof of Proposition 142.5. Its sufficiency half, that (M, ℓ) determines the result of every further matching gluing, is not stated by manuscript 13. Both proofs are kept.

142.3 A complete summary for matching contexts [15]

The following statement concerns wire diagrams, with agents treated as external boundary attachments. It does not say that arbitrary interaction nets are determined by a boundary matching independently of their internal agents.

Theorem 142.7 (Contextual completeness of the wire summary). *A finite wire-only diagram with labelled boundary B , all nonboundary vertices of degree two, and any pre-existing cyclic wires is completely described for future matching gluings by its suppressed boundary matching M and its total loop count ℓ .*

Conversely, two such summaries (M, ℓ) and (M', ℓ') give the same closed-loop count in every perfect-matching closure of B if and only if $M = M'$ and $\ell = \ell'$.

Proof. Sufficiency follows from Corollary 141.5: suppressing the internal vertices before or after any further gluing gives the same result.

For necessity put $|B| = 2m$. For boundary matchings M, Q , let $c(M, Q)$ be the number of components in the union of the two matchings, retaining parallel edges. Every component contains at least one edge of M , so $c(M, Q) \leq m$. Equality holds exactly when all components have two edges, equivalently $M = Q$.

Suppose $M \neq M'$ and write $c = c(M, M') < m$. Closure by M would force $\ell + m = \ell' + c$ if the observed counts agreed. Closure by M' would force $\ell + c = \ell' + m$. Subtracting these equalities gives $2(m - c) = 0$, a contradiction. If $M = M'$, any closure distinguishes unequal ℓ, ℓ' . For $B = \emptyset$, the unique empty closure directly reads the loop count. $\square$

This places the repository's obstruction in a positive framework. Counts and histograms are not sufficient, but the exact matching and closed-loop scalar are sufficient for all purely wiring contexts. This is consonant with classical Brauer-diagram composition [69]; the arithmetic certificate below makes the summary available to a polynomial verifier.

[write] The local analogue is manuscript 14's ten-state theorem Theorem 143.2 (Section 143): on the four boundary positions of one annihilation, a summary from which the external positions and both annihilation loop counts can be read must be injective on the ten partial matchings.

143 The local annihilation kernel

[write] Manuscript 14's Sections 3.2–3.3 and 4: closed formulas for the new loops and surviving endpoints of a binary annihilation (Theorem 143.1), with both annihilation matchings (143.5); the ten-state minimality theorem (Theorem 143.2); and the 23-residual canonical quartic kernel (Theorem 143.3). Its Sections 3.1 and 3.4 are in

Section 141. The formula for c_δ is the one derived independently in the research note [64]; see the note after the table of patterns.

143.1 The ten boundary patterns [14]

For binary annihilation let M be the matching formed by old wires whose *both* endpoints lie in B . It is a partial matching on four labelled positions. The remaining positions have distinct external mates q_i . There are

$$1 + \binom{4}{2} + 3 = 10 \quad (143.1)$$

possible M : the empty matching, six single-edge matchings, and three perfect matchings. A position unmatched by M is called external. Let $m_{ij} \in \{0, 1\}$ be the six edge indicators and put

$$e_i = 1 - \sum_{j \neq i} m_{ij}. \quad (143.2)$$

For a legal pattern e_i is exactly the external-position indicator. We use $m_{ij} = m_{ji}$ in formulas.

Write

$$\mathcal{P} = \{\{12, 34\}, \{13, 24\}, \{14, 23\}\}$$

for the three perfect matchings. Fix an annihilation matching $J \in \mathcal{P}$. Its involution sends i to $J(i)$.

Theorem 143.1 (Closed formulas for exact gluing). *The number of newly created bare loops is*

$$c_J(M) = \sum_{ij \in J} m_{ij} + \sum_{K \in \mathcal{P} \setminus \{J\}} \prod_{ij \in K} m_{ij}. \quad (143.3)$$

For $i < j$, the surviving endpoints q_i, q_j are joined exactly when

$$g_{ij} = e_i e_j \begin{cases} 1, & ij \in J, \\ m_{J(i), J(j)}, & ij \notin J. \end{cases} \quad (143.4)$$

The indicators g_{ij} form a perfect matching of the external positions.

Proof. If M is empty, each of the two J edges makes a path between two external mates, and no cycle is created. If M has a single edge, that edge and a coincident J edge form a two-edge cycle precisely when the edge belongs to J . Otherwise the old edge links the two J edges into one longer path between the two external positions. If M is perfect and equals J , there are two two-edge cycles. If M is perfect and differs from J , their union is one alternating four-cycle. These cases give (143.3).

For (143.4), an external position i first follows its J edge. If this immediately reaches external position j , then $ij \in J$. Otherwise both $J(i)$ and $J(j)$ must be the remaining internal positions, and the only possible path uses the old edge between them. Conversely that edge gives exactly the required path. All components are disjoint paths or cycles by Lemma 141.7, so each external position has one and only one partner. $\square$

In particular,

$$\begin{aligned} c_\delta &= m_{13} + m_{24} + m_{12}m_{34} + m_{14}m_{23}, \\ c_\gamma &= m_{14} + m_{23} + m_{12}m_{34} + m_{13}m_{24}. \end{aligned} \quad (143.5)$$

These are genuine quadratic expressions, including the all-internal case. Counting only coincident old and new edges would miss the alternating four-cycle.

[write] Batch 78, 2 October 2026. The same ten-pattern formula for the $\delta\delta$ rule was derived independently in the Hilbert's-tenth-problem research note [64], §2, as $c = e_{02} + e_{13} + e_{01}e_{23} + e_{03}e_{12}$:

Table 23: Every local pattern for a fixed annihilation matching J .

Pattern	Number	External positions	New loops
$M = \emptyset$	1	4	0
One edge belonging to J	2	2	1
One edge not belonging to J	4	2	0
$M = J$	1	0	2
Perfect $M \neq J$	2	0	1

with that note's positions 0, 1, 2, 3 read as manuscript 14's 1, 2, 3, 4, it is c_δ of (143.5). The note also gives the surviving-endpoint routing as a quadratic system with shared witnesses and charges one complete selected $\delta\delta$ step of the four-cell example at 472 arithmetic operations. Manuscript 14 was written at 6914ccca6, before that note (commit b7a9404d4); the note does not cite the manuscript. Manuscript 14 adds the $\gamma\gamma$ case and the minimality theorem below. Priority is claimed for neither. In the merge, the twenty (pattern, rule) cases of (143.3)–(143.4) were checked against component traversal and against the note's formula; they agree.

143.2 A sharp local abstraction result [14]

The previous obstruction concerns summaries of whole nets. There is also a small, exact lower bound at a single boundary.

Theorem 143.2 (Ten distinguishable boundary states). *Let σ be a summary of partial matchings on the four labelled boundary positions. Suppose the external-position set and both numbers $c_\delta(M)$, $c_\gamma(M)$ can be recovered from $\sigma(M)$. Then σ is injective on the ten partial matchings. Consequently such a summary needs at least ten states, or at least four bits in a fixed-width binary encoding. Conversely the partial matching itself is a sufficient ten-state summary for both annihilation rules, provided the ordered external addresses are retained separately.*

Proof. Four external positions force $M = \emptyset$. Two external positions identify their complementary pair, which must be the unique internal edge. With no external positions, M is perfect. For the three possibilities $\{12, 34\}$, J_δ , J_γ , the observed cycle-count pairs are respectively $(1, 1)$, $(2, 1)$, $(1, 2)$. They are distinct. Sufficiency follows from Theorem 143.1. $\square$

The observation class is essential. For one fixed annihilation rule and no external positions, the two nonmatching perfect patterns both produce one loop; that weaker observation does not distinguish them. Nor does the theorem say that ten states can encode arbitrary external port identifiers.

143.3 A 23-residual quartic topology kernel [14]

143.3.1 All equations and all witnesses

Take the six natural numbers m_{ij} as source coordinates. Introduce the four e_i , six h_{ij} , six g_{ij} , and one c as auxiliary coordinates: 17 in total. Impose

$$m_{ij}(m_{ij} - 1) = 0 \quad (i < j), \quad (143.6)$$

$$e_i + \sum_{j \neq i} m_{ij} - 1 = 0 \quad (1 \leq i \leq 4), \quad (143.7)$$

$$h_{ij} - e_i e_j = 0 \quad (i < j), \quad (143.8)$$

$$g_{ij} - h_{ij}f_{ij} = 0 \quad (i < j), \quad (143.9)$$

$$c - \sum_{ij \in J} m_{ij} - \sum_{K \in \mathcal{P} \setminus \{J\}} \prod_{ij \in K} m_{ij} = 0, \quad (143.10)$$

where $f_{ij} = 1$ for $ij \in J$ and $f_{ij} = m_{J(i), J(j)}$ otherwise. These are $6 + 4 + 6 + 6 + 1 = 23$ residuals, all of degree at most two. Let K_J be the sum of their squares.

Theorem 143.3 (Canonical quartic gluing). *For source tuple $\mathbf{m} \in \mathbb{N}^6$, the equation $K_J = 0$ has a natural solution exactly when $\mathbf{m}$ is the indicator of a partial matching. In that case its 17-coordinate extension is unique and its outputs are precisely (143.3)–(143.4). The polynomial has degree at most four.*

Proof. A sum of integer squares is zero exactly when every summand is zero. Equation (143.6) forces the six indicators to be Boolean. Nonnegativity of e_i in (143.7) forces each position to meet at most one internal edge; it also uniquely determines e_i . The remaining equations determine successively h_{ij} , g_{ij} , and c . Theorem 143.1 proves their semantic meaning and nonnegativity. Every legal partial matching supplies this extension. Squaring quadratic residuals gives the degree bound. $\square$

Thus the scalar cycle output is not an existentially guessed component count. It is forced. The supplied expanded δ and γ quartics each have 104 nonzero monomials. This is an executable count for the stated presentation, not an optimality claim.

[write] Batch 78, 2 October 2026. The review [65] of the Hilbert’s-tenth-problem research tree derives a smaller presentation of this kernel on the natural domain: the four equations (143.7) already force every natural source entry m_{ij} to be Boolean, so the six rows (143.6) are redundant, and for the two edges of J the helper h_{ij} can be eliminated by $g_{ij} = e_i e_j$. This gives 15 residuals and 15 helpers instead of 23 and 17, with the same natural zero set and uniquely restored helpers. It is the review’s result, recorded here; the shipped exports are unchanged.

[write] The witness pair of Section 140.4, read through the kernel.

143.3.2 The repository witness pair, repaired rather than abstracted away

For completeness, use the integer port identifiers $3c + p$ only in this example, as in the motivating note. The initial pair of closed all- δ nets has the following complete wire lists:

$$\begin{aligned} A : & (0, 3), (1, 7), (2, 9), (4, 10), (5, 6), (8, 11), \\ B : & (0, 3), (1, 7), (2, 9), (4, 10), (5, 8), (6, 11). \end{aligned} \quad (143.11)$$

Both inputs have unique active pair $(0, 1)$. Its local old boundary matching is empty, but its ordered external mates differ. Formula (143.4) produces

$$\begin{array}{l|l} A \text{ after the first step} & C0 \text{ --- } D0, C1 \text{ --- } D1, C2 \text{ --- } D2 \\ B \text{ after the first step} & C0 \text{ --- } D2, C1 \text{ --- } D1, C2 \text{ --- } D0. \end{array}$$

For A the remaining pair annihilates with $M = J_\delta$, hence $c = 2$. For B there is no remaining active pair. Our certificate distinguishes the inputs through the actual partner addresses and retains the two loops in A ’s agent-free result. The example and its interpretation are credited to [63]; the new ingredient is the exact arithmetic interface.

144 Memory logs: second routes to Part V

[write] *Manuscript 13's Sections 6–7 and manuscript 14's Section 5. [Merge note: Lemma 144.1 and Lemma 144.5 are Lemma 40.1 of Part V (manuscript 03), proved there first, in the same way: the coefficient sum of $\prod (X + x_i)$ is below the base, so the base- B digits are the coefficients, and evaluation at $-x_1$ followed by cancellation recovers the multiset. Manuscripts 13 and 15 were written without knowledge of this report; manuscript 14 read its README. The two statements are kept in their manuscripts' notation, with their proofs replaced by this pointer; manuscript 13's small-base counterexample and its remarks on Kronecker substitution and bit costs, and manuscript 14's remark on the large base, are kept. Theorem 144.4 and Theorem 144.6 are variants of Theorem 40.2 for different relations; the comparison closes this section.] The design shared by all three is one exact product identity between packed raw records and a proposed sorted list, strict key gaps, an address-change bit with a normalized gap, and local read–write checks in sorted order; Part X's memory compiler (Theorem 76.2) sorts with a network instead.*

144.1 An exact large-integer multiset certificate [13]

144.1.1 Why a single product can be exact

The memory proof will compare two lists of packed access records. A random fingerprint would be inappropriate for an exact Diophantine zero-set statement. A sufficiently large, *computed* base avoids collisions without any probability.

Lemma 144.1 (Collision-free product identity). *Let $L \geq 1$, $U \geq 2$, and let $0 \leq x_i, y_i < U$ for $1 \leq i \leq L$. Put*

$$B = U^L + 1. \quad (144.1)$$

Then

$$\prod_{i=1}^L (B + x_i) = \prod_{i=1}^L (B + y_i) \iff \text{multiset}(x_1, \dots, x_L) = \text{multiset}(y_1, \dots, y_L). \quad (144.2)$$

Pointer. [write] This is Lemma 40.1 with (U, B, x_i, y_i) for (C, β, c_i, d_i) , indexed from 1; manuscript 13's proof is that proof: $f(1) = \prod_i (1 + x_i) \leq U^L < B$, so the coefficients of $f(X) = \prod_i (X + x_i)$ and of g are base- B digits, $f(B) = g(B)$ forces $f = g$, and evaluation at $X = -x_1$ with cancellation of the common monic factor recovers the multiset. $\square$

The argument is the positive-coefficient form of large-base coefficient packing, often called Kronecker substitution; see, for example, [72]. The exact scale and the bounds on *both* lists are part of the proof, not optional implementation hints.

Example 144.2 (An arbitrary small base is unsound). For every positive integer b ,

$$(b + 0)(b + (b + 2)) = (b + 1)(b + b), \quad (144.3)$$

although the multisets $\{0, b + 2\}$ and $\{1, b\}$ differ. Consequently one cannot replace (144.1) by an unconstrained existential base and retain the equivalence. Nor can the prover be allowed to invent unbounded “sorted records” outside the packing range.

144.1.2 Quadratic implementation and honest bit costs

The exponent L in (144.1) is an external size parameter. Compute U^L by its binary addition chain: each squaring uses a variable z and equation $z - x^2 = 0$, and multiplication by U uses $z - Ux = 0$. This requires $O(\log(L + 1))$ gates. Define B by one further linear equation.

Compute each product in (144.2) by a balanced binary tree of multiplication gates. Leaves are the affine expressions $B + x_i$ or $B + y_i$. Each gate residual is quadratic and has a constant number of monomials. Each tree uses $L - 1$ multiplication gates; the final comparison is linear. Coefficients need only $O(\log U)$ bits: the huge integer B is a computed witness, not a huge printed coefficient.

Proposition 144.3 (Height of the product certificate). *The construction uses $O(L + \log(L + 1))$ auxiliary variables. Its largest product has $O(L^2 \log U)$ bits. With balanced product trees, the total bit length of the product-tree witnesses is*

$$O(L^2 \log U \log(L + 1)). \quad (144.4)$$

Proof. Since $B = U^L + 1$, each factor has $O(L \log U)$ bits. A product of s factors therefore has $O(sL \log U)$ bits. At one level of a balanced tree, the blocks partition at most L leaves, so their total bit length is $O(L^2 \log U)$. There are $O(\log(L + 1))$ levels. The binary powering witnesses have smaller total order and do not alter the stated bound. $\square$

A linear number of arithmetic coordinates is therefore obtained at a substantial height cost. Replacing a balanced tree by a long product chain would keep the coordinate count linear but give a weaker total-bit bound. No constant-time big-integer multiplication assumption is used here.

144.2 A canonical quadratic certificate for bounded memory [13]

144.2.1 The relation to be represented

Fix $A \geq 1$, $m \geq 0$ and $V \geq 2$. Memory addresses are $0, \dots, A - 1$ and values are $0, \dots, V - 1$. The proposed input consists of initial contents I_a , final contents O_a , and chronological access events

$$(a_j, r_j, w_j), \quad 1 \leq j \leq m. \quad (144.5)$$

An event must read the current value r_j at address a_j and then replace it by w_j . A read-only event has $w_j = r_j$. The *memory consistency relation* says that all these reads are correct and the final array is O .

This relation includes untouched addresses. Omitting initial or final records for such addresses would weaken the intended assertion.

144.2.2 Initial, ordinary and final records

Use four-field records (a, t, r, w) and form the chronological list

$$\underbrace{(a, 0, 0, I_a)}_{a=0, \dots, A-1}, \quad \underbrace{(a_j, j, r_j, w_j)}_{j=1, \dots, m}, \quad \underbrace{(a, m+1, O_a, O_a)}_{a=0, \dots, A-1}. \quad (144.6)$$

Its length is $L = 2A + m$. Put

$$H = m + 2, \quad U = AHV^2, \quad B = U^L + 1. \quad (144.7)$$

The initial record initializes an address; its artificial read value zero has no preceding record at that address. The final record audits the last value without changing it.

Pack a record injectively by

$$x(a, t, r, w) = HV^2a + V^2t + Vr + w. \quad (144.8)$$

The bounds

$$0 \leq a < A, \quad 0 \leq t < H, \quad 0 \leq r, w < V \quad (144.9)$$

imply $0 \leq x < U$. Uniqueness is ordinary mixed-radix uniqueness: successively reduce modulo V , divide by V , and then reduce modulo H .

Introduce L proposed sorted records $(\bar{a}_i, \bar{t}_i, \bar{r}_i, \bar{w}_i)$, all satisfying the same bounds. Require the product identity of Lemma 144.1 between the packed raw records and these packed proposed records. Thus the sorted rows must be a permutation of the entire raw list, including all four fields.

144.2.3 Sorting and adjacent consistency without a sorting network

Define the key $K_i = H\bar{a}_i + \bar{t}_i$. For each $1 \leq i < L$, introduce natural witnesses s_i, z_i, d_i and impose

$$K_i + 1 + s_i = K_{i+1}, \quad (144.10)$$

$$z_i(z_i - 1) = 0, \quad (144.11)$$

$$\bar{a}_{i+1} - \bar{a}_i = (1 - z_i)(d_i + 1), \quad (144.12)$$

$$z_i d_i = 0, \quad (144.13)$$

$$z_i(\bar{r}_{i+1} - \bar{w}_i) = 0. \quad (144.14)$$

Every equation is quadratic or linear. Equation (144.10) makes keys strictly increasing. If addresses agree, (144.12) forces $z_i = 1$ and (144.13) forces $d_i = 0$. If they differ, $z_i = 0$ and d_i is the address difference minus one. Thus (144.14) is exactly the consecutive-read consistency condition within an address block.

A bound $0 \leq x < R$ is encoded by the natural slack equation

$$x + u = R - 1, \quad u \in \mathbb{N}. \quad (144.15)$$

The raw event fields, initial and final values, and every sorted field receive these bounds. The timestamps in the raw list are literal constants. No bounded universal quantifier remains: the equations are instantiated once for each external index.

Theorem 144.4 (Exact canonical bounded-memory certificate). *For the fixed parameters A, m, V , the equations (144.6)–(144.15), together with the quadratic product-tree implementation, have a natural solution exactly when the proposed memory history is consistent. For fixed raw event data and initial/final arrays, the extension to sorted records, comparison bits, slacks and product-gate variables is unique whenever it exists.*

There are $O(A + m)$ auxiliary variables and equations, each of degree at most two and with a bounded number of monomials. All coefficients have $O(\log(A + m + V))$ bits. Replacing the equations by their sum of squares gives a polynomial of degree at most four with $O(A + m)$ sparse monomials.

Soundness. Lemma 144.1 and the field bounds show that the proposed sorted records are precisely a permutation of the raw records. Raw keys are all distinct: ordinary event times are globally distinct, initial records have time zero and distinct addresses, and final records have time $m + 1$ and distinct addresses. Thus (144.10) forces the unique increasing ordering of those keys.

Because $0 \leq t < H$, increasing $Ha + t$ sorts first by address and then by time. Every address block begins with its initial record and ends with its final record. Within the block, (144.12) forces $z_i = 1$, so (144.14) makes the next read equal the preceding write. Induction along the block shows that every ordinary read has the current memory value and that the final audit equals the final content. A block containing only its initial and final records forces $O_a = I_a$, as required. $\square$

Completeness and uniqueness. For a consistent history, sort the actual raw records by their keys. Assign $s_i = K_{i+1} - K_i - 1$, set z_i to the equal-address indicator, and set d_i to zero for equal addresses or to the address difference minus one otherwise. All these numbers are natural, and every equation holds. Use actual computed products for the arithmetic gates and the unique range slacks in (144.15).

Conversely, soundness fixed the sorted rows uniquely. Equations (144.10)–(144.13) then determine s_i, z_i, d_i uniquely. All range slacks and all gates are definitional and hence unique. The condition $z_i d_i = 0$ is important: without it, the inactive address-gap witness could vary arbitrarily when $z_i = 1$. $\square$

Size and degree. There are $L = 2A + m$ sorted records, a constant number of range equations per record, five equations per adjacent pair, $O(\log L)$ powering gates and two product trees with $L - 1$ gates each. All packed expressions have four fields. Thus the counts and the monomial bound are linear. The constants H, V, HV^2, U have logarithmic bit length in $A + m + V$. Squaring a residual with a bounded number of monomials introduces only a bounded number more; summing the squares preserves the linear sparse-description bound. $\square$

The theorem certifies a proposed sort rather than computing a sort inside the polynomial. Soundness is supplied by exact multiset equality and strict key order; there is no appeal to a sorting-network size estimate.

144.2.4 A fully exported example

The distributed example has $A = 3, m = 5, V = 8$, initial memory $(1, 2, 0)$, final memory $(3, 1, 0)$, and events

$$(0, 1, 3), (2, 0, 4), (0, 3, 3), (1, 2, 1), (2, 4, 0). \quad (144.16)$$

The five reads agree with the evolving memory. Here $L = 11, H = 7, U = 1344$, and $B = 1344^{11} + 1$. The file `data/13-exact-wiring-example_memory_certificate.json` contains every parameter, every witness value, every quadratic residual, and the expanded quartic sum of squares. Its exact counts are

Parameters	21
Natural witnesses	167
Quadratic/linear residuals	144
Monomials in the expanded quartic	2,496
Largest witness bit length	1,258
Total witness bit length	9,874

These are counts for the memory example only. They are neither interaction macro counts nor universal-polynomial records. Every designated residual and the quartic evaluate to zero. Increasing any one of the 167 witness coordinates by one, while keeping the others fixed, makes the quartic strictly positive. This mutation test checks local sensitivity, not the all-tuples uniqueness theorem, which was proved above.

[write] Batch 78, 2 October 2026. The review [65] shows, for the exported example above, that the Boolean equations (144.11) are redundant on the natural domain: the strict keys and the time bounds force adjacent addresses to be nondecreasing, so (144.12) with natural z_i, d_i forces $z_i \in \{0, 1\}$; keeping (144.13), the address equation can be replaced by the linear $\bar{a}_{i+1} - \bar{a}_i - d_i - 1 + z_i = 0$. One residual per adjacent pair is removed, $144 \rightarrow 134$ in the example, with the same natural zero set. The review notes that the deletion needs the key and time bounds, and that it does not transfer to manuscript 14's different encoding $\alpha_j - \alpha_{j-1} = b_j(1 + h_j)$, which does not force b_j to be Boolean. It is the review's result; the shipped certificate is unchanged.

144.3 Canonical bounded-memory certificates [14]

144.3.1 Source relation and the role of external caps

Fix positive integers L, A, V . A source log is

$$(a_i, w_i, v_i) \quad (0 \leq i < L),$$

where $0 \leq a_i < A$, $0 \leq v_i < V$, and $w_i \in \{0, 1\}$. A record with $w_i = 1$ writes v_i at address a_i ; a record with $w_i = 0$ reads that address and asserts that its current value is v_i . Memory starts at zero. The three coordinates per record are source coordinates, not included in the auxiliary count below.

Here L, A, V are *fixed parameters of the polynomial family*. In particular, products such as AL in coefficients do not increase its degree. Allowing those caps themselves to be free arithmetic inputs requires additional product gates and changes the exact ledger. This qualification matters in comparisons with unrestricted memory-log compilers.

144.3.2 One deterministic product test for multiset equality

Lemma 144.5 (Carry-free multiset certificate). *Let $0 \leq x_i, y_i < M$ for $0 \leq i < L$, and set $B = (M + 1)^L$. Then*

$$\prod_{i=0}^{L-1} (B + x_i) = \prod_{i=0}^{L-1} (B + y_i) \quad (144.17)$$

holds if and only if the two lists have the same multiset of entries.

Pointer. [write] Manuscript 14's proof is that of Lemma 40.1, with entries below M and the base $B = (M + 1)^L$ instead of $C^L + 1$: the coefficient sum is $F(1) = \prod_i (1 + x_i) \leq M^L < B$, every coefficient is a base- B digit with no carries, $F(B) = G(B)$ forces $F = G$, and evaluation at $-x_0$ with cancellation recovers the multiset. $\square$

The large base is doing genuine work. Equality of products at an arbitrary small base is not a reliable multiset test. No probability, finite field, random challenge, or unproved collision assumption occurs in (144.17).

144.3.3 Records, bounds, and sorted order

Introduce a putatively sorted list $(\alpha_j, \tau_j, \omega_j, \nu_j)$ for $0 \leq j < L$. It carries address, original time, write bit, and value. Define

$$\begin{aligned} x_i &= a_i + Ai + ALw_i + 2ALv_i, \\ y_j &= \alpha_j + A\tau_j + AL\omega_j + 2AL\nu_j, \\ M &= 2ALV, \quad B = (M + 1)^L. \end{aligned} \quad (144.18)$$

All source and sorted fields have their indicated bounds:

$$\begin{aligned} a_i, \alpha_j &< A, \quad v_i, \nu_j < V, \quad \tau_j < L, \\ w_i, \omega_j &\in \{0, 1\}. \end{aligned} \quad (144.19)$$

Each strict bound $z < D$ is imposed by one natural complement $\bar{z}$ and one equation $z + \bar{z} = D - 1$; each bit satisfies $b(b - 1) = 0$. The packing in (144.18) is injective on these fields. Its maximum is $M - 1$. The time field ensures that complete records are distinct, even when two accesses otherwise coincide.

Sort by the key

$$k_j = L\alpha_j + \tau_j.$$

For $1 \leq j < L$, introduce $d_j \in \mathbb{N}$ and require

$$k_j - k_{j-1} - 1 - d_j = 0. \quad (144.20)$$

Once the product test verifies the multiset, (144.20) selects its unique strictly increasing arrangement.

144.3.4 Quadratic product chains

Do not put an exponentiation symbol in the final Diophantine equation. Instead use $q_0 = 1$ as a constant and introduce $q_1, \dots, q_L$ with

$$q_i = (M + 1)q_{i-1} \quad (1 \leq i \leq L). \quad (144.21)$$

Use q_L as B . Likewise take $R_0 = S_0 = 1$ as constants, and introduce

$$R_{i+1} = R_i(B + x_i), \quad S_{i+1} = S_i(B + y_i) \quad (0 \leq i < L), \quad (144.22)$$

followed by $R_L = S_L$. Each residual in (144.21)–(144.22) is at most quadratic. All chain values are uniquely determined and nonnegative.

144.3.5 Local read/write consistency in sorted order

Let z_j be the value at the sorted address after record j , and let r_j be the value before it. Set $r_0 = 0$ as a constant. For $1 \leq j < L$, introduce $b_j, h_j \in \mathbb{N}$ with

$$\begin{aligned} b_j(b_j - 1) &= 0, \\ \alpha_j - \alpha_{j-1} &= b_j(1 + h_j), \\ (1 - b_j)h_j &= 0, \\ r_j &= (1 - b_j)z_{j-1}. \end{aligned} \quad (144.23)$$

The bit b_j indicates a change of address. It and h_j are unique: if the address stays the same they are both zero; otherwise $b_j = 1$ and $h_j = \alpha_j - \alpha_{j-1} - 1$. For every j , require

$$\begin{aligned} (1 - \omega_j)(v_j - r_j) &= 0, \\ z_j - \omega_j v_j - (1 - \omega_j)r_j &= 0. \end{aligned} \quad (144.24)$$

A read must equal the prior value and preserves it. A write supplies the new value. The key order guarantees that records of the same address are processed in their original chronological order.

Theorem 144.6 (A $17L - 4$ canonical memory system). *For fixed $L, A, V \geq 1$, equations (144.19)–(144.24), with the product equality, form a quadratic system having exactly $17L - 4$ auxiliary variables and $17L - 4$ residuals. A source log has a natural solution if and only if it lies within the caps and is a valid zero-initialized memory log. When it has a solution, the entire auxiliary tuple is unique. The sum of the residual squares is an integer polynomial of degree at most four.*

Proof. The range equations and bit equations enforce well-typed records. By Lemma 144.5, the product chains and final equality force equality of the packed-record multisets. Injectivity of the packing recovers the full records. Their time fields distinguish all records, so the strict key order forces the unique sorted list.

Because $0 \leq \tau_j < L$, increasing keys imply nondecreasing addresses. Equations (144.23) therefore have their asserted unique natural values. Within each address block, (144.24) inductively enforces precisely its chronological read/write history starting from zero. Since operations at different addresses do not affect one another, this is equivalent to validity of the original log. This proves soundness.

Conversely sort a valid log, set all complements and key gaps to their unique differences, evaluate the product chains, and replay the sorted address blocks. This produces a natural solution. The preceding soundness argument forces the same sorted rows in every solution, and each remaining variable is then forced successively. The exact count is in Table 24; the degree statement follows from the displayed quadratic equations. $\square$

Table 24: Complete memory ledger. Source coordinates: $3L$, not included in the auxiliary column.

Component	Auxiliaries	Residuals
Sorted four-field records	$4L$	0
Source range complements and bits	$2L$	$3L$
Sorted range complements and bits	$3L$	$4L$
Base chain	L	L
Two product chains and equality	$2L$	$2L + 1$
Strict sorted-key gaps	$L - 1$	$L - 1$
New-address bits and normalized gaps	$2L - 2$	$3L - 3$
Pre-values except r_0	$L - 1$	$L - 1$
Read checks	0	L
Post-values	L	L
Total	$17L - 4$	$17L - 4$

Corollary 144.7 (Precomputed-base variant). *If the coefficient $B = (M + 1)^L$ is precomputed by the compiler, the base-chain variables and residuals may be omitted. The count becomes $16L - 4$ in both columns, at the price of a coefficient with $O(L \log(M + 1))$ bits.*

The main theorem uses (144.21) to keep coefficient bit lengths small and expose all arithmetic. Neither variant is claimed optimal.

144.3.6 Height, bit complexity, and comparison with earlier memory work

Proposition 144.8 (Height bound). *Every solution of the system in Theorem 144.6 satisfies*

$$R_L, S_L \leq (B + M - 1)^L,$$

and every auxiliary has bit length

$$O\left(L^2 \log(M + 1)\right). \quad (144.25)$$

The residual coefficients have $O(\log(M + 1))$ bits. The expanded sum of squares has $O(L)$ monomials before possible cancellations and merging.

Proof. Each factor of either product is at most $B + M - 1$. Since $B = (M + 1)^L \geq M$, $\log_2(B + M - 1) \leq 1 + L \log_2(M + 1)$. All product prefixes satisfy the same bound. Other coordinates are capped fields, gaps, post-values below V , or base powers no larger than B . Every residual has a bounded number of terms, with coefficients built from A, L, V and their fixed products. Squaring each residual therefore contributes only a bounded number of monomials. $\square$

This is linear *arithmetic description size*, not linear total witness bit volume. A crude total-volume bound is $O(L^3 \log(M + 1))$. The large products are an explicit size–height trade-off, not a hidden unit-cost claim about binary arithmetic. Sorting-network arithmetization gives an alternative with $O(L \log^2 L)$ fixed comparators and small intermediate words; the network ingredient is classical [28].

The repository’s existing memory and heap compilers have different interfaces and published ledgers [66]. A smaller auxiliary count here must not be advertised as an unconditional improvement of every prior bound: our A, V are external caps, and we use large carry-free product witnesses. What is established is the exact count for this explicitly stated relation.

144.4 Three ledgers for one memory design

[write] The three linear-size memory certificates of this report certify different relations, and none of their counts is an improvement on another. The table holds what each relation is.

	Part V, Theorem 40.2 (03)	Theorem 144.6 (14)	Theorem 144.4 (13)
Relation	zero-initialized log of L reads and writes	zero-initialized log of L reads and writes	initial array, m read-and-write events, final array; $L = 2A + m$ records
Caps	none: the radix D is computed from the log	external address and value caps A, V , fixed with L	fixed A, m, V ; bounds on every field
Base	$\beta = C^L + 1$, $C = D^4$, by a chain of length L	$B = (M + 1)^L$, $M = 2ALV$, by a chain of length L (or precomputed, Corollary 144.7)	$B = U^L + 1$, $U = AHV^2$, by binary powering, $O(\log L)$ gates
Witnesses	$24L$	$17L - 4$ ($16L - 4$ precomputed)	$O(A + m)$, not counted exactly; 167 in the exported example
Residuals	$22L + 1$	$17L - 4$ ($16L - 4$ precomputed)	$O(A + m)$; 144 in the example
Products	two chains	two chains	two balanced trees

Manuscript 14 says itself that its smaller auxiliary count “must not be advertised as an unconditional improvement of every prior bound” (Section 144.3.6); here the prior bound is Part V’s, for a relation without caps. Manuscript 13’s binary powering of U^L is the suggestion of Part V’s question on the scalar constant (Research question 71.24), and manuscript 14’s $17L - 4$ answers that question in part for its capped relation; see the dated note there. Both keep Part V’s height cost: a largest witness of $O(L^2 \log U)$ or $O(L^2 \log(M + 1))$ bits (Propositions 144.3 and 144.8; compare Theorem 40.3), so Part V’s size–height question (Research question 71.23) is not answered; manuscript 14 names sorting networks as the alternative, as Part V does.

145 The bounded six-rule compiler

[write] *Manuscripts 13 and 14 prove the same theorem by the same method: exact length- T scheduled reductions of all six rules from an explicit net with C_0 cells and F (or f) free ports have a quartic certificate with $O(C_0 + F + T)$ coordinates, residuals and monomials and exactly one auxiliary witness per legal schedule, built from a constant-size per-step controller and one large-integer sorted-memory certificate for the whole access log. The theorem is stated once, as manuscript 14’s Theorem 145.3, which has the explicit fixed 36-access step, the global ledger (145.9), terminal predicates and a bounded native loader. Manuscript 13’s controller lemma, splice schedule, allocation, paid input validation, terminal predicates, statement and proof follow as a second presentation (Lemma 145.4, Remark 145.5); the statement is printed as a remark, because it is the same theorem, and the note after it lists where it differs. Neither manuscript ships an exporter for the full compiler; both say so (Section 148). The transfer to other fixed interaction systems is likewise stated once (Theorem 145.7), with manuscript 13’s corollary after it.*

145.1 A loop-aware memory machine for the six rules [14]

145.1.1 Fixed storage layout

Fix a valid explicit input net with C_0 cells, f free ports, and ℓ_0 bare loops, and fix an external horizon T . Set

$$C = C_0 + 4T, \quad A = 4C + f + 1, \quad V = 3C + f + 4. \quad (145.1)$$

The completely empty zero-horizon case is handled separately without a memory log. When $C = 0$ and $f > 0$ the same layout works for free ports alone. For cell $c < C$, use port identifiers

$$p(c, j) = 3c + j + 1.$$

Free port i , for $0 \leq i < f$, has identifier $3C + i + 1$. Address zero is a read-only dummy initialized to zero. The four cell slots are

$$\begin{aligned} 4c + 1 & : \text{type code } 0, 1, 2, 3 \text{ for dead, } \varepsilon, \gamma, \delta, \\ 4c + 2 + j & : \text{mate identifier of port } cj \quad (j = 0, 1, 2). \end{aligned} \quad (145.2)$$

The mate slot of free port i is $4C + i + 1$. For an eraser the unused auxiliary slots are zero. Every slot of a deleted cell is reset to zero. All actual values lie below V and all addresses below A .

Besides memory we carry scalar registers h_t, n_t, ℓ_t : next fresh cell identifier, live-cell count, and bare-loop count. They start at C_0, C_0, ℓ_0 . These are circuit registers, not additional memory accesses.

145.1.2 A fixed 36-access rewrite schedule

Initialize the $4C_0 + f$ occupied cell/free-port slots by writes in their fixed layout order; untouched memory is zero. One rewrite uses the following fixed 36 log positions.

Table 25: Literal per-step memory budget. Unused positions are dummy reads $(0, 0, 0)$.

Stage	Accesses	Purpose
Read	8	Two type codes and all six mate slots of the selected cells.
Delete	8	Reset both selected four-slot records to zero.
Allocate	16	Write up to four complete fresh records; inactive record positions are dummy reads.
Patch	4	Update the mates of the external old auxiliary endpoints, at most one per boundary position.
Total	36	No global graph scan and no hidden path-following accesses.

The fixed read stage includes the zero auxiliary slots of any selected eraser. All old mates are read before any deletion or patch. For every dummy position the address, read flag, and value are fixed to zero; there is no unconstrained inactive log data.

Check $u_t < v_t < h_t$, nonzero type codes, and the principal pairing using the read values. Derive the unique rule from the two type codes. For binary annihilation, compute the six comparisons

$$m_{ij} = \mathbf{1}\{\text{old mate of } B_i \text{ is } B_j\}$$

and apply Section 143.3. Each $g_{ij} = 1$ determines the two reciprocal external patches. For the other rules use the relocation operation below. Compute complete new records locally before the allocation stage.

The scalar updates are

$$\begin{aligned} h_{t+1} &= h_t + b_t, \\ n_{t+1} + 2 &= n_t + b_t, \\ \ell_{t+1} &= \ell_t + c_t. \end{aligned} \tag{145.3}$$

where $b_t \in \{0, 2, 4\}$ is the number of new cells. The only nonzero c_t comes from a binary annihilation and is the forced kernel output.

145.1.3 Endpoint relocation for creation rules

For a mixed interaction or a binary erasure, the replacement sends each old auxiliary position i to a distinct new principal port $\rho(i)$. If its old mate is another old boundary position j , set the new mate to $\rho(j)$. If its old mate is external endpoint q_i , join $\rho(i)$ to q_i and patch q_i reciprocally. Add the fixed internal auxiliary wiring (140.3) for the mixed rule. This produces no bare loops: every component touching the old boundary now reaches at least one new incident port. An internally matched pair of old boundary positions becomes a wire between two distinct new principals, not a disappearing wire.

Lemma 145.1 (One-step representation invariant). *Starting from the valid storage representation of a net, the preceding 36-access procedure succeeds for a supplied pair exactly when that pair is active. Its output is the storage representation of the literal interaction rule applied to the net, including the exact loop count. All unmentioned memory cells are preserved.*

Proof. The initial checks enforce distinct live cells and their principal wire, so they characterize an active pair. The old matching invariant implies that the external boundary mates are distinct, are not removed principal ports, and do not belong to any newly allocated record. Freshness follows from h .

For annihilation, Theorem 143.1 gives exactly the surviving endpoint matching and new loops. The reciprocal patches restore the involution at every affected surviving endpoint. For a creation rule, endpoint relocation preserves every old boundary incidence with its endpoint renamed to the specified new principal. Equation (140.3) matches the remaining new ports exactly once. Unused eraser auxiliaries are zero. The old records are cleared; new records use fresh addresses; unrelated records are neither read for path-search nor modified. These facts prove the storage invariant and the claimed literal result. Equation (145.3) gives the correct scalar registers. $\square$

Proposition 145.2 (Resource bounds). *After t successful steps,*

$$h_t \leq C_0 + 4t, \quad n_t \leq C_0 + 2t, \quad \ell_t \leq \ell_0 + 2t. \tag{145.4}$$

For a length- T trace with no final scan, the fixed log length is

$$L = 4C_0 + f + 36T. \tag{145.5}$$

Proof. No rule creates more than four cells or increases the live count by more than two. Equation (143.3) gives at most two new loops; the other rules give none. Induction proves (145.4). Initialization and the fixed access schedule prove (145.5). The loop increment bound is attained by a like-binary active pair whose auxiliary matching equals its annihilation matching. $\square$

The exact 36 is a deliberately uncomplicated memory-access upper bound. It is not the number of arithmetic operations, residuals, or quantified coordinates per interaction.

[write] The bounds (145.4) are Corollary 140.2 (Section 140); (145.5) is manuscript 14's own.

145.2 The loop-sensitive quartic compilation theorem [14]

145.2.1 Functional arithmetic circuits

A *functional quadratic circuit* is a system of degree-at-most-two integer equations whose auxiliary tuple is unique for every permitted source tuple. Straight-line addition and multiplication, Boolean gates, normalized comparisons, fixed-divisor quotient/remainder, and selection have such presentations. Appendix 151 supplies literal gadgets.

A finite branch is compiled by total deterministic computations on its branch inputs and by selecting its outputs. Inactive branches receive fixed dummy inputs when necessary. Their auxiliaries are then uniquely determined; merely multiplying an arbitrary equation by an inactive flag is insufficient. The per-step controller of Section 145.1 has only a fixed number of boundary positions, type cases, and template slots, so it is such a circuit of constant size, with constants independent of C_0, f, T .

Theorem 145.3 (Exact quartic family for interaction combinators). *Fix a valid explicitly labelled input net N , an external horizon T , and one of the terminal conditions described in Section 145.5. There is an effective construction of a polynomial*

$$P_{N,T}(\mathbf{s}, \mathbf{z}) \in \mathbb{Z}[\mathbf{s}, \mathbf{z}] \quad \text{of degree at most 4} \quad (145.6)$$

with these properties.

1. *For a supplied ordered-pair schedule $\mathbf{s} = (u_0, v_0, \dots, u_{T-1}, v_{T-1})$, there is a natural zero in $\mathbf{z}$ if and only if it is a legal length- T execution satisfying the terminal condition.*
2. *Such an auxiliary zero is unique. Thus, when schedules are also quantified, natural zeros are in bijection with the accepted scheduled executions in the fresh-name convention.*
3. *The total number of coordinates, number of quadratic residuals, and number of monomials in the sum-of-squares presentation are $O(C_0 + f + T)$ for the fixed six-rule system and fixed terminal-query type.*
4. *Every witness coordinate has bit length*

$$O\left((C_0 + f + T + 1)^2 \log(C_0 + f + T + 2) + \log(\ell_0 + 2)\right). \quad (145.7)$$

The constants do not depend on the particular input matching.

Proof. Unroll the fixed controller T times and link its access triples into one log. Initialize the fixed input layout by its prescribed writes. Add the memory system of Theorem 144.6 with caps (145.1), and add the scalar and terminal equations. All circuit nodes are separately named, so no high-degree substitution is made. Let $R_1, \dots, R_m$ be the resulting quadratic residuals and set

$$P_{N,T} = \sum_{j=1}^m R_j^2. \quad (145.8)$$

This is an effective finite prescription, not an appeal to an unspecified universal polynomial.

For soundness, a natural zero satisfies every residual. The memory theorem makes every log read agree with the latest prior write to its address. The input writes therefore establish the valid starting representation. Apply Lemma 145.1 successively. At each step the supplied pair is an actual active pair, the controller performs the exact rule, and the stored matching and loop count remain valid. The final checks give the required terminal condition.

For completeness, execute any legal supplied schedule, write down its fixed log and scalar registers, evaluate every functional circuit, and extend the log by its canonical memory witness. Every residual vanishes.

For uniqueness, fix the input and schedule. Chronological induction forces every read value, since all earlier writes and the initial memory are already forced. The local deterministic controller forces the writes, scalars, and its own auxiliary values. Hence the complete source log is forced. The memory theorem then forces all its auxiliary coordinates. Fixed dummy records and normalized branch gadgets remove any inactive choices.

The log length is (145.5), plus a linear final scan when needed. The local controller uses a constant number of circuit nodes per step; initialization and terminal scanning use linearly many. The memory theorem is linear in L , and each residual has a bounded number of terms. These facts give the size claims. The largest arithmetic witnesses occur in the product chains; apply Proposition 144.8 with the polynomial caps (145.1). Scalar loop values add only $O(\log(\ell_0 + T + 2))$ bits. Every other controller register is a bounded pointer, a comparison gap, a small selector, or a bounded number of arithmetic combinations of these. This proves (145.7). $\square$

145.2.2 A transparent, nonoptimized global ledger

Let G denote the number of private controller/scalar/gadget coordinates, and H the number of controller, initialization-link, and terminal residuals in a chosen full assembler. Counting all $3L$ log fields as coordinates, including those forced to constants, gives

$$\begin{aligned} \text{total coordinates, including schedules} &\leq 20L - 4 + 2T + G, \\ \text{quadratic residuals} &= 17L - 4 + H. \end{aligned} \tag{145.9}$$

Here $G, H = O(C_0 + f + T)$. The paper proves this and supplies the full finite recipe, but does not report a measured numerical G, H for an emitted unknown-schedule interaction-net polynomial. Consequently (145.9) must not be compared as a numerical universal-operation record with the repository's separately audited fixed-universal polynomials.

145.2.3 What the bijection does not quotient

Choosing a different available active pair generally gives a different zero, even if the computations commute or later reach isomorphic nets. The bijection is with scheduled executions, not with normal forms, interleaving classes, or observational equivalence classes. It removes artificial auxiliary multiplicity and fresh-name choices; it does not remove semantic scheduler choices. This limitation is especially important in a strongly confluent substrate: confluence alone does not canonically identify the arithmetic witnesses of different schedules.

[write] *Second presentation (source 13). Manuscript 13 compiles the same controller through its bounded-fanin controller lemma (Lemma 145.4), with its own three-way comparison gadget (145.10) (manuscript 14's is in Section 151), and a splice performed as a fixed access schedule. Its allocation reserves $C_0 + 4T$ cell slots and four transient boundary ports per step, as manuscript 14's layout (145.1) reserves $C_0 + 4T$ cell records. It adds a paid validation scan of a public dense input, coefficients of $O(\log S)$ bits by binary powering, a total witness-length bound, and the treatment of "at most T " steps.*

145.3 Quadratic compilation of the finite controller [13]

145.3.1 Definitional gates and canonical branches

The memory theorem certifies that reads and writes agree globally. It must be combined with local equations saying that the proposed accesses really come from the chosen interaction rule. This section makes explicit why this can be done without increasing the degree or hiding a large circuit.

For natural values, addition and multiplication are represented by $z - x - y = 0$ and $z - xy = 0$. Fixed affine address calculations are linear. A finite list of alternatives uses natural selectors $b_1, \dots, b_s$ with $\sum b_i = 1$; each is automatically zero or one. If desired, individual Boolean equations can also be included. For a fixed alphabet and fixed macro, s is an absolute constant.

A useful total comparison gadget for $x, y \in \mathbb{N}$ has natural variables l, e, g, d_l, d_g and equations

$$\begin{aligned} l + e + g &= 1, & e(x - y) &= 0, & (145.10) \\ l(y - x - 1 - d_l) &= 0, & g(x - y - 1 - d_g) &= 0, \\ (1 - l)d_l &= 0, & (1 - g)d_g &= 0. \end{aligned}$$

Exactly one of $x < y$, $x = y$, $x > y$ is possible, forcing the corresponding selector and its unique gap. Inactive gaps are zero. Thus this is a quadratic, uniquely extending comparison certificate. It can test pointer equality, validity ranges, or the ordering $u < v$ of a scheduled pair.

A conditional linear equation $E = 0$ is imposed as $bE = 0$. One must not blindly multiply a quadratic residual by b , which would raise its degree. Instead introduce a definitional product gate for each nonlinear term, then guard the resulting *linear* equality. For example, to impose $xy = z$ only when $b = 1$, first define $p = xy$ unconditionally and then impose $b(p - z) = 0$. All residuals remain quadratic. Dead branch inputs, unused slacks, and unused outputs are set to fixed canonical values; unconditional gate outputs remain their uniquely computed values.

Lemma 145.4 (Bounded-fanin controller compilation). *A fixed finite macro using a bounded number of memory accesses, affine address operations, additions, multiplications and comparisons can be translated into a bounded number of quadratic equations with boundedly many monomials. For a deterministic macro and fixed input registers and read values, its ancillary extension can be made unique. Repeating the macro t times produces $O(t)$ equations and variables.*

Proof. Translate arithmetic instructions to definitional gates and comparisons to (145.10). Translate the finite control alternatives with selectors; their conditions force the selected branch uniquely. Use the flatten-before-guard construction for nonlinear branch equations and canonical values for unused local fields. Each instruction introduces boundedly many variables and equations because both the instruction set and each expression's fanin are fixed. Induction through the instruction sequence proves both semantic correctness and unique extension. Repeating this finite construction adds its constant costs once per macro. $\square$

This lemma does not grant arbitrary unbounded arithmetic predicates for free. Only the displayed finite operations and fixed control are allowed. All long sums or products must themselves be represented by gates.

145.3.2 A splice as a fixed access schedule

Represent an inactive port by a zero mate value, and reserve one dummy memory address with constant value zero. For a legal splice, read $p = m(a)$ and $q = m(b)$ by read-only events. A comparison determines whether $p = b$. Always retire a, b by writing zero. In the nonclosing branch, write q at p

and p at q ; in the closing branch, replace these two writes by zero-preserving accesses to the dummy address. Increment the separate loop counter by the closing bit.

This has a fixed maximum number of access slots. Crucially the two later writes are conditional: writing to $p = b$ after retiring b would revive a retired port and be wrong. Canonical dummy accesses avoid this error while keeping the number and timestamps of memory events fixed.

The valid-matching invariant guarantees the distinctness conditions in Lemma 141.1. They may also be checked locally, at constant cost, but no scan of the full graph is needed after each interaction. A right-hand-side initialization writes its literal mate pairs into fresh slots; erasure and type updates are likewise finite lists of writes. Thus Proposition 140.1 meets every hypothesis of Lemma 145.4.

145.4 The linear-description quartic compiler [13]

145.4.1 Explicit allocation

Let the input have cell identifiers $0, \dots, C_0 - 1$ and F free ports. Fix a horizon T . Reserve

$$C_* = C_0 + 4T \quad (145.11)$$

cell slots, with three possible port slots per cell. At step t , the four possible new cell identifiers are

$$C_0 + 4t, C_0 + 4t + 1, C_0 + 4t + 2, C_0 + 4t + 3. \quad (145.12)$$

Unused slots stay zero; erased cell identifiers are never reused. Reserve four transient boundary ports per step, also never reused. The total port capacity is

$$P_* = 3C_* + F + 4T = 3C_0 + 16T + F. \quad (145.13)$$

Use positive live-port addresses, with zero reserved for inactivity. A type array records inactive, ϵ , δ or γ . The mate array records zero or a live partner. Static arrays can mark principal-port addresses and distinguish cell slots, free ports and transient ports. All these arrays together have

$$A = O(1 + C_0 + F + T) \quad (145.14)$$

addresses, with value bound $V = O(1 + C_0 + F + T)$. The isolated-loop counter is kept separately, so an arbitrarily large supplied ℓ_0 need not inflate the pointer radix.

The scheduled pair at a real interaction is ordered $u_t < v_t$. Checks read the types and principal mates, verify that both cells are live and paired through their principal ports, and then choose the appropriate one of the six literal rules. Fresh identifiers are assigned by the displayed formula, not existentially selected from an unbounded supply.

145.4.2 Input validation is paid for

The public input is a dense typed-port representation, not a single encoded natural number. Its types must be in the stated alphabet. Its live-port domain must match those types and the designated free interface; every inactive slot has zero mate value. For each live port p , read $q = m(p)$, check that q is a live, distinct port in range, and read $m(q)$ to verify $m(q) = p$. Free-port and transient-slot conventions are checked as well.

This takes a constant number of local checks and accesses per allocated slot, hence $O(1 + C_0 + F + T)$ in total. Input cells occupy the first C_0 slots; the remaining type slots and all future boundary slots are zero. One may instead state a theorem on promised-valid inputs, but the bound here includes validation, and malformed inputs have no certificate.

Once this invariant is established, Theorem 141.2 proves its preservation. A repeated graph-wide validity tableau is unnecessary.

145.4.3 Exact terminal predicates

The following predicates have distinct meanings and distinct encodings.

Agent-free. Maintain a live-cell counter initialized to C_0 and changed by the known number created minus two at each real interaction. Require its final value to be zero. Free-interface wires and isolated cycles may still be present.

Normal. All unordered pairs of agent types have a rule. Thus normality means there is no principal–principal wire. Scan allocated cell slots once. For each live cell, read its principal mate and a static principal-address flag; require that flag to be zero. Validity ensures that a flagged mate really is the principal port of a live cell. Inactive slots use dummy accesses. The scan costs $O(C_*)$ accesses, not $O(C_*T)$.

Exact loop target. Require

$$\ell_T = \ell_0 + \sum_{\text{splice slots } j} b_j = \ell_{\text{target}}, \quad (145.15)$$

where b_j is the uniquely determined closing bit, and encode the sum by prefix additions. It is often more economical to store just the accumulated new-loop count, bounded by $2T$, and make the single final comparison with ℓ_0 .

Exact labelled target. Compare final type and mate arrays to the supplied target arrays, one slot at a time, together with the loop count. The target uses the same canonical allocation convention. No uncharged isomorphism search is included.

These predicates may be combined, for example “agent-free with exactly two cyclic wires”. In the closed domain, “agent-free with zero cycles” is the completely empty net, whereas “normal” is substantially weaker.

145.4.4 The compiler theorem

Remark 145.5 (Cycle-preserving bounded Diophantine compiler; manuscript 13’s statement of Theorem 145.3). Fix $C_0, F, T \in \mathbb{N}$ and one of the terminal predicates above, or a fixed conjunction of them. Put $S = 1 + C_0 + F + T$. There is an effectively constructed integer polynomial

$$P_{C_0, F, T}(\mathbf{x}, \mathbf{z}) \quad (145.16)$$

with the following properties.

For every public dense input $\mathbf{x}$, including any supplied target parameters, $P_{C_0, F, T}(\mathbf{x}, \mathbf{z}) = 0$ has a solution $\mathbf{z} \in \mathbb{N}^k$ exactly when the input is valid and has a sequence of exactly T legal interaction-combinator reductions satisfying the terminal predicate. The polynomial has degree at most four, $k = O(S)$, $O(S)$ sparse monomials and coefficients of $O(\log S)$ bits. It can be presented as the sum of $O(S)$ squares of bounded-fanin quadratic residuals.

For each fixed legal ordered schedule $(u_t, v_t)_{t < T}$, the remaining witnesses can be chosen canonically so that exactly one complete certificate lies over that schedule. All isolated cyclic-wire counts are exact.

The largest noninput arithmetic witness can be bounded by $O(S^2 \log S)$ bits and the total such witness length by $O(S^2 \log^2(S + 1))$. Supplied unbounded loop counts remain parameters and are compared by a final linear equation, rather than copied into every memory record.

[write] Manuscript 13's statement differs from Theorem 145.3 in three respects, none of which makes either statement stronger in every respect. (i) Its input net is a tuple of public parameters of fixed dimensions C_0, F , validated inside the polynomial at linear cost; manuscript 14 fixes the input net in the compiler, $P_{N,T}$, and gives an ordinary-integer loader for fixed dimensions as a variant (Section 145.5). (ii) Its schedule is part of the witness $\mathbf{z}$ and uniqueness holds above each legal schedule; manuscript 14 makes the schedule a source tuple $\mathbf{s}$ with a unique zero above each legal one, and quantifies it to obtain the same bijection. (iii) It bounds coefficient lengths by $O(\log S)$ bits and the total witness length by $O(S^2 \log^2(S+1))$; manuscript 14 bounds every coordinate by (145.7). Both are paper constructions.

Second presentation (source 13): construction and completeness. Allocate memory as in (145.11)–(145.13), including a dummy zero address and static metadata. Perform the input-validation scan, T copies of the interaction macro, and any final scan required by the terminal predicate. Each interaction macro has a fixed maximum number of access slots, using canonical dummy accesses in inactive branches. Thus the complete chronological memory log has $m = O(S)$ events, with $A = O(S)$ and $V = O(S)$.

Translate each local instruction and branch by Lemma 145.4, and connect consecutive local registers by linear equalities. Use Theorem 144.4 once for the *whole* chronological access log. Its event fields, which were public parameters in the standalone memory theorem, are now witnesses constrained by the controller. Final contents used in the memory audit are witnesses unless the chosen target supplies them; either way the actual deterministic history fixes them. Add the terminal equations and sum all residual squares.

Given a valid reducing history with the chosen schedule, execute the exact pointer macros. Proposition 140.1 supplies precisely its true successive nets and loop increments. Fill in the true access records, local registers, branch selectors and final arrays. The memory theorem supplies the unique sorted record list and gate values. All residuals vanish, so the polynomial vanishes. $\square$

Second presentation (source 13): soundness and canonicity. Suppose the polynomial vanishes on natural witnesses. Every residual is zero. The memory theorem implies that every certified read obtains the latest certified write to that address, starting with the designated input array. Input-validation residuals therefore establish a genuine initial net. Local controller residuals select a live principal pair and implement the appropriate literal rule. By induction over interaction macros and Theorem 141.2, the certified arrays are the actual arrays after exactly the chosen reductions, including every closed wire. The final residuals impose the stated predicate on that actual final net.

For fixed input and schedule, execution is deterministic. Fresh allocation, branch selectors, dummy records, comparison gaps and local gates use the canonical conventions of Section 145.3. Thus the chronological log, final arrays and loop increments are forced. The unique-extension part of Theorem 144.4 forces all remaining witnesses. This proves uniqueness over a fixed schedule, not uniqueness of the schedule itself. $\square$

Second presentation (source 13): size, coefficient length and height. There are $O(S)$ accesses and local instructions. Each produces only a bounded number of variables and bounded-fanin quadratic residuals. Input and final scans also cost $O(S)$. The global memory certificate has $L = 2A + m = O(S)$ and $U = AHV^2 = S^{O(1)}$. Its powering gates avoid printing U^L as a coefficient. The resulting constants are bounded by fixed powers of S , so their bit lengths are $O(\log S)$.

Every quadratic residual has a constant number of monomials; its square therefore has a constant number as well. Summing $O(S)$ such squares gives the asserted sparse polynomial, including the bound on its joint degree in public inputs and witnesses. Prefix gates are used for all long sums. Finally Proposition 144.3, with $L = O(S)$ and $\log U = O(\log S)$, gives the witness-height bounds. Pointer values, local registers and range slacks are smaller. Store new-loop counts rather than repeated copies of a potentially huge ℓ_0 , as specified above. $\square$

Remark 145.6 (Exactly versus at most T). The theorem counts exactly T real interactions. To represent “at most T ”, add a forced monotone stopped flag. Before stopping, slots execute real interactions; after stopping, every slot is a canonical no-op. Require the terminal predicate at the stopping boundary. This is still a linear bounded family. Its canonicity statement is relative to the selected stopping time as well as the real schedule, unless stopping is required at the first occurrence of a locally decidable terminal predicate.

145.5 Terminal predicates and bounded native input [14]

145.5.1 Queries with no final graph scan

The terminal conditions $n_T = 0$ and $\ell_T = k$ express agent-free execution with exactly k bare loops. Either equality may be used alone. For a closed net, the conjunction $n_T = 0, \ell_T = 0$ expresses the completely empty result. For an open net, $n_T = 0$ still permits a matching of free ports; that matching is present in memory and has not been erased.

For $T > 0$, an execution ending agent-free cannot have become agent-free at an earlier step, because an agent-free net has no active pair. Thus exact length- T agent-free traces already have the appropriate first-hit property. No extra halting-padding witnesses are needed for this query.

145.5.2 Normality and exact labelled targets

To check normality, scan the reserved cell records at the final memory state. At every live principal port, determine whether its mate is the principal port of another live cell. Reject precisely that case. Unused slots and free-port identifiers are distinguished by their fixed ranges. Range comparisons and quotient/remainder modulo three are functional constant-size gadgets. A scan of $O(C_0 + T)$ records is therefore sufficient.

An exact labelled target is checked by scanning all reserved cell and free-port mate slots and comparing them with the specified target layout, and by checking the scalar loop count. This costs $O(C_0 + f + T)$ accesses. Equality is literal in the declared naming convention. An unlabelled target requires a different interface; guessing a graph isomorphism can introduce multiple witnesses, so it is not silently substituted for this comparison.

An at-most- T family can also use an explicitly selected stopping time and fixed dummy steps thereafter. The selected time is part of the represented finite execution. To avoid duplicate encodings, its activity mask and every padded slot must be determined by that time, not chosen independently. The exact-horizon theorem does not need this extension.

145.5.3 An ordinary integer can load a bounded explicit net

The main theorem takes a valid explicit net as fixed input to the compiler. This is not a paid loader for an arbitrary integer into one fixed universal net. There is, however, a straightforward bounded native-input variant.

Fix C_0, f and use an initial port layout with C_0 reserved cells. Let $K = 4C_0 + f$ and $D = 3C_0 + f + 4$. Pack the type and mate entries as digits $d_0, \dots, d_{K-1} < D$ of an ordinary input integer x . With $x_0 = x$, use

$$x_i = Dx_{i+1} + d_i, \quad d_i + \bar{d}_i = D - 1, \quad x_K = 0. \quad (145.17)$$

Every digit, suffix, and complement is unique. This costs $O(K)$ variables and equations; the bit cost of the suffixes is bounded by the supplied input length. If $K = 0$, the only empty-table code is $x = 0$.

Validate the type codes, zero unused eraser slots, and the involutive, fixed-point-free pairing on exactly the live and free ports. The reciprocal pairing check can be performed by reads of the initialized mate array: for each actual port p , read its supplied mate q , verify that q is an actual distinct port, and read the mate at q to check that it equals p . These tests cost linearly many accesses. An invalid encoding is rejected, not repaired. Remap initial free-port identifiers from $3C_0 + i + 1$ to $3(C_0 + 4T) + i + 1$ by a range-controlled affine operation before using the horizon-dependent layout.

This gives a uniform arithmetic interface for fixed input dimensions and external horizon, with unique decoding. It does not make the dimensions, horizon, universal interpreter, or arbitrary source-language translator free of arithmetic cost.

[write] The transfer to other fixed interaction systems is proved by both manuscripts: manuscript 14's Theorem 145.7, printed first, with the count (145.18) of partial matchings on r positions, and manuscript 13's Remark 145.8, printed after it as a remark with its proof. For port graphs with reciprocal links and bare loops it is the adapter that Part X's bounded-local graph transfer (Theorem 81.1) leaves to be proved (the subsection on interaction systems in Section 81); it is proved here directly and is not claimed as new beside Theorem 81.1.

145.6 Extension to other fixed interaction systems [14]

Theorem 145.7 (Fixed-rule finite-interface transfer). *Consider a fixed finite interaction-net signature and finite rule table. Assume a rule removes an active pair, has a fixed finite auxiliary interface, and replaces it with a fixed finite net having the same interface, possibly including a fixed number of bare loops. Then bounded scheduled reductions of explicit inputs have witness-faithful quartic families of size linear in the explicit input and external horizon. Bare loops are preserved exactly.*

Proof. Let r be the maximum number of old auxiliary boundary positions, and b the maximum number of newly created cells. These are constants of the fixed system. On r positions the number of partial matchings is

$$I_r = \sum_{k=0}^{\lfloor r/2 \rfloor} \frac{r!}{2^k k! (r-2k)!}. \quad (145.18)$$

For each rule and each pattern, precompute the suppression of the finite temporary wiring. Lemma 141.7 applies with r in place of four: the only internal vertices are boundary positions. The output is a fixed routing of the ordered external mates and newly created ports, together with a fixed loop increment plus any loops already in the template.

A finite selector circuit chooses this precomputed case from exact partner comparisons. Retain arbitrary external identifiers as data, rather than including their values in the finite table. The number of reads, writes, comparisons, and arithmetic gates per interaction is constant for the fixed system. The storage layout generalizes (145.2) with a fixed number of slots per cell, fresh allocation uses at most bT new identifiers, and the memory certificate and chronological uniqueness proof apply unchanged. $\square$

The dependence on the rule signature is not claimed small: enumeration of I_r can be expensive when arity varies. The transfer theorem is uniform in inputs and horizons for a *fixed* rule table, not a linear-time theorem for arbitrarily large rules supplied as data.

This is a route to arithmetic certificates for esoteric graph-rewriting languages that compile into bounded-interface interaction systems. It still requires an exact source-to-net translation and a statement of which source observations that translation preserves. The substrate name alone is not a proof of that interface.

145.7 Other finite local graph-rewriting substrates [13]

Remark 145.8 (Bounded local interaction systems; manuscript 13’s statement of Theorem 145.7). Fix a finite interaction-system signature of bounded arity and a finite rule table whose right-hand sides have boundedly many cells, boundary ports and isolated cyclic wires. A history of T rule applications from input size N_0 has a cycle-preserving quadratic certificate with $O(N_0 + T)$ variables and residuals, and hence a quartic sum-of-squares certificate of that size. For a fixed schedule and fixed rule choice, the ancillary witnesses can be canonical.

Second presentation (source 13). The maximum arity and right-hand-side sizes are constants because the rule table is fixed. Replace the constants four and three in the allocation by those maxima. Each application is still a bounded literal right-hand side followed by boundedly many boundary splices. Any intrinsic right-hand-side cyclic wires contribute a fixed known increment to the loop counter. The same input scan, local compilation and global memory theorem apply. $\square$

This corollary concerns local port-matching rewrites. It does not automatically cover a rule that tests global graph connectivity, identifies arbitrarily large subgraphs, or performs an unbounded graph search in one “step”. Such operations require their own paid certificates. An esoteric syntax alone is not a source of arithmetic savings; the sizes of its controller, mutable memory and input interface are the relevant quantities.

146 A second route: permutation orbits and an implemented scheduled compiler

[write] *Manuscript 15’s Sections 4–9, except its contextual completeness theorem (Section 142), the subsection on MRDP (Section 147) and the witness pair (Section 140.4). It certifies gluing by a canonical permutation-orbit certificate instead of a memory log: the path–cycle dictionary $\text{cyc}(\alpha \circ \beta) = |B|/2 + 2L$ (Theorem 146.1), a minimum-and-distance orbit witness that is unique on all of $\mathbb{N}$ (Theorem 146.3), its selector form (Corollary 146.5) and the single-fold gluing certificate (Theorem 146.6). Its fixed-schedule compiler for all six rules (Theorem 146.7) is endpoint-parametric, has the exact ledger Corollary 146.8 and size $O(T(C_0 + T + F)^2)$, and is the only one of the three compilers that is implemented as an exporter. It trades the linear size of Theorem 145.3 for an implemented, dense construction. The canonical orbit certificate is new to the repository; the gating of its finite schedule disjunction is Part X’s (Theorem 75.6).*

146.1 From wire components to permutation orbits [15]

Define the permutation

$$\sigma = \alpha \circ \beta, \quad \sigma(v) = \alpha(\beta(v)). \quad (146.1)$$

Let $\text{cyc}(\sigma)$ denote its number of orbits, including fixed points.

Theorem 146.1 (Path–cycle dictionary). *For the gluing data (141.4):*

- (i) *Each path component gives one σ -orbit. That orbit contains its two boundary endpoints and no other boundary vertices.*
- (ii) *Each boundary-free cycle component gives two σ -orbits.*
- (iii) *Two boundary ports are paired after suppression if and only if they belong to the same σ -orbit.*

Consequently, if L is the number of newly created cyclic wires, then

$$\boxed{\text{cyc}(\sigma) = \frac{|B|}{2} + 2L, \quad L = \frac{\text{cyc}(\sigma) - |B|/2}{2}.} \quad (146.2)$$

Proof. A boundary-free component is an alternating cycle of α edges and β edges. On it, σ moves two edges at a time. The vertices in each of the two parity classes form one orbit, so there are exactly two. This remains true for a two-edge cycle: α and β are the same transposition, and their product has two fixed points.

For a path, label its consecutive vertices $v_0, \dots, v_{2q+1}$ so that v_0, v_{2q+1} are the boundary points. Its first and last edges are α edges; its intervening edges alternate. Applying σ travels along one parity class toward an endpoint, uses the fixed-point action of β there to reverse to the other parity class, and returns along it. Thus all vertices of the path belong to one orbit. In the shortest case, the path is just one α edge between two boundary points, and σ is their transposition.

All components are of these two forms. Their vertex sets are invariant under α , β , and σ . There are $|B|/2$ path components; each contributes one orbit, and each of the L cyclic components contributes two. This proves the formula and the endpoint criterion. $\square$

Remark 146.2. Neither $\text{cyc}(\sigma)$ alone nor an unweighted connected-component count has the required meaning. Boundary paths count once, while closed alternating cycles count twice. Omitting the factor of two in (146.2) already miscounts a single two-edge cycle.

146.2 Canonical, single-fold certificates for permutation cycles [15]

146.2.1 Fixed successor indices

Fix a permutation σ of $[n] = \{1, \dots, n\}$. For each i , introduce natural variables r_i, d_i, h_i, e_i and impose

$$e_i(e_i - 1) = 0, \quad (146.3)$$

$$r_i + (1 - e_i)(h_i + 1) = i, \quad (146.4)$$

$$e_i h_i = 0, \quad (146.5)$$

$$r_i = r_{\sigma(i)}, \quad (146.6)$$

$$d_i = (1 - e_i)(d_{\sigma(i)} + 1). \quad (146.7)$$

There are $4n$ variables and $5n$ quadratic equations.

Theorem 146.3 (Canonical orbit witness). *For every σ , equations (146.3)–(146.7) have exactly one solution in $\mathbb{N}^{4n}$. In that solution, on every orbit C ,*

$$\begin{aligned} r_i &= \min C, \\ e_i &= \mathbf{1}_{\{i = \min C\}}, \\ h_i &= \begin{cases} 0, & i = \min C, \\ i - \min C - 1, & i \neq \min C, \end{cases} \\ d_i &= \min\{k \geq 0 : \sigma^k(i) = \min C\}. \end{aligned} \quad (146.8)$$

In particular $\sum_i e_i = \text{cyc}(\sigma)$, and all coordinates are at most n .

Proof. The Boolean equation gives $e_i \in \{0, 1\}$. The r equations make r_i constant on each orbit. Suppose an orbit had $e_i = 0$ at all its vertices. Its distance equations would give $d_i = d_{\sigma(i)} + 1$. Summing around that orbit yields $0 = |C|$, an impossibility. Thus each orbit has a vertex j with $e_j = 1$.

At such a vertex, (146.4) gives $r_j = j$. A second vertex k in the same orbit with $e_k = 1$ would give $k = r_k = r_j = j$, so the root is unique. At every other vertex i the same equation gives $r_i < h_i + r_i + 1 = i$. Since $r_i = j$, this proves $j < i$ for every $i \neq j$ in the orbit. Hence $j = \min C$.

Equation (146.5) forces $h_j = 0$ at the root. At the nonroots, (146.4) determines $h_i = i - j - 1$. The root distance is zero by (146.7); working backwards along the orbit uniquely determines every other distance and gives exactly the forward distance to j . Thus every solution must be (146.8). Direct substitution verifies that these values do form a natural solution. The bounds follow from the orbit length and the range of labels. $\square$

The proof establishes uniqueness over the entire unbounded natural orthant, not just inside a bounded search box. The distance variables prevent a fictitious root label outside the orbit. The strict-gap equation selects the minimum root, and the reset equation $e_i h_i = 0$ prevents a free slack at that root.

Example 146.4. For $\sigma = (1\ 3\ 2)(4\ 5)$ the unique coordinates are

i	$\sigma(i)$	r_i	d_i	h_i	e_i
1	3	1	0	0	1
2	1	1	1	0	0
3	2	1	2	1	0
4	5	4	0	0	1
5	4	4	1	0	0

The sum of the root bits is two, as required.

146.2.2 Successor selection without raising the degree

In a compiler the wiring is not a fixed coefficient table: it may be unknown intermediate data. Let S be the permutation matrix of σ in row-successor convention, so $S_{ij} = 1$ exactly when $j = \sigma(i)$. Then $r_{\sigma(i)} = \sum_j S_{ij} r_j$, a quadratic expression in input entries and witnesses. Directly substituting $\sum_j S_{ij} d_j$ into (146.7) would produce a cubic expression. Introduce one additional natural variable s_i per vertex and replace the last two equations by

$$r_i = \sum_j S_{ij} r_j, \quad (146.9)$$

$$s_i = \sum_j S_{ij} d_j, \quad (146.10)$$

$$d_i = (1 - e_i)(s_i + 1). \quad (146.11)$$

Corollary 146.5 (Selector form). *For every natural permutation matrix S , equations (146.3)–(146.5) and (146.9)–(146.11) have exactly one solution. They use $5n$ natural witnesses and $6n$ residuals, each of total degree at most two in S and the witnesses. The coordinates are those of Theorem 146.3, together with $s_i = d_{\sigma(i)}$.*

Proof. Since S is a permutation matrix, its selected sums equal the corresponding successor coordinates. The new s_i values are uniquely determined, and the remaining system is exactly that of Theorem 146.3. $\square$

If S is an arbitrary natural input matrix rather than already known to be a permutation matrix, add its n row-sum and n column-sum equations. They force S to be a permutation matrix without further witnesses. In our interaction compiler, that fact is instead inherited from a perfect matching and a fixed interface involution.

146.2.3 Why these equations are not merely a guessed spanning forest

An existential spanning forest normally admits many choices of root, parent, and traversal order. Those choices create extra polynomial zeros even when the underlying computation is deterministic. Here the directed orbit is already given, its root is forced to be its least label, and every distance is forced by the orbit. There is no residual choice of a tree or traversal. The construction is therefore suitable for witness-faithful compilation rather than only for existential correctness.

146.3 Loop-exact gluing as a quartic relation [15]

Let B be the surviving boundary and let M' be a proposed perfect matching on it. With fixed successor indices, impose (146.3)–(146.7), then impose

$$r_u - r_v = 0 \quad \text{for every } \{u, v\} \in M', \quad (146.12)$$

$$\sum_i e_i + 2\ell_0 - 2\ell_1 - |B|/2 = 0. \quad (146.13)$$

All vertex names in the orbit equations are their fixed positions in the ordered temporary vertex set; they need not coincide with global port identities.

Theorem 146.6 (Single-fold gluing certificate). *Fix valid gluing data (V, α, β, B) and a target boundary matching M' . The sum of squares of the residuals in (146.3)–(146.7), (146.12), and (146.13) has degree at most four and uses $4|V|$ witnesses and*

$$5|V| + |B|/2 + 1$$

residuals. For given natural ℓ_0, ℓ_1 , it has exactly one natural zero if gluing produces M' and changes the loop count from ℓ_0 to ℓ_1 ; otherwise it has none.

Proof. A sum of squares of integers vanishes exactly when every summand vanishes. By Theorem 146.3, the orbit coordinates are unique and identify actual orbits. By Theorem 146.1, every boundary orbit contains exactly two boundary vertices. Thus the equal-root constraints for the proposed perfect matching force precisely the real output matching. Equation (146.13) is exactly (146.2), with the old loop count carried through. Conversely, the actual output and the canonical orbit coordinates satisfy all these equations. $\square$

146.3.1 Both endpoint matchings may be arithmetic data

Let A be a symmetric natural matching matrix for α , and suppose β is fixed. The successor-selector matrix requires no new matrix multiplication:

$$S_{ij} = A_{\beta(i),j}. \quad (146.14)$$

This is just a permutation of rows. Use the selector certificate of Corollary 146.5. For an unknown or parameter-valued output matching W' on B , replace (146.12) by

$$W'_{uv}(r_u - r_v) = 0 \quad (u < v, u, v \in B), \quad (146.15)$$

and enforce the row-sum matching constraints on W' .

All equations remain quadratic. There are $5|V|$ orbit witnesses and $6|V| + \binom{|B|}{2} + 1$ gluing residuals, in addition to matching validation. The natural zeros are still unique for fixed valid endpoint matchings. This is the form used by the implementation.

146.3.2 A separately quadratic alternative

The Boolean residual $e_i(e_i - 1)$ contains a repeated variable. An optional variant replaces it by $e_i + f_i - 1 = 0$, with $e_i, f_i \in \mathbb{N}$, and replaces $1 - e_i$ in the other equations by f_i . Every residual is then multiaffine: no variable appears with exponent greater than one within an individual residual. There is one extra uniquely determined witness per vertex. Consequently the resulting quartic is of degree at most two in each individual coordinate and is convex in each coordinate separately, with the others fixed. This is a structural refinement, not a claim of joint convexity or an implemented optimization of the exported example.

146.4 The all-six-rule trace compiler [15]

146.4.1 Static lifetime data and dynamic matching data

Fix an initial type-labelled set of agents, a fixed free-port set, and a positive-length schedule

$$\mathcal{S} = ((a_0, b_0), \dots, (a_{T-1}, b_{T-1})).$$

Require $a_t < b_t$ and require both identities to be live at the corresponding stage. Their types determine the unique rule template. The allocator starts above the largest initial identity and assigns consecutive fresh identities in a prescribed template order. It never reuses an identity.

This produces the successive agent sets and their types without examining any wire. A schedule that consumes a nonexistent or already deleted agent is rejected syntactically. A schedule that selects two live but unconnected principal ports is *not* rejected syntactically; the polynomial must rule it out.

Write P_t for the number of ports of configuration t , including free ports. For $1 \leq t < T$, introduce one natural variable $W_{uv}^{(t)}$ for every unordered pair of distinct live ports, and a natural loop-count variable ℓ_t . Use the same scalar for the symmetric matrix entries and set the diagonal to zero. The full matrices $W^{(0)}$ and $W^{(T)}$, and loop counts ℓ_0, ℓ_T , are endpoint parameters. Validate every state by the row-sum equations (141.5).

146.4.2 One step, without arithmetic indirection

For step t , impose the active-pair residual

$$W_{(a_t,0),(b_t,0)}^{(t)} - 1 = 0. \quad (146.16)$$

Remove these two principal ports. The remaining old matching is disjointly united with the fixed replacement matching; call its matrix $A^{(t)}$. Every entry of $A^{(t)}$ is either an entry of $W^{(t)}$ or a constant 0 or 1. The row-sum constraints and (146.16) ensure that this really is a perfect matching after the principal row and column removal.

Create the fixed interface involution β_t described in Section 140.3. Let N_t be the size of this temporary vertex set. Impose its $5N_t$ -variable selector certificate using $S_{ij}^{(t)} = A_{\beta_t(i),j}^{(t)}$. Finally impose (146.15) for $W^{(t+1)}$ and (146.13) for ℓ_t, ℓ_{t+1} .

This explicitly pays for dynamic wire selection: the sums selecting r and d contain matching variables multiplied by orbit variables. There is no unaccounted-for “array lookup” in the proof. Its cost is dense rather than asymptotically optimal, but it is fully present.

Theorem 146.7 (Loop-exact scheduled compilation). *For every syntactically valid positive-length lifetime schedule $\mathcal{S}$, the construction above effectively produces an integer polynomial*

$$Q_{\mathcal{S}}(W^{(0)}, \ell_0, W^{(T)}, \ell_T; \mathbf{z}) = \sum_{j=1}^R f_j^2$$

with each f_j of total degree at most two. For fixed natural endpoint data:

- (i) a natural zero exists exactly when the endpoint matrices are valid matchings and the specified sequence of T interaction-combinator steps transforms the initial net into the exact target net, including cyclic wires;
- (ii) whenever a zero exists, the complete natural witness tuple $\mathbf{z}$ is unique.

The compiler treats all six rules and does not use MRDP as a compilation step.

Proof. Suppose first that the endpoint nets and the scheduled trace are valid. Supply each actual intermediate matching and actual intermediate loop count. For every step, supply the unique canonical orbit certificate of its temporary matching. The rule construction, active-pair equation, output matching constraints, and loop equation all hold. Thus every residual vanishes.

Conversely suppose all residuals vanish. The natural row-sum equations force all displayed matrices to be perfect matchings. Equation (146.16) forces the selected principal pair to be an actual redex. Because the template was constructed from the actual types in the lifetime ledger, the temporary matching and interface involution describe exactly the correct rule in exactly that context. The selector certificate is therefore a certificate for a true permutation. By Theorem 146.6 and its selector extension, the next matching and loop count are exactly the true result. Induction over t gives the entire scheduled computation and its stated target.

For uniqueness, fix the initial data and schedule. A rule at a specified active pair has a unique result with the chosen allocator. Inductively, every intermediate matching and loop count is forced. Every step's orbit coordinates are then forced by Corollary 146.5. These are all the existential variables, so the complete tuple is unique. $\square$

146.4.3 Exact resource ledger

No witnesses are used for agent types, principal-port identities, or template selection because the lifetime schedule and its type ledger are structural inputs to this compiler. No witnesses are hidden in the matrix notation: only upper-triangular matching entries are stored.

Corollary 146.8 (Literal counts). *For $T \geq 1$, the compiler uses exactly*

$$V = 5 \sum_{t=0}^{T-1} N_t + \sum_{t=1}^{T-1} \binom{P_t}{2} + (T-1) \quad (146.17)$$

existential variables, and exactly

$$R = \sum_{t=0}^T P_t + 2T + 6 \sum_{t=0}^{T-1} N_t + \sum_{t=0}^{T-1} \binom{P_{t+1}}{2} \quad (146.18)$$

residuals in its literal, unpruned construction. The number of endpoint scalar parameters is $\binom{P_0}{2} + \binom{P_T}{2} + 2$.

Proof. The first term of (146.17) counts the five orbit coordinates per temporary vertex. The second counts intermediate matching matrices; the last counts intermediate loop scalars. Equation (146.18) counts, respectively, state row sums, one active-pair and one loop equation per step, six orbit equations per temporary vertex, and every output-pair implication. Endpoint loop scalars are parameters, not witnesses. Some residuals can become identically zero or duplicate after specialization, but the displayed ledger is the literal exporter count before such pruning. $\square$

Rule	New agents	$P_{t+1} - P_t$	$N_t - P_t$	New loops
$\varepsilon\varepsilon$	0	-2	-2	0
$\gamma\varepsilon, \delta\varepsilon$	2	-2	2	0
$\gamma\gamma, \delta\delta$	0	-6	2	0, 1, or 2
$\gamma\delta$	4	6	14	0

[write] The bounds on C_t , on the number of allocated identities and on ℓ_t are Corollary 140.2; the bounds on P_t and N_t and the size estimate are manuscript 15's own.

Proposition 146.9 (Growth and witness-height bounds). *For an initial net with C_0 agents and F free ports,*

$$C_t \leq C_0 + 2t, \quad P_t \leq 3C_0 + 6t + F, \quad N_t \leq P_t + 14.$$

At most $C_0 + 4T$ distinct agent identities are allocated through T steps. Moreover,

$$\ell_0 \leq \ell_t \leq \ell_0 + 2t.$$

Thus the witness and residual counts are $O(T(C_0 + T + F)^2)$. Every intermediate matching entry is a bit, every orbit coordinate is at most N_t , and every intermediate loop coordinate is at most $\ell_0 + 2T$.

Proof. The rule table gives the agent and port increments and the maximum of four fresh agents per step. A mixed or erasing rule joins every old auxiliary through a replacement interface to a surviving new principal port. Therefore no internal component can close in these rules. The eraser-eraser rule has no internal joins. Only binary annihilation can produce closed components; there are just two replacement wires, and every such component uses at least one of them. Hence at most two loops are added. The remaining claims follow from the explicit ledger and Theorem 146.3. $\square$

These estimates count witnesses and residuals, not a minimal arithmetic circuit for their evaluation. The selector sums can have $O(N_t^2)$ monomials across a stage. We make no comparison with the repository's optimized fixed universal operation counts: these are different objects and different resource measures.

146.5 Unknown schedules, multiplicity, and the fixed-arity boundary [15]

146.5.1 A finite disjunction can remain quartic and witness-faithful

For a fixed horizon and initial type ledger, there are finitely many lifetime schedules. For a fixed labelled target shape, discard schedules ending in the wrong agent ledger. The remaining branches use common endpoint parameters x , with disjoint witness vectors $z^{(s)}$.

Each residual in the compiler has the form

$$f_{s,j}(x, z) = c_{s,j}(x) + g_{s,j}(x, z),$$

where $c_{s,j}$ is affine in the endpoint parameters, $g_{s,j}(x, 0) = 0$, and $f_{s,j}$ has total degree at most two. The absence of a pure quadratic parameter term follows by inspection: the nonlinear terms are parameter-witness products or products of witness coordinates.

[write] Setting the witnesses of inactive branches to fixed values, so that the gated residuals vanish and the active branch is unchanged, is the mechanism of Part X's guarded quadratization (Theorem 75.6); manuscript 15 applies it to retained schedules and claims no novelty for it.

Proposition 146.10 (One zero per successful schedule). *Introduce a natural branch bit b_s for every retained schedule and impose*

$$\begin{aligned} b_s(b_s - 1) &= 0, & \sum_s b_s &= 1, \\ (1 - b_s)z_i^{(s)} &= 0, & b_s c_{s,j}(x) + g_{s,j}(x, z^{(s)}) &= 0. \end{aligned}$$

The sum of squares of these residuals is quartic. For fixed endpoints its natural zeros are in bijection with successful retained schedules.

Proof. Exactly one branch bit is one. All witnesses in inactive branches are zero, so their gated residuals vanish automatically. The active branch reduces to its original equations and has one witness exactly when its scheduled trace is valid, by Theorem 146.7. The branch bit itself distinguishes different successful schedules. Every displayed residual is quadratic because c is affine and g already has total degree at most two. $\square$

This is an existence construction, not an efficient schedule selector. The number of branches can be exponential in the horizon. The archive implements the practical fixed-schedule compiler, not an all-schedules disjunction exporter. The zero-step case is simply endpoint equality and can be handled separately; a length-at-most- T construction includes the finite family of lengths $0, \dots, T$ with the same gating method.

146.5.2 A finite certificate can nevertheless prove an infinite execution

Proposition 146.11 (Explicit lasso up to renaming). *Suppose a nonempty scheduled trace takes a net N to a net N' with the same loop count and an explicitly given type- and port-order-preserving isomorphism $N' \cong N$ that fixes free-port labels. Then there is an infinite reduction from N .*

Proof. Interaction rules depend on types, ordered port incidences, and freshness, not on the numerical values of agent names. Transport the given finite trace through the isomorphism, choosing fresh identities above all those already used. This yields another copy of the trace from N' . Repeat inductively. The positive trace length makes the concatenation infinite. $\square$

An explicit endpoint renaming can be imposed by linear equalities between the corresponding endpoint matching entries and loop counts. Combining these with our quartic compiler certifies this sufficient condition for nontermination. It is not claimed that every infinite computation admits such a finite lasso, nor that the existence of one infinite execution decides every normalization question.

[write] The lasso is a finite certificate of nontermination for interaction nets. It is related to, not a case of, Part XII's criterion for infinite periodic instruction tails (Theorem 100.1), which concerns priority programs; and it is a sufficient condition only, as manuscript 15 says. Infinite behaviour is the subject of the neighbouring report *liveness-beyond-halting*.

146.6 Worked examples: the exported quartic and a repeatable net [15]

146.6.1 The exported parametric quartic

Give the agents identities $0, 1, 2, 3$ and choose schedule $((0, 1), (2, 3))$. The endpoint target has no agents and two cyclic wires. The compiler is generated from the agent types and schedule *before* any input wire values are supplied. It therefore produces the same polynomial for both initial matchings.

The configuration port counts are $(12, 6, 0)$, and the two temporary vertex counts are $(14, 8)$. Equations (146.17)–(146.18) give

$$V = 5(14 + 8) + \binom{6}{2} + 1 = 126,$$

$$R = (12 + 6 + 0) + 4 + 6(14 + 8) + \binom{6}{2} + \binom{0}{2} = 169.$$

There are $\binom{12}{2} + 2 = 68$ endpoint parameters: 66 initial matching entries and the initial and final loop counts. The empty target has no matching-entry parameters.

The archive contains all 169 residual polynomials and all coordinates of the unique witness for $\mathcal{A}$. Its sum of squares evaluates to exactly zero. For $\mathcal{B}$, the second scheduled pair is not active, so the theorem implies that no natural witness exists. Merely changing the initial parameters of the saved $\mathcal{A}$ witness to those of $\mathcal{B}$ produces the positive value 332; this is a useful implementation check, not the proof of nonexistence of every alternative witness.

146.6.2 A four-step repeatable net

Consider agents $0 : \delta, 1 : \gamma, 2 : \varepsilon, 3 : \varepsilon$ with wires

$$(0, 0)(1, 0), \quad (0, 1)(2, 0), \quad (0, 2)(1, 1), \quad (1, 2)(3, 0).$$

This is the numbered-port version of the familiar repeating interaction pattern illustrated by Lafont [41, Fig. 3]. With our fresh-name convention, a checked four-step schedule is

$$(0, 1), \quad (2, 6), \quad (3, 5), \quad (9, 10).$$

The resulting ordered-port net is isomorphic to the input, with no cyclic wires. Its four-step certificate has 505 existential variables and 650 quadratic residuals. The exact initial and terminal wires and the schedule are in the receipt. By Proposition 146.11, the finite calculation establishes an infinite reduction after repeating the renaming pattern.

[write] Batch 78, 2 October 2026. The review [65] derives a smaller orbit block on the natural domain. The retained equation $d = (1 - e)(s + 1)$ with natural d, e, s forces $e \leq 1$, so the Boolean equation (146.3) is redundant; with the reset (146.5) retained, (146.4) can be replaced by the linear $r + h + 1 - e - i = 0$. A fixed-successor block then has $4n$ instead of $5n$ rows and a selector block $5n$ instead of $6n$, with the same natural zero set: $169 \rightarrow 147$ residuals in the exported example, with the same 126 witnesses. The review notes that the bit implication fails over signed integers. It is the review's result; the shipped export is unchanged.

147 Boundaries: fixed horizon, schedules and the MRDP interface

[write] *The limitations stated by the three manuscripts, each in its own words: a fixed horizon is not fixed arity, the bijections do not quotient schedules, and none of the three gives an ordinary-integer universal loader, a single-fold or finite-fold MRDP theorem, or an operation-count record. Manuscript 13 adds that its construction does not improve the project's 75- and 87-operation figures. The classical MRDP consequence for unbounded reachability is stated by all three and is printed from each, because each draws a different line around it.*

147.1 Why this does not improve a fixed universal-polynomial record [13]

The family (145.16) grows with the horizon. The single product comparison compresses a *permutation check*, not an unbounded list into finitely many scalar coordinates independent of its length. In particular, there are still $O(T)$ chronological access records, controller witnesses and product-tree gates.

The audited repository README reports 75 arithmetic operations for a complete certificate and 87 for a separate fixed universal polynomial [1]. Those are different resources and a different target. The present construction does not lower either number, supply their ordinary-integer input loader, or prove a global arithmetic-circuit optimum. An explicit counted fixed-arity interaction-net history compiler remains a further project.

[write] The figures quoted are those of the project README at manuscript 13's pin [1]: 75 operations for a complete certificate and 87 for one universal polynomial. They are unchanged at this write. Manuscript 14's ledger (145.9) and manuscript 15's counts Corollary 146.8 make the same disclaimer.

147.2 Generalization and the MRDP interface [13]

147.2.1 Qualitative unbounded Diophantineness

For finite encoded nets, the exact successor relation is computable: verify the finite input, perform the finite table lookup, and execute the splice algorithm. A finite history and any of the stated terminal predicates are therefore decidable checkable. Enumerating finite schedules semidecides unbounded reachability of such a target.

Consequently, after choosing an ordinary effective coding of all public finite data by natural numbers, the unbounded target-reachability relation is computably enumerable and hence Diophantine by MRDP. This last consequence is classical, not a new universality theorem. The repository's Diophantine/MRDP.lean explicitly exposes the finite-polynomial statement `Diophantine.mrdp_iff`: the polynomial and witness dimension are selected before the input natural number [34].

The logical interface is therefore clean:

$$\begin{aligned}
 \text{exact finite-net semantics} &\implies \text{computable history checker} \\
 &\implies \text{c.e. target set} \\
 &\stackrel{\text{MRDP}}{\implies} \text{fixed finite polynomial.}
 \end{aligned} \tag{147.1}$$

Only the first two arrows have the local topology and bounded size analysis of this report. The last arrow does not preserve the displayed linear family, its explicit operation count, its height bound, or its schedule-relative unique-extension property.

147.2.2 No illicit single-fold conclusion

It is tempting to argue that a deterministic evaluator plus unique memory witnesses yields a single-fold Diophantine representation of every computably enumerable set. That inference is not justified. The bounded construction fixes T externally, can have several legal schedules, and still uses a number of witnesses growing with T . Applying MRDP to the unbounded relation may introduce new witnesses with uncontrolled multiplicity. Neither eliminating the horizon nor preserving witness uniqueness through MRDP has been proved here.

Similarly, universality of the six-rule formalism does not by itself prove that a particular closed-net agent-free predicate, under one particular ordinary-integer loader, is complete for all computably

enumerable sets. An explicit simulation and output interface would be required for that stronger statement. We do not infer it from the four- δ example.

147.3 Boundaries: what has not been compressed [14]

147.3.1 Fixed horizon is not fixed arity

The schema $P_{N,T}$ has linearly many coordinates in T . It is a family of ordinary integer polynomials, not one polynomial with a fixed number of variables that uniquely represents every unbounded run. Quantifying a code of a variable-length sequence introduces a new problem: the decoding and verification of that code must itself be represented, and its witness fibers must be controlled.

The MRDP theorem provides an ordinary Diophantine representation for any computably enumerable relation, including effectively coded finite reachability [73]. Applying it to a verifier does not preserve the verifier's unique witness automatically. General finite-fold and single-fold representations are a distinct question, discussed explicitly by Matiyasevich [9]. No step in this paper resolves that question.

147.3.2 Degree, domain, and integer-size claims

All witnesses range over $\mathbb{N}$. Simply allowing arbitrary integers would invalidate range-complement arguments and their uniqueness. Replacing nonnegative coordinates by sums of four squares would add representation multiplicity, so it cannot be done while retaining the same bijection. The quartic bound comes from a sum of squares of residuals; it is not a claim about a least possible degree for every semantic relation.

The large integer-product witnesses are bounded in bit length, not small in numerical magnitude. A sparse quartic can be cheap to describe and still have large coordinates. No fast general algorithm for finding its zeros is claimed. In particular, verification complexity, witness construction from a supplied execution, and existential search are different tasks.

147.3.3 No unproved universality interface

Interaction combinators are a classical universal system [41]. The exact six-rule reducer here uses that system, rather than a universal claim about the terminating all- δ fragment. The four-cell counterexample and its exhaustive search lie in that fragment and serve as wiring tests.

We do not infer, just from universality, that every particular terminal predicate used here is c.e.-complete under a paid ordinary-integer loader. Nor do we infer that the existence of a fixed universal net supplies a short arithmetic circuit for its input map. Those would be additional constructions and are recorded as research directions below.

147.4 Why this does not prove finite-fold MRDP [15]

The schedule length and its wire universes determine the syntax and number of variables of the polynomial. They are *not* ordinary numeric parameters in one fixed polynomial. Quantifying over T in a metatheoretic statement therefore gives a family of polynomial formulas, not a single formula with a fixed tuple of existential variables.

Unbounded interaction-combinator reachability is computably enumerable: enumerate finite schedules and simulate them. A fixed-arity Diophantine representation then follows abstractly from MRDP [7, 34]. That existence theorem does not preserve the unique witnesses, explicit degree and size ledger, or wire-level semantics of the present compiler. The distinction between arbitrary Diophantine representations and single-fold or finite-fold representations is substantive [9].

Accordingly, the present result does not resolve the finite-fold or single-fold Diophantine representation problem. It also does not instantiate one fixed universal interaction net with a paid ordinary-integer input loader. These are separate research tasks, not costs silently absorbed into a universality claim.

147.5 Statements not established here [15]

We do not establish a fixed-arity, fixed-degree, explicitly instantiated universal interaction-net polynomial with a paid ordinary-natural input interface. We do not provide a sparse near-linear-memory exporter, a canonical quotient of all concurrent schedules, a minimal witness count, or a globally optimal arithmetic circuit. We also do not establish priority for the specific canonical certificate or claim an externally refereed breakthrough.

What is established by the supplied proofs is narrower and concrete: exact arithmetic wire composition, exact closed-loop accounting, unique natural witnesses for each finite schedule, and a complete six-rule executable implementation with a literal resource ledger.

[write] The formal interface named by manuscripts 13 and 15 is `Diophantine.mrdp` and `Diophantine.mrdp_iff` in `Computability/HilbertTenthProblem/Lean/Diophantine/MRDP.lean` (lines 33 and 42; the file is the same blob, 74aea8c5e, at the two pins and at this write) [34]. These declarations assert existence of a Diophantine representation and say nothing about its number of witnesses. No theorem of Part XV is formalized in Lean or Rocq; the project’s formalized single-fold exponential representation (Remark 64.2) is not an input to this Part.

148 Implementation, validation and formalization proposals

[write] *The three manuscripts’ implementation, validation and formalization sections, in that order: manuscript 13’s Section 11 and its proposed formalization path (its Section 10.4), manuscript 14’s Section 11, and manuscript 15’s Sections 10 and 11.1–11.2. Their package appendices are the last sections of this Part. The shipped names of their programs and data are printed in prose; command listings keep the delivered layout, in which the programs import one another by their delivered names, and this report’s README gives a recipe for the shipped names. The manuscripts’ `article.tex`, `article.pdf` and delivery `README.md` are not shipped; their text is this Part and Sections 3.13 to 3.15.*

What is implemented and what is proved only. [write] The following ledger, collected from the three manuscripts’ own statements, says for each main statement whether it is implemented, exported or only proved. “Proved” means a written mathematical proof; nothing is formalized.

Statement	Status
Six-rule net semantics	Implemented by all three: <code>code/13-exact-wiring-wiring.py</code> (two gluing algorithms), <code>code/14-net-topology-nets.py</code> (two reducers), <code>code/15-no-ghost-wires-loop_exact.py</code> (three gluing algorithms). The reducers share their manuscripts’ rule tables; the tables were checked by eye against Lafont’s numbered figures.
Theorem 142.1 and Lemma 142.2	Proved. Recurrence implemented (<code>code/13-exact-wiring-parity.py</code>); independent enumeration of 121,537 closures for weights up to six, and bounds and equality cases up to weight thirty computed with the recurrence.
Corollaries 142.3 and 142.4 and Theorem 142.6	Proved; Theorem 142.6 is an existence theorem, with no efficient decoder.
Proposition 142.5	Proved; construction tested on 22,272 cases.
Theorem 141.2, Proposition 141.8, and Corollary 141.5	Proved; gluing algorithms compared on all interface orders (13) and on 81,393 gluing instances (15).

Statement	Status
Theorems 143.1 and 143.3	Proved; the δ and γ kernels are exported as expanded quartics (104 monomials each) and checked by an independent JSON checker.
Theorem 144.4	Proved; implemented (<code>code/13-exact-wiring-diophantine_memory.py</code>); one exported example (167 witnesses, 144 residuals, 2,496 monomials).
Theorem 144.6	Proved; implemented (<code>code/14-net-topology-certificates.py</code>); exhaustive check of 584 logs with $A = V = 2$, $L \leq 3$; one exported example (64 auxiliaries, 64 residuals, 402 monomials).
Theorem 145.3, Remark 145.5	Proved as paper constructions. No exporter for the full unknown-schedule compiler, and no measured value of G, H in (145.9).
Theorem 145.7, Remark 145.8, the loader (145.17)	Proved only.
Theorem 146.3 and Corollary 146.5	Proved; implemented; all 46,234 permutations of at most eight points, and a bounded uniqueness search through size four.
Theorems 146.6 and 146.7 and Corollary 146.8	Proved; implemented as a fixed-schedule exporter with a separate witness generator; the example has 68 parameters, 126 witnesses and 169 residuals; 412 local contexts spanning all six rules.
The multiaffine variant, Proposition 146.10, the lasso wrapper of Proposition 146.11	Proved, not implemented as exporters; the four-step trace of the lasso example is itself compiled (505 variables, 650 residuals).

Replays. [write] In the batch-78 placement every delivered suite was run on a copy in the delivered layout (Python 3.14.4, Windows), and the write repeated the runs from the shipped files by the recipe of this report’s README. All pass. Manuscript 13’s `code/13-exact-wiring-verify.py`, in its default comparison mode, recomputes 121,537 closures, the bounds to weight thirty, 23,930 splices and the exported memory certificate and compares them with the recorded receipt; it writes nothing. Manuscript 14’s `code/14-net-topology-verify.py` and `code/14-net-topology-verify_exports.py`, and manuscript 15’s `code/15-no-ghost-wires-verify.py`, rewrite their recorded outputs; the regenerated files equal the shipped ones once line endings are normalized, except manuscript 15’s runtime field. `code/15-no-ghost-wires-check_export.py` prints “PASS: 68 parameters, 126 witnesses, 169 residuals, energy 0”. The review [65] replayed all three suites independently, found no mathematical error, and reports one limitation of manuscript 13’s programming interface: `Circuit.failures` ignores unused trailing coordinates instead of requiring the declared tuple length. This does not change the polynomial on its declared coordinates. Manuscript 13’s `data/13-exact-wiring-build_receipt.json` and manuscript 14’s `data/14-net-topology-pdf_quality.json` describe the delivered PDFs, which are not shipped and are preserved in the arrival commit 1977e6ea6.

148.1 Reproducibility and verification boundary [13]

148.1.1 Distributed artifacts

The code has no third-party Python dependencies. The mathematical proof of the compiler does not depend on running it. The following files make the finite evidence and the arithmetic example replayable:

File	Purpose
<code>code/13-exact-wiring-wiring.py</code>	Literal six-rule evaluator; local splices and a separate multigraph-component evaluator; modular distinguishing contexts.
<code>code/13-exact-wiring-parity.py</code>	Exact signed recurrence, partition generator, and canonical representative matching pairs.
<code>code/13-exact-wiring-diophantine_memory.py</code>	Sparse quadratic memory compiler, canonical witness constructor, and expanded quartic export.
<code>code/13-exact-wiring-verify.py</code>	Reproducible enumeration, rule comparisons, memory tests, and receipt verification.
<code>data/13-exact-wiring-verification.json</code>	Exact test counts, example traces, statistics, and SHA-256 hashes of the distributed Python sources.
<code>data/13-exact-wiring-example_memory_certificate.json</code>	Full natural example zero, all 144 residuals, and all 2,496 quartic monomials.

[write] *The listing keeps the delivered layout; manuscript 13's `article.tex` is not shipped, and the recipe for the shipped names is in this report's README. The option `-write` rewrites the two recorded files in `results/`, which are shipped as `data/13-exact-wiring-verification.json` and `data/13-exact-wiring-example_memory_certificate.json`.*

Run from the unpacked package directory:

```
python3 code/verify.py
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

The default test command compares computed results with the receipt. It does not silently rewrite it. The explicit option `-write` regenerates both saved result files and should be used only after inspecting an intentional source change.

148.1.2 Actual finite-test results

The distributed run used Python 3.13.5. Its checks include:

Check	Count
Partition types checked by direct matching enumeration, weights 1–6	29
Closing multigraphs in that independent recurrence check	121,537
Nonidentity partition bounds and equality cases, weights 2–30	28,598
Near-context separation tests, moduli 2–5	22,272
Splice/component comparisons over all interface-join orders	23,930
Perfect matchings on twelve labelled ports, all- δ fragment	10,395
Initially enabled reductions in those four-cell nets	5,670
Four-cell inputs with all reduction orders checked for one exact terminal net	10,395
Bounded multisets checked for collision-free large-base products	912
Explicit small-base collision examples	100
Random consistent memory histories, fixed seed	160
Inconsistent-read histories rejected	160
Inconsistent-final-memory histories rejected	160
One-coordinate mutations of the example's natural witnesses rejected	167

For the rule comparison, all nine ordered type pairs are exercised; these are the six unordered rules with both orientations where appropriate. Contexts have zero, two or four free ports, and pre-existing loop counts zero or three. Every matching of the available context endpoints and every order of the interface joins is tested. The component evaluator and splice evaluator share the literal right-hand-side

table, so this comparison tests gluing implementation and order invariance, not an independently authored rule table. The table’s numbered-port convention was separately checked against Lafont’s original figures.

The recurrence check is algorithmically independent of the recurrence: it enumerates Q and traverses the actual doubled graphs. The bound check up to weight thirty, by contrast, uses the recurrence; it is supplementary rather than an independent proof of the recurrence.

148.1.3 Trust ledger

Claim	Evidence and limitation
Sharp parity separation, including equality cases	Complete recurrence and induction proof; independent finite enumeration for small ranks. No certified historical-priority claim.
Splice semantics and cycle accounting	Component invariant proof; two algorithms compared on the displayed finite domains.
Exact canonical memory certificate	All-tuples proof; actual sparse polynomial generator and expanded example zero.
Full bounded six-rule compiler theorem	Constructive mathematical proof and complete finite rule evaluator. No full controller-polynomial exporter or compiled Lean development is included.
Unbounded Diophantineness	Classical MRDP consequence after an effective coding. No new fixed-arity arithmetic bound or uniqueness transfer.
Global minimality or breakthrough on an established major conjecture	Not claimed. The report resolves the stated bounded-compilation and contextual-information questions.

148.2 A proposed formalization path [13]

A useful formalization can be divided into four theorem layers rather than starting with a very large expanded polynomial. First formalize finite matchings, doubled components, and the splice invariant. Next formalize the contraction recurrence and the induction proving Theorem 142.1. Independently formalize the exact product lemma, the mixed-radix record injection and Theorem 144.4. Finally prove that the finite six-rule controller realizes the semantic reduction relation and compose it with the memory certificate.

These are proposed proof obligations, not claims that new Lean files have been checked. The source text of the repository’s MRDP interface was read, but its dependency closure and axioms were not rebuilt during this work. A later integration should run the project’s compiler and its explicit axiom checks on every newly added declaration.

148.3 Reproducibility and finite validation [14]

148.3.1 Two reducers, not two calls to one algorithm

The reference component reducer builds the temporary multigraph, explicitly retaining parallel edges, and suppresses connected components. The local reducer instead uses the four-port formulas for annihilation and endpoint relocation for creation rules. They share the declared numbered rule *templates* but not the gluing algorithm. Thus agreement tests the arithmetic topology construction against the component semantics; it does not independently derive Lafont’s rule table. The original numbered diagrams were separately inspected on journal pages 81–82.

All six rules occur in the random tests: 194 $\delta\delta$, 435 $\delta\varepsilon$, 1,011 $\delta\gamma$, 509 $\varepsilon\varepsilon$, 409 $\varepsilon\gamma$, and 204 $\gamma\gamma$ steps. The seed is 20261002; each random input is followed for at most eight steps. Free ports, pre-existing

Table 27: Actual results of `code/14-net-topology-verify.py`. All checks passed.

Check	Count
Boolean boundary patterns checked, per annihilation rule	64
Legal / impossible boundary patterns, per rule	10 / 54
Capped source logs, $A = V = 2, L = 1, 2, 3$	584
Valid logs among those source logs	258
One-auxiliary-coordinate +1 mutations rejected	11,310
Closed two-cell inputs across all ordered type assignments	73
Enabled rewrites on those two-cell inputs compared	17
All perfect matchings on four labelled δ cells	10,395
Enabled first rewrites on those four-cell inputs compared	5,670
Seeded full-system random inputs	1,000
Full-system rewrite steps compared	2,762

bare loops, self-wires, and allocation growth are included. The largest observed live net has 22 cells. These numbers describe the actual finite workload, not an asymptotic proof or an exhaustive search of the full universal system.

Every four- δ input is also normalized along all available choices; its resulting literal terminal set has cardinality one. That finite observation is consistent with the classical confluence theory but is not used as a substitute for the correctness proofs above.

148.3.2 What the exported equations establish

An independent JSON checker reconstructs the sum of squares from the serialized residuals, compares it coefficient-by-coefficient with the exported quartic, checks the supplied natural witness, and rejects a single-coordinate increment of every private coordinate in each export. It does not import the compiler or witness constructor.

The two loop-kernel JSON files each contain six source fields, 17 auxiliaries, 23 quadratic residuals, and an expanded degree-four polynomial with 104 monomials. A supplied valid witness realizes the two-loop case. The memory example has $L = 4, A = 4, V = 10$ and log

$$(1, 1, 7), (2, 0, 0), (1, 0, 7), (1, 1, 3). \quad (148.1)$$

It exports 12 source fields, 64 auxiliaries, 64 residuals, and a degree-four polynomial with 402 nonzero monomials. Its source execution is a write of 7, a zero read elsewhere, a read of 7, and a final write of 3.

For all short logs in the exhaustive test, the constructed sorted witness satisfies the equations exactly when direct chronological replay says the log is valid. The mutation test changes each private coordinate of each valid short witness individually by +1 and confirms rejection. It is a useful implementation check, not an exhaustive search over all natural auxiliary assignments. Uniqueness over *all* such assignments is proved in Theorem 144.6.

The package does not contain a kernel-checked proof. Its confidence boundary is the written mathematical argument plus exact Python integer tests.

148.4 Implementation, tests, and reproducibility [15]

148.4.1 What the code actually does

The implementation uses Python’s standard library and exact integers only. `code/15-no-ghost-wires-loop_exact.py` defines typed nets, the complete rule templates, two gluing algorithms, the

canonical orbit witness, a sparse polynomial data type, the parametric scheduled compiler, and a witness generator. The compiler `compile_schedule` receives initial types, free-port labels, and a lifetime schedule; it does not receive the initial or terminal matching.

Every intermediate matching scalar and orbit coordinate has a name. Each residual is stored as an exact sparse integer polynomial. The exporter rejects any residual whose total degree exceeds two. The quartic is represented as the sum of their squares, not as an unnecessarily expanded monomial list. This is a complete polynomial specification: expanding it is mechanical and does not change its mathematical meaning.

The witness generator is a separate procedure. It executes the actual input net, constructs the canonical coordinates, checks the exact requested target, and evaluates every residual. It cannot turn an invalid schedule into a certificate by changing the polynomial.

148.4.2 Three independent gluing algorithms

The test suite compares three methods on the same finite gluing data:

1. the $\alpha\beta$ permutation, its orbit minima, and (146.2);
2. an undirected multigraph component traversal, retaining the multiplicity of parallel edges;
3. sequential pair suppression, using direct partner rewiring and a loop increment when the two suppressed vertices are already matched.

The second and third methods do not call the permutation algorithm. Agreement therefore exercises the central topological claim more meaningfully than checking an implementation against another wrapper around itself. These are independent algorithms within the accompanying implementation effort, not an external independent review.

148.4.3 Exact finite test receipt

The complete run produced the following results:

Test family	Count
All permutations on sets of size 0 through 8	46,234
Candidate assignments in bounded uniqueness checks through size 4	240,393
All matching/involution gluing instances on even sizes through 8	81,393
Local active-pair contexts spanning all six rules	412
Compiled zero-witness checks for those contexts	412
Single-coordinate witness mutations rejected in local tests	600
All perfect matchings on four δ agents	10,395
Enabled rewrites checked on those four-agent nets	5,670
Saved-example witness mutations rejected	203

The gluing family ranges over every perfect matching α and every involution β on each even vertex set through size eight; its fixed points determine the boundary. Thus it includes empty boundaries, no internal joins, overlapping old and new matching pairs, and two-edge cycles. The local-rule family takes the two chosen active agents, fixes their principal edge, varies the number of external free ports over 0, 2, 4, and enumerates all remaining perfect matchings. Its rule counts are

$$5 (\varepsilon\varepsilon), \quad 19 (\gamma\varepsilon), \quad 19 (\delta\varepsilon), \quad 123 (\gamma\gamma), \quad 123 (\delta\delta), \quad 123 (\gamma\delta).$$

Pre-existing loop count three is retained in these local tests.

The bounded uniqueness test deliberately includes the spurious root label zero. It enumerates all tuples compatible with the local minimum/reset conditions and distances through $n - 1$, then checks orbit consistency and distance equations. Exactly one remains for every tested permutation. This finite test is not the unbounded uniqueness proof; that proof is Theorem 146.3.

The complete recorded run took approximately four seconds in the authoring container. That timing is an observation about this run, not a performance guarantee. The receipt includes code hashes and a hash of the ordered gluing records.

148.4.4 Reproduction commands and archive contents

[write] *The listing keeps the delivered layout; manuscript 15's `article.tex` is not shipped, nor is its `data/verification.txt`, which is its `verification.json` with a two-line header. The provenance note and the claims ledger are shipped as `15-no-ghost-wires-PROVENANCE.md` and `15-no-ghost-wires-CLAIMS.md`. This report's `README` gives the recipe for the shipped names.*

From the extracted archive directory, run

```
python3 code/verify.py
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

The first command regenerates the exact JSON and readable receipts, the example quartic, its witness, and the endpoint descriptions. The three LaTeX passes settle references and the two-page table of contents. Standard TeX Live packages, including New PX and TikZ, are used; no bibliography processor or external image file is needed.

The principal generated data files are `data/15-no-ghost-wires-quartic_A_schedule.json`, `data/15-no-ghost-wires-witness_A.json`, `data/15-no-ghost-wires-endpoints.json`, and `data/15-no-ghost-wires-verification.json`. The archive also contains a provenance note, a compact mathematical claims ledger, and the standalone source code. No repository files are modified by these commands.

148.5 Integration and formal verification boundaries [15]

148.5.1 A narrow repository audit

The repository inspection was deliberately scoped. It read the Hilbert's-tenth-problem `README`, the concrete wiring-obstruction note, and the MRDP interface, in addition to the repository directory information. The observed tree identifier was

928ea97017a25ebe56d240c84f27d2275d818c75.

The obstruction note was returned with blob identifier

764c982297342093307a0e831bc0d9ed5084c87f.

The MRDP interface was read at the recorded tree revision. This is not a whole-repository correctness audit, and no Lean or Rocq build was run.

The present Python compiler is standalone: it neither imports a repository proof as an axiom nor claims that its new theorems are already formalized. A natural integration location would be alongside the existing `native-stream-queue` wiring-obstruction report, but the archive does not write to that location automatically.

148.5.2 A feasible proof-assistant decomposition

A formalization can keep the graph and arithmetic layers separate. Represent α and β as finite permutations with proofs of their involution properties, and represent the boundary as the fixed-point set of β . Prove the path–cycle dictionary and the pair-suppression identity first. Neither theorem depends on interaction combinators.

Next formalize the orbit certificate over natural-valued finite functions. The critical argument is a finite-orbit sum: absence of a root gives $\sum d_i = \sum d_i + |C|$. The minimum, slack, and distance uniqueness proofs then use only finite permutation reasoning and natural arithmetic. The selector version is a matrix-row selection lemma.

Finally define the six finite templates and a reflected compiler producing lists of multivariate-polynomial residuals. A correctness theorem for the compiler should connect its zeros to the exact net semantics, including loop counts. Testing the generated residuals alone is not a substitute for that connection. The source and receipt are suitable inputs for such a development, not an assertion that the development has already been completed.

[write] The “natural integration location” that manuscript 15 names, beside the research note in the Hilbert’s-tenth-problem tree, was not used: the three manuscripts are placed in this report, as Part XV, and the research tree is maintained separately.

149 Research questions and conclusions

[write] *The research questions of the three manuscripts, twenty-five in all: nine of manuscript 13 and eight of manuscript 15, which posed them as headings of unnumbered subsections, and eight of manuscript 14, which posed them as research questions. Each is one question environment titled by its heading and source, with its text unchanged. They are not merged with one another or with the questions of Section 71 and Parts X–XIV; a [write] line after a question names the questions it overlaps. Four topics recur in all three manuscripts: a complete exporter for the six-rule compiler, a canonical quotient of concurrent schedules (Part X’s question on quotienting independent schedules and Research questions 71.4 and 71.43), fixed-arity compression with a paid ordinary-integer loader (Research questions 71.48 and 71.49), and formalization (Research question 71.1). Manuscript 13’s question on low-height memory certificates and manuscript 14’s on the memory size–height cost are Part V’s size–height question (Research question 71.23) for their relations. The three conclusions follow.*

149.1 Further research questions and concrete next targets [13]

The following are proposed research problems emerging from the proved results. They are not all asserted to be previously named open problems in the literature. A priority search should accompany work on each one.

Research question 149.1 (Deterministic, efficiently decodable one-bit tomography; source 13). Theorem 142.6 proves the existence of an $O(n \log n)$ separating family, but does not construct or decode one in polynomial time. Find an explicit family of that size and a polynomial-time algorithm reconstructing M from its loop-parity vector. Proposition 142.5 provides local separators once two candidate matchings are known; turning that pairwise knowledge into a global nonadaptive decoder is a distinct problem.

A sharper numerical target is to minimize

$$\tau(n) = \min\{K : \exists Q_1, \dots, Q_K \text{ with } \Phi \text{ injective on } \mathcal{M}_n\}. \quad (149.1)$$

We proved $\log_2 D_n \leq \tau(n) = O(\log D_n)$. Does $\tau(n)/\log_2 D_n \rightarrow 1$, or is there a larger limiting obstruction imposed by the restricted family of allowable probes?

Research question 149.2 (Sharp separation probabilities modulo larger integers; source 13). Proposition 142.5 proves separation modulo every $r \geq 2$, but not a uniform sharp probability. Determine, for fixed r ,

$$\inf_{n, M \neq N} \mathbb{P}(c(M, Q) \not\equiv c(N, Q) \pmod{r}). \quad (149.2)$$

The signed recurrence can be generalized by replacing -1 with roots of unity and tracking the pair of loop increments. The extremal cycle types need not remain a single six-cycle. Small-rank exact enumeration provides a direct way to formulate testable conjectures.

Research question 149.3 (Error-correcting and restricted-context tomography; source 13). The $2/5$ lower bound suggests sampling probe families with positive relative Hamming distance between every two observation vectors. Quantify the best tradeoff between probe count, minimum distance and efficient reconstruction. Separately, determine what changes when contexts are planar, geometrically local, bounded-depth, or generated by a restricted rule system. Corollary 142.3 cannot be transferred to such a family without proving that its contexts still separate the relevant matchings.

Research question 149.4 (Asymptotics and spectra of signed closure correlations; source 13). The recurrence computes $F(\lambda)$ without enumerating all closing matchings. Study the asymptotic behavior of $h(\lambda)$ along families such as (k) , (2^r) and (k^r) , including sharp decay estimates and sign patterns. Relate the matrix $((-1)^{c(M, Q)})_{M, Q \in \mathcal{M}_n}$ and its row correlations to the matching association scheme and Brauer-algebra specializations. A useful outcome would be a closed spectral formula that recovers, and perhaps strengthens, the elementary extremal theorem without obscuring its equality cases.

Research question 149.5 (Low-height exact memory certificates; source 13). The present memory argument pays $O(S^2 \log S)$ bits in its largest witness. Can a deterministic, exact, uniquely extending memory certificate retain $O(S)$ arithmetic coordinates while substantially reducing this height? Possible approaches include structured multi-evaluation schemes or coefficientwise certificates with carefully balanced costs. A finite-field fingerprint with nonzero error is not a solution to the exact zero-set problem, and an unconstrained base is ruled out by the explicit collision example in Section 144.1.

[write] Compare Research question 71.23, Part V's size–height question, and manuscript 14's question on reducing the memory size–height cost below; each asks it for its own relation.

Research question 149.6 (A fully instrumented interaction-net polynomial exporter; source 13). The next implementation task is to turn the controller construction into a literal fixed instruction schedule, route every array access through the existing memory generator, and export a complete polynomial for each (C_0, F, T) . The important output is not just a degree bound: it should include a register ledger, branch conventions, raw/sorted record layouts, canonical witness generator, independent replay, and operation counts for each macro. Compare degree, coordinate count, sparse monomial count, arithmetic operations and witness height separately.

[write] Compare manuscript 14's question on finishing the full symbolic assembler and manuscript 15's on sparse certificates using the sixteen-port support bound, below. Manuscript 15's fixed-schedule exporter (Theorem 146.7) is implemented, but it is dense and does not route accesses through a memory certificate.

Research question 149.7 (A fixed-arity history encoding and an ordinary input loader; source 13). An unbounded universal equation through the six-rule substrate requires a new history-compression layer whose coordinate count is independent of T . The exact memory certificate is a sound bounded starting point, not that layer itself. A productive target is to combine it with the repository's existing history arithmetic while preserving enough port-incidence information to satisfy Corollary 142.3.

In parallel, specify one fixed universal net construction receiving an ordinary natural-number input through an explicit loader. Prove the simulation and its output predicate, including behavior on

malformed encodings. Only then can a counted comparison with the repository’s universal-polynomial constructions be made.

[write] Compare Research questions 71.48 and 71.49, and below manuscript 14’s questions on a fully charged universal input interface and on restricted, genuinely uniform compression, and manuscript 15’s on fixed-arity compression with a completely paid input loader.

Research question 149.8 (Schedule quotients and honest multiplicity statements; source 13). The compiler is canonical over a fixed schedule, but distinct legal schedules may represent the same concurrent computation. Investigate a canonical representative of commuting reductions and the arithmetic cost of checking it. Any resulting multiplicity theorem must distinguish the bounded trace, the schedule quotient and the auxiliary witnesses introduced by later unbounded Diophantine compilation. No conclusion about the general single-fold representation problem follows solely from local confluence.

[write] Compare Research questions 71.4 and 71.43, Part X’s question on quotienting independent schedules (Section 85), and below manuscript 14’s question on canonicalizing genuine concurrency and manuscript 15’s on witness-faithful quotienting by independent reductions.

Research question 149.9 (Formal proof and broader substrates; source 13). Formalize the sharp theorem first: it has a compact finite combinatorial core and an induction with explicit exceptional cases. The memory theorem is an independent second milestone. A successful combination would provide a reusable compiler for fixed bounded-arity port-rewriting systems, including variants of interaction nets with additional finite agent types. For substrates with priorities, global pattern tests or nonlocal rewrites, identify the exact extra relation requiring arithmetic certification rather than treating the entire “esoteric instruction” as a free primitive.

[write] Compare Research question 71.1, and below manuscript 14’s question on a formal verification boundary and manuscript 15’s on formalizing the whole semantic bridge. For substrates with timing, the neighbouring report *liveness-beyond-halting* asks for clock-faithful translations, among others into interaction nets; this Part does not supply one.

149.2 Further research questions and concrete next steps [14]

Research question 149.10 (Finish the full symbolic assembler; source 14). Implement the entire unknown-schedule assembler of Section 145.2, export a full quartic for the four-cell witness pair, and give measured values of G, H in (145.9). A useful acceptance test is that all accepted source schedules decode through an independent reducer to the same literal execution. Can common comparison outputs and reciprocal patches substantially reduce the constant?

[write] Compare manuscript 13’s question on a fully instrumented interaction-net polynomial exporter, above, and manuscript 15’s on sparse certificates, below.

Research question 149.11 (Reduce the memory size–height cost; source 14). Can the $17L - 4$ capped-memory construction be improved while retaining small coefficients, exact deterministic verification, and unique witnesses? In particular, is a linear-coordinate construction possible with substantially smaller than $O(L^2 \log M)$ maximum bit length for the product witnesses? A meaningful comparison should hold the source domain and every cap fixed.

[write] Compare Research question 71.23 and manuscript 13’s question on low-height exact memory certificates, above. The $17L - 4$ count is for a capped relation (Section 144.4).

Research question 149.12 (Canonicalize genuine concurrency; source 14). Can one obtain exactly one arithmetic zero per interaction-net execution class under swaps of independent reductions, while retaining loops and fresh-name normalization? The independent pairs are dynamically determined by wiring. A solution must distinguish a static commutation relation from residual-based independence and prove that its proposed normal form has no extra arithmetic witnesses.

[write] Compare Research questions 71.4 and 71.43, Part X’s question on quotienting independent schedules, manuscript 13’s question on schedule quotients, above, and manuscript 15’s on witness-faithful quotienting, below.

Research question 149.13 (Variable arity without a huge case table; source 14). For boundary size r supplied as part of the problem, replace the I_r -case construction by a polynomial-size canonical gluing certificate. The target outputs are the exact boundary matching and number of internally closed components. An arbitrary spanning forest is not single-fold; any rank, root, or traversal witnesses must be normalized. What size–height frontier is attainable for matching composition specifically?

Research question 149.14 (A fully charged universal input interface; source 14). Choose a specific universal interaction system and one precise source language, then construct and charge an ordinary-integer input map. Identify a terminal predicate whose relation to source halting is proved in both directions. The bounded explicit-net loader (145.17) is a building block, not the completed universal loader. Does this route compete with the repository’s direct arithmetic substrates after all topology costs are included?

[write] Compare Research question 71.48, manuscript 13’s question on a fixed-arity history encoding and an ordinary input loader, above, and manuscript 15’s on fixed-arity compression with a completely paid input loader, below.

Research question 149.15 (Observational targets versus literal targets; source 14). The current system is parsimonious for executions with structural names. Which useful output equivalences admit canonical certificates of comparable size? Candidates include fixed-interface normal forms and wire permutations. Unlabelled graph isomorphism and observational equivalence require separate analysis; neither can be made witness-unique by simply existentially guessing a relabelling.

[write] Compare Part X’s birth-order normalization (Theorem 78.1), which identifies names, not observations, and manuscript 15’s question on canonical renaming, below.

Research question 149.16 (A formal verification boundary; source 14). Formalize the partial-matching kernel, prove the carry-free product lemma, and certify the memory compiler in Lean. Then connect a proved 36-access machine to the exact involution-plus-loop semantics. The most valuable end-to-end statement is an explicit equivalence between natural zeros of the emitted polynomial and the chosen finite execution type, not merely existence of some Diophantine representation.

[write] Compare Research question 71.1, manuscript 13’s question on formal proof, above, and manuscript 15’s on the whole semantic bridge, below.

Research question 149.17 (Restricted, genuinely uniform compression; source 14). Identify nontrivial classes of interaction nets with compressible histories, for example fixed routing skeletons or periodic local rewrites, and seek fixed-arity certificates for those classes. State clearly whether uniqueness is preserved and why the restricted construction does not silently assume a solution to the general finite-fold problem.

[write] Compare Research question 71.49; Part XIV answers that question in part for routing networks, with fixed arity for one topology.

149.3 Further research questions and proposed projects [15]

Research question 149.18 (Sparse certificates using the sixteen-port support bound; source 15). The dense compiler stores entire intermediate matching matrices. Replace them by a canonical read/write log for partner pointers. Each elementary suppression reads two partners and modifies at most two surviving pointers; every rule uses at most four such suppressions. The support bound shows that topology itself does not require scanning the whole net. The main task is a witness-faithful memory-consistency checker, with every sorting, lookup, allocation, and untouched-memory cost included. A useful target is quasi-linear certificate size in the number of initial ports and interaction events, rather than in history length times the square of the live graph size.

[write] Compare manuscript 13's question on a fully instrumented exporter and manuscript 14's on the full symbolic assembler, above: the sparse certificate asked for here is in substance the paper construction of Theorem 145.3 and Remark 145.5, whose exporter is what those two questions ask for.

Research question 149.19 (Can variable-topology orbit certification use fewer coordinates?; source 15). The fixed-index certificate needs four coordinates per vertex; the selector certificate uses five to prevent a cubic distance expression. Determine whether an equally explicit unique-witness certificate can use four coordinates with quadratic residuals when the successor matrix is arithmetic data. A meaningful lower bound must specify the allowed equation schema and the input interface; no unrestricted lower bound is suggested by our ledger.

Research question 149.20 (Fixed-arity compression with a completely paid input loader; source 15). Use the loop-exact gluing module as the semantic front end of a packed-history construction. The target should state the full polynomial, the witness domain, the input encoding, the allocated-name policy, and the treatment of loops. A universality theorem for interaction combinators is not an input loader. Before optimizing operation counts, prove that invalid packed port incidences cannot masquerade as valid memory and that cycle counts remain exact.

[write] Compare Research question 71.48, and manuscript 13's and manuscript 14's questions on loaders, above.

Research question 149.21 (Witness-faithful quotienting by independent reductions; source 15). The current finite-disjunction construction has one zero per successful schedule, not one zero per concurrent trace class. Independent active pairs can be reduced in either order; their wiring suppressions commute by Corollary 141.5, up to the fresh-name convention. Construct a canonical representative of each appropriate concurrency class without deleting valid resource or loop information. A useful intermediate goal is to prove a precise local diamond with an explicit fresh-name bijection.

[write] Compare Research questions 71.4 and 71.43, Part X's question on quotienting independent schedules, and manuscript 13's and manuscript 14's questions on schedule quotients and concurrency, above.

Research question 149.22 (Canonical renaming without changing topology; source 15). Our single-fold theorem fixes the agent names. To obtain one witness per isomorphism class of traces, a canonical naming policy must account for ordered ports, agent types, free-port labels, and cyclic wires. Birth-order naming is natural once a schedule is fixed, but it is not automatically canonical across all schedules. Establish the exact equivalence relation first, then decide whether its normalization can be certified at low degree without introducing multiple witnesses for automorphisms.

[write] Compare Part X's birth-order normalization (Theorem 78.1), a canonical naming of created objects once the schedule is fixed, and manuscript 14's question on observational targets, above.

Research question 149.23 (Loop-sensitive semantics for diagrammatic computation; source 15). For ordinary matching contexts, Theorem 142.7 identifies a complete observable summary. Extend the arithmetic treatment to typed or coloured wires, oriented interfaces, and diagram categories in which a closed component has a type-dependent weight. Specify whether different loop types are counted separately or evaluated in a target algebra. The distinction matters for faithfulness: specializing all loop weights to one can erase information.

Research question 149.24 (Minimal local counterexamples to incorrect abstractions; source 15). The repository’s four- δ example is small and decisive. Classify which coarse summaries become sufficient on restricted families, such as acyclic wire-only contexts or bounded interface width, and find smallest counterexamples when they fail. A finite exhaustive search should be paired with an explicit mathematical explanation of the witness, as in Section 140.5, rather than reported solely as a hash or a solver verdict.

Research question 149.25 (Formalization of the whole semantic bridge; source 15). The most immediately actionable project is a Lean or Rocq formalization of Theorems 146.1, 146.3 and 146.7. The main payoff is not another abstract invocation of MRDP but a verified theorem that the exported polynomial recognizes exactly the intended port-labelled computation. The 126-variable example provides a compact regression case, while the four-step lasso exercises commutation, erasure, annihilation, allocation, and renaming in one trace.

[write] Compare Research question 71.1, and manuscript 13’s and manuscript 14’s questions on formalization, above.

149.4 Conclusion [13]

The concrete wiring gap leads in two directions. On the negative side, port incidence cannot be discarded by any summary that remains exact in all closing contexts: even final cyclic-wire parity distinguishes any two matchings in at least two fifths of contexts, and that constant is sharp. On the positive side, preserving exact incidence need not force a complete graph tableau at every moment. Constant-size splices and one exact global memory argument yield a linear-description, degree-four bounded history certificate, with canonical ancillary witnesses for each schedule.

The resulting boundary is precise. Exact topology, exact cyclic components and a horizon-sensitive arithmetic construction are proved. A smaller fixed universal polynomial, an ordinary-number loader with a paid ledger, and uniqueness through an unbounded MRDP conversion are not. The package supplies both the proofs and enough executable structure to make those remaining tasks concrete.

149.5 Conclusion [14]

The obstruction in the motivating repository note is not repaired by adding more global graph statistics. It is repaired by retaining the actual port involution and compiling its bounded local changes. A disappearing boundary can create a two-edge or four-edge cycle, and both must remain visible to arithmetic. The formulas (143.3)–(143.4) capture that distinction with a constant kernel; Theorem 143.2 identifies an exact local distinguishability threshold; and the capped-memory theorem controls all remaining access witnesses without a probabilistic permutation test.

The resulting quartic family represents complete bounded scheduled interaction-combinator computations, not just a coarse reachability shadow. Its semantic proof and linear asymptotic ledger are complete within the stated explicit-input, external-horizon scope. The next engineering task is to turn the specified full assembler into an audited emitter with an exact global constant ledger. The next mathematical tasks concern variable-arity gluing, genuine concurrency quotients, and justified restricted compression.

149.6 Conclusion [15]

The arithmetic obstacle was not the number of combinator symbols. It was the need to retain which port is connected to which, and to account for the closed wire components created by suppression. We have supplied two exact solutions to that local problem: a division-free quadratic elimination identity and a canonical permutation-orbit certificate. The latter fits directly into a single-fold quartic trace compiler with an explicit size ledger.

The construction preserves information lost by the repository's obstructed summary, treats all six rules rather than only the terminating all- δ fragment, and distinguishes a port-free cyclic component from the absence of a component. It supplies a finite-horizon topology-aware foundation on which more ambitious memory and history compression can safely be built.

150 A compact mathematical specification of the memory compiler [13]

[write] *Appendix A of manuscript 13. Its Appendix B, the source audit, is in the provenance appendix.*

For implementation or formalization, the entire memory certificate can be summarized without invoking the controller. The structural parameters are A, m, V ; the public fields are $(I_a), (O_a), (a_j, r_j, w_j)$. The construction is the following finite schema, with every displayed range instantiated externally.

First bound all public values and addresses by (144.15). Form the $L = 2A + m$ literal raw rows (144.6). Introduce four natural fields per sorted row and bound each by (144.9). For every adjacent pair introduce s_i, z_i, d_i and impose the five residuals (144.10)–(144.14).

Next construct $B = U^L + 1, U = AHV^2$, by binary powering. For each of the raw and sorted row lists, form affine leaves $B + HV^2a + V^2t + Vr + w$, combine them in a fixed balanced product tree, and equate the two roots. There is no need to add a variable for a leaf. Every intermediate product receives one natural variable and one defining residual.

All auxiliary choices have been specified: sort by the strictly ordered key, use natural difference slacks, force inactive address gaps to zero, and use the unique value of every gate. The sum of squared residuals is the single exported polynomial. Parameters and witnesses are distinguished in the JSON by a role field; monomials are sorted tuples of variable IDs, with integer coefficients.

The implementation intentionally permits an in-range but inconsistent proposed history to be passed to the constructor. It fills in the canonical sort and gates and returns the list of failed residuals. For such a history the adjacent memory-link equations expose the inconsistency. Values outside the structural ranges are rejected before constructing an example tuple; the polynomial equations themselves enforce those ranges for arbitrary candidate tuples.

151 Literal functional gadgets [14]

[write] *Appendix A of manuscript 14. Manuscript 13's comparison gadget is (145.10); the two gadgets differ in their equations and both have unique natural extensions.*

151.1 Comparison with no unspecified witnesses

For source $x, y \in \mathbb{N}$, introduce $E, L, G, h_L, h_G \in \mathbb{N}$ and require

$$\begin{aligned} E + L + G &= 1, \\ x + L(1 + h_L) &= y + G(1 + h_G), \\ (1 - L)h_L &= 0, \\ (1 - G)h_G &= 0. \end{aligned}$$

The first equation makes E, L, G one-hot because they are natural numbers. If $x = y$, the unique solution has $E = 1$ and all other fields zero. If $x < y$, it has $L = 1$, $h_L = y - x - 1$, and the other fields zero. The $x > y$ case is symmetric. Thus five auxiliaries and four quadratic residuals implement an exact three-way comparison. Equality indicators and strict-order bits are outputs of this total gadget, not free Boolean guesses.

151.2 Range and division

For fixed $D \geq 1$, $x < D$ is represented by the unique natural complement $x + s = D - 1$. Division uses

$$x = Dq + r, \quad r + s = D - 1.$$

It has exactly one natural triple (q, r, s) for every $x \in \mathbb{N}$. The same uniqueness holds when D is a positive source variable, with a quadratic first residual. The interaction-net address decoder needs only fixed division by three and affine range changes.

151.3 Boolean circuits and selection

Once b, c are Boolean, $z = bc$ implements AND and $z = 1 - b$ implements NOT. A selected scalar value is forced by

$$z = bx + (1 - b)y.$$

Every product receives its own variable when substitution would exceed degree two. A finite truth table is compiled into a Boolean circuit in the usual explicit way: AND the literals of each input pattern and OR the corresponding disjoint cases. Its gate values are all unique.

For the ten-state kernel one may instead directly use (143.6)–(143.10). These relations already enforce the legal input patterns and have a unique extension. In a larger conditional controller, set an inactive kernel's source to a fixed legal pattern before invoking it. Its outputs are then fixed and may be ignored by deterministic selection.

151.4 Why guarding alone is unsafe

The equation $b R(\mathbf{z}) = 0$ does not constrain $\mathbf{z}$ when $b = 0$. Moreover, multiplying a quadratic residual by b creates a cubic residual, whose square has degree six. Neither issue is solved by calling the branch “inactive.” Use total functional evaluation, fixed dummy inputs, or a properly quadratized and normalized branch construction. Complements of inactive zero inputs need not be zero: for instance the complement of zero under $z + s = D - 1$ is $D - 1$. The requirement is uniqueness, not an unjustified blanket zeroing rule.

152 Assembler specification and trust boundaries [14]

[write] Appendix B of manuscript 14. The pseudocode specifies the paper construction of Theorem 145.3; it is not a shipped program.

The following pseudocode specifies the full mathematical compiler. It is separate from the delivered executable kernel emitters.

```
compile_exact_trace(valid_net N, external_horizon T, terminal_query Q):
  reserve  $C0 + 4 * T$  cell records and  $f$  fixed free-port records
  choose  $A = 4 * C + f + 1$  and  $V = 3 * C + f + 4$ 
  create source variables for unordered schedule pairs  $u[t] < v[t]$ 
  emit fixed initialization log records
  for  $t = 0, \dots, T-1$ :
    emit eight read records for selected cell records
    compile active-pair checks and unique type/rule selection
    derive exact boundary comparisons from the read mate values
    compile annihilation kernel or deterministic endpoint relocation
    assign fresh identifiers in the declared template-role order
    emit eight zero writes, sixteen allocation slots, four patch slots
    replace each unused slot by the fixed dummy read  $(0,0,0)$ 
    emit scalar fresh/live/loop updates
  emit terminal-query checks and any required final scan
  attach the capped-memory system to the entire fixed-length log
  retain every intermediate product as a separate functional node
  return the sum of squares of all quadratic residuals
```

The initial validity condition can either be checked by the effective compiler, producing the constant polynomial 1 for malformed fixed inputs, or be arithmetized using Section 145.5 when the input table is a source tuple. The theorem does not assume a malformed input magically represents a net. The numbered local templates are finite constants of the six-rule system. The only dependence on surrounding graph topology enters through the actual memory reads of port partners and their exact comparisons.

Layer	What is supplied
Exact net semantics	Definition, six numbered templates, and two Python reducers.
Annihilation arithmetic	Complete equations, proof, sparse emitter, expanded quartics, valid witnesses.
Capped-memory arithmetic	Complete equations, exact ledger, proof, sparse emitter, expanded example.
Full bounded IC assembler	Constructive algorithm and soundness/completeness/uniqueness proof; no full symbolic emitter.
Formal proof assistant	Not supplied.
Unbounded fixed-arity compression	Not supplied or claimed.

153 Package contents and replay [14]

[write] Appendix C of manuscript 14. The listing keeps the delivered layout, in which the programs sit in `code/` and write into `data/`; they are shipped as `code/14-net-topology-*.py` and write by default into `data/` beside their own directory, where the shipped outputs carry prefixes. The README of this report gives a recipe on a copy. The delivered README that the paragraph mentions is not shipped; its sparse-polynomial convention is the one stated here. Its Appendix D, the source audit, is in the provenance appendix.

Run with a standard Python 3 installation:

```
python code/verify.py
python code/verify_exports.py
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

The Python code uses no external libraries. It regenerates the three expanded quartic JSON files and `data/14-net-topology-verification.json`. Running the verifier is a finite check, not a Diophantine solver. The sparse polynomial convention is explained in the README: a monomial is a sorted list of variable names with repetition, and an empty list denotes a constant.

The four- δ replay record hash is

```
4fd5e7ed594006facddceadfb753cf3c
355425452db33bf3ca35490aa2541d89
```

where the line break is typographical. This is a SHA-256 digest of the exact input/normal-form serialization described in the verifier, not a repository source hash. Code hashes and all test counts are stored in the JSON receipt.

154 The complete residual schema in one place [15]

[write] *Appendix A of manuscript 15.*

For clarity, here is the entire scheduled compiler as an equation list. All indices, vertex sets, rule templates, and the row permutation β_t are fixed by the type/lifetime ledger. Symmetric matching entries share a variable; diagonal entries are zero.

For configurations $0 \leq t \leq T$:

$$\sum_{v \neq u} W_{uv}^{(t)} - 1 = 0 \quad (u \text{ a live or free port}).$$

For steps $0 \leq t < T$:

$$W_{(a_t,0),(b_t,0)}^{(t)} - 1 = 0.$$

For every temporary vertex $i \in [N_t]$:

$$\begin{aligned} e_{t,i}(e_{t,i} - 1) &= 0, \\ r_{t,i} + (1 - e_{t,i})(h_{t,i} + 1) - i &= 0, \\ e_{t,i}h_{t,i} &= 0, \\ r_{t,i} - \sum_j A_{\beta_t(i),j}^{(t)} r_{t,j} &= 0, \\ s_{t,i} - \sum_j A_{\beta_t(i),j}^{(t)} d_{t,j} &= 0, \\ d_{t,i} - (1 - e_{t,i})(s_{t,i} + 1) &= 0. \end{aligned}$$

For every unordered pair of distinct surviving ports u, v :

$$W_{uv}^{(t+1)}(r_{t,u} - r_{t,v}) = 0.$$

Finally, for every step:

$$\sum_i e_{t,i} + 2\ell_t - 2\ell_{t+1} - P_{t+1}/2 = 0.$$

Here $r_{t,u}$ means the coordinate of the temporary index assigned to global port u . The total certificate is the sum of the squares of every displayed residual. No additional existential helper, divisibility condition, unbounded quantifier, or implicit array operation is required.

155 Small omissions that would invalidate the result [15]

[write] *Appendix B of manuscript 15.*

Omitting root-slack normalization. Without $e_i h_i = 0$, the root's value of h_i is completely free. The representation remains existentially correct but loses uniqueness, with infinitely many witnesses even for a one-vertex permutation.

Omitting the distance condition. Setting $e_i = 0$, $r_i = 0$, and $h_i = i - 1$ satisfies the Boolean, minimum, slack, and equal-root equations for any permutation. Thus component labels can all be fictitious unless an equation forces the existence of an actual root. The distance recurrence provides that forcing by contradiction around an orbit.

Treating the temporary graph as simple. An old edge that coincides with a join is not one edge: it is two parallel edges belonging to different matchings. Replacing the multigraph by a simple graph destroys precisely the elementary loop counted by (141.7).

Dropping the output matching constraints. Equality of root labels for proposed output edges is insufficient if the output is allowed to omit edges or use a port more than once. The natural row-sum equations ensure a genuine perfect matching before the orbit criterion is applied.

Confusing a removed principal edge with a suppressed wire cycle. The principal–principal edge is deleted as part of the interaction rule. Only the separate interface gluing is normalized by $\alpha\beta$. Applying the pair-suppression loop increment to the principal pair itself would already misinterpret $\varepsilon\varepsilon$.

Claiming an unbounded single-fold theorem from finite schedules. The dimension in (146.17) grows with the schedule. A collection of uniquely witnessed equations, one for each length, is not a single fixed-arity Diophantine equation with the same property.

156 Data format and verification of the exported polynomial [15]

[write] *Appendix C of manuscript 15. The listing reads `data/quartic_A_schedule.json` and `data/witness_A.json` in the delivered layout; they are shipped as `data/15-no-ghost-wires-quartic_A_schedule.json` and `data/15-no-ghost-wires-witness_A.json`, and `code/15-no-ghost-wires-check_export.py` accepts `-data` for the directory but not the prefixed names, so it is run on a copy (this report's README).*

A residual is stored as a list of pairs consisting of an integer coefficient and a list of variable names. Repeated names encode powers, and an empty name list encodes the constant monomial. For example,

```
[[1, ["t0.e[1]", "t0.e[1]"]],
 [-1, ["t0.e[1]"]]]
```

represents $e_{0,1}^2 - e_{0,1}$. The file separately lists endpoint parameters and existential witnesses. An assignment must provide exactly these names and must assign every one a nonnegative Python integer. Booleans are not accepted as integers by the assignment validator.

For the example, the following code independently evaluates the exported residual list without using the compiler's polynomial class:

```
import json
from pathlib import Path
root = Path("data")
spec = json.loads((root / "quartic_A_schedule.json").read_text())
values = json.loads((root / "witness_A.json").read_text())
assert set(values) == set(spec["parameters"] + spec["witnesses"])
assert all(type(v) is int and v >= 0 for v in values.values())
energy = 0
for residual in spec["residuals"]:
    total = 0
    for coefficient, monomial in residual["terms"]:
        term = coefficient
        for name in monomial:
            term *= values[name]
        total += term
    energy += total * total
assert energy == 0
print(len(spec["witnesses"]), len(spec["residuals"]), energy)
# 126 169 0
```

The archive also supplies this independent evaluation as `code/15-no-ghost-wires-check_export.py`, with explicit input validation. The article's soundness theorem explains what this zero means. The evaluation alone certifies the arithmetic identity for the saved tuple, not the correctness of an arbitrary compiler.

Appendices

[write] *The appendices of manuscripts 05, 01, 02, 03, 04 and 06, in that order, followed by this report's provenance record. Manuscript 04's first appendix and manuscript 06's second appendix are printed in Section 69 and Section 70; manuscript 07 has no appendix.*

A Manuscript 05: A minimal end-to-end example

The following program can be run from the package directory. Python indices start at zero, whereas the mathematical exposition numbers labels and places from one. The three ordered labels remain $a < b < c$.

```
import sys
sys.path.insert(0, "code")
from trace_polytope import Net, compile_certificate

net = Net(pre=((0,), (0,), (1,)),
          post=((1,), (0,), (0,)),
          names=("a", "b", "c"))
cert = compile_certificate(net, (1,), (1,), 3)

# b,c,a is the canonical representative of c,a,b.
```

```

word = (1, 2, 0)
z = cert.witness(word)
assert cert.zero(z)
assert cert.energy(z) == 0
assert cert.decode(z) == word
cert.export("certificate.json")
print(len(cert.variables), len(cert.residuals)) # 73, 59

# Make b a zero-test instead of an unconditional no-op.
# The compiler recomputes the guarded commutation relation.
guarded = compile_certificate(
    net, (0,), (0,), 3,
    upper=((None,), (0,), (None,)))
z2 = guarded.witness((0, 2, 1)) # increment, decrement, zero-test
assert guarded.zero(z2)

```

The exported equation is exact even without monomial expansion. To expand a small instance, use `cert.expanded_polynomial()`; for large instances, retaining the sum-of-squares circuit is preferable. Expansion is not needed to evaluate the equation or to verify a proposed zero.

B Manuscript 05: Edge cases and specification checklist

Zero horizon. There are no selectors, remainders, or step-local auxiliaries. The endpoint equations require $M = N$, and the flags are forced to zero. There is exactly one empty execution class when the endpoints agree and no solution otherwise.

No places. Markings have the unique empty tuple as value. The net has only labelled identity transitions. The selector and normal-form constraints still distinguish the intended words and their classes. The full-box test uses this legitimate degenerate case to obtain a genuinely exhaustive small search.

Repeated effects and repeated circuit inputs. Two labels with equal transition data remain distinct actions and can commute. Circuit gates that mention one predecessor twice must read one token once; otherwise a supposedly 1-safe construction would demand two tokens and fail.

Empty interval compositions. Both orders may have empty domains. They then agree as partial maps, even when their numerical composed-box endpoints differ. The guarded commutation routine canonicalizes an empty box before comparing it. Empty *one-step* domains are excluded in the stated input convention.

Inactive guards and branches. An inactive upper guard has the bound B_p , not an unconstrained witness. An unused branch of a logical disjunction must not leave arbitrary variables. Every variable listed in the exported certificate has a unique semantic value once the canonical word is fixed.

The domain is part of the theorem. The identity $\sum_a e_a = 1$ forces a one-hot selector over $\mathbb{N}$, but not over $\mathbb{Z}$ or $\mathbb{R}$. Real solutions are deliberately retained only as a relaxation. Natural nonnegativity must not be erased when transferring the polynomial to another system.

Representation versus decision. The program constructs and evaluates equations. It does not enumerate all natural zeros of a large certificate. The explicit height theorem makes the set finite, but the associated search and count can still be computationally hard.

C Manuscript 05: Source and verification provenance

The repository sources inspected for the baseline were its Hilbert-tenth-problem README, formalization status register, and the actual `DiophantineTrace.lean` interface. The latter was retrieved at the pinned commit listed in Section 1; its Git blob identifier was

`08e5796b22fc85ef0cd97255dba8b7527960160b.`

Repository citations below use immutable commit URLs. The literature was checked through the authors’ papers, publisher pages, or the ACL Anthology. The bibliography is selective: it documents the reused conceptual background and the finite-fold boundary, not a claim to an exhaustive priority survey.

The package’s `README.md` describes its contents and limitations. Its `SHA256SUMS.txt` records the distributed files, including the article source, compiled PDF, code, and test receipts. The verification receipt reports the precise finite scopes; it should not be read as a formal proof certificate. [write] In `ProveIt` the delivered `README.md` is replaced by this report’s README, the checksum ledger was verified (19 of 19 entries) and retired, and the PDF is not shipped (Appendix K).

D Manuscript 01: Reproducibility and source-audit notes

The package contains the self-contained `article.tex`, its compiled PDF, the reference compiler and test runner, machine-readable verification output, and two expanded polynomial examples. To rebuild the document from a clean directory, run `pdflatex article.tex` three times to stabilize the contents and cross-references. The bibliography is included directly; no external bibliography database or source checkout is required.

The JSON polynomial format lists each monomial as an integer coefficient and a list of variable names, with repetition recording powers. The declared single polynomial is the sum of the squares of the listed residuals. Exported examples also contain its expanded form. Parameters and witnesses are separately listed, and every assignment entry is an exact integer. These conventions make the files inspectable without a computer algebra system.

The repository audit was limited to relevant live files at the pinned commit: the Hilbert-tenth README, its formalization status register, the actual exact-iteration interface, and the combinatory-logic README. A full-repository novelty audit was not performed. The code and mathematical proofs in this report were not committed to, or used to modify, the user’s repository. Literature was checked through the primary sources below; no conclusion of world-first priority is drawn from the search. [write] In `ProveIt` the package’s `article.tex` and PDF are not shipped separately (they survive in the arrival commit, Appendix K); its programs and data are now committed as files of this report under the names listed in the README, and no other repository file was changed.

E Manuscript 02: The finite checking formula in one place

For readers implementing a separate compiler, the following is a complete specification of the primary checking relation. The parameters are d fixed, $c \in \mathbb{Z}^{d+1}$, and $T \in \mathbb{N}$. Compute the coefficient tower by (120). At level k , use $L_k = 2(d - k) + 1$ slots, fixed cuts $b_{k,0} = 0$, $b_{k,L_k} = T + 1$, and natural primary coordinates for all remaining cuts and all tags.

Impose ordered cuts, tags in $\{0, 1, 2\}$, terminal zero-tag padding, and different tags in adjacent active slots. Impose $\tau_{d,0} = 1 + \text{sgn } q_d(0)$. For every $k < d$, parent slot j , and child slot ℓ , let

$$a = b_{k,j}, \quad b = b_{k,j+1}, \quad c = b_{k+1,\ell}, \quad e = b_{k+1,\ell+1},$$

$$u = \max(a, c), \quad v = \min(b - 1, e), \quad E = (a < b) \wedge (c < e) \wedge (u \leq v).$$

Require both implications

$$E \implies 1 + \text{sgn } q_k(u) = \tau_{k,j}, \quad E \implies 1 + \text{sgn } q_k(v) = \tau_{k,j}.$$

For a positivity certificate, also require $(b_{0,j} < b_{0,j+1}) \implies \tau_{0,j} \neq 0$ for every j . Omitting only these last assertions gives the unique sign-table certificate for *every* polynomial, including those negative somewhere in the interval.

Every comparison is evaluated, even when E is false. To preserve the cubic bound, evaluate and cache q_k at all candidate expressions $a, c, b - 1, e$ before selecting their sign tags. Use canonical sign bits for the selections and the intersection comparison. The signed values $b - 1$ are ordinary integer expressions; truncated natural subtraction must not be substituted into the checking formula.

F Manuscript 02: Reproduction and the exported polynomial format

The code requires Python 3.10 or later and uses only its standard library. Run the following commands from the extracted directory [write] (the delivered layout; see the README for a copy of the shipped files in that layout):

```
python run_tests.py
python certificates.py --coefficients 8 -6 1 --horizon 6
python quartic_compiler.py --coefficients 4 -4 1 --horizon 6 \
  --output quartic_example.json
python verify_export.py quartic_example.json
```

The coefficient order is ascending: $8 -6 1$ means $8 - 6t + t^2$. The optional compiler switch `-sign-table-only` omits the final positivity requirements while retaining all sign-table conditions.

An exported residual consists of terms with an integer coefficient and a list of variable names. For example,

```
{"coefficient": -2, "variables": ["b0_1", "w17"]}
```

means $-2b_{0,1}w_{17}$. The empty variable list denotes a constant. Repeated names denote powers; each residual has total degree at most two. The exact polynomial is the sum of the squares of the listed residuals. Free coefficient parameters are signed, the free horizon is natural, and all primary and auxiliary witness coordinates are natural.

The file includes a sample assignment but is not specialized to it. Changing an input coefficient in that assignment does not change the polynomial; it normally requires regenerating the unique witness. The structure fingerprints in the test report confirm equality of the exported symbolic template across the tested parameter assignments at fixed nominal degree.

Compile this article with

```
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

No external bibliography processor or shell escape is required.

G Manuscript 03: Additional validation of the linear backend

The independent driver code `/03-linear-memory-test_linear_memory.py` runs both algebraic and operational checks. It verifies collision-free product fingerprints for 451 bounded multisets, including zeros and repetitions. It repeats all 585 small logs, checks six non-Boolean write flags, and tests 40 valid larger-integer logs together with 37 corrupted reads. All agree with the independent chronological memory semantics. A further adversarial test replaces one sorted write record by a duplicate and recomputes every dependent output-side coordinate. All local bounds, packing, ordering, and memory checks still pass; exactly the final product-equality residual rejects the forged multiset.

For the four-event example, the linear backend emits exactly 96 witnesses and 89 residuals. Its expanded quartic has 499 nonzero monomials. The largest witness has 471 bits, compared with six bits for the bitonic backend on the same log; here $D = 164$. All 96 single-coordinate increments of the witness are rejected. Twenty additional assignments verify the expanded-polynomial identity, and compiling two different logs of the same length produces identical symbolic variable lists and residual polynomials.

These checks are recorded in `data/03-linear-memory-linear_test_results.json`; the complete polynomial and assignment are in `data/03-linear-memory-linear_example-*`.

Run

```
python3 code/test_linear_memory.py
python3 code/linear_memory.py artifacts/example/events.json \
    artifacts/rebuilt_linear_example --expanded
```

As with the comparison backend, the proof of uniqueness for arbitrary inputs is mathematical, not inferred from the bounded test corpus.

H Manuscript 03: Claim and dependency ledger

Claim	Dependencies	Status in this archive
Canonical comparison	Integer algebra and nonnegative witnesses	Proved; implemented; tested.
Low-height memory	Comparison, bitonic sorting, local memory criterion	Proved; implemented; tested.
Optimal network size	External AKS sorting-network theorem	Proved conditional on that established theorem; not implemented.
Carry-free product lemma	Base expansion and cancellation in $\mathbb{Z}[X]$	Proved; finite cases tested.
Linear-size memory	Product lemma, radix packing, local memory criterion	Proved; implemented; tested.
Parsimonious RAM	Canonical local instruction interface and either memory backend	Proved; complete frontend not implemented.
Immutable graph evaluator	Acyclic unfolding and normalized RAM implementation	Mathematical construction and proof; no frontend implementation.

Claim	Dependencies	Status in this archive
Counting and weights	RAM bijection and elementary counting/probability	Proved; weight example tested.
Universal compression equivalence	Universal computation and specialization	Proved elementary equivalence; neither conjecture solved.
Lean certification of this work	Proposed modules in Section 70.4	Not performed.
Literature-wide novelty	Additional independent specialist review	Not established.

I Manuscript 04: Notation and resource-count reference

Symbol	Meaning
$\mathbb{N}, \mathbb{Z}$	Natural witness domain including zero; integer polynomial coefficient and evaluation domain.
E, V	Number of displayed residuals; number of natural witness variables. Neither is claimed minimal.
I	Fixed symmetric irreflexive independence relation on the labeled action alphabet.
$\text{NF}_I(w)$	Lexicographically least word in the class generated by independent adjacent swaps.
p, r, T	Petri places, labeled actions, and externally fixed horizon; in FRACTRAN, r is the number of ordered reduced fractions.
U, V (net)	Consumption and production matrices. The production matrix V is unrelated to the scalar variable count when its indices are displayed.
m, x, e, f	Petri marking, action-selector, enabling-slack, and normal-form-flag variables.
q, ρ, s, u, z	Canonical quotient, remainder, bound slack, positive-remainder slack, and divisibility bit.
h, x (priority)	No-earlier-applicable-index bit and first-applicable selector.
$\text{App}(\ell, r)$	Natural code for binary application; leaves S, K, I have codes 0, 1, 2.
σ, H, n_I, n_K, n_S	Fixed SKI reduction schedule, its total context depth, and counts of each root rule.
P_T vs. $P(T, \cdot)$	A horizon-indexed polynomial family may have growing arity; one polynomial with horizon as an input has fixed arity.

J Manuscript 06: A completely displayed local power polynomial

To make the ordinary-polynomial nature of the encoding visible without any compiler machinery, consider one resource-summary pair (u, v) , an exponent k , and proposed powered coordinates (U, V) . Introduce $c, \alpha, \beta, e, \ell \in \mathbb{N}$ and set

$$\begin{aligned} \mathcal{P} = & (c + \alpha - u)^2 + (c + \beta - v)^2 + (\alpha\beta)^2 + (e(e - 1))^2 \\ & + (k - e - \ell)^2 + ((1 - e)\ell)^2 + (U - ec - k\alpha)^2 + (V - ec - k\beta)^2. \end{aligned} \quad (245)$$

This is a polynomial of total degree at most four in $u, v, k, U, V, c, \alpha, \beta, e, \ell$. For fixed u, v, k , its natural solutions in all seven remaining coordinates $(U, V, c, \alpha, \beta, e, \ell)$ form a singleton. The first three residuals force $c = \min(u, v)$, $\alpha = u - c$, $\beta = v - c$. The next three force $e = \mathbf{1}_{k>0}$ and $\ell = k - e$. The final two give exactly the powered summary. Thus the uniqueness claim includes both the result and its internal witnesses.

For $u = v = 1$ and $k = 0$, the forced values are

$$c = 1, \quad \alpha = \beta = e = \ell = 0, \quad U = V = 0.$$

The internal minimum remains one, but the powered action is the identity. For the same pair and any $k > 0$, the forced values are $e = 1$, $\ell = k - 1$, $U = V = 1$. This small example shows simultaneously why inactive internal witnesses must remain fixed and why their values must not be confused with the resources required by a zero power.

K Provenance of this report

[write] This report was built from fifteen manuscripts. Six arrived in batch 60 of ProveIt’s incoming reports (arrival commits 725d2ebb6 and c1f56f842) and were placed by commit 7498484af; the seventh arrived alone in batch 61 (b998f70c6) and was placed by 3bf66a8bc as the report’s seventh member, before the report was written. Four more arrived in batch 62 (arrival commit b57c0b5ff), after the report had been written, and were placed by adeddcecc as additions 08–11; they form Parts X–XIII. A twelfth arrived in batch 63 (arrival commit a4268e78e), after Parts X–XIII had been written, and was placed by 62f1ad07c as addition 12; it forms Part XIV. Three more arrived together in batch 78 (arrival commit 1977e6ea6), after Part XIV had been written, and were placed by aa11f3fef as additions 13–15; they form Part XV. Each manuscript’s source and delivery README survive only in its arrival commit, except manuscript 05’s source, which is the base of this article; no manuscript PDF is shipped.

No.	Archive (batch, manuscript)	Arrival	Pin	Placed	Contribution (Parts)
01	Diophantine_Causal_Computation.zip (60, 01)	725d2ebb6	e8bb0931d	7498484af	box test and resource envelope (I); causal-height quartic (II); accelerator, block programs, word powers, examples (III); substrate adapters (VII, VIII); boundary (IX)
02	canonical_diophantine_certificates.zip (60, 02)	725d2ebb6	e8bb0931d	7498484af	signed coordinates and total circuits (conventions); polynomial trajectories (IV); FRACTRAN step (VII); substrates (VIII); exponentiation boundary (IX)
03	ProveIt_Linear_Size_Diophantine_Certificates.zip (60, 03)	725d2ebb6	e8bb0931d	7498484af	memory logs, RAM, graph evaluation, counting (V); boundary (IX)
04	Witness_Faithful_Diophantine_Compilation.zip (60, 04)	c1f56f842	e8bb0931d	7498484af	monitor, Petri quartic, counting (II, second route); FRACTRAN and SKI (VII); substrates (VIII); boundary (IX)
05	canonical_trace_polytopes.zip (60, 05), base	c1f56f842	e8bb0931d	7498484af	introduction; commutation (I); trace polytopes, costs, counting (II); rewriting (VII); substrates (VIII); boundary (IX)
06	diophantine_counter_programs.zip (60, 06)	c1f56f842	e8bb0931d	7498484af	resource summaries (I); signatures and trace bijection (II, second route); saturation, compiler, nesting boundary (III); substrates (VIII); finite-fold substrate equivalence (IX)
07	Causal_Diophantine_Queue_Compilation.zip (61, 01)	b998f70c6	725d2ebb6	3bf66a8bc	first-failure product (I); queues, tag systems, word memory, FIFO networks (VI); boundary (IX)
08	Names_to_Numbers_Diophantine_Heaps.zip (62, 01)	b57c0b5ff	4e128356d	adeddcecc	memory logs, fresh names, heaps, pointer-only universality, graph transfer, safety boundary (X)
09	order_free_diophantine.zip (62, 02)	b57c0b5ff	4e128356d	adeddcecc	degree classification under orthant nonnegativity; order-free tile-assembly compiler (XI)
10	ProveIt_Priority_Pumping_Diophantine.zip (62, 04)	b57c0b5ff	4e128356d	adeddcecc	priority capacity, sharp pumping, program-uniform quartic, periodic tails, affine macros (XII)
11	one_shared_fallback_research.zip (62, 05)	b57c0b5ff	4e128356d ccfb084ad	adeddcecc	one-fallback reaction compiler, reservoir threshold, trace and minimum-fuel quartics, canonical-fuel equivalence (XIII); names a later commit besides its pin

No.	Archive (batch, manuscript)	Arrival	Pin	Placed	Contribution (Parts)
12	Diophantine_Certificates_Without_Histories.zip (63, 02)	a4268e78e	e18718e83	62f1ad07c	exact odometers by balance and last exits, run-length and grammar quartics, both-outcome certificates, counting barrier, universal one-router system, rank-function bottleneck (XIV)
13	Exact_Wiring_Diophantine_Report.zip (78, 01)	1977e6ea6	439c0a2d9	aa11f3fef	six-rule table, splice and gluing, sharp loop-parity separation, tomography, memory certificate with initial and final arrays, compiler (second presentation), MRDP interface (XV)
14	Topology_Is_Not_Free_Interaction_Nets.zip (78, 04)	1977e6ea6	6914ccca6	aa11f3fef	locality, ten-pattern kernel and minimality, $17L - 4$ memory, 36-access machine, compiler statement and ledger, bounded loader, transfer (XV)
15	no_ghost_wires.zip (78, 05)	1977e6ea6	928ea9701	aa11f3fef	matrix suppression, support bound, path-cycle dictionary, contextual completeness, canonical orbit certificate, implemented fixed-schedule compiler, disjunction, lasso, examples (XV)

Where the merge had to choose. [write]

- *Base.* Manuscript 05, whose trace-class theorem needs the weakest hypothesis (any sound $I \subseteq I_{\max}$, with interval guards) and gives the strongest conclusion (a convex quadratic whose real zero set is a rational polytope with exactly the natural zeros as lattice points). Its text supplies the title page layout, the preamble, the introduction and the first place of every shared result.
- *Printed once.* The universal-halting equivalence (manuscripts 05, 01, 03, 04) is Theorem 62.1; the absence of a computable witness-height bound (02, 01, 03, 04, 05) is Proposition 62.2. The labels of the four equivalence theorems and the five height statements (four propositions and manuscript 04's theorem) that they replace point to the merged statements, and each manuscript's proof is printed in its own section after a note mapping its letters.
- *Second routes.* Manuscripts 04's and 06's trace-class bijections (Theorems 15.2 and 18.1), manuscript 01's box test (Proposition 5.1) and manuscript 06's commutation criterion (Theorem 7.7) are kept with their own statements, counts and proofs, marked in their titles; manuscript 04's monitor, lower-bound family and counting theorem, and the Parikh refinements of manuscripts 01, 04 and 06, likewise point to manuscript 05's versions. Manuscript 01's height-bounded compiler is a different theorem and is kept whole.
- *Moved text.* Manuscript 05's interval-guard theorem and guarded-program corollary were moved from its section on other substrates to Parts I and II, and its rewriting subsection to Part VII; the substrate surveys of manuscripts 01, 02 and 05 are split between Parts VII, VIII and IX; manuscript 02's first two introductory subsections open Part IV and its Section 2 stands in the conventions; manuscript 07's word-code subsection opens Part VI and its first-failure section closes Part I. Every move is announced by a note at both ends.
- *Back matter.* Implementation and validation, formalization proposals and conclusions are collected by manuscript (the conclusions of the single-manuscript Parts IV–VI close those Parts). The research questions of all seven manuscripts (74) are merged into 50 questions; each merged question names its sources and prints each source's text (Section 71).

- *Bibliography.* One list; duplicates merged, each item naming the manuscripts that cite it. Manuscript 07 cites a different Cantone–Cuzziol–Omodeo article ([11]) from the one cited by manuscripts 02, 04, 05 and 06 ([10]); they are kept apart. Manuscript 07’s two unpublished “companion” items are manuscripts 04 and 06 themselves; its sentence citing them now names them.
- *Corrections.* Manuscript 06 cited Jones–Matiyasevich for ProveIt’s documentation of MRDP and its interfaces; it now cites the repository. Manuscript 04’s hard-coded section numbers (“Section 2”, “Section 3”, “Section 7”) now point to its sections here. Sentences in which manuscripts 01 and 05 describe their own delivered packages carry notes on what is and is not shipped.
- *Batch 62 (manuscripts 08–11).* Each manuscript is appended as one Part (X–XIII) after Part IX, in its own order, so that no existing Part, section, theorem, question or equation number changes; forward pointers in Parts II–V, VII and IX are unnumbered [write] paragraphs. Where a batch-62 result duplicates one above, both are printed because the variants differ: manuscript 08’s memory compiler beside manuscript 03’s, with an exact comparison after it; manuscripts 09’s and 11’s no-bound statements and manuscript 11’s canonical-fuel equivalence with their own proofs, each preceded by a note naming the Part IX statement it specializes (Proposition 62.2 and Theorem 62.1). Manuscript 09’s three parts are sections of Part XI. The introductions of 08–11 are in Section 3; their validation, formalization plans, questions and conclusions stay in their Parts, and their 39 questions are cross-referenced to, not merged with, the questions above (Section 71). One symbol is renamed (09’s tile palette T is $\mathcal{T}$) and one operator gains a subscript (08’s NF_{birth}); the equations of Parts X–XIII are numbered within sections. Corrections: 11’s report of a later live-branch read is recast as a pin statement, and its unpinned link to ProveIt’s MRDP guide is pinned in the bibliography; 09’s “Part I and 11’s “Appendix A point to their sections here; where 08–11 name their own files in prose, the shipped names are printed, and their command listings keep the delivered layout with a note. The title-page lines that named the AI assistant are recorded here only: manuscript 09’s PDF metadata “Research prepared with ChatGPT for Vladimir Reshetnikov; manuscript 10’s author line and metadata “Research manuscript prepared with ChatGPT; manuscript 11’s second author line “Developed with ChatGPT; proofs and executable verification included and metadata “Research draft prepared with ChatGPT.
- *Batch 63 (manuscript 12).* Appended as Part XIV after Part XIII, in its own order, so that no existing number changes; its introduction is in Section 3, and its validation, formalization plan, eight questions, conclusion and appendices stay in the Part. It duplicates no printed result: its bottleneck theorem is a new form of Theorem 62.1(c), its no-bound item a case of Proposition 62.2, and both are printed with notes and pointers in both directions. Its eight questions are cross-referenced to, not merged with, those of Section 71; four of those get dated notes, and two unlabelled ones (Research questions 71.31 and 71.37) received labels so that they can be cited. Renamed: its Cantor pairing π is π_C ; its SF and FF use Part XIII’s macros. Where it names its own files in prose, the shipped names are printed; its reproduction listings keep the delivered layout with a note. Its bibliography keys are mapped: its `repo`, the project README, to [1], because this report’s key `repo` is manuscript 06’s different item [6], and its `mrddpguide` to [5]; both entries now also record its pin. Its author line names no assistant. Notes added in [write] brackets: the formalized exponential representation beside its MRDP paragraph, a sharper grammar-length bound, a weaker hypothesis for its transport identity, and the relation of its bottleneck theorem to Parts IX and XIII. Dated notes in Parts IV, VIII, X and XII point to the neighbouring report *liveness-beyond-halting*, written in the same batch.
- *Batch 78 (manuscripts 13–15).* The three manuscripts answer one question of this report and share their target, rule table, gluing theorem and compiler, so they are merged into one Part, arranged by subject, appended after Part XIV and before the appendices; no existing Part, section,

theorem, question or equation number changed. Their title pages, abstracts, status statements and introductions are Sections 3.13–3.15 (equations numbered within subsections), and the Section 3 heading “The twelve manuscripts” became “The fifteen manuscripts”. Printed once, crediting every source: the rule table and the witness pair (manuscript 13’s text, with the others’ definitions and readings beside it); the storage and loop bounds (13’s corollary; 14’s and 15’s statements keep their additions); order-independent gluing (13’s theorem, with 15’s matrix form and 14’s locality lemma as the further statements they are); the compiler theorem (14’s statement and ledger, with 13’s statement as a remark and its proof as a second presentation; likewise the transfer theorem); and the carry-free product lemma, which is Part V’s Lemma 40.1 (13’s and 14’s statements are kept in their notation, their proofs replaced by a pointer because they are Part V’s proof). Kept with their own proofs: 15’s contextual completeness, which adds sufficiency to the loop-count form of 13’s injectivity; 13’s and 14’s memory theorems, variants for different relations compared in a table (14’s $17L - 4$ is not printed as an improvement on Part V’s $24L$); and 15’s orbit route. The twenty-five questions are printed in the Part and cross-referenced, not merged. Renamed: no symbol. Manuscript 14’s hand-set equation numbers are replaced by the Part’s numbering and its references by `\eqref` (34 equation labels and nine section labels written in its namespace, which had sixteen labels); its hard-coded “Section 4”, “Section 6”, “Section 7”, “Section 8” and “Section 1” point to its sections here; its pin and URL macros are printed literally. Corrections and disclosures: manuscript 15’s “tree identifier” 928ea9701 is a commit (noted; its text is unchanged); the README blob 79153a4ed recorded by manuscript 13 is the project README of 928ea9701, not of its pin 439c0a2d9; both state the 75- and 87-operation figures it quotes. Where the three name their own files in prose, the shipped names are printed; their command listings keep the delivered layout with notes. Dated notes credit the research note [64] as an independent derivation of manuscript 14’s loop formula and record the three natural-domain reductions of [65], which are not applied to the shipped files. Notes added elsewhere: after Part V’s linear-memory theorem; at Research questions 71.24 and 71.43; in Part X, after its subsection on interaction systems, at its frontend question (labelled `cdc:nh:q:frontend` for this purpose) and at its question on quotienting independent schedules. The title-page lines that named the AI assistant are recorded here only: manuscript 13’s “Prepared with ChatGPT” and its PDF metadata “Research report prepared with ChatGPT for the ProveIt project”; manuscript 14’s author line “Research prepared with ChatGPT for Vladimir Reshetnikov’s ProveIt program” and metadata “Research prepared with ChatGPT for Vladimir Reshetnikov”; manuscript 15’s metadata “OpenAI, research draft prepared for Vladimir Reshetnikov”.

- *Additions.* The conventions (Section 2), the Part openings, the remarks on the formalized exponential counterpart (Remark 64.2) and on the project’s tag-system notes (Remark 47.1), and every other paragraph marked [write].

Labels. [write] Every label carries the prefix `cdc:`. Manuscript 05’s labels keep their names after it (`cdc:thm:main`); the other manuscripts’ labels carry a second prefix: 01 `cdc:ct:`, 02 `cdc:pt:`, 03 `cdc:mem:`, 04 `cdc:wf:`, 06 `cdc:cs:`, 07 `cdc:qc:`. Labels written in the merge use `cdc:conv:`, `cdc:bd:`, `cdc:part:`, `cdc:sec:`, `cdc:q:`, or a manuscript’s prefix for a section it did not label (`cdc:wf:sec:interfaces`). Batch 62 adds the sub-prefixes `cdc:nh:` (08), `cdc:of:` (09), `cdc:pf:` (10) and `cdc:rx:` (11); its written labels are `cdc:part:heaps`, `cdc:part:assembly`, `cdc:part:pumping`, `cdc:part:reactions`, `cdc:conv:b62-08` to `cdc:conv:b62-11`, `cdc:conv:b62q`, `cdc:sec:ms08` to `cdc:sec:ms11`, nine `cdc:q:` labels added to existing questions, and `cdc:of:sec:degree`, `cdc:of:sec:assembly`, `cdc:of:sec:reproduction` for manuscript 09’s three parts. Batch 63 adds the sub-prefix `cdc:rt:` (12); its written labels are `cdc:part:routing`, `cdc:conv:b63-XIV`, `cdc:sec:ms12`, `cdc:rt:app:reproduction` and `cdc:rt:app:dependencies` for manuscript 12’s two appendices, and `cdc:q:beyond` and `cdc:q:varperiods` on two existing questions. Batch 78 adds the sub-prefixes

cdc:ew: (13), cdc:nt: (14) and cdc:ng: (15) for the manuscripts' labels, and cdc:ic: for sections written in the merge. Its written labels are cdc:part:wiring, cdc:conv:b78-wiring, cdc:sec:ms13 to cdc:sec:ms15, twelve cdc:ic: labels, nine section and appendix labels in manuscript 14's namespace (cdc:nt:sec:intro, ..., cdc:nt:app:sources), the 34 equation labels cdc:nt:eq:2.1 to cdc:nt:eq:11.1, named after manuscript 14's hand-set numbers, and cdc:nh:q: frontend on an existing question of Part X.

Source audits of manuscripts 13 and 14. [write] Manuscript 13's Appendix B and manuscript 14's Appendix D are printed below as delivered. Manuscript 15's audit is its shipped 15-no-ghost-wires-PROVENANCE.md; it records the revision 928ea9701 (which it calls a recursive-tree revision; it is a commit), the note blob 764c98229 and the MRDP.lean blob 74aea8c5e, says that no Lean or Rocq build was run and that priority was not established, and lists the primary sources of its bibliography. Manuscript 13's 13-exact-wiring-SOURCES.md repeats its Appendix B with the literature annotated, and manuscript 14's data/14-net-topology-source_audit.json records its Appendix D in machine-readable form.

K.1 Manuscript 13: Source audit and research provenance

The motivating repository audit used the snapshot

439c0a2d9c1052595f3de6a29c11511a24fb2e11.

The specific obstruction note and the MRDP interface are pinned to that commit in the bibliography. The obstruction note's fetched blob identifier was 764c982297342093307a0e831bc0d9ed5084c87f; the initially fetched Hilbert-tenth README blob was 79153a4ed74a8ab63e5f08fb3485a168b20cf8c1. The repository is active; these references identify the inspected basis, not a claim that the moving branch remained unchanged throughout report preparation.

For Lafont's paper, the net definition on journal page 71 and the rule figures on pages 81–82 were inspected as page images, not inferred from imperfect mathematical text extraction. The numbered annihilation rules are especially important. The code's positive port identifiers are an explicit implementation change, not a change to the auxiliary-port order.

The supplied sources are original explanatory text and original code for this report. Third-party articles are referenced rather than redistributed. The saved Python source hashes support exact replay of the tests. They do not establish independent review, historical priority, or proof-assistant verification.

K.2 Manuscript 14: Source audit

The repository was inspected through its GitHub connector. The motivating note was re-read at the pinned commit, and its blob identifier is recorded in Section 3.14.1. The canonical-certificate collection was consulted to avoid presenting generic heap and memory arithmetization as a new discovery. The original Lafont PDF was inspected visually at PDF pages 3, 13, and 14 (journal pages 71, 81, and 82), covering cyclic wires, the rule diagram, and the numbered annihilation distinction. De Falco's article was consulted for prior exact gluing and closed-component semantics. These sources establish context and the chosen native semantics; the arithmetic proofs do not assume correctness of an unformalized repository theorem.

[write] Manuscript 13's "initially fetched" README blob 79153a4ed is the project README of 928ea9701, a commit made twelve minutes after its pin 439c0a2d9, at which the README is 076832c56; both versions state the figures of 75 and 87 operations that manuscript 13 quotes.

References

[write] One bibliography for the fifteen manuscripts. An item cited by several manuscripts is printed once; the manuscripts citing it are listed at its end, and a detail given by only one of them is kept and attributed. Repository items are pinned at `e8bb0931d67f80d9fce87a8cddb0f661ff19f956` (manuscripts 01–06) or `725d2ebb6909fe11a13a92354c0f47367a3cbbf5` (manuscript 07); the cited files of `Computability/HilbertTenthProblem` are identical at the two commits. Manuscript 07’s two “companion” items were manuscripts 04 and 06 of this report and are not repeated. Manuscripts 08–11 (batch 62) cite the repository at `4e128356d0ef75308be8ed405d89aea2ffdb8a57`; every file they cite, in `Computability`, `Logic/PresburgerArithmetic` and the vendored `Rocq` library, is identical at the three pins. Manuscript 12 (batch 63) cites the repository at `e18718e837d43e162252f9a314e8cb797fbd1a1f`; no file under `Computability/HilbertTenthProblem` differs between `e8bb0931d` and that pin. Manuscripts 13–15 (batch 78) cite the repository at `439c0a2d9c1052595f3de6a29c11511a24fb2e11`, `6914ccca6685baf53b7a35f25efc89366c76ba74` and `928ea97017a25ebe56d240c84f27d2275d818c75`, all of 2 October 2026; the research note and the Lean file they cite are unchanged at the batch-78 write, and the project README has grown since without changing the figures they quote.

- [1] V. Reshetnikov (and contributors). *ProveIt: Hilbert’s tenth problem* — project README (manuscripts 01, 03, 04, 07), with its corrected Jones articles and Diophantine developments (04), and, as one item, README and formalization status register (02). `Computability/HilbertTenthProblem/README.md`, inspected September 30, 2026.
<https://github.com/VladimirReshetnikov/ProveIt/blob/e8bb0931d67f80d9fce87a8cddb0f661ff19f956/Computability/HilbertTenthProblem/README.md>; at manuscript 07’s pin <https://github.com/VladimirReshetnikov/ProveIt/blob/725d2ebb6909fe11a13a92354c0f47367a3cbbf5/Computability/HilbertTenthProblem/README.md>; the development’s tree as cited by manuscript 02: <https://github.com/VladimirReshetnikov/ProveIt/tree/e8bb0931d67f80d9fce87a8cddb0f661ff19f956/Computability/HilbertTenthProblem>. Manuscript 02: the repository supplies corrected editions and source developments; inspection here does not constitute an independent build audit. At the batch-62 pin: the README <https://github.com/VladimirReshetnikov/ProveIt/blob/4e128356d0ef75308be8ed405d89aea2ffdb8a57/Computability/HilbertTenthProblem/README.md> (manuscript 10) and the tree <https://github.com/VladimirReshetnikov/ProveIt/tree/4e128356d0ef75308be8ed405d89aea2ffdb8a57/Computability/HilbertTenthProblem> (manuscript 08, as “repository README and associated source”; manuscript 09, as README and formalization status register `Lean/STATUS.md`). At manuscript 12’s pin, as “ProveIt: Hilbert’s tenth problem development”, pinned project README: <https://github.com/VladimirReshetnikov/ProveIt/blob/e18718e837d43e162252f9a314e8cb797fbd1a1f/Computability/HilbertTenthProblem/README.md>. Manuscript 13 cites the README as “Hilbert’s tenth problem: Jones’s articles on Diophantine representation”, inspected 2 October 2026 at `439c0a2d9`: <https://github.com/VladimirReshetnikov/ProveIt/blob/439c0a2d9c1052595f3de6a29c11511a24fb2e11/Computability/HilbertTenthProblem/README.md>, for the figures of 75 and 87 arithmetic operations, which it attributes to the project; the blob it records, `79153a4ed`, is the README of `928ea9701`, and both versions state those figures. Manuscript 15 lists the same README, accessed October 2, 2026, without citing it in its text. Cited by manuscripts 01, 02, 03, 04, 07, 08, 09, 10, 12, 13.
- [2] V. Reshetnikov (and contributors). *ProveIt: Hilbert’s tenth problem, formalization status*. `Computability/HilbertTenthProblem/Lean/STATUS.md`, inspected September 30, 2026.
<https://github.com/VladimirReshetnikov/ProveIt/blob/e8bb0931d67f80d9fce87a8cddb0f661ff19f956/Computability/HilbertTenthProblem/Lean/STATUS.md>. Cited by manuscripts 03, 05.
- [3] V. Reshetnikov (and contributors). *ProveIt: Diophantine bounded universals and finite traces*. `Computability/HilbertTenthProblem/Lean/Diophantine/Common/DiophantineTrace.lean`, inspected September 30, 2026. <https://github.com/VladimirReshetnikov/ProveIt/blob/e8bb0931d67f80d9fce87a8cddb0f661ff19f956/Computability/HilbertTenthProblem/Lean/Diophantine/Common/DiophantineTrace.lean>; at the batch-62 pin <https://github.com/Vladimi>

- [rReshetnikov/ProveIt/blob/4e128356d0ef75308be8ed405d89aea2ffdb8a57/Computability/HilbertTenthProblem/Lean/Diophantine/Common/DiophantineTrace.lean](#). Cited by manuscripts 01, 02, 04, 05, 08, 11.
- [4] V. Reshetnikov. *ProveIt: Combinatory logic — SK, SKI, and Iota universality*. [Computability/CombinatoryLogic/README.md](#), inspected September 30, 2026. <https://github.com/VladimirReshetnikov/ProveIt/blob/e8bb0931d67f80d9fce87a8cddb0f661ff19f956/Computability/CombinatoryLogic/README.md>. Manuscript 08 cites the README and associated source at the batch-62 pin: <https://github.com/VladimirReshetnikov/ProveIt/tree/4e128356d0ef75308be8ed405d89aea2ffdb8a57/Computability/CombinatoryLogic>. Cited by manuscripts 01, 03, 04, 08.
- [5] V. Reshetnikov, ProveIt repository. *MRDP: recursively enumerable if and only if Diophantine*. [Computability/HilbertTenthProblem/Lean/MRDP.md](#), inspected at commit 725d2ebb6909fe11a13a92354c0f47367a3cbbf5, September 30, 2026. <https://github.com/VladimirReshetnikov/ProveIt/blob/725d2ebb6909fe11a13a92354c0f47367a3cbbf5/Computability/HilbertTenthProblem/Lean/MRDP.md>. Manuscript 11 cites this guide (as “ProveIt: Hilbert’s tenth problem and MRDP guide”) by a link to the repository’s main branch, which is not a fixed version; it is pinned here at the manuscript’s inspection commit: <https://github.com/VladimirReshetnikov/ProveIt/blob/4e128356d0ef75308be8ed405d89aea2ffdb8a57/Computability/HilbertTenthProblem/Lean/MRDP.md>. Manuscript 12 cites it at its own pin, as the specification of the MRDP interface, proof chain and audit boundary: <https://github.com/VladimirReshetnikov/ProveIt/blob/e18718e837d43e162252f9a314e8cb797fbd1a1f/Computability/HilbertTenthProblem/Lean/MRDP.md>. Cited by manuscripts 07, 11, 12.
- [6] V. Reshetnikov. *ProveIt*, public research repository, inspected September 30, 2026. Repository tree at commit e8bb0931d67f; relevant documents: [Computability/HilbertTenthProblem/README.md](#) and [Computability/HilbertTenthProblem/Lean/STATUS.md](#). [Pinned Diophantine development](#). The repository descriptions were inspected; its proofs were not rebuilt for manuscript 06. Cited by manuscript 06.
- [7] Yu. V. Matiyasevich. Enumerable sets are Diophantine. *Soviet Mathematics Doklady* **11** (1970), 354–358. (Manuscript 04 gives the pages as 354–357.) Manuscript 15 cites the reprint in *Mathematical Logic in the 20th Century*, ed. G. E. Sacks, https://doi.org/10.1142/9789812564894_0013. Cited by manuscripts 04, 06, 15.
- [8] J. P. Jones and Yu. V. Matiyasevich. Register machine proof of the theorem on exponential Diophantine representation of enumerable sets. *Journal of Symbolic Logic* **49**(3) (1984), 818–829. doi:10.2307/2274135. Bibliographic details, a corrected edition and its formalization status are catalogued in [1] (manuscripts 01 and 06). Cited by manuscripts 01, 03, 04, 05, 06, 11.
- [9] Yu. V. Matiyasevich. Towards finite-fold Diophantine representations. *Journal of Mathematical Sciences* **171** (2010), 745–752. doi:10.1007/s10958-010-0179-4. Manuscript 03: this reference documents the finite-fold/single-fold frontier; its universal-substrate equivalence is proved directly (Theorem 62.1). Cited by manuscripts 01, 02, 03, 05, 06, 14, 15.
- [10] D. Cantone, L. Cuzziol, and E. G. Omodeo. On Diophantine singlefold specifications. *Le Matematiche* **79**, no. 2 (2024), 585–620. Published December 28, 2024. doi:10.4418/2024.79.2.18. Journal page: <https://lematematiche.dmi.unict.it/index.php/lematematiche/article/view/2703>. Author-deposited text: <https://arts.units.it/bitstream/11368/3101478/1/CC024.pdf>. Cited by manuscripts 02, 04, 05, 06, 11.
- [11] D. Cantone, L. Cuzziol, and E. G. Omodeo. *Six equations in search of a finite-fold-ness proof*. 2023; version 3, September 30, 2024. <https://arxiv.org/abs/2303.02208v3>. A different article from [10]. Cited by manuscripts 07, 08.
- [12] D. Cantone, A. Casagrande, F. Fabris, and E. Omodeo. Does every recursively enumerable set admit a finite-fold Diophantine representation? *CEUR Workshop Proceedings* **2396**, paper 11 (2019). <https://ceur-ws.org/Vol-2396/paper11.pdf>. Cited by manuscript 01.

- [13] D. Larchey-Wendling and Y. Forster. Hilbert’s tenth problem in Coq. In *FSCD 2019*, LIPIcs **131** (2019), 27:1–27:20. doi:10.4230/LIPIcs.FSCD.2019.27. Cited by manuscripts 04, 07.
- [14] D. Larchey-Wendling and Y. Forster. Hilbert’s tenth problem in Coq (extended version). *Logical Methods in Computer Science* **18**(1), article 35 (2022). doi:10.46298/lmcs-18(1:35)2022. Author version: <https://arxiv.org/abs/2003.04604> (version 5: <https://arxiv.org/abs/2003.04604v5>). Manuscript 10 writes the DOI with capitals, 10.46298/LMCS-18(1:35)2022; DOIs are case-insensitive. Cited by manuscripts 01, 02, 10, 11.
- [15] M. Carneiro. *A Lean formalization of Matiyasevič’s Theorem*. arXiv:1802.01795, 2018. <https://arxiv.org/abs/1802.01795>. Cited by manuscripts 03, 07.
- [16] P. Cartier and D. Foata. *Problèmes combinatoires de commutation et réarrangements*. Lecture Notes in Mathematics **85**, Springer, 1969. Cited in manuscript 01 for the classical partially commutative monoid and normal-form framework; see also [17]. Cited by manuscript 01.
- [17] C. Krattenthaler. *The theory of heaps and the Cartier–Foata monoid*, author-hosted exposition. <https://www.mat.univie.ac.at/~kratt/artikel/heaps.pdf>. Cited by manuscript 01.
- [18] F. Barthélemy. Using Mazurkiewicz trace languages for partition-based morphology. In *Proceedings of the 45th Annual Meeting of the Association of Computational Linguistics* (2007), 928–935. See the lexicographic normal-form characterization in Section 2.2, p. 930. <https://aclanthology.org/P07-1117/>. Cited by manuscript 05.
- [19] A. V. Anisimov and D. E. Knuth. *Inhomogeneous sorting*. International Journal of Computer & Information Sciences **8** (1979), 255–260. doi:10.1007/BF00993053. Cited by manuscript 06.
- [20] M. Lohrey, F. Stober, and A. Weiß. The power word problem in graph products. *Theory of Computing Systems* **68** (2024), 403–464. doi:10.1007/s00224-024-10173-z. Cited by manuscript 04.
- [21] A. de Colnet, K. S. Meel, and U. Mathur. Counting and sampling traces in regular languages. *Proceedings of the ACM on Programming Languages*, POPL 2026. doi:10.1145/3776723. Preprint arXiv:2512.00314. Cited by manuscript 04.
- [22] L. G. Valiant. The complexity of enumeration and reliability problems. *SIAM Journal on Computing* **8**(3) (1979), 410–421. <https://doi.org/10.1137/0208032>. Cited by manuscript 05.
- [23] G. Zetsche. *The Emptiness Problem for Valence Automata over Graph Monoids*. arXiv:1710.07528 (2017). <https://arxiv.org/abs/1710.07528>. Cited by manuscript 06.
- [24] F. Grunewald and D. Segal. *On the integer solutions of quadratic equations*. Journal für die reine und angewandte Mathematik **569** (2004), 13–45. doi:10.1515/crll.2004.023. Cited by manuscript 06.
- [25] M. Hark, F. Frohn, and J. Giesl. *Termination of Triangular Polynomial Loops*. arXiv:1910.11588, version 7, February 13, 2024. <https://arxiv.org/abs/1910.11588v7>. Cited by manuscript 02.
- [26] N. Lommen, F. Meyer, and J. Giesl. *Automatic Complexity Analysis of Integer Programs via Triangular Weakly Non-Linear Loops*. arXiv:2205.08869, version 3, November 15, 2024. <https://arxiv.org/abs/2205.08869v3>. Cited by manuscript 02.
- [27] E. Ben-Sasson, A. Chiesa, D. Genkin, E. Tromer, and M. Virza. SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge. In *Advances in Cryptology — CRYPTO 2013*, Part II, LNCS 8043, Springer, 2013, 90–108. https://doi.org/10.1007/978-3-642-40084-1_6. Extended version, Cryptology ePrint Archive 2013/507: <https://eprint.iacr.org/2013/507>. The extended version’s memory-consistency discussion is the relevant prior art. Manuscript 08 cites the extended version. Cited by manuscripts 03, 08.
- [28] K. E. Batcher. Sorting networks and their applications. In *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*, AFIPS **32**, 307–314. <https://doi.org/10.1145/1468075.1468121>. Manuscript 14 cites it as AFIPS Spring Joint Computer Conference (1968), pp. 307–314. Cited by manuscripts 03, 08, 14.

- [29] M. Ajtai, J. Komlós, and E. Szemerédi. An $O(n \log n)$ sorting network. In *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing*, 1983, 1–9.
<https://doi.org/10.1145/800061.808726>. Cited by manuscript 03.
- [30] J. H. Conway. FRACSTRAN: A simple universal programming language for arithmetic. In *Open Problems in Communication and Computation*, Springer, New York, 1987, pp. 4–26. Manuscript 10 names the editors, T. M. Cover and B. Gopinath, and an online copy:
<https://gwern.net/doc/cs/computable/1987-conway.pdf>. Cited by manuscripts 04, 10.
- [31] M. Cook. *Universality in Elementary Cellular Automata*. *Complex Systems* **15**(1) (2004), 1–40.
[doi:10.25088/ComplexSystems.15.1.1](https://doi.org/10.25088/ComplexSystems.15.1.1). The cyclic-tag definition and simulation are in Sections 2.1–2.2. Cited by manuscript 07.
- [32] D. Woods and T. Neary. *On the time complexity of 2-tag systems and small universal Turing machines*. *Proceedings of FOCS 2006*. [doi:10.1109/FOCS.2006.58](https://doi.org/10.1109/FOCS.2006.58); <https://arxiv.org/abs/cs/0612089>. Cited by manuscript 07.
- [33] P. Atela and J. Hu. *Commuting polynomials and polynomials with same Julia set*. 1995.
<https://arxiv.org/abs/math/9504210>. Cited for the broader centralizer context, not as a dependency of the elementary proof in manuscript 07. Cited by manuscript 07.
- [write] The following items were added in batch 62, for manuscripts 08–11.
- [34] V. Reshetnikov and contributors. *ProveIt: MRDP finite-polynomial interface*. *Computability/HilbertTenthProblem/Lean/Diophantine/MRDP.lean*, commit `4e128356d0ef75308be8ed405d89aea2ffdb8a57`, inspected September 30, 2026.
<https://github.com/VladimirReshetnikov/ProveIt/blob/4e128356d0ef75308be8ed405d89aea2ffdb8a57/Computability/HilbertTenthProblem/Lean/Diophantine/MRDP.lean>. The Lean source of the theorems *Diophantine.mrdp*, *Diophantine.mrdp_iff* and *Diophantine.mrdp_dioph_iff*; not the guide [5]. Manuscripts 13 and 15 cite the same file, for *Diophantine.mrdp* and *Diophantine.mrdp_iff*, at `439c0a2d9` and `928ea9701` (blob `74aea8c5e` at both pins and at the batch-78 write):
<https://github.com/VladimirReshetnikov/ProveIt/blob/439c0a2d9c1052595f3de6a29c11511a24fb2e11/Computability/HilbertTenthProblem/Lean/Diophantine/MRDP.lean>. Neither rebuilt it. Cited by manuscripts 09, 13, 15.
- [35] V. Reshetnikov and contributors. *ProveIt: Presburger arithmetic is decidable*. *Logic/PresburgerArithmetic/README.md*, commit `4e128356d0ef75308be8ed405d89aea2ffdb8a57`.
<https://github.com/VladimirReshetnikov/ProveIt/blob/4e128356d0ef75308be8ed405d89aea2ffdb8a57/Logic/PresburgerArithmetic/README.md>. Cited by manuscript 09.
- [36] V. Reshetnikov and contributors. *Arithmetic machines as an alternative to the 90-operation compiler*. *ProveIt research note, FRACSTRAN_VARIANTS.md*, commit `4e128356d0ef75308be8ed405d89aea2ffdb8a57`.
[Computability/HilbertTenthProblem/Papers/1980/FRACTRAN_VARIANTS.md](https://github.com/VladimirReshetnikov/ProveIt/blob/4e128356d0ef75308be8ed405d89aea2ffdb8a57/Computability/HilbertTenthProblem/Papers/1980/FRACTRAN_VARIANTS.md). Cited by manuscript 10.
- [37] D. Larchey-Wendling and contributors. *FRACSTRAN termination is Diophantine*. *fracstran_dio.v*, Coq Library of Undecidability, vendored in *ProveIt* at commit `4e128356d0ef75308be8ed405d89aea2ffdb8a57`. The source file identifies its original MPL-2.0 license; no copy of that source is redistributed here.
[lib/Coq-Library-Undecidability/theories/H10/Fracstran/fracstran_dio.v](https://github.com/VladimirReshetnikov/ProveIt/blob/4e128356d0ef75308be8ed405d89aea2ffdb8a57/lib/Coq-Library-Undecidability/theories/H10/Fracstran/fracstran_dio.v). Cited by manuscript 10.
- [38] E. Ben-Sasson, A. Chiesa, D. Genkin, and E. Tromer. *Fast Reductions from RAMs to Delegatable Succinct Constraint Satisfaction Problems*. *Cryptology ePrint Archive*, Report 2012/071, revised 18 August 2012. In particular, the memory-consistency discussion in Section 4.1. <https://eprint.iacr.org/2012/071>. Cited by manuscript 08.

- [39] M. T. Goodrich. *Zig-zag Sort: A Simple Deterministic Data-Oblivious Sorting Algorithm Running in $O(n \log n)$ Time*. STOC 2014; arXiv:1403.2777, 2014. <https://arxiv.org/abs/1403.2777>. Cited by manuscript 08.
- [40] D. Cantone, A. Casagrande, F. Fabris, and E. G. Omodeo. *The quest for Diophantine finite-fold-ness*. *Le Matematiche* **76**(1) (2021). <https://lematematiche.dmi.unict.it/index.php/lematematiche/article/view/2044>. Not the same article as [12], [10] or [11]. Manuscript 08 omits the second author initial of Omodeo. Cited by manuscripts 08, 11.
- [41] Y. Lafont. *Interaction Combinators*. *Information and Computation* **137**(1) (1997), 69–101. <https://www.sciencedirect.com/science/article/pii/S0890540197926432>. Manuscript 13 gives doi:10.1006/inco.1997.2643 and an inspected full-text copy, <https://chorasimilarity.wordpress.com/wp-content/uploads/2024/01/ic-lafont-1.pdf>; manuscripts 14 and 15 give the same copy and the author’s PostScript, <https://www.i2m.univ-amu.fr/perso/yves.lafont/pub/combinators.ps>, and manuscript 15 also the publisher record above. Manuscripts 13–15 inspected the definition on journal page 71 and the rule figures on pages 81–82 as page images. Cited by manuscripts 08, 13, 14, 15.
- [42] S. Ginsburg and E. H. Spanier. Semigroups, Presburger formulas, and languages. *Pacific Journal of Mathematics* **16**(2) (1966), 285–296. <https://doi.org/10.2140/pjm.1966.16.285>. Cited by manuscript 09.
- [43] A. A. Ahmadi and J. Zhang. *Complexity aspects of local minima and related notions*. arXiv:2008.06148, version 2, June 15, 2021. In particular, Section 4 and Lemma 4.2. <https://arxiv.org/abs/2008.06148>. Cited by manuscript 09.
- [44] R. Hildebrand. *Minimal zeros of copositive matrices*. arXiv:1401.0134, version 4, June 8, 2014. In particular, Lemmas 2.3–2.5 and 3.1. <https://arxiv.org/abs/1401.0134>. Cited by manuscript 09.
- [45] S. Basu. *Algorithms in real algebraic geometry: A survey*. arXiv:1409.1534 (2014). <https://arxiv.org/abs/1409.1534>. Cited by manuscript 09.
- [46] D. Doty, J. H. Lutz, M. J. Patitz, R. T. Schweller, S. M. Summers, and D. Woods. *The tile assembly model is intrinsically universal*. arXiv:1111.3097 (2011; revised 2012). <https://arxiv.org/abs/1111.3097>. Cited by manuscript 09.
- [47] A. Alseth and M. J. Patitz. *The need for seed (in the abstract Tile Assembly Model)*. arXiv:2211.03877 (2022). <https://arxiv.org/abs/2211.03877>. Cited by manuscript 09.
- [48] D. Doty. *Producibility in hierarchical self-assembly*. arXiv:1304.7804, version 2 (2013). <https://arxiv.org/abs/1304.7804>. Cited by manuscript 09.
- [49] C. Barrett, S. Demri, and M. Deters. *Witness Runs for Counter Machines!* In FroCoS 2013, Lecture Notes in Artificial Intelligence 8152, Springer, pp. 120–150. <https://theory.stanford.edu/~barrett/pubs/BDD13.pdf>. Cited by manuscript 10.
- [50] N. Decker and A. Pirogov. *Flat Model Checking for Counting LTL Using Quantifier-Free Presburger Arithmetic*. Preprint, 2019. <https://arxiv.org/abs/1901.05692>. Cited by manuscript 10.
- [51] D. Cantone, L. Cuzziol, and E. G. Omodeo. *A Brief History of Singlefold Diophantine Definitions*. CILC 2023, CEUR Workshop Proceedings 3428, 2023. <https://ceur-ws.org/Vol-3428/paper5.pdf>. A fourth Cantone–Omodeo article, different from [10], [11], [12] and [40]. Cited by manuscript 10.
- [52] I. Korec. Small universal register machines. *Theoretical Computer Science* **168**(2) (1996), 267–301. doi:10.1016/S0304-3975(96)00080-1. Cited by manuscript 11.
- [53] A. Alhazov and S. Verlan. *Minimization Strategies for Maximally Parallel Multiset Rewriting Systems*. arXiv:1009.2706, 2010. Journal version: *Theoretical Computer Science* **412**(17) (2011), 1581–1591; doi:10.1016/j.tcs.2010.10.033. In particular, Figure 1 and the adjacent instruction list on printed pages 4–5 of the PDF inspected by manuscript 11. <https://arxiv.org/abs/1009.2706>. Cited by manuscript 11.

- [54] X. Wang, P. Zheng, T. Ma, and T. Song. Small Universal Bacteria and Plasmid Computing Systems. *Molecules* **23**(6) (2018), article 1307. doi:10.3390/molecules23061307. Cited by manuscript 11.
- [55] E. Best and M. Koutny. Petri net semantics of priority systems. *Theoretical Computer Science* **96**(1) (1992), 175–215. doi:10.1016/0304-3975(92)90184-H. Cited by manuscript 11.
- [56] R. M. Karp and R. E. Miller. Parallel program schemata. *Journal of Computer and System Sciences* **3**(2) (1969), 147–195. doi:10.1016/S0022-0000(69)80011-5. Cited by manuscript 11.
- [write] The following items were added in batch 63, for manuscript 12. Its two repository items are merged into [1] and [5].
- [57] T. Friedrich and L. Levine. *Fast simulation of large-scale growth models*. arXiv:1006.1003v2, 2012. In particular, Theorem 1 and the least-action/acyclicity characterization of abelian-stack aggregation. <https://arxiv.org/abs/1006.1003>. Cited by manuscript 12.
- [58] B. Bond and L. Levine. *Abelian networks I. Foundations and examples*. SIAM Journal on Discrete Mathematics **30**(2) (2016), 856–874. arXiv:1309.3445. <https://arxiv.org/abs/1309.3445>. Cited by manuscript 12.
- [59] B. Gärtner, T. D. Hansen, P. Hubáček, K. Král, H. Mosaad, and V. Slívová. *ARRIVAL: Next Stop in CLS*. ICALP 2018; arXiv:1802.07702. Unique run-profile certificates and the $UP \cap coUP$ upper bound. <https://arxiv.org/abs/1802.07702>. Cited by manuscript 12.
- [60] J. Fearnley, M. Gairing, M. Mnich, and R. Savani. *Reachability switching games*. Logical Methods in Computer Science **17**(2), article 10 (2021). arXiv:1709.08991v7. See Section 2 for run-length succinctness and Section 5.3 for last-used-edge structure in succinct zero-player switching games. <https://arxiv.org/abs/1709.08991>. Cited by manuscript 12.
- [61] H. Cairns. *Some halting problems for abelian sandpiles are undecidable in dimension three*. arXiv:1508.00161v2, 2021. <https://arxiv.org/abs/1508.00161>. Cited by manuscript 12.
- [62] J. Bayer, M. David, M. Haßler, Yu. Matiyasevich, and D. Schleicher. *Diophantine equations over $\mathbb{Z}$: universal bounds and parallel formalization*. arXiv:2506.20909v1, 2025. Used for context on formalization and explicit universal-bound accounting, not as an input to the routing proofs. <https://arxiv.org/abs/2506.20909>. Cited by manuscript 12.
- [63] V. Reshetnikov (and contributors). *A finite wiring obstruction for interaction combinators*, research note. Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/interaction_combinator_wiring_obstruction.md, blob 764c982297342093307a0e831bc0d9ed5084c87f at the pins of manuscripts 13, 14 and 15 and at the batch-78 write. Manuscript 13: inspected 2 October 2026 at 439c0a2d9c1052595f3de6a29c11511a24fb2e11, https://github.com/VladimirReshetnikov/ProveIt/blob/439c0a2d9c1052595f3de6a29c11511a24fb2e11/Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/interaction_combinator_wiring_obstruction.md. Manuscript 14: “Vladimir Reshetnikov’s ProveIt research continuation”, inspected at commit 6914ccca6685baf53b7a35f25efc89366c76ba74, 2 October 2026, <https://github.com/VladimirReshetnikov/ProveIt>. Manuscript 15: “V. Reshetnikov, ProveIt research corpus”, accessed October 2, 2026, blob as above. Cited by manuscripts 13, 14, 15.
- [64] V. Reshetnikov (and contributors). *A guarded quadratic topology relation for interaction-combinator annihilation*, research note. Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/interaction_combinator_quadratic_topology.md, commit b7a9404d40d1c80a86c8080983745b1ec7b6525b (2 October 2026), after the pins of manuscripts 13–15. Cited in the batch-78 write only.
- [65] V. Reshetnikov (and contributors). *Six new substrate reports: review, constructor repairs and local reductions*. Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/incoming_substrate_review_1977e6ea6.md, commit 44b28c395fd5bc2efe444fad419cbb5713b92d75 (2 October 2026), with its replay checker and receipt

- beside it. It reviews the archives of manuscripts 13–15 and of three other batch-78 manuscripts. Cited in the batch-78 write only.
- [66] ProveIt research reports. *Canonical Diophantine Certificates*, collection README, especially manuscripts 03 (linear memory) and 08 (dynamic heaps). SetTheory/Cardinals/docs/reports/hilbert-tenth-problem/canonical-diophantine-certificates/README.md, inspected 2 October 2026 at 6914ccca6, when Parts I–XIV of this report had been written.
<https://github.com/VladimirReshetnikov/ProveIt>. This report’s README. Cited by manuscript 14.
 - [67] Y. Lafont. *Interaction nets*. In *Proceedings of the 17th ACM SIGPLAN–SIGACT Symposium on Principles of Programming Languages (POPL ’90)*, 1990, 95–108. <https://doi.org/10.1145/96709.96718>. Cited by manuscripts 14, 15.
 - [68] M. de Falco. *An Explicit Framework for Interaction Nets*. *Logical Methods in Computer Science* **6**(4:6) (2010). doi:10.2168/LMCS-6(4:6)2010; <https://arxiv.org/abs/1010.1066> (manuscript 14 cites arXiv:1010.1066v2, 13 December 2010). Cited by manuscripts 13, 14.
 - [69] G. I. Lehrer and R. B. Zhang. *The Brauer category and invariant theory*. *Journal of the European Mathematical Society* **17** (2015), 2311–2351. doi:10.4171/JEMS/558; <https://ems.press/content/serial-article-files/32068> (manuscript 13); <https://arxiv.org/abs/1207.5889> (manuscript 15). Cited by manuscripts 13, 15.
 - [70] M. Blum, W. S. Evans, P. Gemmell, S. Kannan, and M. Naor. *Checking the Correctness of Memories*. *Algorithmica* **12**(2/3) (1994), 225–244. Author’s publication listing and full-text links: <https://www.wisdom.weizmann.ac.il/~naor/onpub.html>. Cited by manuscript 13.
 - [71] M. Naor, M. Parter, and E. Yogev. *The Power of Distributed Verifiers in Interactive Proofs*. arXiv:1812.10917 (2018), especially Section 4.3. <https://arxiv.org/abs/1812.10917>. Cited by manuscript 13.
 - [72] A. V. Sutherland. *18.783 Elliptic Curves, Lecture 3*. Massachusetts Institute of Technology, 12 February 2013; discussion of Kronecker substitution.
<https://math.mit.edu/classes/18.783/2013/LectureNotes3.pdf>. Cited by manuscript 13.
 - [73] Yu. V. Matiyasevich. *Matiyasevich theorem*. *Scholarpedia* **3**(7) (2008), 7095. Author’s account of the constructive MRDP theorem. <https://doi.org/10.4249/scholarpedia.7095>. Cited by manuscript 14.